

With LangChain, LangGraph, and MCP

AI Agents and Applications

Roberto Infante



With LangChain, LangGraph, and MCP

AI Agents and Applications

Roberto Infante

MEAP

 MANNING



AI Agents and Applications

1. [welcome](#)
2. [1 Introduction to AI Agents and Applications](#)
3. [2 Executing prompts programmatically](#)
4. [3 Summarizing text using LangChain](#)
5. [4 Building a research summarization engine](#)
6. [5 Agentic Workflows with LangGraph](#)
7. [6 RAG fundamentals with Chroma DB](#)
8. [7 Q&A chatbots with LangChain and LangSmith](#)
9. [8 Advanced indexing](#)
10. [9 Question transformations](#)
11. [10 Query generation, routing and retrieval post-processing](#)
12. [11 Building Tool-based Agents with LangGraph](#)
13. [12 Multi-agent Systems](#)
14. [13 Building and consuming MCP servers](#)
15. [14 Productionizing AI Agents: memory, guardrails, and beyond](#)
16. [Appendix A. Trying out LangChain](#)
17. [Appendix B. Setting up a Jupyter Notebook environment](#)
18. [Appendix C. Choosing an LLM](#)
19. [Appendix D. Installing SQLite on Windows](#)
20. [Appendix E. Open-source LLMs](#)
21. [index](#)

welcome

Dear Reader,

Thank you so much for purchasing "*AI Agents and Applications*". Your interest in this cutting-edge topic means a great deal to me, and I am excited to guide you through the expansive world of large language models (LLMs).

This book emerged from my professional journey as a software developer, where my main focus has been quantitative development in finance. Over the years, I occasionally delved into AI projects aimed at tackling problems where traditional mathematical computations fell short. My fascination with LLMs began when I encountered ChatGPT in November 2022. Its capabilities inspired me to explore further, leading me to experiment with the OpenAI APIs, delve into prompt engineering, and design applications using Retrieval Augmented Generation (RAG). My involvement with LangChain, an open-source LLM application framework, significantly accelerated my learning and allowed me to witness its rapid adoption within the community.

"*AI Agents and Applications*" is structured to cater both to beginners and seasoned professionals. It offers a comprehensive look into the foundational technologies and advanced techniques in the realm of LLMs. Whether you are new to programming or an experienced developer, you will find the content approachable yet enriching, especially with practical code examples to enhance your engagement.

The insights in this book are drawn from my real-world experiences and continuous learning in a field that evolves almost daily. I cover a range of topics from running open source LLMs locally to advanced RAG techniques and fine-tuning methods. My goal is to provide a resource that I wish had been available to me—an accessible, practical guide that empowers you to navigate and excel in this emerging domain.

As this field is still in its infancy, your feedback is incredibly valuable. I encourage you to actively participate in our online [discussion forum at](#)

[liveBook](#). Your questions, comments, and suggestions not only help improve this book but also contribute to the broader community exploring LLM applications.

Thank you once again for joining me on this journey. I am eager to hear about your experiences and look forward to any insights you might share.

Warm regards,

— Roberto Infante

In this book

[welcome](#) [1 Introduction to AI Agents and Applications](#) [2 Executing prompts programmatically](#) [3 Summarizing text using LangChain](#) [4 Building a research summarization engine](#) [5 Agentic Workflows with LangGraph](#) [6 RAG fundamentals with Chroma DB](#) [7 Q&A chatbots with LangChain and LangSmith](#) [8 Advanced indexing](#) [9 Question transformations](#) [10 Query generation, routing and retrieval post-processing](#) [11 Building Tool-based Agents with LangGraph](#) [12 Multi-agent Systems](#) [13 Building and consuming MCP servers](#) [14 Productionizing AI Agents: memory, guardrails, and beyond](#) [Appendix A. Trying out LangChain](#) [Appendix B. Setting up a Jupyter Notebook environment](#) [Appendix C. Choosing an LLM](#) [Appendix D. Installing SQLite on Windows](#) [Appendix E. Open-source LLMs](#)

1 Introduction to AI Agents and Applications

This chapter covers

- Core challenges in building LLM-powered applications
- LangChain’s modular architecture and components
- Patterns for engines, chatbots, and agents
- Foundations of prompt engineering and RAG

After the launch of ChatGPT in late 2022, developers quickly began experimenting with applications powered by large language models (LLMs). Since then, LLMs have moved from novelty to necessity, becoming a new staple in the application development toolbox—much like databases or web interfaces in earlier eras. They give applications the ability to answer complex questions, generate tailored content, summarize long documents, and coordinate actions across systems. More importantly, LLMs have unlocked a new class of applications: AI agents. Agents take input in natural language, decide which tools or services to call, orchestrate multi-step workflows, and return results in a clear, human-friendly format.

You could build such systems entirely from scratch, but you would quickly encounter recurring challenges: how to ingest and manage data, how to structure prompts, how to chain model calls reliably, and how to integrate external APIs and services. Reinventing these patterns each time is slow and error-prone. This is where frameworks come in. LangChain, LangGraph, and LangSmith provide modular building blocks that eliminate boilerplate, promote best practices, and let you focus on application logic instead of low-level wiring. Throughout this book, we will use these frameworks as practical tools—not as the goal, but as the means—to learn how to design, build, and scale real LLM-based applications and agents.

In this opening chapter, we will set the foundation. You’ll get an overview of the main problems LLM applications aim to solve, explore the architecture

and object model that LangChain provides, and examine the three primary families of LLM-powered applications: engines, chatbots, and AI agents. We will also introduce the key patterns—such as Retrieval-Augmented Generation (RAG) and prompt engineering—that you’ll return to throughout the book. By the end of this chapter, you will have a clear picture of the challenges in building LLM applications, the patterns that solve them, and the frameworks we will use to bring those solutions to life.

1.1 Introducing LangChain

Imagine you’ve been asked to build a chatbot that can answer customer questions from your company’s documentation, or a search engine that retrieves precise answers from thousands of internal reports. You quickly discover the same pain points:

- How do you get your own data into the model reliably, without pasting entire documents into prompts?
- How do you keep prompts, chains, and integrations maintainable as your features grow?
- How do you handle context limits and costs while still preserving accuracy?
- How do you orchestrate multi-step workflows and API calls without fragile glue code?
- And once your app is live, how do you evaluate, debug, and monitor its behavior?

Without a framework, developers end up re-implementing the same plumbing—load data, split it, embed it, store it, retrieve it, prompt it—again and again. It works for a demo, but it doesn’t scale. LangChain addresses these problems by standardizing common patterns into modular, composable components. With loaders, splitters, embeddings, retrievers, vector store retrievers, and prompt templates, you can focus on application logic instead of boilerplate. The LangChain Expression Language (LCEL) and the Runnable interface then let you chain these pieces together consistently, making pipelines easier to build, debug, and maintain.

LangChain has also evolved rapidly, fueled by an active open-source

community. It continues to keep pace with advances in LLM architectures, new data sources, and retrieval technologies, while helping to establish shared best practices for building and deploying LLM-based systems.

Three principles guide its design: modularity, composability, and extensibility. Components follow standard interfaces, so you can swap an LLM, change a vector store, or add a new data connector without rewriting your entire application. Real-world tasks can be composed from multiple components, forming chains or agent workflows that dynamically select the right tools for the job. And while default implementations exist, you can always extend or replace them with custom logic or third-party integrations, avoiding lock-in and promoting interoperability.

By learning LangChain, you not only gain the ability to build production-grade LLM applications, but also acquire transferable skills. Competing frameworks solve similar problems in similar ways, so once you understand these patterns, you can adapt them to whatever stack you choose in the future.

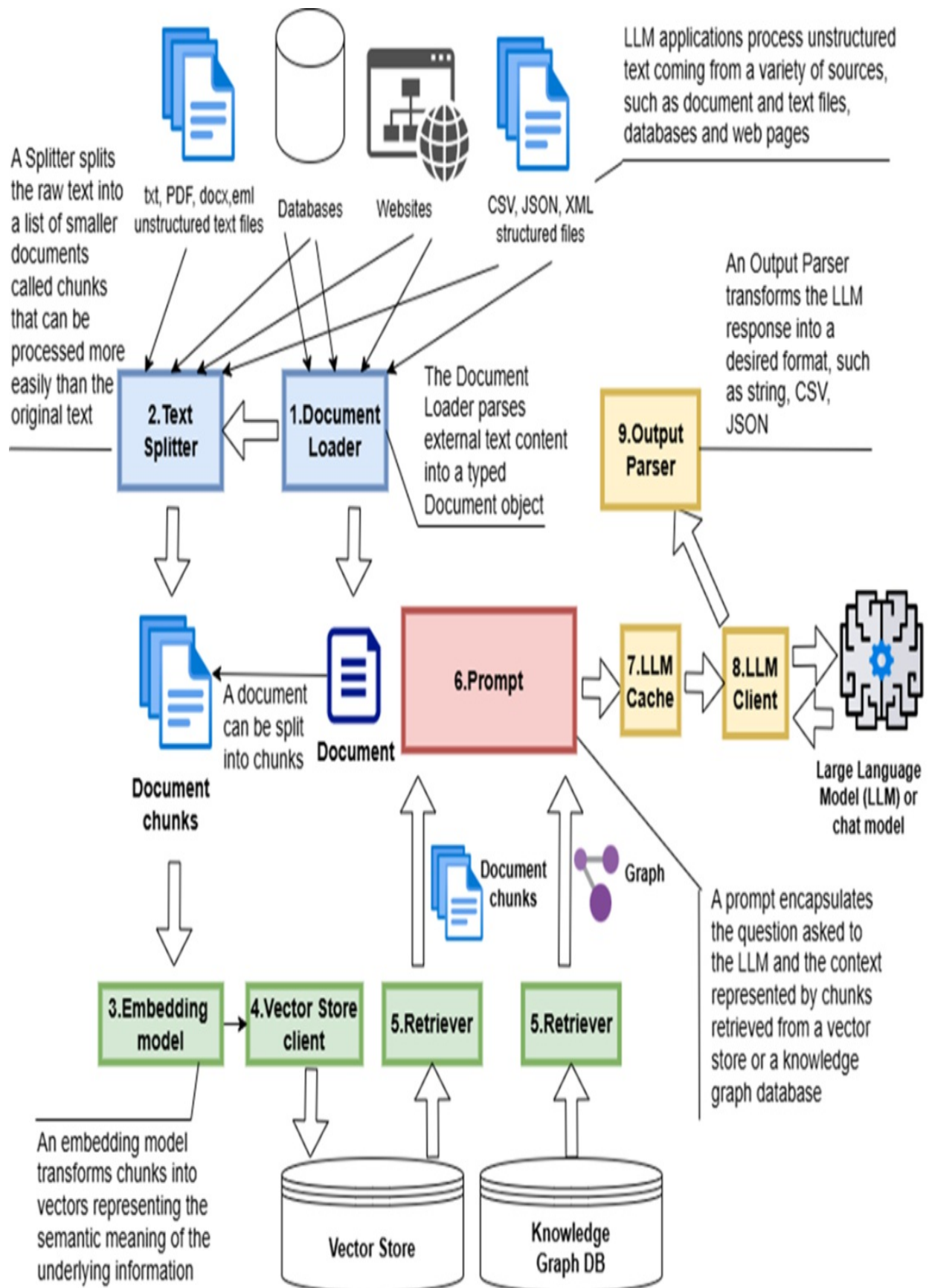
1.1.1 LangChain architecture

LangChain’s documentation is thorough, but the best way to really understand how it works is by building with it. This section gives you a high-level overview that we’ll keep coming back to in later chapters. Think of it as the map you’ll use while we dive into the details and code examples later on.

Most LLM frameworks follow a similar overall pattern, but each one defines its own components. In LangChain, the workflow looks roughly like what you see in Figure 1.1. You start by pulling in text from different sources—files, databases, or websites—and wrapping it into Document objects. Those documents are often split into smaller chunks so they’re easier to handle. Next, each chunk is passed through an embedding model, which turns the text into vectors that capture its meaning. Both the raw chunks and their embeddings are stored in a vector store, which lets you quickly retrieve the most relevant pieces of text based on similarity search.

Figure 1.1 LangChain architecture: The Document Loader imports data, which the Text Splitter divides into chunks. These are vectorized by an Embedding Model, stored in a Vector Store, and retrieved through a Retriever for the LLM. The LLM Cache checks for prior requests to return

cached responses, while the **Output Parser** formats the LLM's final response.



When an LLM app runs a task—say summarization or semantic search—it builds a prompt that combines the user’s question with extra context. That context usually comes from document chunks pulled out of a vector store. Sometimes, though, you’ll also want to bring in information from a graph database. Vector stores are still the backbone of most *retrieval-augmented generation (RAG)* workflows, but graph databases are becoming more common in apps that need to represent and reason about relationships between entities. LangChain already integrates with popular options like Neo4j, and it lets you use graph-based memory or planning components—features that are showing up more and more in advanced agent architectures.

To make all of this easier to wire together, LangChain introduced the Runnable interface and the LangChain Expression Language (LCEL). These give you a clean, consistent way to chain components without writing piles of glue code. We’ll go deeper into both later in this chapter.

It’s also worth keeping in mind that LangChain’s workflow isn’t locked into a simple pipeline. You can arrange components as graphs to handle more complex, branching flows—a capability formalized in the LangGraph module. We’ll dig into these patterns and architectural variations in the next section, where the bigger picture of LLM application design comes into focus.

A detailed description of each component follows below (it references the numbering in figure 1.1):

- Document loaders (1): in LangChain, document loaders play a pivotal role by extracting data from various sources and transforming it into Document objects. These Documents serve as the fundamental entities processed within the system.
- Text splitters (2): As we’ll discuss in upcoming chapters, text splitters are crucial for breaking down text from a data source into smaller Document instances or “chunks.” This approach helps overcome the limitation of the maximum prompt size, also known as “context window”. It’s especially relevant in the ingestion phase of the Question & Answer use case, where the original text is split before being indexed with embeddings and stored in the vector database.

- **Document:** In LangChain, a Document is a fundamental data structure that encapsulates both content and metadata. It represents a unit of text—structured or unstructured—along with relevant contextual information. For example, the text extracted from a PDF file can be wrapped in a Document object, with metadata storing details like the source file name or page number. When working with larger texts, a Text Splitter can divide them into smaller segments, each represented as an individual Document, making it easier to process them in chunks.
- **Embedding models (3):** LangChain provides support for the most popular embedding models through convenient wrappers. These models are designed to capture the semantic meaning of text and convert it into numerical vectors.
- **Vector stores (4):** vector stores operate as specialized databases designed for the efficient retrieval of Document objects. These objects typically represent fragments or "chunks" of the original document, indexed based on their associated embeddings or vector-based semantic representations. This indexing allows for search queries to match the embeddings of the chunks against those of the query, retrieving the most similar embeddings and their related chunks. By ingesting Documents along with their associated embeddings, the vector store can serve as an offline knowledge base for proprietary data. LangChain is compatible with various popular vector stores.
- **Knowledge Graph databases:** Although not a key component of the architecture, LangChain offers client wrappers for leading graph databases to facilitate Knowledge Graph functionality. These databases store entities and their relationships in a graph form.
- **Retrievers (5):** In LangChain, retrievers efficiently fetch data, often a list of Documents containing unstructured text, from indexed databases like vector stores. LangChain provides support for various retrievers, catering not only to vector stores but also to relational databases and more complex data stores, such as knowledge graph databases like Neo4j.
- **Prompts (6):** LangChain provides tools for defining *prompt templates*—reusable structures that can be dynamically filled with input to create prompt instances. These inputs may come from users or external sources such as vector stores. Once a prompt instance is fully constructed, it is sent to the Language Model (LLM) for processing. To improve prompt

quality and guide the model's behavior, you can enhance templates by including examples, a technique often used in few-shot learning.

- LLM Cache (7): An optional component that improves performance and reduces costs by minimizing calls to the underlying LLM. It provides immediate answers to previously asked questions.
- LLM / ChatModel (8): This component acts as an interface to an LLM or a chat model built on an LLM. LangChain supports multiple LLMs and associated chat models, including those from OpenAI, Cohere, and HuggingFace. Additionally, LangChain supports a Fake LLM for unit testing purposes.
- Output Parser (9): This component transforms an LLM's natural language response into a structured format, such as JSON, making it easier for the LLM application to process.

The components above can be organized into a chain or structured around agents:

- Chain: A composite arrangement guiding LangChain's processing workflow, customized for specific use cases and based on a sequence of the described components.
- Agent: This component manages a dynamic workflow, extending a sequential chain. The agent's processing is flexible and can adapt based on user input or component output. Resources in the dynamic workflow are called "tools" or "plugins" in most frameworks' documentation, and the collection of all tools is referred to as the "toolkit."

LangChain's comprehensive design supports three primary application types, which we'll examine in detail shortly: summarization and query services, chatbots, and agents. Although the framework can seem complex at first glance, the detailed explanations in upcoming chapters will break down each component and clear up any uncertainties.

1.2 LangChain core object model

With a good understanding of LangChain's high-level architecture, we can now turn to its core object model for a clearer picture of how the framework operates. Gaining familiarity with the key objects and their interactions will

significantly improve your ability to use LangChain effectively.

The object model is organized into class hierarchies, beginning with abstract base classes from which multiple concrete implementations are derived. Figure 1.2 provides a visual overview of the main class families used across various tasks, all centered around the Document entity. It illustrates how loaders generate Document objects, how splitters divide them into smaller segments, and how these are then passed into vector stores and retrievers for downstream processing.

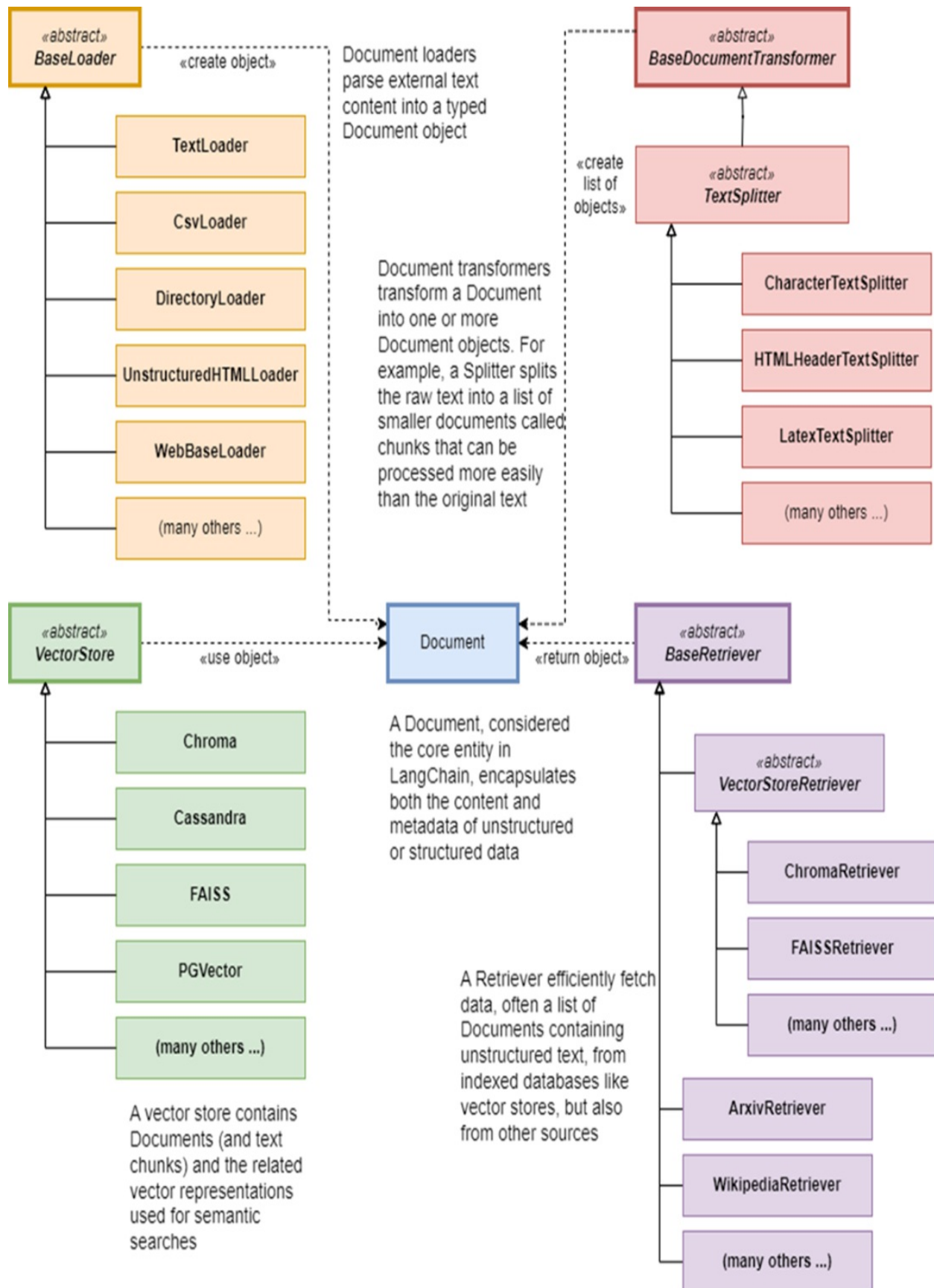
As covered in section 1.1.1 on LangChain architecture, the primary classes include:

- Document
- DocumentLoader
- TextSplitter
- VectorStore
- Retriever

LangChain integrates with a wide variety of third-party tools and services across these components, offering great flexibility. You can find a full list of supported integrations in the official documentation. In addition, LangChain provides the LangChain Hub, a community-driven repository for discovering and sharing reusable components such as prompts, chains, and tools.

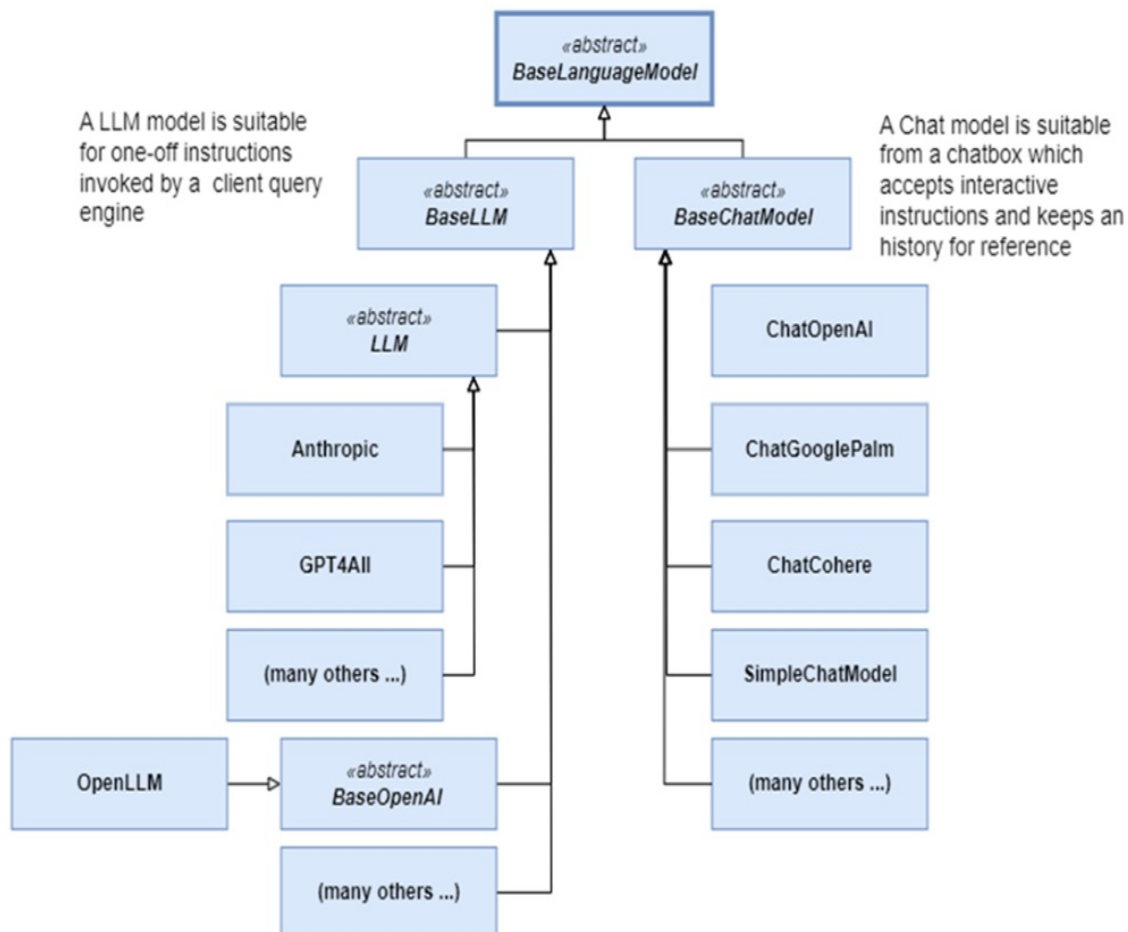
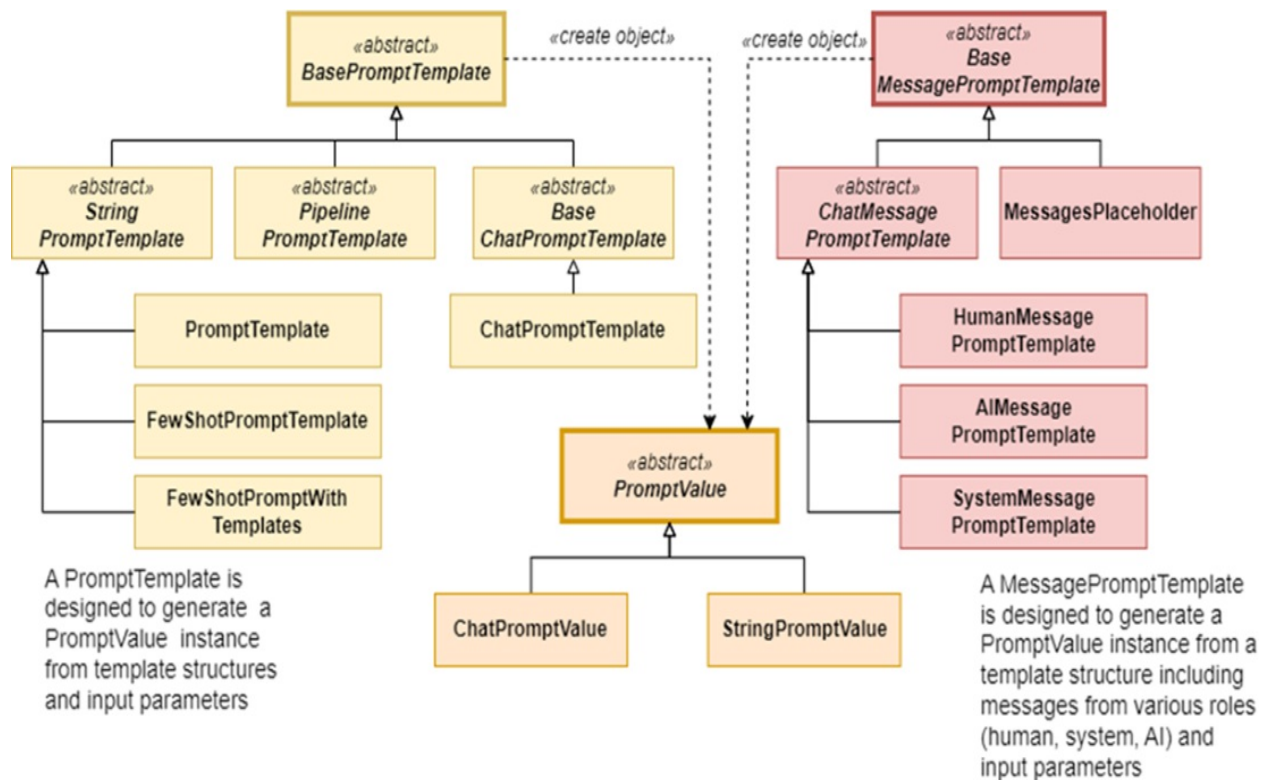
A key architectural feature shared by many of these components—from loaders to LLMs—is the Runnable interface. This common interface allows objects to be composed and chained together consistently, enabling highly modular workflows. We'll dive deeper into this feature, and into the LangChain Expression Language (LCEL), in a later section focused on building composable and expressive LLM pipelines.

Figure 1.2 Object model of classes associated with the Document core entity, including Document loaders (which create Document objects), splitters (which create a list of Document objects), vector stores (which store Document objects in vector stores) and retrievers (which retrieve Document objects from vector stores and other sources)



In figure 1.3, you can see the object model related to Language Models, including PromptTemplate and PromptValue. This figure illustrates how these classes connect to the LLM interface, exposing a somewhat more complex hierarchy than the classes presented earlier.

Figure 1.3 Object model of classes associated with Language Models, including Prompt Templates and Prompt Values



Now that you've reviewed LangChain's architecture and object model, let's pause and explore the types of applications you can build with it.

1.3 Building LLM applications and AI agents

LLMs are great at handling natural language—they can understand text, generate it, and pull out information when you need it. That makes them useful for all kinds of applications: summarization, translation, sentiment analysis, semantic search, chatbots and

code generation. Because of this range, you'll now see LLMs showing up in fields as varied as education, healthcare, law, and finance.

Even with all these different use cases, most LLM apps end up looking pretty similar under the hood. They take in natural language input, work with unstructured data, pull in extra context from one or more sources, and then package everything into a prompt for the model to process.

From a high level, these applications generally fall into three main categories:

- LLM-based applications or engines – Systems that provide a specific capability such as summarization, search, or content generation.
- Chatbots – Conversational interfaces that maintain context over multiple exchanges.
- AI agents – Autonomous or semi-autonomous systems that use LLMs to plan and execute multi-step tasks, often interacting with external tools or APIs.

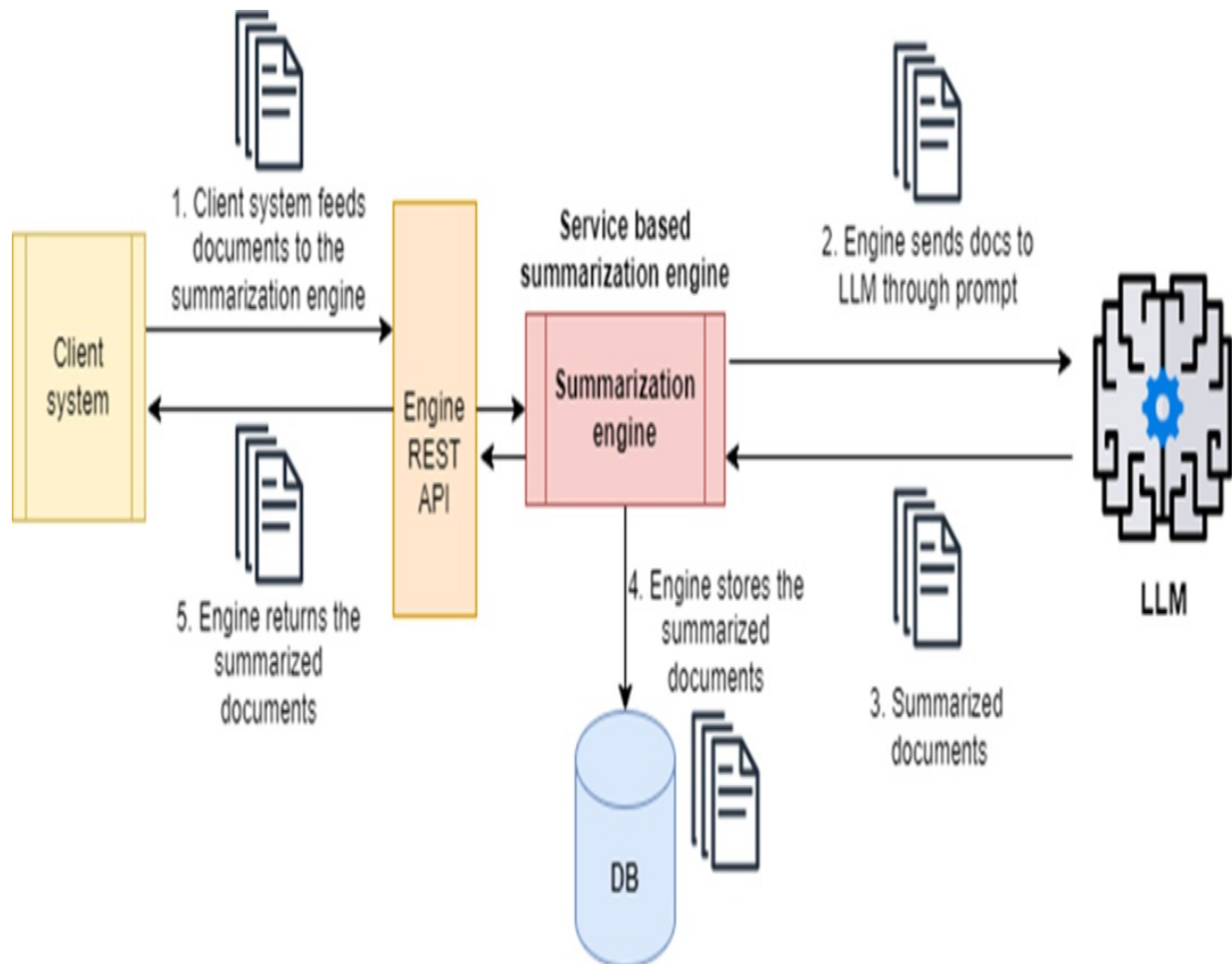
We'll examine each of these categories in turn before exploring how LangChain supports their development—starting with LLM-based applications and engines.

1.3.1 LLM-based applications: summarization and Q&A engines

An LLM-based engine acts as a backend tool that handles specific natural

language requests for other systems. For example, a summarization engine condenses lengthy text passages into concise summaries. These summaries can be returned immediately to the client or stored in a database for later use by other applications, as shown in Figure 1.4.

Figure 1.4 A summarization engine efficiently summarizes and stores content from large volumes of text and can be invoked by other systems through REST API.



Summarization engines are often deployed as shared services, typically exposed through a REST API so multiple systems can call them on demand. Another common type is the Question & Answer (Q&A) engine, which answers user queries against a knowledge base. A Q&A engine works in two phases: ingestion and query.

In the ingestion phase, the engine builds its knowledge base by pulling in

text, splitting it into chunks, and converting those chunks into embeddings—mathematical vectors that capture meaning. Both the embeddings and the original chunks are stored in a vector store for efficient retrieval. Don't worry if “embedding model” or “vector store” sound unfamiliar; we'll cover them in detail later. For now, just think of this step as transforming raw text into a searchable representation.

Definition

Embeddings are vector representations of words, tokens, or larger text units—such as sentences, paragraphs, or document chunks—mapped into a continuous, high-dimensional space (typically with hundreds or thousands of dimensions). These vectors capture semantic and syntactic relationships, allowing language models to understand meaning, context, and similarity. Embeddings are learned during the pre-training phase, where models are trained on large-scale text corpora to predict tokens based on surrounding context. By encoding both individual words and broader semantic meaning, embeddings enable more effective reasoning, retrieval, and language understanding.

In the query phase, the engine takes a user's question, turns it into an embedding using the same model, and performs a semantic search over the vector store. The most relevant chunks are retrieved and combined with the original question to form a prompt, which is then sent to the LLM. The model uses both the question and the retrieved context to generate an accurate, grounded answer.

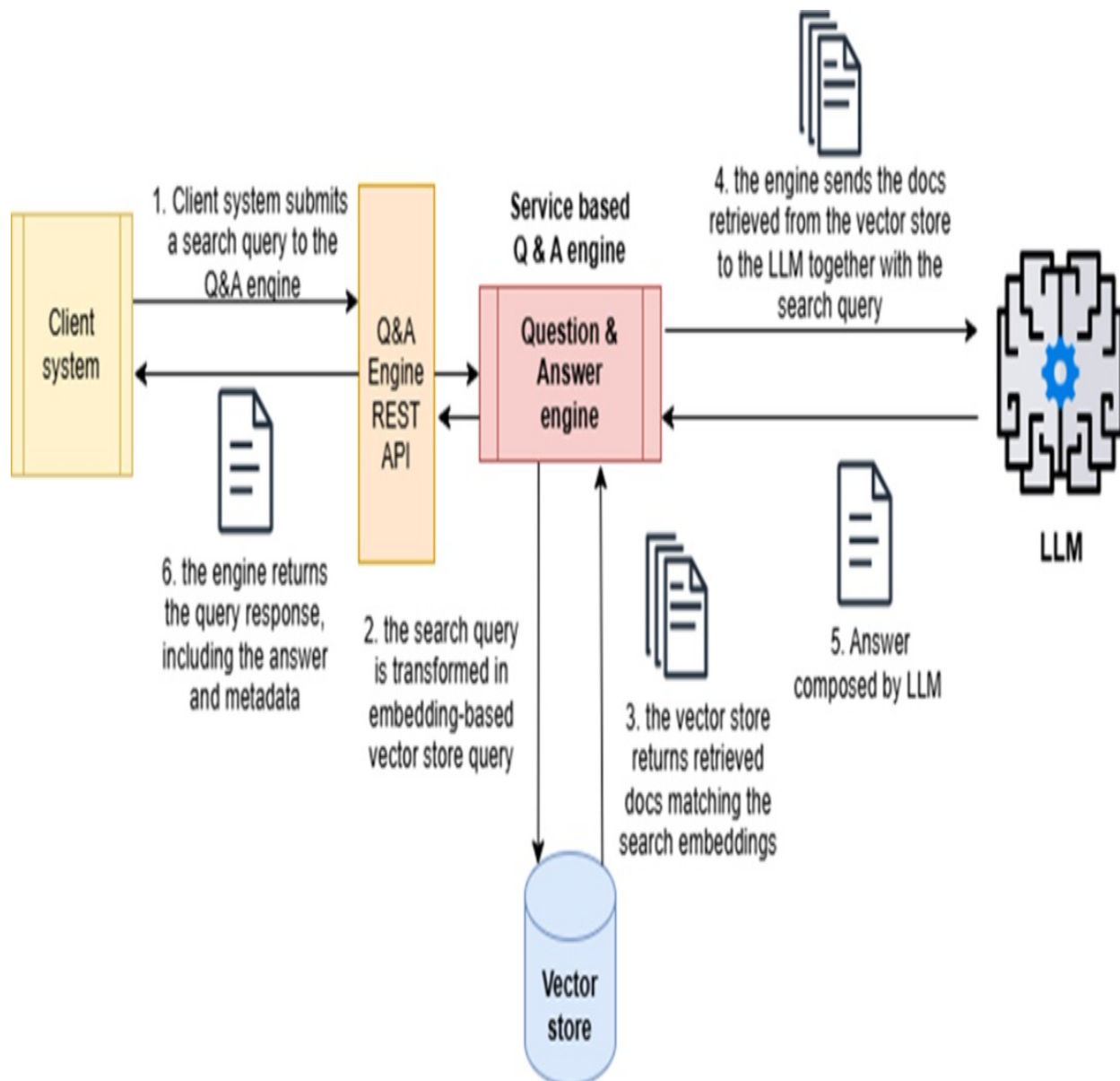
This workflow is known as *Retrieval-Augmented Generation* (RAG), shown in Figure 1.5. RAG has quickly become a cornerstone of modern LLM applications, and we'll spend three full chapters (8, 9, and 10) diving into advanced techniques for making it more powerful and reliable.

Definition

Retrieval-Augmented Generation (RAG) is a design pattern where the LLM's text “generation” is “augmented” by incorporating additional context, “retrieved” from a local knowledge base—often stored in a vector store—at

query time.

Figure 1.5 A Q&A engine implemented with RAG design: an LLM query engine stores domain-specific document information in a vector store. When an external system sends a query, it converts the natural language question into its embeddings (or vector) representation, retrieves the related documents from the vector store, and then gives the LLM the information it needs to craft a natural language response.



LangChain makes it straightforward to build engines like question-answering systems by giving you modular components out of the box: document loaders, text splitters, embedding models, vector stores, and retrievers. You

can snap these together with minimal boilerplate, which means you spend more time on your application's logic and less time wrestling with low-level details. Later in the book, we'll look at how to evaluate and choose among different embedding models and vector stores. LangChain supports many popular options, so you can mix and match without getting locked into a single vendor.

Engines aren't limited to Q&A. They can also call external tools by running predefined sequences of steps, often called chains. For example, an engine handling a user request might convert natural language instructions into API calls, pull data from outside systems, and then use an LLM to interpret and present the results in a clean, human-readable format.

At their core, these engines are designed for system-level work: automating processes, processing data intelligently, and stitching together different platforms. They simplify workflows that involve natural language by handling the messy parts—retrieval, transformation, orchestration—so you don't have to.

But once a human is directly interacting with the system in real time, the game changes. At that point, the engine takes on a conversational role, shifting from pure automation into dialogue. This is where chatbots come into play.

1.3.2 LLM-based chatbots

An LLM-based chatbot acts as an intelligent assistant, enabling ongoing, natural conversations with a language model. Unlike simple question-answer scripts, these systems aim to keep interactions both useful and safe. They rely heavily on prompt design: clear instructions shape the model's behavior and help prevent irrelevant, inaccurate, or unsafe replies. Modern chat APIs—such as those from OpenAI—support role-based messaging formats (system, user, assistant), which let you define an assistant's persona and enforce consistent behavior across a conversation.

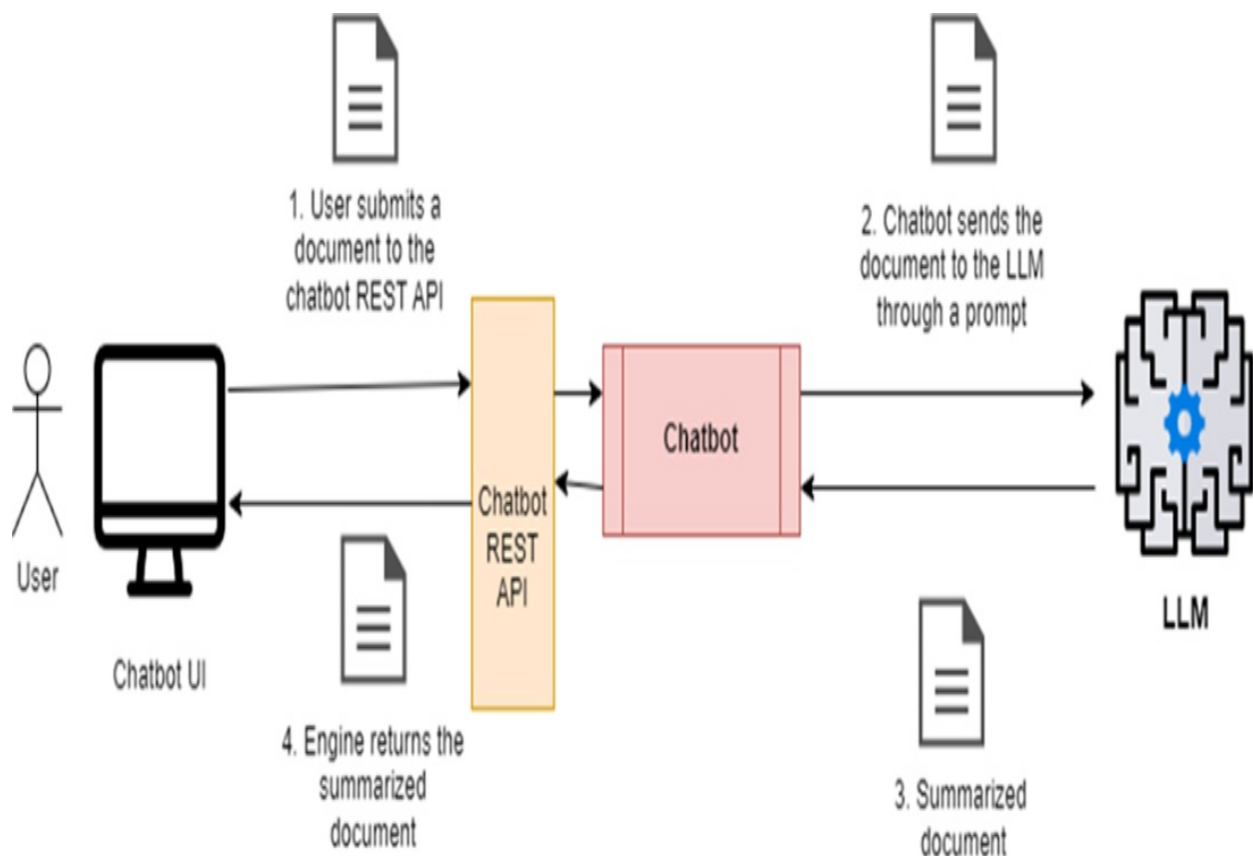
To improve accuracy, chatbots often pull in factual context from local knowledge sources like vector stores. This lets them blend conversational

fluency with domain-specific grounding, so the answers aren't just smooth but also relevant and reliable.

A key strength of chatbots is conversation memory. By tracking earlier turns, they can keep responses coherent and personalized. This memory is limited by the model's context window. Larger windows help, but many systems still compress or summarize conversation history to stay within limits—and to manage cost.

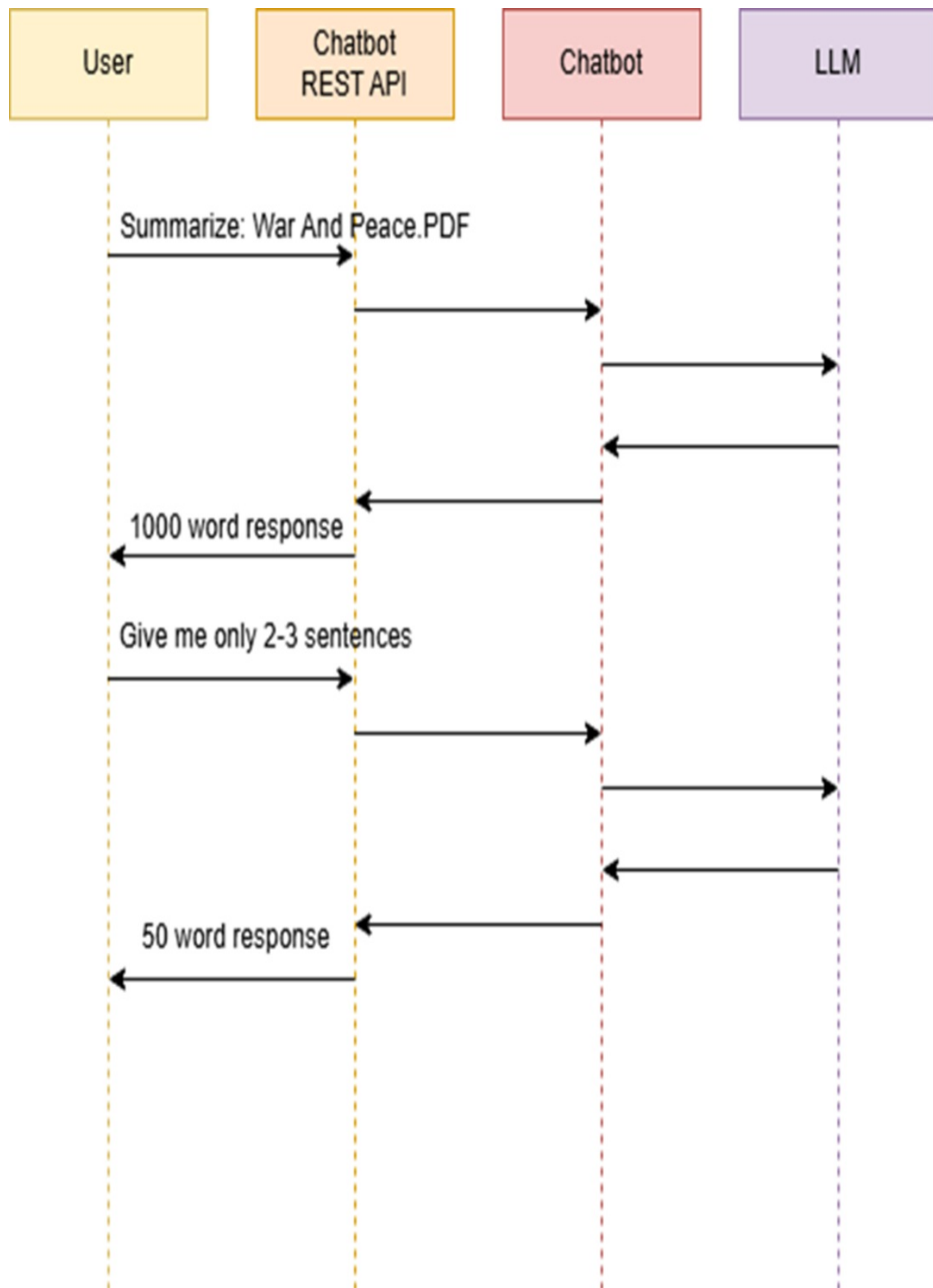
LLM-based chatbots are usually specialized for tasks such as summarization, question answering, or translation. They can either respond directly to user input or combine it with stored knowledge. For example, the architecture of a summarization chatbot (figure 1.6) builds on a basic summarization engine, but adds dialogue management and context-awareness layers.

Figure 1.6 A summarization chatbot has some similarities with a summarization engine, but it offers an interactive experience where the LLM and the user can work together to fine-tune and improve the results.



The crucial difference between a summarization engine and a summarization chatbot is interactivity. A chatbot lets you refine responses in real time: if you want a shorter or more casual summary, you can just ask, as illustrated in the sequence diagram in Figure 1.7.

Figure 1.7 Sequence diagram that outlines how a user interacts with an LLM through a chatbot to create a more concise summary.



This back-and-forth makes the process collaborative—producing answers that feel more tailored and context-aware. In the next chapter, we’ll explore techniques like role instructions, few-shot examples, and advanced prompt engineering to give you more control over chatbot behavior and output.

1.3.3 AI agents

An AI agent is a system that works with a large language model (LLM) to carry out multi-step tasks—often involving multiple data sources, branching logic, and adaptive decision-making. Unlike simple pipelines, agents can operate with a degree of independence: they follow the constraints you set, but they make choices about what to do next.

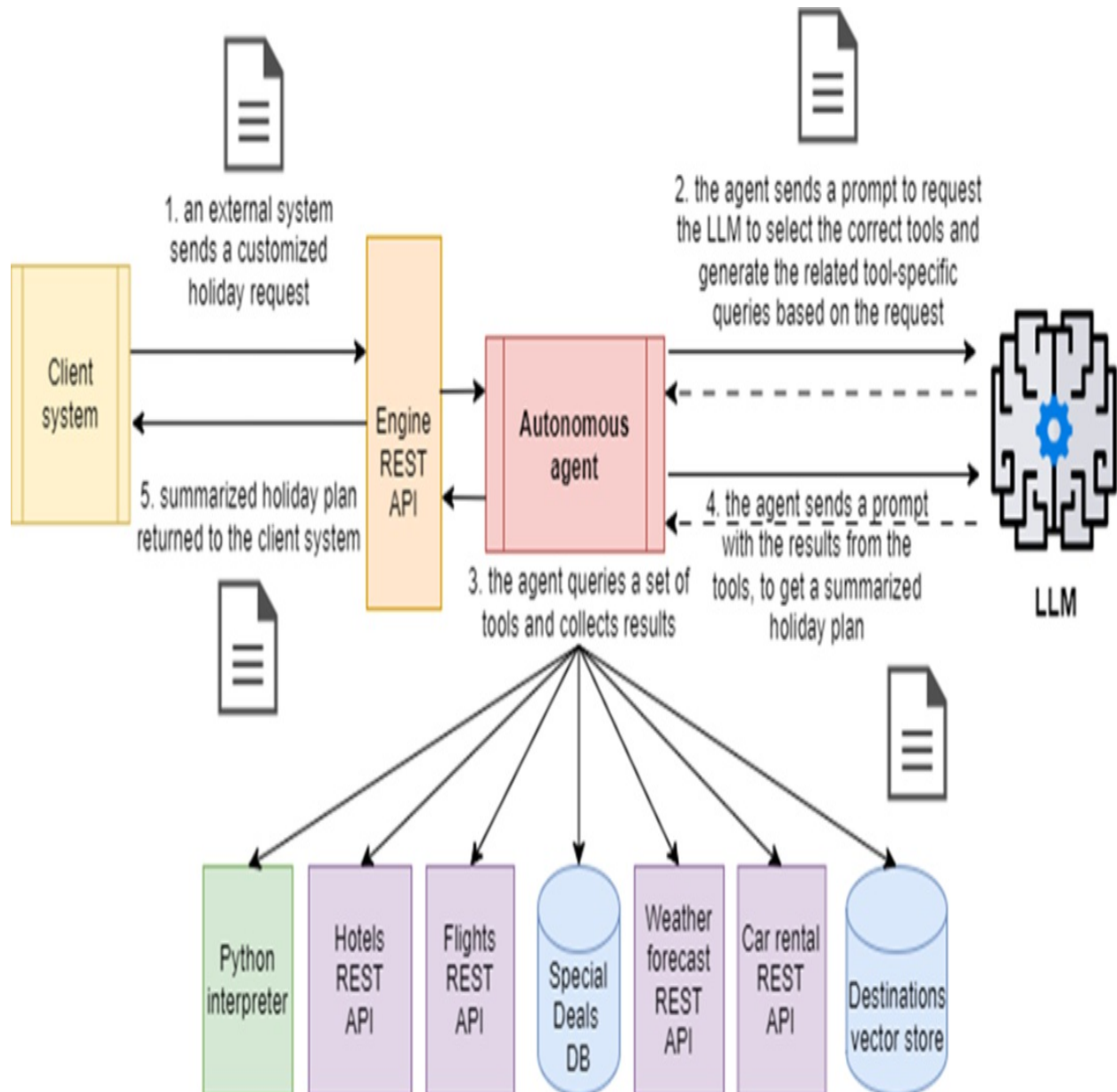
At each step, the agent consults the LLM to decide which tools to use, runs those tools, and processes the results before moving on. This loop continues until the agent produces a complete solution. In practice, that might mean pulling information from both structured sources (like databases or APIs) and unstructured ones (like documents or web pages), combining the results, and presenting them in a coherent format.

Consider this example: a tour operator uses an AI agent to generate holiday packages based on natural language requests from a booking website. As shown in Figure 1.8, the process could look like this:

1. The agent sends a prompt to the LLM asking it to choose the most relevant tools for the request—for example, flight, hotel, and car rental providers; weather forecast services; and internal holiday deals databases.
2. Guided by a developer-crafted prompt listing available tools and their descriptions, the LLM selects the appropriate ones and generates the required queries—such as SQL for the holiday deals database or REST API calls for external providers.
3. The agent executes the queries, gathers the results, and sends another prompt to the LLM containing both the original holiday request and the collected data.
4. The LLM responds with a summarized holiday plan that includes all

bookings, which the agent then returns to the booking website.

Figure 1.8 Workflow of an AI agent tasked with assembling holiday packages: An external client system sends a customized holiday request in natural language. The agent prompts the LLM to select tools and formulate queries in technology-specific formats. The agent executes the queries, gathers the results, and sends them back to the LLM, along with the original request, to obtain a comprehensive holiday package summary. Finally, the agent forwards the summarized package to the client system.



Note

The workflow can involve multiple iterations between the agent and the LLM before producing a final output—such as a complete holiday plan. Also, the architecture shown in figure 1.8 is only one of the possible ones. An alternative design could be based on a set of more granular agents coordinated by a supervisor agent at the top. We'll explore these designs in chapters 11, 12 and 13. In high-stakes domains like finance or healthcare, it's common to include a human-in-the-loop step. This ensures that a human can review and validate critical actions—such as approving a financial transaction or confirming a complex medical recommendation—before finalizing them. In the holiday planning example, the agent could be programmed to pause and request human approval of the proposed itinerary before sending it to the client, adding an extra layer of oversight and trust. LangChain's agent framework also allows developers to incorporate human validation steps as part of the toolchain.

There's been a surge of interest in AI agents, and major players like OpenAI, Google, and Amazon have all released agent SDKs to encourage developers and companies to adopt the approach. LangChain's agent framework—especially with the introduction of LangGraph—is built to handle these multi-step, multi-tool workflows in a modular, controlled way.

LangGraph provides pre-built agent and orchestrator classes, along with ready-to-use tool integrations, so you don't have to reinvent the wheel. Between Chapters 11 and 13, you'll get hands-on experience building advanced agents with LangGraph, learning how to design, orchestrate, and refine them in practice.

In many ways, an AI agent represents the most advanced form of LLM application. It leverages the full spectrum of LLM capabilities—understanding, generating, and reasoning over text—to drive complex, automated workflows. Agents dynamically select and use multiple tools, guided by prompts you design, making them powerful across industries like finance, healthcare, and logistics where multi-step reasoning and data integration are essential.

Interest in agents has accelerated with the introduction of the Model Context Protocol (MCP) by Anthropic. MCP defines a standard way for services to expose tools through MCP servers, which agents can then access via MCP

clients as easily as if they were local components. This shifts the integration burden to the service itself, letting developers focus on building capable agents rather than writing and maintaining custom connectors. Since its release in late 2024, MCP has quickly become a de facto standard—adopted by major LLM providers like OpenAI and Google—and thousands of tools are now available through public MCP portals.

We'll dive into MCP in Chapter 13, where you'll learn the protocol, explore the ecosystem in practice, and build your own MCP server that can plug directly into an agent application.

Now that you've seen LangChain's purpose, architecture, and the core application types it supports, I encourage you to try out LangChain using the Jupyter notebook described in Appendix A. It's a quick way to get a feel for the framework before we dive deeper!

1.4 Typical LLM use cases

LLMs are applied across a wide range of tasks, from text classification to code generation and logical reasoning. Below are some of the most common use cases, along with real-world examples for further exploration:

- **Text Classification and Sentiment Analysis:** This includes categorizing news articles or recommending stocks based on sentiment analysis. For example, GoDaddy uses LLMs to automatically classify support tickets, as described in the article “LLM From the Trenches: 10 Lessons Learned Operationalizing Models at GoDaddy.”
- **Natural Language Understanding and Generation:** LLMs can identify main topics in a text and generate summaries tailored by length, tone, or terminology. Duolingo uses AI to accelerate lesson creation, detailed in their blog post “How Duolingo Uses AI to Create Lessons Faster.” Summarization is explored in depth in Chapters 3 and 4.
- **Semantic Search:** This involves querying a knowledge base based on the intent and context of a question, rather than relying on simple keywords. The Picnic supermarket app uses LLMs to enhance recipe search, as explained in the Medium post “Enhancing Search Retrieval with Large Language Models (LLMs).” We'll explore this extensively in the

context of Q&A chatbot development in Chapters 6 and 7, with deeper dives into advanced techniques in Chapters 8 to 10.

- **Autonomous Reasoning and Workflow Execution:** LLMs can handle tasks like planning a complete holiday package by understanding requests and managing each step of the process. We'll discuss building LLM-powered agents using LangGraph in Chapter 12.
- **Structured Data Extraction:** This involves pulling structured data—like entities and their relationships—from unstructured text, such as financial reports or news articles.
- **Code Understanding and Generation:** LLMs can analyze code to identify issues, suggest improvements, or generate new code components—ranging from simple functions and classes to entire applications—based on user instructions. This capability powers popular IDE extensions like GitHub Copilot and Cline AI, and emerging AI-driven coding assistants such as Cursor and Windsurf, which provide real-time code suggestions and error detection as you work. In addition, CLI-based coding tools like Anthropic's Claude Code and OpenAI Codex allow developers to interact with AI through the command line, enabling code generation, refactoring, and debugging directly from terminal environments.
- **Personalized Education and Tutoring:** LLMs are increasingly used as interactive tutors, providing personalized help and feedback. For instance, Khan Academy's "Khanmigo" uses an LLM to assist students with interactive learning.

These use cases assume the LLM can competently handle user requests. However, real-world tasks often involve specific domains or scenarios that extend beyond the LLM's initial training. How can you ensure your LLM meets user needs effectively in these cases? That's exactly what you'll learn in the next section.

1.5 How to adapt an LLM to your needs

There are several techniques to enhance the LLM's ability to respond to user requests, even without prior knowledge or training in a specific domain. These include:

- Prompt engineering

- Retrieval Augmented Generation
- Fine-tuning

Let's begin with the most basic approach: prompt-engineering.

1.5.1 Prompt engineering

Prompts for large language models (LLMs) can be as simple as a single command or as complex as a block of instructions enriched with examples and context. Prompt engineering is the practice of designing these inputs so that the model understands the task and produces useful, accurate responses. Done well, it allows developers to guide the model's behavior, adapt it to domain-specific needs, and even handle problems the model wasn't explicitly trained on. A common technique here is *in-context learning*, where the model infers patterns from examples embedded directly in the prompt—no fine-tuning required.

In practice, prompts are often organized as templates: a fixed instruction with variable fields that accept dynamic input. This makes prompts reusable and easier to manage across different parts of an application. A widely used pattern is *few-shot prompting*, where you include a handful of examples to teach the model how to generalize to similar inputs.

Well-crafted prompts are surprisingly powerful. They can coax high-quality, domain-aware output from an LLM without needing extra training data. For instance, chatbots often embed recent conversation turns into the prompt so the model maintains context and produces coherent, multi-turn replies. Prompt engineering can often take you very far, making it a lightweight yet effective tool for many real-world applications.

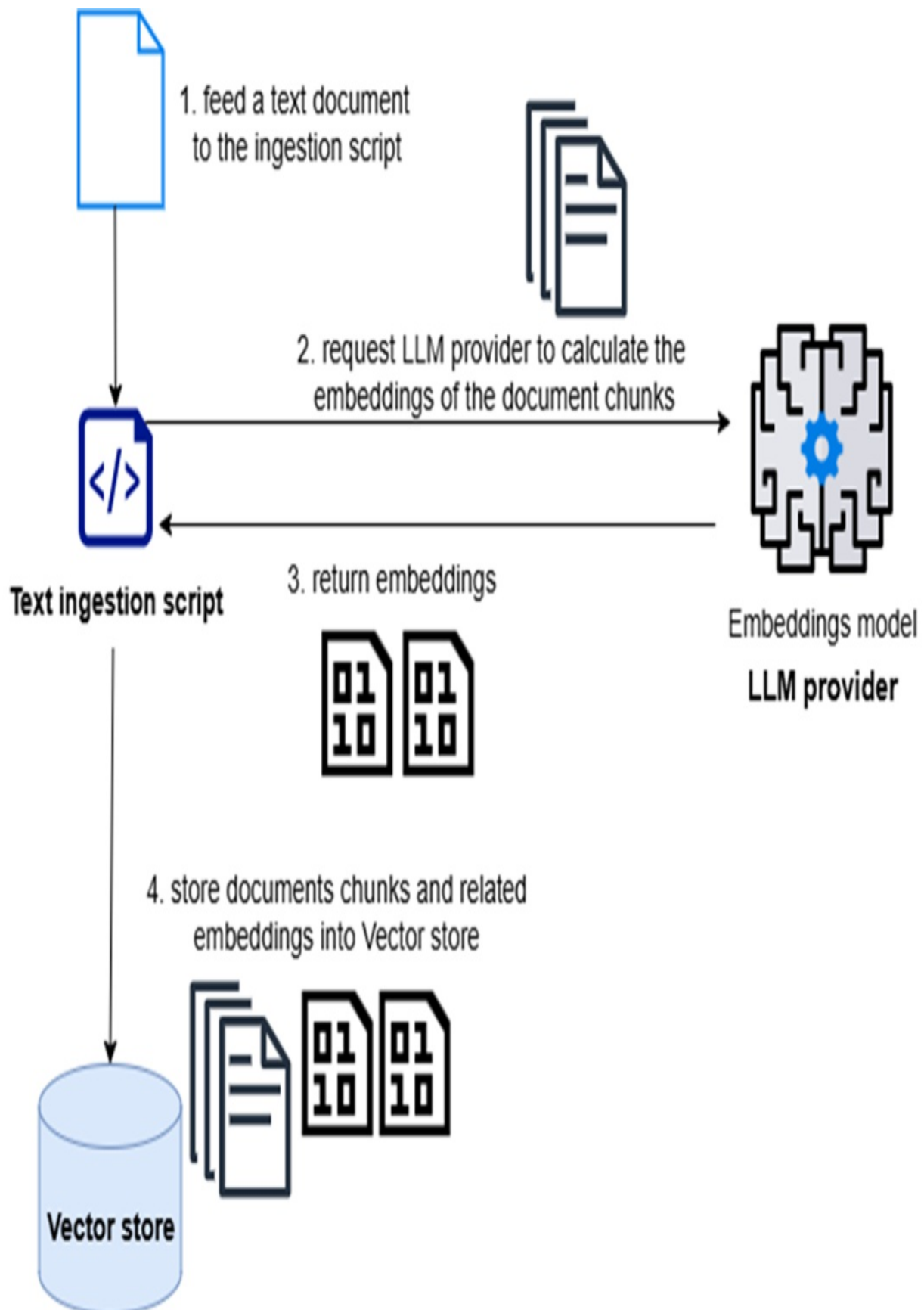
LangChain builds on this idea with abstractions for managing and reusing prompts consistently, which we'll explore in the next chapter. That said, prompt engineering alone has limits—especially when applications need to ground answers in user-specific or enterprise data. In those cases, the natural next step is *Retrieval-Augmented Generation* (RAG), which augments prompts with knowledge pulled dynamically from external sources.

1.5.2 Retrieval Augmented Generation (RAG)

One of the most effective ways to improve LLM responses is to ground them in your own data. Instead of relying only on what the model learned during pretraining, you can retrieve relevant context from a local knowledge base—usually stored in a vector database—and add it to the prompt. This approach, called Retrieval-Augmented Generation (RAG), has become a core pattern in modern LLM applications.

The workflow begins with building a knowledge base. Documents are ingested, split into smaller chunks, and converted into vector representations using an embedding model. These embeddings capture semantic meaning, making it possible to compare text by similarity rather than exact wording. LangChain provides tools to load documents in many formats, split them efficiently, and generate embeddings, then store everything in a vector database.

Figure 1.9 A collection of documents is split into text chunks and transformed into vector-based embeddings. Both text chunks and related embeddings are then stored in a vector store.



RAG offers multiple benefits:

1. *Efficiency*: Instead of passing an entire document to the model, you retrieve only the key chunks—keeping inputs concise, reducing token costs, and working within context limits.
2. *Accuracy*: Responses are *grounded* on real data, reducing the risk of *hallucinations*. In Chapter 6, you’ll learn techniques for having the LLM cite its sources, further improving transparency and trust.
3. *Flexibility*: By swapping embedding models, retrievers, or vector stores, you can adapt the same pattern to different domains and requirements.

Definition

"Grounding" an LLM involves crafting prompts that include context pulled from a trusted knowledge source—often stored in a vector store. This ensures that the LLM generates its response based on verified facts rather than relying solely on its pre-trained knowledge, which may include outdated or unreliable data.

Definition

A “hallucination” is when a large language model generates an incorrect, misleading, or fabricated response. This happens when the model draws from poor-quality data during training or lacks sufficient information to answer accurately. Due to their auto-regressive nature, LLMs will try to generate a response even when relevant content is missing—leading to hallucinations as they “fill in the gaps” with plausible-sounding but incorrect information.

To make RAG reliable, prompts should explicitly instruct the LLM to rely only on the retrieved context. LangChain also supports guardrails and validators to help enforce safe behavior. In high-stakes cases, human-in-the-loop review remains the best safeguard.

In short, RAG bridges the gap between static pretrained models and dynamic, domain-specific applications. By teaching your app to “speak the same language” as the LLM—vectors—you unlock a practical way to deliver grounded, verifiable, and cost-efficient answers. If prompt engineering and

RAG still don't meet your needs, the next step is fine-tuning the model, which we'll revisit later in the book.

1.5.3 Fine-tuning

Fine-tuning is the process of adapting a pre-trained LLM to perform better in a specific task or domain. This is done by training the model on a curated dataset of examples that capture the style, terminology, and reasoning patterns you want it to master. Traditionally, fine-tuning required specialized ML frameworks and access to powerful hardware, but today many platforms—including OpenAI's fine-tuning API and several open-source toolkits—make it possible with just a dataset upload and minimal configuration.

The main benefit of fine-tuning is efficiency: once a model has absorbed domain-specific knowledge, you don't need to stuff every prompt with long instructions or examples. Instead, the model “knows” how to respond in your context. The trade-offs, however, are real. Preparing high-quality datasets takes time and expertise, and training runs can be costly since they often require GPUs.

Recent advances like Low-Rank Adaptation (LoRA) and other parameter-efficient fine-tuning methods have lowered both cost and complexity, making fine-tuning more accessible. Related techniques, such as instruction tuning and *reinforcement learning from human feedback (RLHF)*, push models to follow directions more reliably, though they typically demand more engineering effort and infrastructure.

Whether fine-tuning is truly necessary is an ongoing debate. General-purpose LLMs are surprisingly strong out-of-the-box, especially when paired with Retrieval-Augmented Generation (RAG). In fact, research by Heydar Soudani, Evangelos Kanoulas, and Faegheh Hasibi (“Fine-Tuning vs. Retrieval-Augmented Generation for Less Popular Knowledge”) shows that RAG often outperforms fine-tuning by letting you provide context dynamically at runtime, reducing both costs and retraining needs.

That said, in highly specialized domains—such as medicine, law, or finance—fine-tuning remains invaluable. It allows models to capture domain-

specific jargon and workflows in ways generic models struggle to match. Well-known examples include:

- *BioMistral* (biology and life sciences)
- *LexisNexis's legal-domain LLM* (“LexiGPT”)
- *BloombergGPT* (finance)
- *Anthropic's Claude Code* and similar code-focused models

In summary, fine-tuning customizes an LLM for domain expertise and specialized accuracy, but it comes at a cost in time, money, and complexity. As a developer, you'll need to weigh when it's truly worth it versus when RAG or prompt engineering will get you there faster.

1.6 Which LLMs to choose

When developing LLM-based applications, you'll find a wide range of models to choose from. Some are proprietary, accessible via subscription or pay-as-you-go APIs, while others are open source and can be self-hosted. Most modern LLMs offer REST APIs for easy integration and user-friendly chat interfaces. Many also come in different size variants, letting you balance performance, speed, and cost.

LangChain makes it simple to integrate with a variety of LLMs. Thanks to its standardized interface, you can switch models with minimal code changes—an essential feature in today's fast-evolving LLM landscape. Below are key considerations for choosing the right LLM for a specific task.

- **Model Purpose:**
 - For general tasks like summarization, translation, classification, or sentiment analysis, most large model families (GPT, Gemini, Claude, Llama, Mistral) handle these as standard capabilities.
 - For specialized tasks—such as code generation—seek out models that are optimized or fine-tuned for that purpose. For instance, Claude Sonnet and Meta's Code Llama variants are well-regarded for coding tasks.
- **Context Window size:**
 - Larger A larger context window lets your application handle longer

prompts and documents. For example, some models support up to 2 million tokens, while many others offer 128K–256K tokens.

- Keep in mind: larger windows can increase both performance overhead and cost, especially for commercial APIs that charge per token.
- Multilingual Support:
 - If your app needs to handle multiple languages, opt for models with strong multilingual training. Qwen and Llama are especially capable in both Western and Asian languages. Some models may specialize—for example, certain Gemma releases are optimized for Japanese.
- Model size:
 - LLMs range from small (1B parameters) to very large (trillions of parameters). Smaller models can be more cost-effective and faster, often sufficient for simpler tasks.
 - For example, OpenAI mini and nano versions offer impressive accuracy at a lower cost and latency than its full-size counterpart.
- Speed:
 - Smaller and mid-size models typically respond faster. If responsiveness is critical (e.g., in chatbots), benchmark both performance and accuracy.
- Instruction vs Reasoning:
 - A key distinction in LLM behavior is between instruction models and reasoning models.
 - Instruction models, such as OpenAI’s GPT-4 series or Google Gemini Pro, are best suited when you already know exactly how a task should be performed and want the LLM to follow your detailed directions. They excel at accurately executing well-described instructions. You can think of them as reliable junior assistants: fast, precise, and effective when the steps are clear.
 - Reasoning models, such as OpenAI’s o-series or Google Gemini Thinking, are designed for situations where the task is defined but the method is not. These models not only execute work but also decide how to approach it. They are closer to senior problem-solvers—capable of planning, adapting, and filling in missing details.
 - The trade-offs are cost and speed: reasoning models are generally

slower and more expensive, while instruction models are faster and more economical. Choosing between them depends on whether you want the model to simply follow a plan you provide or to figure out the plan itself.

- Open-Source vs. Proprietary:
 - Open-source models (Llama, Mistral, Qwen, Falcon) provide stronger data privacy and allow on-premise or private cloud deployment.
 - Proprietary models are easier to set up via API and may deliver best-in-class performance out of the box—but long-term costs can be high. Many teams start with commercial APIs, then migrate to open-source hosting for cost or compliance reasons.

By weighing these factors, you can select an LLM that aligns with your project's goals, technical requirements, and budget. Keep in mind that your application may involve multiple tasks, each with its own demands for accuracy, speed, and cost-efficiency. In many cases, it's wise to configure your system to use different models for different tasks. For example, in one recent project, I used GPT-4o Mini for summarization and sentiment analysis, o3 to interpret and route user queries to the appropriate agents, and GPT-4.1 for answer synthesis—balancing performance and cost across the workflow.

1.7 What You'll Learn from this Book

If you've read this far, you already recognize the value of building applications powered by large language models. LLM interaction is rapidly becoming a core feature of modern software—similar to how websites, in the early 2000s, opened up a new communication channel for server-based applications. And this shift is only accelerating.

We'll begin with prompt engineering, the foundational skill for effective interaction with LLMs. You'll start by experimenting interactively with ChatGPT, then move to programmatic access via REST APIs. These exercises will establish the backbone for many projects in this book.

From there, we'll use LangChain to build two categories of applications:

custom engines (such as summarization and Q&A systems) and chatbots that combine conversational fluency with knowledge retrieval. Each project will be small and self-contained, but all will share a common background theme in the travel industry, so you can see how the ideas fit together in a coherent domain.

Once those foundations are in place, we'll move to LangGraph to build AI agents—applications that can orchestrate multi-step workflows, coordinate tools, and make adaptive decisions. This will be your introduction to the most advanced class of LLM-powered systems, where engines and tools come together into dynamic, autonomous applications. To make this approachable, I'll start with a simple Python script and then extend it step by step. Each extension will branch into a different capability—such as tool use, planning, or memory—so you can see how agents grow in sophistication without being overwhelmed all at once.

We'll also take a deep dive into Retrieval-Augmented Generation (RAG), the architectural pattern behind many real-world systems. RAG concepts will be introduced through short, focused scripts—some independent, others progressively refined into more advanced workflows—so you can master both fundamentals and cutting-edge techniques.

Although the examples reference OpenAI models for accessibility and quick wins, you'll also learn how to use open-source models through the inference engines listed in Appendix E. This way, you'll gain the skills to build and deploy applications that are cost-effective, privacy-conscious, and fully under your control.

Beyond building, you'll explore the entire lifecycle of LLM applications: debugging, monitoring, and refining with LangSmith; orchestrating complex workflows with LangGraph; and applying best practices for production deployment to ensure scalability and maintainability.

By the end of this book, you'll have built a portfolio of working applications, learned the core architectural patterns, and developed the skills to design and implement LLM-powered systems with confidence. You won't just understand how these applications work—you'll be ready to keep innovating and pushing the boundaries of what's possible with LangChain, LangGraph,

and large language models.

1.8 Recap on LLM terminology

If you're new to LangChain, LLMs, and LLM applications, you'll encounter many terms in this chapter. The following table will help clarify these concepts.

Table 1.1 LLM Application Glossary

Term	Definition
LLM	A Large Language Model trained on vast text datasets to perform tasks like understanding, generating, and summarizing text.
Embedding	Numerical vector representations of words or tokens in a high-dimensional space, capturing their semantic and syntactic meaning.
Embedding Model	Models that convert text into vectors, capturing the meaning for efficient processing and retrieval in LangChain.
Vector Store	A specialized database for storing and retrieving vector-based representations of documents, supporting efficient search and retrieval.
Prompt	A request to guide the LLM, often created from templates and user inputs, for processing specific tasks.
Prompt Engineering	The craft of designing effective prompts to guide the LLM in generating accurate responses, often through trial and refinement.
Completion	The response generated by an LLM when given a prompt, based on the model's predictive text abilities.
LLM-Based Engine	A backend service using LLMs for tasks like summarization or analysis, powering other applications with language processing.
LLM-Based Chatbot	A chatbot using an LLM, optimized with prompts and instructions to guide interactions and maintain safety and relevance.

RAG	Retrieval Augmented Generation combines LLM outputs with context retrieved from a local knowledge base for better accuracy.
Grounding	Supplying the LLM with verified context from a trusted source to ensure reliable responses.
Hallucination	The model's tendency to generate incorrect or fabricated information when lacking sufficient context or relying on low-quality data.
LLM-Based Agent	An autonomous system that orchestrates complex workflows by coordinating across multiple tools, including remote tools hosted by MCP servers, and data sources, executing multi-step tasks through interactions with a language model.
Fine-Tuning	Adapting a pre-trained LLM for specific tasks by training it on examples, enhancing its domain-specific knowledge.

Each term has been introduced in the chapter to support your understanding of LangChain and its applications.

1.9 Summary

- LLMs have rapidly evolved into core building blocks for modern applications, enabling tasks like summarization, semantic search, and conversational assistants.
- Without frameworks, teams often reinvent the wheel—managing ingestion, embeddings, retrieval, and orchestration with brittle, one-off code. LangChain addresses this by standardizing these patterns into modular, reusable components.
- LangChain’s modular architecture builds on loaders, splitters, embedding models, retrievers, and vector stores, making it straightforward to build engines such as summarization and Q&A systems.
- Conversational use cases demand more than static pipelines. LLM-based chatbots extend engines with dialogue management and memory, allowing adaptive, multi-turn interactions.
- Beyond chatbots, AI agents represent the most advanced type of LLM

application. Agents orchestrate multi-step workflows and tools under LLM guidance, with frameworks like LangGraph designed to make this practical and maintainable.

- Retrieval-Augmented Generation (RAG) is a foundational pattern that grounds LLM outputs in external knowledge, improving accuracy while reducing hallucinations and token costs.
- Prompt engineering remains a critical skill for shaping LLM behavior, but when prompts alone aren't enough, RAG or even fine-tuning can extend capabilities further.

2 Executing prompts programmatically

This chapter covers

- Understanding prompts and prompt engineering
- Different kinds of prompts and how they're structured
- Enhancing prompt responses using one, two, or a few-shot learning
- Examples of using prompts with ChatGPT and the OpenAI API

Applications built with LangChain interact with LLMs primarily through prompts—structured inputs that guide the model’s behavior. Think of it as giving instructions to a highly capable but inexperienced colleague: the clearer and more specific your instructions, the better the outcome. To generate accurate and relevant responses, your prompts must be well-crafted and tailored to the task at hand. In LangChain, prompt design plays a central role in determining how your application performs.

Prompt engineering—the practice of designing and refining prompts to steer the LLM’s output—is a core skill in LLM application development. You’ll spend a significant amount of time creating, testing, and iterating on prompts to ensure your system consistently delivers high-quality results. LangChain recognizes this and places prompt engineering at the heart of its design, offering a suite of tools to support the process. Whether you're writing simple instructions or building sophisticated templates that require advanced reasoning and dynamic input handling, LangChain provides the infrastructure to manage and scale your prompt-driven workflows effectively.

In this chapter, you'll begin with the basics of prompt design and gradually move to more sophisticated techniques, such as Chain of Thought. LangChain's suite of prompt engineering tools, including `PromptTemplate` and `FewShotPromptTemplate`, will be your key resources as you learn to harness the full power of LLMs in your applications.

2.1 Running prompts programmatically

LangChain applications rely on well-crafted prompts to generate completions, which are then passed to the next component in the chain. Unlike prompts entered manually in interfaces like ChatGPT, LangChain prompts are typically constructed and sent to the LLM programmatically as part of a larger workflow. In the following sections, we'll start by setting up and executing prompts using the OpenAI API directly, and then explore how to run and manage prompts within LangChain.

Before diving in, make sure you've covered these basics:

1. You have an OpenAI API key.
2. You know how to create a Python Jupyter notebook environment.

If you're not familiar with these tasks, check the sidebar for guidance on "Creating an OpenAI key" in Section 1.7 and follow the instructions in Appendix B titled "Setting up a Jupyter notebook environment."

2.1.1 Setting up an OpenAI Jupyter Notebook environment

Assuming you have cleared the prerequisites, open the operating system terminal shell (e.g., cmd on Windows), create a project folder, and navigate into it:

```
C:\Github\building-llm-applications\ch02>
```

Create and activate a virtual environment with venv:

```
C:\Github\building-llm-applications\ch02>python -m venv env_ch02
C:\Github\building-llm-applications\ch02>.\env_ch02\Scripts\activ
```

You should now observe the updated operating system prompt, displaying the environment name in front as (env_ch02):

```
(env_ch02) C:\Github\building-llm-applications\ch02>
```

Having activated the virtual environment, you're now prepared to implement a Jupyter notebook for executing prompts with OpenAI models. If you have

cloned the repository from Github, you can install Jupyter and the OpenAI packages as follows:

```
(env_ch02) C:\Github\building-llm-applications\ch02>pip install -
```

Then you can start the notebook:

```
(env_ch02) C:\Github\building-llm-applications\ch02>jupyter noteb
```

Otherwise, if you are creating everything from scratch locally, install the packages in this way:

```
(env_ch02) C:\Github\building-llm-applications\ch02>pip install n
```

After about a minute, the installation of the notebook and OpenAI packages should be finished. You can start the Jupyter notebook by executing the following command:

```
(env_ch02) C:\Github\building-llm-applications\ch02>jupyter noteb
```

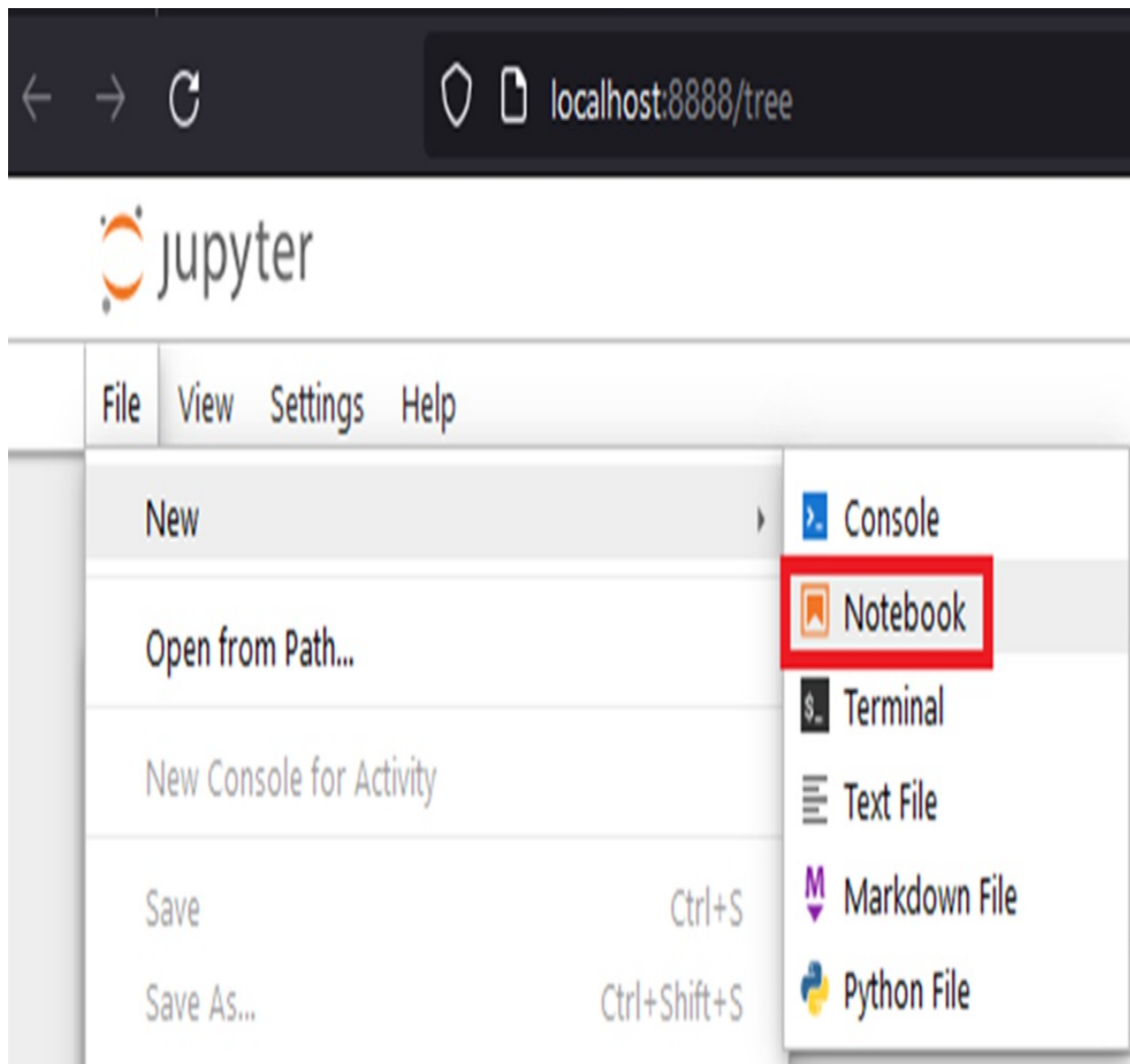
After a few seconds, you should see some output in the terminal.

Subsequently a browser window will open up at this URL:

<http://localhost:8888/tree>.

Once you have created the notebook, go to the Jupyter menu, select: File > Rename ... and name the notebook file: 02-prompt_examples.ipynb.

Figure 2.1 Creating a new Jupyter notebook: create the notebook with the menu File > New > Notebook and then rename the file to prompt_examples.ipynb



Now, you're prepared to input code into the notebook cells (or simply execute them if you got the notebook from Github).

TIP

If you are unfamiliar with Jupyter notebook, remember to press Shift + Enter to execute the code in each cell.

Throughout the remaining chapter, I assume you've set up a virtual environment, installed the OpenAI library, and launched a Jupyter notebook instance, as outlined in the previous section.

2.1.2 Minimal prompt execution

In the first cell of your notebook, import the required libraries and grab the OpenAI API key in a secure way (never hardcode it as it might get misused):

```
from openai import OpenAI
import getpass

OPENAI_API_KEY = getpass.getpass('Enter your OPENAI_API_KEY')
```

After entering your OpenAI API key (you only need to hit Enter, without Shift), set up the OpenAI client as follows:

```
client = OpenAI(api_key=OPENAI_API_KEY)
```

In the next notebook cell, enter and execute the following code. It's based on one of the prompts discussed in the "prompt basics" section, and you can find details about the `chat.completions.create` function in the accompanying sidebar.

```
prompt_input = """Write a short message to remind users to be vig
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."}
        {"role": "user", "content": prompt_input}
    ],
    temperature= 0.7,
    max_tokens= 400
)

print(response)
```

The output will look like this (I have shortened it for convenience):

```
ChatCompletion(id='chatcmp1-8Wr77Lor0Xusp7V7iM2NURV8dbwVj', choic
```

For a clearer output, explore the attributes and properties of the response object:

```
print(response.choices[0].message.content)
```

You will see an output similar to the following, though it may vary slightly due to the non-deterministic nature of LLMs—especially with the temperature parameter set above zero, which increases randomness:

Attention users,

Please be vigilant and stay alert regarding phishing attacks. Cyb

Stay safe and secure online!

Best regards,

[Your Name/Company Name]

Note

You may not be familiar with the three double quotes (""") I have used to formulate the prompt and may wonder about their purpose. This allows you to enter a block of text spanning multiple lines in a very readable way, without having to introduce newline characters (\n), which is ideal for capturing prompts.

chat.completions.create

The `chat.completions.create` function is essential for interacting with the OpenAI LLM, so it's crucial to understand its signature in detail:

```
client.chat.completions.create(  
    model="gpt-4o-mini",  
    messages=[  
        {"role": "system", "content": "You are a helpful assistant."}  
        {"role": "user", "content": prompt_input}  
    ],  
    temperature= 0.7,  
    max_tokens= 400  
)
```

Here is a description of the parameters:

- **model:** This refers to the OpenAI model to be used. Some model families, such as GPT-4o and GPT-4o-mini, have generic capabilities, understanding and generating text. Other models, like the "moderation"

family, are more specialized. Within each family, there are variants that can accept bigger or smaller prompts or are newer, and therefore recommended over more obsolete variants. Learn more on this page on OpenAI Models: <https://platform.openai.com/docs/models>.

- messages: A prompt can be composed of a list of messages (when following the OpenAI convention), each impersonated by a “role” and having some “content” with the instructions to be followed.
- temperature: This is a "creativity" or "entropy" coefficient, ranging from 0 (no creativity, meaning the output will always be the same given the same prompt) to 2 (the output will vary considerably at each execution of the same prompt).
- max_tokens: This limits the number of output tokens, generally to reduce costs, as you are charged for the number of tokens read and generated.

Now that you've grasped the fundamentals of programmatically submitting prompts, you can proceed to learn and work with more intricate prompts.

2.2 Running prompts with LangChain

Before moving to more advanced prompts, let me show you how to replicate the same example in LangChain. This will help you familiarize yourself with its object model. In the following sections, I'll demonstrate how LangChain simplifies the implementation of more complex prompts that would be more challenging to implement from scratch.

Before adding LangChain code to your notebook, open a new OS shell and import the LangChain packages as you did in chapter 1 to avoid stopping the running OS shell (you do not need to do this if you installed the dependencies from `requirements.txt`, as explained earlier):

```
(env_ch02) c:\Github\building-llm-applications\ch02>pip install l
```

Once the installation is complete, add a new cell in the notebook and import the LangChain OpenAI wrapper library. Then, instantiate a connection to the LLM:

```
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI(openai_api_key=OPENAI_API_KEY,  
                  model_name="gpt-4o-mini")
```

You can now instantiate and execute the prompt you saw earlier as follows:

```
prompt_input = """Write a concise message to remind users to be  
  
response = llm.invoke(prompt_input)  
print(response.content)
```

This will return the following output similar to:

```
"Stay alert and protect your information - beware of phishing attacks."
```

2.3 Prompt templates

When building LLM applications, creating flexible prompt templates that incorporate user input through parameters is crucial. LangChain makes this easier with its `PromptTemplate` class, which lets you manage and reuse parametrized prompts without the need for custom functions. This not only streamlines the creation of dynamic prompts but also enhances the efficiency and adaptability of your LLM interactions, as I'll demonstrate shortly.

2.3.1 Implementing a prompt template with a Python function

To illustrate the concept of a template, let's create a text summarization template that requests the text, desired summary length, and preferred tone. You could implement this using a simple Python function:

```
def generate_text_summary_prompt(text, num_words, tone):  
    return f"You are an experienced copywriter. Write a {num_word
```

Let's use the prompt template to generate a prompt and then execute it through LangChain's `ChatOpenAI` wrapper as usual:

```
segovia_aqueduct_text = "The Aqueduct of Segovia (Spanish: Acuedu
```

```
input_prompt = generate_text_summary_prompt(text=segovia_aqueduct_text)
response = llm.invoke(input_prompt)
print(response.content)
```

You should be output similar to:

The Aqueduct of Segovia, built in the 1st century AD, is a well-preserved Roman structure that has survived for over 1,900 years.

2.3.2 Using LangChain's PromptTemplate

With LangChain, you don't need to implement a prompt template function manually. Instead, you can use the convenient `PromptTemplate` class to handle parametrized templates, as I showed in chapter 1. Here's how you can use it:

```
from langchain_core.prompts import PromptTemplate

prompt_template = PromptTemplate.from_template("You are an expert in the history of the city of Segovia. Please provide a brief summary of the {text} in {num_words} words.")
```

To use the prompt template, create a prompt instance and format it with your parameters:

```
prompt = prompt_template.format(text=segovia_aqueduct_text, num_words=50)
```

Then, invoke the `ChatOpenAI` client with the formatted prompt:

```
response = llm.invoke(prompt)
print(response.content)
```

This will generate output similar to what's shown here (as you now understand, the exact output may differ from what's printed in the book, so I won't repeat this point on the next page):

The Aqueduct of Segovia, a Roman marvel built in the 1st century AD, is a well-preserved structure that has survived for over 1,900 years.

Let's take a short break from coding with LangChain. I want to delve deeper into prompt engineering. In the upcoming pages, you'll see that a good prompt can have various elements, depending on your goal, task complexity, and desired accuracy. By learning from different prompt variations, you'll be

able to handle both simple and complex prompts, adjusting their complexity to fit specific cases.

2.4 Types of prompt

Creating effective prompts is crucial for getting the best results in LangChain applications. Whether your app is focused on text classification, sentiment analysis, summarization, text generation, or question answering, each task requires a carefully designed prompt. In the following sections, we'll dive into how to craft prompts tailored to these specific tasks, ensuring your application functions as intended.

Although these prompting techniques are key to developing LangChain applications, we'll use ChatGPT for now to keep things simple and explain prompt engineering concepts more clearly. The patterns you learn here will directly apply to the LLM apps we build with LangChain in the upcoming chapters.

2.4.1 Text classification

In classification, the aim is to place an input text into one of several predefined categories, as demonstrated in the following prompt:

Instruction: Classify the following text into one of these categories: history, tech, gardening.

Text: Headphones provide immersive audio experiences for music lovers and gamers alike.

Output:

The text "Headphones provide immersive audio experiences for music lovers and gamers alike." should be classified as "tech" because it discusses technology-related products and their functionality.

The output is overly detailed. Let's revise the prompt and specify to only display the category.

Instruction: Classify the following text into one of these categories: history, tech, gardening.

Text: Headphones provide immersive audio experiences for music lovers and gamers alike.

Output: only the category

Output:

Tech

Now, categorize another sentence by applying the same prompt and only altering the text:

Instruction: Classify the following text into one of these categories: history, tech, gardening.

Text: Julius Caesar was a Roman general and statesman who played a pivotal role in the demise of the Roman Republic and the rise of the Roman Empire.

Output: only the category

Output:

History

In summary, in a standard text classification prompt, you find three components: an Instruction, the input Text, and an Output specification.

Now, let's delve into a slightly specialized text classification: sentiment analysis.

2.4.2 Sentiment analysis

Sentiment analysis is a specific type of text classification that aims to determine whether a given text is perceived as positive, neutral, or negative. Below are three sample prompts you can experiment with independently.

First prompt example:

Instruction: Classify the following text as positive, neutral or negative

Text: Apple have announced stellar quarterly results

Expected output:

Positive

Second prompt example:

Instruction: Classify the following text as positive, neutral or negative

Text: this is the worst movie I have watched this month

Expected output:

Negative

Third prompt example:

Instruction: Classify the following text as positive, neutral or negative

Text: The customer bought a green dress

Expected output:

Neutral

If you have a list of texts and wish to analyze their sentiments collectively, you can input them all into a single prompt:

Instruction: classify the sentiment of following stock reports as positive, neutral or negative

Stock 1: Apple: the launch of the new iPhone has been a success

Stock 2: Nvidia: sales are being propelled by consumer demand on LLMs

Stock 3: GX oil: demand of carbon based energy is dropping thanks to the increase of renewable energy

Output: output a table with the columns "stock name", "sentiment"

Expected output:

```
| Stock Name | Sentiment |  
|-----|-----|  
| Apple | positive |  
| Nvidia | positive |  
| GX oil | negative |
```

As you observe, in this scenario, the output is formatted in a table and can be readily processed programmatically.

Now, let's create a prompt for one of the most common LLM use cases: text summarization.

2.4.3 Text summarization

Creating a text summarization prompt is simple: you only need to ask for a

summary of an input text with your preferred length.

Instruction: write a 30 word summary for the following text

Text: Home PCs from the 1980s were iconic pioneers of personal computing. Computers like the Apple II, IBM PC, and Commodore 64 revolutionized how we lived and worked. They introduced us to word processing, gaming, and the possibilities of home computing. The graphical user interface, as seen in the Apple Macintosh, changed the way we interacted with machines. These PCs, though less powerful by today's standards, ignited a digital revolution, shaping the technology landscape we know today. They hold a special place in history, symbolizing the dawn of a new era in computing that laid the foundation for the modern digital age.

Output:

1980s home PCs like Apple II, IBM PC, Commodore 64 pioneered personal computing, introducing word processing, gaming, and GUIs. They shaped today's tech, symbolizing a digital era's dawn.

You can also ask the LLM to provide a summary for a text it's already familiar with:

Instruction: Write a one sentence summary of the Lord of the Rings

Output:

"The Lord of the Rings" follows the epic journey of a diverse group of characters as they seek to destroy a powerful ring and save Middle-earth from the dark lord Sauron.

2.4.4 Composing text

LLMs are part of the AI category called "generative AI." While LLMs can

handle audio and video, with text they generate new content, like a paragraph or sentence, based on a list of facts, for example:

Write a piece on the diver watches, mentioning the following facts:

- the Rolex Oyster Perpetual was among the first to be water resistant
- Omega produced the Seamaster in 1948 as the first purpose designed diver watch was the Seamaster
- the Seamaster set a world record of immersion of 62.5m in 1955.
- Rolex produced the first Submariner in 1953
- the Submariner became popular also because of the association with James Bond until 1995
- James Bond started to wear the Seamaster in 1995

Output:

Diver Watches: A Journey into Time's Abyss

Diver watches have a storied history, encapsulating the spirit of adventure and resilience against the unforgiving depths of the oceans. These timepieces not only serve as reliable companions for professional divers but have also become iconic symbols of style and innovation. Here, we delve into the fascinating world of diver watches, highlighting key milestones and legendary timepieces that have defined this genre.

The Rolex Oyster Perpetual, introduced in the early 20th century, played a pioneering role in the evolution of diver watches. Among its notable features was its remarkable water-resistant design, which set the stage for the development of truly waterproof timepieces. This innovation marked a crucial step in ensuring that watches could withstand the rigors of underwater exploration.

Omega, a respected Swiss watchmaker, made its mark on the diver watch scene in 1948 with the introduction of the Seamaster. This timepiece was a game-changer as it was purposefully designed for divers, combining style and functionality. It was a clear indication that dive watches were evolving

beyond mere timekeeping instruments into specialized tools for underwater adventurers.

The Seamaster didn't stop at design; it also showcased its capabilities in the depths. In 1955, it set a remarkable world record for immersion, descending to an astonishing 62.5 meters underwater. This achievement solidified the Seamaster's reputation as a reliable and robust diver's companion.

Not to be outdone, Rolex joined the fray by producing the first Submariner in 1953. This iconic watch quickly became synonymous with underwater exploration and adventure. Its success was further boosted by its association with the fictional British spy, James Bond. The Submariner's appearance on the wrist of the suave secret agent added a layer of mystique and sophistication to this already legendary timepiece. It's a partnership that endured until 1995, firmly establishing the Submariner as a symbol of timeless style and rugged functionality.

However, the tides of popularity shifted in 1995 when James Bond, portrayed by Pierce Brosnan, made a bold move by strapping on an Omega Seamaster in "GoldenEye." This marked the beginning of a new era for the Seamaster, as it became the watch of choice for the world's most famous secret agent. The Seamaster seamlessly blended style, innovation, and precision, making it a perfect match for 007's daring missions.

In conclusion, diver watches have come a long way since their inception. The Rolex Oyster Perpetual and Omega Seamaster stand as pioneers, shaping the path for future innovations. These watches not only broke records but also captured the imaginations of generations. The Rolex Submariner and the Omega Seamaster, each with its unique charm, continue to be revered in the world of horology, leaving an indelible mark on the wrists of adventurers and style enthusiasts alike.

You may notice that ChatGPT has used the limited information to create a quite lengthy article. On the other hand, you might have expected a shorter piece and may not be a fan of the complex vocabulary or somewhat formal tone it used. In such a situation, try restructuring the prompt more systematically to get the generated text to match your preferred length, vocabulary, and tone:

Persona: you are a watch enthusiast, experienced in writing high quality articles for watch magazines for a general audience

Instruction: Write a piece on the diver watches

Facts to mention:

- the Rolex Oyster Perpetual was among the first to be water resistant
- Omega produced the Seamaster in 1948 as the first purpose designed diver watch was the Seamaster
- the Seamaster set a world record of immersion of 62.5m in 1955.
- Rolex produced the first Submariner in 1953
- the Submariner became popular also because of the association with James Bond until 1995
- James Bond started to wear the Seamaster in 1995

Tone: use a sober, knowledgeable but not too formal tone and use everyday vocabulary

Output: the length of the article should be 150 words at most

Output:

****Diver Watches: Timepieces of Exploration and Style****

Diver watches have long been cherished for their blend of functionality and aesthetics, a perfect companion for both the adventurous spirit and the style-conscious.

Rolex deserves recognition for pioneering water resistance with the Oyster Perpetual, setting the stage for a new era in watchmaking.

Omega, however, made a significant splash in 1948 with the Seamaster, the world's first purpose-designed diver watch. Its 1955 record dive of 62.5 meters underlines its prowess in the deep.

Rolex didn't lag behind, introducing the Submariner in 1953, a watch that

became synonymous with exploration, partly due to its affiliation with James Bond on-screen.

Speaking of James Bond, it's worth noting that in 1995, the iconic secret agent started sporting the Omega Seamaster, a move that reinforced the Seamaster's reputation as a sophisticated yet rugged timepiece for those who dare to explore both land and sea. These diver watches are more than instruments; they're symbols of adventure and style.

By adding more details to the prompt, the article has been generated exactly as you wanted it.

note

Creating a well-structured prompt with defined sections (Persona, Instruction, Facts, Tone, Output Format) helps the LLM understand your requirements better and generate the expected text.

2.4.5 Question answering

The ability of LLMs to "understand" text makes them suitable not only for summarizing but also for retrieving specific information, known as the "Question answering" use case.

Now, let's go back to the same text we used in the summarization example and ask focused questions about its content.

Instruction: read the following text, and tell me which home computers were popular in the 80s.

Text: Home PCs from the 1980s were iconic pioneers of personal computing. Computers like the Apple II, IBM PC, and Commodore 64 revolutionized how we lived and worked. They introduced us to word processing, gaming, and the possibilities of home computing. The graphical user interface, as seen in the Apple Macintosh, changed the way we interacted with machines. These PCs, though less powerful by today's standards, ignited a digital revolution, shaping the technology landscape we know today. They hold a special place in history, symbolizing the dawn of a new era in computing that laid the

foundation for the modern digital age.

Output: just output the names of the computers in a list

Output:

The popular home computers in the 1980s were:

1. Apple II
2. IBM PC
3. Commodore 64
4. Apple Macintosh

Question answering is such a common use case most LLMs understand Q and A as shorthand for Question and Answer. For example:

Text: Java is a popular programming language that compiles code into bytecode, which is executed by the Java Virtual Machine (JVM) for platform-independent application development.

Q: Where is Java code executed?

A:

Output:

Java code is executed by the Java Virtual Machine (JVM).

LLMs are skilled not just at finding requested information in a text but also at making logical deductions. This will become apparent in the next section.

2.4.6 Reasoning

While reasoning is a powerful capability, it's an area where LLMs aren't exceptional yet. Therefore, you might need more advanced prompt engineering techniques to get the results you want.

Here's a simple example involving mathematical reasoning:

Instruction: Add the square numbers in this sequence: 19, 13, 1, 17, 4, 64, 900

Output:

The square numbers in the sequence 19, 13, 1, 17, 4, 64, and 900 are 1, 4, 64, and 900. To find their sum:

$$1 + 4 + 64 + 900 = 969$$

So, the sum of the square numbers in this sequence is 969.

ChatGPT identified the square numbers, picked them out, and then figured out their total. The next example is more challenging, involving palindromes—numbers that read the same forward and backward, such as 1661 or 27872:

Instruction:

Sum the palindromes in this sequence: 13, 1331, 121, 73, 99, 56, 232, 7

I got this output:

The palindromes in the sequence 13, 1331, 121, 73, 99, 56, 232, and 7 are 1331, 121, 232, and 7. To find their sum:

$$1331 + 121 + 232 + 7 = 1691$$

So, the sum of the palindromes in this sequence is 1691.

This is incorrect! A palindrome is a number that reads the same forward and backward. Therefore, 99 should have been considered a palindrome, but it wasn't.

I'll be frank with you: the output above is what I encountered in the early stages of writing this book. However, you should now see a different and, more importantly, accurate output. It's still valuable to explain how I corrected ChatGPT's mistake before OpenAI addressed this in a later version of the model.

So, how did I fix it? Since the LLM didn't fully grasp the concept of a palindrome, I "taught" ChatGPT to correctly identify all relevant cases, as I'll explain in the next section. Before moving on, take a moment to review the types of prompts you've learned. I've summarized them in Table 2.1 below for your reference.

Table 2.1 Prompt types and examples

Prompt type	Prompt example
Text classification	Classify the following text into one of these categories: history, tech, gardening.
Sentiment analysis	Classify the following text as positive, neutral or negative
Text summarization	Write a 30 word summary for the following text
Composing Text	Write a piece on the diver watches, mentioning the following facts: - the Rolex Oyster Perpetual was among the first to be water resistant - Omega produced the Seamaster in 1948 as the first purpose designed diver watch was the Seamaster ...
Question answering	Read the following text, and tell me which home computers were popular in the 80s.
Reasoning	Add the square numbers in this sequence: 19, 13, 1, 17, 4, 64, 900

2.5 Reasoning in detail

When developing LLM applications, particularly those requiring complex reasoning, it's important to understand the limitations of different prompt techniques. For instance, "*zero-shot learning*" is a method where you provide no examples, expecting the LLM to grasp and perform the task based purely on the instructions given. While this approach can work well in some scenarios, it doesn't always yield accurate results for more intricate tasks, such as teaching an LLM to identify palindromes. In the following sections, I'll demonstrate how to go beyond zero-shot learning to refine LLM prompts for more reliable performance in reasoning tasks.

2.5.1 One-shot learning

When you provide just one example in the prompt, it's called "*one-shot learning*." Here's how I attempted to address the palindrome problem by giving a single example:

Instruction: Sum the palindromes in this sequence: 13, 1331, 121, 73, 99, 56, 232, 7

Example: 33 is a palindrome

I got this output:

I apologize for the previous oversight. Let's correctly identify and sum the palindromes in this sequence: 13, 1331, 121, 73, 99, 56, 232, and 7.

The palindromes in the sequence are 1331, 121, 232, and 7. To find their sum:

$$1331 + 121 + 232 + 7 = 1691$$

So, the sum of the palindromes in this sequence is indeed 1691.

Unfortunately, even after using one-shot learning, the issue persisted. So, I

made adjustments to the prompt and included two examples.

2.5.2 Two-shot learning

Here is the amended prompt I used:

Instruction: Sum the palindromes in this sequence: 13, 1331, 121, 73, 99, 56, 232, 7

Examples:

33 is a palindrome

44 is a palindrome

This is the output I got:

Certainly, let's identify and sum the palindromes in the sequence: 13, 1331, 121, 73, 99, 56, 232, and 7.

The palindromes in this sequence are 1331, 121, 232, and 7. To find their sum:

$$1331 + 121 + 232 + 7 = 1691$$

So, the sum of the palindromes in this sequence is 1691.

Still incorrect: 99 got overlooked once again.

Following another unsuccessful attempt, I explored different approaches and found a standard solution. It suggested breaking down the problem into "steps" and providing clear instructions for each step. This is similar to guiding a colleague who may be struggling with a specific step in a procedure.

2.5.3 Providing steps

This is the amended prompt I entered:

Instruction: Sum the palindromes in this sequence: 13, 1331, 121, 73, 99, 56
232, 7

Examples:

33 is a palindrome

44 is a palindrome

Steps: 1) identify the palindromes; 2) add them up

I got this output:

Certainly, let's follow your instructions step by step:

Step 1: Identify the palindromes in the sequence.

Palindromes in the sequence:

- 1331
- 121
- 99
- 232
- 7

Step 2: Add them up.

$$1331 + 121 + 99 + 232 + 7 = 1790$$

So, the sum of the palindromes in the sequence is 1790.

Finally, I nailed it!

Interestingly, it appears that ChatGPT has undergone a similar training process (on a much larger scale, of course) to fix this issue. It's likely that other users encountered the same problem.

If you enter the original prompt:

Instruction:

Sum the palindromes in this sequence: 13, 1331, 121, 73, 99, 56, 232, 7

You'll now see the output below.

Let's identify the palindromes in the given sequence:

1331 is a palindrome

121 is a palindrome

99 is a palindrome

232 is a palindrome

7 is a palindrome

Now, let's add these palindromes together:

$$1331 + 121 + 99 + 232 + 7 = 1790$$

Therefore, the sum of the palindromes in the sequence is 1790.

It appears ChatGPT is addressing the problem step by step, much like how I resolved the issue.

I found it interesting to share an example that's no longer relevant. This is something you might experience while working with LLMs: you could invest time in teaching the LLM certain tasks using advanced prompt engineering techniques, only to find that a newer model version can handle the task without any additional training. For instance, OpenAI's o1 model performed reasoning tasks with minimal prompts, which previously required more complex prompts like the few-shot techniques you're about to learn.

2.5.4 Few-shot learning

In some cases, providing only two examples may not be enough to teach the LLM a new concept. In such instances, you need to provide several examples, known as "few-shot learning." To demonstrate this, let me create a game where a number can be classified as Abra, Kadabra, or Abra Kadabra. Let's see what happens when no information about the classification rules is provided.

Classify the following numbers as Abra, Kadabra or Abra Kadabra:

3, 5, 8, 2, 7, 12, 13

Output:

The classification of numbers into "Abra," "Kadabra," or "Abra Kadabra" doesn't follow a conventional mathematical pattern. These are just words, not related to numbers. Therefore, it's not possible to classify the numbers you provided as "Abra," "Kadabra," or "Abra Kadabra" based on any mathematical or logical criteria.

ChatGPT hasn't experienced any hallucinations but rightfully pointed out that you haven't clarified the classification rules.

I'll show you a prompt that, instead of directly explaining the rules, offers an interpretation based on a specific list of numbers, different from the ones I originally asked to be addressed:

Classify the following numbers as Abra, Kadabra or Abra Kadabra:

3, 4, 5, 7, 8, 10, 11, 13, 35

Examples:

6 // not divisible by 5, not divisible by 7 // None

15 // divisible by 5, not divisible by 7 // Abra

12 // not divisible by 5, not divisible by 7 // None

21 // not divisible by 5, divisible by 7 // Kadabra

70 // divisible by 5, divisible by 7 // Abra Kadabra

Observe the format I've employed for every example: I've delineated each field with a double forward slash (/), although I haven't explicitly outlined the meaning of each field. Trusting the LLM to decipher it.

Here is the output:

Let's classify the numbers as Abra, Kadabra, or Abra Kadabra based on whether they are divisible by 5 and/or 7:

3: Not divisible by 5, not divisible by 7 - None

4: Not divisible by 5, not divisible by 7 - None

5: Divisible by 5, not divisible by 7 - Abra

7: Not divisible by 5, divisible by 7 - Kadabra

8: Not divisible by 5, not divisible by 7 - None

10: Divisible by 5, not divisible by 7 - Abra

11: Not divisible by 5, not divisible by 7 - None

13: Not divisible by 5, not divisible by 7 - None

35: Divisible by 5, divisible by 7 - Abra Kadabra

So, here's the classification:

Abra: 5, 10, 35

Kadabra: 7

Abra Kadabra: 35

None: 3, 4, 8, 11, 13

ChatGPT not only provided accurate results but also explained the reasoning behind them. It successfully deduced the general rules from the given examples and articulated them well. Quite impressive, isn't it?

Note

You may be familiar with a similar "number classification game" or "Divisibility Test Algorithm" (<https://stackoverflow.com/questions/20486231/divisibility-test-algorithm>) where a number divisible by 3 is labeled "Foo," a number divisible by 5 is labeled "Bar," and a number divisible by both 3 and 5 is labeled "Foo Bar." I haven't used this as an example because ChatGPT already knows it and can correctly classify numbers without examples, using "zero-shot" learning.

2.5.5 Implementing few-shot learning with LangChain

Let's implement the AbraKadabra game with LangChain. First, establish a connection to the LLM, as you did previously:

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(openai_api_key=OPENAI_API_KEY,
                  model_name="gpt-4o-mini")
```

Now, define and run the same few-shot prompt you executed previously directly against ChatGPT:

```
prompt_input = """Classify the following numbers as Abra, Kadabra
3, 4, 5, 7, 8, 10, 11, 13, 35
```

Examples:

```

6 // not divisible by 5, not divisible by 7 // None
15 // divisible by 5, not divisible by 7 // Abra
12 // not divisible by 5, not divisible by 7 // None
21 // not divisible by 5, divisible by 7 // Kadabra
70 // divisible by 5, divisible by 7 // Abra Kadabra
"""

```

```

response = llm.invoke(prompt_input)
print(response.content)

```

The output, is, as expected:

```

3 // not divisible by 5, not divisible by 7 // None
4 // not divisible by 5, not divisible by 7 // None
5 // divisible by 5, not divisible by 7 // Abra
7 // not divisible by 5, divisible by 7 // Kadabra
8 // not divisible by 5, not divisible by 7 // None
10 // divisible by 5, not divisible by 7 // Abra
11 // not divisible by 5, not divisible by 7 // None
13 // not divisible by 5, not divisible by 7 // None
35 // divisible by 5, divisible by 7 // Abra Kadabra

```

The output is accurate, but the implementation isn't ideal because the examples are hardcoded in the prompt. LangChain provides a cleaner solution for creating a few-shot prompt. It lets you separate the training examples from the prompt template and inject them later, as shown in listing 2.1.

Listing 2.1 Few-shot prompt using FewShotPromptTemplate

```

from langchain_core.prompts.few_shot import FewShotPromptTemplate
from langchain_core.prompts.prompt import PromptTemplate

examples = [
    {
        "number": 6,
        "reasoning": "not divisible by 5 nor by 7",
        "result": "None"
    },
    {
        "number": 15,
        "reasoning": "divisible by 5 but not by 7",
        "result": "Abra"
    },
    {

```

```

        "number": 12,
        "reasoning": "not divisible by 5 nor by 7",
        "result": "None"
    },
    {
        "number": 21,
        "reasoning": "divisible by 7 but not by 5",
        "result": "Kadabra"
    },
    {
        "number": 70,
        "reasoning": "divisible by 5 and by 7",
        "result": "Abra Kadabra"
    }
]

```

```

example_prompt = PromptTemplate(input_variables=["number", "reasoning"],
                                output_variable="result")
few_shot_prompt = FewShotPromptTemplate(
    examples=examples,
    example_prompt=example_prompt,
    suffix="Classify the following numbers as Abra, Kadabra or Ab",
    input_variables=["comma_delimited_input_numbers"]
)

```

```

prompt_input = few_shot_prompt.format(comma_delimited_input_numbers="3 \ not divisible by 5 nor by 7 \ None\n4 \ not divisible by 5 nor by 7 \ None\n5 \ divisible by 5 but not by 7 \ Abra\n7 \ divisible by 7 but not by 5 \ Kadabra\n8 \ not divisible by 5 nor by 7 \ None\n10 \ divisible by 5 but not by 7 \ Abra\n11 \ not divisible by 5 nor by 7 \ None\n13 \ not divisible by 5 nor by 7 \ None\n35 \ divisible by 5 and by 7 \ Abra Kadabra")
response = llm.invoke(prompt_input)
print(response.content)

```

And the result is still correct:

```

3 \ not divisible by 5 nor by 7 \ None
4 \ not divisible by 5 nor by 7 \ None
5 \ divisible by 5 but not by 7 \ Abra
7 \ divisible by 7 but not by 5 \ Kadabra
8 \ not divisible by 5 nor by 7 \ None
10 \ divisible by 5 but not by 7 \ Abra
11 \ not divisible by 5 nor by 7 \ None
13 \ not divisible by 5 nor by 7 \ None
35 \ divisible by 5 and by 7 \ Abra Kadabra

```

Before concluding this section, I want to introduce you to a technique known as "chain of thought," which blends "providing logical steps" with "few-shot learning."

2.5.6 Chain of Thought (CoT)

I'm making up a new mathematical definition: I'll name a sequence of numbers "Strange" if it contains at least two odd numbers, and the sum of all odd numbers is divisible by 3.

Initially, I check whether the LLM is unfamiliar with the rules of "Strange" sequences:

Is the following sequence "Strange"?

1, 3, 4, 5, 7, 10, 15, 18, 22, 24

Output:

The sequence you provided appears to be increasing, but it doesn't follow a common mathematical pattern like arithmetic or geometric progressions. Whether it's considered "strange" depends on the context or criteria you're using to define "strange." In a mathematical sense, it might be considered an unusual or non-standard sequence. If you have specific criteria or rules that define what a "strange" sequence is, please provide more details for a more precise classification.

As expected, ChatGPT doesn't seem to understand the concept of a "Strange sequence," reaffirming that, luckily, it hasn't developed psychic abilities... yet.

I will now create a prompt using the Chain of Thought technique, as introduced by Wei et al (<https://arxiv.org/abs/2201.11903>). This prompt, an extension of a few-shot learning prompt, will include:

1. Several sample sequences for few-shot learning
2. For each sample sequence, an explanation of the logical steps to determine if a sequence is Strange

Q: Is the following sequence "Strange"?

3, 4, 5, 7, 10, 18, 22, 24

A:

Examples:

Q: is the following sequence strange: 1, 4, 6, 8, 20

A: 1 is an odd number; I need at least two odd numbers // Not Strange

Q: is the following sequence strange: 5, 6, 7, 8, 20

A: 5 and 7 are odd numbers; the sum of 5 and 7 is 12; 12 is divisible by 3 // Strange

Q: is the following sequence strange: 1, 5, 6, 7, 8, 20

A: 1, 5 and 7 are odd numbers; the sum of 1, 5 and 7 is 13; 13 is not divisible by 3 // Not Strange

Q: is the following sequence strange: 5, 6, 7, 8, 9, 20

A: 5, 7, 9 are odd numbers; the sum of 5, 7 and 9 is 21; 21 is divisible by 3 // Strange

Output:

Thank you for providing the examples and clarifications. Based on the criteria you've presented:

The sequence "3, 4, 5, 7, 10, 18, 22, 24" has 3 odd numbers: 3, 5, and 7. The sum of these odd numbers is 15. 15 is divisible by 3. So, according to the criteria, this sequence is ****Strange****.

ChatGPT gave the right answer and explained its thought process. To reinforce your understanding, try using a similar prompt with a different set of numbers. Also, observe how it responds when the sequence contains just one odd number.

In essence, in this section you've discovered how to introduce ChatGPT to a new task by offering various examples or outlining the necessary steps. This approach is known as "in-context" learning, as you're instructing the LLM using information provided within the context of a prompt.

Definition

“In-context learning” refers to any method that instructs an LLM on an unfamiliar task or instruction requested by the user, utilizing examples within the prompt context. Common techniques include one-shot, two-shot, or few-shot learning, as well as providing step-by-step guidance, often referred to as a "chain of thought." Compared to fine-tuning, in-context learning is a more cost-effective and less resource-intensive approach, as it doesn't demand high-end hardware resources like GPUs or in-depth knowledge of transformer architectures.

This section covered a range of in-context learning prompts, which I have summarized, for convenience, in table 2.2.

Table 2.2 In-context learning prompts

In-Context learning technique	Explanation
Zero-shot learning	No example is provided in the prompt
One-shot learning	One example is provided in the prompt
Two-shot learning	Two examples are provided in the prompt
Few-shot learning	A number of examples are provided in the prompt
Chain of Thought	A number of examples are provided in the prompt, and for each example, all the logical steps to achieve an objective are clearly explained

If you want to explore advanced techniques beyond Chain of Thought, check the related sidebar. It includes detailed pointers on Tree of Thought and Thread of Thought, two powerful methods that enhance reasoning and problem-solving in language models.

Beyond Chain of Thought

Chain of Thought improves how language models handle complex reasoning by breaking problems into smaller, logical steps. However, it has limitations, such as missing deeper exploration or struggling with messy contexts. Two advanced techniques address these gaps: Tree of Thought (ToT) and Thread of Thought (ThoT):

- **Tree of Thought (ToT):** ToT improves problem-solving by allowing language models to explore multiple reasoning paths. Traditional models make token-by-token decisions, which limits their ability to plan ahead or revisit earlier choices. ToT structures the process into coherent steps, enabling models to evaluate and refine options as they go. This method (<https://arxiv.org/abs/2305.10601>) is highly effective in tasks requiring strategic thinking, such as the Game of 24, creative writing, and mini crosswords. For example, ToT helped GPT-4 solve 74% of Game of 24 problems, compared to just 4% using Chain of Thought. This approach improves deliberate decision-making, backtracking, and strategic foresight.
- **Thread of Thought (ThoT):** ThoT addresses challenges with chaotic contexts, where irrelevant details distract models or lead to errors. Inspired by human cognition, ThoT (<https://arxiv.org/abs/2311.08734>) systematically breaks down and analyzes large chunks of information while focusing on relevant details. It works as a modular add-on that integrates with various models and prompts. Experiments on datasets like PopQA, EntityQ, and a Multi-Turn Conversation Response dataset show ThoT significantly enhances reasoning accuracy. It excels in filtering noise and maintaining focus in complex contexts.

Both techniques push language models beyond simple response generation, improving reasoning, planning, and decision-making in structured and chaotic scenarios. For technical details and examples, refer to the published research and code repositories.

Before closing this chapter, let's try to extract a standard prompt format from our examination of various use cases up to this point.

2.6 Prompt structure

Combining all the prompt elements from earlier sections results in the following generalized structure:

- **Persona:** Specify the role you want the language model (LLM) to embody. Whether a skilled marketing manager, an award-winning sci-fi author, or a proficient copy editor for a computing magazine, set the expectations high.
- **Context:** Provide detailed background information to help the LLM comprehend the context surrounding your request. Precision enhances the likelihood of aligning with your expectations.
- **Instruction:** Clearly define the action you want the LLM to take with your input text. Whether it's translation, writing, rewriting, improving, classifying, ordering, or finding, be explicit.
- **Input:** This refers to specific context details, which could take the form of a paragraph, a query, or a list of key points. You may choose to label it accordingly, such as "Text" or "Question.", etc
- **Steps:** Outline the processing steps the LLM should follow to generate the output.
- **Tone:** Specify the desired tone of the LLM's answer—formal, informal, witty, enthusiastic, sober, friendly, etc. Combinations are possible.
- **Output Format:** Optionally specify the desired output format. If not specified, the LLM assumes a text answer. However, you can request formats like a table, CSV file (with column names), JSON (with attribute names), or other structured formats.
- **Examples:** In cases where the LLM may lack sufficient training for certain tasks, especially in text classification and reasoning, provide mini-inline training through examples. Even a few examples significantly enhance answer quality.

This list acts as a guide or checklist for organizing prompts. Customize it based on your requirements, choosing relevant sections or making modifications. Here's a prompt that includes many of these elements, structured with clear sections:

Persona: You are an experienced Large Language Model (LLM) developer

and renowned speaker.

Context: You have been invited to give a keynote speech for a LLM event.

Instruction: Write the punch lines for the speech.

Input: Include the following facts:

- LLMs have become mainstream with the launch of ChatGPT in November 2022
- many popular LLMs and LLM based chatbots have been launched since then, such as LLAMA-2, Falcon180B, Bard.
- LLMs becoming as popular as search engines
- many companies want to integrate LLMs in their applications

Tone: Use a witty but entertaining tone.

Output format: Present the text in two paragraphs of 5 lines each.

Output:

Ladies and gentlemen, as we embark on this linguistic odyssey, remember this: In November 2022, ChatGPT didn't just enter the scene; it was the linguistic equivalent of a rockstar's encore, setting the stage for an epic word-tastic journey! Today, with LLAMA-2, Falcon180B, and Bard, it's safe to say that LLMs have become the Beatles of the digital era, making search engines feel like yesterday's news.

But here's the kicker – companies are now falling over themselves, desperately trying to get these LLMs into their applications. It's like the gold rush, but instead of panning for precious metal, we're sifting through bytes of brilliant language. So, my friends, in this age where words wield the power, and LLMs are the mighty pens, we're not just scripting the future; we're penning a linguistic saga that's bound to be a best-seller! Welcome to the age of LLMs, where we're rewriting the rules, one sentence at a time, and words are the currency of change!

NOTE

Studies (for example “The Prompt Report: A Systematic Survey of Prompting Techniques”, <https://arxiv.org/abs/2406.06608>) have found that explicitly naming different parts of a prompt tends to improve results. However, you don't have to name every section. Most LLMs can figure out the purpose of the text in the prompt on their own. So, you can mix things up by naming some parts (like Question or Examples) and leaving out names for others (like Context or Tone). Experiment and see what works best for your case and the results you're getting.

If you want to delve deeper into prompt engineering, I highly recommend the following resources:

- <https://www.promptingguide.ai/techniques>
- <https://github.com/dair-ai/Prompt-Engineering-Guide>
- <https://github.com/prompts-lab/Awesome-Prompt-Engineering>

2.7 Summary

- A prompt is a specific request that guides the LLM, giving it instructions based on provided background information.
- Different types of prompts are tailored for specific tasks, such as text classification, sentiment analysis, text summarization, text generation, or question and answering.
- If a prompt doesn't produce the expected result, it means the underlying LLM isn't familiar with the requested task and needs training.
- Training involves enhancing the prompt with examples, categorized as one-shot, two-shot, or few-shot learning based on the number of examples given.
- As you create various prompts, you might notice a common prompt structure with sections like persona, context, instruction, input, steps, examples, etc.
- You should adapt the prompt structure to your use case by choosing relevant sections or adding custom ones. You can also drop some explicit section names
- While users often create and execute prompts directly through a chat

LLM UI like ChatGPT, software developers typically automate prompt generation programmatically using directly the LLM API or LangChain.

- LangChain supports prompt engineering with a range of classes, from `PromptTemplate` to the more advanced `FewShotPromptTemplate`
- In LLM application development, it's crucial to design parameterized prompts, also known as prompt templates.

3 Summarizing text using LangChain

This chapter covers

- Summarization of large documents exceeding the LLM's context window
- Summarization across multiple documents
- Summarization of structured data

In Chapter 1, you explored three major LLM application types: summarization engines, chatbots, and autonomous agents. In this chapter, you'll begin building practical summarization chains using LangChain, with a particular focus on the LangChain Expression Language (LCEL) to handle various real-world scenarios. A chain is a sequence of connected operations where the output of one step becomes the input for the next—ideal for automating tasks like summarization. This work lays the foundation for constructing a more advanced summarization engine in the next chapter.

Summarization engines are essential for automating the summarization of large document volumes, a task that would be impractical and costly to handle manually, even with tools like ChatGPT. Starting with a summarization engine is a practical entry point for developing LLM applications, providing a solid base for more complex projects and showcasing LangChain's capabilities, which we'll further explore in later chapters.

Before we start building, we'll look at different summarization techniques, each suited to specific scenarios like large documents, content consolidation, and handling structured data. Since you've already worked with summarizing small documents using a `PromptTemplate` in section 1.3.2, we'll skip that and focus on more complex examples.

3.1 Summarizing a document bigger than context window

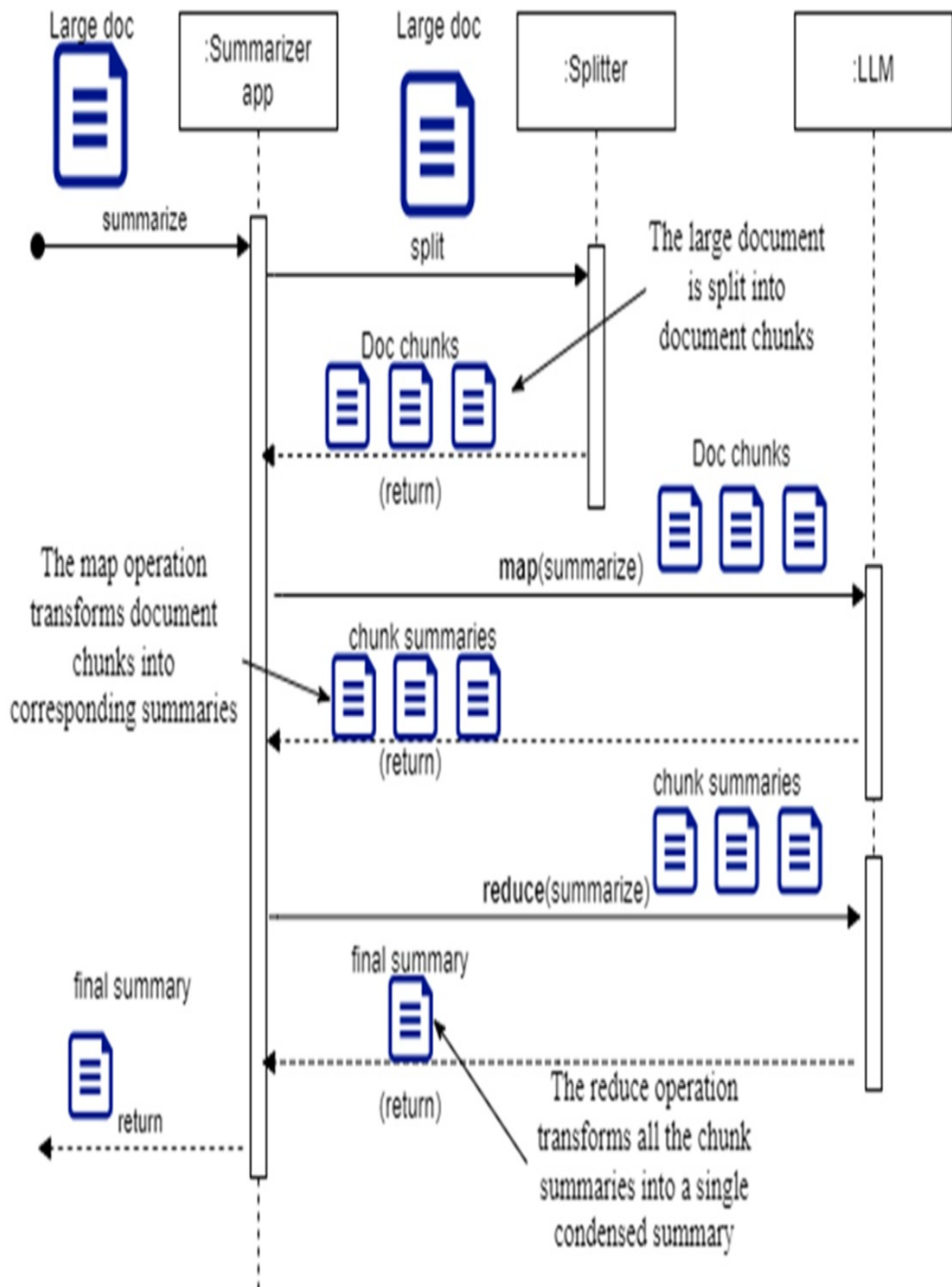
As mentioned in chapter 2, each LLM has a maximum prompt size, also referred to as the "context window."

DEFINITION

The "LLM context window" represents the maximum size of the prompt provided to an LLM, comprising instructions and context. Different LLMs have varying token limits for the context window: GPT-3.5 used to accept up to 16K tokens, GPT-4 and Claude-3 up to 100K, and Gemini-1.5 up to 1M.

As the context window for popular LLMs continues to grow, you may still encounter situations where a document exceeds the token limit of your chosen model. In these cases, you can use a map-reduce approach, as illustrated in Figure 3.1.

Figure 3.1 Summarizing a document bigger than the LLM's context window: this involves splitting the document into smaller chunks, summarizing each chunk, and then summarizing the combined chunk summaries.



This workflow involves splitting the document into smaller segments, summarizing each one, and then summarizing the combined summaries.

To start, you need to split the text into chunks using a tokenizer. A tokenizer reads the text and breaks it into tokens, which are the smallest units of text, often parts of words. After tokenizing the document, the tokens are grouped into chunks of a specific size. This lets you control the content size being processed by the LLM and ensures the token count stays within your LLM's prompt limit. I'll show you how to use `TokenTextSplitter`, part of the `tiktoken` package, a tokenizer developed by OpenAI.

Begin by opening a terminal and creating a new folder named `ch03` for this chapter's code. Then, create and activate a virtual environment:

```
C:\Github\building-llm-applications>md ch03
C:\Github\building-llm-applications>cd ch03
C:\Github\building-llm-applications\ch03>python -m venv env_ch03
C:\Github\building-llm-applications\ch03>.\env_ch03\Scripts\activ
(env_ch03) C:\Github\building-llm-applications\ch03>
```

Next, install the required packages—`tiktoken`, `notebook`, and `langchain`. If you've cloned the repo from GitHub, use `pip install -r requirements.txt`. Otherwise, install them as follows:

```
(env_ch03) C:\Github\building-llm-applications\ch03>pip install t
```

Once the installation is complete, start the Jupyter notebook:

```
(env_ch03) C:\Github\Building-llm-applications\ch03>jupyter noteb
```

Now, open or create a notebook and name it `03-summarization_examples.ipynb`, then save it.

3.1.1 Chunking the text into Document objects

Let's summarize the book "Moby Dick" using its text file, `Moby-Dick.txt`, downloaded from The Project Gutenberg. You can locate the `Moby-Dick.txt` file in the chapter's subfolder on my Github page (<https://github.com/roberto-inf/building-llm-applications/tree/main/ch03>). Place this file in your `ch03`

folder and load the text into a variable:

```
with open("./Moby-Dick.txt", 'r', encoding='utf-8') as f:
    moby_dick_book = f.read()
```

IMPORTANT

Keep in mind that running the code on the full "Moby Dick" text can get expensive. The provided "Moby-Dick.txt" file is a shorter version, containing only 5 chapters and about 18,000 tokens. Running the code a few times with this version shouldn't cost much. However, if you plan to run many tests, you may want to reduce the file size even more to save money. Each time you execute a chain with an LLMChain block, you'll be charged. If budget allows, you can use the full version of the book, labeled "Moby-Dick_ORIGINAL_EXPENSIVE.txt." The entire "Moby Dick" text has around 300,000 words, or about 350,000 tokens. Using GPT-4o, which costs \$2.50 per million tokens, processing the full text would cost about \$0.75. Running it multiple times will add up. If you switch to GPT-4o-mini, which costs \$0.15 per million tokens, the cost drops to around \$0.05, making it much cheaper.

I'll cover chunking strategies in detail—like chunking by size and content structure—in chapter 8. For now, let's split the text into chunks of about 3,000 tokens each to simulate a context window shorter than the GPT4o-mini model we'll be using.

3.1.2 Split

First, import the necessary libraries:

```
from langchain_openai import ChatOpenAI
from langchain_text_splitters import TokenTextSplitter
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableLambda, RunnableParallel
import getpass
```

Next, retrieve your OpenAI API key using the getpass function:

```
OPENAI_API_KEY = getpass.getpass('Enter your OPENAI_API_KEY')
```

You'll be prompted to enter your `OPENAI_API_KEY`. After that, instantiate the OpenAI model in a new notebook cell::

```
llm = ChatOpenAI(openai_api_key=OPENAI_API_KEY,model_name="gpt-4o
```

Now you're ready to set up the first chain, which will break the document into chunks of the specified size. In this chapter, you'll learn the basics of LCEL, with a more detailed exploration in the next chapter:

```
text_chunks_chain = (
    RunnableLambda(lambda x:
        [
            {
                'chunk': text_chunk,
            }
            for text_chunk in
                TokenTextSplitter(chunk_size=3000, chunk_overlap=1
        ]
    )
)
```

Note

LangChain chains are made up of components that implement the `Runnable` interface, an abstract Python class that defines how a component takes input, processes it, and returns output. Any class implementing `Runnable` can be part of a chain. `RunnableLambda` lets you turn any Python callable into a `Runnable`, making it easy to include custom functions in a LangChain chain. It's similar to Python's lambda expression, where you run code with a parameter and optionally return output without defining a full function. With `RunnableLambda`, you can create a chain component without writing a separate class to implement the `Runnable` interface. In this example, the code wrapped by `RunnableLambda` takes input text as a string through the `x` parameter and passes it to the `split_text()` function, which breaks the text into chunks.

3.1.3 Map

The next step is to set up the "map" chain, which will run a summarization prompt for each document chunk. If you're unfamiliar with "map-reduce,"

refer to the sidebar for more details.

```
summarize_chunk_prompt_template = """
Write a concise summary of the following text, and include the main
Text: {chunk}
"""

summarize_chunk_prompt = PromptTemplate.from_template(summarize_c
summarize_chunk_chain = summarize_chunk_prompt | llm

summarize_map_chain = (
    RunnableParallel (
        {
            'summary': summarize_chunk_chain | StrOutputParser()
        }
    )
)
```

Note

In this chain, I've used `RunnableParallel`, which is similar to `RunnableLambda`, but it operates on a sequence, processing each element in parallel. In this case, we'll feed the sequence of text chunks to the `summarize_map_chain`, and each chunk will be summarized in parallel by the inner `summarize_map_chain`.

Map-reduce

Map-reduce is a programming model for processing large datasets in two steps. First, the "map" operation splits the data into smaller subsets, each processed independently by the same function. This step typically returns a list of results, grouped by a key. Next is the "reduce" operation, where results for each key are aggregated into a single outcome. The final output is a list of key-value pairs, where the keys come from the map step and the values are the result of aggregating the mapped data in the reduce step.

3.1.4 Reduce

Setting up the reduce chain, which summarizes the summaries from each document chunk, follows a process similar to the map chain but requires a bit

more setup. Start by defining the prompt template:

```
summarize_summaries_prompt_template = """
Write a concise summary of the following text, which joins sever
Text: {summaries}
"""
```

```
summarize_summaries_prompt = PromptTemplate.from_template(summari
```

Next, you can configure the reduce chain:

```
summarize_reduce_chain = (
    RunnableLambda(lambda x:
        {
            'summaries': '\n'.join([i['summary'] for i in x]),
        })
    | summarize_summaries_prompt
    | llm
    | StrOutputParser()
)
```

The reduce chain includes a lambda function that combines the summaries from the map chain into a single string. This string is then processed by the `summarize_summaries_prompt` prompt, which generates a final summary of the combined content.

3.1.5 Map-reduce combined chain

Finally, we combine the document-splitting chain, the map chain, and the reduce chain into a single map-reduce chain:

```
map_reduce_chain = (
    text_chunks_chain #A
    | summarize_map_chain.map() #B
    | summarize_reduce_chain #C
)
```

This setup efficiently splits the input document in chunks, summarizes each chunk, and then compiles those summaries into a final summary. The `map()` function on `summarize_map_chain` is essential to enable parallel processing of the chunks.

3.1.6 Map-reduce execution

Everything is set up. Start the map-reduce summarization of the large document with this command (if you are on the OpenAI free-tier, this might fail with a 429 Rate Limit error):

```
summary = map_reduce_chain.invoke(moby_dick_book)
```

If you run `print(summary)`, you'll get output similar to the following:

```
The introduction to the Project Gutenberg eBook of "Moby-Dick; or  
As Ishmael shares a bed with Queequeg, whom he initially fears to
```

WARNING

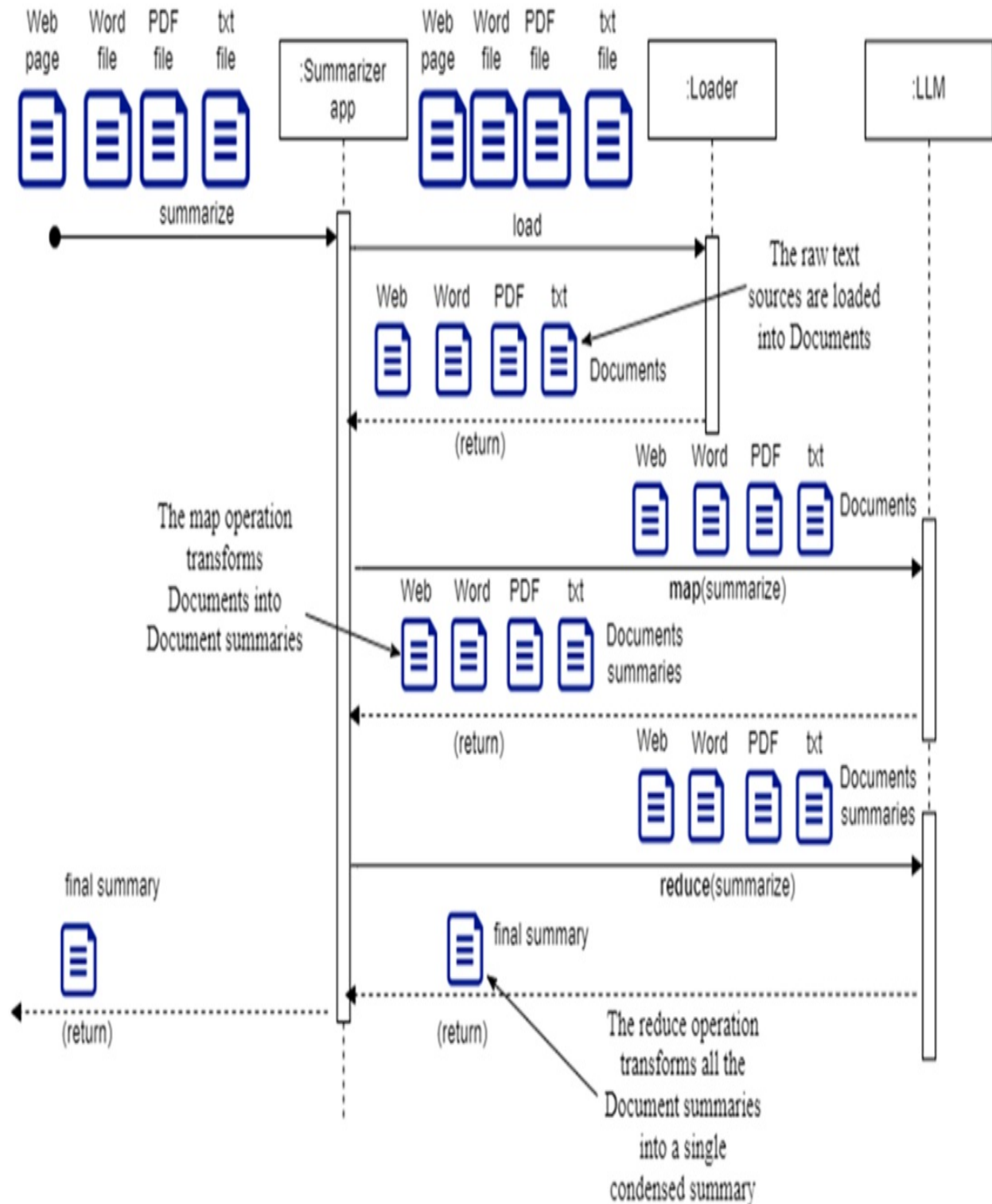
As previously mentioned, running the `map_reduce_chain` will incur costs; the longer the text you wish to summarize, the higher the cost. Therefore, consider further shortening the input text (in our case, the `Moby-Dick.txt` ebook file) if you want to limit expenses. Additionally, ensure your OpenAI API credit balance remains positive to avoid errors such as: `RateLimitError: Error code: 429 - {'error': {'message': 'You exceeded your current quota, please check your plan and billing details [...]'. If necessary, log in to the OpenAI API page, navigate to Settings > Billing, and set a credit of at least $5.`

Let's now proceed to the next use case: summarizing across documents.

3.2 Summarizing across documents

You can easily learn how to summarize information across various data sources, such as Wikipedia or local files in Microsoft Word, PDF, and text formats. This process, as shown in Figure 3.2, is similar to the map-reduce technique used in the previous section.

Figure 3.2 Summarizing across documents using the Map-Reduce technique seen earlier: in this method, each document chunk undergoes a map operation to generate a summary. These individual summaries are then further condensed into a single summary through the reduce operation.

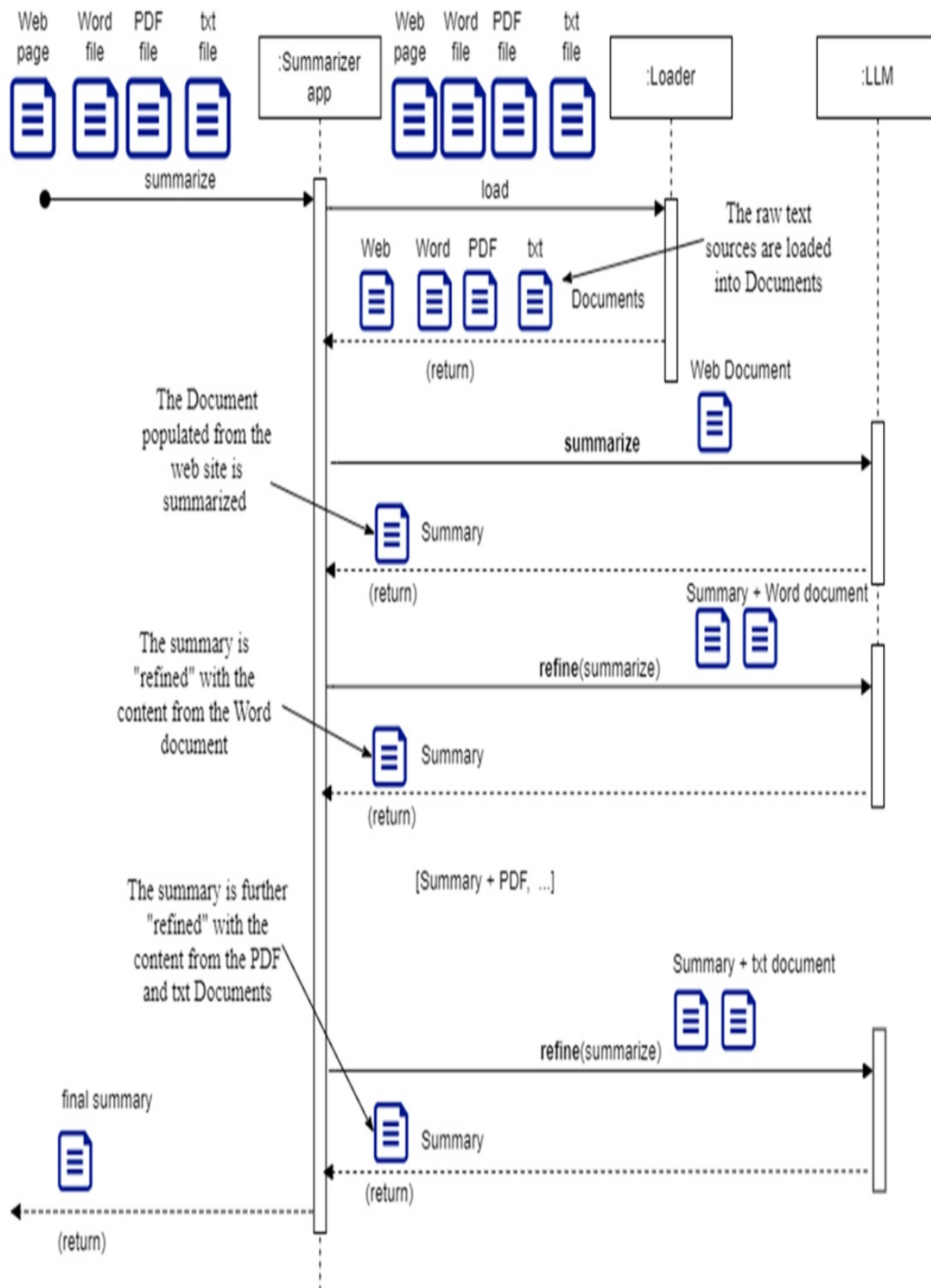


In the sequence diagram in Figure 3.2, content from each raw text source is loaded into a corresponding Document instance. During the "map" operation, these Document objects are converted into individual summaries, which are

then combined into a single summary during the "reduce" operation.

Next, I'll introduce you to an alternative technique called "Refine," illustrated in Figure 3.3.

Figure 3.3 Summarizing across documents using the Refine technique: with this approach, a final summary is constructed incrementally by iteratively summarizing the combination of the current final summary and one of the document chunks. This process continues until all document chunks have been processed, resulting in the completion of the final summary.



In this method, you progressively build the final summary by refining it with each step. Each document is sent to the LLM for summarization, along with the current draft of the summary. This continues until all documents are processed, leading to the final summary. Map-reduce works well for summarizing large volumes of text, where some content loss is acceptable to manage the processing load. In contrast, the refine technique is better when you want to ensure that the essence of each part is fully captured in the final summary.

3.2.1 Creating a list of Document objects

When summarizing a large document, you typically start by breaking it into smaller chunks, treating each chunk as a separate document. In this case, we're beginning with a set of existing documents, so there's no need to split anything. How you create each Document object will depend on the source of the text. I'll show you how to summarize content from four different sources: a Wikipedia page and a set of files in various formats (txt, docx, PDF) stored in a local folder. All the content is related to Paestum, a Greek colony on the Cilento coast in southern Italy around 500 BC. You'll use the appropriate DocumentLoader for each data source, selecting from the many options introduced earlier in section 1.6 on LangChain's Document object model..

3.2.2 Wikipedia content

Let's begin with the Wikipedia content. While you can create a document from web-based data content using the WebBaseLoader, specific loaders are customized to retrieve content from particular websites, such as the IMDBLoader for the Internet Movie Script Database (IMSDb) website, the AZLyricsLoader for the AZ Lyrics website, and the WikipediaLoader for the Wikipedia website.

Firstly, install the package for the loaders, including the ones for Docx2txtLoader (for Word files) and the PyPDFLoader (for PDFs). You can install them all at once to avoid stopping and restarting the Jupyter notebook multiple times. Alternatively, you can open a separate OS shell and install the packages there after activating your virtual environment:

```
(env_ch03) C:\Github\Building-llm-applications\ch03>pip install w
```

If you cloned the repository from Github and run `pip install -r requirements.txt` after activating the virtual environment, all the required packages should already be installed.

Once everything is installed, start the Jupyter notebook as usual:

```
(env_ch03) C:\Github\Building-llm-applications\ch03>jupyter noteb
```

Now, import the content from the Paestum Wikipedia page:

```
from langchain.document_loaders import WikipediaLoader

wikipedia_loader = WikipediaLoader(query="Paestum", load_max_docs
wikipedia_docs = wikipedia_loader.load()
```

NOTE

The `WikipediaLoader` may load content from other Wikipedia hyperlinks referenced in the requested article. For example, the Paestum article references the National Archeological Museum of Paestum, the Lucania region, Lucanians, and the temples of Hera and Athena, resulting in additional content loaded. Thus, it returns a Document list rather than a single Document object. I've set the maximum number of documents returned to 2 to save on summarization costs, but you can adjust it as needed.

3.2.3 File based content

To get started, download or pull the Paestum folder from GitHub and place it in your local ch03 directory (if you didn't clone the entire repository). The Paestum subfolder within ch03 contains three files:

- `Paestum-Britannica.docx`: Content sourced from the Encyclopedia Britannica website.
- `PaestumRevisited.pdf`: An excerpt from "Paestum Revisited," a master thesis submitted at Stockholm University. The extract comprises only 4 pages, but you have the option to use the full document located in the same folder (`PaestumRevisited-StocholmsUniversitet.pdf`).

- Paestum-Encyclopedia.txt: Content taken from Encyclopedia.com.

Below is the process to load these files into corresponding documents:

```
from langchain.document_loaders import Docx2txtLoader
from langchain.document_loaders import PyPDFLoader
from langchain.document_loaders import TextLoader

word_loader = Docx2txtLoader("Paestum/Paestum-Britannica.docx")
word_docs = word_loader.load()

pdf_loader = PyPDFLoader("Paestum/PaestumRevisited.pdf")
pdf_docs = pdf_loader.load()

txt_loader = TextLoader("Paestum/Paestum-Encyclopedia.txt")
txt_docs = txt_loader.load()
```

The document variables (word_docs, pdf_docs, txt_docs) are in plural mode because a loader always returns a list of documents, even if the list contains only one item.

NOTE

You may have noticed the direct creation of a Document object from Paestum-Encyclopedia.txt using a TextLoader. You might wonder why the Moby-Dick.txt file was read with the Python file reader previously. The reason is that in that case, the intention was to split the content into a specific number of tokens to fit the LLM prompt, requiring manual creation of a Document for each.

3.2.4 Creating the Document list

You can now merge all the documents from various sources into a single Document list:

```
all_docs = wikipedia_docs + word_docs + pdf_docs + txt_docs
```

With everything compiled, you're ready to summarize the content using the "refine" technique. Before you move forward, take a moment to check out the sidebar on Document Loaders for alternative ways to create your document

list..

Document Loaders

While the sections above have introduced document loaders for specific data sources, I encourage you to explore the `UnstructuredLoader` as well. It enables you to import content from various file types, including Word, PDF, and txt files, among others.

Another option is the `DirectoryLoader`, which utilizes the `UnstructuredLoader` internally. It allows you to load content from files of different formats located in the same folder in a single operation.

As an exercise, I recommend recreating the documents from the Word, PDF, and txt Paestum content using either the `UnstructuredLoader` or the `DirectoryLoader`. If you choose to do so, you'll need to install the related package and refer to the documentation on the LangChain website:

```
pip install "unstructured[all-docs]"
```

The LangChain framework provides numerous loaders for retrieving content from diverse data sources. I highly encourage you to explore the list and experiment with any loaders that pique your interest:

https://python.langchain.com/docs/integrations/document_loaders

3.2.5 Progressively refining the final summary

Now that everything is set up, you can create a chain to generate the final summary step by step. Begin by importing the necessary modules:

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
import getpass
```

Next, capture the `OPENAI_API_KEY` and set up the LLM model as before:

```
OPENAI_API_KEY = getpass.getpass('Enter your OPENAI_API_KEY')
```

```
llm = ChatOpenAI(openai_api_key=OPENAI_API_KEY,model_name="gpt-4o
```

Now, define the chain, with related prompt, for summarizing individual documents:

```
doc_summary_template = """Write a concise summary of the following
{text}
DOC SUMMARY:"""
doc_summary_prompt = PromptTemplate.from_template(doc_summary_tem
doc_summary_chain = doc_summary_prompt | llm
```

Next, set up the chain for refining the summary by iteratively combining the current summary with the summary of an additional document:

```
refine_summary_template = """
Your must produce a final summary from the current refined summary
which has been generated so far and from the content of an additional
This is the current refined summary generated so far: {current_re
This is the content of the additional document: {text}
Only use the content of the additional document if it is useful,
otherwise return the current full summary as it is."""
refine_summary_prompt = PromptTemplate.from_template(refine_summa
refine_chain = refine_summary_prompt | llm | StrOutputParser()
```

Finally, define a function that loops over each document, summarizes it using the doc_summary_chain, and refines the overall summary using the refine_chain:

```
def refine_summary(docs):
    intermediate_steps = []
    current_refined_summary = ''
    for doc in docs:
        intermediate_step = \
            {"current_refined_summary": current_refined_summary,
             "text": doc.page_content}
        intermediate_steps.append(intermediate_step)

        current_refined_summary = refine_chain.invoke(intermediat

    return {"final_summary": current_refined_summary,
            "intermediate_steps": intermediate_steps}
```

You can now start the summarization process by calling `refine_summary()` on your prepared document list:

```
full_summary = refine_summary(all_docs)
```

Printing the `full_summary` object will show the final summary under `final_summary` and the intermediate steps under `intermediate_steps`. Although the results steps are shortened for convenience, I encourage you to observe how the summary evolves at each stage:

```
print(full_summary )
```

Here is an excerpt from the output:

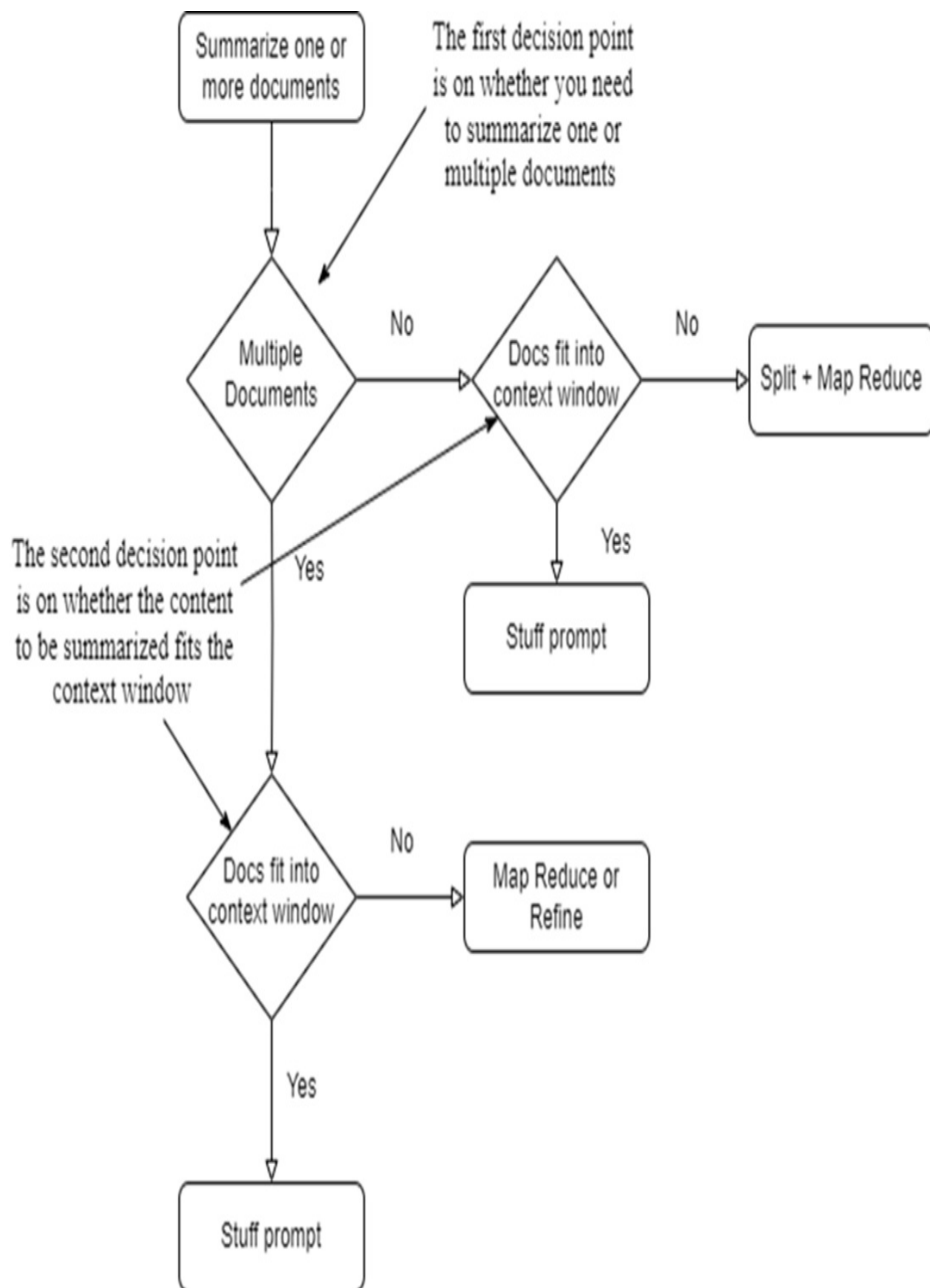
```
{'final_summary': "***Final Summary:**\n\nPaestum, an ancient Gree
```

We have covered a few summarization techniques, so let's pause briefly to reflect on what we have learnt.

3.3 Summarization flowchart

To wrap up this chapter, I've included a flowchart to help you choose the most appropriate summarization technique for your specific needs:

Figure 3.4 Summarization flowchart This flowchart guides you in selecting the right summarization approach based on whether you need to summarize multiple unrelated documents and whether the input text fits within the context window



As shown in the flowchart in Figure 3.4, the first key decision is whether you're summarizing one or multiple documents. If it's just one document and it fits within the context window, you can "stuff" the entire document into a single prompt for summarization. If it doesn't fit, use the map-reduce method. For multiple documents, if they all fit within the context window, you can also stuff them into a single prompt. If not, use map-reduce for a large number of documents, or the refine technique if you want to ensure the core of each document is included in the final summary.

3.4 Summary

- Summarization is a great starting point for building LLM applications due to its straightforwardness.
- The choice of technique depends on whether you're summarizing a single document or multiple documents and if the text fits within the context window.
- Typically, the first step in summarization involves loading each raw text source into a LangChain Document object.
- For many summarization tasks, a map-reduce approach is effective. This method summarizes each document or chunk individually in the "map" stage, then combines these summaries in the "reduce" stage to create a final summary.
- When summarizing multiple documents, load each document with the appropriate loader, summarize them individually, and progressively refine the summary by integrating each document's summary until all are included.
- Map-reduce is suited for large text summaries where some content loss is acceptable to handle the processing load, while the refine technique ensures the essence of each part is fully retained in the final summary.

4 Building a research summarization engine

This chapter covers

- Explaining what a research summarization engine is.
- Organizing core functionality, such as web searching and scraping functions.
- Using prompt engineering for creating web searches and summarizing results.
- Structuring the process into individual LangChain chains.
- Integrating various sub-chains into a comprehensive main chain.
- Applying advanced LCEL features for parallel processing.

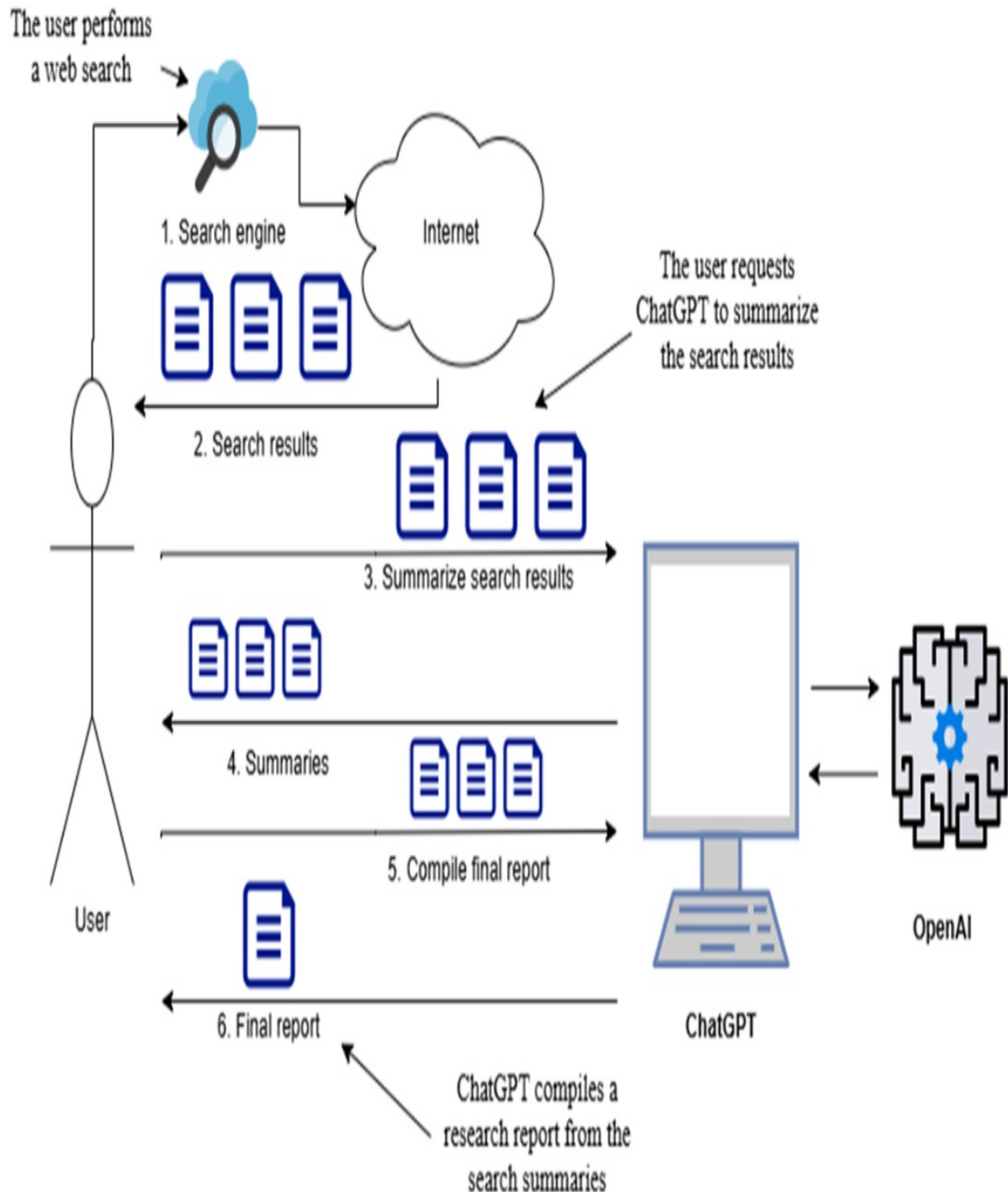
Building on the content summarization techniques from chapter 3, this chapter will guide you through creating a research summarization engine. This LLM application will process user queries, perform web searches, and compile a comprehensive summary of the findings. We'll develop this project step by step, starting with the basics and gradually increasing in complexity. Along the way, you will deepen your knowledge of LangChain as I introduce creating LLM chains with *LangChain Expression Language* (LCEL).

4.1 Overview of a research summarization engine

Imagine you're researching various topics, like an NBA player, a tourist destination, or whether to invest in a stock. Manually, you'd perform a web search, sift through results, read related pages, take notes, and compile a summary. A modern approach is to let an LLM handle this work. You could copy text from each web page, paste it into a ChatGPT prompt for summarization, and repeat for multiple pages. Then, combine these summaries into a final prompt for a consolidated summary (see figure 4.1).

Figure 4.1 Semi-automated summarization with LLM: You prompt ChatGPT to summarize each

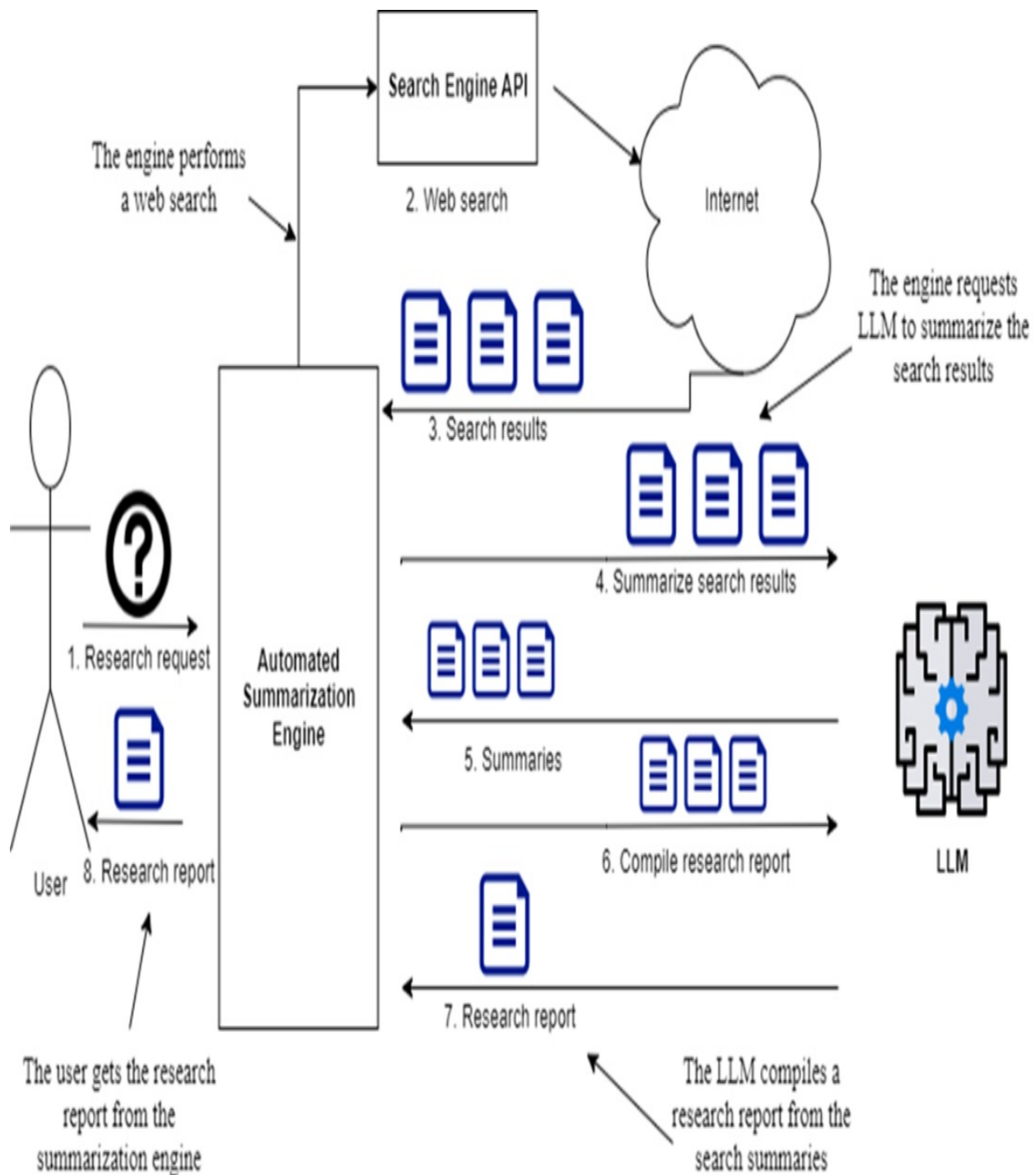
web search result and then compile them into a consolidated summary.



A more efficient method is to develop a fully automated research summarization engine. This engine can perform web searches, summarize the

results, and compile a final report automatically (see figure 4.2). It's a valuable tool for handling any research query.

Figure 4.2 Automated research summarization engine: Ask a question, and the engine performs a web search, returns URLs, scrapes and summarizes web pages, and compiles a research report for you.



We'll build this engine using LangChain. First, we'll implement web searching and scraping, then set up the OpenAI LLM model for summarization, and finally integrate all components into a Python application. Initially, it will run as an executable and later if you wish, you could expose it as a REST API.

4.2 Setting up the project

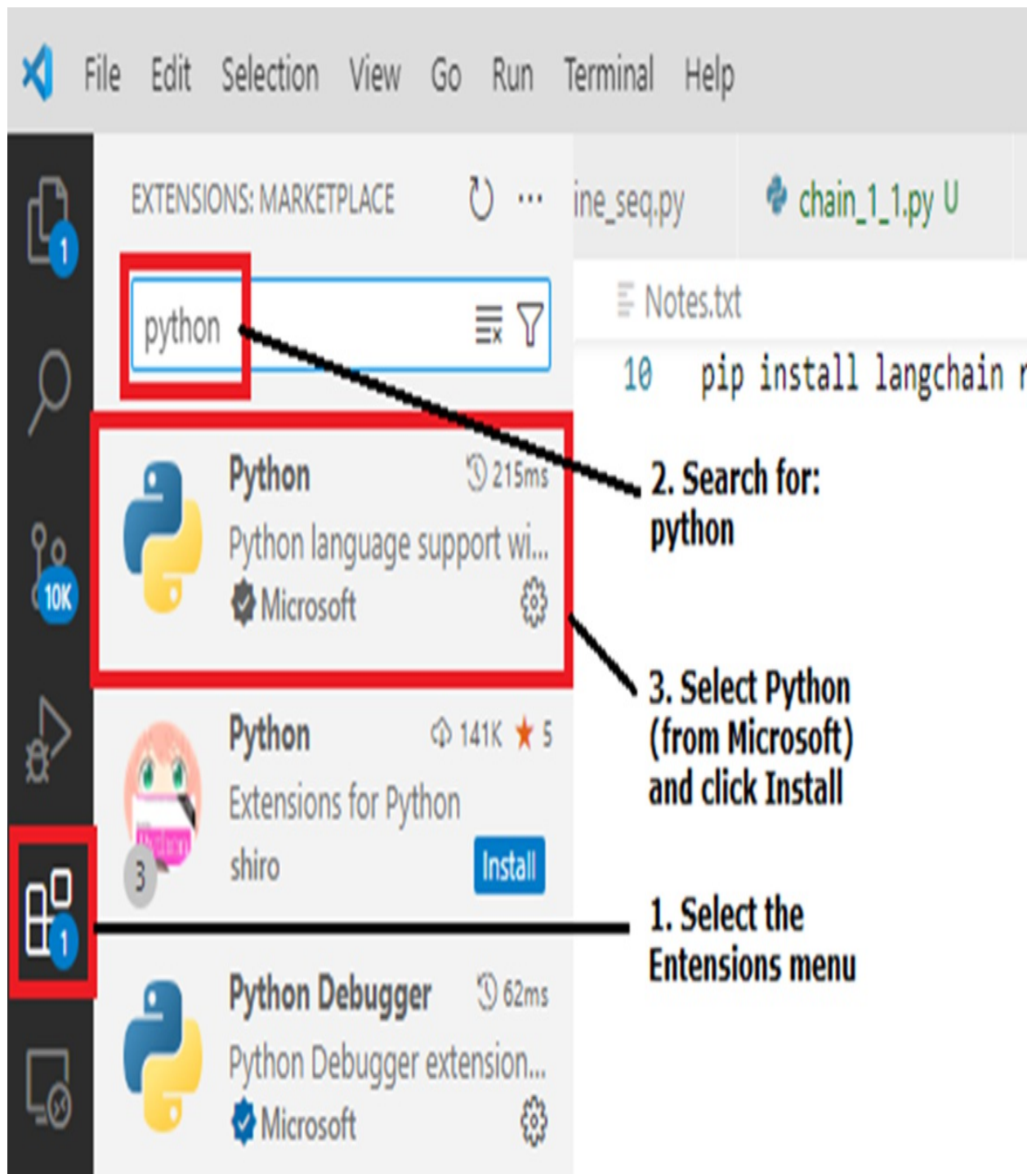
I am assuming you're using Visual Studio Code with the free Python extension and running on Windows, though you can opt for alternative Python IDEs like PyCharm. If you're new to Visual Studio Code, get it set up by following the sidebar instructions.

Installing Visual Studio Code and the Python extension

Download and install the appropriate version of Visual Studio Code for your OS from the official website:

<https://code.visualstudio.com/download>

Figure 4.3 Screenshot of the Extensions marketplace panel: 1) Select the Extensions menu; 2) enter python in the search box; 3) Select Python (from Microsoft) and click Install.



Once installed, open Visual Studio Code, and click the Extensions icon on the left-hand menu. Then, search for Python, select the Python extension (from Microsoft), and click install.

I'll briefly guide you through setting up a Python project in Visual Studio

Code, creating a virtual environment, activating it, and installing necessary packages.

Using your file explorer or shell, create an empty folder named `ch04` in your source code area. For example:

```
C:\Github\building-llm-applications\ch04
```

Open Visual Studio Code, then choose File > Open Folder, navigate to the `ch04` folder, and click Select Folder.

Open a terminal within Visual Studio Code by selecting Terminal > New Terminal.

The terminal should display the path to the folder you just created. On Windows, you'll see something like this:

```
PS C:\Github\building-llm-applications\ch04>
```

If you've just installed Visual Studio Code or you're new to it, enable the terminal by running this command (you only need to do it once):

```
PS C:\Github\building-llm-applications\ch04> Set-ExecutionPolicy
```

Within this terminal, as usual, create a virtual environment, activate it, and install the required Python packages (I'm omitting the full path to `ch04` for convenience):

```
... ch04> python -m venv env_ch04
... ch04> .\env_ch04\Scripts\activate
```

If you activate the virtual environment in PowerShell using `activate.ps1`, you may see a prompt asking: “Do you want to run software from this untrusted publisher?” Respond by typing `A` (Always run) to proceed.

If you have cloned this repository from GitHub, you can install the required packages with `pip install -r requirements.txt`, otherwise:

```
(env_ch04) ... ch04> pip install langchain langchain_openai langc
```

Once the required packages are installed, I recommend creating a custom Run configuration to ensure you're running and debugging your code within the `env_ch04` virtual environment you just set up.

To create the configuration:

1. In the top menu, go to Run > Open Configurations.
2. This will open the `launch.json` file in the editor.
3. Replace or add the following configuration:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Python Debugger: Current File",
      "type": "debugpy",
      "request": "launch",
      "program": "${file}",
      "console": "integratedTerminal",
      "python": "${workspaceFolder}/env_ch04/Scripts/python"
    }
  ]
}
"${workspaceFolder}/env_ch04/bin/python"
```

Note

If you're using macOS or Linux, replace the path in the "python" field with:

This custom Run configuration ensures your code runs with the correct interpreter and environment. You'll use it later for debugging. With everything in place, you're now ready to start coding!

4.3 Implementing the core functionality

Looking again at figure 4.2, it's clear that our research summarization engine depends on two key capabilities we must provide: one for conducting web searches and another for extracting text from related web pages. Additionally, you'll need a utility function to initialize an instance of the LLM client you'd like to use.

4.3.1 Implementing web searching

We'll use the LangChain wrapper for the DuckDuckGo search engine to perform web searches. Its results method returns a list of objects, each containing the result URL in a property called "link".

Add a new empty file named `web_searching.py` to the project and fill it with the code below:

```
from langchain_community.utilities import DuckDuckGoSearchAPIWrap
from typing import List

def web_search(web_query: str, num_results: int) -> List[str]:
    return [r["link"] for r in DuckDuckGoSearchAPIWrapper().result(web_query, num_results=num_results)]
```

Create a separate Python file, such as `web_searching_try.py`, to test the search function:

```
from web_searching import web_search

result = web_search(web_query="How many titles did Michael Jordan win")
print(result)
```

NOTE

If you're unfamiliar with Visual Studio Code, you can execute the code above by pressing F5 and then selecting Python Debugger > Python File. You can also set breakpoints and run the code step by step by pressing F10 on each line.

In the terminal, you'll get a list of URLs like the following ones, representing the results of your search.

```
['https://en.wikipedia.org/wiki/List_of_career_achievements_by_Michael_Jordan']
```

Note

Other web search engine wrappers provided by LangChain are TavilySearchResults and GoogleSearchAPIWrapper. Both require an API key, so I chose DuckDuckGoSearchAPIWrapper because it doesn't.

4.3.2 Implementing web scraping

We'll scrape the web pages from the result list using BeautifulSoup, which is a web scraper library. Place the code shown in the listing below in a file named `web_scraping.py`:

Listing 4.1 `web_scraping.py`

```
import requests
from bs4 import BeautifulSoup

def web_scrape(url: str) -> str:
    try:
        response = requests.get(url)

        if response.status_code == 200:
            soup = BeautifulSoup(response.text, "html.parser")
            page_text = soup.get_text(separator=" ", strip=True)

            return page_text
        else:
            return f"Could not retrieve the webpage: Status code"
    except Exception as e:
        print(e)
        return f"Could not retrieve the webpage: {e}"
```

Let's try out the function by placing the code below in a file named `web_scraping_try.py`:

```
from web_scraping import web_scrape
result = web_scrape('https://en.wikipedia.org/wiki/List_of_career_achievements_by_Michael_Jordan')
print(result)
```

After you run this script, the output will be similar to the following excerpt:

List of career achievements by Michael Jordan - Wikipedia Jump to

As you can see, much of the scraped content is not relevant, but don't worry for now; the LLM will extract the relevant bits we're interested in.

4.3.3 Instantiating the LLM client

For this use case, we'll use the OpenAI GPT-4o-mini model instead of GPT-4o. Create a function to instantiate the OpenAI client as follows and place it in a file named `llm_models.py` (replace 'YOUR_OPENAI_API_KEY' with your OpenAI key):

```
from langchain_openai import ChatOpenAI
openai_api_key = 'YOUR_OPENAI_API_KEY'
def get_llm():
    return ChatOpenAI(openai_api_key=openai_api_key,
                      model_name="gpt-4o-mini")
```

Note

There are various ways to store secrets like the OpenAI key in Visual Studio Code, each with its pros and cons. I'm hardcoding the OpenAI key inline here, but you're free to implement it with your preferred approach.

4.3.4 JSON to Python object converter

Let's make a utility function that converts JSON text from the LLM into a Python object, usually a dictionary or sometimes a list. If the JSON is malformed, it will return an empty dictionary. Add the following code to a file called `utilities.py`:

```
import json

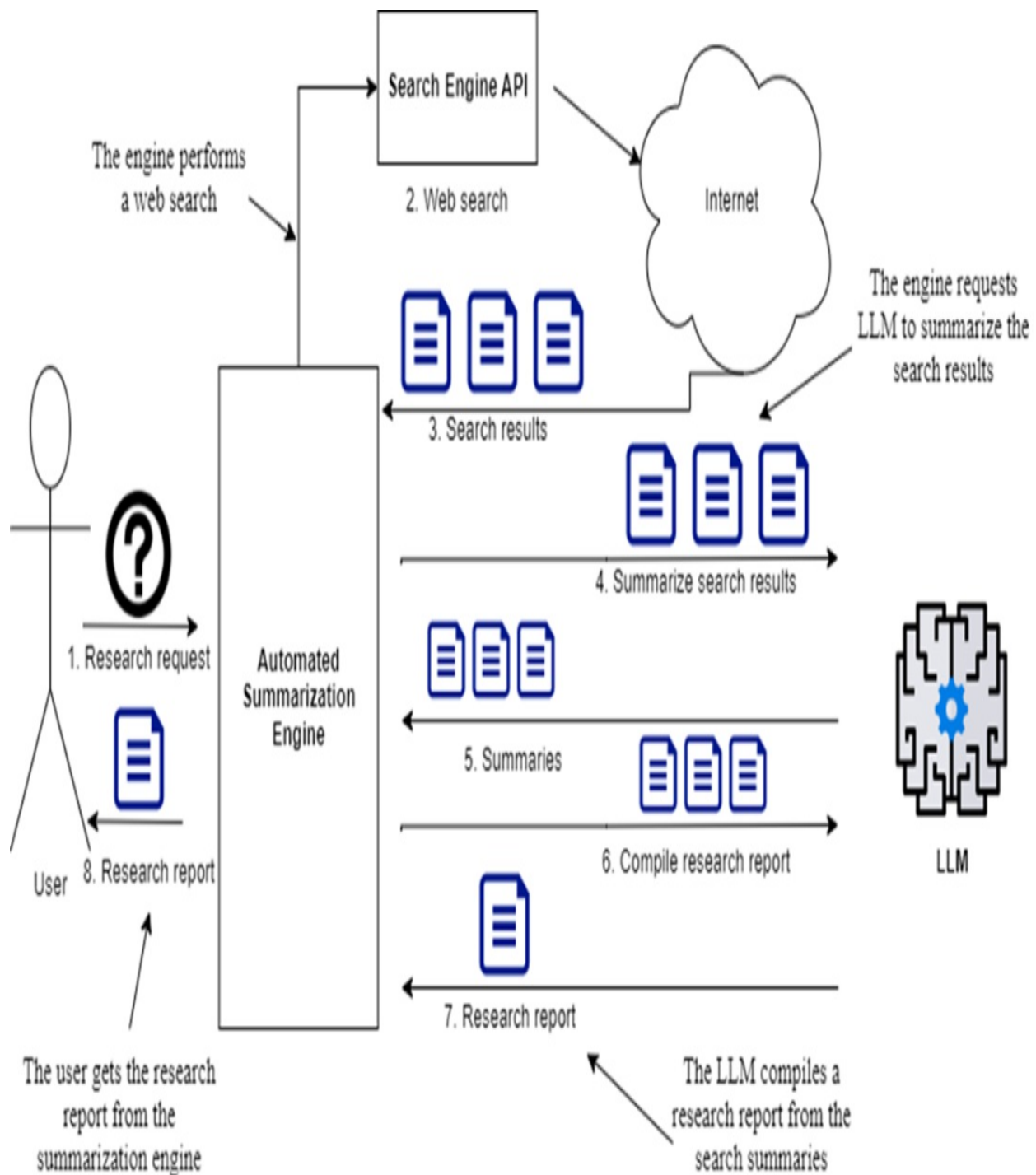
def to_obj(s):
    try:
        return json.loads(s)
    except Exception:
        return {}
```

To recap: we've set up a Visual Studio Code project and implemented the core capabilities we need. Before building the engine to orchestrate web searching, web scraping, and summarization requests to the LLM, let's step back and take a second look at the big picture.

4.4 Enhancing the architecture with query rewriting

Let's revisit the diagram in Figure 4.2, which I've included again for convenience as Figure 4.4 below

Figure 4.4 Automated research summarization engine: Ask a question, and the engine performs a web search, returns URLs, scrapes and summarizes web pages, and compiles a research report for you.



If you look closely at the architecture diagram in Figure 4.4, you might notice a potential improvement: instead of sending the original research request directly to the web search engine, you can use the LLM to create multiple search queries. This technique, called "query rewriting" or "multiple query generation," is similar to a method used to enhance RAG searches, which I'll cover in later chapters. Rewriting the original question into multiple queries offers several advantages. It can clarify ambiguous or unclear queries, fix grammar or syntax errors that could confuse the search engine, and add context to short queries to improve search results. For complex queries, breaking them into simpler queries provides useful context for better answers. This is the key reason for rewriting a single query into multiple targeted searches.

For instance, rather than querying the search engine "How many titles did Michael Jordan win?", you can prompt ChatGPT as follows:

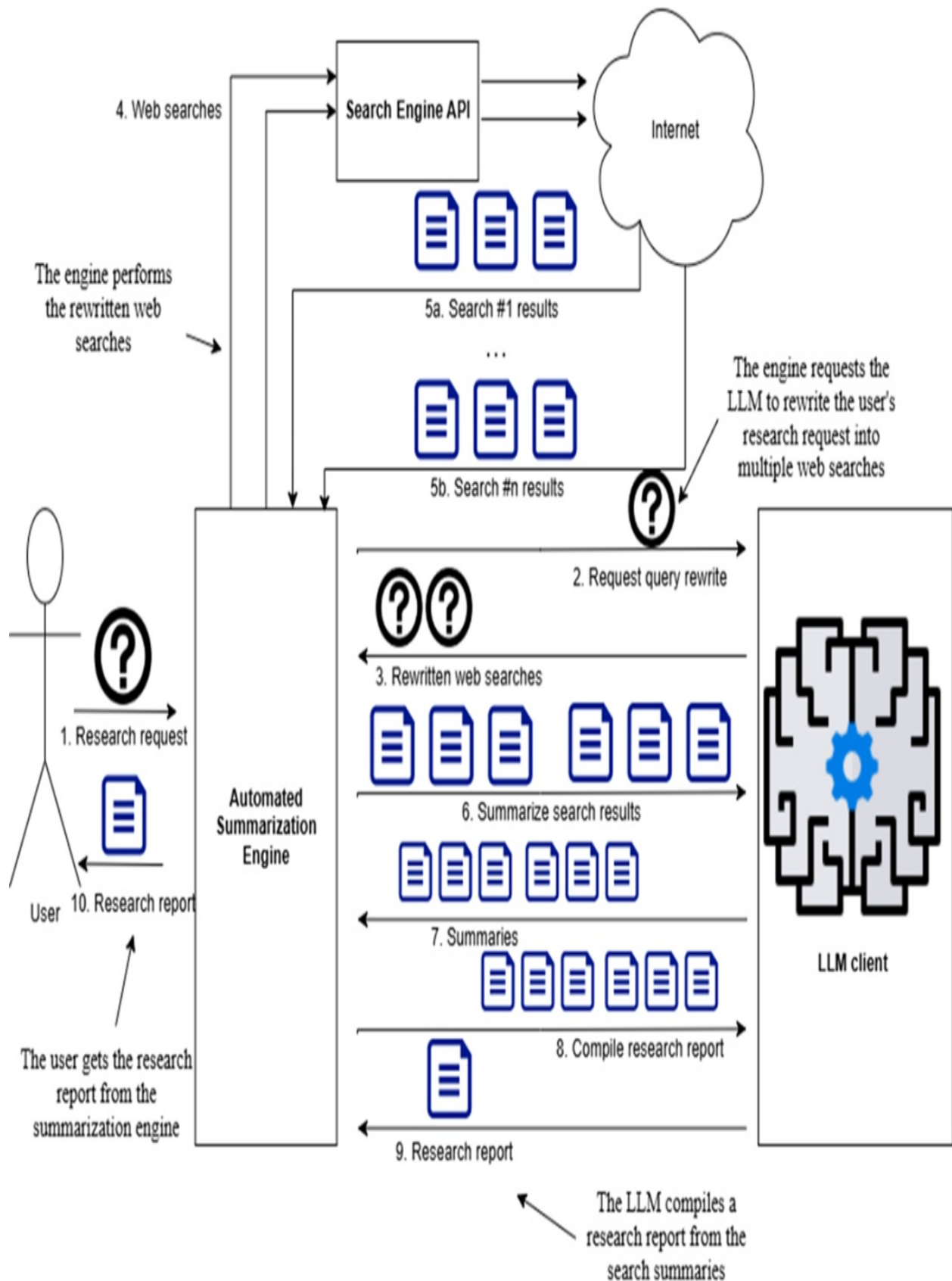
Generate three web search queries to research the following topic, aiming for a comprehensive report: "How many titles did Michael Jordan win?"

You'll receive queries like these:

1. "Michael Jordan NBA championships total"
2. "List of NBA titles won by Michael Jordan"
3. "Michael Jordan basketball career championships count"

Consequently, the engine will conduct research by performing these three web searches and gathering results from each. This updated workflow is depicted in the diagram in figure 4.5.

Figure 4.5 Revised system architecture diagram, incorporating query rewriting: The process begins by tasking the LLM to generate a specified number of queries based on the user's research question. These queries are then submitted to the search engine. The subsequent processing remains consistent with the illustration in figure 4.2.



Technically, the updated architecture is more complex, which could raise concerns about performance due to the additional web searches. If processed sequentially, the response time would increase linearly; for instance, running four web searches would take four times as long. However, you can speed up processing through parallelization, which I'll cover in later sections. For now, we'll start with a sequential processing approach and explore parallelization later.

4.5 Prompt engineering

Before we assemble the building blocks from section 4.3, we need to focus on some prompt engineering. We'll create prompts for generating web search queries, summarizing individual web pages, and producing the final report. Let's tackle each of these tasks step by step.

4.5.1 Crafting Web Search Prompts

Imagine a user asks a finance-related question like, "Should I invest in Apple stocks?" When guiding the LLM to generate web search queries based on this question, it's helpful to specify the persona that should frame these queries. For example, you might begin the prompt with: "You are an experienced finance analyst AI assistant. Your objective is to create detailed, insightful, unbiased, and well-structured financial reports based on the provided data and trends."

To generate these instructions dynamically based on the research question, you'll use a dedicated prompt. Start by designing a prompt that selects a suitable research assistant and provides instructions tailored to the user's query.

Create a file named `prompts.py` and begin with the following code:

Listing 4.2 prompts.py: Prompts to select and generate assistant instructions

```
from langchain.prompts import PromptTemplate

ASSISTANT_SELECTION_INSTRUCTIONS = """
```

You are skilled at assigning a research question to the correct r
There are various research assistants available, each specialized
Each assistant is identified by a specific type. Each assistant h

How to select the correct assistant: you must select the relevant

Here are some examples on how to return the correct assistant inf

Examples:

Question: "Should I invest in Apple stocks?"

Response:

```
{{
  "assistant_type": "Financial analyst assistant",
  "assistant_instructions": "You are a seasoned finance analyst",
  "user_question": {user_question}
}}
```

Question: "what are the most interesting sites in Tel Aviv?"

Response:

```
{{
  "assistant_type": "Tour guide assistant",
  "assistant_instructions": "You are a world-travelled AI tour",
  "user_question": "{user_question}"
}}
```

Question: "Is Messi a good soccer player?"

Response:

```
{{
  "assistant_type": "Sport expert assistant",
  "assistant_instructions": "You are an experienced AI sport as",
  "user_question": "{user_question}"
}}
```

Now that you have understood all the above, select the correct re

Question: {user_question}

Response:

"""

```
ASSISTANT_SELECTION_PROMPT_TEMPLATE = PromptTemplate.from_templat
    template=ASSISTANT_SELECTION_INSTRUCTIONS
)
```

This prompt template has a couple of key features. First, it's a "few-shot prompt," as demonstrated by the three examples included, which help the LLM grasp our specific requirements. Second, it directs the LLM to return

results in JSON format, making it straightforward to convert the output into a Python dictionary for further processing.

With this prompt in place to select the appropriate assistant type and provide instructions, you can now proceed to create the prompt for generating web searches based on the user's question, as shown in the following listing.

Listing 4.3 prompts.py: Prompt to rewrite the user query into multiple web searches

```
WEB_SEARCH_INSTRUCTIONS = """
{assistant_instructions}

Write {num_search_queries} web search queries to gather as much i
on the following question: {user_question}. Your objective is to
You must respond with a list of queries such as query1, query2, q
[
    {"search_query": "query1", "user_question": "{user_question}"
    {"search_query": "query2", "user_question": "{user_question}"
    {"search_query": "query3", "user_question": "{user_question}"
]
"""

WEB_SEARCH_PROMPT_TEMPLATE = PromptTemplate.from_template(
    template=WEB_SEARCH_INSTRUCTIONS
)
```

The {assistant_instructions} placeholder will be filled with the "assistant_instructions" output from the previous assistant selection prompt you saw. This prompt also returns results in JSON format, making it easy to convert the output into a list of Python dictionaries for further processing.

Including the user question in the output ensures continuity throughout the process, allowing us to use it as needed, especially in the chain-based code that follows.

With the web search prompt completed, let's move on to creating the summarization prompts.

4.5.2 Crafting Summarization Prompts

The prompt for summarizing result pages closely resembles prompts from the previous chapter on summarization:

Listing 4.4 prompts.py: Prompt for summarizing result pages

```
SUMMARY_INSTRUCTIONS = """
Read the following text:
Text: {search_result_text}

-----

Using the above text, answer in short the following question.
Question: {search_query}

If you cannot answer the question above using the text provided a
Include all factual information, numbers, stats etc if available.
"""

SUMMARY_PROMPT_TEMPLATE = PromptTemplate.from_template(
    template=SUMMARY_INSTRUCTIONS
)
```

4.5.3 Research Report prompt

Similarly, the prompt for generating the research report is straightforward:

Listing 4.5 prompts.py: Prompt for generating the research report

```
# Research Report prompts adapted from https://github.com/assafel
RESEARCH_REPORT_INSTRUCTIONS = """
You are an AI critical thinker research assistant. Your sole purp

Information:
-----
{research_summary}
-----

Using the above information, answer the following question or top
The report should focus on the answer to the question, should be
in depth, with facts and numbers if available and a minimum of 1,

You should strive to write the report as long as you can using al
You must write the report with markdown syntax.
You MUST determine your own concrete and valid opinion based on t
```

```
Write all used source urls at the end of the report, and make sur
You must write the report in apa format.
Please do your best, this is very important to my career."""
```

```
RESEARCH_REPORT_PROMPT_TEMPLATE = PromptTemplate.from_template(
    template=RESEARCH_REPORT_INSTRUCTIONS
)
```

4.6 Initial implementation

In this section, I'll walk you through the initial implementation of our research summarization engine, following the steps laid out in the architectural diagram in Figure 4.3.

4.6.1 Importing Functions and Prompt Templates

To kick things off, create a Python file named `research_engine_seq.py` and begin importing the functions and prompt templates you've crafted in the preceding sections.

```
from web_searching import web_search
from web_scraping import web_scrape
from llm_models import get_llm
from utilities import to_obj
from prompts import (
    ASSISTANT_SELECTION_PROMPT_TEMPLATE,
    WEB_SEARCH_PROMPT_TEMPLATE,
    SUMMARY_PROMPT_TEMPLATE,
    RESEARCH_REPORT_PROMPT_TEMPLATE
)
```

4.6.2 Setting constants and input variables

Now, let's establish some constants to configure the application. The accuracy and diversity of the report will depend on the number of web searches and results per search you set. However, it's important to consider your budget when setting these values. For instance, configuring 4 web searches with 5 results per search would entail summarizing 20 web pages, which means roughly 2000 tokens per page, for a total of $20 \times 2000 = 40,000$ tokens. With the OpenAI GPT4o Mini model costing around \$0.0006 per

1,000 output tokens, this would amount to approximately \$0.01 per research request. I'll set lower configuration numbers, but feel free to adjust them as needed:

```
NUM_SEARCH_QUERIES = 2
NUM_SEARCH_RESULTS_PER_QUERY = 3
RESULT_TEXT_MAX_CHARACTERS = 10000
```

The sole input variable in the application is the one that captures the research question from the user:

```
question = 'What can I see and do in the Spanish town of Astorga?'
```

4.6.3 Instantiating the LLM client

Instantiating the LLM client is straightforward. You can do it simply as follows:

```
llm = get_llm()
```

4.6.4 Generating the web searches and collecting the results

In the process of generating web searches and collecting results, the first step is to execute the LLM prompt to determine the correct research assistant and related instructions based on the user's research question:

```
assistant_selection_prompt = ASSISTANT_SELECTION_PROMPT_TEMPLATE.
assistant_instructions = llm.invoke(assistant_selection_prompt)
```

To execute this code, place a breakpoint on the line `llm = get_llm()`. If you're new to Visual Studio Code, you can set a breakpoint by clicking to the left of the line number where you want the debugger to pause.

Next, start the debugger by clicking the Run & Debug icon on the left sidebar (or use the shortcut `Ctrl+Shift+D` on Windows). From the dropdown at the top, select the Run configuration named "Python Debugger: Current File", which you created at the end of Section 4.2. Then, click the play button next to it to start debugging.


```

        'search_query': wq['search_query']}
    for wq in web_search_queries_list]

```

Executing up to this point, the `searches_and_result_urls` variable would hold a list of Python dictionaries like this:

```

[{'result_urls': ['https://igotospain.com/one-day-in-astorga-on-t

```

Each dictionary shows a search query and its corresponding result URLs (3 for each query here). The next step is to flatten the results, so each dictionary contains a search query and just one result URL:

```

search_query_and_result_url_list = []
for qr in searches_and_result_urls:
    search_query_and_result_url_list.extend([{'search_query': qr[
        'result_url': r
    } for r in qr['result_urls']]

```

Now, `search_query_and_result_url_list` has 9 dictionaries, just as expected:

```

[{'search_query': 'Astorga attractions', 'result_url': 'https://w

```

With the web search queries and all related result URLs ready, the next move is to start scraping the web pages linked to these URLs.

4.6.5 Scraping the web results

Use the `web_scrape()` function to pull text from web pages linked through your search results:

```

result_text_list = [ {'result_text': web_scrape(url=re['result_ur
    'result_url': re['result_url'],
    'search_query': re['search_query']]
    for re in search_query_and_result_url_list]

```

This code populates `result_text_list` with 9 dictionaries, each carrying text from a web page:

```

[{'result_text': 'Astorga, Spain - WorldAtlas Astorga, Spain Asto
{'result_text': "Astorga, Spain: Uncovering the Jewels of a Hidde
Museum and Gaudi's Palace... ', 'result_url': 'https://citiesandatt

```

The next move is to summarize the information collected from each webpage.

4.6.6 Summarizing the web results

You'll ask the language model to summarize the text from each web page using the SUMMARY_PROMPT_TEMPLATE you set up earlier. This summary will also keep the original search queries and URLs, which you'll need later:

```
result_text_summary_list = []
for rt in result_text_list:
    summary_prompt = SUMMARY_PROMPT_TEMPLATE.format(
        search_result_text=rt['result_text'],
        search_query=rt['search_query'])

    text_summary = llm.invoke(summary_prompt)

    result_text_summary_list.append({'text_summary': text_summary,
                                    'result_url': rt['result_url'],
                                    'search_query': rt['search_query']})
```

This process results in result_text_summary_list, a list of 9 dictionaries. Each contains a summary of the scraped text, the URL where it was found, and the search query used to find it:

```
[{'text_summary': '\nAstorga is a municipality in Northwestern Sp
```

With these summaries, you've compiled all the information needed for the final research report.

4.6.7 Generating the research report

Let's review the prompt used to create the final report:

```
RESEARCH_REPORT_INSTRUCTIONS = """
You are an AI critical thinker research assistant. Your sole purp

Information:
-----
{research_summary}
-----
```

Using the above information, answer the ...

```
"""
```

Let's prepare the final report by combining summaries and URLs into a format the prompt template expects. Here's how.

First, transform each dictionary into a string with the summary and its source URL:

```
stringified_summary_list = [f'Source URL: {sr["result_url"]}\nSum  
                             for sr in result_text_summary_list]
```

Inspecting `stringified_summary_list`, you'll find entries like these:

```
['Source URL: https://www.worldatlas.com/cities/astorga-spain.htm  
the Maragatería ... pilgrims on the Camino de Santiago. ', 'Source
```

Next, combine all the summary strings into one:

```
appended_result_summaries = '\n'.join(stringified_summary_list)
```

This gives you a single text block with all summaries and URLs:

```
Source URL: https://www.worldatlas.com/cities/astorga-spain.html  
Summary:  
Astorga is a municipality in Northwestern Spain with a population  
Source URL: https://citiesandattractions.com/spain/astorga-spain-  
Summary:  
Some of the main attractions in Astorga, Spain include the Episco
```

Now, use this content with the `RESEARCH_REPORT_INSTRUCTIONS` prompt template to get the final research report:

```
research_report_prompt = RESEARCH_REPORT_PROMPT_TEMPLATE.format(  
    research_summary=appended_result_summaries,  
    user_question=question  
)  
research_report = llm.invoke(research_report_prompt)  
  
print(f'strigified_summary_list={stringified_summary_list}')  
print(f'merged_result_summaries={appended_result_summaries}')  
print(f'research_report={research_report}')
```

If you run the entire `research_engine_seq.py` script, you'll receive a complete research report like the one below, based on web summaries after about a minute:

```
# Introduction
Astorga is a charming town located in the northwestern region of
# Historical and Cultural Significance
Astorga is a town with a long and rich history, dating back to th
[... SHORTENED]
```

Great job on completing your first automated web research! I recommend going through the code for this initial implementation again. If you didn't have time to type and run it, be sure to check it out in the GitHub repository. You'll likely understand it right away, but a review will help ensure everything is clear.

This initial version works well and generates the expected research report. However, it's a bit slow because it processes tasks sequentially. If we had set it up to handle 10 web searches with 10 results each, the time required would have significantly increased.

Can we make it faster? Absolutely. In the next section, we'll explore how to do that by introducing the LangChain Expression Language (LCEL).

4.7 Reimplementing the research summary engine in LCEL

The LangChain Expression Language (LCEL) offers a structured approach to organizing the core components of your LLM applications—such as web search, page scraping, and summarization—into an efficient chain or pipeline. This framework not only simplifies the creation of complex workflows from simple elements but also enhances them with advanced features like streaming, parallel execution, and logging. While you got an initial glimpse of LCEL in Chapter 4, this sidebar will provide a more in-depth look at its full capabilities.

LangChain Expression Language (LCEL)

For those developing LLM applications, using LCEL is highly recommended. It allows you to interact with LLMs and chat models efficiently by creating and executing chains, providing several benefits:

- **Fallback:** Enables adding a fallback action for error handling.
- **Parallel Execution:** Executes independent chain components simultaneously to boost performance.
- **Execution Modes:** Supports developing in synchronous mode and then switching to streaming, batch, or asynchronous execution modes as needed.
- **LangSmith Tracing:** Automatically logs execution steps when upgrading to LangSmith, facilitating debugging and monitoring.

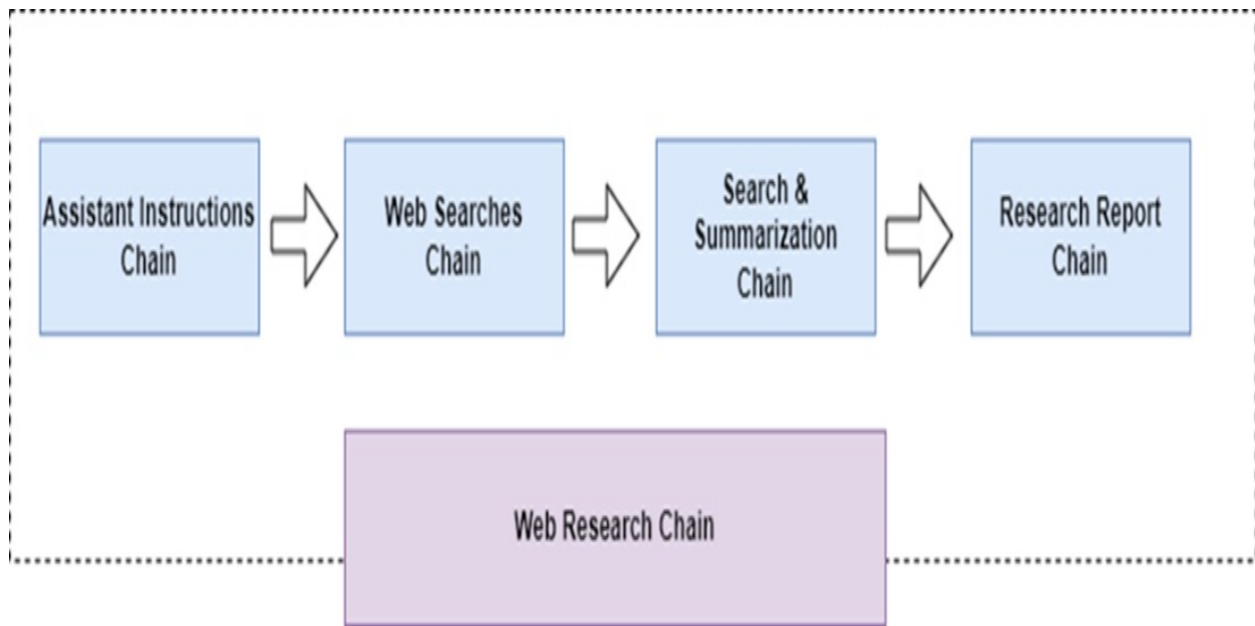
A chain follows the Runnable protocol, meaning it requires the implementation of specific methods like `invoke()`, `stream()`, and `batch()`, including their asynchronous versions. LangChain's framework ensures that its components, such as `PromptTemplate` and `JsonOutputFunctionsParser`, adhere to these standards.

For detailed information and examples, visiting the official LCEL documentation is recommended, especially the How To and Cookbook sections: https://python.langchain.com/docs/expression_language/

LCEL streamlines complex chain creation by offering a unified interface (the Runnable protocol), composition tools, and the ability to easily parallelize processes. Though mastering LCEL may require some practice, the effort is rewarding as it significantly enhances application performance and scalability.

My chain implementation strategy, shown in figure 4.6, involves constructing a mini-chain for each processing step seen earlier and integrating these into a master Web Research chain.

Figure 4.6 Architecture of the chain-based research summarization engine: Each step of the process is re-implemented as a mini-chain; all mini-chains are assembled into a master Web Research chain.



This master Web Research chain handles the entire process, as shown in figure 4.5, which illustrates the architecture of the chain-based research summarization engine. Each processing step is implemented as a mini-chain, all integrated into the master Web Research chain:

- Assistant Instructions chain: This chain selects the best research assistant from available options to answer the user's question. It also creates the system prompt that defines the assistant's skills and purpose.
- Web Searches chain: This chain generates multiple web searches based on the user's question. It provides context from different perspectives or breaks down complex queries into simpler ones.
- Search and Summarization chain: This chain performs web searches, retrieves URLs from search results, scrapes the relevant web pages, and summarizes the content of each page.
- Research Report chain: The final chain synthesizes the answer using the original question and the summaries generated from the search results.

The Search and Summarization chain, detailed in figure 4.7, is itself a composite of three chains: one for web search, one for scraping and summarizing web pages, and one for compiling summaries into a single text block.

Figure 4.7 The Search and Summarization chain is made up of three chains: a web search chain,

a scraping and summarization chain, and a summary compilation chain.

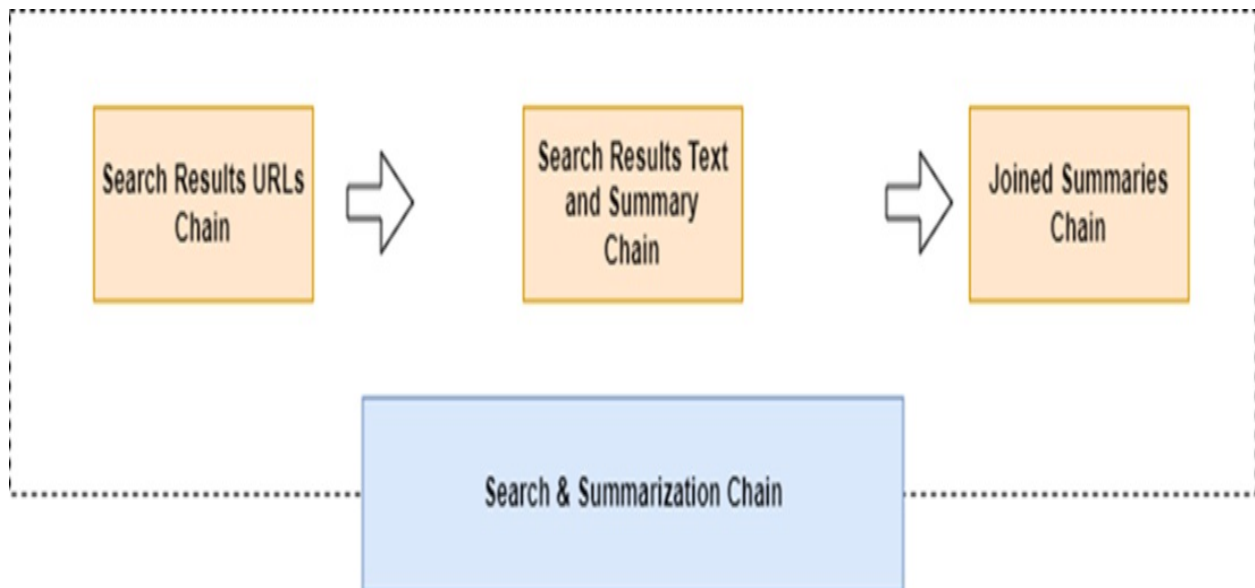


Figure 4.7 shows the Search and Summarization chain, which combines three separate chains: one for web searches, another for scraping and summarizing web content, and a third for merging these summaries into a single text block. Reimplementing this process into chains, as shown in Figure 4.8, allows for parallel execution, significantly improving efficiency compared to the sequential approach in Figure 4.6.

Figure 4.8 Chain architectural diagram showing how parallelization is applied in the research summarization engine, highlighting the execution of separate chain instances in parallel for efficiency.

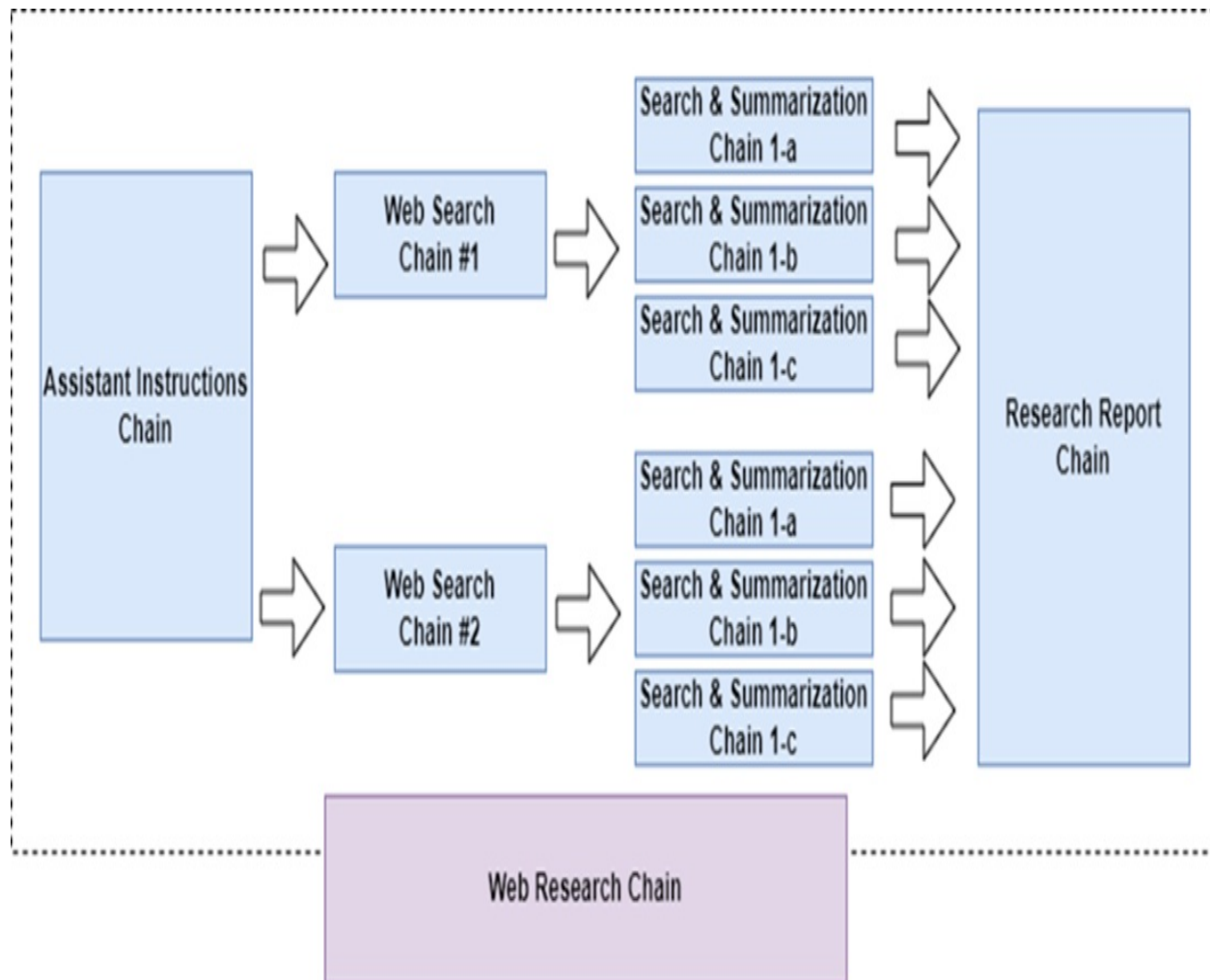


Figure 4.7 highlights parallelization at two key stages:

1. A separate instance of the Search and Summarization chain is created and executed simultaneously for each web search initiated by the Web Searches chain.
2. For each search result generated by the Search Result URLs chain, an individual Search Result Text and Summary chain instance is launched to run in parallel.

With this overview of the chain-based approach, we'll now explore the individual "mini chains," starting with the "Assistant Instructions" chain.

4.7.1 Assistant Instructions chain

To begin processing a research question, the first task is to determine the most relevant research assistant and their prompt instructions. This is handled by the `assistant_instructions_chain`, as shown below. Place this code in a file named `chain_1_1.py`:

```
from llm_models import get_llm
from prompts import (
    ASSISTANT_SELECTION_PROMPT_TEMPLATE,
)
from langchain.schema.output_parser import StrOutputParser

assistant_instructions_chain = (
    ASSISTANT_SELECTION_PROMPT_TEMPLATE | get_llm()
)
```

For those new to LCEL syntax, the flow here is straightforward: the selection prompt feeds into the LLM, which then selects a research assistant based on the user's question.

To test this setup, use the following script, saved as `chain_try_1_1.py`:

```
from chain_1_1 import assistant_instructions_chain

question = 'What can I see and do in the Spanish town of Astorga?'

assistant_instructions = assistant_instructions_chain.invoke(question)
print(assistant_instructions)
```

Running this code will produce output similar to what was shown in section 4.6.4, detailing the assistant type, instructions, and the user question (the output will come after a few seconds, and the metadata might look slightly different from what I have reported here):

```
content={'\n    "assistant_type": "Tour guide assistant",\n    "a
```

While you could manually extract the needed output from the `content` property, using LCEL offers a more efficient approach. By adding a `StrOutputParser()` block to your chain, you can automatically extract the LLM's response directly from the `content` property. Update the chain accordingly and save it as `chain_1_2.py`.

```
from llm_models import get_llm
```

```

from utilities import to_obj
from prompts import (
    ASSISTANT_SELECTION_PROMPT_TEMPLATE,
)
from langchain.schema.runnable import RunnablePassthrough
from langchain.schema.output_parser import StrOutputParser

assistant_instructions_chain = (
    {'user_question': RunnablePassthrough()}
    | ASSISTANT_SELECTION_PROMPT_TEMPLATE | get_llm() | StrOutput
)

```

This updated version introduces several enhancements:

- While the `ASSISTANT_SELECTION_PROMPT_TEMPLATE` automatically matches the input text question to the `{user_question}` field (refer back to section 4.5.1 for a refresh on `{user_question}`), it's safer to explicitly map the input question to a `user_question` property in a Python dictionary using the `RunnablePassthrough()` function, which binds the external input to the `user_question` variable through an unnamed parameter. This is also beneficial as this piece of information is important for subsequent steps.
- The `StrOutputParser()` block extracts text from the LLM's content property, simplifying output handling.
- The response is converted to a Python dictionary by `to_obj()`, making it easier to work with in the chain.

Test the revised chain with this script, saved as `chain_try_1_2.py`:

```

from chain_1_2 import assistant_instructions_chain
question = 'What can I see and do in the Spanish town of Astorga?'

assistant_instructions_dict = assistant_instructions_chain.invoke
print(assistant_instructions_dict)

```

Running this should produce the expected output:

```
{'assistant_type': 'Tour guide assistant', 'assistant_instruction
```

4.7.2 Web Searches chain

After the LLM selects the appropriate research assistant and details their role, you can prompt the LLM to generate web searches related to the user's query. Use the code below, saved as `chain_2_1.py`, for this step:

Listing 4.6 Web Searches chain for rewriting the user query into web searches

```
from llm_models import get_llm
from utilities import to_obj
from prompts import (
    WEB_SEARCH_PROMPT_TEMPLATE
)
from langchain.schema.output_parser import StrOutputParser
from langchain.schema.runnable import RunnableLambda

NUM_SEARCH_QUERIES = 2

web_searches_chain = (
    RunnableLambda(lambda x:
        {
            'assistant_instructions': x['assistant_instructions'],
            'num_search_queries': NUM_SEARCH_QUERIES,
            'user_question': x['user_question']
        }
    )
    | WEB_SEARCH_PROMPT_TEMPLATE | get_llm() | StrOutputParser()
)
```

This implementation uses a `RunnableLambda` block to process input from the previous chain, transforming it into the format needed by `WEB_SEARCH_PROMPT_TEMPLATE`. The chain then continues similarly to previous examples.

To test this chain, you can use the following script, saved as `chain_try_2_1`:

Listing 4.7 Script to test the Web Searches chain

```
from utilities import to_obj
from chain_2_1 import web_searches_chain

# test chain invocation
assistant_instruction_str = '{"assistant_type": "Tour guide assis'
assistant_instruction_dict = to_obj(assistant_instruction_str)
web_searches_list = web_searches_chain.invoke(assistant_instructi
print(web_searches_list)
```

This script uses a mock input in `assistant_instruction_str` to mimic the output from the Assistant Instructions chain, testing the chain's response to this input.

Expected output:

```
[{'search_query': 'Things to do in Astorga Spain', 'user_question': 'Things to do in Astorga Spain'}
```

With the web searches now generated, you can now proceed to the Search and Summarization chain.

4.7.3 Search and Summarization chain

The Search and Summarization chain is designed to perform a web search based on a query from the previous chain, retrieve URLs from the search results, scrape the corresponding web pages, and then summarize each page. As shown in Figure 4.5, this process is broken down into smaller sub-chains:

- Search result URLs chain
- Search result Text and Summary chain
- Joined summary chain

We'll begin by building these sub-chains, starting with the Search Result URLs chain.

Search result URLs chain

This sub-chain carries out a web search, retrieving a specific number of URLs from the search results. Save the following code as `chain_3_1.py`:

Listing 4.8 Search and Summarization chain

```
from web_searching import web_search
from langchain.schema.runnable import RunnableLambda

NUM_SEARCH_RESULTS_PER_QUERY = 3

search_result_urls_chain = (
    RunnableLambda(lambda x:
```



```

        [
            {
                'result_url': url,
                'search_query': x['search_query'],
                'user_question': x['user_question']
            }
            for url in web_search(web_query=x['search_query'],
                                num_results=NUM_SEARCH_RESULTS_)
        ]
    )
)

```

A couple of key points to highlight:

- A lambda function is used to pass the `web_query` parameter to the `web_search()` function. This function also formats the output as a list of dictionaries, each containing the resulting URL along with data from the previous chain, such as the search query and the original user question. These elements will be useful in subsequent stages.
- The lambda function receives its input from the output of the previous chain, specifically the Web Searches chain.

To test this sub-chain, we'll use simulated input that mirrors what would come from the Web Searches chain. Save the testing code in a file named `chain_try_3_1.py`:

Listing 4.9 Script to test the Search and Summarization chain

```

from utilities import to_obj
from chain_3_1 import search_result_urls_chain

# test chain invocation
web_search_str = '{"search_query": "Astorga Spain attractions", '
web_search_dict = to_obj(web_search_str)
result_urls_list = search_result_urls_chain.invoke(web_search_dict)
print(result_urls_list)

```

After executing it, you should see output similar to this:

```
[{'result_url': 'https://loveatfirstadventure.com/astorga-spain/']
```

Search Result Text and Summary chain

The Search Result Text and Summary sub-chain handles a URL from the previous sub-chain by:

- Scraping the webpage's text using the provided URL.
- Generating a summary of the scraped text.
- Incorporating the source URL into the summary.

These steps are effectively implemented in the following code, which should be saved as `chain_4_1.py`:

Listing 4.10 Search Result Text and Summary chain

```
from llm_models import get_llm
from web_scraping import web_scrape
from langchain.schema.output_parser import StrOutputParser
from langchain.schema.runnable import RunnableLambda, RunnablePar
from prompts import (
    SUMMARY_PROMPT_TEMPLATE
)

RESULT_TEXT_MAX_CHARACTERS = 10000

search_result_text_and_summary_chain = (
    RunnableLambda(lambda x:
        {
            'search_result_text': web_scrape(url=x['result_url'])
            'result_url': x['result_url'],
            'search_query': x['search_query'],
            'user_question': x['user_question']
        }
    )
    | RunnableParallel (
        {
            'text_summary': SUMMARY_PROMPT_TEMPLATE | get_llm() |
            'result_url': lambda x: x['result_url'],
            'user_question': lambda x: x['user_question']
        }
    )
    | RunnableLambda(lambda x:
        {
            'summary': f"Source Url: {x['result_url']}\nSummary:
            'user_question': x['user_question']
        }
    )
)
```

The code follows the conventions established in previous chains, and you might recall the purpose of `RunnableParallel` from Chapter 3. The `RunnableParallel` block allows for the simultaneous execution of an inner chain (next to `text_summary`) along with other operations (in this case, two lambda functions) that depend on the same input, which comes from the initial `RunnableLambda` block.

To execute this chain, save the following script as `chain_try_4_1.py`:

Listing 4.11 Script to test the Search Result Text and Summary chain

```
from utilities import to_obj
from chain_4_1 import search_result_text_and_summary_chain

# test chain invocation
result_url_str = '{"result_url": "https://citiesandattractions.co
result_url_dict = to_obj(result_url_str)

search_text_summary = search_result_text_and_summary_chain.invoke
print(search_text_summary)
```

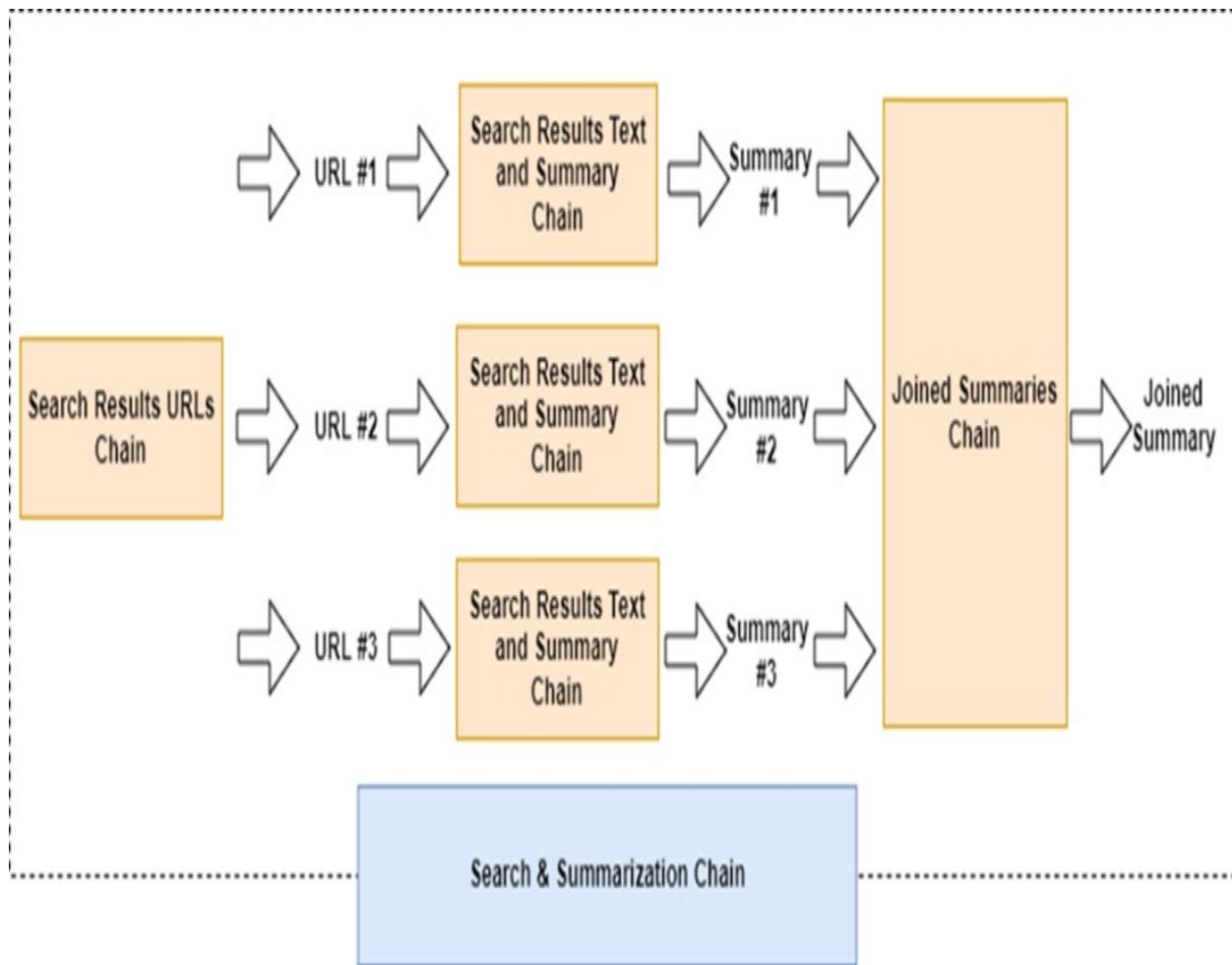
You will get output similar to this:

```
{'summary': 'Source Url: https://citiesandattractions.com/spain/a
```

Assembling the Search and Summarization chain

We've now assembled all the key components to build the Search and Summarization chain. Before diving into the LCEL implementation, take a moment to review Figure 4.9. This diagram provides a clearer understanding of the process compared to the earlier figure 4.7.

Figure 4.9 Enhanced Search and Summarization chain diagram: The Search Result URLs chain generates multiple URLs; each URL initiates an instance of the Result Text and Summary chain, which in turn produces a corresponding summary. These summaries are then consolidated into a single text string by the Joined Summary chain.



As illustrated in figure 4.9, the Search Result URLs chain generates multiple URLs, each of which triggers an instance of the Result Text and Summary chain. These instances run in parallel, each producing a summary for its respective web page. The Joined Summary chain then consolidates these summaries into a single text block.

To implement this process, place the necessary code in a file named `chain_5_1.py`:

Listing 4.12 Search and Summarization chain

```
from llm_models import get_llm
from prompts import (
    RESEARCH_REPORT_PROMPT_TEMPLATE
)
from chain_1_2 import assistant_instructions_chain
from chain_2_1 import web_searches_chain
```

```

from chain_3_1 import search_result_urls_chain
from chain_4_1 import search_result_text_and_summary_chain

from langchain.schema.output_parser import StrOutputParser
from langchain.schema.runnable import RunnableLambda

search_and_summarization_chain = (
    search_result_urls_chain
    | search_result_text_and_summary_chain.map() # parallelize fo
    | RunnableLambda(lambda x:
        {
            'summary': '\n'.join([i['summary'] for i in x]),
            'user_question': x[0]['user_question'] if len(x) > 0
        })
)

```

The `map()` operator triggers multiple instances of the Result Text and Summary chain, one for each dictionary from the Search Result URLs chain containing a URL. This allows each instance to run simultaneously.

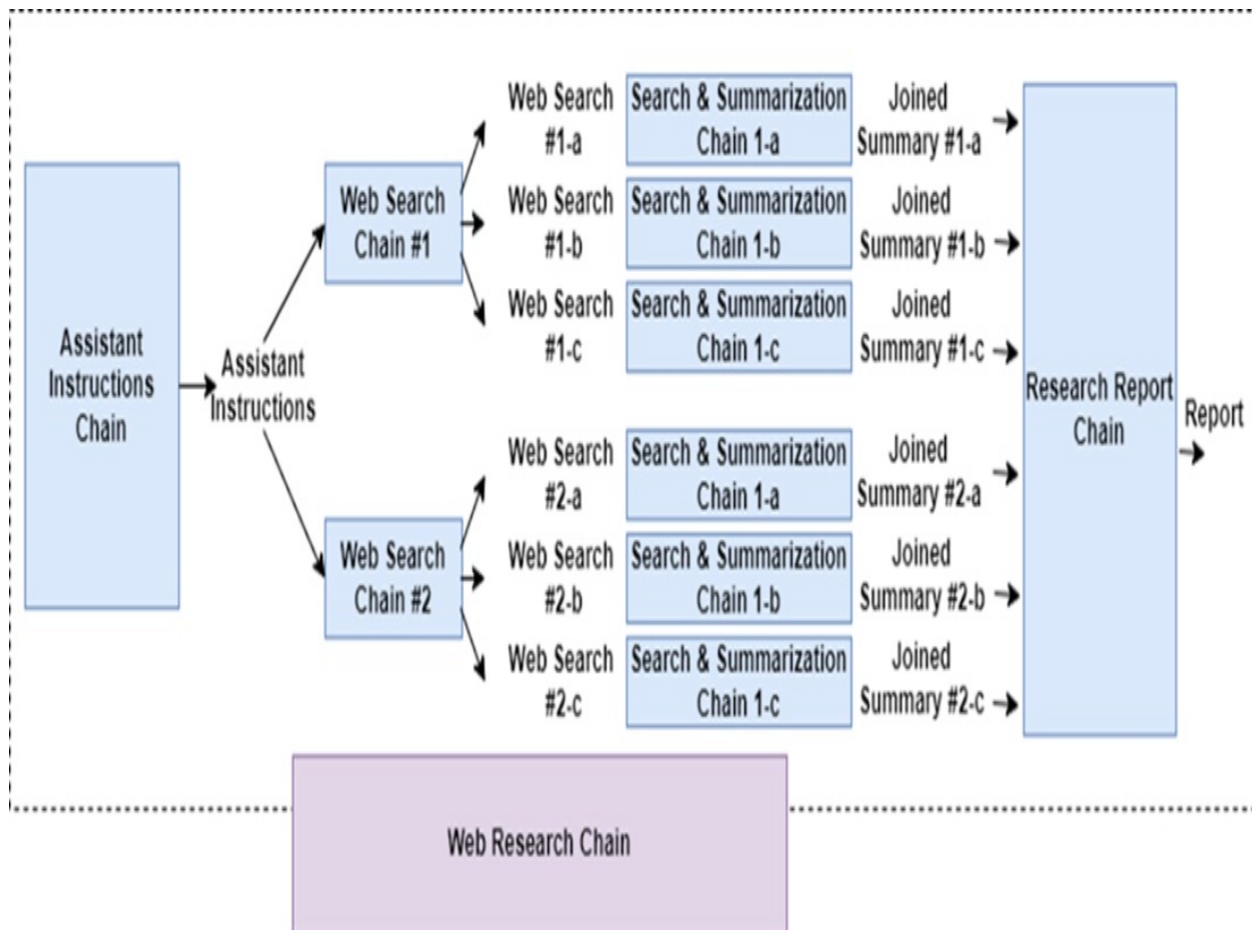
Additionally, the Joined Summaries sub-chain is integrated directly within the larger Search and Summarization chain rather than as a separate entity. This sub-chain merges summaries from each instance of the Result Text and Summary chain, functioning as a core part of the overall process.

With these components in place, you're ready to complete the Web Research chain.

4.7.4 Web Research chain

Before diving into the implementation, let's take a look at Figure 4.10 for a detailed overview of the Web Research chain, building on what we introduced in Figure 4.8, similar to our approach with the Search and Summarization chain.

Figure 4.10 Web Research chain: This illustrates how each web search initiated by the Web Searches chain triggers a separate instance of the Search and Summarization chain. These instances operate concurrently, each producing a summary. These summaries are then compiled by the Research Report chain into the comprehensive final research report.



As shown in Figure 4.10, web searches from the Web Searches chain trigger multiple instances of the Search and Summarization chains to run in parallel. Each chain generates a summary for a specific web search, and these summaries are then combined by the Research Report chain to create the final research report.

The LCEL implementation of this process is outlined below, to be added to the `chain_5_1.py` file:

Listing 4.13 Web Research chain

```
web_research_chain = (
    assistant_instructions_chain
    | web_searches_chain
    | search_and_summarization_chain.map() # parallelize for each
    | RunnableLambda(lambda x:
        {
            'research_summary': '\n\n'.join([i['summary'] for i in
```

```

        'user_question': x[0]['user_question'] if len(x) > 0 else
    })
    | RESEARCH_REPORT_PROMPT_TEMPLATE | get_llm() | StrOutputParser
)

```

This setup follows the same logic as the Search and Summarization chain. The `map()` operator is used here to initiate multiple instances of the Search and Summarization sub-chain, allowing them to run concurrently. The Research Report chain is then integrated as part of the overall Web Research chain.

To test the Web Research chain, use the following script, saving it as `chain_try_5_1.py`:

Listing 4.14 Script to test Web Research chain

```

from chain_5_1 import web_research_chain

# test chain invocation
question = 'What can I see and do in the Spanish town of Astorga?'

web_research_report = web_research_chain.invoke(question)
print(web_research_report)

```

After running this, you'll get a research report formatted in Markdown, as specified by the prompt. Note, the example report is shortened here for brevity:

```

# Introduction
Astorga, a small town in northwestern Spain, may not be on everyo
stunning architecture, delicious local cuisine, and natural beaut
along with practical information for planning a trip to this char

# History of Astorga
Astorga's history dates back to the ancient Roman settlement of A
Christians. It has also been the site of many violent campaigns,
# Top Attractions in Astorga
## Episcopal Palace
One of the most iconic ...

```

Congratulations! You've built an LCEL-based chain that integrates sub-chains, inline chains, lambda functions, and parallelization. This hands-on experience will make it easier for you to create your own LCEL chains. I

recommend taking some time to experiment with the different chains in this application. Adjust the prompts, change the number of queries generated, or tailor the application to a specific type of web research you have in mind.

Note

The research summarization engine you've built draws inspiration from an open-source project named GPT Researcher. Exploring its GitHub repository (<https://github.com/assafelovic/gpt-researcher>) will show you a robust platform supporting various search engines and LLMs, with features like memory and compression. While it doesn't use LCEL, adapting its core functionalities into an LCEL-based framework was intended to clarify several technical aspects for you. I recommend exploring the GPT Researcher's codebase and running the project on your computer if possible. It's an excellent way to learn about building an LLM application, including aspects like web UI integration, beyond just the engine itself.

4.8 Summary

- A research summarization engine is built around an LLM for efficient processing.
- The engine's workflow involves:
 - Receiving a research question from the user.
 - Generating web searches to gather relevant information with LLM assistance.
 - Summarizing the content from each search result using the LLM.
 - Combining these summaries into a detailed report.
- This engine can be developed using Visual Studio Code.
- Key components of the engine include web searching, web scraping, and LLM interaction.
- Prompt engineering is critical in three areas:
 - Crafting web searches from the user's question.
 - Summarizing search results.
 - Creating the final report.
- LangChain Expression Language (LCEL) helps organize the workflow into a main chain with secondary chains.
- Various techniques can parallelize the execution of these secondary

chains for efficiency.

5 Agentic Workflows with LangGraph

This chapter covers

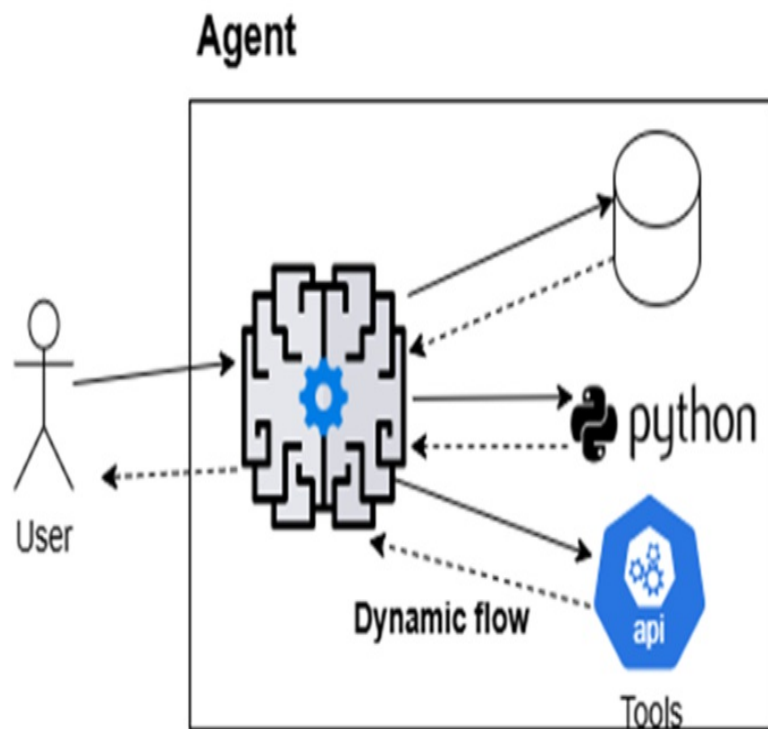
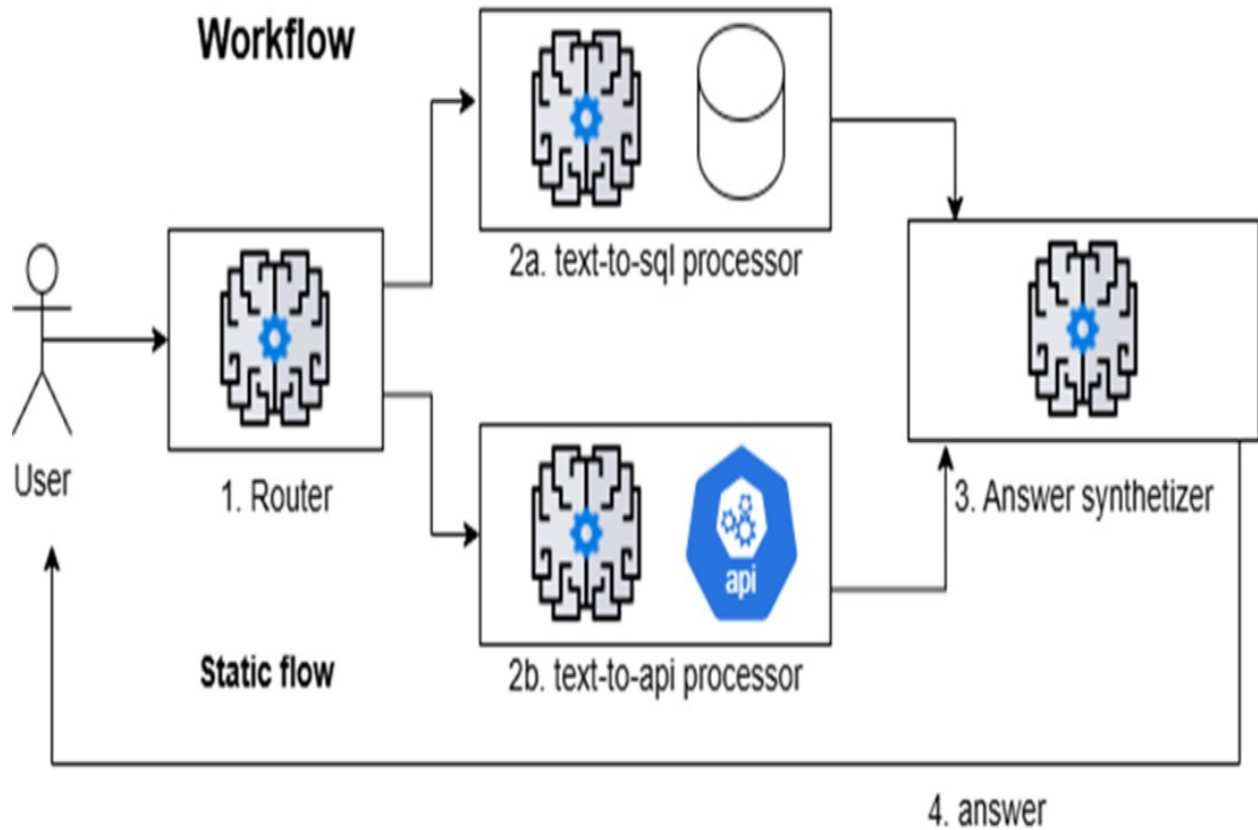
- Overview of agentic workflows and agents
- LangGraph fundamentals and state management
- Transition from LangChain chains to an agentic workflow

Large language models are driving a new generation of applications that require more than simple prompt-response exchanges. As applications become more complex, agentic workflows have become essential—a pattern where the LLM orchestrates a structured, multi-step process using predefined components and explicit state management. Unlike fully autonomous agents, agentic workflows follow a predictable sequence of steps and do not dynamically select tools or adapt to context in real time. Instead, they offer reliability, transparency, and modularity by guiding the application through a set flow, using the LLM to make decisions within fixed boundaries.

5.1 Understanding Agentic Workflows and Agents

LLM-powered agent-based systems typically follow one of two core design patterns: agentic workflows and agents. Each pattern shapes how the application operates, as illustrated in figure 5.1. Because these terms are often used interchangeably—but have important differences—it’s essential to understand exactly what is meant by “agentic workflow” and “agent” before diving deeper..

Figure 5.1 Workflows and agents: workflows use the LLM to choose the next step from a fixed set of options, such as routing a request to a SQL database or a REST API and synthesizing the answer with the related results. Agents, however, dynamically select and combine tools to achieve their objectives.



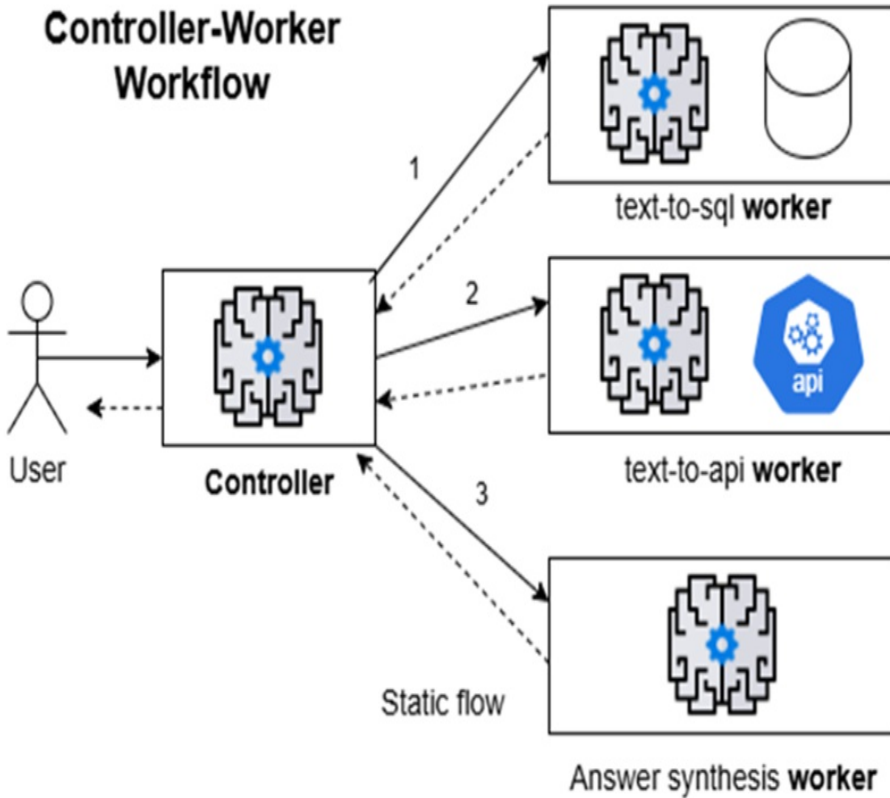
Agentic workflows—often called simply *workflows*—guide an application through a fixed sequence of predetermined steps. The LLM is used to select among predefined options, helping the system complete tasks and manage the overall flow. In contrast, *agents* use language models for more than just task execution: they reason, make decisions, and dynamically determine the next steps based on available tools and evolving context. Here, a "tool" typically refers to a function that returns or processes data. While both patterns rely on the LLM to drive application behavior, workflows maintain a structured and predictable path, whereas agents can adapt in real time based on new information and shifting goals.

5.1.1 Workflows

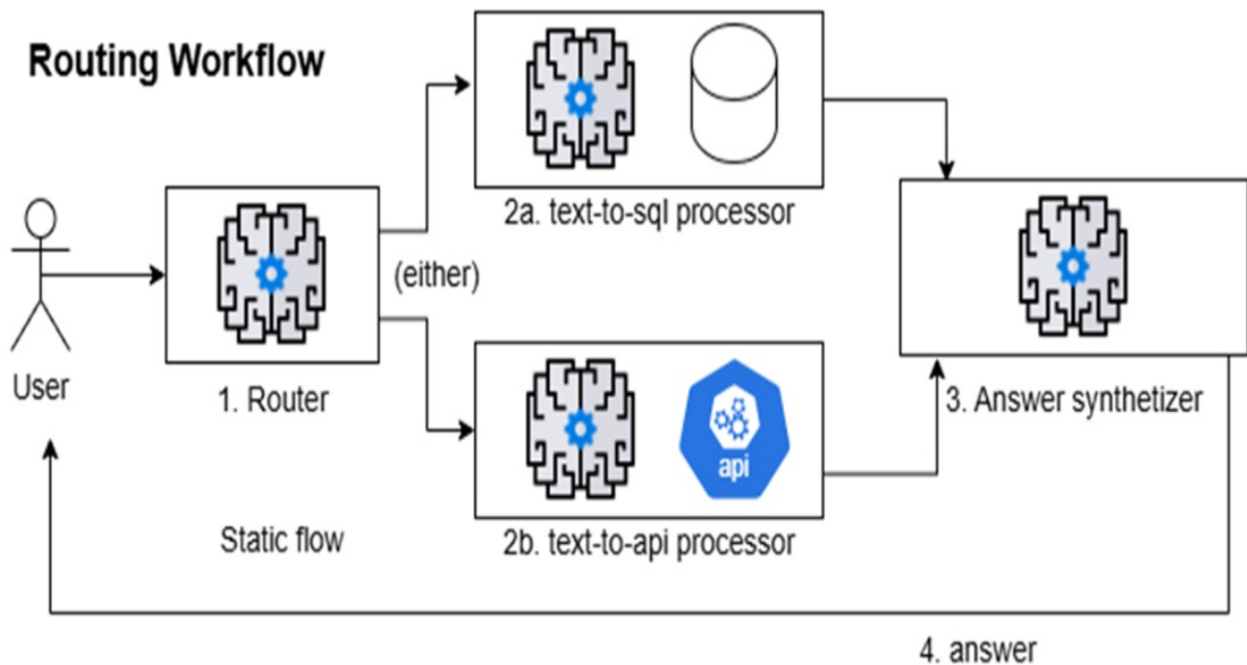
Workflows use the LLM to pick the next step from a limited set of choices. They typically implement patterns such as Controller-Worker or Router, as illustrated in figure 5.2.

Figure 5.2 Common Workflows patterns: the Controller-Worker pattern uses the LLM in the controller orchestrates the flow by assigning various tasks to workers following a certain sequence. In the Router pattern, the LLM simply directs the task to the appropriate worker based on the context.

Controller-Worker Workflow



Routing Workflow



In the Controller-Worker pattern, the controller spawns tasks for workers following a certain sequence. In the Router pattern, the LLM simply directs the task to the proper processor (or worker).

5.1.2 Agents

LLM agents use language models to perceive data, reason about it, decide on actions, and achieve goals. Advanced agents learn from feedback, retain memory of past interactions, and build dynamic workflows with branching logic. Unlike fixed prompt-response systems, agents generate new flows based on real-time data and available tools.

5.1.3 When to Use Agent-Based Architectures

The concepts of LLM-based workflows and agents are closely related and often overlap, with no sharp dividing line between them. In this chapter, our focus is on agentic workflows; we'll explore agents in greater depth later in the book. Both approaches are most valuable when your application needs to break complex tasks into smaller steps, make decisions based on previous results, access external tools or data, or maintain context throughout extended interactions. It's best to adopt agentic workflows or agents when your use case genuinely benefits from explicit state management and dynamic control—balancing the added power against the increased complexity.

TIP

I highly recommend Anthropic's article "Building Effective AI Agents" for a deeper understanding of workflows, agents, and when to use them. You can read it here: <https://www.anthropic.com/engineering/building-effective-agents>

5.1.4 Agent Development Frameworks

A variety of frameworks are available for building agent-based systems, each with its own focus and trade-offs. *LangGraph* is designed for stateful, persistent agentic workflows using graph-based execution, making it

powerful for complex applications—though it may require more technical expertise. *AutoGPT* emphasizes fully autonomous, goal-driven agents with minimal supervision, but can face challenges with task consistency. *LlamaIndex* stands out in knowledge retrieval, though its scope is narrower than broader agent frameworks. *Microsoft Autogen* supports highly customizable multi-agent conversations, but comes with a steeper learning curve. *n8n* provides a visual interface and extensive integrations, making it accessible to non-developers, though advanced reasoning may require additional components. *Microsoft Semantic Kernel* prioritizes memory and planning and integrates well with Azure services. *CrewAI* enables collaborative, multi-agent systems for specialized teams, but is a newer tool with a smaller community.

5.2 LangGraph Basics

LangGraph builds on LangChain to manage more complex agentic workflows with branching paths, stateful processing, and clear transitions between steps. It's a framework for building stateful, multi-step AI applications using a graph-based structure. In LangGraph, nodes represent individual tasks, such as generating text, calling an API, or analyzing data. Edges define the paths that connect these tasks. The state is information that moves between nodes and updates at each step.

This setup is better than traditional chains when you need to make decisions, manage state, or handle complex agentic workflows.

Note

LangGraph isn't a replacement for LangChain but an extension. Think of LangChain as providing the building blocks and LangGraph as offering a blueprint to connect those parts into a complex system. LangChain gives you components like LLMs, embeddings, and retrievers, while LangGraph helps you organize those components into a structured, stateful workflow.

To use LangGraph effectively, it's important to understand a few key concepts—like the core components of a graph (nodes and edges), how state flows through the graph, and how conditional edges control its behavior.

These are the building blocks you'll explore in the next section.

5.3 Moving from LangChain Chains to LangGraph

LangChain's simple, linear chains work for straightforward tasks but have limits when your applications get more complex. A typical LangChain setup often looks like this:

```
chain = (  
    prompt_template  
    | llm  
    | output_parser  
)
```

This setup struggles when tasks need to split into different paths, when you need to repeat steps based on new information, or when you want to manage state across multiple steps. It also falls short when multiple processes need to happen in parallel.

LangGraph solves these problems by offering better state management, conditional branching, and support for cyclical workflows. With explicit state management, you can define and track data consistently across the workflow, which is essential for memory and reasoning. Conditional branching allows agents to take different paths based on previous results, making decision-making smoother. Cyclical workflows let agents repeat tasks until they meet specific conditions, which helps refine results.

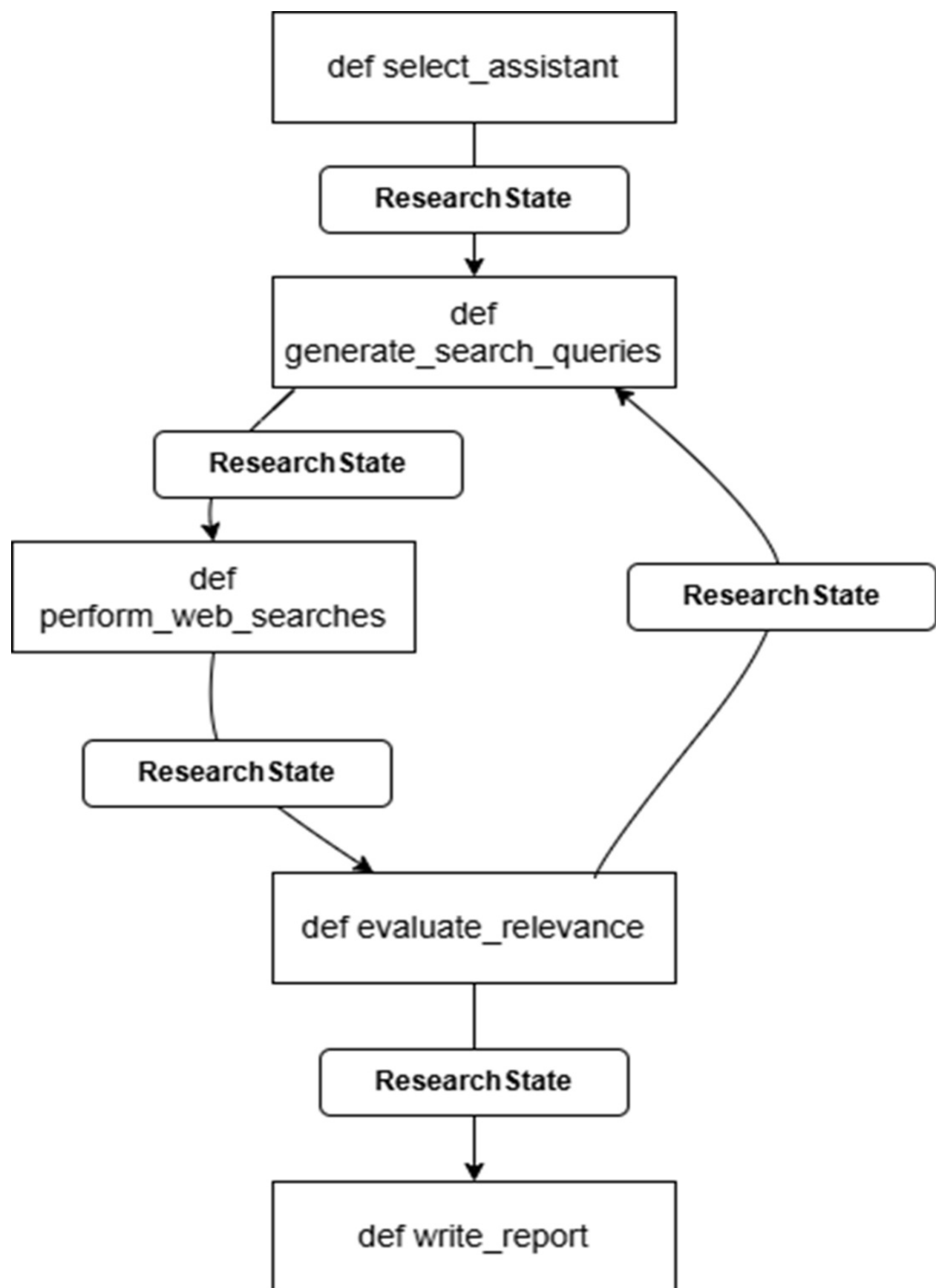
LangGraph also makes it easier to understand and debug complex workflows. Its graph-based structure offers a clearer view of how data flows through the system, which helps when you need to trace or fix problems.

This graph-based approach works well for a range of use cases, such as multi-step reasoning, task planning, managing context in long conversations, coordinating research tasks, and automating business processes. As your applications get more complex, the benefits of using LangGraph's agent-based architecture become clearer. It gives you the control and flexibility needed to build smart, multi-step systems that can adapt and make decisions on their own.

5.4 LangGraph Core Components

LangGraph provides a robust framework for building stateful, multi-step AI applications. Figure 5.3 presents the core components that form LangGraph application's foundation.

Figure 5.3 LangGraph core Components: a strongly typed state (in this example, modelled with `ResearchState`) flows through the workflow. Nodes, usually Python functions (like `def select_assistant`), perform tasks, and edges create directed data flows between nodes, in some cases with conditional paths.



At the heart of every LangGraph application is a state object — in our example, `ResearchState` — which defines a clear and strongly typed state for the entire workflow. This state is typically defined as a Python `TypedDict`, ensuring that data passed between components is well-structured and type-checked.

In a LangGraph, each *node* functions as a processing unit. Nodes can handle tasks such as generating search queries, calling external APIs, summarizing results, or transforming data. These nodes are usually implemented as Python functions. The *edges* between nodes determine the directed flow of data, defining how information moves through the graph.

One of LangGraph's powerful features is its *conditional edges*, which allow you to define dynamic execution paths based on the runtime state. Combined with entry points and end conditions, this gives you full control over where the graph begins, how it progresses, and when it completes.

The following sections walk through how to define and connect these components, enabling you to build systems that can handle complex workflows and adaptive decision-making.

5.4.1 StateGraph Structure

A core utility in LangGraph is the `StateGraph` class, which you use to define the graph that models your application's workflow. For example:

```
from langgraph.graph import StateGraph
from typing import TypedDict

class ResearchState(TypedDict): #A
    input_query: str
    intermediate_result: str
    final_output: str

graph = StateGraph(ResearchState) #B
```

5.4.2 State Management and Typing

State management is central to LangGraph applications. Unlike chain-based methods that rely on implicit or loosely typed state, LangGraph enforces explicit, strongly typed state, making workflows more robust and predictable.

Here's an extended version of the ResearchState that adds more detail:

```
from typing import TypedDict, Optional, List

class ResearchState(TypedDict):
    user_question: str
    assistant_info: Optional[dict]
    search_queries: Optional[List[dict]]
    search_results: Optional[List[dict]]
    research_summary: Optional[str]
    final_report: Optional[str]
```

Each node receives the current state and returns updates that merge into the overall state:

```
def process_node(state: dict) -> dict:

    result = do_something(state["input_data"]) #A

    return {"output_data": result} #B
```

5.4.3 Node Functions and Edge Definitions

Nodes represent processing steps. Each node is a function that takes the current state and returns updates. For instance:

```
def generate_search_queries(state: dict) -> dict:
    """Generate search queries based on user question."""
    question = state["user_question"]
    # Generate queries using an LLM
    queries = llm_generate_queries(question)
    return {"search_queries": queries}

graph.add_node("generate_queries", generate_search_queries) #A
```

Edges define valid transitions between nodes. A simple linear edge looks like this:

```
graph.add_edge("generate_queries", "perform_searches")
```

A conditional edge uses a function to choose the next node based on the state:

```
def should_refine_queries(state: dict) -> str:
    if len(state["search_results"]) < 2:
        return "refine_queries"
    else:
        return "summarize_results"
```

```
graph.add_conditional_edge("perform_searches", should_refine_quer
```

5.4.4 Entry Points and End Conditions

Every graph needs a starting point and clear end conditions. For example:

```
graph.set_entry_point("parse_question") #A

from langgraph.graph import END
graph.add_edge("write_final_report", END) #B
```

Let's explore a practical application of LangGraph.

5.5 Turning the Web Research Assistant into an AI Agent

To demonstrate how LangGraph works, I'll show you how to transform the web research assistant from chapter 4—originally built with LangChain—into an agent-based system. This upgrade allows the application to assess the relevance of summaries from web page results and, if less than 50% of them are relevant, redirect the flow back to generating new search queries. If enough summaries are relevant, the application can proceed to write the final report as usual. Achieving this level of dynamic control would be very complex with plain LangChain, which justifies the move to an agent-based approach with LangGraph. This case study guides you through each step, highlighting the benefits of explicit state management and modular design.

5.5.1 Original LangChain Implementation Overview

Our original web research assistant used LangChain's sequential chains. The process followed these steps:

1. Choose the appropriate research assistant based on the user's question.
2. Generate search queries.
3. Perform web searches and collect URLs.
4. Scrape and summarize each search result.
5. Compile a final research report.

Each step fed its output into the next step, as you can see in the extract from the original implementation in listing 5.1:

Listing 5.1 Original LangChain implementation of the Web Research Assistant

```
assistant_instructions_chain = (  
    {'user_question': RunnablePassthrough()}  
    | ASSISTANT_SELECTION_PROMPT_TEMPLATE  
    | get_llm()  
    | StrOutputParser()  
    | to_obj  
)  
  
web_searches_chain = (  
    # ...input processing...  
    | WEB_SEARCH_PROMPT_TEMPLATE  
    | get_llm()  
    | StrOutputParser()  
    | to_obj  
)  
  
web_research_chain = ( #A  
    assistant_instructions_chain  
    | web_searches_chain  
    | search_and_summarization_chain.map()  
    | RunnableLambda(lambda x: # ...process results...)  
    | RESEARCH_REPORT_PROMPT_TEMPLATE  
    | get_llm()  
    | StrOutputParser()  
)
```

This approach works but has clear limitations:

- The flow is rigid and linear, making it difficult to adapt dynamically based on intermediate results. For instance, a conditional flow that redirects the application to generate new search queries if less than 50% of the summaries are relevant would be cumbersome to implement.

- Error handling is challenging, as the lack of explicit state makes it hard to track and manage failures effectively.
- State is not explicitly managed, which complicates maintaining context across multiple steps.
- Debugging becomes difficult when issues arise, especially with complex flows, since it is unclear which part of the chain failed or why.

5.5.2 Identifying Components for Conversion

To convert the web research assistant to LangGraph, I first identify the key components that will serve as nodes. Each node handles a specific part of the process:

- Assistant Selector: Determines which type of research assistant to use based on the user's question.
- Query Generator: Creates search queries derived from the user's input.
- Web Searcher: Conducts searches and gathers URLs based on the generated queries.
- Content Summarizer: Scrapes and summarizes the content of web pages.
- Relevance Evaluator: Assesses if the summaries are relevant enough to proceed or if new search queries are needed.
- Report Writer: Compiles the final research report using the relevant summaries.

Unlike a simple linear flow, this setup introduces a conditional element. After evaluating the relevance of the summaries, the flow can either proceed to the Report Writer if enough content is relevant or redirect back to the Query Generator to create new search queries. This decision is based on a defined threshold (for example, if less than 50% of summaries are relevant) and can repeat up to a maximum of three iterations to avoid infinite loops.

The flow control is managed by a conditional routing function, the `route_based_on_relevance`, which checks the relevance of the search results and the current iteration count. If the relevance is insufficient and the maximum number of iterations has not been reached, the application generates new queries and repeats the search and evaluation steps. If the maximum iteration count is reached, the application proceeds to compile a

report using the available results, regardless of their relevance.

For each component, I define:

- The input state: The data each node requires to function.
- The processing: The tasks each node performs.
- The state updates: The information each node returns to update the overall state.

This modular and conditional approach makes the system flexible and adaptive, which would be cumbersome to achieve with plain LangChain's linear chains.

5.5.3 Step-by-Step Transformation Process

Now, I'll guide you through the process of converting a LangChain application to LangGraph. The following is a simplified version of the actual code, which you can find in the GitHub repository.

Step 1: Define the State

The first step is to design the state structure that will flow through the graph. A well-defined state helps you keep track of data across all nodes. In our case we'll model a composite state using inner types, as shown in listing 5.2.

Listing 5.2 State type of the LangGraph based Research assistant

```
from typing import TypedDict, List, Optional

class AssistantInfo(TypedDict): #A
    assistant_type: str
    assistant_instructions: str
    user_question: str

class SearchQuery(TypedDict): #A
    search_query: str
    user_question: str

class SearchResult(TypedDict): #A
    result_url: str
```



```

        search_query: str
        user_question: str
        is_fallback: Optional[bool]

class SearchSummary(TypedDict): #A
    summary: str
    result_url: str
    user_question: str
    is_fallback: Optional[bool]

class ResearchReport(TypedDict): #A
    report: str

class ResearchState(TypedDict): #B
    user_question: str
    assistant_info: Optional[AssistantInfo]
    search_queries: Optional[List[SearchQuery]]
    search_results: Optional[List[SearchResult]]
    search_summaries: Optional[List[SearchSummary]]
    research_summary: Optional[str]
    final_report: Optional[str]
    used_fallback_search: Optional[bool]
    relevance_evaluation: Optional[Dict[str, Any]]
    should_regenerate_queries: Optional[bool]
    iteration_count: Optional[int]

```

This state structure clearly defines the data available at each stage, reducing ambiguity and simplifying debugging.

Step 2: Convert Components to Node Functions

Next, I convert each component into a node function. Each function takes the current state, processes it, and returns updated state information, as you can see in listing 5.3.

Listing 5.3 Node functions

```

def select_assistant(state: dict) -> dict:
    """Select the appropriate research assistant."""
    user_question = state["user_question"]

    # Use the LLM to select an assistant
    prompt = ASSISTANT_SELECTION_PROMPT_TEMPLATE.format(
        user_question=user_question
    )

```

```

    )
    response = get_llm().invoke(prompt)

    assistant_info = parse_assistant_info(response.content) #A

    return {"assistant_info": assistant_info} #B

def generate_search_queries(state: dict) -> dict:
    """Generate search queries based on the question."""
    assistant_info = state["assistant_info"]
    user_question = state["user_question"]

    prompt = WEB_SEARCH_PROMPT_TEMPLATE.format( #C
        assistant_instructions=assistant_info["assistant_instruct
        user_question=user_question,
        num_search_queries=3
    )
    response = get_llm().invoke(prompt)

    search_queries = parse_search_queries(response.content) #D

    return {"search_queries": search_queries} #E

```

Additional node functions follow the same pattern, each handling its own task.

Step 3: Define the Graph Structure

With node functions in place, I create the graph and define how the nodes connect, establishing the execution order and data flow, as shown in Listing 5.4. Unlike a simple linear chain, this version of the graph introduces a new node for relevance evaluation and a conditional edge that dynamically alters the flow based on the relevance of the search results.

Listing 5.4 Graph structure

```

from langgraph.graph import StateGraph, END

graph = StateGraph(ResearchState) #A

graph.add_node("select_assistant", select_assistant) #B
graph.add_node("generate_search_queries", generate_search_queries)
graph.add_node("perform_web_searches", perform_web_searches) #B

```

```

graph.add_node("summarize_search_results", summarize_search_resul
graph.add_node("evaluate_search_relevance", evaluate_search_relev
graph.add_node("write_research_report", write_research_report) #

def route_based_on_relevance(state): #C
    iteration_count = state.get("iteration_count", 0) + 1
    state["iteration_count"] = iteration_count

    if iteration_count >= 3:
        return "write_research_report"
    if state.get("should_regenerate_queries", False):
        return "generate_search_queries"
    return "write_research_report"

graph.add_edge("select_assistant", "generate_search_queries") #D
graph.add_edge("generate_search_queries", "perform_web_searches")
graph.add_edge("perform_web_searches", "summarize_search_results"
graph.add_edge("summarize_search_results", "evaluate_search_relev
graph.add_edge("write_research_report", END) #D

graph.add_conditional_edges( #E
    "evaluate_search_relevance",
    route_based_on_relevance,
    {
        "generate_search_queries": "generate_search_queries",
        "write_research_report": "write_research_report"
    }
)

graph.set_entry_point("select_assistant") #F

```

The new relevance evaluation node checks if enough of the summarized results are relevant. If less than 50% of them meet the criteria, the graph redirects the flow back to the Query Generator to refine the search. If the summaries are sufficient or if the maximum of three iterations is reached, it moves forward to compile the final report. This conditional flow is a significant enhancement over the rigid linear chains of LangChain, allowing the system to adapt dynamically based on intermediate results.

Step 4: Compile and Run the Graph

After defining the graph, I compile it and run it using an initial state, as

shown in listing 5.5. This step involves setting up an initial state with all required fields, including additional parameters for controlling the conditional flow, such as `should_regenerate_queries` and `iteration_count`.

Listing 5.5 Running the graph

```
app = graph.compile() #A

initial_state = { #B
    "user_question": " What can you tell me about Astorga's roman
    "assistant_info": None,
    "search_queries": None,
    "search_results": None,
    "search_summaries": None,
    "research_summary": None,
    "final_report": None,
    "used_fallback_search": False,
    "relevance_evaluation": None,
    "should_regenerate_queries": None,
    "iteration_count": 0
}

result = app.invoke(initial_state) #C

final_report = result["final_report"] #D
```

By introducing conditional edges and relevance evaluation, this step-by-step process transforms a rigid, linear chain into a flexible, stateful, and adaptive agent-based workflow. The system can now evaluate its own results, adapt by refining search queries if needed, and ensure that the final report is based on sufficiently relevant information. This adaptability would be cumbersome to implement in plain LangChain, justifying the shift to LangGraph for complex LLM applications.

5.5.4 Code Comparison and Benefits Realized

The LangGraph approach offers significant benefits:

- **Explicit State Management:** The state is clearly defined and passed through each node, making data handling transparent and reliable.

- **Modular Components:** Each node handles a single task, simplifying testing, debugging, and maintenance.
- **Clear Flow Control:** The graph structure visually represents the execution order and data flow, making it easier to trace and understand complex processes.
- **Easier Debugging:** With well-defined nodes and edges, it's straightforward to identify where errors occur and what data caused them.
- **Enhanced Error Handling:** Each node can implement specific error-handling strategies without affecting the rest of the system.
- **Conditional Flow Control:** The introduction of conditional edges based on relevance evaluation allows the application to dynamically alter its path—either refining search queries if results are insufficient or proceeding to report writing. This adaptability ensures that the application can respond intelligently to intermediate results, which would be cumbersome to implement in plain LangChain.
- **Future Extensibility:** Adding or modifying nodes requires minimal changes to the overall system, allowing for smooth upgrades and new capabilities.

This case study demonstrates how LangGraph enhances flexibility, control, and adaptability in complex, multi-step AI applications compared to traditional LangChain chains. The ability to implement conditional flows based on runtime evaluations makes LangGraph a powerful choice for building smart, context-aware agent-based systems..

5.6 Summary

- Agentic workflows follow fixed, predictable steps, while agents use LLMs for reasoning, dynamic tool selection, and real-time adaptation.
- Moving from basic LangChain apps to agentic workflows with LangGraph gives greater control and flexibility for complex tasks.
- LangGraph extends LangChain with stateful, multi-step workflows organized as nodes, edges, and explicit state updates.
- Replacing linear chains with LangGraph enables explicit state tracking and conditional branching, simplifying debugging and error handling.
- Conditional flows based on runtime evaluations let applications adapt,

such as re-running searches when results are insufficient.

- Core components include the StateGraph, node functions, and well-defined edges and entry points for maintainable, extensible designs.
- The case study shows converting a web research assistant from a rigid chain to a flexible agent-based system with modular design and clear state management.
- LangGraph boosts control, modularity, and future scalability, enabling complex decision-making and better overall performance.

6 RAG fundamentals with Chroma DB

This chapter covers

- Implementing semantic search using the RAG architecture
- Understanding vector stores and their functionality
- Implementing RAG with Chroma DB and OpenAI

In this chapter, you'll dive into two essential concepts: *semantic search* and *Retrieval Augmented Generation (RAG)*. You'll explore how large language models (LLMs) are used for semantic search through a chatbot, enabling you to query a system for information across multiple documents and retrieve the fragments that best match the meaning of your question, rather than just matching keywords. This approach is also known as Q&A over documents or querying a knowledge base.

In earlier chapters, you learned about summarization, a typical use case for large language models. Now, I'll walk you through the basics of building a Q&A chatbot that searches across multiple documents. You'll interact with the LLM to find the answers you're looking for.

This chapter focuses on RAG, the design pattern that powers semantic search systems, with a particular emphasis on the vector store—a key component of these systems. You'll learn the technical terminology related to Q&A and RAG systems and understand how terms like "semantic search" and "Q&A" are often used interchangeably.

By the end of this chapter, you'll have implemented a basic RAG-based architecture using the APIs of an LLM (OpenAI) and a vector store (Chroma DB). In chapter 6, you'll build on this foundation to create Q&A chatbots using RAG architecture. There's a lot to cover, so let's get started.

6.1 Semantic Search

Semantic search is a popular use case for LLMs, alongside summarization and code generation. It's one of the key applications driving the LLM and Generative AI boom.

Definition

Semantic search means searching for information by focusing on its meaning. This involves understanding a query's meaning, retrieving relevant document fragments from a document store that closely match the query's meaning, and optionally generating a natural language answer.

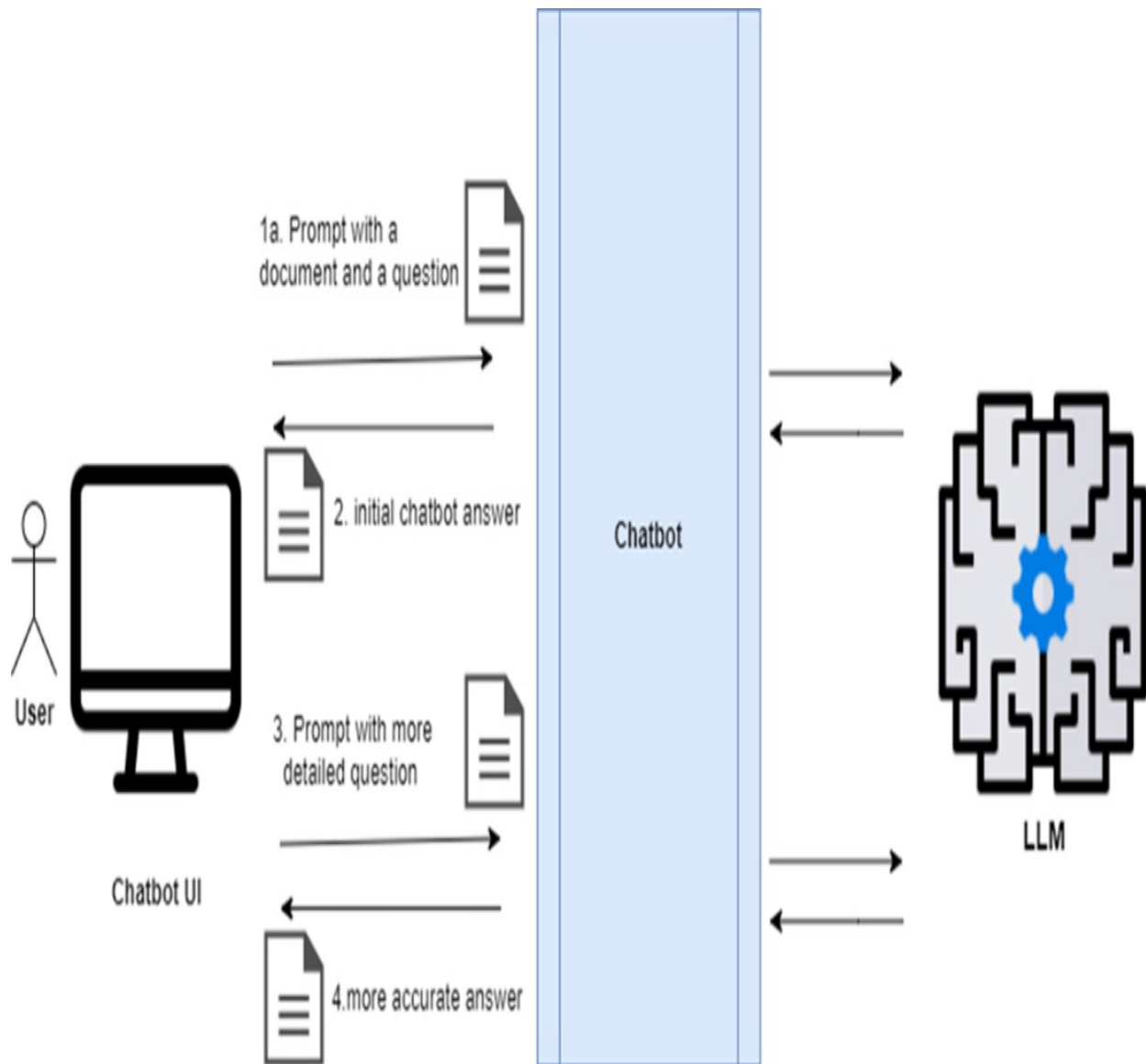
Semantic search differs from traditional keyword-based searches, which fail to find information if exact words don't match. Semantic search produces relevant results even if the query and result don't share a single word.

Before diving into the code, I want to give you a clear understanding of a semantic search chatbot's architecture. I'll start with a simple example to ease you in, but by the end of this section, you'll grasp the architecture of a real-world Q&A chatbot.

6.1.1 A Basic Q&A Chatbot Over a Single Document

I'll start with a simple scenario to help you understand how a Q&A chatbot works and to familiarize you with its components. The first chatbot example answers questions about a single document, as shown in figure 6.1.

Figure 6.1 A simple Q&A chatbot process: 1) The user sends a prompt containing a document (context) and a question to the chatbot; 2) The chatbot returns an initial answer; 3) The user follows up with a more detailed question; 4) The chatbot provides a more accurate answer.



The main elements of this basic setup are:

- **Document:** Contains the text for semantic search or information extraction.
- **Prompt:** Encapsulates the user's question (semantic search) and the context (the document) with the information needed for the answer.
- **LLM-based Chatbot:** Sends the prompt to the LLM, which understands the question and context, selects relevant information, and formulates an answer for the user.

Let's break down these concepts further.

Definition

Context is the text or information in the prompt, along with the user's question, used to formulate an answer.

Definition

Synthesize means to generate an answer from the question and context provided.

You don't need to write any code to implement this initial setup. Just log into ChatGPT or an alternative LLM-based chatbot like Gemini or Claude, and you're ready to go. Let's try a simple Q&A interaction using text about Paestum from britannica.com. Submit this prompt to ChatGPT (you can find this prompt in a text file of the Github repository):

Read the following text and let me know how many temples are in Paestum, who constructed them, and what architectural style they are:

Paestum, Greek Poseidonia, ancient city in southern Italy near the west coast, 22 miles (35 km) southeast of modern Salerno and 5 miles (8 km) south of the Sele (ancient Silarus) River. Paestum is noted for its splendidly preserved Greek temples.

Visit the ruins of the ancient Greek colony of Paestum and discover its history, culture, and society. See all videos for this article

Poseidonia was probably founded about 600 BC by Greek colonists from Sybaris, along the Gulf of Taranto, and it had become a flourishing town by 540, judging from its temples. After many years' resistance the city came under the domination of the Lucanians (an indigenous Italic people) sometime before 400 BC, after which its name was changed to Paestum. Alexander, the king of Epirus, defeated the Lucanians at Paestum about 332 BC, but the city remained Lucanian until 273, when it came under Roman rule and a Latin colony was founded there. The city supported Rome during the Second Punic War. The locality was still prosperous during the early

years of the Roman Empire, but the gradual silting up of the mouth of the Silarus River eventually created a malarial swamp, and Paestum was finally deserted after being sacked by Muslim raiders in AD 871. The abandoned site's remains were rediscovered in the 18th century.

Temple ruins of columns and statues at Karnak, Egypt (Egyptian architecture; Egyptian archaeology; Egyptian history)

The ancient Greek part of Paestum consists of two sacred areas containing three Doric temples in a remarkable state of preservation. During the ensuing Roman period a typical forum and town layout grew up between the two ancient Greek sanctuaries. Of the three temples, the Temple of Athena (the so-called Temple of Ceres) and the Temple of Hera I (the so-called Basilica) date from the 6th century BC, while the Temple of Hera II (the so-called Temple of Neptune) was probably built about 460 BC and is the best preserved of the three. The Temple of Peace in the forum is a Corinthian-Doric building begun perhaps in the 2nd century BC. Traces of a Roman amphitheatre and other buildings, as well as intersecting main streets, have also been found. The circuit of the town walls, which are built of travertine blocks and are 15–20 feet (5–6 m) thick, is about 3 miles (5 km) in circumference. In July 1969 a farmer uncovered an ancient Lucanian tomb that contained Greek frescoes painted in the early classical style. Paestum's archaeological museum contains these and other treasures from the site.

Here's the answer I got (keep in mind, you might get a slightly different result because of the stochastic nature of LLMs, but it's still likely to be incomplete):

There are three temples in Paestum, and they are constructed in the Doric architectural style. These temples are:

- The Temple of Athena (also known as the Temple of Ceres), from the 6th century BC.
- The Temple of Hera I (also known as the Basilica), also from the 6th century BC.
- The Temple of Hera II (also known as the Temple of Neptune), built

around 460 BC and the best-preserved of the three.

These temples are remarkable for their well-preserved Doric architecture and are important historical landmarks.

This is almost what I wanted, but it's missing who built the temples. If this had been a Q&A engine instead of a chatbot, the client system wouldn't have received any answer about who built the temples.

Note

A Q&A engine, like a summarization engine, is called by a client system and returns one answer. A chatbot, used by a person, allows for multiple questions so you can refine the answer through interaction.

Next, I send this prompt:

This answer is almost what I expected, but you forgot to answer something: who built the temples?

ChatGPT responds:

I apologize for missing that part. The temples in Paestum were built by Greek colonists from Sybaris who founded the city of Poseidonia (later Paestum) around 600 BC. So, the Greek colonists constructed these temples in the Doric architectural style.

This shows the advantage of a chatbot over an engine: after a couple of rounds of questions, ChatGPT answered all my questions about the temples in Paestum. Importantly, I didn't have to resubmit the full text; ChatGPT remembered the initial text.

Note

A chatbot like ChatGPT creates a stateful session that remembers the exchange by passing the history of the conversation as an additional context,

allowing the user to refine the answer without resending the initial text.

Let's try another prompt:

Is the archaeological site in Paestum recognized by UNESCO?

ChatGPT answered:

Yes, the archaeological site of Paestum is recognized by UNESCO. In 1998, it was designated a UNESCO World Heritage Site. This designation acknowledges the historical and architectural significance of the well-preserved Greek temples and ancient ruins in Paestum.

Hold on! The original text didn't include this information. ChatGPT used its own knowledge, likely from Wikipedia or other sources, to answer correctly. This eagerness to answer questions using unprovided content can be helpful, as shown in the example, but it can also be dangerous, as you will see shortly

Let's try another question:

How many columns do the three temples have in total?

ChatGPT answered:

The three temples in Paestum have a total of 94 columns:

- The Temple of Athena (Temple of Ceres) has 6 columns on its shorter sides and 13 columns on its longer sides, for a total of 32 columns.
- The Temple of Hera I (Basilica) has 6 columns on its shorter sides and 9 columns on its longer sides, for a total of 30 columns.
- The Temple of Hera II (Temple of Neptune) has 6 columns on its shorter sides and 14 columns on its longer sides, for a total of 32 columns.

ChatGPT tried to answer, but the answer is wrong. The Temple of Hera I has 50 columns, not 30, as you can check yourself on the Wikipedia.

Warning

When a chatbot can't find the information in the provided text, it might rely on its own knowledge, which can lead to incorrect or made-up information, known as "hallucination." You can limit this by forcing the LLM to only use the information in the prompt and admit when it doesn't know the answer. Beyond prompt engineering, you can reduce hallucinations by testing the application using an evaluation framework, which I'll cover in later chapters.

In one instance, ChatGPT provided accurate information about UNESCO, yet in another, it gave an incorrect answer about the number of columns in a structure. This raises an important question: how can you tell when ChatGPT is providing reliable information and when it's not? As a general rule, you should be cautious about trusting ChatGPT for facts or details you haven't independently verified or supplied yourself.

Tip

To avoid this issue, instruct ChatGPT to use only the information you provided and indicate if it doesn't know the answer.

You can try to address the previous issue with this prompt:

How many columns do the three temples have in total? Use only the information I gave you in the text. If you don't know the answer, just say so.

You'll get a more accurate response:

I apologize for the error in my previous response. The text you provided doesn't mention the total number of columns in the three temples in Paestum.

Spoiler Alert

Designing safe prompts for Q&A chatbots reduces the chance of hallucinations. You'll learn more about this in the rest of the book.

Now, let's move to a more complex use case.

6.1.2 A More Complex Q&A Chatbot Over a Knowledge Base

Let's summarize the design of the basic LLM-based Q&A chatbot, like ChatGPT, which processes a single piece of text:

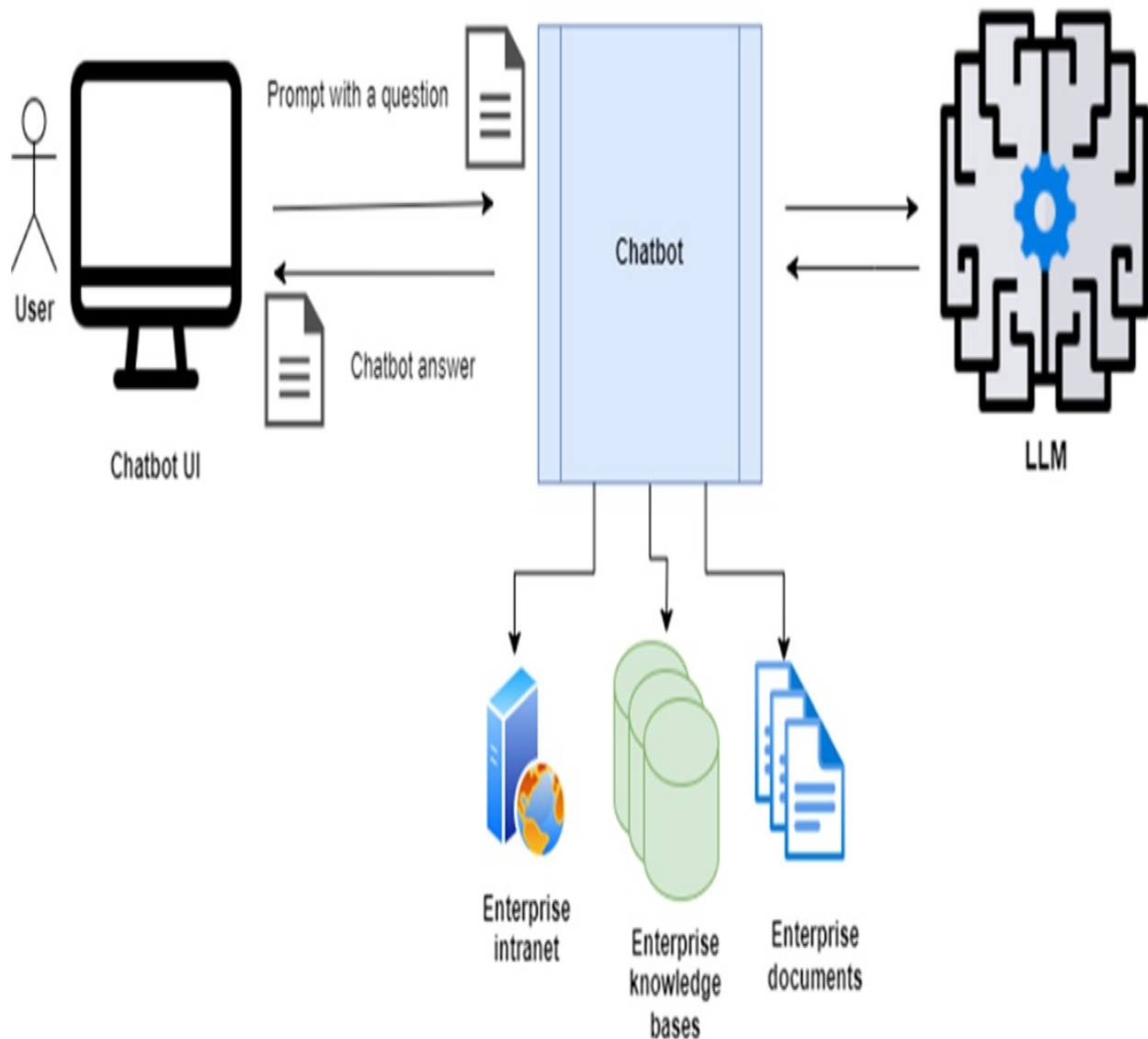
1. You send a prompt to the chatbot with the text you want to search for and the question you want to ask.
2. The prompt should instruct the chatbot to formulate an answer using only the provided text.
3. The chatbot should create a session, retaining the conversation history to refine answers.

The approach described so far works well for searching within a single text. But what if your chatbot needs to answer questions about company information scattered across multiple sources—such as intranet pages, shared folders, and documents in various formats like PDF, Word, TXT, or PowerPoint? That's the challenge we'll tackle in the next chapters.

When designing an enterprise Q&A chatbot, one of the main obstacles is that you can't include all of the company's content in the prompt along with the user's question—especially when dealing with large documents, this would quickly exceed the model's context window. In practice, narrowing down the context to what's most relevant improves speed, cost-efficiency, and accuracy. In fact, providing less—but more focused—context often yields better results than overloading the model with loosely related information.

To achieve this, the chatbot must be able to access the company's knowledge base and retrieve only the specific content needed to answer a given question. Ideally, it should “understand” the knowledge base well enough that you can ask a question naturally—without manually supplying background context each time. This concept is illustrated in Figure 6.2.

Figure 6.2 Hypothetical design for an enterprise Q&A chatbot: the knowledge of the chatbot is expanded with various data sources: the enterprise intranet, knowledge bases and documents



Now, you might wonder: how can you connect the ChatGPT chatbot (or Gemini chat, or Claude chat) to company intranet, knowledge bases and documents, as shown in figure 6.2 above, so you can send just the question in the prompt? Unfortunately, you can't connect ChatGPT to your local data directly. The above solution doesn't work with standard ChatGPT. The closest alternative is to use ChatGPT Plus and configure a custom version through OpenAI's *My GPTs* offering, allowing you to upload documents for lookup by later queries. This is convenient for simple use cases. However, for more control over how the chatbot interacts with text sources and the LLM, you need a different approach. Enter the RAG design pattern.

6.1.3 The RAG Design Pattern

The Retrieval Augmented Generation (RAG) design pattern is a classic solution for building a Q&A chatbot. Let's break down what RAG stands for:

- **Retrieval:** This step involves retrieving context from a pre-prepared data source, typically a vector store optimized for semantic search. Retrieval is a key part of the RAG architecture.
- **Augmented:** This means the answer is improved or enhanced by the context provided during the retrieval step.
- **Generation:** This refers to generating the answer to your question. Since this book focuses on LLMs and generative AI, the answer generation is performed by an LLM, requiring the chatbot to interact with it.

You might wonder where this information is retrieved from and how. The key is preparing the information so your custom chatbot can easily access it and then use it to augment the LLM's generated answer.

The RAG design pattern has two stages:

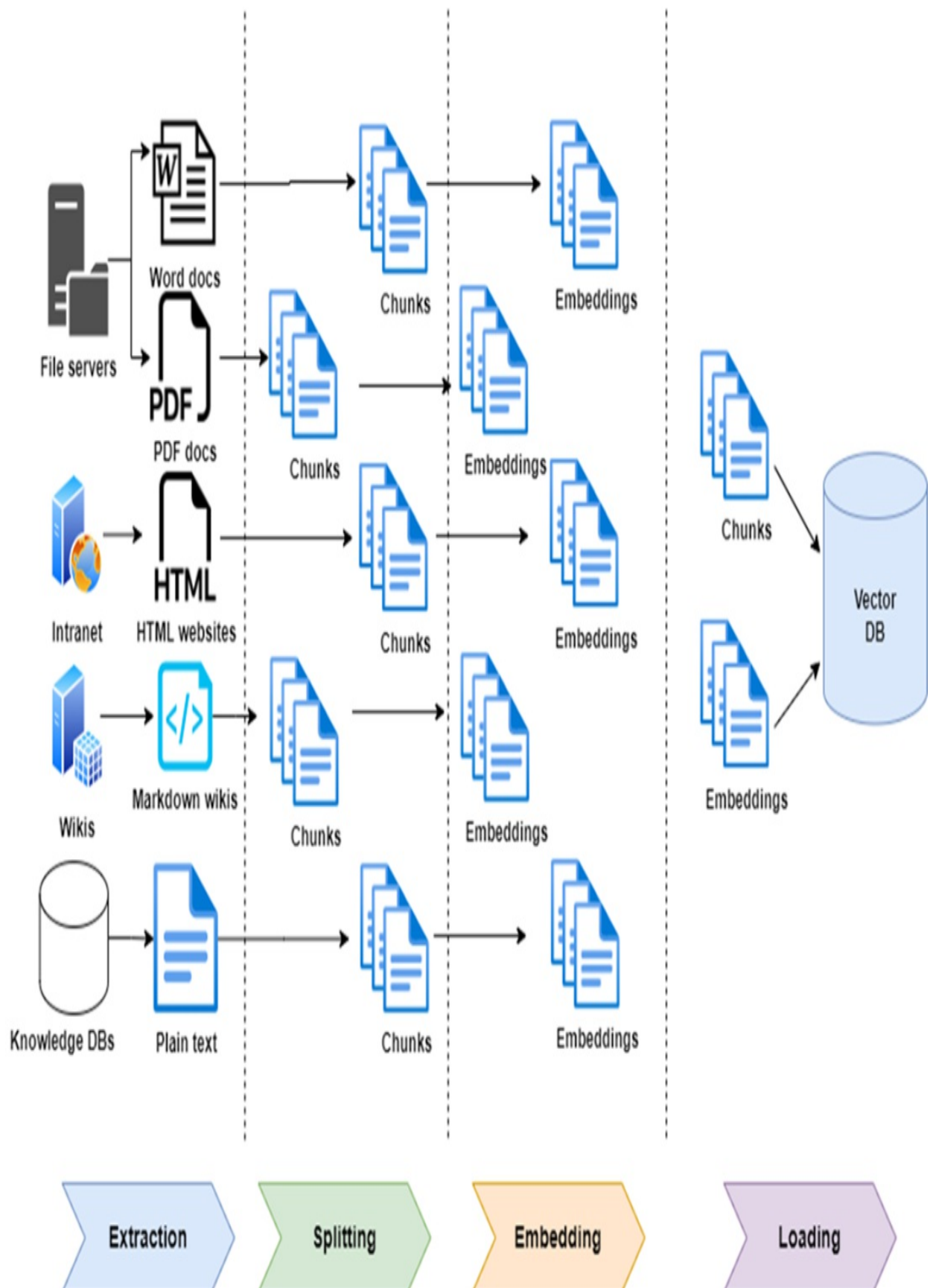
1. **Content Ingestion Stage (Indexing Stage):** All the content users will query is stored in a “special” database and indexed in a “special” format for efficient retrieval (I will clarify what “special” means shortly).
2. **Question-Answering Stage (Retrieval and Generation Stage):** The chatbot takes a user's question, retrieves relevant information from the special database, and feeds it along with the user's question to the LLM. The LLM generates and returns the augmented answer.

Let's dive into each phase.

Content Ingestion Stage (Indexing Stage)

Before users can query the Q&A chatbot, you need to store relevant content, such as enterprise documents from various sources and formats, into a vector store, a database optimized for quick search and retrieval, as shown in figure 6.3.

Figure 6.3 RAG Ingestion Stage: Documents are extracted from sources, split into chunks, and converted into embeddings while being stored in a vector database, which stores a copy of the original chunks and their embeddings (vector form).

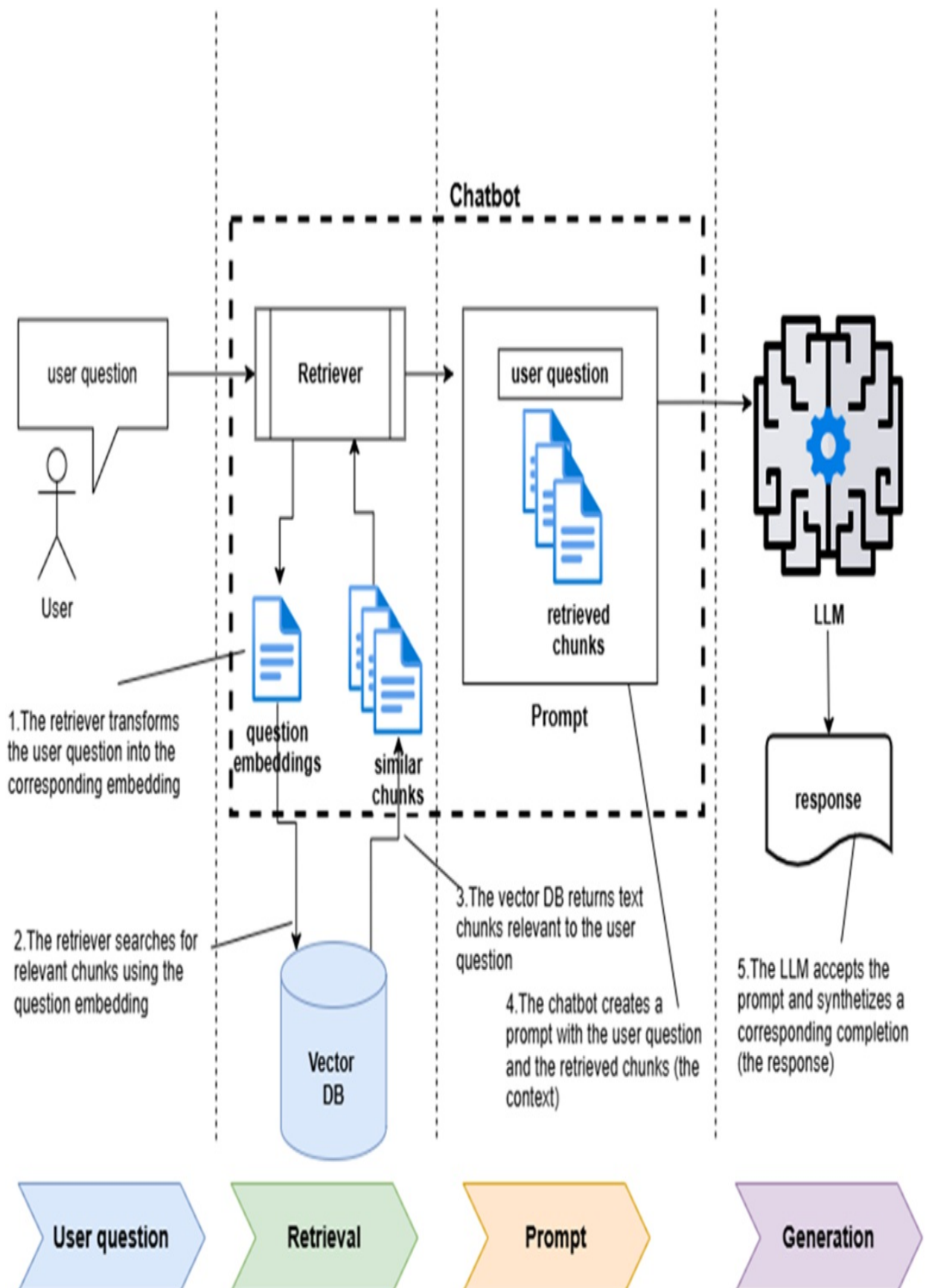


During the content ingestion stage, text is extracted from the sources, split into small chunks, and each chunk is transformed into an "embedding," a numerical vector representation of the text. Splitting content into chunks is crucial because embedding models work on finite sizes, and you want the search to target small, relevant content pieces instead of large, mixed-relevance sections. You can create embeddings using the vector store's proprietary model, an LLM provider's model (like OpenAI), or a dedicated embedding library. The embeddings and content chunks are then stored in a vector database. The purpose of the embeddings is to index the content for efficient lookup during the Q&A stage. This means the text in the user's question doesn't need to match the text in the results exactly to produce relevant answers. For example, querying the vector store for "feline animals" will return chunks mentioning "cat," "lion," and "tiger," even if the word "feline" isn't in any document chunk.

Question & Answer Stage (Retrieval & Generation Stage)

Once the information has been split into small chunks, transformed into embeddings, and stored in a vector store, users can query your Q&A chatbot. Let's walk through the Q&A stage workflow, as shown in Figure 6.4.

Figure 6.4 RAG Question & Answer Stage: 1) The chatbot uses a retriever to transform the user question into embeddings; 2) then the retriever uses the embedding to perform a similarity search in the vector store. 3) The vector store returns several relevant text chunks; 4) The retrieved content is fed into a prompt as a "context" along with the original user question. 5) The chatbot sends the prompt to the LLM which synthesizes a response and returns it to the user.



When the chatbot receives a user question, a retriever converts the natural language query into its vector representation using an embedding model. This step must use the same embedding model as the ingestion stage. The chatbot then uses the vectorized question to query the vector store, which understands the query's semantic meaning and returns document chunks with embeddings similar to the query. It performs a vector distance calculation between the vectorized question and the items in the vector index stored in the database. The chatbot can retrieve the closest text chunk or a set number of the closest chunks (ordered by distance).

Once the relevant content chunks are retrieved, the chatbot sends the LLM a prompt that includes the initial question and a "context" incorporating the retrieved document chunks. The LLM then generates (or "synthesizes" in RAG terminology) the answer and returns it to the chatbot, which then delivers it to the user.

This final step is similar to the basic "Q&A over a single document" use case. You provide the LLM with the initial question and a "context" (previously the entire input text in the simple scenario) that provides the information for the answer. Now, the "context" is represented by chunks retrieved from the vector store. The main difference between Q&A over a single document and over a range of documents in a vector database is the additional components: the vector store provides the information for the answer. The role ChatGPT played in the basic chatbot use case is now split between an orchestrating chatbot (which accepts the query and retrieves the information) and an LLM (which synthesizes the answer).

Now that you understand the high-level architecture of the RAG design pattern, let's examine one of its key components: the vector store. After that, you'll be ready to attempt your first RAG implementation.

6.2 Vector Stores

I have mentioned vector stores several times, but only briefly. In this section, I will explain what they are, their purpose, what they offer, and name a few examples.

6.2.1 What's a Vector Store?

A vector store is a storage system designed to efficiently store and query high-dimensional vectors. Vectors are key in AI because embeddings—numerical representations of text, images, sounds, or videos—are built using them. In short, embeddings are vectors that capture the meaning of words in their dimensions.

The main use of vector stores in LLM and ML applications is to store embeddings that act as indexes for text chunks (or chunks of video, image, or audio). Searches in vector stores are "similarity searches," which measure the distance between the embeddings of the query and those of the stored chunks. The result is either the closest vector or a list of the closest vectors. This "semantic similarity" reflects how close the meanings of the text chunks are.

6.2.2 How Do Vector Stores Work?

Vector distance calculations use common functions like Euclidean distance, Cosine distance, and Hamming distance. These are used in ML algorithms such as k-Nearest Neighbors (KNN) and the more scalable Approximate Nearest Neighbor (ANN) search, which is the standard algorithm for similarity searches.

Tip

I won't cover distance metrics or similarity search algorithms here. If you're interested, check out this academic paper by Yikun Han et al.:

<https://arxiv.org/pdf/2310.11703.pdf> or this informal article by Erika Cardenas: <https://weaviate.io/blog/distance-metrics-in-vector-search>.

The first vector stores, like Milvus, appeared in 2019 to support *dense vector* search, mainly for image recognition. These stores efficiently stored and compared image embeddings. This is called "dense vector" search because the vectors, or embeddings, have most dimensions with non-zero values.

Earlier search techniques, like Term Frequency-Inverse Document Frequency (TF-IDF), used *sparse vectors*, where most values are zero. These were used

for *lexical search*, focusing on exact word matches and implemented in systems like Lucene or BM25.

Milvus was initially built for image-based embeddings, where the vectors represented the meaning of an image. Later, vector stores expanded to tasks like product recommendations, and with the rise of large language models (LLMs), new vector stores emerged specializing in text-based semantic similarity search.

6.2.3 Vector Libraries vs. Vector Databases

The first vector stores, known as "vector libraries," like Faiss (developed by Meta), offered minimal functionality to keep things simple. They stored embeddings in memory using immutable data structures and provided efficient similarity search capabilities. However, as LLM adoption grew, these libraries revealed several limitations:

- **Handling Underlying Text:** Vector libraries only stored embeddings, requiring you to store the original data, such as text or images, elsewhere. This meant creating a unique identifier for each piece of data to synchronize the original text and its embeddings, complicating implementation and maintenance.
- **No Updates:** Vector libraries used immutable data structures, preventing updates. This made them unsuitable for use cases with frequently changing data, especially on an intraday basis.
- **Limited Querying During Data Ingestion:** Vector libraries didn't support querying during data ingestion due to the risk of simultaneous read and write operations, which could impact performance and scalability.

To address these issues, vendors like Pinecone developed "vector databases," offering more features:

- **Handling Text and Embeddings:** Vector databases store both the text and related embeddings, simplifying the client application's workflow. Many can even handle embedding creation. They also allow storing metadata associated with the text, such as provenance and lineage information.

- **Full Create/Read/Update/Delete (CRUD) Capabilities:** Vector databases support updating data, making them suitable for use cases with frequent data changes.
- **Querying During Import:** Vector databases allow similarity searches while importing new data, enhancing scalability and performance.

Vector databases soon introduced features like caching, sharding, and partitioning, which improved scalability, performance, robustness, and durability, similar to traditional relational and NoSQL databases. Meanwhile, relational databases like PostgreSQL and NoSQL databases like MongoDB, which already offered these benefits, adapted by adding support for vector types. This allows embeddings to be stored alongside text in the same record or document, making it easy to link text with its corresponding embedding. For the rest of the book, I will use "vector store" and "vector database" interchangeably, as they have converged in meaning.

6.2.4 Most Popular Vector Stores

Compiling a table summarizing the characteristics of the most popular vector stores is challenging due to their rapid evolution and convergence. However, table 6.1 gives a rough idea of what's available in the market at the time of publication, providing a starting point for your exploration.

Table 6.1 Most popular vector stores and related characteristics

Vector store	Type	Website
Faiss	Vector library	https://github.com/facebookresearch/faiss/wiki/
Milvus	Vector database	https://milvus.io
Qdrant	Vector database	https://qdrant.tech
Chroma	Vector database	https://www.trychroma.com
Weaviate	Vector database	https://weaviate.io

Pinecone	Vector database	https://www.pinecone.io
Vald	Vector database	https://vald.vdaas.org
Scann	Vector library	https://github.com/google-research/google-research/tree/master/scann
KDB	Time series DB	https://kdb.ai
Elastic Search	Search engine	https://www.elastic.co
OpenSearch	Fork of ElasticSearch	https://opensearch.org
PgVector	PostgreSQL extension	https://github.com/pgvector/pgvector
MongoDB Atlas	MongoDB extension	https://www.mongodb.com/

6.2.5 Storing Text and Performing a Semantic Search Using Chroma

Before guiding you through building an enterprise Q&A chatbot, let's first learn how to store text in a vector store and perform semantic searches. This will help you understand the fundamentals.

We'll use Chroma, a vector database that's easy to set up and use. You just need to install the related Python package with pip. Let's get started!

Setting up Chroma DB

First, set up a Chroma Jupyter notebook environment as explained in the sidebar below.

Sidebar: Setting up a Chroma Jupyter Notebook Environment

Create the virtual environment for Chapter 6's code. Open a terminal, create a ch06 folder, navigate into it, and run:

```
C:\Github\building-llm-applications\ch06>python -m venv env_ch06
```

Activate the virtual environment:

```
C:\Github\building-llm-applications\ch06>.\env_ch06\Scripts\activ
```

You should see:

```
(env_ch06) C:\Github\building-llm-applications\ch06>
```

Install the necessary packages either with `pip install -r requirements.txt` if you have cloned the Github repo, or as follows (note the chromadb version):

```
pip install notebook chromadb==0.5.3 openai
```

If you cloned the Github repo, start the Jupyter notebook with `jupyter notebook 06-chromadb-ingestion-and-querying.ipynb`, otherwise create it from scratch:

```
jupyter notebook
```

Create a notebook with `File > New > Notebook` and save it as: `06-chromadb-ingestion-and-querying.ipynb`.

On In your notebook, import the chromadb module and create an in-memory client for the vector database. Keep in mind that the in-memory client will lose all data when the notebook session ends. (For other ways to set up a ChromaDB database, see the related sidebar at the end of this section:)

```
import chromadb
chroma_client = chromadb.Client()
```

Next, create a collection to store the content on Paestum from the Britannica website. A collection is like a "bucket" where you store documents and their related embeddings.

```
tourism_collection = chroma_client.create_collection(name="touris
```

Inserting the Content

Add the Paestum content to the collection, splitting it manually into a list of smaller chunks (or “documents”). Chroma will then generate embeddings from the text you provide, using its default embeddings model unless you specify a different one. I have shortened the text for convenience, but you can find the full version in the `Paestum-Britannica.txt` file on my GitHub repo or check my notebook (also on GitHub) if needed. When storing the text, it's useful to include metadata such as the source of each document and an associated ID, as shown in listing 6.1.

Listing 6.1 Creating and populating a Chroma DB collection

```
tourism_collection.add(
    documents=[
        "Paestum, Greek Poseidonia, ...[shortened] ... preserved Gree
        "Poseidonia was probably ...[shortened] ... in the 18th centu
        "The ancient Greek part of ...[shortened] ... from the site."
    ],
    metadatas=[
        {"source": "https://www.britannica.com/place/Paestum"},
        {"source": "https://www.britannica.com/place/Paestum"},
        {"source": "https://www.britannica.com/place/Paestum"}
    ],
    ids=["paestum-br-01", "paestum-br-02", "paestum-br-03"]
)
```

After running this code, you are ready to perform a search on the vector store.

Performing a Semantic Search

Let's perform a query similar to the one you executed against ChatGPT. Ask for the number of Doric temples in Paestum, specifying that you only want the closest result:

```
results = tourism_collection.query(
    query_texts=["How many Doric temples are in Paestum"],
    n_results=1
)
print(results)
```

Here's a shortened version of the result:

```
{'ids': [['paestum-br-03']], 'distances': [[0.7664762139320374]],
```

Chroma understands the query's meaning and returns the correct text chunk containing the answer, along with metadata about the source and the distance between the query and answer embeddings.

Note

Unlike querying ChatGPT, where you had to provide the question and the full text, querying Chroma only requires sending the question, as the content is already stored in the Chroma DB.

Checking Semantic Proximity

To see how close the returned text chunk (paestum-br-03) is to the question compared to the other text chunks (paestum-br-01 and paestum-br-02), request three results:

```
results = tourism_collection.query(
    query_texts=["How many Doric temples are in Paestum"],
    n_results=3
)
print(results)
```

You should see:

```
{'ids': [['paestum-br-03', 'paestum-br-01', 'paestum-br-02']], 'd
```

The embeddings for paestum-br-03 are the closest to the question's embeddings, with a distance of 0.76. The chunk paestum-br-02 is the farthest, with a distance of 1.33, proving that Chroma identified the most relevant chunk correctly.

Note

The vector database doesn't generate an answer as ChatGPT does. It returns the semantically closest text chunks to your query. For a properly formulated answer, you still need an LLM model to process the original question and the retrieved text chunks. This approach saves costs since LLM vendors like

OpenAI charge based on the number of tokens processed.

This section has given you a glimpse of what you can do with Chroma. For more details, check out the official documentation at <https://docs.trychroma.com/>, especially the “Usage Guide” to learn how to run Chroma in client/server mode if you prefer it to run on a separate host from your LLM solution.

Instantiating ChromaDB in Different Ways

So far, you've worked with a local in-memory instance of ChromaDB. You can also create a client for a local on-disk instance like this:

```
collection = client.create_collection("my_persistent_collection")
```

Alternatively, you can set up an HTTP client on the same computer or a different one. To do this, open a command shell and run the following command (assuming you've installed ChromaDB via pip):

```
chroma run --port 8010
```

Next, instantiate the HTTP client in your notebook or application like this:

```
client = chromadb.HttpClient(host="http://localhost", port=8010)
```

Once the client is set up, you can interact with it in the same way you do with the in-memory client.

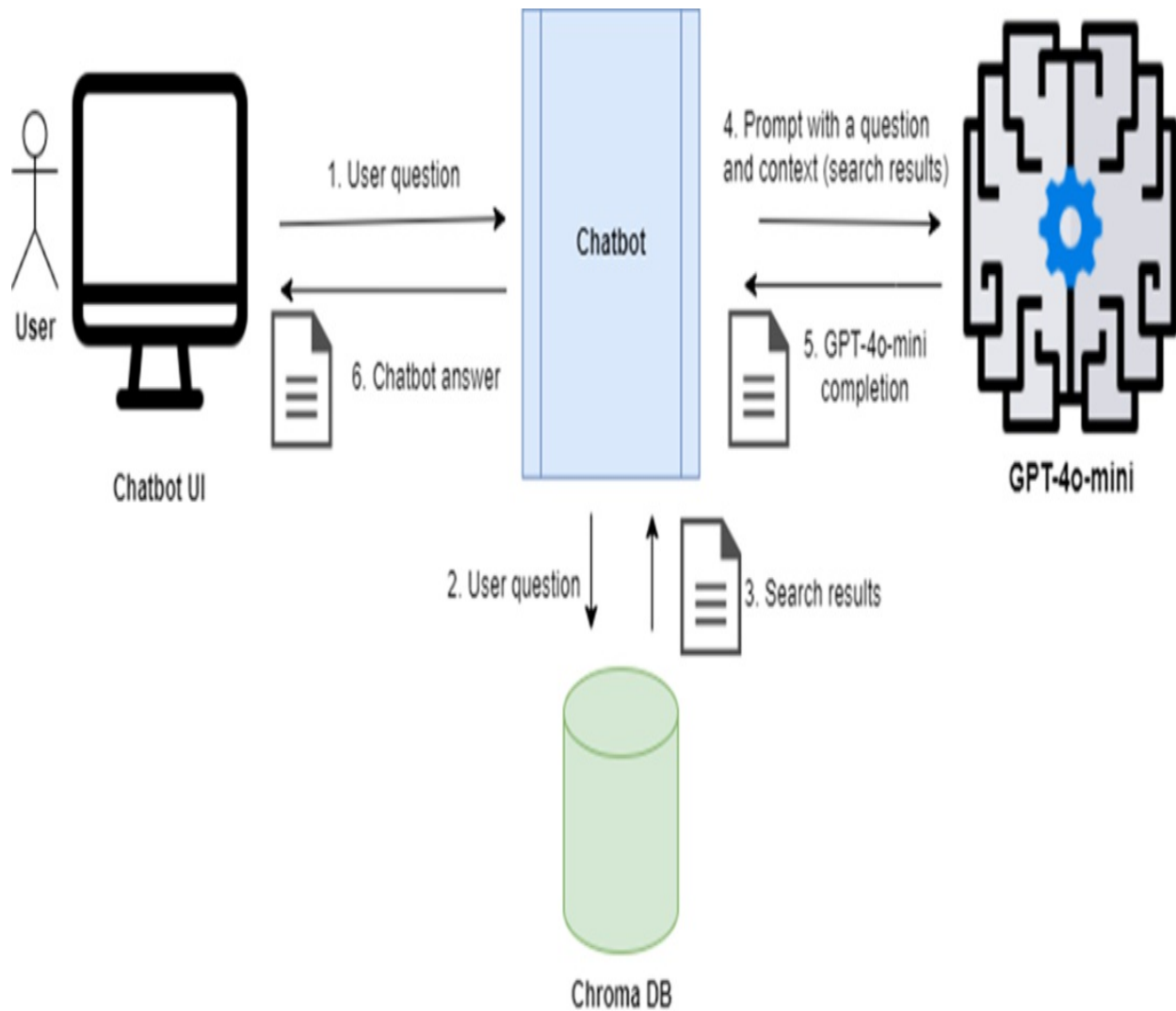
Now that you know how to query the vector store, you can attempt to implement the full RAG pattern, including generating complete answers.

6.3 Implementing RAG from Scratch

Let's implement RAG by building a chatbot that uses the gpt-4o-mini model and a vector database. We will then ask it the same question about Paestum's temples that you asked ChatGPT in section 6.1.1. When using ChatGPT, you had to send a prompt with both the question and the full text on Paestum from Britannica. Once you build your own chatbot, you will only need to ask the

question, as shown in figure 6.5 below.

Figure 6.5 RAG Architecture including Chroma DB and GPT-4o-mini model



As you can see in the architectural diagram in figure 6.5, the chatbot will query Chroma, retrieve the content, and feed it to GPT-4o-mini with the original question to get the full answer.

We'll build the chatbot step by step by implementing a few functions. First, import the OpenAI library and set the OpenAI API key (assuming you're using the same notebook from the previous section):

```
from openai import OpenAI
```

```
import getpass

OPENAI_API_KEY = getpass.getpass('Enter your OPENAI_API_KEY')
```

Now instantiate the OpenAI client:

```
openai_client = OpenAI(api_key=OPENAI_API_KEY)
```

6.3.1 Retrieving Content from the Vector Database

As you already know how to perform a semantic search against the vector store, let's wrap that code into a reusable function:

```
def query_vector_database(question):
    results = tourism_collection.query(
        query_texts=[question],
        n_results=1
    )
    results_text = results['documents'][0][0]
    return results_text
```

Let's try out this function against the same question we asked previously:

```
results_text = query_vector_database("How many Doric temples are
print(results_text)
```

You will see output like this:

```
The ancient Greek part of Paestum consists of two sacred areas co
```

This is the result we expected: notice the retrieved chunk correctly contains the text “three Doric temples”.

6.3.2 Invoking the LLM

We need to craft a prompt that combines the user's question with the context retrieved from the vector database, and then submit it to the LLM. To get started, we'll use a simple prompt and encapsulate the code for calling the LLM in a new function, as shown in Listing 6.2. For this example, I've set the temperature to 0.7 to introduce some controlled randomness in the responses.

Listing 6.2 Functions to define and execute a prompt

```
def prompt_template(question, context):
    return f'Read the following text and answer this question: {q

def execute_llm_prompt(prompt_input):
    prompt_response = openai_client.chat.completions.create(
        model='gpt-4o-mini',
        messages=[
            {"role": "system", "content": "You are an assistant f
            {"role": "user", "content": prompt_input}
        ],
        temperature=0.7
    )
    return prompt_response
```

Using a simple Q&A prompt

Let's test the functions with the question that made ChatGPT hallucinate:

```
trick_question = "How many columns have the three temples got in
tq_result_text = query_vector_database(trick_question)
tq_prompt = prompt_template(trick_question, tq_result_text)
tq_prompt_response = execute_llm_prompt(tq_prompt)
print(tq_prompt_response)
```

We get:

```
ChatCompletion(id='chatcmp1-9nCGTTSUBJUmA0ZWVIGbF80E7nLF5', choic
```

The gpt-4o-mini model did not hallucinate. It correctly recognized it didn't have enough information to answer the question. A few months ago, when I ran the same code against the gpt-3.5-turbo model, it gave a wrong answer of 24 columns, with incorrect assumptions about the number of columns in each temple. I'll show you below how I fixed the issue.

Using a Safer Q&A Prompt

Hallucinations can be mitigated in general by using a well-known prompt for Q&A, available on the Langchain Hub web page (part of LangSmith).

Tip

Langchain Hub (<https://smith.langchain.com/hub>) is a popular LLM resource, constantly updated with open-source models, prompts, and advice on use cases. I highly recommend checking it out.

Here is the recommended hallucination-safe RAG prompt from <https://smith.langchain.com/hub/rlm/rag-prompt>:

Use the following pieces of retrieved context to answer the question. If you don't know the answer, just say that you don't know. Use three sentences maximum and keep the answer concise.

Question: {question}

Context: {context}

Answer:

Let's update the prompt template function accordingly:

```
def prompt_template(question, text):  
    return f'Use the following pieces of retrieved context to ans
```

Now let's re-submit the trick question:

```
trick_question = "How many columns have the three temples got in  
tq_result_text = query_vector_database(trick_question)  
tq_prompt = prompt_template(trick_question, tq_result_text)  
tq_prompt_response = execute_llm_prompt(tq_prompt)  
print(tq_prompt_response)
```

We get this:

```
ChatCompletion(id='chatcmp1-9nCco9P3xSdArsptotrmJEjtd2N5D', choic
```

Well done! You have prevented the LLM from hallucinating. Your chatbot will now only use the knowledge stored in the vector database or admit explicitly it doesn't know the answer.

6.3.3 Building the Chatbot

We can now implement the chatbot with a single function, using the code we've covered in this section:

```
def my_chatbot(question):  
    results_text = query_vector_database(question) #A  
    prompt_input = prompt_template(question, results_text) #B  
    prompt_output = execute_llm_prompt(prompt_input) #C  
    return prompt_output
```

Let's test it with the original question:

```
question = "Let me know how many temples there are in Paestum, wh  
result = my_chatbot(question)  
print(result)
```

We get:

```
ChatCompletion(id='chatcmp1-9nCmZjXJHXVEZLjNpQdAlrnm3K691', choic
```

The synthesized response is comprehensive as it answers all the questions we asked.

You should be proud of what you have achieved so far! You've implemented a basic chatbot that can answer questions based on text imported into the vector database and provide additional information if needed. It won't return information not in the vector database, so it won't hallucinate or make up answers.

The key takeaway is that you now understand the internals of a Q&A LLM-based system, the components of the RAG design pattern, and its workflow. This knowledge will help you when using frameworks like LangChain, LlamaIndex, and Semantic Kernel, which might hide their implementation details. You'll be better equipped to troubleshoot problems and understand what's going on behind the scenes.

Before re-implementing RAG with LangChain, let's recap the RAG terminology you've learned so far.

6.3.4 Recap on RAG Terminology

Throughout this chapter, you've been learning and refining RAG terminology. Some terms may have similar meanings to ones you've seen earlier. The table below will help consolidate your understanding, especially for concepts that can be expressed with different terms.

Table 6.2 RAG Glossary

Term	Definition	Alternative Terms
Retrieval Augmented Generation (RAG)	Use case involving the generation of text (typically an answer) augmented with information retrieved from a content store optimized for semantic searches, typically a vector store	Q&A
Text chunk	A fragment of text from a document. Documents are split into chunks for more effective searching, especially when stored in specialized unstructured text stores like vector stores	Text fragment, chunk, text node, node
Embeddings	Numerical (vector) representation of a piece of text, used to index text chunks for semantic searches	Vector
RAG content ingestion stage	Phase in the RAG design where text is imported and indexed into a context store for efficient retrieval against a natural language question. In a vector store, text is broken into chunks and indexed through their associated embeddings	Text indexing, text vectorization
Vector store	In-memory store or specialized database holding text chunks and their related embeddings, which serve as their index	Vector database
Semantic similarity	Comparing pieces of text based on their meaning, typically by calculating the distance between the embeddings of the	

	text pieces. This can be done using cosine distance or Euclidean distance vector similarity, cosine similarity	
Semantic search	Searching for information based on its meaning. This involves performing semantic similarity between the embeddings of the search question and the text chunks in a vector store	Q&A, vector search
Context	Text (or information) provided in the prompt along with the user question, used to formulate an answer. This can be a full document or a list of text chunks retrieved from a vector store through semantic search	
Synthesize	Generate an answer, typically from a user question and a "context" that provides the necessary information	Generate
RAG Question & Answer stage	Phase in the RAG design where a user asks a search question, the application performs a semantic search against a content store (typically a vector store), and feeds the LLM the original question along with the "context" retrieved from the store. The LLM then synthesizes and returns the answer to the application, which passes it on to the user	RAG Question-Answering stage

You are now ready to re-implement RAG with LangChain, which you will do in the next chapter.

6.4 Summary

- Implement a minimalistic Q&A chatbot by feeding a question and supporting document to an LLM like ChatGPT or Claude.
- For answering questions over a knowledge base, use RAG, which

includes a vector store, a retriever, and an LLM to synthesize responses.

- The RAG design pattern has two stages: ingestion (populating the vector store with text and embeddings) and Q&A (retrieving relevant documents and generating responses).
- A vector store is a specialized database optimized for storing and retrieving text through related embeddings.
- Many vector stores evolved from in-memory libraries to full-fledged persistent databases.
- You can implement a simple RAG system by directly using the APIs of the LLM (such as the OpenAI API) and the vector store (such as the Chroma DB API).

7 Q&A chatbots with LangChain and LangSmith

This chapter covers

- Implementing RAG with LangChain
- Q&A across multiple documents
- Tracing RAG chain execution with LangSmith
- Alternative implementation using LangChain Q&A specialized functionality

Now that you understand the RAG design pattern, implementing it with LangChain should be straightforward. In this chapter, I'll show you how to use the LangChain object model to abstract interaction with source documents, the vector store, and the LLM.

We'll also explore LangSmith's tracing capabilities for monitoring and troubleshooting the chatbot workflow. Additionally, I'll demonstrate alternative chatbot implementations using LangChain's specialized Q&A classes and functions.

By the end of this chapter, you'll be equipped with the skills to build a search-enabled chatbot that can connect to private data sources—a complete version of which you'll construct in Chapter 12.

Before we start implementing the chatbot with LangChain, let's review the LangChain classes that support the Q&A chatbot use case.

7.1 LangChain object model for Q&A chatbots

As discussed earlier, the key benefit of using LangChain for your LLM-based application is its ability to handle communication between components like data loaders, vector stores, and LLMs. Instead of working with each API

directly, LangChain abstracts these interactions. This allows you to swap out any component with a different provider without changing the overall design of your application.

Additionally, LangChain provides out-of-the-box implementations for common requirements when building LLM applications, such as splitting source text, retrieving relevant context from a vector store, generating prompts, handling the prompt context window, and more.

Q&A is a core use case for LangChain. The components needed for this workflow are grouped by stage as follows:

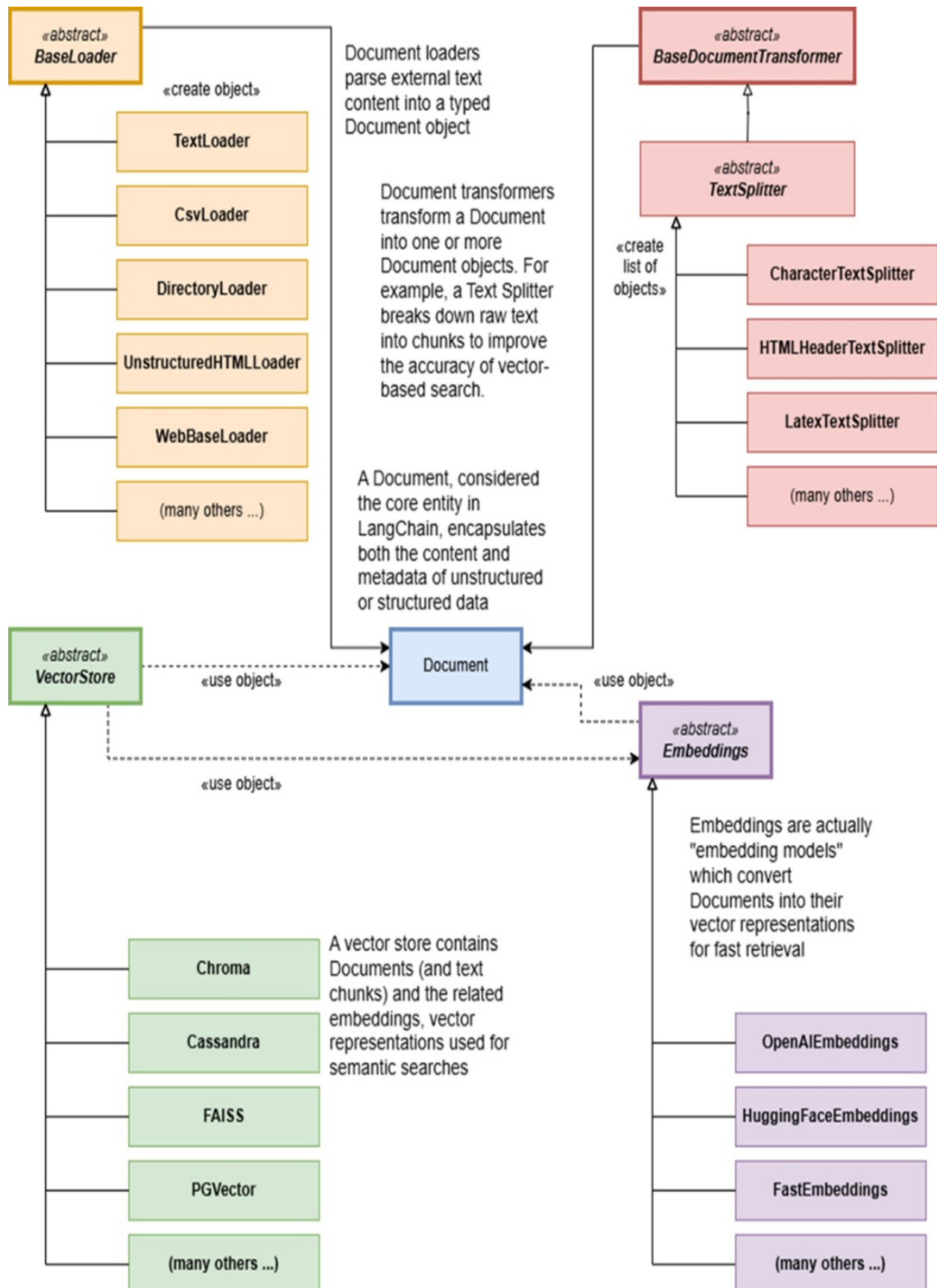
- Content Ingestion (Indexing) Stage
- Q&A (Retrieval and Generation) Stage

Let's examine these component groups in detail.

7.1.1 Content Ingestion (Indexing) Stage

Figure 7.1 summarizes the object model for the content ingestion stage of the Q&A use case.

Figure 7.1 Object model associated with the content ingestion stage.

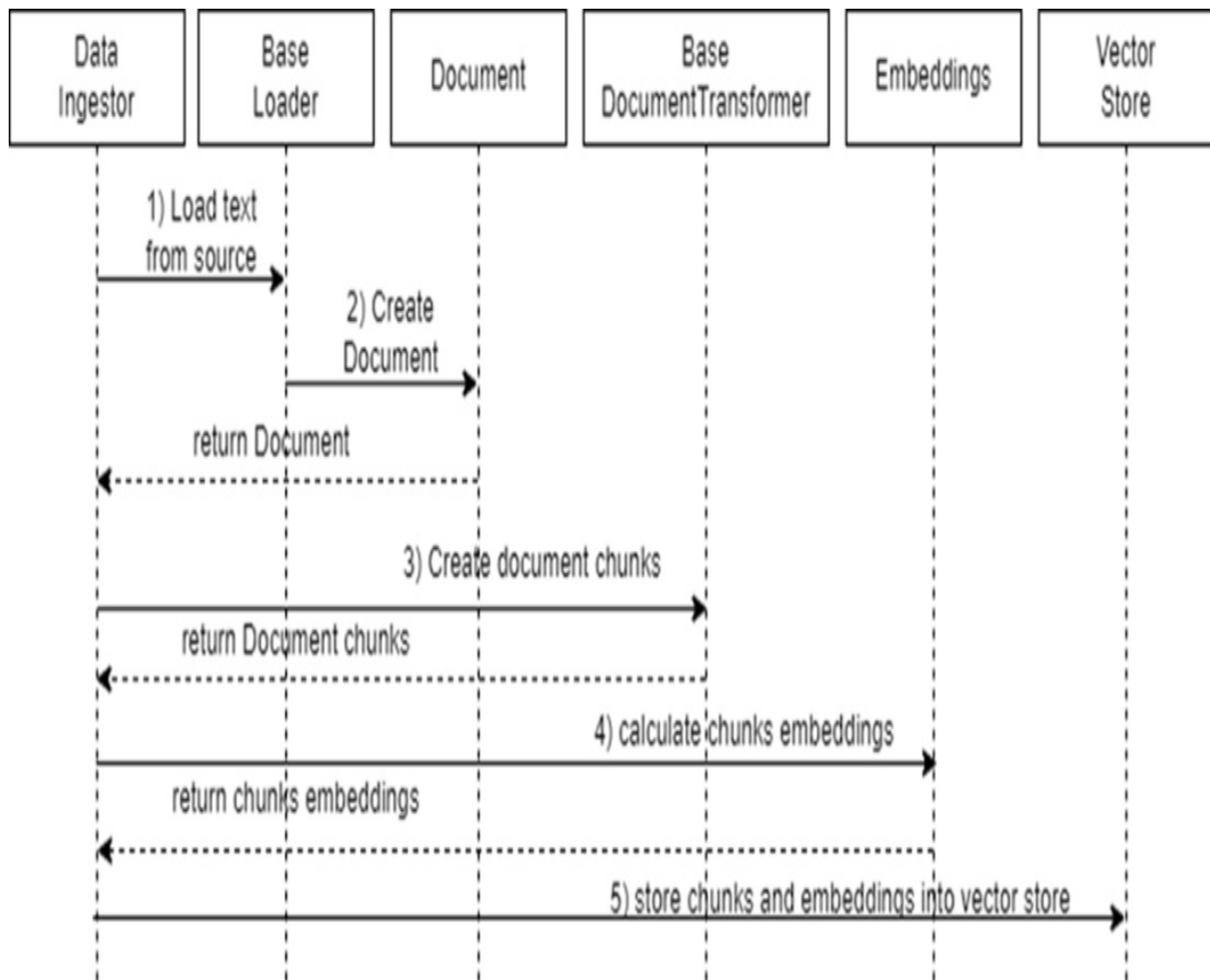


Here are the components shown in the figure:

- Document: Models the text content and related metadata.
- BaseLoader: Loads text from external sources into the document model.
- TextSplitter: Splits documents into smaller chunks for efficient processing.
- Vector Store: Stores text chunks and related embeddings for efficient retrieval.
- Embedding Model: Converts text into embeddings (vector representations).

This is a static view of how LangChain class families connect. For a dynamic view, see the sequence diagram in figure 7.2, which shows the typical content ingestion process.

Figure 7.2 Content Ingestion Sequence Diagram: the data ingestion orchestrator initializes a Loader to import text, which is parsed into a Document. The Document is transformed into smaller chunks by a Text Splitter. An Embedding model generates embeddings for the chunks, and both the chunks and embeddings are stored in the vector store.



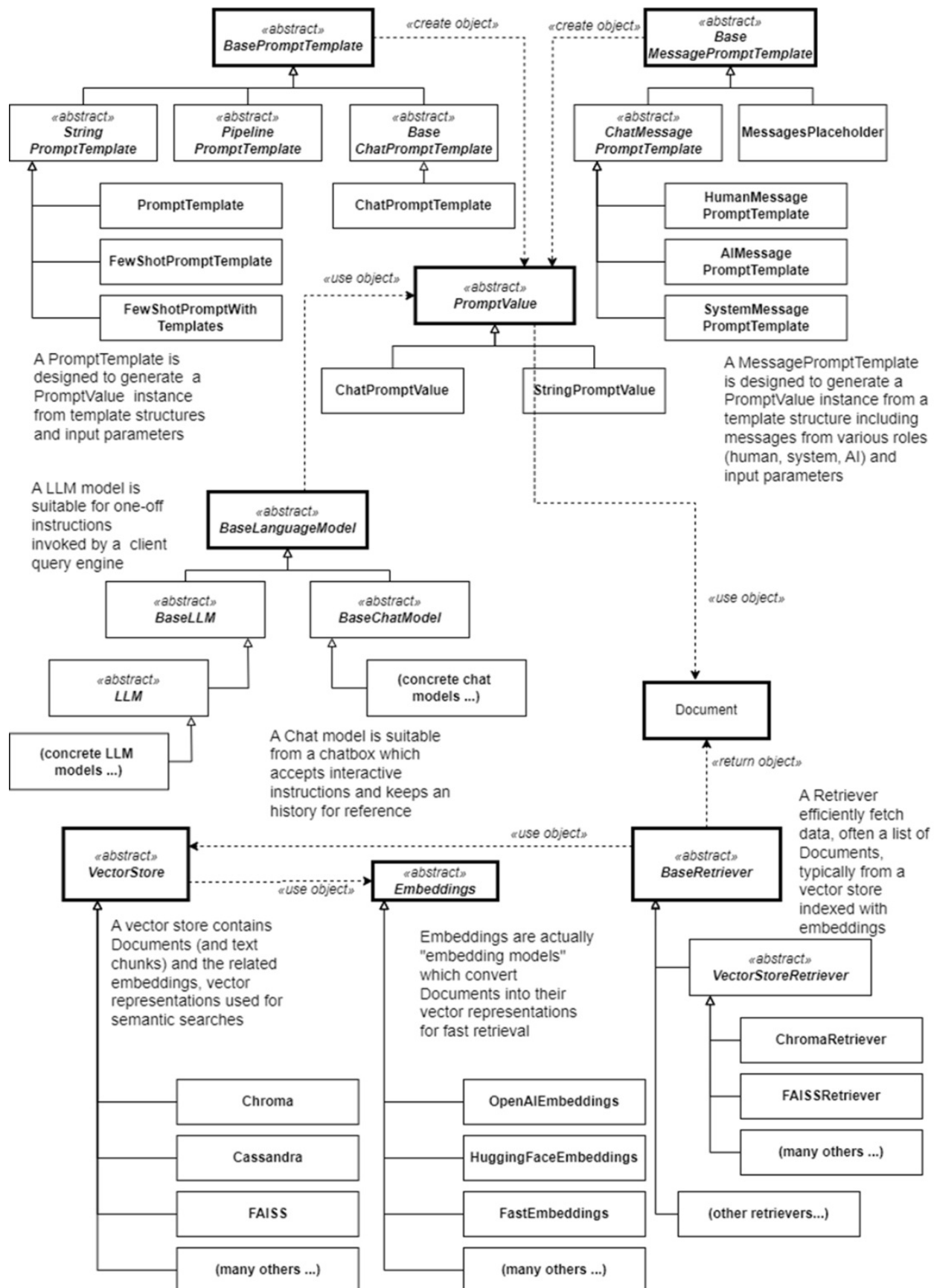
Here's how LangChain classes interact during content ingestion:

1. The data ingestion orchestrator initializes a specific Loader to import text from an external source, like `CsvLoader` for CSV files.
2. The Loader parses the text and converts it into a `Document` object.
3. The `Document` is passed to a `DocumentTransformer`, usually a `TextSplitter`, to divide it into smaller chunks.
4. The chunks are processed by an `Embeddings` model to create embeddings.
5. Both the chunks and embeddings are sent to the `VectorStore` for storage.

7.1.2 Q&A (Retrieval and Generation) Stage

Figure 6.3 summarizes the object model for the retrieval and generation stage of the Q&A use case.

Figure 7.3 Object model associated with the retrieval and generation stage.

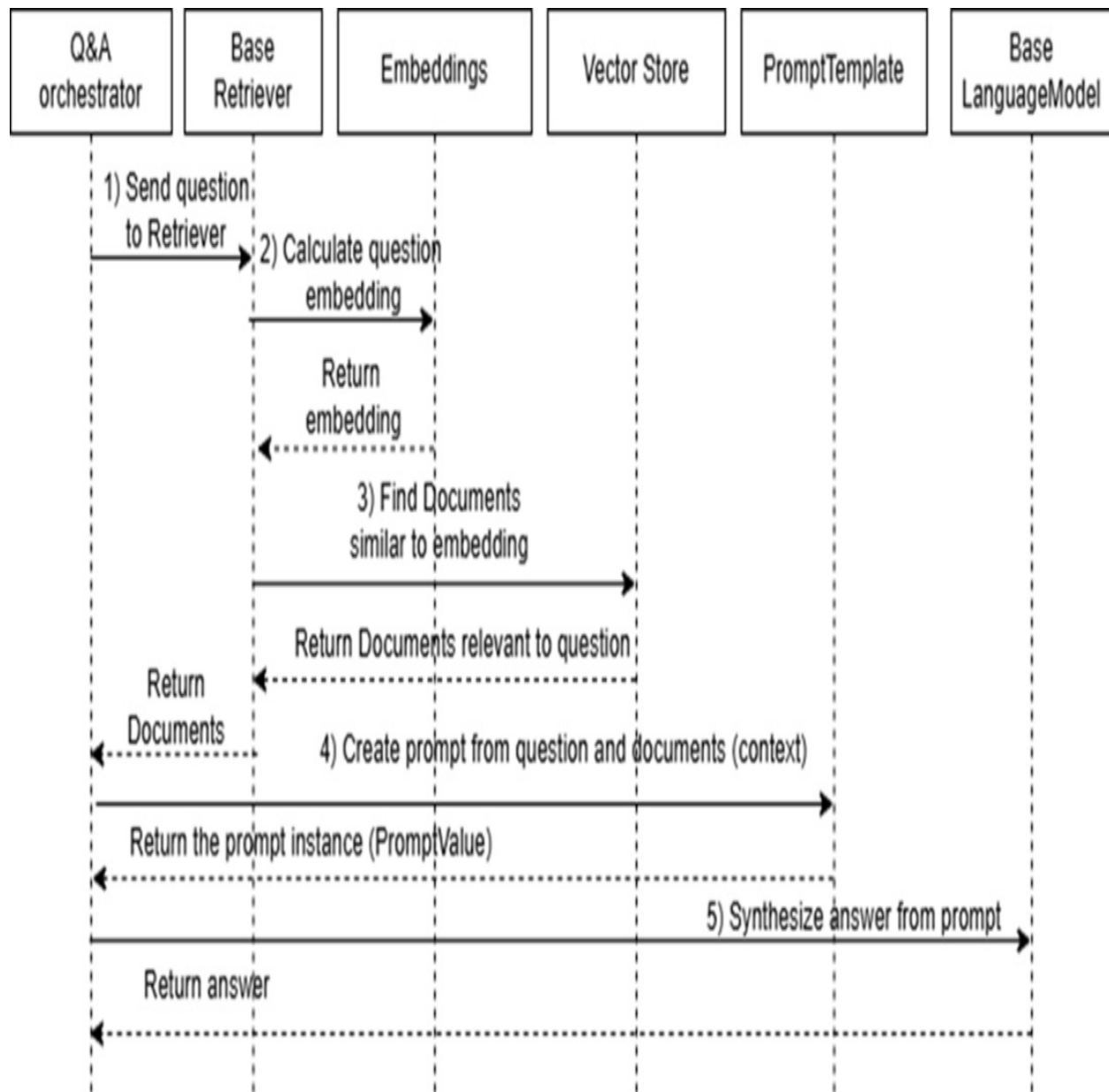


The figure includes the following components:

- **VectorStore:** Stores and retrieves relevant text chunks.
- **Retriever:** Retrieves relevant text chunks from the Vector Store based on the similarity between the query's embedding and the stored text embeddings.
- **Embeddings (Embedding model):** Ensures consistent embeddings for queries and documents.
- **Prompt / PromptTemplate:** Constructs the input for the language model, typically using the user question and a context made of retrieved text chunks.
- **LanguageModel:** Generates answers using the provided context and query.

For a dynamic view of the Q&A workflow, see the sequence diagram in figure 7.4.

Figure 7.4 Q&A Sequence Diagram: the Q&A orchestrator sends the user's question to the vector store Retriever, which generates an embedding using an Embeddings model. The Retriever searches for similar embeddings in the vector store and returns relevant documents. The Q&A orchestrator combines the documents and the question into a prompt with a PromptTemplate, then sends it to the Language Model, which provides the answer.



Here's how LangChain classes interact during the Q&A process:

1. The Q&A orchestrator sends the user's question to the vector store Retriever.
2. The Retriever generates the question's embedding using an Embeddings model.
3. The Retriever searches the vector store for documents with similar embeddings and returns them to the Q&A orchestrator.
4. The Q&A orchestrator combines the retrieved documents and the user's question into a prompt using a PromptTemplate.

5. The orchestrator sends the prompt to the `LanguageModel`, which returns the answer.

Now that you understand the Q&A object model, let's get started with the implementation!

7.2 Vector Store Content Ingestion

Before querying your documents, you must store them in a vector database. To make it more interesting, we'll import content about Paestum from various sources and in different formats, rather than just using the small Britannica article on Paestum as before.

First, install the required packages for loading, splitting documents, and creating embeddings. We'll use the open-source embeddings model from the sentence-transformers package, which includes the Chroma embeddings model `all-MiniLM-L6-v2`. This helps save on costs, but feel free to use the OpenAI embeddings model if cost isn't an issue.

Open a new OS shell, navigate to the folder for this chapter, create and activate the virtual environment for Chapter 7, and install the required packages (as usual, if you have cloned the repository from Github you can install the packages with `pip install -r requirements.txt` instead, after activating the virtual environment):

```
C:\Github\building-llm-applications\ch07>python -m venv env_ch07
C:\Github\building-llm-applications\ch07>.\env_ch07\Scripts\activ
(env_ch07) C:\Github\building-llm-applications\ch07>pip install w
```

Once the installation is complete, start a Jupyter notebook, as usual:

```
(env_ch07) C:\Github\building-llm-applications\ch07>jupyter noteb
```

Create a new notebook with `File > New > Notebook`, give it a name (e.g., `ch07-QA_across_documents`), and save it.

Import the necessary libraries on the notebook:


```
from langchain_community.document_loaders import WikipediaLoader,
from langchain_chroma import Chroma
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
```

Capture the OpenAI API key as usual:

```
import getpass
OPENAI_API_KEY = getpass.getpass('Enter your OPENAI_API_KEY')
```

7.2.1 Splitting and Storing the Documents

To make the content more searchable:

1. Split each document into chunks of about 500 characters. You can add overlap, usually around 10%, but for now, set the overlap to 0. The size of chunks and overlap considerations will be covered in Chapter 8 on advanced indexing techniques;
2. calculate the embeddings of the document chunks and store them in the Chroma vector database.

Instantiate the splitter, embeddings model, and vector database client as follows:

```
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, ch
embeddings_model = OpenAIEmbeddings(openai_api_key=OPENAI_API_KEY
vector_db = Chroma("tourist_info", embeddings_model)
```

Process each document by loading it, splitting it into chunks, and storing the chunks with the related embeddings into the vector database:

```
wikipedia_loader = WikipediaLoader(query="Paestum")
wikipedia_chunks = text_splitter.split_documents(wikipedia_loader
vector_db.add_documents(wikipedia_chunks)
```

Once the content has been split and stored in the vector database, you will see an output similar to this:

```
[ 'd3197373-7df1-4c24-8a0a-0145c176042c',
  '435add06-6b85-421a-8ad8-be88b7defe08',
  '7fbd7575-5f56-4fed-9642-34f587325699',
```

```
'6a798aca-9dcb-433c-b0d3-196acfd83b0e',  
...  
]
```

Note

The WikipediaLoader also loads content from other Wikipedia hyperlinks referenced in the requested article. For example, the Paestum article references the National Archaeological Museum of Paestum, the Lucania region, Lucanians, and the temples of Hera and Athena. As a result, it will load these related articles, providing more content than you might expect.

For other document formats:

```
word_loader = Docx2txtLoader("Paestum/Paestum-Britannica.docx")  
word_chunks = text_splitter.split_documents(word_loader.load())  
vector_db.add_documents(word_chunks)
```

```
pdf_loader = PyPDFLoader("Paestum/PaestumRevisited.pdf")  
pdf_chunks = text_splitter.split_documents(pdf_loader.load())  
vector_db.add_documents(pdf_chunks)
```

```
txt_loader = TextLoader("Paestum/Paestum-Encyclopedia.txt")  
txt_chunks = text_splitter.split_documents(txt_loader.load())  
vector_db.add_documents(txt_chunks)
```

Note

If you prefer, you can process the full PaestumRevisited-StockholmsUniversitet.pdf document instead of its shorter version, PaestumRevisited.pdf. This will take considerably longer, but it will provide information for a wider range of questions.

Removing duplication

You might have noticed that the code above has a lot of duplication. Let's extract some common functionality into a function:

```
def split_and_import(loader):  
    chunks = text_splitter.split_documents(loader.load())  
    vector_db.add_documents(chunks)
```

```
print(f"Ingested chunks created by {loader}")
```

Now you can call this function after instantiating each loader, as shown in Listing 7.1:

Listing 7.1 Refactored ingestion of different file types

```
wikipedia_loader = WikipediaLoader(query="Paestum")
split_and_import(wikipedia_loader)

word_loader = Docx2txtLoader("Paestum/Paestum-Britannica.docx")
split_and_import(word_loader)

pdf_loader = PyPDFLoader("Paestum/PaestumRevisited.pdf")
split_and_import(pdf_loader)

txt_loader = TextLoader("Paestum/Paestum-Encyclopedia.txt")
split_and_import(txt_loader)
```

After running this code, you will see output like this:

```
Ingested chunks created by <langchain_community.document_loaders.
Ingested chunks created by [... SHORTENED]
```

7.2.2 Ingesting Multiple Documents from a Folder

I have just shown you how to ingest different types of documents (web, MS Word, PDF, txt) using a specialized document loader for each type. This approach works fine for a few documents, but if you need to load many documents, a more efficient method is needed. After placing all the documents in a folder (e.g., /CilentoTouristInfo), there are a couple of ways to achieve this:

1. Iterating over the files in the folder and calling the relevant loader.
2. Using the purpose-built `DirectoryLoader`.

Let's explore both approaches.

Iterating over all files in a folder

You can ingest all the files located in a folder into a vector store by iterating over the files, identifying the file type, and using the relevant loader as described in the previous section. Before iterating over the files, create a loader factory as shown below:

Listing 7.2 Loader factory: instantiating the relevant extension-specific loader

```
loader_classes = {
    'docx': WordLoader,
    'pdf': PdfLoader,
    'txt': TxtLoader
}

import os

def get_loader(filename):
    _, file_extension = os.path.splitext(filename)
    file_extension = file_extension.lstrip('.')

    loader_class = loader_classes.get(file_extension)

    if loader_class:
        return loader_class(filename)
    else:
        raise ValueError(f"No loader available for file extension")
```

Exercise

I leave you with an exercise to iterate over the files in `/CilentoTouristInfo` and load them into the vector store using `get_loader()` and `split_and_import()`. If you're unsure, check the section "Ingesting Multiple Documents from a Folder" in my Python notebook `ch07-QA_across_documents.ipynb` on GitHub.

Ingesting all files with DirectoryLoader

An alternative method for loading all files in a folder into the vector store is by using the `DirectoryLoader`. This loader, part of a third-party package by Unstructured (Unstructured offers a platform and tools for ingesting and processing unstructured documents for RAG and fine-tuning), takes a folder

path and a glob pattern (a string with wildcard characters to specify a set of filenames).

First, install the unstructured package or the langchain-unstructured wrapper along with its dependencies. Follow the instructions for your operating system in the LangChain documentation (<https://python.langchain.com/>)> Integrations > Providers table > Unstructured) or the Unstructured documentation (<https://docs.unstructured.io/>).

In your Jupyter notebook, import the DirectoryLoader:

```
from langchain_community.document_loaders import DirectoryLoader
```

You can now load and ingest the files into the vector store with the following code:

```
folder_path = "CilentoTouristInfo"
pattern = "**/*. {docx,pdf,txt}" #A

directory_loader = DirectoryLoader(folder_path, pattern) #B
split_and_import(directory_loader)
```

NOTE

While the code above is included in the GitHub repository, its execution depends on successfully installing the unstructured or langchain-unstructured package. Because setup can vary across operating systems, the code may not run consistently in all environments. I've added comments in the code to clarify this.

7.3 Q&A Across Stored Documents

Now that you have stored all the content about Paestum, let's query the vector store directly to see which documents are retrieved.

7.3.1 Querying the vector store directly

Let's query the vector store to see what documents are retrieved for a question that requires information from different sources:

```
query = "Where was Poseidonia and who renamed it to Paestum?"
results = vector_db.similarity_search(query, 4) #A
print(results)
```

Here is an excerpt from the results, showing that the most relevant chunks returned by the vector store come from different sources, as you can verify in the related metadata:

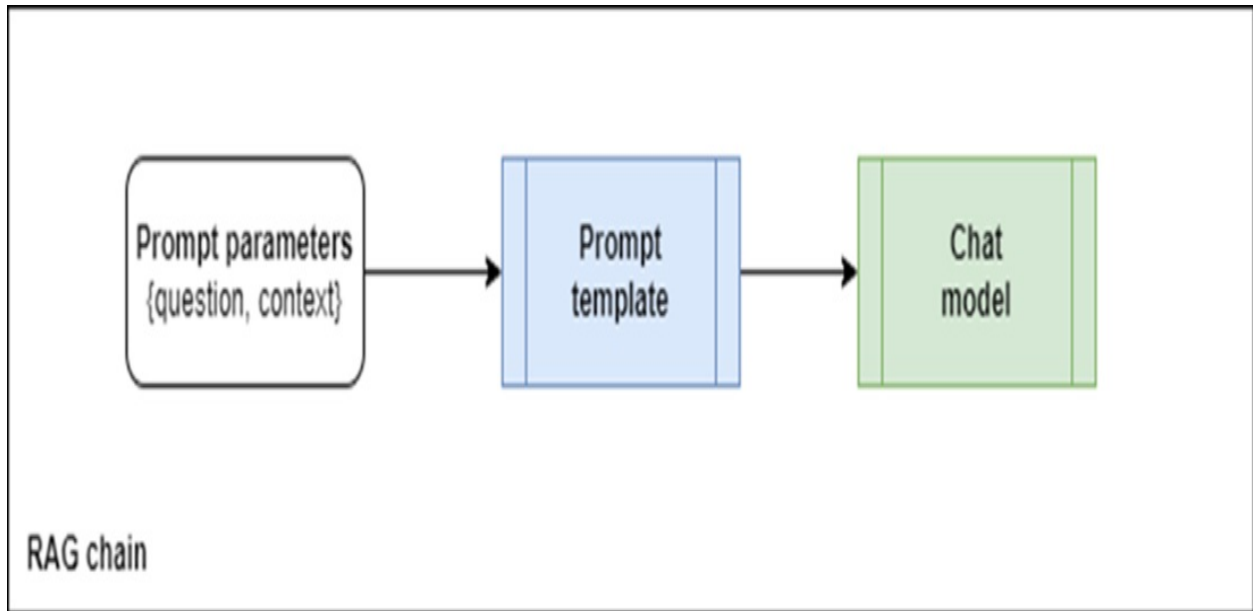
```
[Document(metadata={'source': 'Paestum/Paestum-Britannica.docx'},
```

Although you now know which documents the vector store returns for your query, the next step is to get a well-formulated answer.

7.3.2 Asking a Question through a LangChain Chain

Unlike when you implemented RAG using the OpenAI gpt-4o-mini model directly in section 5.3, with LangChain, you don't need to manually craft a prompt and configure it with the original question and the context from the vector store. You can set up a RAG chain, which will automatically instantiate and execute a prompt template, as shown in Figure 7.5 below.

Figure 7.5 LangChain RAG chain, LangChain RAG chain, packaging the prompt parameters into a prompt instance, which is then sent to the chat or LLM model.



Set up the prompt using a template from LangChain Hub (<https://smith.langchain.com/hub>). For clarity I have simply copied it and set it explicitly below:

```
from langchain_core.prompts import PromptTemplate

rag_prompt_template = """Use the following pieces of context to a
If you don't know the answer, just say that you don't know, don't
Use three sentences maximum and keep the answer as concise as pos
{context}
Question: {question}
Helpful Answer: """

rag_prompt = PromptTemplate.from_template(rag_prompt_template)
```

Alternatively, you can pull the prompt instance directly from LangChain Hub:

```
from langchain import hub
rag_prompt = hub.pull("rlm/rag-prompt")
```

7.3.3 Completing the RAG Chain Setup

To complete the setup of the RAG chain, we need a few more components, as shown in Figure 7.6 below.

Figure 7.6 Amended RAG chain, including runnable passthrough and retriever

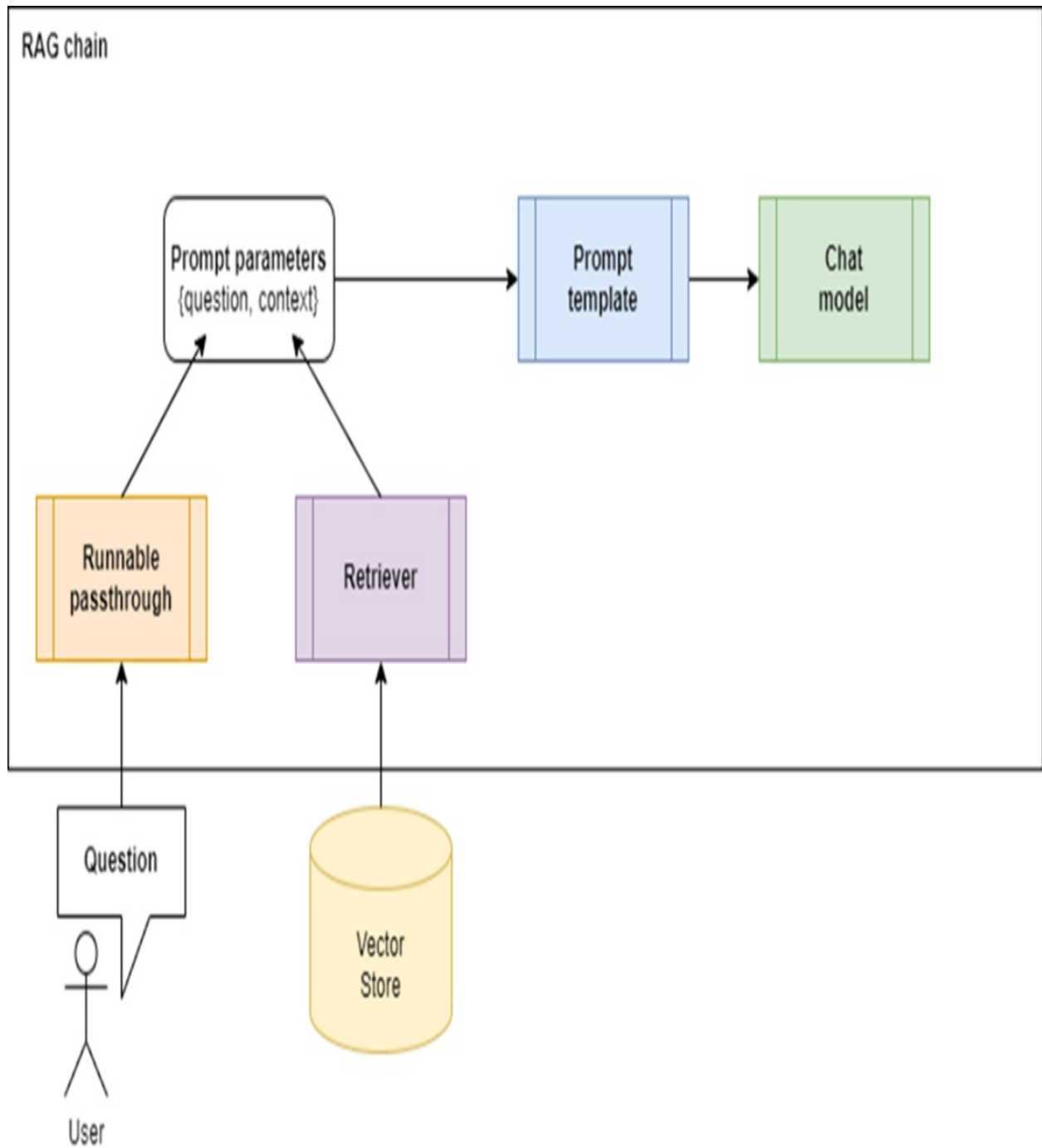


Figure 7.6 shows your RAG chain, which includes:

- **Retriever:** Retrieves relevant text content from the vector store and injects it into the "context" parameter of the prompt.

- Question Feeder: Implemented as a simple “pass-through” component, it feeds the user's question through the Runnable interface (an abstract class on which every LangChain component is based).
- Chat Model: Processes the prompt to generate the answer.

Instantiate the retriever, question feeder, and chat model:

```
retriever = vector_db.as_retriever()

from langchain_core.runnables import RunnablePassthrough
question_feeder = RunnablePassthrough()

from langchain_openai import ChatOpenAI
chatbot = ChatOpenAI(openai_api_key=OPENAI_API_KEY, model_name="g
```

Set up the RAG chain. As mentioned earlier, each block in a LangChain chain implements the Runnable interface and accepts a dictionary as input. This is why the first block in the chain is a dictionary:

```
rag_chain = {"context": retriever, "question": question_feeder} |
```

Create a utility function to execute the chain:

```
def execute_chain(chain, question):
    answer = chain.invoke(question)
    return answer
```

Ask a question:

```
question = "Where was Poseidonia and who renamed it to Paestum? A
answer = execute_chain(rag_chain, question)
print(answer.content)
```

The answer you will get should be similar to:

Poseidonia was located on the coast of the Tyrrhenian Sea in Magn

Inspect the full answer object if needed:

```
print(answer)
```

You should see something like:

```
content='Poseidonia was located on the coast of the Tyrrhenian Se
```

Follow-up Question

Let's find out if the chatbot can sustain the conversation by asking a follow-up question. I'll make it deliberately vague to see if the chatbot can understand what I'm after:

```
Question = "And then, what did they do? Also tell me the source"
answer = execute_chain(rag_chain, question)
print(answer.content)
```

The chatbot returns:

```
I don't know. The source is not provided in the context.
```

It seems the chatbot is a bit lost and doesn't understand that "they" refers to the Romans. This is because the chatbot has no memory of previous questions and answers. Currently, it is stateless and simply passes questions from the user to the LLM and back, without retaining any memory of the conversation flow.

Let's see how we can fix this.

7.4 Chatbot memory of message history

One of the most useful features of an LLM-based chatbot, compared to an LLM-based engine, is its ability to remember previous questions and responses, allowing you to continue querying until you get a satisfactory answer. This capability is crucial for providing context to each new prompt.

Let's see how to incorporate message history into the RAG setup we finalized in the previous section, which I have summarized in listing 7.3 for convenience:

Listing 7.3 Initial RAG Setup Before Incorporating Message History

```
from langchain_core.runnables import RunnablePassthrough
from langchain_core.prompts import PromptTemplate
```

```

from langchain_openai import ChatOpenAI

rag_prompt_template = """Use the following pieces of context to a
If you don't know the answer, just say that you don't know, don't
Use three sentences maximum and keep the answer as concise as pos
{context}
Question: {question}
Helpful Answer: """

rag_prompt = PromptTemplate.from_template(rag_prompt_template)

retriever = vector_db.as_retriever()

question_feeder = RunnablePassthrough()

chatbot = ChatOpenAI(openai_api_key=OPENAI_API_KEY, model_name="g

rag_chain = {"context": retriever, "question": question_feeder} |

def execute_chain(chain, question):
    answer = rag_chain.invoke(question)
    return answer

```

First, we should amend the prompt to include message history.

7.4.1 Amending the Prompt

The original RAG prompt doesn't account for message history, so we need to modify it. Since message history is a core feature of the memory-enabled RAG design, we should use a different prompt helper than `PromptTemplate.from_template`. This helper was based on a template parameterized by a user question (`{question}`) and the context (`{context}`) pulled by the retriever from the vector store. LangChain provides a more suitable prompt helper, `ChatPromptTemplate.from_messages`, which creates a prompt from a list of chat messages.

Chat Messages

Most chat-oriented LLMs use a standard format for messages and roles associated with chat messages. A chat message is typically a key-value pair like this:

("role": "message text")

Where:

- "role" can be "system", "human", or "ai" (see table 7.1 for descriptions).
- "message text" stands for the text of the exchanged message.

Table 7.1 Chat Message Roles

Role	Description	Sample message
System	This role represents the chatbot application. You typically create one system message at startup to instruct the chatbot on its persona.	You are a world-class expert in Roman and Greek history, especially in towns located in southern Italy. Provide interesting insights on local history and recommend places to visit with knowledgeable and engaging answers.
Human	This message comes from a user, typically a question.	Can you please recommend some attractions around Paestum and give me some information on them?
AI	This is the synthesized response from the LLM.	The best attractions in Paestum are the three Greek temples built around 500 BC.
Custom	You can use other non-standard roles to incorporate messages containing useful information. For example, "Context" for text retrieved from the vector store, or "Chat History" for the entire chat history.	

A chat history is a list of such messages:

```
chat_history = [
    ("system", "You are a world-class expert in Roman and Greek h
    ("human", "Can you please recommend some attractions around P
    ("context", "Paestum was a major ancient Greek city on the co
    ("ai", "The best attractions in Paestum are the three Greek t
]
```

Chat-Based Prompt

Now that you understand how to model chat messages, you can create a message-based prompt:

```
from langchain_core.prompts import ChatPromptTemplate
rag_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", "You are a helpful assistant, world-class expe
        ("chat_history", "{chat_history_messages}"),
        ("context", "{retrieved_context}"),
        ("human", "{question}"),
    ]
)
```

You will re-instantiate this prompt at each interaction with the user. Specifically, after the initial interaction, which starts with an empty `chat_history`, you will feed the new `{question}`, the newly `{retrieved_context}`, and the accumulated `{chat_history_memory}`.

You already know how to feed a new question and a newly retrieved context: it's the same as before, but the prompt is now message-based.

In the next section, I will show you how to update the message history at each user interaction.

7.4.2 Updating the Chat Message History

LangChain provides the `ChatMessageHistory` class to model chat message history. We can instantiate the `chat_history_memory` variable as a `ChatMessageHistory` type to hold the chat history:

```
from langchain_community.chat_message_histories import ChatMessageHistory
chat_history_memory = ChatMessageHistory()
```

You can add messages for Human (user question) and AI (LLM response) to the chat history using the `ChatMessageHistory` convenience methods listed in Table 7.2.

Table 7.2 ChatMessageHistory Convenience Methods for Standard Roles

Role	Convenience Method
Human	<code>add_user_message(user_question)</code>
AI	<code>add_ai_message(llm_response)</code>

You don't need to add messages associated with the System and Context roles to the chat message history, as they are already part of the prompt and won't provide the LLM with additional information. The only messages that need tracking are those encapsulating user questions and LLM responses.

Update the chat history in the `execute_chain()` function as follows:

```
def execute_chain_with_memory(chain, question):
    chat_history_memory.add_user_message(question)
    answer = chain.invoke(question)
    chat_history_memory.add_ai_message(answer)
    print(f'Full chat message history: {chat_history_memory.messages}')
    return answer
```

When the `chat_history_memory` object is updated with the latest Human or AI messages, you can retrieve the entire message history using the `messages` property of the `ChatMessageHistory` class:

```
full_message_history = chat_history_memory.messages
```

You will inject this into the prompt each time the user asks a new question.

7.4.3 Feeding the Chat History to the RAG Chain

After updating your code to include message history in the prompt and updating it after each user question and LLM response, you need to feed the updated message history to the RAG chain. You can do this by redefining the

chain with LCEL:

```
rag_chain = {  
    "retrieved_context": retriever,  
    "question": question_feeder,  
    "chat_history_messages": chat_history_memory.messages  
} | rag_prompt | chatbot
```

In this setup, the `chat_history_messages` prompt parameter is fed through the corresponding `chat_history_messages` property above.

7.4.4 Putting Everything Together

To clarify the changes made for integrating chatbot memory into the RAG chain, here's the complete code in Listing 7.4.

Listing 7.4 RAG Chain with Chatbot Memory

```
from langchain_core.runnables import RunnablePassthrough  
from langchain_openai import ChatOpenAI  
from langchain_core.prompts import ChatPromptTemplate  
from langchain_community.chat_message_histories import ChatMessageHistory  
from langchain_core.runnables import RunnableLambda  
  
rag_prompt = ChatPromptTemplate.from_messages(  
    [  
        ("system", "You are a helpful assistant, world-class exper  
        ("placeholder", "{chat_history_messages}"),  
        ("assistant", "{retrieved_context}"),  
        ("human", "{question}"),  
    ]  
)  
  
retriever = vector_db.as_retriever()  
question_feeder = RunnablePassthrough()  
chatbot = ChatOpenAI(openai_api_key=OPENAI_API_KEY, model_name="gpt-4")  
chat_history_memory = ChatMessageHistory()  
  
def get_messages(x):  
    return chat_history_memory.messages  
  
rag_chain = {  
    "retrieved_context": retriever,  
    "question": question_feeder,
```

```

    "chat_history_messages": RunnableLambda(get_messages)
} | rag_prompt | chatbot

def execute_chain_with_memory(chain, question):
    chat_history_memory.add_user_message(question)
    answer = chain.invoke(question)
    chat_history_memory.add_ai_message(answer)
    print(f'Full chat message history: {chat_history_memory.messages}')
    return answer

```

Testing the Amended Chain

Now, let's test the updated chain with the same question we asked previously:

```

question = "Where was Poseidonia and who renamed it to Paestum? A"
answer = execute_chain_with_memory(rag_chain, question)
print(answer.content)

```

Here is the chat message history accumulated so far, followed by the answer:

```

Full chat message history: [HumanMessage(content='Where was Posei

```

Poseidonia was an ancient Greek city located on the coast of the Tyrrhenian Sea in what is now southern Italy. It was renamed Paestum by the Romans after they took control of the city in 273 BC, following its conquest by the Lucanians. The source of this information is from Wikipedia.

The answer is similar to what you got with the memoryless chatbot. Let's see what happens if we now ask the same follow up-question we asked previously:

```

question = "And then what did they do? Also tell me the source"
answer = execute_chain_with_memory(rag_chain, question)
print(answer.content)

```

Now we get the following response:

```

Full chat message history: [HumanMessage(content='Where was Posei

```


After the Romans renamed Poseidonia to Paestum, they developed the city further, enhancing its infrastructure and economy, particularly through agriculture and trade. They also constructed significant buildings, including temples dedicated to Greek gods, which demonstrate the city's cultural continuity. The source of this information is from historical texts on Roman history and archaeology.

As you can see, the chat history now contains all the exchanged questions and answers. Most importantly, the chatbot understands that "they" refers to the Romans and provides a coherent response. Congratulations! You have now completed a fully functional chatbot that remembers previous messages and can sustain a proper conversation.

Before considering the chatbot complete, I would like to cover another important topic: tracing its chain execution with LangSmith.

7.5 Tracing Execution with LangSmith

LangSmith is a comprehensive developer platform for every stage of the LLM-based application lifecycle, whether you're using LangChain or not. It helps you debug, collaborate on, test, and monitor your LLM applications. LLM applications can be unpredictable due to their natural language inputs, which can create edge cases that are hard to reproduce and debug with traditional tools. LangSmith addresses these challenges throughout the LLM application development lifecycle:

- **Development and Debugging:** LangSmith's tracing feature ensures your LLM application workflow executes as expected and helps troubleshoot deviations. Its Hub provides standard, battle-tested prompts for common use cases, speeding up implementation.
- **Evaluation and Testing:** LangSmith supports testing with built-in evaluators for relevance, correctness, and sensitivity of LLM completions. It also provides tools to build datasets from various sources, including production data, for continuous and regression testing.
- **Monitoring:** LangSmith's tracing capabilities allow you to monitor the real-time status of your LLM production application. It supports

feedback through human labels and annotations, enabling investigation and correction when the application deviates from the expected path.

To enable LangSmith's tracing, you just need to set up the LangSmith API key and some tracing configurations at the top of your application. For more details on LangSmith and how to set up the API key, refer to the sidebar below.

Sidebar: Setting up LangSmith's API Key

Set up a LangSmith API key as follows:

1. Register at LangSmith: <https://smith.langchain.com/>. Click the Sign-Up button to create an account using GitHub, Google, Discord, or an email address.
2. Log in to the LangSmith website, click the Settings cog-wheel on the left-hand menu, and then click the Api Keys menu to access the Api Keys page.

Api Keys screen - Access this screen by clicking the Settings cog-wheel and then the Api Keys menu; you can create a new API key here.

< Back

Organizations

Org ID

Personal

Workspaces

PLUS

Members and roles

PLUS

API Keys

Click here to create an API key

Models

Shared

Secrets

Feedback tags

Rules

Usage and billing

API Keys

You can only access an API key when you first create it. If you lost one, you will need to create a new one.

Personal Access Tokens are available now! All previous API keys have been converted to Service

Personal Service All

3 keys

Key	Description
lsv2_pt_d ... 64f5	For transcripts proj
lsv2_pt_3 ... 9521	Chatbot
lsv2_pt_6 ... af1c	llm-crag-work-home

3. Click the *Create API Key* button in the top-right corner. Provide a description for the key (e.g., *LangChain in Action examples*) and select *Personal Access Token* as the key type. Then click *Create API Key*. Be

sure to copy the key and store it in a secure location, as you won't be able to view it again on the LangSmith site—only the first and last few characters will remain visible.

With the LangSmith API key, you'll be able to use it in your projects.

The easiest way to enable tracing through LangSmith is by setting a few environment variables before launching your Jupyter notebook. If your notebook is already running, close it first. Then, follow these steps (I am assuming you are using in a Windows cmd shell):

1. Navigate to the notebook folder and activate your virtual environment:

```
C:\Github\building-llm-applications\ch07> env_ch07\Scripts\activa
```

2. Set the following environment variables, which activate and configure tracing through LangSmith (I assume you are in C:\Github\building-llm-applications\ch07, so I have shortened the folder name below):

```
(env_ch07) C:\...\ch07>set LANGSMITH_TRACING=true  
(env_ch07) C:\...\ch07>set LANGSMITH_ENDPOINT=https://api.smith.l  
(env_ch07) C:\...\ch07>set LANGSMITH_PROJECT=Q & A chatbot  
(env_ch07) C:\...\ch07>set LANGSMITH_API_KEY=<YOUR_LANGSMITH_API_
```

3. Restart the Jupyter notebook:

```
(env_ch07) C:\...\ch07>jupyter notebook 07-QA_across_documents.ipynb
```

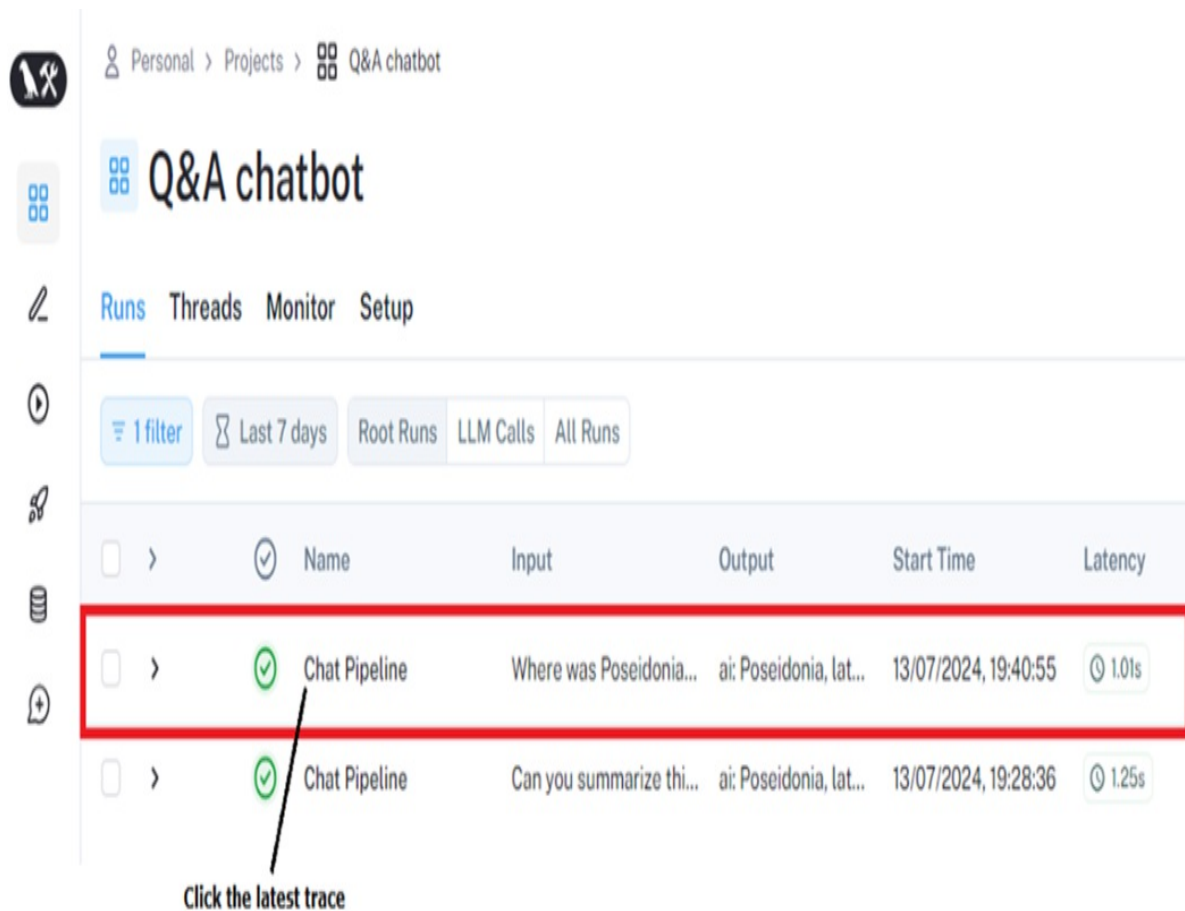
4. Re-run the notebook cells up to the end of Section 7.3 (Q&A Across Documents) or optionally through Section 7.4 (Chatbot Memory).

Once you've completed these steps, LangSmith will have captured tracing information for your notebook execution. You can view and analyze this trace data directly in the LangSmith dashboard.

7.5.1 Inspecting the LangSmith Traces

Go to the LangSmith website (smith.langchain.com) > Projects > Q&A Chatbot. Click the latest trace, as shown in the screenshot below:

Figure 7.7 LangSmith high-level trace: click the latest trace to get high-level details of the chain execution.



The current view shows all the traces associated with your Q&A Chatbot project (a project, in LangSmith's terminology is the collection of all the traces associated to an application or to a part of an application), ordered by execution time, with the most recent at the top. When you click any trace, such as the latest one at the top, you will get more details. I have split the three panels into two figures for clarity. Figure 7.8 shows the left-hand panel and the middle panel.

Figure 7.8 LangSmith trace details obtained by clicking one of the traces, for example, the latest one at the top. You will see three panels: 1) The left panel shows all traces of the Chatbot Q&A project; 2) the middle panel shows the runs of the selected trace.

The screenshot shows the Q&A chatbot interface. The middle panel displays a list of runs, and the right panel shows the details of the selected 'RunnableSequence' trace.

Middle Panel:

Name	Input	Output
Chat Pipeline	Where was Poseidonia...	ai: Po
RunnableSequence	Where was Poseidonia...	ai: Po
RunnableParallel<conte	Where was Poseidonia...	When
PromptTemplate	Where was Poseidonia...	{ "tex
ChatOpenAI	human: Use the followi...	ai: Po
Chat Pipeline	Can you summarize thi...	ai: Po

Right Panel:

TRACE

Collapse Stats Most relevant ▾

Chat Pipeline 1.01s 1,533

RunnableSequence 1.01s

Retriever 0.03s

ChatOpenAI gpt-3.5-turbo 0.98s

Annotations:

- Sequence of runs within trace:** Points to the 'RunnableSequence' row in the middle panel.
- Selected trace:** Points to the 'RunnableSequence' row in the middle panel.

Figure 7.9 shows the middle panel and the right-hand panel.

Figure 7.9 Middle and right-hand panels you get when clicking the latest trace; 1) the middle panel shows the runs of the selected trace; 2) the right panel shows the input to the selected trace and its output.

The screenshot displays the Q&A Chatbot project interface. At the top, there are navigation icons and buttons for 'Playground', 'Add to Dataset', and 'Add to Annotation Queue'. The left panel, titled 'TRACE', shows a list of traces. The 'Chat Pipeline' trace is expanded, revealing a 'RunnableSequence' trace (1.01s) which is highlighted with a black box. Below it are 'Retriever' (0.03s) and 'ChatOpenAI gpt-3.5-turbo' (0.98s). Annotations point to the 'RunnableSequence' trace as the 'Sequence of runs within trace' and the entire trace list as the 'Selected trace'. The right panel shows the details for the 'RunnableSequence' trace, including tabs for 'Run', 'Feedback', and 'Metadata'. The 'Input' section shows a single run with the text: 'input: Where was Poseidonia and who renamed it to Paestum. Also tell me the source.' The 'Output' section shows the AI response: 'Poseidonia, later renamed Paestum, was located in southern Italy. The Lucanians renamed Poseidonia to Paestum, and the Romans adopted this name. Source: Britannica and Wikipedia.'

Sequence of runs within trace

Selected trace

Input and output of the trace

Let's go through the three panels shown in the figures above that you see when clicking the latest trace on the Q&A Chatbot project webpage:

- Left Panel: Shows the list of all traces associated with your project (Chatbot Q&A), ordered from the latest at the top to the oldest at the bottom. Each trace can be expanded into its inner steps. Click the latest

trace, which is the top one.

- **Middle Panel:** Graphically displays the chain execution runs for the selected trace, including the vector-store Retriever and the ChatOpenAI LLM client. Each run represents the execution details of a chain component within the trace. These runs are ordered by execution time and include their duration down to the centisecond.
- **Right Panel:** Shows the trace input (the user question) and its output (the synthesized response).

You can get further details by clicking one of the runs in the middle panel, as shown in figure 7.10 below:

Figure 7.10 Run details; when you click the Retriever run, its details appear in the right-hand panel, showing its input (the user question) and its output (the documents retrieved from the vector store).

The screenshot displays the LangSmith interface for tracing a 'Chat Pipeline'. The middle panel shows a trace with steps: 'Chat Pipeline' (1.01s, 1,533 tokens), 'Retriever' (0.03s), and 'ChatOpenAI gpt-3.5-turbo' (0.98s). The 'Retriever' step is highlighted with a black box. The right-hand panel, titled 'Retriever', shows the details of this step. The 'Input' section contains a query: 'query: Where was Poseidonia and who renamed it to Paestum. Also tell me the source.' The 'Output' section shows four documents retrieved from the vector store:

DOCUMENTS	4
Paestum, Greek Poseidonia, ancien...	Paestum/Paestum-Britannica.docx
Paestum ancient city, Italy Print Cit...	Paestum/Paestum-Britannica.docx
== Name == The Gre...	Paestum https://en.wikipedia.org/wiki/Paestum +1
Paestum was establi...	Paestum https://en.wikipedia.org/wiki/Paestum +1

Annotations on the image:

- Click the Retriever run to examine its details
- The input and output of the Retriever run are shown on the right-hand panel

For instance, clicking the Retriever run in the middle panel shows its details in the right-hand panel. You will see the original query in the Input section and the documents retrieved from the vector store in the Output section.

This is just a glimpse of LangSmith's tracing capabilities using a simple

chain. I encourage you to examine each chain step in detail. Start by clicking a trace substep in the left panel and then inspect its associated runs by selecting the one you want to examine in the middle panel. If you're eager to experiment, create a new LangSmith project to trace the execution of the research summarization engine we built in chapter 4. This will help you appreciate a more complex trace that captures a wider range of chain components.

Before closing the chapter, I want to mention a useful LangChain class you might consider for simple Q&A chains.

7.6 Creating a Q&A Chain with RetrievalQA

Depending on your specific use case, you might set up a Q&A chain more effectively with the `RetrievalQA` chain utility function. This is similar to the `MapReduceDocumentsChain` and `load_summarize_chain` functions used for summarization chains in the previous chapter:

```
from langchain.chains import RetrievalQA

rag_chain = RetrievalQA.from_chain_type(
    llm=chatbot, #A
    chain_type="stuff", #B
    retriever=retriever, #C
    return_source_documents=False #D
)
```

You can now execute it with the `run()` method:

```
question = "Where was Poseidonia and who renamed it to Paestum? A  
with trace("RetrievalQA", "chain", project_name="Q&A chatbot", in  
    answer = execute_chain(rag_chain, question)  
    print(answer)  
    rt.end(outputs={"output": answer})
```

This returns the following result, similar to what we got earlier. Notice the input and output properties are called “query” and “result” respectively. Feel free to inspect the `RetrievalQA` trace in detail on the LangSmith website.

```
{'query': 'Where was Poseidonia and who renamed it to Paestum? A1
```

Before moving on, check out the sidebar on how to configure the retriever for other types of searches.

Configuring the Retriever

You can configure the retriever more accurately by setting its parameters:

- `search_type`: Specifies the search algorithm to retrieve from the vector database.
- `search_kwargs`: Specifies the parameters associated with the search algorithm. In Python, `kwargs` refers to a special syntax (`**kwargs`) that allows you to pass a variable number of keyword arguments to a function. It collects these arguments into a dictionary, making it easy to handle optional or dynamic function parameters.

By default, the retriever is configured for a plain Similarity Search (with the implicit configuration of `search_type = "similarity"`). However, you can explicitly configure it for a Similarity Search with a Threshold using `search_type="similarity_score_threshold"` or for a Max Marginal Relevance Search with `search_type = "mmr"`. You can provide additional configuration details through the `search_kwargs` parameter.

For example, to perform a similarity search and exclude any result with a similarity score lower than 0.8, while returning 3 results, you can configure the retriever as follows:

```
retriever = vector_db.as_retriever(
    search_type="similarity_score_threshold",
    search_kwargs={'score_threshold': 0.8, 'k': 3}
)
```

Consult the official documentation for more information on search parameters.

7.7 Summary

- LangChain abstracts core components of RAG, making the design modular and flexible.

- LangChain's object model includes classes that simplify and abstract the RAG content ingestion phase, such as:
 - BaseLoader: Imports a text source into one or more Document objects.
 - BaseDocumentTransformer: Transforms a loaded Document, for example, by splitting it into smaller Document chunks.
 - VectorStore: Efficiently stores Document objects optimized for semantic searching.
 - Embedding Models: Index Document objects with their embeddings (vector representations) when storing them in the vector store.
- The Q&A or retrieval stage of the RAG solution is supported by LangChain abstractions, including:
 - BasePromptTemplate: Models a generic prompt and produces a specific PromptValue instance after parameters are fed to it.
 - BaseRetriever: Abstracts the retrieval process from the vector store.
 - BaseLanguageModel: Abstracts clients to LLMs.
- A typical RAG chain includes a passthrough component, a vector store with a retriever, a prompt template, and an LLM or chat client.
- LLM-based chatbots retain memory of previous questions and responses, preserving context for ongoing queries until a satisfactory answer is achieved.
- Use LangChain's ChatPromptTemplate.from_messages to create prompts and ChatMessageHistory to manage chat history, feeding it into the RAG chain for each interaction between user and LLM.
- LangSmith allows you to analyze execution runs and traces, drilling down into each step to examine the input and output of each component.
- LangChain provides additional specific classes for building Q&A chatbots tailored to simpler use cases.

8 Advanced indexing

This chapter covers

- Advanced RAG techniques for more effective retrieval
- Selecting the optimal chunk splitting strategy for your use case
- Using multiple embeddings to enhance coarse chunk retrieval
- Expanding granular chunks to add context during retrieval
- Indexing strategies for semi-structured and multi-modal content

In chapter 7, you learned about the basics of the RAG (Retrieval-Augmented Generation) architecture, a common pattern for building LLM-based applications. To simplify things, I introduced a stripped-down version. While this was useful for learning, using this basic version in a real-world scenario will often produce poor results. You might notice inaccurate answers, even when your content store (usually a vector store) has relevant data. These issues often arise from poorly phrased queries, inefficient text indexing, or ineffective use of metadata attached to text chunks.

In this chapter, I'll show you how to avoid these issues and use advanced methods to improve accuracy. When working with LLM applications using LangChain, most of your time will be spent refining your design to get the best results. This involves selecting advanced RAG techniques, experimenting, and refining prompts. You can call yourself proficient only when you understand and apply these techniques effectively.

Here, we'll cover advanced indexing strategies, including creating multiple embeddings for larger text chunks stored in the vector database. Using this method boosts search precision and provides better context for generating high-quality responses.

8.1 Improving RAG Accuracy

To improve RAG (Retrieval-Augmented Generation) accuracy, analyze each

step in both the content ingestion and Q&A workflows. Each stage may pose specific issues, but it also opens up opportunities for improvement. Let's start with the content ingestion stage.

8.1.1 Content Ingestion Stage

Retrieval accuracy can be improved through an optimized content ingestion process that aligns with the specific features of each content store. Relying only on basic indexing can reduce indexing depth and weaken retrieval performance. Figure 8.1 shows two key areas for improvement in the ingestion stage: refining embedding calculations and optimizing how embeddings are linked to related text chunks in the vector store.

Figure 8.1 Common accuracy issues in the ingestion stage of a simple RAG architecture are often due to inadequate indexing that only uses basic embeddings for each text chunk. Advanced indexing techniques involve generating multiple embeddings for each chunk, enhancing searchability.

RAG Ingestion phase



1. request the embedding service to calculate the embeddings of the document chunks



Text ingestion script



Embeddings service

2. return embeddings



3. store documents chunks and related embeddings into Vector store



Typically we only calculate the embedding of the content of the text chunk. However we can calculate multiple embeddings to make the chunk more searchable

Advanced indexing

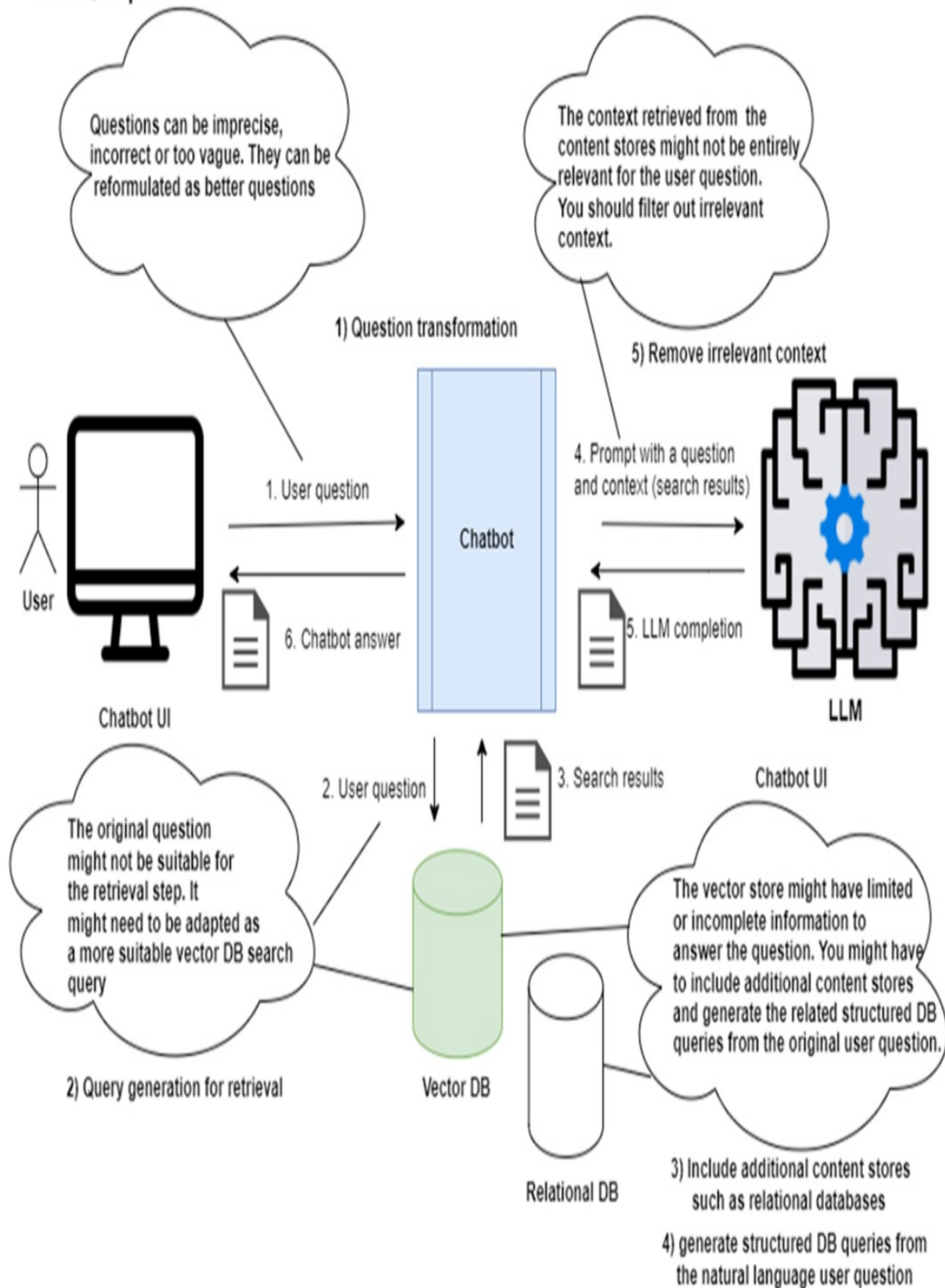
Even if a question is clear, retrieval can fail with overly simple indexing strategies. In vector indexing, chunk size and overlap length are crucial: smaller chunks may work well for precise questions but fail with broader queries, while larger chunks may lack detail for specific questions. To address this, you can add additional embeddings based on distinct chunk features, as illustrated in the figure above. This multi-faceted approach helps make each text chunk more adaptable and searchable. We'll explore advanced indexing techniques in this chapter.

8.1.2 Question Answering Stage

The effectiveness of the question-answering stage depends largely on how accurately the system interprets and processes user queries. Many potential issues, highlighted in Figure 8.2, can disrupt this process.

Figure 8.2 Common Issues in the Q&A stage of the naive RAG architecture and their solutions: 1) Handle poorly formulated questions with question transformation; 2) Enhance retrieval accuracy by transforming the original user questions into more suitable vector DB search queries; 3) Include relevant data sources by adding structured data content stores, such as relational DBs; 4) Generate DB queries for structured data content stores; 5) Filter out irrelevant context retrieved from the content stores.

RAG Q&A phase



Walking through the workflow of the Q&A stage of the RAG architecture, shown in the diagram above, you'll encounter several pitfalls that can lead to suboptimal answers. Each issue has a targeted solution:

- **Poor question formulation:** If a user's question is unclear, the vector store may return weak context, leading to poor results. The LLM struggles when working with unclear queries and subpar context. A fix is to rephrase the question into a clearer, more detailed form before passing it to the retrieval system.
- **Ineffective question for retrieval:** Using the original question for both retrieval and generation can fail, especially when the query is broad or abstract. Broad questions may not pinpoint relevant content, resulting in less accurate context. You can address this by breaking down broad questions into specific sub-questions to retrieve more precise information.
- **Limited data relevance in the content store:** Most RAG systems rely only on vector stores, but adding structured data sources, like relational databases, tables, or graph databases, can improve results. Route queries to the appropriate content store based on the type of data needed to enhance answer accuracy.
- **Limited querying capabilities against structured data:** While vector stores and LLMs excel with natural language, relational and graph databases don't process it directly, creating a barrier to using structured content. Use an LLM to generate structured queries (e.g., SQL) tailored to each data source to overcome this.
- **Irrelevant search results fed to the LLM:** Even with clearer questions and better indexing, irrelevant data can sometimes slip through, adding noise to the answer. Reduce this by applying filtering or post-processing steps to keep only the most relevant results.
- **Insufficient improvement in answer accuracy:** Sometimes fixing individual issues doesn't yield expected improvements. In these cases, boost accuracy by combining multiple techniques—such as advanced indexing, question transformations, and multi-store routing—into an ensemble strategy that maximizes precision.

The table below summarizes these issues and their solutions.

Table 8.1 Common issues in Naïve RAG architecture and recommended solutions

Issue	Solution
Retrieval returns the wrong content chunks	Advanced document indexing techniques
Poor question formulation	Question transformations
Ineffective question for retrieval	Question transformations
Limited data relevance in content store	Routing to multiple content stores
Limited querying capabilities for structured data	Content store query generation
Irrelevant retrieved results fed to LLM	Retrieval post-processing

This chapter, along with the next two, will dive into each problem and its solution in detail.

8.2 Advanced Document Indexing

For an LLM to generate high-quality answers, the relevance and accuracy of the text chunks retrieved from the vector store (or any document store) are critical. The quality of these chunks depends on several key factors:

- **Splitting Strategy:** The granularity of document chunks impacts retrieval accuracy. Smaller chunks yield more precise results for specific queries but lack broader context. Larger chunks offer richer context but may miss fine details. Choosing the right split size is essential and can be optimized using various techniques. Additional factors—such as chunk overlap and document hierarchy—also play a crucial role in balancing context and relevance.
- **Embedding Strategy:** How you index each chunk is equally important. You can use embeddings, metadata, or a combination. Advanced strategies use multiple indexes—such as child document embeddings, summaries, or hypothetical questions associated with a chunk—to capture both fine-grained details and broader context.
- **Sentence Expansion:** One method to deliver larger chunks without losing detail is to expand smaller chunks by including surrounding

sentences during retrieval. This approach provides additional context without sacrificing specificity.

- **Indexing Structured and Semi-Structured Data:** Retrieving structured data (e.g., database tables or multimedia content) using unstructured queries requires specialized techniques. This can include generating embeddings for database rows, images, or even audio files.

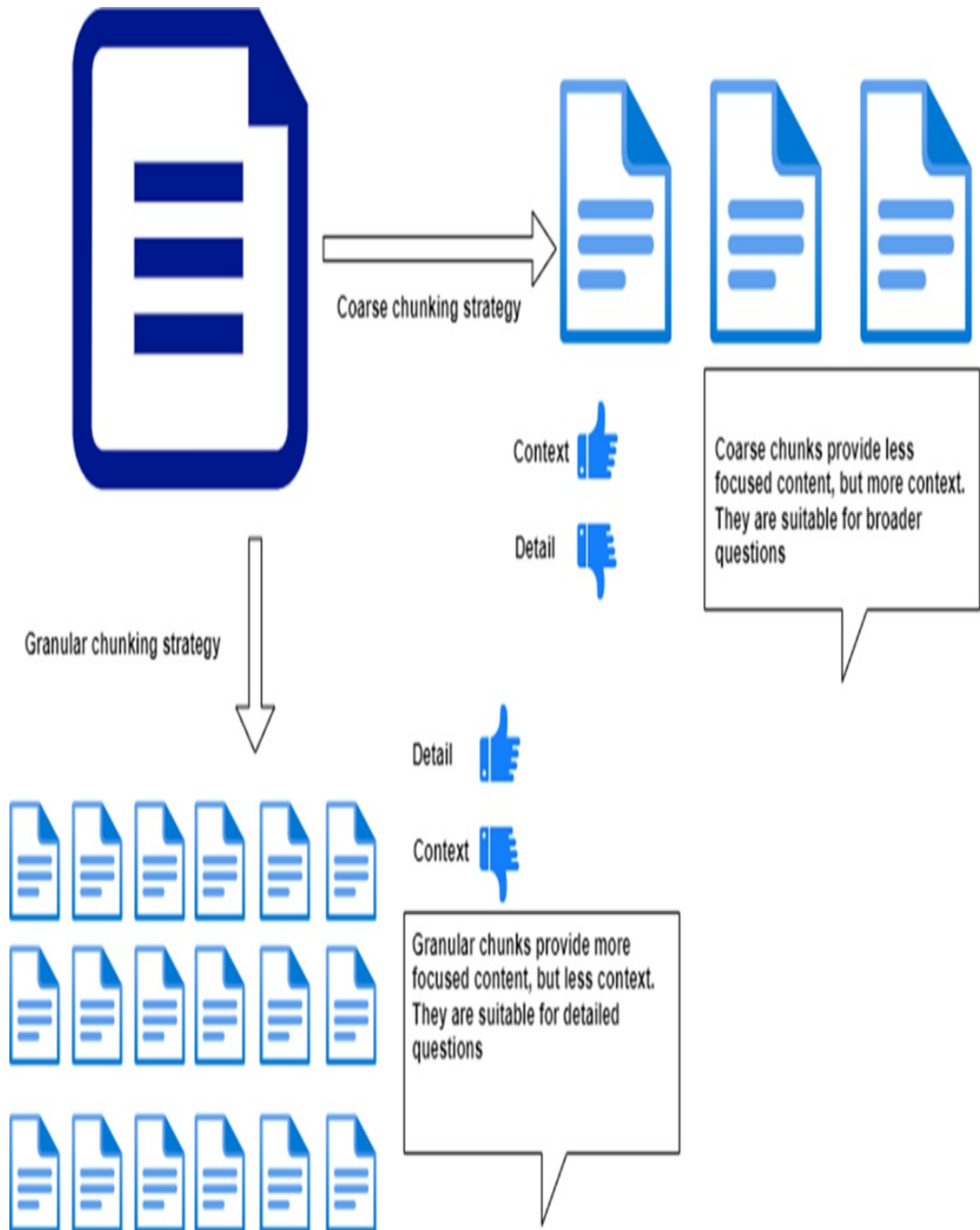
This chapter will cover each of these methods in detail, starting with strategies for optimal document splitting.

8.3 Splitting Strategy

During the ingestion phase of RAG, documents are split into chunks before being stored in a vector database or document store. Each chunk is indexed using embeddings and sometimes metadata (e.g., by tagging it with relevant keywords). Vector similarity searches rely on the embeddings index, while metadata searches use the keyword index.

The easiest way to improve the relevance of document chunk retrieval is to choose the right document splitting strategy for your use case. Ideally, the document store should return all relevant chunks that provide the LLM with enough context to generate accurate answers. The size of these chunks plays a critical role in retrieval performance, as shown in figure 8.3.

Figure 8.3 Impact of Chunk Size on Answer Accuracy. Coarse chunks provide more context but less focus, and they are suitable for broader questions. Granular chunks provide less context but more focus and they are suitable for more detailed questions.



Smaller, more granular chunks are better suited for answering detailed questions because they focus on specific topics. However, they contain less

surrounding context, which can reduce effectiveness for broader queries. In contrast, larger chunks are more effective for general questions since they provide more context but lose focus on fine-grained details.

The challenge is to balance chunk size based on expected question types. Small chunks work well when queries are precise, as the vector representation of the question will match closely with those of the relevant chunks. But for broader questions, small chunks might miss context, making retrieval less accurate. Larger chunks help cover broader topics but at the cost of losing detailed semantic information. The added context in these larger chunks, however, can be valuable during answer generation, as it provides the LLM with more background data to work with.

Finding the right balance between granular and coarse chunks depends on your use case. Ask yourself: are the questions likely to be detailed or broad? The chunk size should match the expected query type.

Splitting strategies

The size of the chunks isn't the only factor. You also need to decide *how* to split the document. There are two main approaches:

1. **Splitting by Document Hierarchy:** This approach respects the natural structure of the document (e.g., chapters, sections, paragraphs). It works well when the document is organized by topics, as chunks represent coherent subtopics. Tools like `HTMLHeaderTextSplitter` and `MarkdownHeaderTextSplitter` in `LangChain` target specific document types and maintain semantic accuracy. However, chunk sizes can vary greatly.
2. **Splitting by Absolute Size:** You can define chunk size by characters, tokens, sentences, or words. This results in more consistent chunk sizes, but context might be lost if chunks split mid-sentence. `CharacterTextSplitter` and its variants support different granularity levels, but you'll need to test for optimal size. Evaluating a range of fixed sizes is necessary to find what works best for your use case.

Factors to Consider

For each splitting strategy, keep in mind:

- **Document Type:** If you're working with mixed content (e.g., text, tables, images), maintaining related content within the same chunk is crucial. In this case, splitting by document hierarchy is more effective than a fixed-size approach.
- **Search Type:** if you are planning to use metadata search, keywords can be refined depending on the chunk granularity. You can also attach a mix of broad and detailed tags to each chunk to increase retrieval flexibility.

Choosing the Right Strategy

Selecting the best strategy often requires a mix of trial and error, but experience will eventually guide you to the right balance between document hierarchy and absolute size. Table 8.2 summarizes the pros and cons of each strategy and provides relevant LangChain classes to implement them.

Table 8.2 Splitting Strategies, Pros and Cons, and Related LangChain Classes

Splitting Strategy	Pros	Cons	LangChain Classes
Document Hierarchy	More accurate semantic meaning	Chunk size varies significantly	HTMLHeaderTextSplitter, MarkdownHeaderTextSplitter
By Size - Number of Tokens	Consistent chunk size	Incomplete sentences at boundaries	CharacterTextSplitter.from_tiktoken_encoder
By Size - Number of Characters	Consistent chunk size	Incomplete sentences may reduce semantic value	CharacterTextSplitter
By Sentence, Paragraph,	Retains semantic meaning	Small chunks may lack enough	RecursiveCharacterTextSplitter

or Word	in most cases	context	
---------	---------------	---------	--

Each method has its use case, and the choice depends on your specific needs and document structure. In the following sections, I'll cover these strategies in detail, starting with document hierarchy splitting.

8.3.1 Splitting by HTML Header

In this section, I'll show you how to split documents using the `HTMLHeaderTextSplitter` class on various online documents from Wikivoyage.org, a travel-focused site from the same group as Wikipedia. We'll explore how different levels of splitting granularity affect the accuracy of responses.

First, set up your environment by creating a new folder and virtual environment, and then install the required packages. If you've cloned the repository from GitHub, you can install dependencies directly with the provided requirements file:

```
C:\Github\building-llm-applications\ch08> python -m venv env_ch08
C:\Github\building-llm-applications\ch08> env_ch08\Scripts\activate
(env_ch08) C:\Github\building-llm-applications\ch08> pip install
```

Next, start a new Jupyter notebook or open the existing one in the cloned repository:

```
(env_ch08) C:\Github\building-llm-applications\ch08> jupyter note
```

Save the notebook as `08-advanced_indexing.ipynb` and import the required libraries:

```
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings
import getpass
```

```
OPENAI_API_KEY = getpass.getpass('Enter your OPENAI_API_KEY')
```

Setting Up Chroma DB Collections

Now, create a Chroma DB collection to store the more granular chunks:

```
cornwall_granular_collection = Chroma( #A
    collection_name="cornwall_granular",
    embedding_function=OpenAIEmbeddings(openai_api_key=OPENAI_API
)

cornwall_granular_collection.reset_collection() #B
```

This will initialize a new Chroma collection called `cornwall_granular_collection`. If the collection already exists, it will be reset to start fresh.

Next, set up a second collection for coarser chunks:

```
cornwall_coarse_collection = Chroma( #A
    collection_name="cornwall_coarse",
    embedding_function=OpenAIEmbeddings(openai_api_key=OPENAI_API
)

cornwall_coarse_collection.reset_collection() #B
```

Loading HTML Content with AsyncHtmlLoader

The final step is to ingest some content about Cornwall, a region in the UK known for its stunning seaside resorts, using an HTML loader:

```
from langchain_community.document_loaders import AsyncHtmlLoader
destination_url = "https://en.wikivoyage.org/wiki/Cornwall"
html_loader = AsyncHtmlLoader(destination_url)
docs = html_loader.load()
```

This snippet fetches the Cornwall page content, which we'll use to create both granular and coarse chunks. In the following steps, I'll show how to split the content based on HTML headers and analyze the impact on retrieval accuracy.

Splitting Content into Granular Chunks Using HTMLSectionSplitter

Before storing the content from the ``docs`` object into the vector database, decide on a splitting strategy. Since this content is from an HTML page, you

can split it by H1 and H2 tags, which separate the content into sections. This will generate more granular chunks, as shown in listing 8.1.

Listing 8.1 Splitting content with the HTML Section splitter

```
from langchain_text_splitters import HTMLSectionSplitter

headers_to_split_on = [("h1", "Header 1"), ("h2", "Header 2")]
html_section_splitter = HTMLSectionSplitter(headers_to_split_on=h

def split_docs_into_granular_chunks(docs):
    all_chunks = []
    for doc in docs:
        html_string = doc.page_content #A
        temp_chunks = html_section_splitter.split_text(html_strin
        all_chunks.extend(temp_chunks)
    return all_chunks
```

You can now generate the granular chunks:

```
granular_chunks = split_docs_into_granular_chunks(docs)
```

Now, insert the granular chunks into the Chroma collection:

```
cornwall_granular_collection.add_documents(documents=granular_chu
```

Searching Granular Chunks

You can now run a search for specific content within the granular chunks:

```
results = cornwall_granular_collection.similarity_search(query="E
for doc in results:
    print(doc)
```

Splitting Content into Coarse Chunks Using RecursiveCharacterTextSplitter

For larger, coarser chunks, use the RecursiveCharacterTextSplitter class. Start by creating the necessary objects:

```
from langchain_community.document_transformers import Html2TextTr
```

```

from langchain_text_splitters import RecursiveCharacterTextSplitt

html2text_transformer = Html2TextTransformer()
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=3000, chunk_overlap=300
)

```

Next, define a function to split the content into coarse chunks:

```

def split_docs_into_coarse_chunks(docs):
    text_docs = html2text_transformer.transform_documents(docs)
    coarse_chunks = text_splitter.split_documents(text_docs)  #B
    return coarse_chunks

```

Then generate the coarse chunks:

```
coarse_chunks = split_docs_into_coarse_chunks(docs)
```

Insert these chunks into the corresponding Chroma collection:

```
cornwall_coarse_collection.add_documents(documents=coarse_chunks)
```

Searching Coarse Chunks

You can now search for more general content within the coarse chunks:

```

results = cornwall_coarse_collection.similarity_search(query="Eve
for doc in results:
    print(doc)

```

Ingesting Content from Multiple URLs

To make the searches more comprehensive, load additional content into the collections. Listing 8.2 shows how to set up new granular and coarse collections for various UK destinations and ingest the related content chunks. If you'd like to minimize processing costs, consider reducing the size of the `uk_destinations` list.

Listing 8.2 Creating collections for multiple UK destinations

```
uk_granular_collection = Chroma(  #A
```

```

        collection_name="uk_granular",
        embedding_function=OpenAIEmbeddings(openai_api_key=OPENAI_API
    )
    uk_granular_collection.reset_collection() #B

    uk_coarse_collection = Chroma( #A
        collection_name="uk_coarse",
        embedding_function=OpenAIEmbeddings(openai_api_key=OPENAI_API
    )
    uk_coarse_collection.reset_collection() #B

    uk_destinations = [
        "Cornwall", "North_Cornwall", "South_Cornwall", "West_Cornwal
        "Tintagel", "Bodmin", "Wadebridge", "Penzance", "Newquay",
        "St_Ives", "Port_Isaac", "Looe", "Polperro", "Porthleven",
        "East_Sussex", "Brighton", "Battle", "Hastings_(England)",
        "Rye_(England)", "Seaford", "Ashdown_Forest"
    ]

    wikivoyage_root_url = https://en.wikivoyage.org/wiki

    uk_destination_urls = [f'{wikivoyage_root_url}/{d}' for d in uk_d

    for destination_url in uk_destination_urls:
        html_loader = AsyncHtmlLoader(destination_url) #C
        docs = html_loader.load() #D

        granular_chunks = split_docs_into_granular_chunks(docs)
        uk_granular_collection.add_documents(documents=granular_chunk

        coarse_chunks = split_docs_into_coarse_chunks(docs)
        uk_coarse_collection.add_documents(documents=coarse_chunks)

```

You can now perform both granular and coarse searches:

```

granular_results = uk_granular_collection.similarity_search(query
for doc in granular_results:
    print(doc)

coarse_results = uk_coarse_collection.similarity_search(query="Ev
for doc in coarse_results:
    print(doc)

```

Try experimenting with different queries, like "Beaches in Cornwall", to see how the results vary between granular and coarse chunks. This approach will help you fine-tune the balance between detailed and general content

retrieval.

8.4 Embedding Strategy

Previously, I discussed how keyword searches are more flexible than vector searches because you can tag a document with multiple keywords. The same idea can be applied to embeddings—you can store multiple vectors per document, which increases the flexibility and accuracy of vector searches. I'll walk through various multi-vector strategies in the following sections, mainly focusing on how to use LangChain's `MultiVectorRetriever`.

The key to these strategies is a two-layer chunk structure. The top layer includes *synthesis chunks*—the chunks fed into the LLM to generate answers. The lower layer consists of *retrieval chunks*, smaller segments that create precise embeddings for retrieving the synthesis chunks. I suggest testing all multi-vector retriever techniques for your use case, as they usually yield similar performance improvements. However, since the structure of your text may favor one technique over another, experimenting with each will help you identify the best fit.

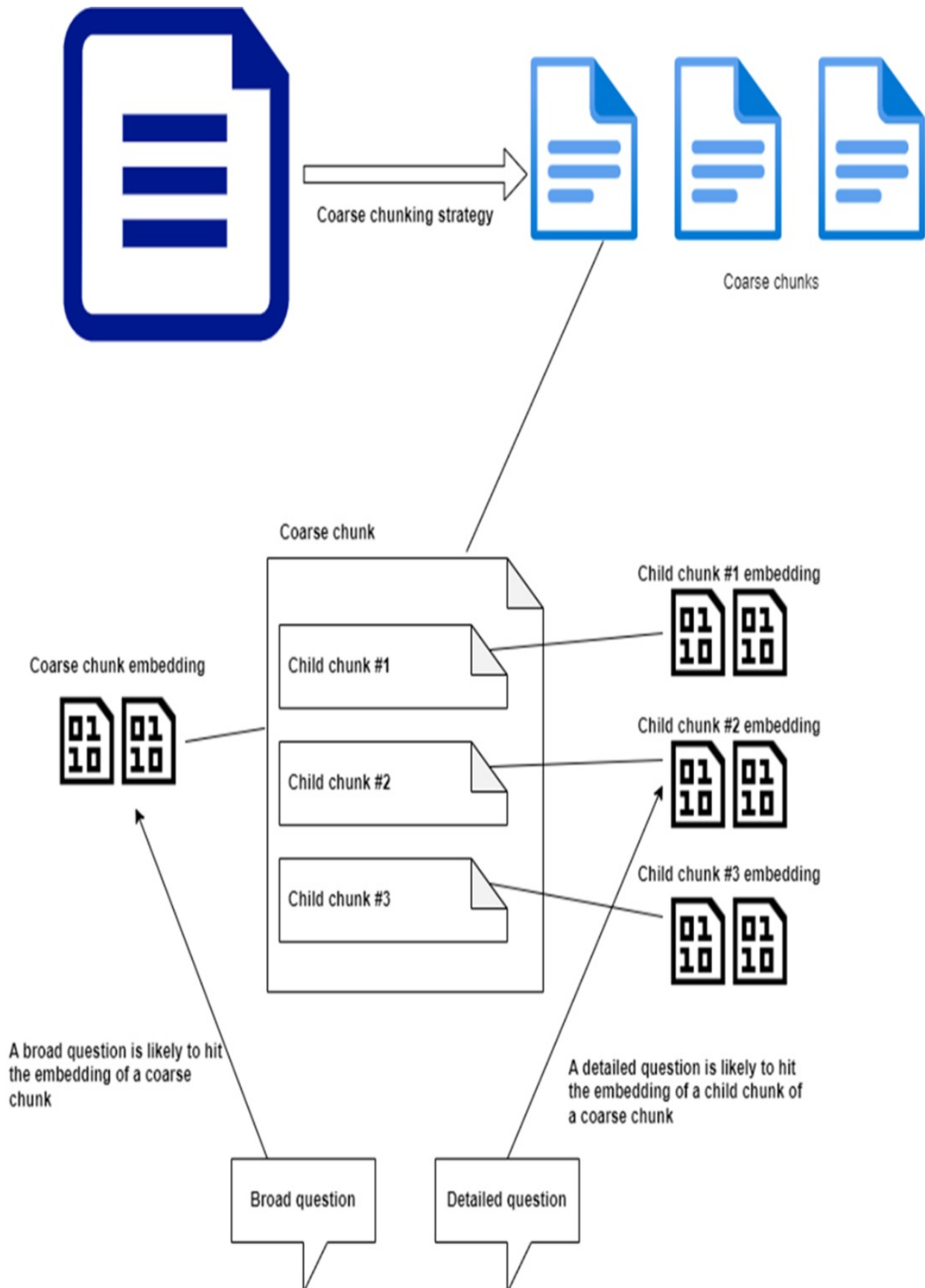
8.4.1 Embedding Child Chunks with `ParentDocumentRetriever`

A common challenge with chunk size is balancing between context and detail. Large chunks work for broad questions but struggle with detailed queries. Small chunks, while supporting detailed queries, often lack the context needed for generating comprehensive answers. This creates a tradeoff: if chunks are too small, the response might be incomplete; if they're too large, the retrieval may be less precise.

To solve this, split the document into larger *parent* chunks and create smaller *child* chunks within each parent. Use the child chunks solely for generating more granular embeddings, which are then stored against the parent chunk. This hybrid approach allows each document to have embeddings for both broad and detailed queries, as illustrated in figure 8.4.

Figure 8.4 Child Chunk embeddings—A coarse document chunk is indexed with its own embedding and the embeddings generated from its smaller child chunks. This allows it to match

effectively with both broad and detailed queries.



The advantage of this approach is that when a broad query is made, the parent chunk is retrieved, providing rich context. For more detailed queries, the child embeddings ensure precise matching which still lead to the retrieval of the context-rich parent chunk. This structure helps the LLM generate more accurate and contextually relevant answers.

I'll show you how to implement the technique using `ParentDocumentRetriever`. Start by importing the necessary libraries:

```
from langchain.retrievers import ParentDocumentRetriever
from langchain.storage import InMemoryStore
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings
from langchain_community.document_loaders import AsyncHtmlLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

Setting Up the `ParentDocumentRetriever`

The approach begins by splitting content into large, coarse chunks for synthesis, then further dividing each into smaller child chunks for retrieval. Listing 8.3 demonstrates how to configure the splitters and set up the retriever. As shown, documents are stored in an `InMemoryStore`—a general-purpose, in-memory key-value store designed to hold serializable Python objects such as strings, lists, and dictionaries. It's particularly useful for caching and intermediate data storage.

Listing 8.3 Setting Up Parent and Child Splitters for Coarse and Granular Chunks

```
parent_splitter = RecursiveCharacterTextSplitter(chunk_size=3000)
child_splitter = RecursiveCharacterTextSplitter(chunk_size=500)

child_chunks_collection = Chroma( #C
    collection_name="uk_child_chunks",
    embedding_function=OpenAIEmbeddings(openai_api_key=OPENAI_API_KEY)
)

child_chunks_collection.reset_collection() #D

doc_store = InMemoryStore() #E
```



```
parent_doc_retriever = ParentDocumentRetriever( #F
    vectorstore=child_chunks_collection,
    docstore=doc_store,
    child_splitter=child_splitter,
    parent_splitter=parent_splitter
)
```

Ingesting Content into Document and Vector Stores

Now, generate both coarse and granular chunks using the configured splitters and add them to the respective stores.

```
for destination_url in uk_destination_urls:
    html_loader = AsyncHtmlLoader(destination_url) #A
    html_docs = html_loader.load() #B
    text_docs = html2text_transformer.transform_documents(html_do

    print(f'Ingesting {destination_url}')
    parent_doc_retriever.add_documents(text_docs, ids=None) #D
```

Verifying the In-Memory Document Store

You can check if the coarse chunks have been correctly stored:

```
list(doc_store.yield_keys())
```

Performing a Search on Granular Information

Now perform a search on the child chunks using the ParentDocumentRetriever:

```
retrieved_docs = parent_doc_retriever.invoke("Cornwall Ranger")
```

The first document retrieved (retrieved_docs[0]) will contain rich contextual information:

```
Document(metadata={'source': 'https://en.wikivoyage.org/wiki/Sout
```

Comparing with Direct Semantic Search on Child Chunks

Now, compare the results by directly searching only the child chunks:

```
child_docs_only = child_chunks_collection.similarity_search("Corn
```

The first result is much shorter and lacks context:

```
Document(metadata={'doc_id': '34645d23-ed05-4a53-b3af-c8ab21e3f51',
page_content='The Cornwall Ranger ticket allows unlimited tra
```

The result we have obtained with the `ParentDocumentRetriever` is particularly useful when used as context for LLM synthesis, as it provides broader details around the specific information.

Next, I'll introduce another technique that leverages embedding strategies to further optimize RAG retrieval accuracy.

8.4.2 Embedding Child Chunks with `MultiVectorRetriever`

An alternative method for embedding child chunks and linking them to the larger parent chunks used in synthesis is to use the `MultiVectorRetriever`. Begin by importing the necessary libraries, including `InMemoryByteStore`, which is specifically designed for storing binary data. In this store, keys are strings and values are bytes, making it ideal for use cases involving embeddings, models, or files where raw byte storage is preferred or required:

```
from langchain.retrievers.multi_vector import MultiVectorRetriever
from langchain.storage import InMemoryByteStore
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings
from langchain_community.document_loaders import AsyncHtmlLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
import uuid
```

Setting Up the `MultiVectorRetriever`

You can use a similar approach as with the `ParentDocumentRetriever` by defining parent and child splitters and injecting them into `MultiVectorRetriever`, as shown in Listing 8.4.

Listing 8.4 Configuring Parent and Child Splitters for `MultiVectorRetriever`

```

parent_splitter = RecursiveCharacterTextSplitter(chunk_size=3000)
child_splitter = RecursiveCharacterTextSplitter(chunk_size=500)

child_chunks_collection = Chroma( #C
    collection_name="uk_child_chunks",
    embedding_function=OpenAIEmbeddings(openai_api_key=OPENAI_API
)

child_chunks_collection.reset_collection() #D

doc_byte_store = InMemoryByteStore() #E
doc_key = "doc_id"

multi_vector_retriever = MultiVectorRetriever( #F
    vectorstore=child_chunks_collection,
    byte_store=doc_byte_store
)

```

Ingesting Content into Document and Vector Stores

The next step is to load and ingest content into the `MultiVectorRetriever`, as shown in listing 8.5. In this case, the document store is handled internally by the retriever, unlike the `ParentDocumentRetriever`, where you configured the store separately. While ingestion may feel slower, this is expected due to the added complexity of managing multiple vector representations.

Listing 8.5 Ingesting Content into Document and Vector Stores

```

for destination_url in uk_destination_urls:
    html_loader = AsyncHtmlLoader(destination_url) #A
    html_docs = html_loader.load() #B
    text_docs = html2text_transformer.transform_documents(html_do

    coarse_chunks = parent_splitter.split_documents(text_docs) #

    coarse_chunks_ids = [str(uuid.uuid4()) for _ in coarse_chunks
    all_granular_chunks = []
    for i, coarse_chunk in enumerate(coarse_chunks): #E

        coarse_chunk_id = coarse_chunks_ids[i]

        granular_chunks = child_splitter.split_documents([coarse_

        for granular_chunk in granular_chunks:

```

```

        granular_chunk.metadata[doc_key] = coarse_chunk_id #
    all_granular_chunks.extend(granular_chunks)

    print(f'Ingesting {destination_url}')
    multi_vector_retriever.vectorstore.add_documents(all_granular_chunks)
    multi_vector_retriever.docstore.mset(list(zip(coarse_chunks_ids, all_granular_chunks)))

```

Performing a search on granular information

Now, perform a search using MultiVectorRetriever, just like you did with the ParentDocumentRetriever:

```
retrieved_docs = multi_vector_retriever.invoke("Cornwall Ranger")
```

Printing the first result shows that the retrieved document contains rich, detailed information:

```
Document(metadata={'source': 'https://en.wikivoyage.org/wiki/South_West_of_England'}, page_content="Trains from London take about 3 hr 20 min to Plymouth")
```

Comparing with Direct Semantic Search on Child Chunks

For comparison, run the same search directly on the child chunk collection:

```
child_docs_only = child_chunks_collection.similarity_search("Cornwall Ranger")
```

The first document retrieved from the child collection (child_docs_only[0]) is more concise and lacks the broader context:

```
Document(metadata={'doc_id': '04c7f88e-e090-4057-af5b-ea584e777b3'}, page_content='The **Cornwall Ranger** ticket allows unlimited travel on the Cornwall Ranger train')

```

This result is similar to what you observed with the ParentDocumentRetriever. The broader parent chunks provide more useful context for synthesis, making them a better fit for complex or detailed queries.

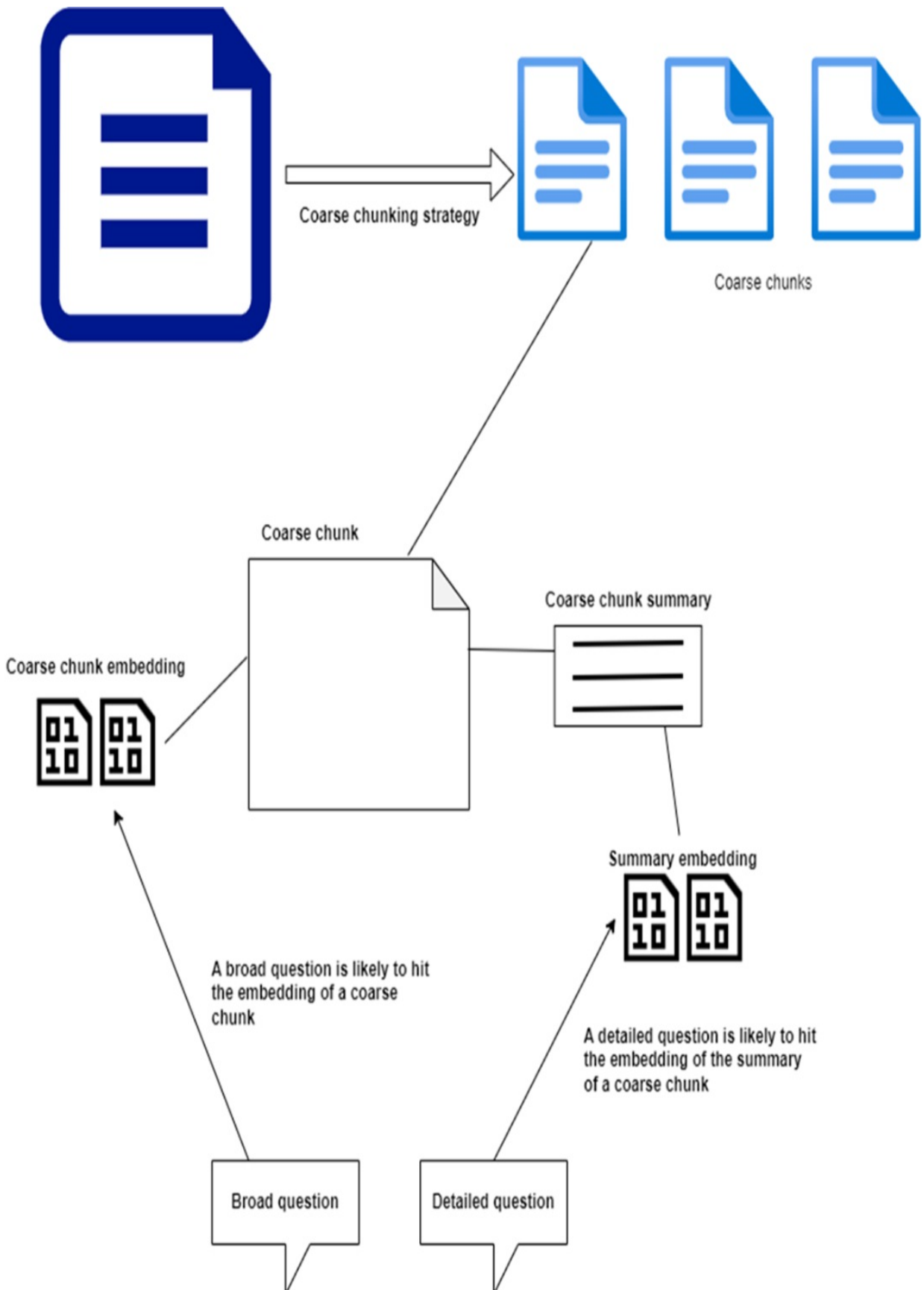
In the next section, I'll cover additional strategies for improving RAG accuracy using advanced embedding techniques.

8.4.3 Embedding Document Summaries

Embeddings from coarse chunks are often ineffective because they capture too much irrelevant content. A large chunk may include filler text or minor details that dilute the semantic value of the embeddings, making them less focused and less useful.

To address this, you can create a summary of the coarse chunk and generate embeddings from it. These summary embeddings are then stored alongside the original chunk embeddings, as shown in figure 8.5. Because the summary is more concise and relevant, the resulting embeddings are denser and more effective for retrieval, reducing noise and improving search precision.

Figure 8.5 Chunk Summary Embedding—A coarse chunk is indexed with its own embedding and an additional embedding from its summary. This allows for more accurate retrieval when answering detailed questions.



To start, import the required libraries for building the summarization and retrieval setup:

```
from langchain.retrievers.multi_vector import MultiVectorRetrieve
from langchain.storage import InMemoryByteStore
from langchain_chroma import Chroma
from langchain_openai import ChatOpenAI
from langchain_openai import OpenAIEmbeddings
from langchain_community.document_loaders import AsyncHtmlLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.documents import Document
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
import uuid
```

Setting Up the MultiVectorRetriever

First, create a collection to store the summaries and set up the document store (InMemoryByteStore). Then, configure the MultiVectorRetriever to use these components, as shown in Listing 8.6.

Listing 8.6 Injecting summary collection and document store into MultiVectorRetriever

```
parent_splitter = RecursiveCharacterTextSplitter(chunk_size=3000)

summaries_collection = Chroma( #B
    collection_name="uk_summaries",
    embedding_function=OpenAIEmbeddings(openai_api_key=OPENAI_API_KEY)
)

summaries_collection.reset_collection() #C

doc_byte_store = InMemoryByteStore() #D
doc_key = "doc_id"

multi_vector_retriever = MultiVectorRetriever( #E
    vectorstore=summaries_collection,
    byte_store=doc_byte_store
)
```

Setting Up the Summarization Chain

Use an LLM to generate summaries of the coarse chunks. Define a summarization chain that extracts the content, prompts the LLM, and parses the response into a usable format:

```
llm = ChatOpenAI(model="gpt-4o-mini", openai_api_key=OPENAI_API_K
summarization_chain = (
    {"document": lambda x: x.page_content} #A
    | ChatPromptTemplate.from_template("Summarize the following d
    | llm #C
    | StrOutputParser()) #D
```

Ingesting Coarse Chunks and Summaries into Stores

Next, load the content, split it into coarse chunks, generate summaries, and store both in the appropriate collections, as shown in listing 8.7.

Listing 8.7 Ingesting Coarse Chunks and Their Summaries

```
for destination_url in uk_destination_urls:
    html_loader = AsyncHtmlLoader(destination_url) #A
    html_docs = html_loader.load() #B
    text_docs = html2text_transformer.transform_documents(html_do

    coarse_chunks = parent_splitter.split_documents(text_docs) #

    coarse_chunks_ids = [str(uuid.uuid4()) for _ in coarse_chunks
    all_summaries = []
    for i, coarse_chunk in enumerate(coarse_chunks): #E

        coarse_chunk_id = coarse_chunks_ids[i]

        summary_text = summarization_chain.invoke(coarse_chunk)
        summary_doc = Document(page_content=summary_text, metadat

        all_summaries.append(summary_doc) #G

    print(f'Ingesting {destination_url}')
    multi_vector_retriever.vectorstore.add_documents(all_summarie
    multi_vector_retriever.docstore.mset(list(zip(coarse_chunks_i
```

When running the code in listing 8.6, you may notice that processing is slower compared to using child embeddings. This slowdown is due to the

time required to submit each coarse chunk for summarization to the LLM, which is a more computationally intensive step.

Performing a Search Using the MultiVectorRetriever

Once the ingestion is complete, you can perform a search using the MultiVectorRetriever, which now utilizes the summaries for each travel destination:

```
retrieved_docs = multi_vector_retriever.invoke("Cornwall travel")
```

If you print the first result (`retrieved_docs_only[0]`), you'll see a large chunk similar to those retrieved when using child embeddings. These larger chunks provide more context, making them effective when passed as input to the LLM.

Comparing with Direct Semantic Search on Summaries

For comparison, perform a direct search on the summaries alone:

```
summary_docs_only = summaries_collection.similarity_search("Cornw  
print(summary_docs_only[0])
```

The first result from the summary search is concise and lacks broader context:

```
Document(metadata={'doc_id': 'ee55d250-bc53-46ce-9204-8fd2c1a0566
```

This result confirms a pattern observed earlier: directly searching against summaries or child chunks retrieves focused but context-limited information, while using a multi-vector approach retrieves broader context that is more useful for synthesis.

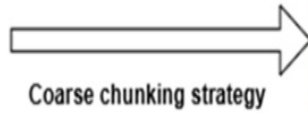
Let's explore one more advanced multi-vector embedding technique in the following section.

8.4.4 Embedding Hypothetical Questions

When querying a vector store, your natural language question is converted into a vector, and the system calculates its similarity (e.g., cosine distance) to the stored vectors. The documents linked to the closest vectors are then returned. This approach works well if the question is semantically similar to the ideal answer. But often, the wording of the question and the phrasing of the ideal answer may not match closely enough, causing the search to miss relevant documents.

To address this, you can generate hypothetical questions that each chunk is likely to answer, and then store the chunk using embeddings derived from these questions, as shown in figure 8.6. This method increases the chances that the stored vectors will align more closely with a user's query, making it more likely to retrieve relevant information, even if the original document's embeddings aren't a perfect match for the question.

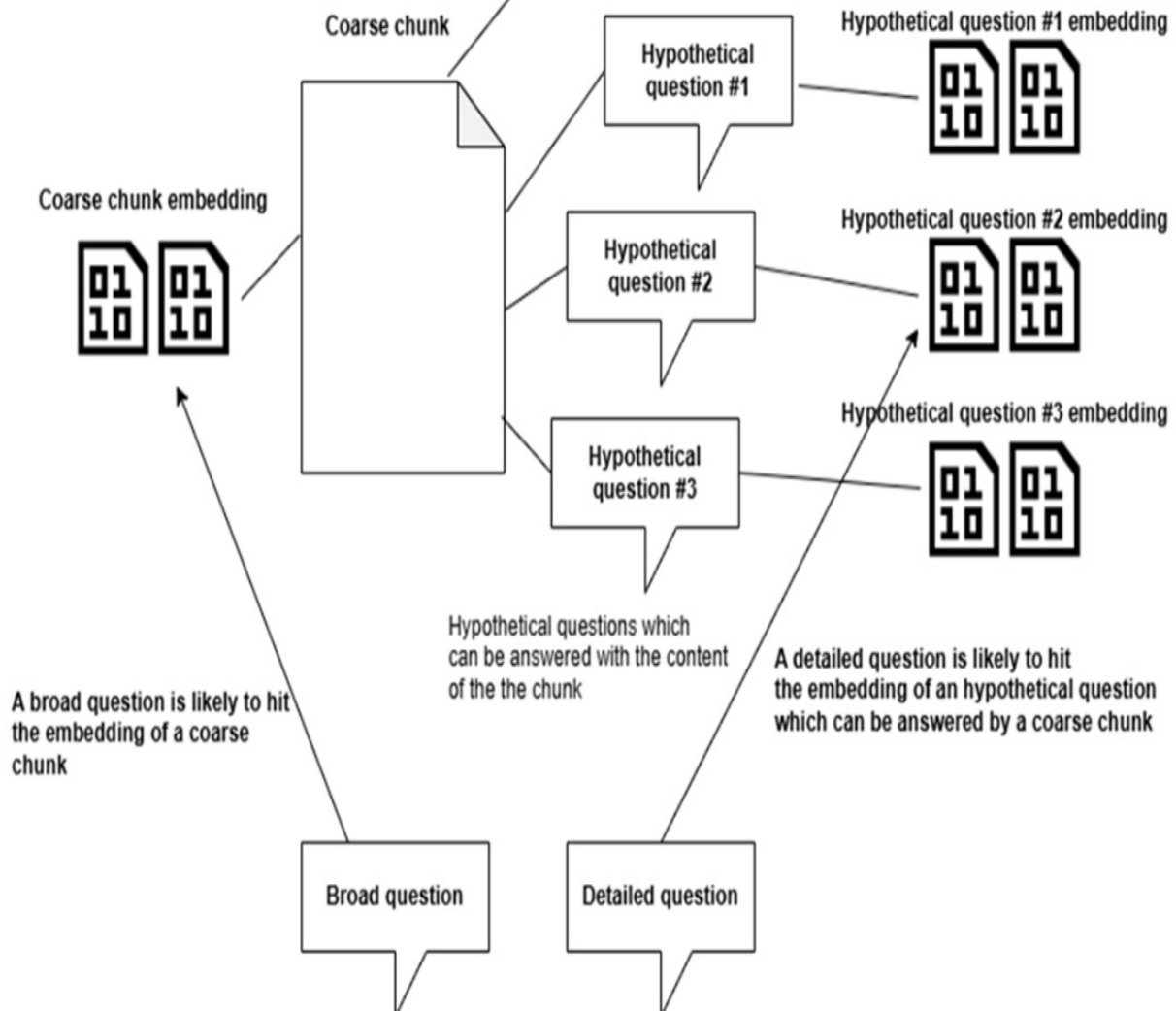
Figure 8.6 Hypothetical Question embeddings—A document chunk is indexed with its own embedding and additional embeddings generated from hypothetical questions that it can answer. This allows for more accurate matching with user queries.



Coarse chunking strategy



Coarse chunks



To begin, import all the required libraries to set up the MultiVectorRetriever with hypothetical question embeddings:

```
from langchain.retrievers.multi_vector import MultiVectorRetrieve
from langchain.storage import InMemoryByteStore
from langchain_chroma import Chroma
from langchain_openai import ChatOpenAI
from langchain_openai import OpenAIEmbeddings
from langchain_community.document_loaders import AsyncHtmlLoader
from langchain_text_splitters import RecursiveCharacterTextSplitt
from langchain_core.documents import Document
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
import uuid
from typing import List
from pydantic import BaseModel, Field
```

Setting Up the MultiVectorRetriever

Set up the MultiVectorRetriever similarly to how it was configured for summary embeddings, but this time use a vector store specifically for storing hypothetical questions, as shown in Listing 8.8.

Listing 8.8 Configuring MultiVectorRetriever with Hypothetical Question Collection

```
parent_splitter = RecursiveCharacterTextSplitter(chunk_size=3000)

hypothetical_questions_collection = Chroma( #B
    collection_name="uk_hypothetical_questions",
    embedding_function=OpenAIEmbeddings(openai_api_key=OPENAI_API
)

hypothetical_questions_collection.reset_collection() #C

doc_byte_store = InMemoryByteStore() #D
doc_key = "doc_id"

multi_vector_retriever = MultiVectorRetriever( #E
    vectorstore=hypothetical_questions_collection,
    byte_store=doc_byte_store
)
```

Setting Up the Hypothetical Question Generation Chain

Create a chain to generate hypothetical questions for each document chunk. Use structured output from the LLM to ensure the generated questions are returned as a list of strings:

```
class HypotheticalQuestions(BaseModel):
    """Generate hypothetical questions for given text."""

    questions: List[str] = Field(..., description="List of hypoth

llm_with_structured_output = ChatOpenAI(model="gpt-4o-mini",
    openai_api_key=OPENAI_API_KEY).with_structured_output(
    HypotheticalQuestions
)
```

You can see the full question generation chain in listing 8.9.

Listing 8.9 Chain for Generating Hypothetical Questions from Text

```
hypothetical_questions_chain = (
    {"document_text": lambda x: x.page_content} #A
    | ChatPromptTemplate.from_template( #B
        "Generate a list of exactly 4 hypothetical questions that
    )
    | llm_with_structured_output #C
    | (lambda x: x.questions) #D
)
```

Ingesting Coarse Chunks and Related Hypothetical Questions

Now, generate coarse chunks, create the hypothetical questions for each, and store them in the respective collections, as shown in listing 8.10.

Listing 8.10 Ingesting Coarse Chunks and Related Hypothetical Questions

```
for destination_url in uk_destination_urls:
    html_loader = AsyncHtmlLoader(destination_url) #A
    html_docs = html_loader.load() #B
    text_docs = html2text_transformer.transform_documents(html_do

    coarse_chunks = parent_splitter.split_documents(text_docs) #
```

```

coarse_chunks_ids = [str(uuid.uuid4()) for _ in coarse_chunks]
all_hypothetical_questions = []
for i, coarse_chunk in enumerate(coarse_chunks): #E

    coarse_chunk_id = coarse_chunks_ids[i]

    hypothetical_questions = hypothetical_questions_chain.invoke(
        coarse_chunk_id,
        hypothetical_questions_docs = [Document(page_content=ques
                                                for question in hyp

    all_hypothetical_questions.extend(hypothetical_questions_

print(f'Ingesting {destination_url}')
multi_vector_retriever.vectorstore.add_documents(all_hypothet
multi_vector_retriever.docstore.mset(list(zip(coarse_chunks_i

```

Performing a Search Using the MultiVectorRetriever

After ingestion, perform a search using the MultiVectorRetriever, which now uses the stored hypothetical question embeddings:

```
retrieved_docs = multi_vector_retriever.invoke("How can you go to
```

The first retrieved document (retrieved_docs[0]) is a detailed response with rich contextual information:

```
[Document(metadata={'source': 'https://en.wikivoyage.org/wiki/Bri
```

Comparing with Direct Search on Hypothetical Questions

Now, run a search directly on the hypothetical questions collection for comparison:

```
hypothetical_question_docs_only = hypothetical_questions_collecti
```

The results show hypothetical questions closely matching the query:

```

[Document(metadata={'doc_id': 'af848894-8591-4c28-8295-f3b833ffaa
Document(metadata={'doc_id': '7fa14e56-270c-4461-88ab-9b546afb07
Document(metadata={'doc_id': '7fa14e56-270c-4461-88ab-9b546afb07
Document(metadata={'doc_id': '7fa14e56-270c-4461-88ab-9b546afb07

```

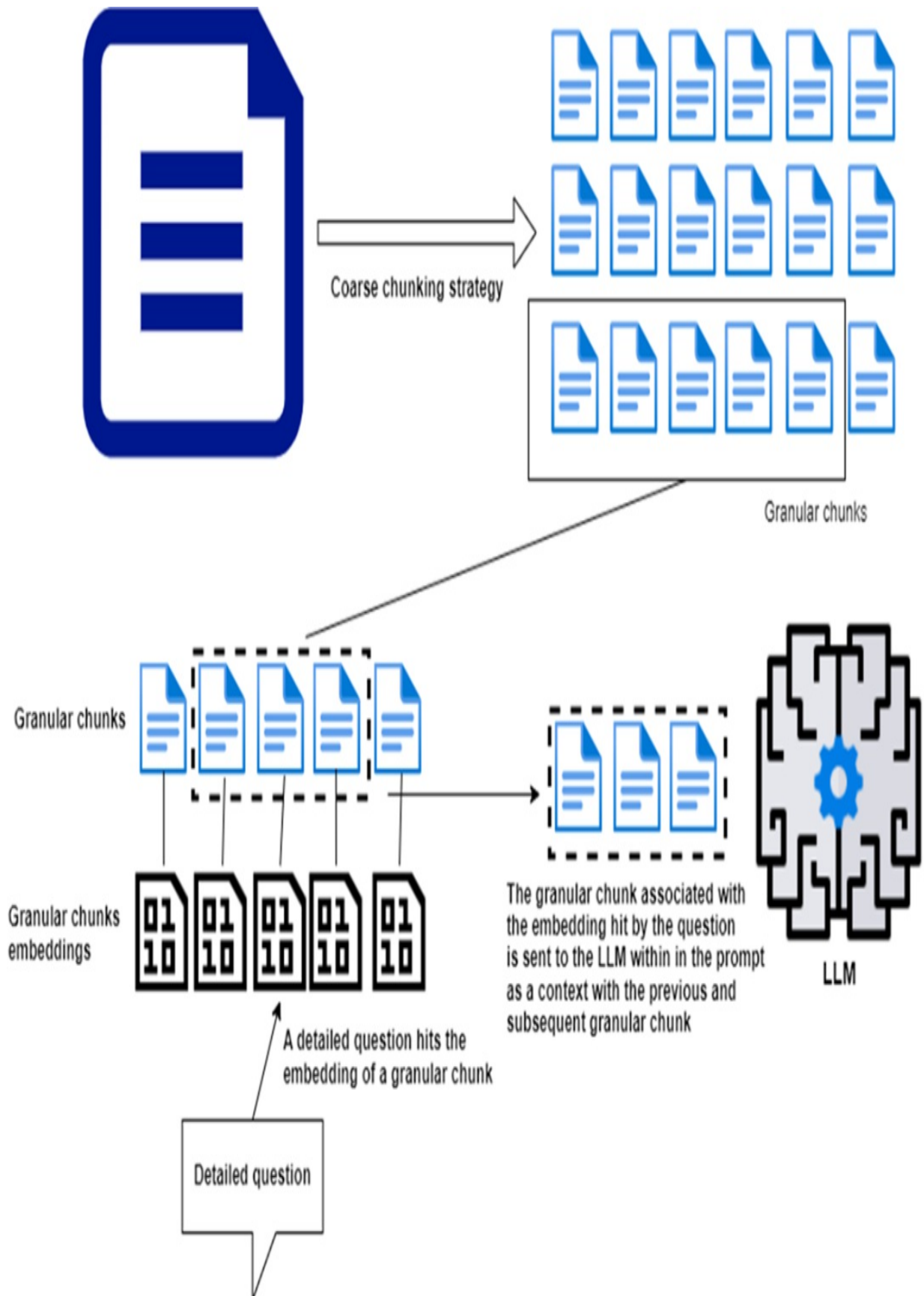
This pattern is consistent with earlier techniques: using hypothetical question embeddings helps the search engine focus on the specific intent behind the query, making it easier to retrieve relevant documents even when the exact wording varies.

In the next section, I'll explore another technique to further improve document retrieval accuracy.

8.5 Granular Chunk Expansion

As previously discussed, the main drawback of splitting a document into small granular chunks is that, while these chunks are effective for detailed questions, they often lack the context needed to generate complete answers. One way to address this is through *chunk expansion*, as shown in Figure 8.7.

Figure 8.7 Sentence Expansion—A granular chunk can be enhanced by including the content from its preceding and following chunks to provide additional context.



The idea is to store an expanded version of each chunk that includes content from the chunks immediately before and after it. This expanded version is stored in a separate document store. So, when the vector store retrieves a relevant granular chunk, the linked expanded chunk is returned instead, offering a richer context for the LLM to produce a more complete answer.

This technique can be easily implemented using the `MultiVectorRetriever`. Let's look at how to set this up.

Setting Up the `MultiVectorRetriever` for Chunk Expansion

First, configure the `MultiVectorRetriever` by creating a collection to hold granular chunks and an in-memory document store for the expanded chunks, as shown in listing 8.11:

Listing 8.11 Setting up a multi vector retriever for granular chunk expansion

```
from langchain.retrievers.multi_vector import MultiVectorRetrieve
from langchain.storage import InMemoryByteStore
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings
from langchain_community.document_loaders import AsyncHtmlLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
import uuid

granular_chunk_splitter = RecursiveCharacterTextSplitter(chunk_si

granular_chunks_collection = Chroma( #B
    collection_name="uk_granular_chunks",
    embedding_function=OpenAIEmbeddings(openai_api_key=OPENAI_API
)

granular_chunks_collection.reset_collection() #C

expanded_chunk_store = InMemoryByteStore() #D
doc_key = "doc_id"

multi_vector_retriever = MultiVectorRetriever( #E
    vectorstore=granular_chunks_collection,
    byte_store=expanded_chunk_store
)
```

Ingesting Granular and Expanded Chunks

Now, generate expanded chunks by including the content from adjacent chunks, as shown in listing 8.12.

Listing 8.12 Generating and Storing Expanded Chunks

```
for destination_url in uk_destination_urls:
    html_loader = AsyncHtmlLoader(destination_url)  #A
    html_docs = html_loader.load()  #B
    text_docs = html2text_transformer.transform_documents(html_do

    granular_chunks = granular_chunk_splitter.split_documents(tex

    expanded_chunk_store_items = []
    for i, granular_chunk in enumerate(granular_chunks):  #E

        this_chunk_num = i  #F
        previous_chunk_num = i-1  #F
        next_chunk_num = i+1  #F

        if i==0:  #F
            previous_chunk_num = None
        elif i==(len(granular_chunks)-1):  #F
            next_chunk_num = None

        expanded_chunk_text = ""  #G
        if previous_chunk_num:  #G
            expanded_chunk_text += granular_chunks[previous_chunk
            expanded_chunk_text += "\n"

        expanded_chunk_text += granular_chunks[this_chunk_num].pa
        expanded_chunk_text += "\n"

        if next_chunk_num:  #G
            expanded_chunk_text += granular_chunks[next_chunk_num
            expanded_chunk_text += "\n"

        expanded_chunk_id = str(uuid.uuid4())  #H
        expanded_chunk_doc = Document(page_content=expanded_chunk

        expanded_chunk_store_item = (expanded_chunk_id, expanded_
        expanded_chunk_store_items.append(expanded_chunk_store_it

        granular_chunk.metadata[doc_key] = expanded_chunk_id  #J
```

```
print(f'Ingesting {destination_url}')
multi_vector_retriever.vectorstore.add_documents(granular_chunks)
multi_vector_retriever.docstore.mset(expanded_chunks_store.items())
```

Performing a Search Using the MultiVectorRetriever

After the ingestion step, run a search using the MultiVectorRetriever, which now uses expanded chunks for a more complete context:

```
retrieved_docs = multi_vector_retriever.invoke("Cornwall Ranger")
```

The first document retrieved will include content from surrounding chunks, giving the LLM more context to generate a richer response:

```
Document(page_content="Buses only serve designated stops when in
```

Comparing with Direct Semantic Search on Granular Chunks

For comparison, run a search directly on the granular chunks without expansion:

```
child_docs_only = granular_chunks_collection.similarity_search("C
```

The result will likely be more concise and lack the broader context:

```
Document(metadata={'doc_id': '04c7f88e-e090-4057-af5b-ea584e777b3
```

This smaller chunk lacks the surrounding details and may not provide enough context to synthesize a complete answer.

Chunk expansion offers a way to improve the effectiveness of granular embeddings by attaching broader context. The next section will cover how to efficiently handle mixed structured and unstructured content.

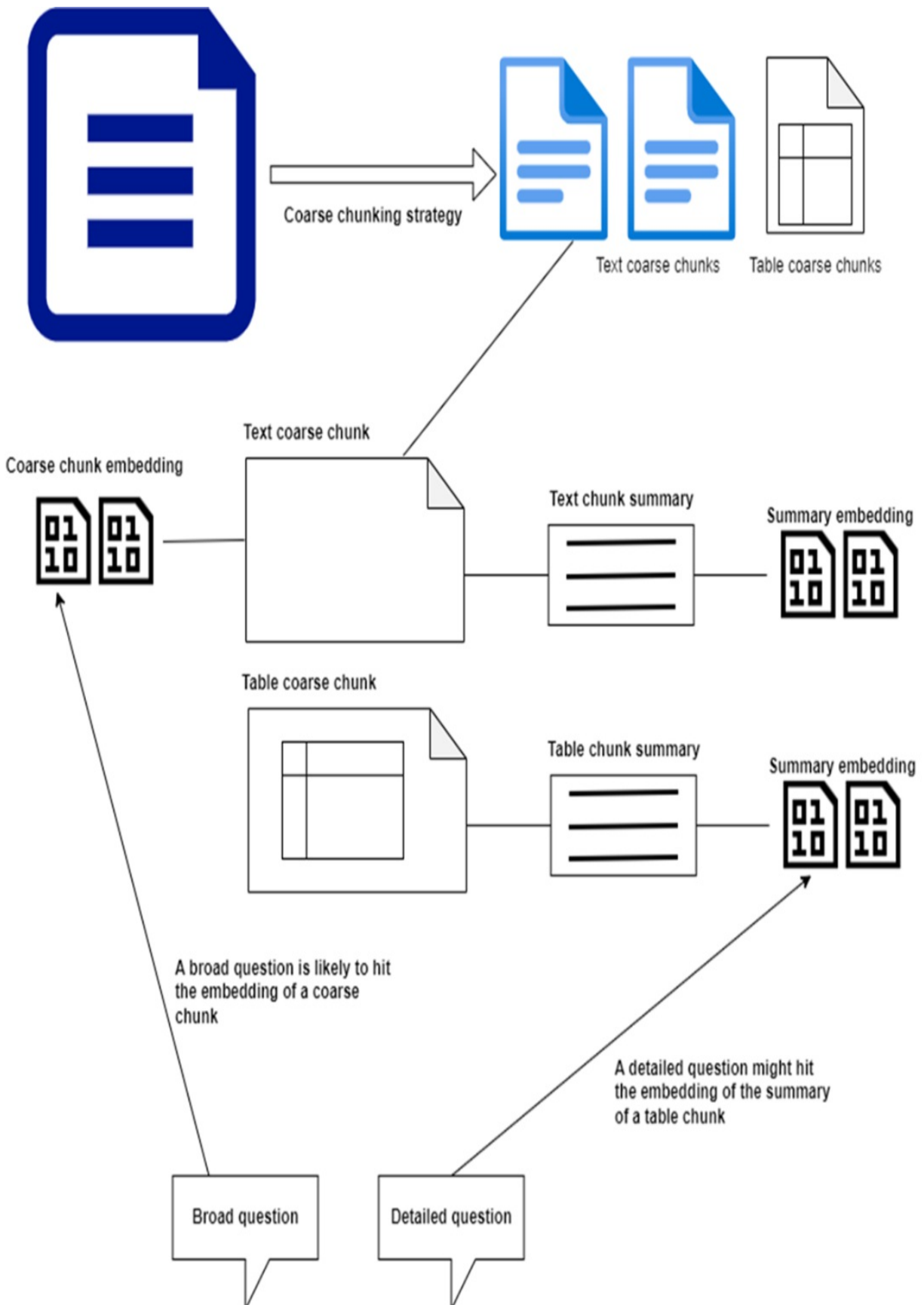
8.6 Semi-Structured Content

When dealing with documents that mix unstructured text and structured data (e.g., tables), it's essential to handle each type separately. You should extract

structured content, like tables, and generate embeddings for their summaries—just as you would for text chunks, as discussed in the *Embedding Document Summaries* section.

Store the coarse text chunks and the full tables in a document store and place the embeddings for the summaries of both text and tables in the vector store using a `MultiVectorRetriever`, as shown in figure 8.8. This setup allows seamless retrieval of both structured and unstructured content.

Figure 8.8 Embedding Structured and Unstructured Content—Structured data, such as tables, should be summarized and embedded just like unstructured text chunks. This ensures that the embeddings match detailed questions as effectively as text embeddings. Use the `MultiVectorRetriever` to manage both types of content.



When a search hits the embedding of a table’s summary, the entire table (stored in the document store) is returned to the LLM for synthesis, providing the necessary context to generate a complete response.

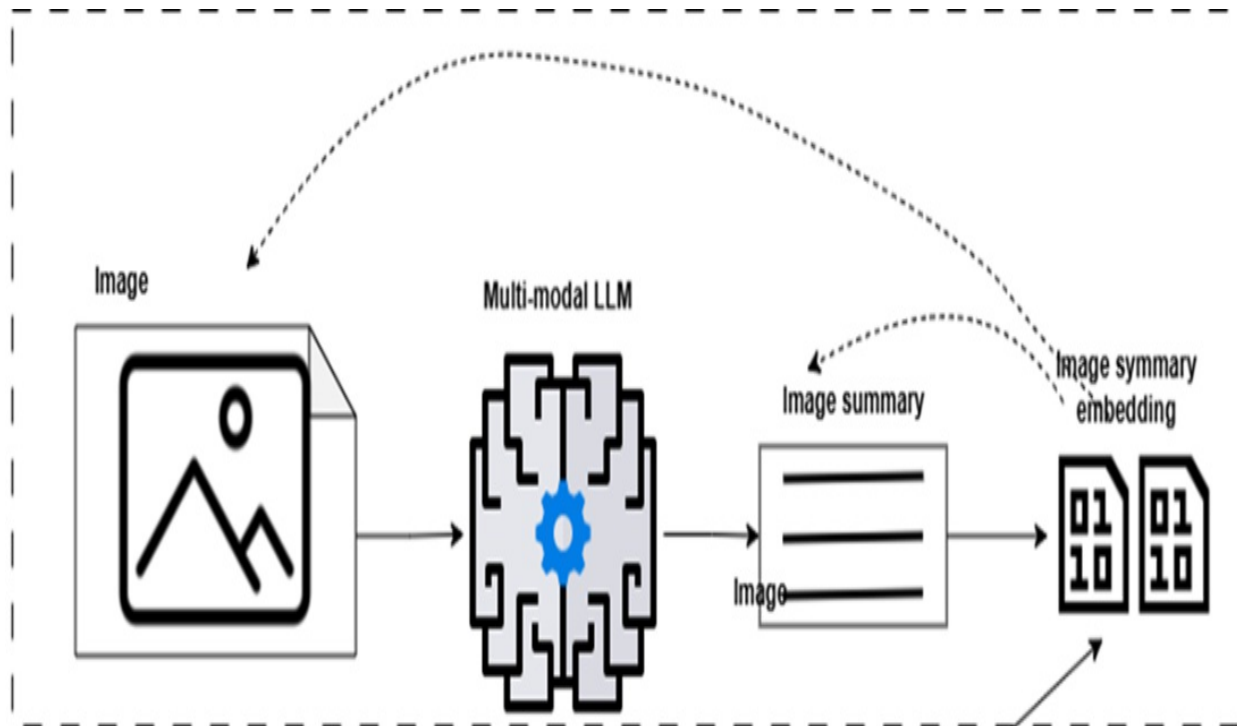
For a practical implementation of this technique, refer to https://github.com/langchain-ai/langchain/blob/master/cookbook/Semi_Structured_RAG.ipynb?ref=blog.langchain.dev.

8.7 Multi-Modal RAG

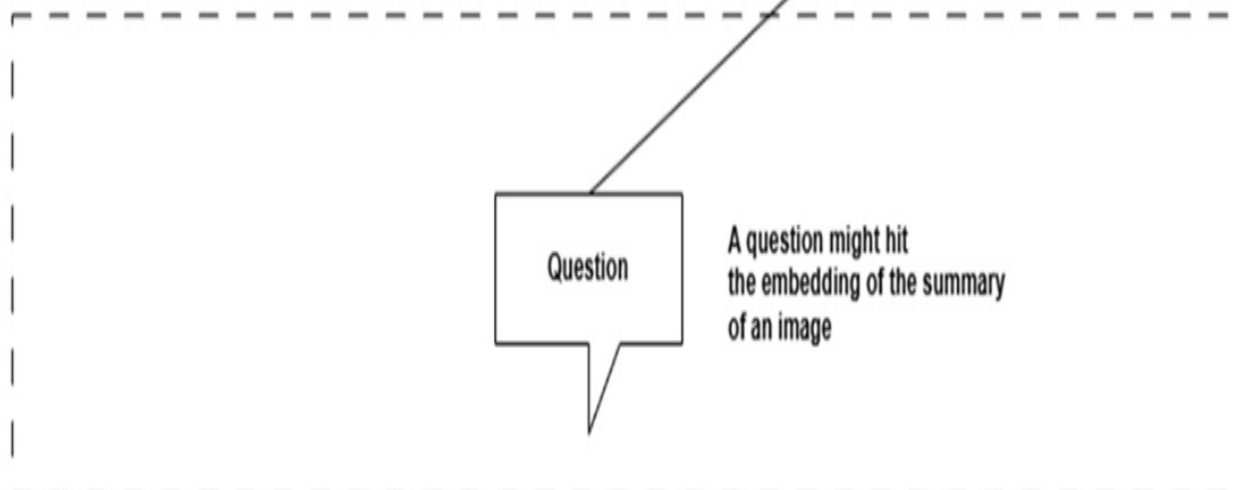
You’ve likely come across the term “multi-modal” LLMs. Models like GPT-4V extend traditional LLMs to handle not just text, but also images and audio. This opens the door for extending the RAG architecture to support multi-modal data.

The approach is similar to handling semi-structured content. During the data preparation stage, you can use a multi-modal LLM to generate a summary of an image, just as you would for a table. Then, create embeddings for the image summary and link these embeddings to the raw image stored in a document store, as shown in Figure 8.9.

Figure 8.9 Multi-Modal RAG Workflow—1) Data Ingestion: Use a multi-modal LLM to generate an image summary, store the summary embeddings in a vector store, and keep the raw image in a document store. **2) Retrieval:** If the summary embeddings match a query, the raw image is returned by the MultiVectorRetriever and fed into the LLM for synthesis.



1) Data ingestion



2) Retrieval

During retrieval, if the summary's embeddings match a user query, the `MultiVectorRetriever` returns the raw image—just as it would return a table for semi-structured text. The image is then passed to the multi-modal LLM

along with its summary, providing a rich context for generating a response.

Note

This book does not cover multi-modal RAG in detail, as it is an advanced topic that would require in-depth explanations beyond the intended scope. However, with what you've learned so far, you should have the foundation to explore it on your own. For more information, I recommend the article "Multimodality and Large Multimodal Models (LMMs)" by Chip Huyen: <https://huyenchip.com/2023/10/10/multimodal.html>.

8.8 Summary

- A basic RAG implementation often produces low-quality results, making it unsuitable for real-world applications.
- You can improve answer quality using techniques like advanced indexing, multiple query generation, query transformation, content-specific queries, retrieval post-processing (e.g., result reranking), and ensemble methods.
- Advanced indexing may include refined document splitting strategies, multiple embeddings for coarse chunks, and expanded context for granular chunks.
- Choose a splitting strategy based on content type: by size (e.g., paragraphs, sentences, words, or characters) or by structure (e.g., chapters or sections).
- Structured formats like HTML or Markdown benefit from splitting based on document hierarchy.
- For coarse chunks, enhance retrieval accuracy by using multiple embeddings generated from child chunks, summaries, or hypothetical questions about the content.
- Embed child chunks using the `ParentDocumentRetriever`. Use the `MultiVectorRetriever` to embed chunk summaries, hypothetical questions, or child chunks.
- Since it's hard to predict which multi-vector retrieval technique will perform best, experiment with each and compare results to find the optimal approach.
- For granular chunks, expand retrieval context by including adjacent

chunks for a fuller context.

- For semi-structured content (e.g., text with tables) or multi-modal content (e.g., images and audio), use specialized indexing techniques for optimal results.

9 Question transformations

This chapter covers

- Rewrite user questions with "Rewrite-Retrieve-Read" for better embedding alignment.
- Use "step-back" queries to retrieve higher-level context.
- Generate hypothetical documents to align questions with embeddings.
- Decompose complex queries into single or multi-step sequences.

In some cases, you might spend a lot of time preparing RAG data—collecting documents, splitting them into chunks, and generating embeddings for synthesis and retrieval (as covered earlier). Yet, you may still see low-quality results from the vector store. This problem might not come from missing relevant content in the vector store, but from issues in the user's question itself.

For instance, the question might be poorly phrased, unclear, or overly complex. Questions that aren't clearly and simply stated can confuse both the vector store and the LLM, leading to weaker retrieval results. In this section, I'll show you techniques to refine the user's question, making it easier for the query engine and the LLM to understand. By improving the question, you'll likely see better retrieval performance, providing more relevant context for the LLM to deliver a solid answer.

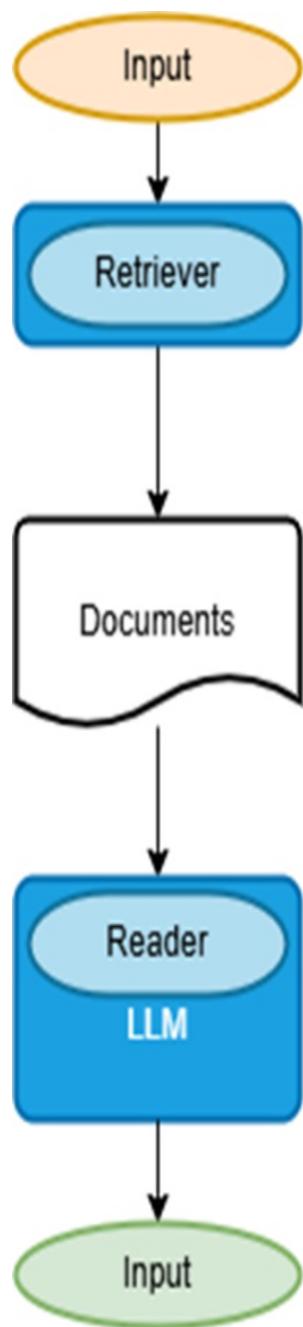
Let's begin with a straightforward method: using the LLM to help rephrase the question.

9.1 Rewrite-Retrieve-Read

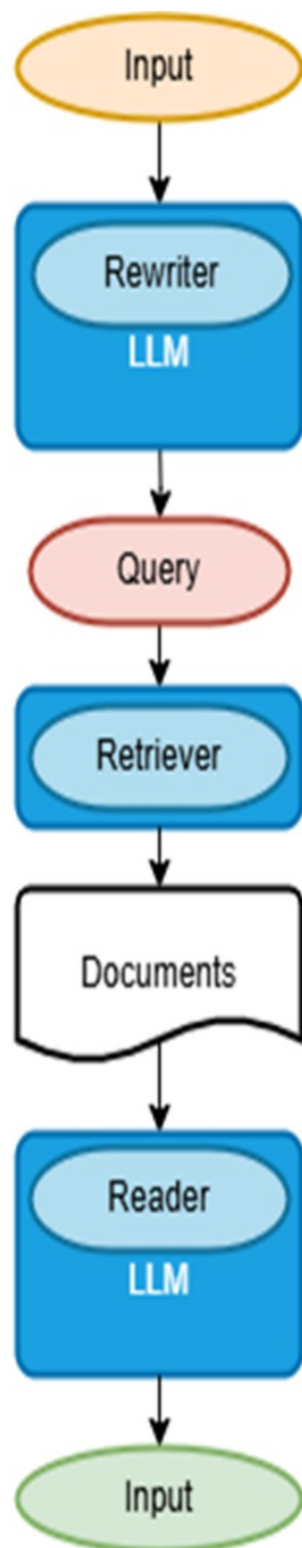
To improve a poorly worded question, one effective method is to have an LLM rewrite it into a clearer form. This approach, covered in the paper "Query Rewriting for Retrieval-Augmented Large Language Models" by Xinbei Ma et al. (<https://arxiv.org/pdf/2305.14283.pdf>), inspired the diagram

in figure 9.1.

Figure 9.1 In the standard Retrieve-and-Read setup, the retriever processes the user question directly, delivering results to the LLM for synthesis. In the “Rewrite-Retrieve-Read” approach, an initial Rewrite step uses an LLM to rephrase the query before it reaches the retriever, enhancing the clarity of the retrieval process (diagram adapted from the paper “Query Rewriting for Retrieval-Augmented Large Language Models”, <https://arxiv.org/pdf/2305.14283.pdf>).



Retrieve-and-Read

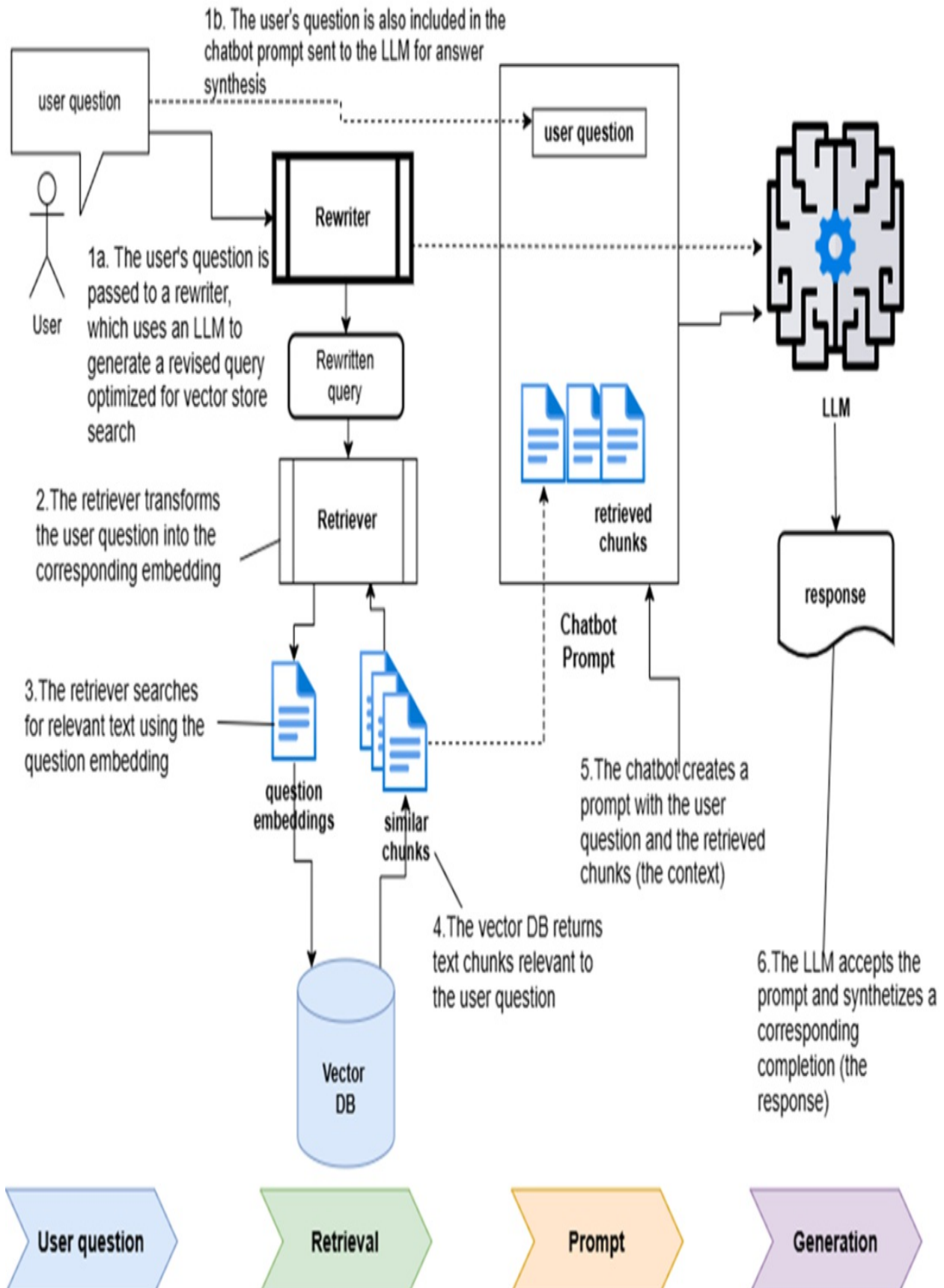


Rewrite-Retrieve-Read

In the standard “Retrieve-and-Read” workflow, the retriever processes the original question directly and sends results to the LLM for synthesis. By adding a Rewrite step upfront, a rewriter (often an LLM) reformulates the question before passing it to the retriever. This improved workflow is known as “Rewrite-Retrieve-Read.”

I usually apply this technique to create a tailored search query for the vector store, keeping the original question in the RAG prompt. This approach allows the rewritten query to optimize retrieval, particularly for semantic searches in the vector database, while preserving the original question for the LLM to synthesize the answer. See the workflow in figure 9.2, which is an amended version of the RAG diagram you saw in figure 5.2.

Figure 9.2 Using query rewriting to generate an optimized vector store query, while preserving the original question for answer synthesis via the chatbot prompt.



A prompt for rewriting the query can be as simple as the one below, adapted from a popular prompt in the Langchain Hub:

```
Revise the original question to make it more refined and precise
Original question: {user_question}
Revised Chroma DB query:
```

To apply the *Rewrite-Retrieve-Read* technique and use the prompt above to rewrite the original user question, open a new OS shell, navigate to the Chapter 9 code folder, and set up your environment.:

```
C:\Github\building-llm-applications\ch09>
C:\Github\building-llm-applications\ch09>python -m venv env_ch09
C:\Github\building-llm-applications\ch09>pip install notebook lan
C:\Github\building-llm-applications\ch09>env_ch09\Scripts\activat
(env_ch09) C:\Github\building-llm-applications\ch09>jupyter noteb
```

After launching Jupyter Notebook, create a new notebook named 09-question_transformations.ipynb. Then, re-import the UK tourist destination content from Wikivoyage as in the previous chapter. For convenience, I've included the code below.

Listing 9.1 Splitting and Ingesting Content from URLs

```
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import HTMLSectionSplitter
from langchain_community.document_loaders import AsyncHtmlLoader

import getpass

OPENAI_API_KEY = getpass.getpass('Enter your OPENAI_API_KEY')

uk_granular_collection = Chroma(
    collection_name="uk_granular",
    embedding_function=OpenAIEmbeddings(openai_api_key=OPENAI_API
)

uk_granular_collection.reset_collection() #A

uk_destinations = [
    "Cornwall", "North_Cornwall", "South_Cornwall", "West_Cornwal
    "Tintagel", "Bodmin", "Wadebridge", "Penzance", "Newquay",
```

```

        "St_Ives", "Port_Isaac", "Looe", "Polperro", "Porthleven",
        "East_Sussex", "Brighton", "Battle", "Hastings_(England)",
        "Rye_(England)", "Seaford", "Ashdown_Forest"
    ]

    wikivoyage_root_url = "https://en.wikivoyage.org/wiki"

    uk_destination_urls = [f'{wikivoyage_root_url}/{d}' for d in uk_d

    headers_to_split_on = [("h1", "Header 1"), ("h2", "Header 2")]
    html_section_splitter = HTMLSectionSplitter(headers_to_split_on=h

    def split_docs_into_granular_chunks(docs):
        all_chunks = []
        for doc in docs:
            html_string = doc.page_content #B
            temp_chunks = html_section_splitter.split_text(html_strin
            h2_temp_chunks = [chunk for chunk in temp_chunks if "Head
            all_chunks.extend(h2_temp_chunks)

        return all_chunks

    for destination_url in uk_destination_urls:
        html_loader = AsyncHtmlLoader(destination_url) #E
        docs = html_loader.load() #F

        for doc in docs:
            print(doc.metadata)
            granular_chunks = split_docs_into_granular_chunks(docs)
            uk_granular_collection.add_documents(documents=granular_c

```

This setup prepares your data for effective query rewriting and retrieval using the *Rewrite-Retrieve-Read* workflow. The Rewrite step will help craft refined search queries, improving retrieval results and the overall quality of the responses generated.

9.1.1 Retrieving Content Using the Original User Question

Let's start by performing a search with the original user question:

```

user_question = "Tell me some fun things I can enjoy in Cornwall"
initial_results = uk_granular_collection.similarity_search(query=
for doc in initial_results:
    print(doc)

```


The output will look something like this (I have reduced it in various places):

```
page_content='Do  
[ edit ]
```

```
Cornwall, in particular Newquay, is the UK's surfing capital,  
The South West Coast Path runs [REduced...]  
The Camel Trail is an 18-mile (29 km) [REduced...]  
The Cornish Film Festival is held annually [REduced...]  
The Royal Cornwall Show is an agricultural show [REduced...]  
Camel Creek Adventure Park , Tredinnick, Wadebridge offers grea
```

```
Festivals  
[ edit ]
```

```
Mummer's Day , or "Darkie Day" as it is sometimes known [REduced  
' Obby 'Oss is held annually on May Day (1 May), mainly in Pad  
St Piran's Day (Cornish: Gool Peran ) is the national day of C  
page_content='Do  
[ edit ]
```

```
The South West Coast Path runs [REduced...]  
The Camel Trail is an 18-mile (29 km) off-road cycle-track  
The Cornish Film Festival is held annually [REduced...]  
Cornwall, in particular Newquay, is the UK's surfing capital, [
```

```
Cricket: Cornwall CCC play in the National Counties [REduced...  
page_content='Buy  
[ edit ]
```

```
In the village centre you will find the usual [REduced...] ' meta  
page_content='Do  
[ edit ]
```

```
The South West Coast Path runs along [REduced...]  
St Piran's Day (Cornish: Gool Peran ) is the national day of C
```

This output provides some information on fun activities, but it appears somehow limited. This might be due to the way the question has been worded.

9.1.2 Setting Up the Query Rewriter Chain

To refine the user question into a more effective query for Chroma DB, we'll use the LLM to rewrite it into a format better suited for semantic search.

Setting up a query rewriter chain will help us automate this transformation.

Start by importing the necessary libraries:

```
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
llm = ChatOpenAI(model="gpt-4o-mini", openai_api_key=OPENAI_API_K
```

Next, define the prompt that instructs the LLM to rewrite the user question:

```
rewriter_prompt_template = """
Generate search query for the Chroma DB vector store from a user
Just return the revised Chroma DB query, with quotes around it.
```

```
User question: {user_question}
Revised Chroma DB query:
"""
```

```
rewriter_prompt = ChatPromptTemplate.from_template(rewriter_prompt
```

Now, construct the chain to execute the rewriting process:

```
rewriter_chain = rewriter_prompt | llm | StrOutputParser()
```

This setup allows you to pass a user question to the rewriter chain, which generates a tailored query optimized for Chroma DB, enhancing retrieval accuracy.

9.1.3 Retrieving Content with the Rewritten Query

Now, let's use the rewriter chain to create a more targeted query and see if it returns more accurate results compared to the original question.

First, generate the rewritten query:

```
user_question = "Tell me some fun things I can do in Cornwall"

search_query = rewriter_chain.invoke({"user_question": user_quest
print(search_query)
```

If you print search_query, you should see something like:

"fun activities to do in Cornwall"

Now, use this refined query to perform the vector store search:

```
improved_results = uk_granular_collection.similarity_search(query
```

Finally, print the results to review their relevance:

```
for doc in improved_results:
    print(doc)
```

You will get the following output, which I have reduced to save space:

```
page_content='Do
[ edit ]
```

```

    Cornwall, in particular Newquay, is the UK's surfing capital,
    The South West Coast Path runs along the coastline [REduced...]
    The Cornish section is supposed to be the most [REduced...]
    The Camel Trail is an 18-mile (29 km) off-road cycle-track
    The Cornish Film Festival is held annually each November around
    The Royal Cornwall Show is an agricultural show [REduced...]
    Camel Creek Adventure Park , Tredinnick, Wadebridge offers [RE
    Festivals
[ edit ]
```

```

    Mummer's Day , or "Darkie Day" as it is sometimes [REduced...]
    ' Obby 'Oss is held annually on May Day (1 May), [REduced...]
    St Piran's Day (Cornish: Gool Peran ) is the national day [RE
page_content='Do
[ edit ]
```

```

    The South West Coast Path runs [REduced...]
    The Camel Trail is an 18-mile (29 km) off-road cycle- [RED
    The Cornish Film Festival is held annually each November around
    Cornwall, in particular Newquay, is the UK's surfing capital,
    Cricket: Cornwall CCC play in the National Counties Cricket
page_content='Do
[ edit ]
```

```

    Helford River is an idyllic river estuary between Falmouth and
    The South West Coast Path runs along the coastline [REduced...]
    Festivals
[ edit ]
```

```

    Allantide (Cornish: Kalan Gwav or Nos Kalan Gwav ) is a Corn
```

Furry Dance , also known as Flora Day, takes place in [REDUCED Golowan , sometimes also Goluan or Gol-Jowan , is the Cornish Guldize is an ancient harvest festival in Autumn, [REDUCED...]
Montol Festival is an annual heritage, arts and community [RED
Nickanan Night is traditionally held on the Monday before Lent.
St Piran's Day (Cornish: Gool Peran) is the national day of C

This approach should yield results that align more closely with your original intent, displaying a broader selection of content on activities available in Cornwall. Using the rewritten question, the vector store retriever provides a more diverse set of text chunks compared to the results from the original question.

9.1.4 Combining Everything into a Single RAG Chain

Now, you can build a complete workflow that transforms the initial user question into a search query for vector retrieval. The original question is retained in the prompt to generate the final answer. Listing 9.2 shows the full RAG chain, including the query rewriting step.

Listing 9.2 Combined RAG Chain with Query Rewriting

```
from langchain_core.runnables import RunnablePassthrough
retriever = uk_granular_collection.as_retriever()

rag_prompt_template = """
Given a question and some context, answer the question.
If you do not know the answer, just say I do not know.

Context: {context}
Question: {question}
"""

rag_prompt = ChatPromptTemplate.from_template(rag_prompt_template)

rewrite_retrieve_read_rag_chain = (
    {
        "context": {"user_question": RunnablePassthrough()} | rew
        "question": RunnablePassthrough(), #B
    }
    | rag_prompt
    | llm
    | StrOutputParser()
```

)

Now, run the complete workflow:

```
user_question = "Tell me some fun things I can do in Cornwall"

answer = rewrite_retrieve_read_rag_chain.invoke(user_question)
print(answer)
```

When you print the answer, you should see a response like this:

In Cornwall, you can enjoy a variety of fun activities such as:

1. Surfing in Newquay, known as the UK's surfing capital, where you can catch waves on the famous Fistral Beach.
 2. Walking along the scenic South West Coast Path, which offers breathtaking views of the coastline.
 3. Cycling on the Camel Trail, an 18-mile off-road cycle track through the countryside.
 4. Attending the Cornish Film Festival held annually in November.
 5. Exploring the Helford River, where you can take a ferry ride or enjoy fishing.
 6. Participating in local festivals such as St Piran's Day or the Cornwall Festival.
 7. Visiting Camel Creek Adventure Park for family-friendly entertainment.
 8. Enjoying various agricultural shows, like the Royal Cornwall Show.
- There are plenty of options for both adventure and relaxation in Cornwall.

This output demonstrates a satisfying answer based on the combined Rewrite-Retrieve-Read workflow.

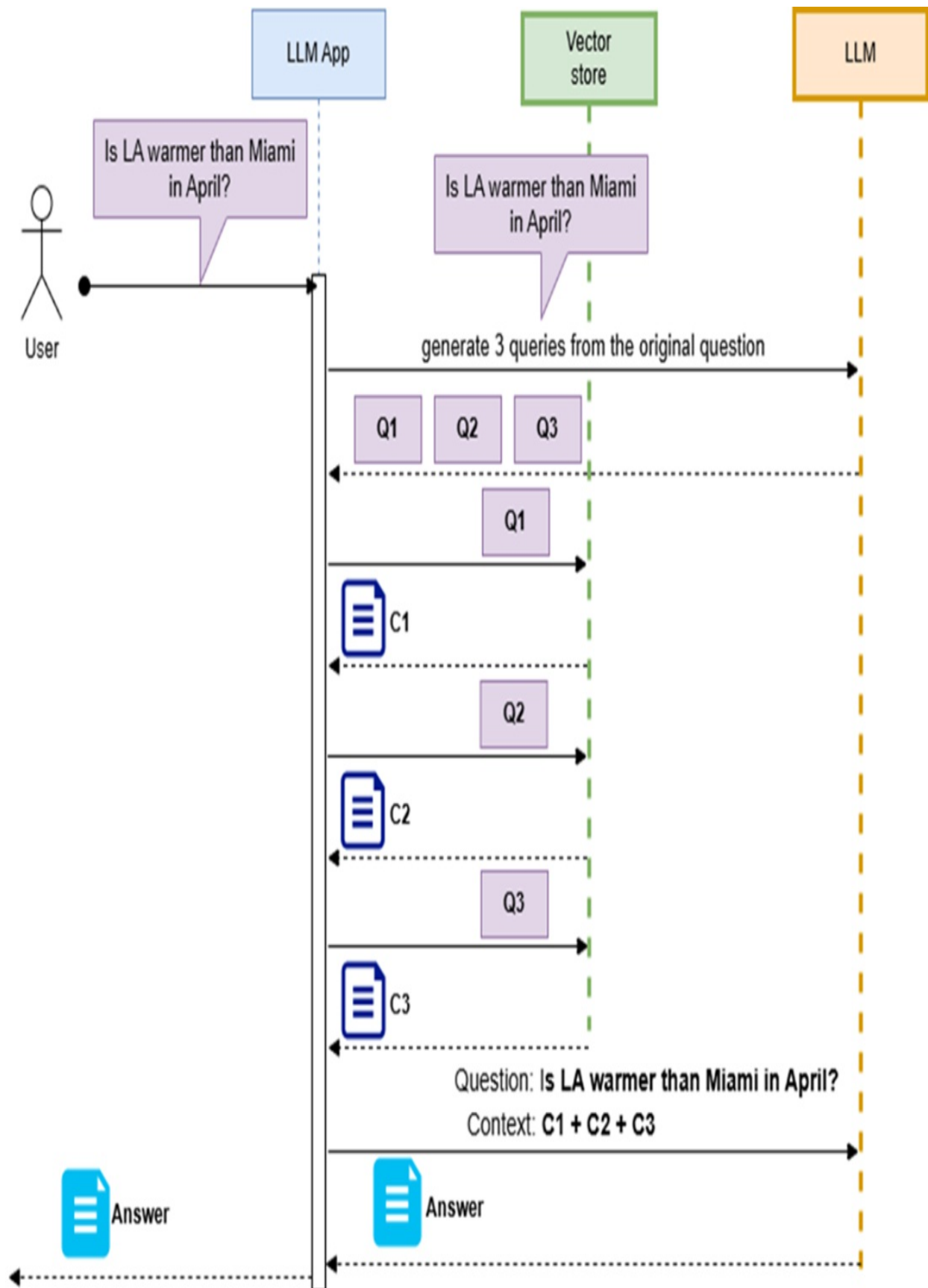
Next, I'll show you further ways to refine the "Rewrite-Retrieve-Read" process.

9.2 Generating Multiple Queries

The "Rewrite-Retrieve-Read" approach assumes the original user question was poorly phrased. But if the question is well-formed and contains multiple implicit questions, rewriting it as a single improved query may not be effective. In these cases, it's better to have the LLM break down the original question into multiple explicit questions. Each question can be executed separately against the vector store, with the answers then synthesized into a comprehensive response. Figure 9.3 illustrates this workflow.

Figure 9.3 Workflow for Multiple Query Generation: The LLM application reformulates the original question into multiple explicit questions, which are executed against the vector store to produce the context, and synthesizes the answer from a prompt with the original question and the

produced context.



In the figure 9.3, the LLM application reformulates the original question into multiple explicit questions, executes them against the vector store to gather context, and then synthesizes the answer using a prompt that includes both the original question and the retrieved context.

For example, if you prompt ChatGPT with the following:

```
Reformulate the following question into multiple explicit questions
Is LA warmer than Miami in April?
```

You might receive the following questions, which can be executed individually against the vector store:

```
What is the average temperature in Los Angeles during April?
What is the average temperature in Miami during April?
Can you compare the April temperatures in Los Angeles and Miami?
Is there a notable difference in temperature between Los Angeles
and Miami?
Could you provide insights into how the temperatures in Los Angeles
compare to Miami?
```

This approach is especially useful when designing generic LLM applications. You can automatically generate multiple queries for any user question using a prompt like this, adapted from a LangChain example:

```
QUERY_PROMPT = PromptTemplate(
    input_variables=["question"],
    template="""You are an AI language model assistant. Your task is
    to generate different versions of the given user question to retrieve relevant
    information from a vector database. By generating multiple perspectives on the user
    question, you can overcome some of the limitations of the distance-based
    search. Provide these alternative questions separated by newlines.
    Original question: {question}""",
)
```

LangChain's `MultiQueryRetriever` class allows you to use a prompt, an LLM reference, and a retriever (e.g., derived from a vector database). When a `MultiQueryRetriever` instance processes a user query, it executes the entire multi-query workflow automatically, as shown in Figure 7.10. This approach combines multi-retrieval with answer synthesis.

Next, I'll show you how to implement a custom `MultiQueryRetriever`.

9.2.1 Setting Up the Chain for Generating Multiple Queries

First, import the necessary libraries:

```
from langchain.retrievers.multi_query import MultiQueryRetriever
from langchain_core.prompts import ChatPromptTemplate

from typing import List
from langchain_core.output_parsers import BaseOutputParser
from pydantic import BaseModel, Field
```

Begin by setting up the prompt shown earlier to instruct the LLM to generate multiple variations of a single user question:

```
multi_query_gen_prompt_template = """
You are an AI language model assistant. Your task is to generate
different versions of the given user question to retrieve relevant
database. By generating multiple perspectives on the user question,
the user can overcome some of the limitations of the distance-based
search. Provide these alternative questions separated by newlines.
Original question: {question}
"""
```

```
multi_query_gen_prompt = ChatPromptTemplate.from_template(multi_q
```

Since the LLM is generating five alternative questions, it's useful to format the output as a list of strings, with each string representing one question. This lets you process each question independently. To do this, implement a custom result parser instead of the standard `StrOutputParser`:

```
class LineListOutputParser(BaseOutputParser[List[str]]):
    """Parse out a question from each output line."""

    def parse(self, text: str) -> List[str]:
        lines = text.strip().split("\n")
        return list(filter(None, lines))
```

```
questions_parser = LineListOutputParser()
```

With these components, you can set up the chain to generate multiple queries:

```
llm = ChatOpenAI(model="gpt-4o-mini", openai_api_key=OPENAI_API_K
multi_query_gen_chain = multi_query_gen_prompt | llm | questions_
```

Now, try running this chain:

```
user_question = " Tell me some fun things I can do in Cornwall."
multiple_queries = multi_query_gen_chain.invoke(user_question)
```

When you print `multiple_queries`, you should get a list of questions similar to this:

```
['What are some enjoyable activities to explore in Cornwall? ',
 'Can you suggest interesting attractions or events in Cornwall f
 'What are the top leisure activities to try out while visiting C
 'What fun experiences or adventures does Cornwall have to offer?
 'Could you recommend some entertaining things to do while in Cor
```

One final step remains to complete the setup.

9.2.2 Setting Up a Custom Multi-Query Retriever

To configure a custom multi-query retriever, start by creating a standard retriever and then embedding it within the multi-query retriever, as shown below:

```
basic_retriever = uk_granular_collection.as_retriever()

multi_query_retriever = MultiQueryRetriever(
    retriever=basic_retriever, llm_chain=multi_query_gen_chain,
    parser_key="lines" #A
)
```

Now, test it with an example query:

```
user_question = "Tell me some fun things I can do in Cornwall"
retrieved_docs = multi_query_retriever.invoke(user_question)
```

When you print `retrieved_docs`, you should see output similar to this:

```
[Document(metadata={'Header 2': 'Do'}, page_content='Do \n [ edit
Document(metadata={'Header 2': 'Do'}, page_content='Do \n [ edit
Document(metadata={'Header 2': 'Contents'}, page_content='Conten
Document(metadata={'Header 2': 'Do'}, page_content="Do \n [ edit
Document(metadata={'Header 2': 'Festivals'}, page_content='Festi
```

```
Document(metadata={'Header 2': 'Contents'}, page_content='Content')
Document(metadata={'Header 2': 'See'}, page_content="See \n [ ed
```

9.2.3 Using a Standard MultiQueryRetriever Instance

For straightforward use cases, you can set up multi-query generation with a standard MultiQueryRetriever instance. First, instantiate the multi-query retriever:

```
std_multi_query_retriever = MultiQueryRetriever.from_llm(
    retriever=basic_retriever, llm=llm
)
```

Now, test it with the same question:

```
user_question = " Tell me some fun things I can do in Cornwall"
retrieved_docs = multi_query_retriever.invoke(user_question)
```

The output in retrieved_docs will be similar to the results you saw previously.

In some cases, rewriting the original question or breaking it into multiple explicit questions may not yield the desired results. Continue reading to explore a technique that can improve accuracy in these situations.

9.3 Step-Back Question

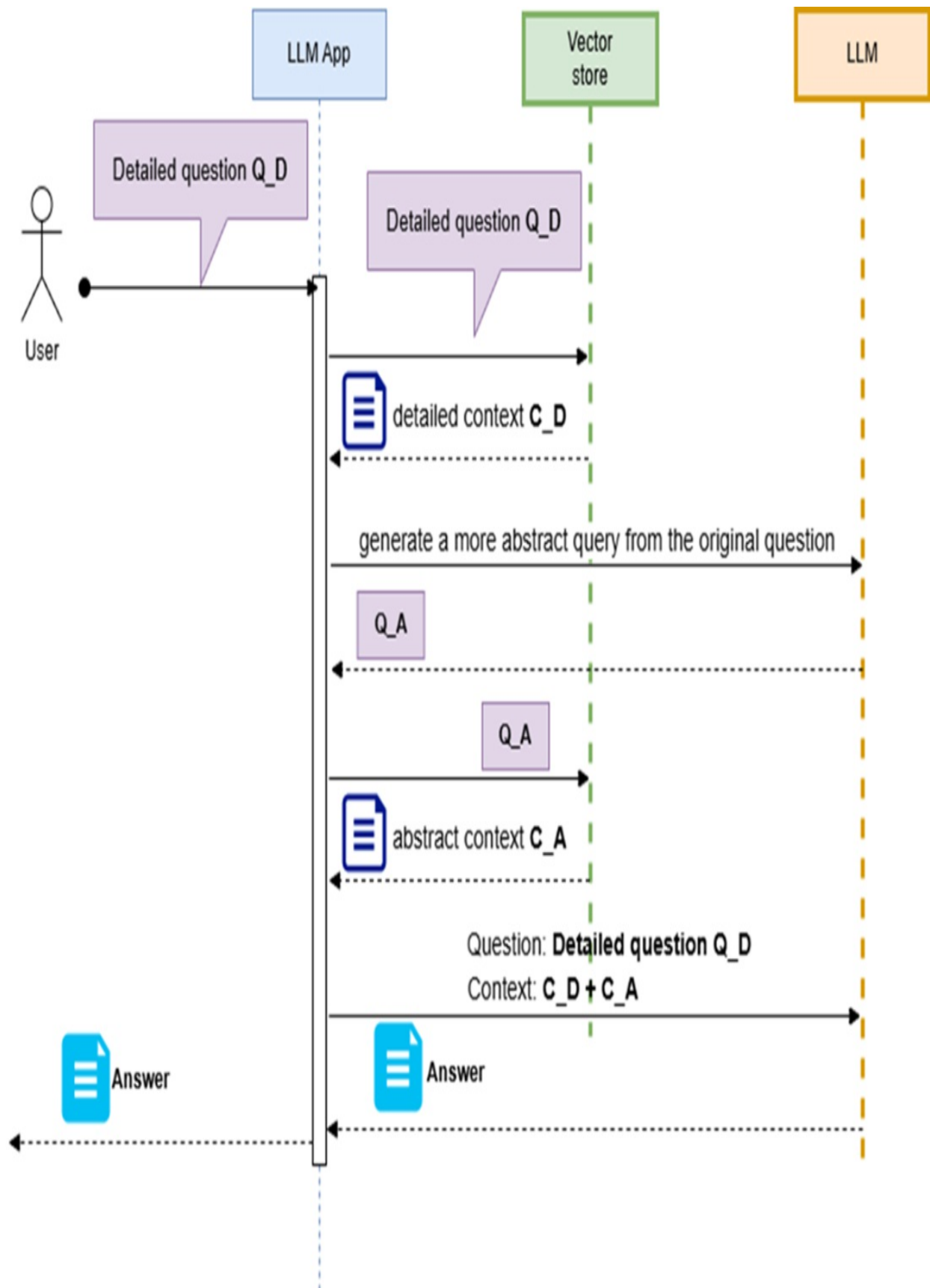
When you send a highly detailed question directly to the vector store—assuming your documents are split into small, specific chunks—you might retrieve information that’s too focused, missing broader context. This can limit the LLM’s ability to generate a comprehensive answer.

As discussed in section 7.4, one solution is to create two sets of document chunks: coarse chunks for synthesis and fine-grained chunks for detailed retrieval. Another solution is to adjust the user question rather than the document chunks, using an approach called a “step-back question.”

In this approach, you start with the user’s detailed question but then create a

broader question to retrieve a more generalized context. This “step-back context” provides a higher-level view than the “original context” derived from the specific question. You then provide the LLM with both the detailed context and the broader context to enable a fuller response, as illustrated in figure 9.4.

Figure 9.4 Step-Back Question Workflow: The LLM application first sends the detailed question (Q_D) to the vector store to retrieve a detailed context (C_D). It then prompts the LLM to generate a more abstract question (Q_A) based on Q_D, which is also executed on the vector store to obtain an abstract context (C_A). Finally, the LLM application combines Q_D, C_D, and C_A into a single prompt, enabling the LLM to synthesize a comprehensive answer.



The LLM application sends the original detailed question to the vector store to retrieve detailed context, then generates and executes a broader question to obtain abstract context. It combines both contexts with the original question to enable the LLM to create a comprehensive answer.

This technique, developed by Huaixiu Steven Zheng et al., is explained in their paper “Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models” (<https://arxiv.org/pdf/2310.06117.pdf?ref=blog.langchain.dev>).

To implement this, use a prompt like the following to generate the step-back question:

```
Generate a less specific question (aka Step-back question) for the
Detailed question: {detailed_question}
Step-back question:
```

For example, if you input the prompt with the detailed question, “Can you give me some tips for a trip to Brighton?” the more abstract (step-back) question might look like:

```
Step-back question: "What should I know before visiting a popular
```

This broader question helps retrieve more general information, which, combined with the detailed context, allows the LLM to produce a well-rounded answer.

9.3.1 Setting Up the Chain to Generate a Step-Back Question

Implementing the step-back question technique is straightforward: it involves crafting an effective prompt to generate a broader question and then following a standard RAG workflow. Here’s a sample implementation, which closely resembles the pattern used for the rewrite-retrieve-read technique.

Start by setting up the prompt in your Jupyter notebook:

```
llm = ChatOpenAI(model="4o-mini", openai_api_key=OPENAI_API_KEY)

step_back_prompt_template = """
```

```
Generate a less specific question (aka Step-back question) for the
Detailed question: {detailed_question}
Step-back question:
"""
```

```
step_back_prompt = ChatPromptTemplate.from_template(step_back_pro
```

Note

I've chosen to use gpt-4o-mini over gpt-4o and gpt-4o-mini because it tends to generate more abstract yet contextually relevant queries, along with more coherent and well-synthesized final answers. That said, I encourage you to experiment with different models to see how their outputs vary.

Now, create the chain:

```
step_back_question_gen_chain = step_back_prompt | llm | StrOutput
```

Try out the chain with a sample question:

```
user_question = "Can you give me some tips for a trip to Brighton"
step_back_question = step_back_question_gen_chain.invoke(user_que
```

When you print `step_back_question`, you should get a response like:

```
'What are some general tips for planning a successful trip to a c
```

This generated step-back question can then be used within a RAG architecture to retrieve broader context from the vector store, which can subsequently be provided to the LLM to help synthesize a more complete answer.

9.3.2 Incorporating Step-Back Question Generation into the RAG Chain

You can integrate the step-back question generation chain into a RAG workflow, as shown in Listing 9.3.

Listing 9.3 Integrating Step-Back Question Generation within a RAG Architecture

```

retriever = uk_granular_collection.as_retriever()

rag_prompt_template = """
Given a question and some context, answer the question.
If you do not know the answer, just say I do not know.

Context: {context}
Question: {question}
"""

rag_prompt = ChatPromptTemplate.from_template(rag_prompt_template)

step_back_question_rag_chain = (
    {
        "context": {"detailed_question": RunnablePassthrough()} |
        "question": RunnablePassthrough(), #B
    }
    | rag_prompt
    | llm
    | StrOutputParser()
)

```

Now, try running the chain:

```

user_question = "Can you give me some tips for a trip to Brighton"

answer = step_back_question_rag_chain.invoke(user_question)
print(answer)

```

You should see a synthesized response like this:

Here are some tips for a trip to Brighton:

1. ****Stay Safe****: While Brighton is generally safe, be cautious i
2. ****Watch for Traffic****: Be mindful of traffic, especially in bu
3. ****Valuables****: Take standard precautions with your valuables t
4. ****Homelessness****: Be aware that there may be homeless individu
5. ****Beaches****: Lifeguards patrol the beaches from late May to ea
6. ****Emergency Contacts****: In case of emergencies related to the
7. ****Explore Local Venues****: Enjoy local venues favored by reside
8. ****Cultural Areas****: Visit areas like The Lanes and North Laine
9. ****Stay Informed****: Keep an eye on your surroundings, especiall

Enjoy your trip to Brighton!

This technique offers an alternative to embedding-focused methods,

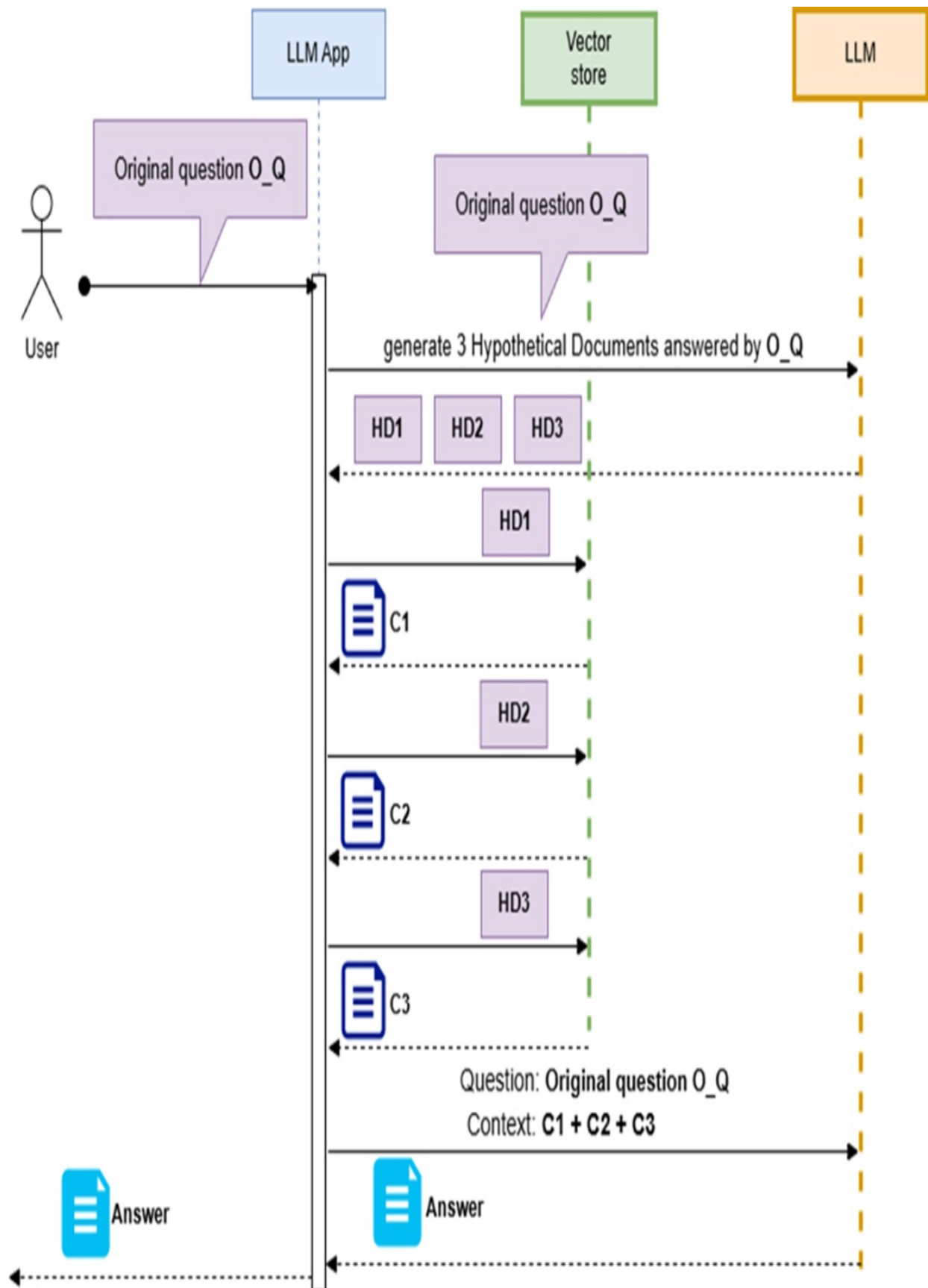
enhancing retrieval by broadening the question itself to improve context. Next, I'll introduce another technique that also optimizes retrieval through question transformation.

9.4 Hypothetical Document Embeddings (HyDE)

As discussed in section 7.2.2, embedding hypothetical questions can enhance RAG retrieval by indexing document chunks with additional embeddings that represent questions answerable by the content in each chunk. This approach makes embeddings of these hypothetical questions more semantically similar to the user's question than embeddings of the raw chunk text alone.

A similar effect can be achieved with Hypothetical Document Embeddings (HyDE), a technique that keeps the original chunk embeddings unchanged while generating hypothetical documents based on the user's question, as shown in figure 9.5.

Figure 9.5 Hypothetical Document Embeddings (HyDE): In this approach, the LLM generates hypothetical documents that would answer the user's question. Rather than querying the document store with the user's original question, these generated documents are used. Because these hypothetical documents are semantically closer to the document chunk text, they improve the relevance of retrieved content.



As shown in the sequence diagram in Figure 9.5, the generated hypothetical documents are used to retrieve relevant content, aiming to increase the semantic similarity between the user's question and the document chunk embeddings. This technique was introduced by Luyu Gao et al. in their paper, "Precise Zero-Shot Dense Retrieval without Relevance Labels" (<https://arxiv.org/pdf/2212.10496v1.pdf>). Now, let's proceed to implement this workflow.

Let's move on to implementing this workflow.

9.4.1 Generating an Hypothetical Document for the User Question

Implementing HyDE follows a familiar pattern: you design a chain to generate a hypothetical document that could answer the user's question, and then use this generated document as input for retrieval within a larger RAG workflow.

First, set up the prompt:

```
llm = ChatOpenAI(model="gpt-4o-mini", openai_api_key=OPENAI_API_K
hyde_prompt_template = """
Write one sentence that could answer the provided question. Do no
Question: {question}
Sentence:
"""
```

```
hyde_prompt = ChatPromptTemplate.from_template(hyde_prompt_templa
```

Next, build the HyDE chain:

```
hyde_chain = hyde_prompt | llm | StrOutputParser()
```

Test it with a sample question:

```
user_question = "What are the best beaches in Cornwall?"
```

```
hypotetical_document = hyde_chain.invoke(user_question)
```

When you print `hypothetical_document`, you should see output like:

```
Some of the best beaches in Cornwall include Fistral Beach, Porth
```

This generated hypothetical document can then be used in the RAG chain to retrieve relevant content, improving the alignment between the user's question and the document chunks in the vector store. Let's look at integrating this step into the broader RAG workflow next.

9.4.2 Integrating the HyDE Chain into the RAG Chain

You can incorporate the HyDE chain into a RAG workflow, as shown in Listing 9.4.

Listing 9.4 Integrating a HyDE Chain within a RAG Workflow

```
retriever = uk_granular_collection.as_retriever()

rag_prompt_template = """
Given a question and some context, answer the question.
Only use the provided context to answer the question.
If you do not know the answer, just say I do not know.

Context: {context}
Question: {question}
"""

rag_prompt = ChatPromptTemplate.from_template(rag_prompt_template)

hyde_rag_chain = (
    {
        "context": {"question": RunnablePassthrough()} | hyde_chain,
        "question": RunnablePassthrough(), #B
    }
    | rag_prompt
    | llm
    | StrOutputParser()
)
```

Now, try the complete RAG chain:

```
user_question = "What are the best beaches in Cornwall?"
```

```
answer = hyde_rag_chain.invoke(user_question)
print(answer)
```

You should see a synthesized answer similar to this:

```
The best beaches in Cornwall mentioned in the context include Bud
```

This concludes the integration of HyDE into the RAG chain, enhancing the retrieval process by using a hypothetical document. Before moving on, I'll briefly revisit the multi-query generation technique we discussed earlier to refine the retrieval focus.

9.5 Single-Step and Multi-Step Decomposition

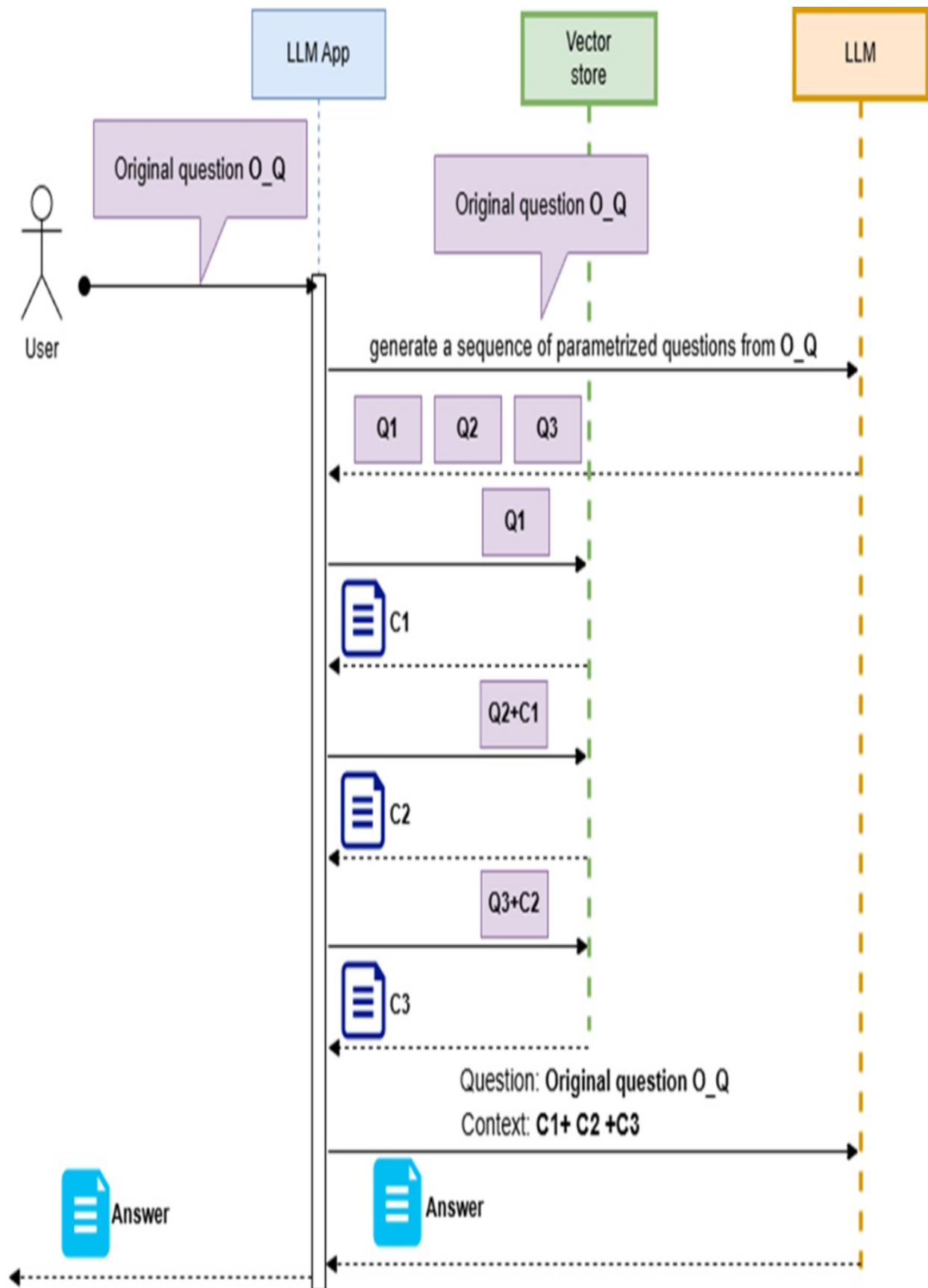
In the “multiple questions” or “sub-questions” method covered in section 9.2, the original user question contained several implicit questions. For that case, we instructed the LLM to generate a set of explicit, independent questions, each of which could be executed separately (or in parallel) on the vector store. This approach, called “single-step decomposition,” is effective when the questions are independent, and the original “complex question” can be split into simple, single-step questions.

However, if the original question includes several interdependent questions, a different approach is required. For example, if your data store contains tourist information, consider the question: “What is the average August temperature at the most popular sandy beach in Cornwall?” This question requires a sequence of dependent queries, where each answer informs the next question.

In this case, you could instruct the LLM to generate a strategic plan for breaking down the question into a sequence of interdependent queries. Each query would contain a parameter, filled with information from the previous answer. Once the LLM returns this sequence of parameterized questions, you can execute them step-by-step, storing each answer and feeding it into the subsequent query. This process continues until you reach the final answer, as shown in figure 9.6.

Figure 9.6 Multi-Step Decomposition Workflow: In this workflow, the original complex question is sent to the LLM, which generates a sequence of parameterized questions. Each question is then executed on the vector store, using answers from previous steps as parameters. Once all questions

are answered, the LLM synthesizes a final response based on the collected information.



In the workflow illustrated in the sequence diagram, the original complex question is sent to the LLM, which breaks it down into a series of parameterized questions. Each question is executed on the vector store, using previous answers as parameters. After all questions are answered, the LLM combines the collected information to generate a final response.

To prompt the LLM to generate this question sequence, use a template like this:

```
Break down the following question into multiple dependent steps t
For each step, include the question and the corresponding paramet
- Question: The text of the question.
- Parameter: The parameter to be filled with the previous answer.
```

```
----
```

```
Original question: "What is the average August temperature at the
Multiple dependent questions:
```

To illustrate what such a sequence of questions might look like, you can try running the prompt above in ChatGPT. Replacing the user question with “What is the average August temperature at the most popular sandy beach in Cornwall?” will produce a response similar to the one below:

```
[
  {
    "Question": "What is the most popular sandy beach in Cornwall"
    "Parameter": "None (initial query to the full data source)"
  },
  {
    "Question": "What are the recorded daily temperatures in Augu"
    "Parameter": "Beach Name from the previous answer"
  },
  {
    "Question": "What is the average temperature from the followi"
    "Parameter": "Daily Temperatures from the previous answer"
  }
]
```

Each question can then be executed against the vector store or SQL database, using the answer from the previous step as its parameter value. After processing the final question, you will have the context needed to answer the original question with the LLM.

Note

If using a SQL database, you could simplify this with native SQL functions like AVG. However, this example illustrates the concept rather than the optimal implementation.

While I won't provide a full implementation here, this method builds on advanced RAG techniques, such as:

- Routing a natural language question to the relevant content store.
- Generating a query for specific content stores, like SQL databases, from a natural language question.

While LangChain doesn't offer a dedicated class for multi-step question decomposition, you may find inspiration in LlamaIndex's `MultiStepQueryEngine` class, which is worth exploring for further ideas.

This concludes our exploration of question transformation techniques for enhancing retrieval effectiveness. In the next chapter, you'll learn additional methods to improve RAG performance.

9.6 Summary

- The quality of LLM answers depends on well-structured questions, as vague or complex queries can mislead the vector store and LLM, resulting in weaker results.
- Poorly phrased questions lead to less accurate retrievals and answers, making clear and concise questions essential.
- The "Rewrite-Retrieve-Read" approach improves answers by refining the original question, helping retrieve better context and enhancing LLM responses.
- Splitting questions into explicit sub-questions, executed separately, ensures better retrieval, and LangChain's `MultiQueryRetriever` can combine these answers into a comprehensive response.
- Highly detailed queries can return overly specific results, missing the broader context needed for a complete answer.
- Using coarse chunks for synthesis and fine-grained chunks for detail, or

employing “step-back questions” to broaden queries, helps capture high-level context.

- Embedding hypothetical questions alongside document chunks improves retrieval by aligning them with potential user questions.
- Generating hypothetical documents with HyDE retrieves semantically aligned content based on user questions without altering chunk embeddings.
- Decomposing questions into independent sub-questions allows for individual retrieval and synthesis.
- Generating a sequence of parameterized questions for interdependent queries ensures that each step builds on the previous answer, systematically resolving dependencies for a final answer.

10 Query generation, routing and retrieval post-processing

This chapter covers

- Generate metadata queries directly from user questions
- Convert user questions into database-specific queries (e.g., SQL, SPARQL)
- Route questions to the appropriate handler based on intent
- Enhance result relevance using Reciprocal Rank Fusion (RRF)

In chapters 8 and 9, you improved RAG answer accuracy using advanced indexing and query transformations. Indexing strengthens embedding effectiveness for broader chunks, adding richer context, while query transformations boost the precision of vector store retrieval.

Now, you'll dive into three more advanced RAG techniques. First, you'll learn to generate queries specific to the type of content store in use. For instance, you'll see how to generate SQL from a user's natural language question to retrieve data from a relational database. Your setup might include several types of content stores—such as vector stores, a relational database, or even a knowledge graph database. You'll use the LLM to direct the user's question to the right content store.

Finally, you'll refine the retrieved results to send only the most relevant content for synthesis, filtering out unnecessary data to maintain clarity and relevance.

10.1 Content Database Query Generation

To give an LLM the best information possible for answering user questions, you often need to access other databases beyond your vector store. Many of these databases hold structured data and only accept structured queries. You

might wonder if the difference between unstructured user questions and the structured queries required for these databases poses a problem. This gap can be bridged with an LLM's help. Let's look at common content stores in LLM applications and typical ways to retrieve data from them.

- **Vector Store (or Vector Database):** Vector Store (or Vector Database): You're already familiar with vector stores, which hold document chunks along with their embeddings in a vector-based index to enable *semantic retrieval*. This approach, known as *dense retrieval*, uses embeddings—compact vectors with hundreds or thousands of dimensions that capture the semantic meaning of text. Similarity between a user query and stored chunks is computed based on the distance between these dense vectors. An alternative is *sparse retrieval* (also called *lexical retrieval*), which many vector databases support, though it doesn't require one. In the indexing phase, each document chunk is tokenized, and an inverted index is built to map each unique token to the list of chunks in which it appears. During querying, the user's question is tokenized in the same way, and each token is matched against the inverted index to retrieve relevant chunks. These are then ranked using relevance scoring methods like BM25 or TF-IDF, based on how well the term statistics of each chunk align with the query. Sparse search excels at precise, keyword-driven queries, supports Boolean logic (such as “must” and “must not”), and offers strong explainability by directly linking results to matching terms in the query.
- **Relational (SQL) Database:** An LLM application can connect to a relational database, which stores structured facts in tables. Data is typically retrieved using SQL queries. For example, a database might contain seasonal temperatures at tourist resorts or lists of available hotels and car rentals by location. LLMs can assist by generating SQL queries from natural language questions—a technique known as text-to-SQL. This approach is gaining popularity as a way to make databases accessible to non-technical users.
- **Document, Key-Value, or Object Databases:** These databases store data as documents or objects, typically in JSON or BSON format. Because many LLMs have been extensively trained on JSON structures, they can accurately convert user questions into JSON-based queries that align with the database schema. Notably, many document databases have

recently been rebranded as vector databases after introducing support for vector fields—used to store embeddings—and adding vector search and similarity capabilities.

- **Knowledge Graph Databases:** Knowledge graphs represent data as a graph, where nodes correspond to entities and edges define their relationships. Originally popularized by companies like Facebook and LinkedIn to infer connections in social networks, graph databases are now increasingly used in LLM applications. LLMs can transform unstructured text into structured knowledge graphs—a process often referred to as Knowledge Graph Enhanced RAG—resulting in a more compact and structured representation compared to vector stores. Once data is stored in a graph database, it can be queried using graph-specific languages like SPARQL or Cypher, enabling complex reasoning and inference that go beyond simple similarity search. This forms the basis of *GraphRAG*. We'll explore how LLMs can assist in building these graph structures from raw text, generating SPARQL or Cypher queries, and finally converting query results back into natural language responses.

Now, let's dig into the process of transforming a user's question into a structured query for different databases, starting with retrieving document chunks from a vector store using metadata.

10.2 Self-Querying (Metadata Query Enrichment)

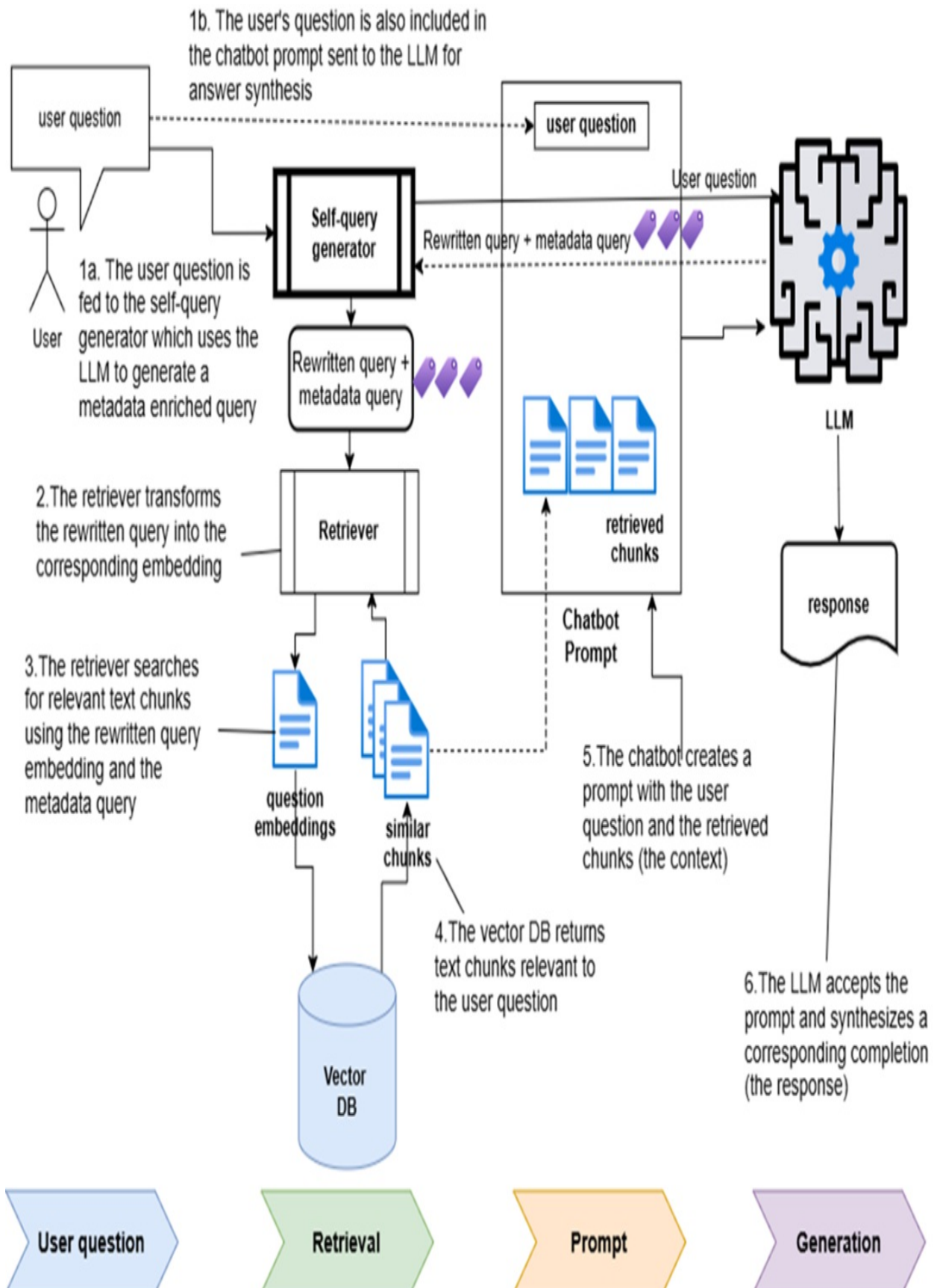
A vector store typically indexes document chunks by embedding for dense search, but it can also use keyword-based indexing in a few ways:

1. **Explicit Metadata Tags:** You can add metadata to each chunk, such as the timestamp, filename or URL, topic, and keywords. These keywords can come from user input or ones you assign manually.
2. **Keyword Extraction via Algorithm:** Use algorithms like *TF-IDF* (Term Frequency-Inverse Document Frequency) or its extension, BM25, to identify relevant keywords for each chunk based on word frequency and importance.
3. **Keyword Suggestions from the LLM:** You can ask the LLM to generate keywords for tagging each chunk.

With keywords attached to chunks through any of these methods, you can perform a semantic search, focusing only on chunks filtered by a keyword-based (or “sparse”) search. If your chatbot’s UI allows users to filter results directly—such as with dropdown options—your vector store query can explicitly include a metadata filter based on those selections. However, more commonly, you’ll automate this filtering by inferring relevant metadata from the user’s question. This technique, known as “self-querying” or “self-metadata querying,” enables your application to automatically generate a query enriched with metadata filters based on the user’s question.

In a self-querying flow, the user’s original question is transformed into an enriched query with both a metadata filter and a semantic component, enabling a combined dense and sparse search. This process, illustrated in figure 10.1, resembles the setup shown in figure 8.

Figure 10.1 Self-querying workflow: the original question turns into a semantic query with an embedded metadata filter. When this enriched query runs on the vector store, it first selects chunks matching the metadata filter, then applies semantic search on this refined set.



Now that you understand the basics of self-metadata querying, let's go through the steps to implement it. We'll start with the ingestion phase, where you'll tag each chunk with relevant metadata keywords. Then, in the Q&A phase, I'll show you two methods for generating self-metadata queries: one using the built-in `SelfQueryRetriever` and another that uses LLM function calling.

10.2.1 Ingestion: Metadata Enrichment

To use metadata effectively, start by re-importing the UK tourist destination data into a new collection, this time storing metadata for each chunk. Here's how to set up the environment:

Initialize the Environment

Open a new OS shell, navigate to the chapter 10 code folder, activate the virtual environment, and create a new Jupyter notebook.

```
C:\Github\building-llm-applications\ch10>
C:\Github\building-llm-applications\ch10>python -m venv env_ch10
C:\Github\building-llm-applications\ch10>pip install notebook lan
c:\Github\building-llm-applications\ch10>env_ch10\Scripts\activat
```

Then, in Jupyter Notebook, go to File > New > Notebook, and save it as `10-query_generation.ipynb`.

Define Metadata

Identify keywords to tag each chunk, such as:

- source: URL of the original content
- destination: the tourist destination referenced
- region: the UK region of the destination

Manually define mappings for destination and region, and dynamically generate the source URL for each chunk.

Set Up Chroma DB Collection

Configure the Chroma DB collection with the code shown in the following listing:

Listing 10.1 Setting up the ChromaDB collection

```
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings
import getpass

OPENAI_API_KEY = getpass.getpass('Enter your OPENAI_API_KEY')

uk_with_metadata_collection = Chroma(
    collection_name="uk_with_metadata_collection",
    embedding_function=OpenAIEmbeddings(openai_api_key=OPENAI_API_KEY)

uk_with_metadata_collection.reset_collection()  #A
```

Define Ingestion Content and Splitting Strategy

Outline the content and define a text splitting strategy to process the documents. Listing 10.2 shows how to set this up:

Listing 10.2 Defining content to be ingested and text splitting strategy

```
from langchain_community.document_loaders import AsyncHtmlLoader
from langchain_community.document_transformers import Html2TextTr
from langchain_text_splitters import RecursiveCharacterTextSplitt
from langchain_core.documents import Document

html2text_transformer = Html2TextTransformer()

text_splitter = RecursiveCharacterTextSplitter(  #A
    chunk_size=1000, chunk_overlap=100
)

def split_docs_into_chunks(docs):
    text_docs = html2text_transformer.transform_documents(docs)
    chunks = text_splitter.split_documents(text_docs)

    return chunks
```

```

uk_destinations = [
    ("Cornwall", "Cornwall"), ("North_Cornwall", "Cornwall"),
    ("South_Cornwall", "Cornwall"), ("West_Cornwall", "Cornwall")
    ("Tintagel", "Cornwall"), ("Bodmin", "Cornwall"), ("Wadebridg
    ("Penzance", "Cornwall"), ("Newquay", "Cornwall"), ("St_Ives"
    ("Port_Isaac", "Cornwall"), ("Looe", "Cornwall"), ("Polperro"
    ("Porthleven", "Cornwall"),
    ("East_Sussex", "East_Sussex"), ("Brighton", "East_Sussex"),
    ("Battle", "East_Sussex"), ("Hastings_(England)", "East_Susse
    ("Rye_(England)", "East_Sussex"), ("Seaford", "East_Sussex"),
    ("Ashdown_Forest", "East_Sussex")
]

wikivoyage_root_url = "https://en.wikivoyage.org/wiki"

uk_destination_url_with_metadata = [ #C
    ( f'{wikivoyage_root_url}/{destination}', destination, region
    for destination, region in uk_destinations]

```

The next step is to ingest the content and the related metadata.

Ingest Content with Metadata

Enrich the content with metadata by processing each document chunk as shown in listing 10.3.

Listing 10.3 Enriching chunks with related metadata

```

for (url, destination, region) in uk_destination_url_with_metadat
    html_loader = AsyncHtmlLoader(url) #A
    docs = html_loader.load() #B

    docs_with_metadata = [
        Document(page_content=d.page_content,
            metadata = {
                'source': url,
                'destination': destination,
                'region': region})
        for d in docs]

    chunks = split_docs_into_chunks(docs_with_metadata)

    print(f'Importing: {destination}')
    uk_with_metadata_collection.add_documents(documents=chunks)

```

Now your collection is ready, with each document chunk enriched with metadata. You can query this content and apply metadata filters to refine search results based on keywords like destination, region, or source.

10.2.2 Q & A on a Metadata-Enriched Collection

There are three ways to query metadata-enriched content:

1. Explicit Metadata Filters: Specify the metadata filter manually.
2. SelfQueryRetriever: Automatically generate the metadata filter using the SelfQueryRetriever.
3. Structured LLM Function Call: Infer the metadata filter with a structured call to an LLM function.

Let's explore these methods, starting with explicit filtering.

Querying with an Explicit Metadata Filter

You can leverage the metadata attached to each chunk by explicitly adding a filter to the retriever. Here's an example:

```
question = "Events or festivals"
metadata_retriever = uk_with_metadata_collection.as_retriever(sea
result_docs = metadata_retriever.invoke(question)
```

When you print `result_docs`, you'll see that only chunks tagged with ``destination: Newquay`` are returned, confirming that the filter is working correctly:

```
[Document(metadata={'destination': 'Newquay', 'region': 'Cornwall
Document(metadata={'destination': 'Newquay', 'region': 'Cornwall
```

To adjust the filter, instantiate a new retriever with the updated parameters.

Automatically Generating Metadata Filters with SelfQueryRetriever

You can also generate metadata filters automatically with

SelfQueryRetriever. This tool interprets the user's question to infer the appropriate filter. First, import the necessary libraries, as shown in the following listing:

Listing 10.4 Setting up metadata field information

```
from langchain.chains.query_constructor.base import AttributeInfo
from langchain.retrievers.self_query.base import SelfQueryRetriever
from langchain_openai import ChatOpenAI
```

Next, define the metadata attributes to infer from the question:

```
metadata_field_info = [
    AttributeInfo(
        name="destination",
        description="The specific UK destination to be searched",
        type="string",
    ),
    AttributeInfo(
        name="region",
        description="The name of the UK region to be searched",
        type="string",
    )
]
```

Now, set up the SelfQueryRetriever with the question, without specifying a manual filter:

```
question = "Tell me about events or festivals in the UK town of Newquay"
llm = ChatOpenAI(model="gpt-4o-mini", openai_api_key=OPENAI_API_KEY)
self_query_retriever = SelfQueryRetriever.from_llm(
    llm, uk_with_metadata_collection, question, metadata_field_info
)
```

Invoke the retriever with the question:

```
result_docs = self_query_retriever.invoke(question)
```

Printing result_docs will confirm that only chunks related to Newquay are retrieved, matching the inferred filter:

```
[Document(metadata={'destination': 'Newquay', 'region': 'Cornwall'})
Document(metadata={'destination': 'Newquay', 'region': 'Cornwall'})
Document(metadata={'destination': 'Newquay', 'region': 'Cornwall'})]
```

flexibility.

Generating Metadata Filters with an LLM Function Call

You can also infer metadata filters by having the LLM map the question to a predefined metadata template with attributes you stored during ingestion. This approach offers greater flexibility than the `SelfQueryRetriever` but requires more setup.

First, import the libraries necessary to create a structured query with specific filters:

Listing 10.5 Importing libraries

```
import datetime
from typing import Literal, Optional, Tuple, List

from pydantic import BaseModel, Field
from langchain.chains.query_constructor.ir import (
    Comparator,
    Comparison,
    Operation,
    Operator,
    StructuredQuery,
)
from langchain.retrievers.self_query.chroma import ChromaTranslator
```

The `DestinationSearch` class translates the user question into a structured object with a `content_search` field containing the question (minus filtering details) and fields for inferred search filters. Listing 10.6 shows this setup:

Listing 10.6 Strongly-typed structured question including inferred filter attributes

```
class DestinationSearch(BaseModel):
    """Search over a vector database of tourist destinations."""

    content_search: str = Field(
        """
```

```

        description="Similarity search query applied to tourist d
    )
    destination: str = Field(
        '''
        description="The specific UK destination to be searched."
    )
    region: str = Field(
        '''
        description="The name of the UK region to be searched.",
    )

    def pretty_print(self) -> None:
        for field in self.__fields__:
            if getattr(self, field) is not None and getattr(self,
                self.__fields__[field], "default", None
            ):
                print(f"{field}: {getattr(self, field)}")

```

Build a Chroma DB Filter Statement from the Structured Query

Next, create a function to convert a `DestinationSearch` object into a filter compatible with Chroma DB, as shown in listing 10.7:

Listing 10.7 Building a ChromaDB specific filter statement from a structured query object

```

def build_filter(destination_search: DestinationSearch):
    comparisons = []

    destination = destination_search.destination #A
    region = destination_search.region #A

    if destination and destination != '': #B
        comparisons.append(
            Comparison(
                comparator=Comparator.EQ,
                attribute="destination",
                value=destination,
            )
        )
    if region and region != '': #C
        comparisons.append(
            Comparison(
                comparator=Comparator.EQ,
                attribute="region",
                value=region,
            )
        )

```

```

        )
    )

    search_filter = Operation(operator=Operator.AND, arguments=co
    chroma_filter = ChromaTranslator().visit_operation(search_fil
    return chroma_filter

```

Build a Query Chain to Convert the Question into a Structured Query

Now, define the query generator chain to convert the user question into a structured query with metadata filters:

Listing 10.8 Query generator chain

```

from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

system_message = """You are an expert at converting user question
You have access to a database of tourist destinations. \
Given a question, return a database query optimized to retrieve t

If there are acronyms or words you are not familiar with, do not
prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system_message),
        ("human", "{question}"),
    ]
)
llm = ChatOpenAI(model="gpt-4o-mini", openai_api_key=OPENAI_API_K
structured_llm = llm.with_structured_output(DestinationSearch, me
query_generator = prompt | structured_llm

```

Let's try out the chain with the same question used earlier:

```

question = "Tell me about events or festivals in the UK town of N
structured_query = query_generator.invoke(question)

```

Printing `structured_query` shows the question converted into a structured object:

```

DestinationSearch(content_search='events festivals', destination=

```

With the structured query created, generate a Chroma DB-compatible search filter:

```
search_filter = build_filter(structured_query)
```

The search_filter result will look like this:

```
{'$and': [{ 'destination': {'$eq': 'Newquay'} },  
  { 'region': {'$eq': 'Cornwall'} } ] }
```

Perform the vector search using the generated structured query and Chroma DB filter:

```
search_query = structured_query.content_search  
metadata_retriever = uk_with_metadata_collection.as_retriever(search_filter)  
answer = metadata_retriever.invoke(search_query)
```

The answer should closely match the output from the SelfQueryRetriever, returning chunks associated with Newquay:

```
[Document(metadata={'destination': 'Newquay', 'region': 'Cornwall
```

So far, you've focused on generating metadata-enriched queries for vector stores. In the next section, you'll learn how to generate SQL queries from natural language questions, enabling retrieval of structured data from relational databases.

10.3 Generating a Structured SQL Query

Many LLMs can transform user questions into SQL queries, enabling access to relational databases directly from LLM applications. While LLMs are continually improving in generating accurate SQL, challenges remain, especially when working with complex schemas or specific database structures. LangChain enhances these capabilities with evolving Text-to-SQL features, but there are some common issues you should consider.

A helpful reference here is the paper "Evaluating the Text-to-SQL Capabilities of Large Language Models" by Nitarshan Rajkumar et al.

(<https://arxiv.org/pdf/2204.00498.pdf>). Though dated, this paper offers practical insights into common pitfalls and solutions. The main finding was that "hallucinations"—incorrect table and column names—can often be reduced by using few-shot prompts that include the schema and sample records for the target table. An example schema from the paper looks like this:

```
CREATE TABLE "state" (  
    "state_name" TEXT,  
    "population" INT DEFAULT NULL,  
    "area" DOUBLE DEFAULT NULL,  
    "country_name" VARCHAR(3) NOT NULL DEFAULT '',  
    "capital" TEXT,  
    "density" DOUBLE DEFAULT NULL  
);  
/* example rows  
state_name      population      area      country_name      capital  
alabama         3894000        51700.0   usa               montgome  
alaska          401800         591000.0  usa               juneau  
arizona         2718000        114000.0  usa               phoenix  
*/  
-- Answer the following question using the above table schema:  
-- {user_question}
```

Using the `CREATE TABLE` command along with sample data helps the LLM better understand the structure and constraints, minimizing incorrect column and table references.

10.3.1 Installing SQLite

SQLite does not require full installation. Unzip the package, place it in a folder, and add the folder to your system's Path environment variable. Refer to Appendix D for setup instructions on Windows. For other operating systems, consult the SQLite documentation.

10.3.2 Setting Up and Connecting to the Database

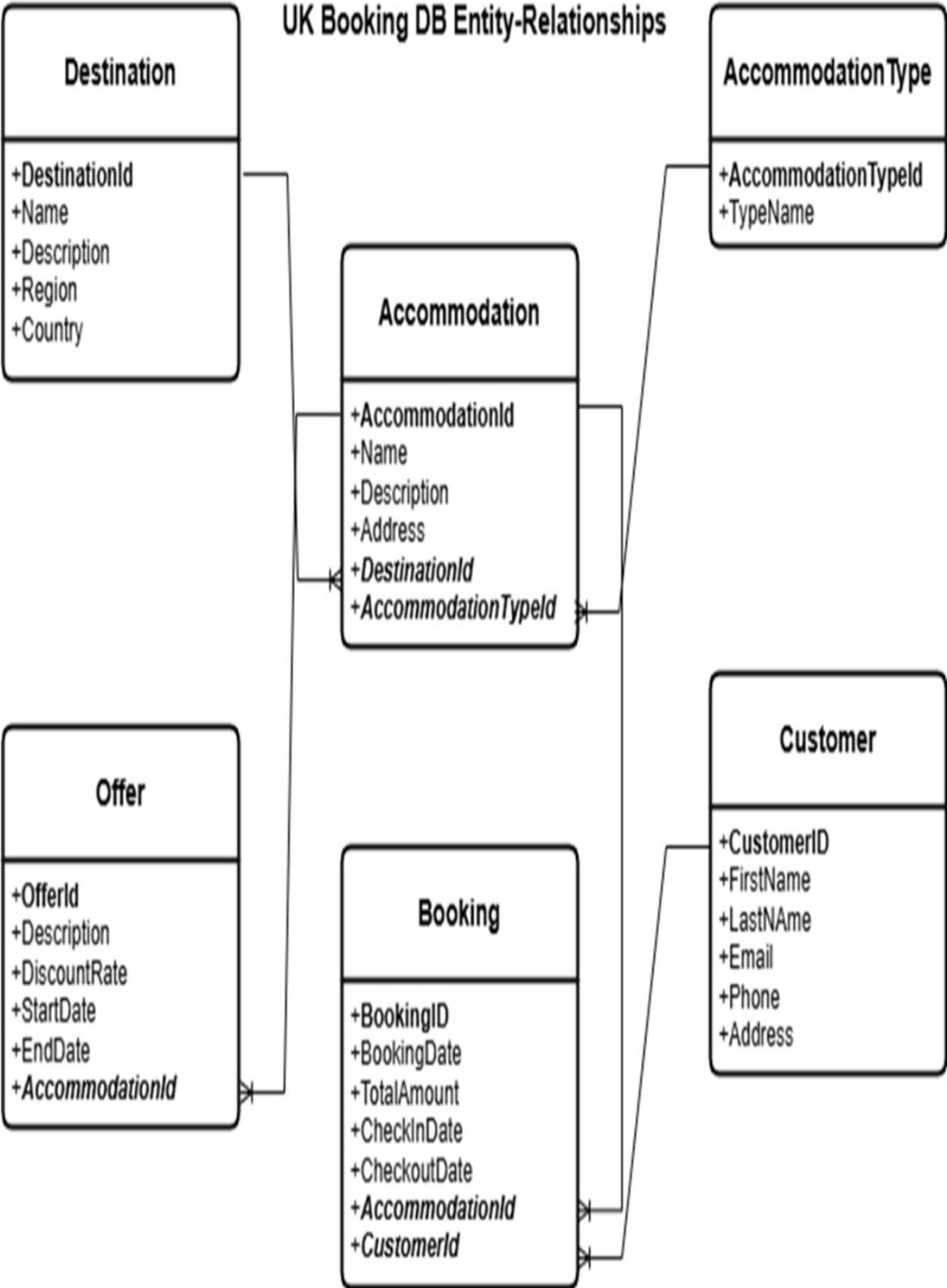
Let's create a booking database called `ukBooking` to store UK destinations, accommodations, and special offers.

Here is the relational diagram for the `ukBooking` database. Each table shows

its primary key (PK) and foreign key (FK) columns, with relationships marked by arrows connecting related tables. This setup visually represents the structure and relationships within the database.

Figure 10.2 Entity-Relationship diagram of the UkBooking DB

UK Booking DB Entity-Relationships



The database schema, shown in figure 10.2, includes the relationships between tables for destinations, accommodations, and offers.

Open your OS shell, navigate to the code folder, and enter the following command to create the UkBooking database:

```
C:\Github\building-llm-applications\ch10>sqlite3 UkBooking.db
```

This opens the SQLite terminal:

```
SQLite version 3.46.1 2024-08-13 09:16:08 (UTF-16 console I/O)
Enter ".help" for usage hints.
sqlite>
```

In the SQLite terminal, load the SQL scripts to create and populate the UkBooking database. Ensure these files are in C:\Github\building-llm-applications\ch10. Download them from GitHub if necessary:

```
sqlite> .read CreateUkBooking.sql
sqlite> .read PopulateUkBooking.sql
```

To confirm the setup, check for records in the Offer table:

```
sqlite> SELECT * FROM Offer;
```

You should see output similar to this:

```
1|1|Summer Special|0.15|2024-06-01|2024-08-31
2|2|Weekend Getaway|0.1|2024-09-01|2024-12-31
3|3|Early Bird Discount|0.2|2024-05-01|2024-06-30
4|4|Stay 3 Nights, Get 1 Free|0.25|2024-01-01|2024-03-31
...
```

Now the UkBooking database is ready for use with LangChain.

Return to the Jupyter notebook and import the libraries needed for SQL database connections:

```
from langchain_community.utilities import SQLDatabase
from langchain_community.tools import QuerySQLDataBaseTool
from langchain.chains import create_sql_query_chain
```

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
import getpass
import os
```

Use the following code to connect to the database and list available tables:

```
db = SQLiteDatabase.from_uri("sqlite:///UkBooking.db")
print(db.get_usable_table_names())
```

You should see a list of table names:

```
['Accommodation', 'AccommodationType', 'Booking', 'Customer', 'De
```

Run a sample query to verify the connection:

```
db.run("SELECT * FROM Offer;")
```

The output should display entries from the Offer table:

```
"[(1, 1, 'Summer Special', 0.15, '2024-06-01', '2024-08-31'), (2,
```

Now, you're set up to query the UkBooking database programmatically using LangChain.

10.3.3 Generating SQL Queries from Natural Language

Now that the setup is complete, you can start generating SQL queries directly from natural language questions. Here's how to test a simple query:

```
llm = ChatOpenAI(openai_api_key=OPENAI_API_KEY, model="gpt-4o-mini")
sql_query_gen_chain = create_sql_query_chain(llm, db)
response = sql_query_gen_chain.invoke({"question": "Give me some
```

Printing response will show the generated SQL query:

```
```sql\nSELECT "Offer"."OfferDescription", "Offer"."DiscountRate
```

However, if you attempt to execute this SQL directly against the database, you'll encounter an error due to the backticks (` `), which are non-SQL characters:

```
db.run(response)
```

```
'Error: (sqlite3.OperationalError) near "```sql\nSELECT "Offer".'
```

To clean up the SQL formatting, you can use the LLM to strip unnecessary characters and output a properly formatted SQL statement. Here's a simple chain setup for this:

#### **Listing 10.9 Chain to fix the formatting of the generated SQL**

```
clean_sql_prompt_template = """You are an expert in SQL Lite. You
which might contain unneeded prefixes or suffixes. Given the follo
transform it to a clean, executable SQL statement for SQL lite.
Only return an executable SQL statement which terminates with a s
Do not include the language name or symbols like ```.
```

```
Unclean SQL: {unclean_sql}"""
```

```
clean_sql_prompt = ChatPromptTemplate.from_template(clean_sql_pro
```

```
clean_sql_chain = clean_sql_prompt | llm
```

```
full_sql_gen_chain = sql_query_gen_chain | clean_sql_chain | Stro
```

Let's try out this full chain with a sample question and verify the output:

```
question = "Give me some offers for Cardiff, including the accomo
```

```
response = full_sql_gen_chain.invoke({"question": question})
```

```
print(response)
```

The output should be a clean SQL statement:

```
"SELECT Offer.OfferDescription, Offer.DiscountRate, Accommodation
```

This approach ensures the SQL statement is correctly formatted and ready to execute against the database.

### **10.3.4 Executing the SQL Query**

Now, let's create a chain to generate and execute SQL queries.

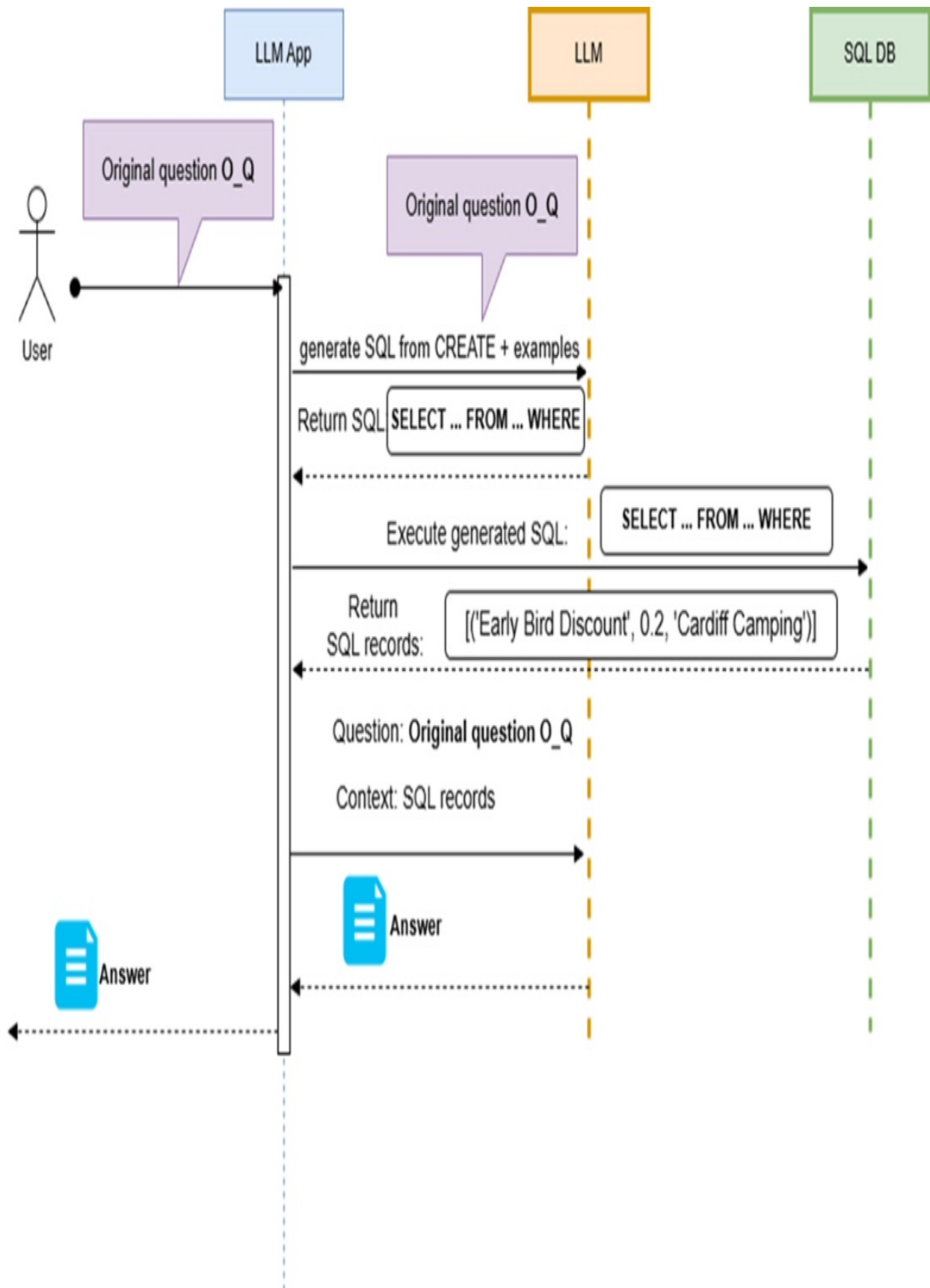
```
sql_query_exec_chain = QuerySQLDataBaseTool(db=db)
sql_query_gen_and_exec_chain = full_sql_gen_chain | sql_query_exe
response = sql_query_gen_and_exec_chain.invoke({"question":questi
```

Printing response should show the following output:

```
"[('Early Bird Discount', 0.2, 'Cardiff Camping')]"
```

This setup allows you to retrieve data from a relational database by using a combined chain (`sql_query_gen_and_exec_chain`) that handles both SQL generation and execution. You can easily integrate this chain within a broader RAG (Retrieval-Augmented Generation) setup, as discussed in earlier sections. Try extending this integration as an exercise.

**Figure 10.3 RAG with SQL Workflow: The LLM converts the natural language question into a SQL query, which is executed on the SQL database. The database returns records that are then processed by the LLM to generate the final answer.**





The sequence diagram in figure 10.3 can give you a visual idea of what the full RAG with SQL workflow would look like.

#### **TIP**

LangChain’s `SQLDatabaseChain` class provides a streamlined way to generate SQL queries directly from user questions. This tool uses an LLM and your database connection to automatically create “few-shot” prompts, similar to those recommended in the Rajkumar paper. Experimenting with `SQLDatabaseChain` can be highly beneficial if you plan to incorporate relational databases into your RAG setup. A sequence diagram (Figure 10.3) shows a high-level overview of this complete RAG workflow.

## **10.4 Generating a Semantic SQL Query**

In the previous section, you learned how to generate SQL queries from natural language. However, these queries rely on “strict” SQL, meaning they depend on exact matching and traditional relational operations. Relational databases operate on record sets using operations like `SELECT`, `JOIN`, `WHERE`, and `GROUP BY`, where filters are based on exact string matches or numeric comparisons.

But what if you want to expand the SQL search to include results that are similar in meaning to what the user intended? This requires a shift from standard SQL to a *semantic SQL search*. In this section, I’ll provide an overview of how to implement semantic SQL search, a topic that continues to evolve.

### **10.4.1 Standard SQL Query**

A standard SQL query filters based on exact matches. For example, to find users with the first name “Roberto,” you would use:

```
SELECT first_name, last_name FROM user WHERE first_name = 'Robert
```

This query returns only users named “Roberto.” It won’t return records for

names like Robert, Rob, Robbie, Roby, Robin, Roe, Bobby, Bob, or Bert.

You can loosen this search slightly with the `LIKE` operator for partial matching. For instance, to find users with names that start with “Rob”:

```
SELECT first_name, last_name FROM user WHERE first_name LIKE 'Rob'
```

This query will return names like Roberto, Robert, Rob, Robbie, Roby, and Robin but still won’t catch variations like Roe, Bobby, Bob, or Bert, as they don’t contain the string “Rob.”

### 10.4.2 Semantic SQL Query

With the rise of large language models (LLMs), several relational databases now support semantic search, which enables searches based on embeddings instead of exact matches. An example is *PGVector*, an extension for PostgreSQL that allows vector-based similarity searches using metrics such as L2, inner product, and cosine distance. This approach enables you to perform searches that return results based on meaning rather than exact text matches.

In this section, I’ll refer to this approach as semantic SQL search or SQL similarity search interchangeably.

### 10.4.3 Creating the Embeddings

To extend the “traditional SQL” approach with *PGVector*’s similarity search, you’ll need to add vector-based embeddings for any columns you want to use in semantic searches. Here’s how to do it.

1. **Add a Vector Column:** First, add a `VECTOR` type column to the table for each field you want to search by similarity. For example, to enable similarity search on `first_name`, add a column called `first_name_embedding`:

```
ALTER TABLE user ADD COLUMN first_name_embedding VECTOR
```

2. **Calculate Embeddings:** Next, compute the embedding values for each

`first_name`. You can do this directly within PostgreSQL if you have a function to generate embeddings, or you can compute embeddings externally using an API client like LangChain.

- In-Database Calculation: If PostgreSQL has a custom function like `calculate_my_embedding()` available, you can update the embeddings in-place with SQL:

```
UPDATE user
SET first_name_embedding = calculate_my_embedding(first_name)
```

- External Calculation with LangChain: If you're using a pre-built embedding function (e.g., OpenAI's), calculate embeddings externally and store them using the PGVector API. In listing 10.10 you can see an example of using LangChain's `OpenAIEmbeddings` wrapper to generate embeddings for `first_name` values and update the database (library imports are omitted for brevity).

**Listing 10.10 Using LangChain's `OpenAIEmbeddings` wrapper to generate embeddings**

```
db = SQLiteDatabase.from_uri(YOUR_DB_CONNECTION_STRING) #A
embeddings_model = OpenAIEmbeddings() #A

first_names_resultset_str = db.run('SELECT first_name FROM user')
first_names = [fn[0] for fn in eval(first_names_resultset_str)]

first_names_embeddings = embeddings_model.embed_documents(first_n
fn_emb = zip(first_names, first_names_embeddings) #D

for fn, emb in fn_emb:
 sql = f'UPDATE user SET first_name_embeddings = ARRAY{emb} WH
 db.run(sql)
```

By following these steps, you'll enable semantic search on `first_name` or other fields, allowing PGVector to retrieve records based on similarity, rather than exact matches.

### 10.4.4 Performing a Semantic SQL Search

After setting up the embeddings, you can perform a similarity search as follows:

```
embedded_query= embeddings_model.embed_query("Roberto")
query = (
 'SELECT first_name FROM user WHERE first_name_embeddings IS N
)
db.run(query)
```

This query returns variations of “Roberto,” including Roe, Bobby, Bob, and Bert, by ordering results based on similarity.

### 10.4.5 Automating Semantic SQL Search

Now that you understand how to generate embeddings and perform similarity searches in a SQL database, the final step is to create a prompt that can automatically generate SQL similarity queries. This process is similar to what we covered for generating traditional SQL queries. Once you design, implement, and test this prompt—and integrate it into a full chain within LCEL—your LLM application will be capable of generating semantic searches on PGVector or any SQL database that supports ARRAY (or similar) data types, seamlessly feeding the results to the LLM for synthesis.

### 10.4.6 Benefits of Semantic SQL Search

The simple example here only scratches the surface of semantic SQL’s capabilities. You can combine semantic filtering with exact matching or use multiple semantic filters, especially powerful in multi-table queries using joins. This approach allows highly nuanced searches, especially when combined with traditional SQL filtering.

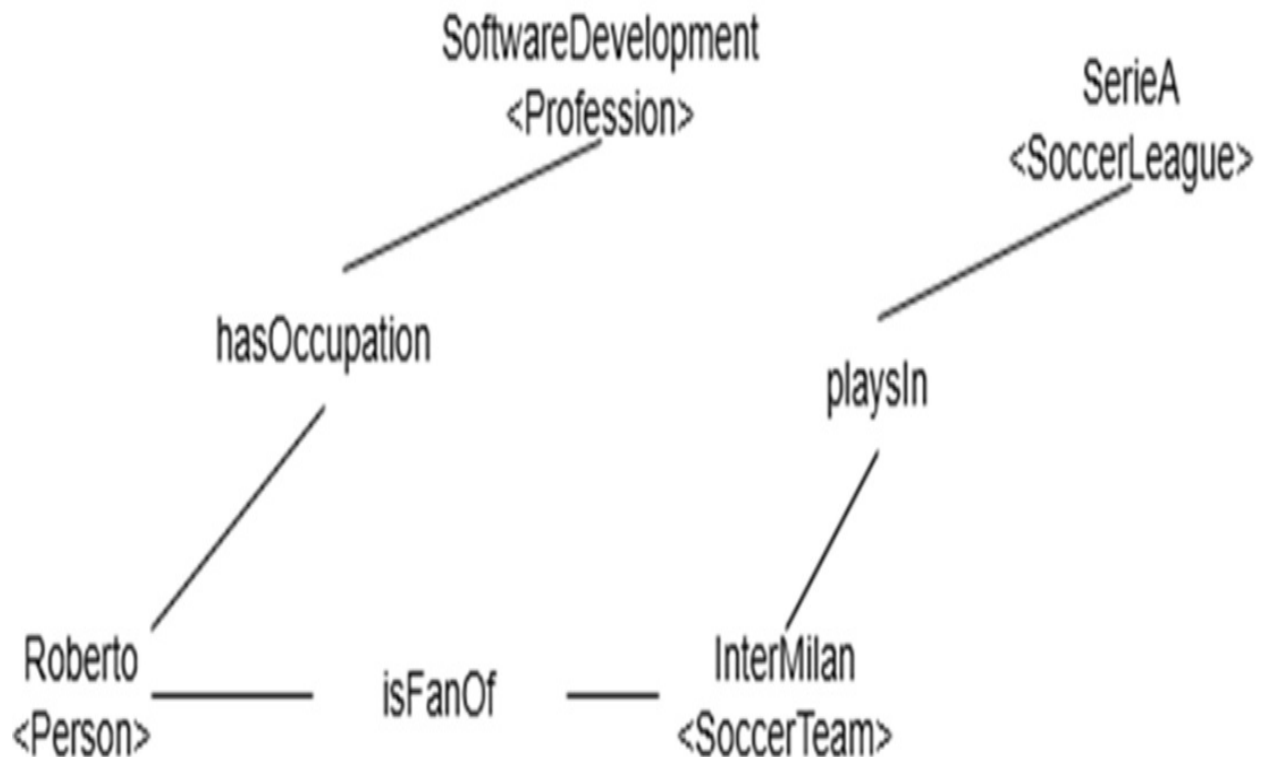
Later, I’ll show you how to combine metadata and semantic filtering in a vector store, which can achieve similar results. However, using multiple semantic filters in SQL offers greater flexibility, particularly for complex queries.

## 10.5 Generating Queries for a Graph Database

*Graph databases* are designed to store, navigate, and query data in a graph format, with nodes representing entities and edges defining relationships, as shown in Figure 10.4. They are well-suited for building knowledge graphs,

making them ideal for specialized domains that require advanced reasoning, inference, and explainability.

**Figure 10.4 Graph Representation of data: Nodes represent entities such as "Roberto" and "InterMilan," while edges like `hasOccupation` and `isFanOf` depict their relationships.**



Unlike relational databases, graph databases do not follow a universal standard. Some use the Resource Description Framework (RDF) to represent data as “triples”—a Subject-Predicate-Object format. For instance, in RDF:

```
(Roberto, hasOccupation, SoftwareDeveloper)
(Roberto, isFanOf, InterMilan)
(InterMilan, playsIn, SerieA)
```

This triple structure might look like:

```
@prefix ex: <http://example.org/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .

ex:Roberto rdf:type ex:Person .
ex:Roberto ex:hasOccupation ex:SoftwareDevelopment .
ex:Roberto ex:isFanOf ex:InterMilan .
```

```
ex:InterMilan rdf:type ex:SoccerTeam .
ex:InterMilan ex:playsIn ex:SerieA .

ex:SerieA rdf:type ex:SoccerLeague .
```

However, only some graph DBs use RDF. Others use proprietary graph representations and query languages like Cypher (for Neo4j) or Gremlin, rather than RDF and SPARQL.

As you can see in the figure 10.4, the graph data structure enables powerful, flexible representations of relationships that are difficult to capture with traditional databases.

This versatility makes graph DBs ideal for knowledge-intensive applications where deep understanding of relationships and reasoning is essential.

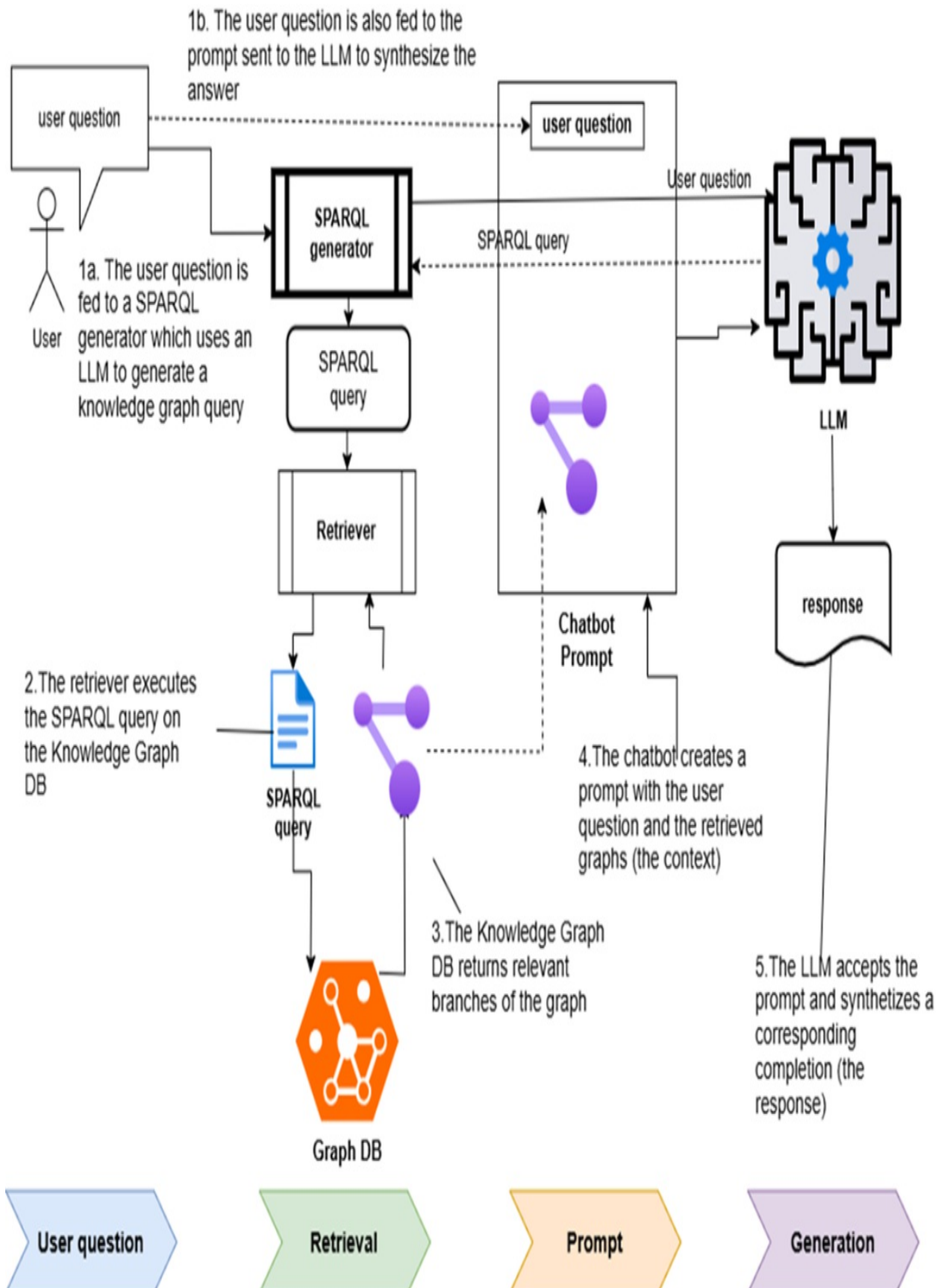
Graph databases have been around since the early 2000s, with Neo4j among the first. While they offer powerful, flexible ways to represent and query information, their complexity can be a hurdle. Recently, large language models (LLMs) have made graph DBs more accessible by enhancing several key functions:

- Entity and Relationship Extraction: LLMs can pull entities, relationships, and even full graphs from unstructured text, allowing you to store this data directly in a graph DB.
- Automated Query Generation: LLMs can generate complex Cypher or SPARQL queries from natural language questions, easing a task traditionally challenging for less experienced developers. To accomplish this, use a carefully crafted few-shot prompt with examples, which works best with a high-accuracy LLM like GPT-4o.
- Natural Language Answers: LLMs can convert query results (e.g., in RDF) into natural language responses. This process involves feeding the initial question, the generated Cypher or SPARQL query, and the results into a dedicated prompt, which a lower-cost LLM like GPT-4o-mini can handle effectively.

Figure 10.5 below shows the resulting Knowledge Graph RAG architecture, also known as KG-RAG, which closely resembles the setup used for vector

store based RAG.

**Figure 10.5 Knowledge Graph RAG Architecture (KG-RAG):** Similar to vector-store-based RAG setups, but using a SPARQL generator. The generator converts a natural language question into SPARQL, which is executed on the Knowledge Graph database. The retrieved graph data is then provided to the LLM, along with the original question, to synthesize the answer.





LangChain supports several graph DBs, including Neo4j and Amazon Neptune. While this book does not cover Knowledge Graph RAG in detail, I recommend consulting LangChain's documentation and examples. LangChain also offers practical blog posts, such as a collaborative piece with Neo4j on building DevOps RAG applications with knowledge graphs: <https://blog.langchain.dev/using-a-knowledge-graph-to-implement-a-devops-rag-application/>.

Below is a sample prompt template for generating Cypher queries, taken from LangChain's Neo4j QA Chain:

```
CYPHER_GENERATION_TEMPLATE = """Task:Generate Cypher statement to
Instructions:
Use only the provided relationship types and properties in the sc
Do not use any other relationship types or properties that are no
Schema:
{schema}
Note: Do not include any explanations or apologies in your respon
Do not respond to any questions that might ask anything else than
Do not include any text except the generated Cypher statement.
Examples: Here are a few examples of generated Cypher statements
How many people played in Top Gun?
MATCH (m:Movie {{title:"Top Gun"}})<-[:ACTED_IN]-()
RETURN count(*) AS numberOfActors
The question is:
{question}"""
```

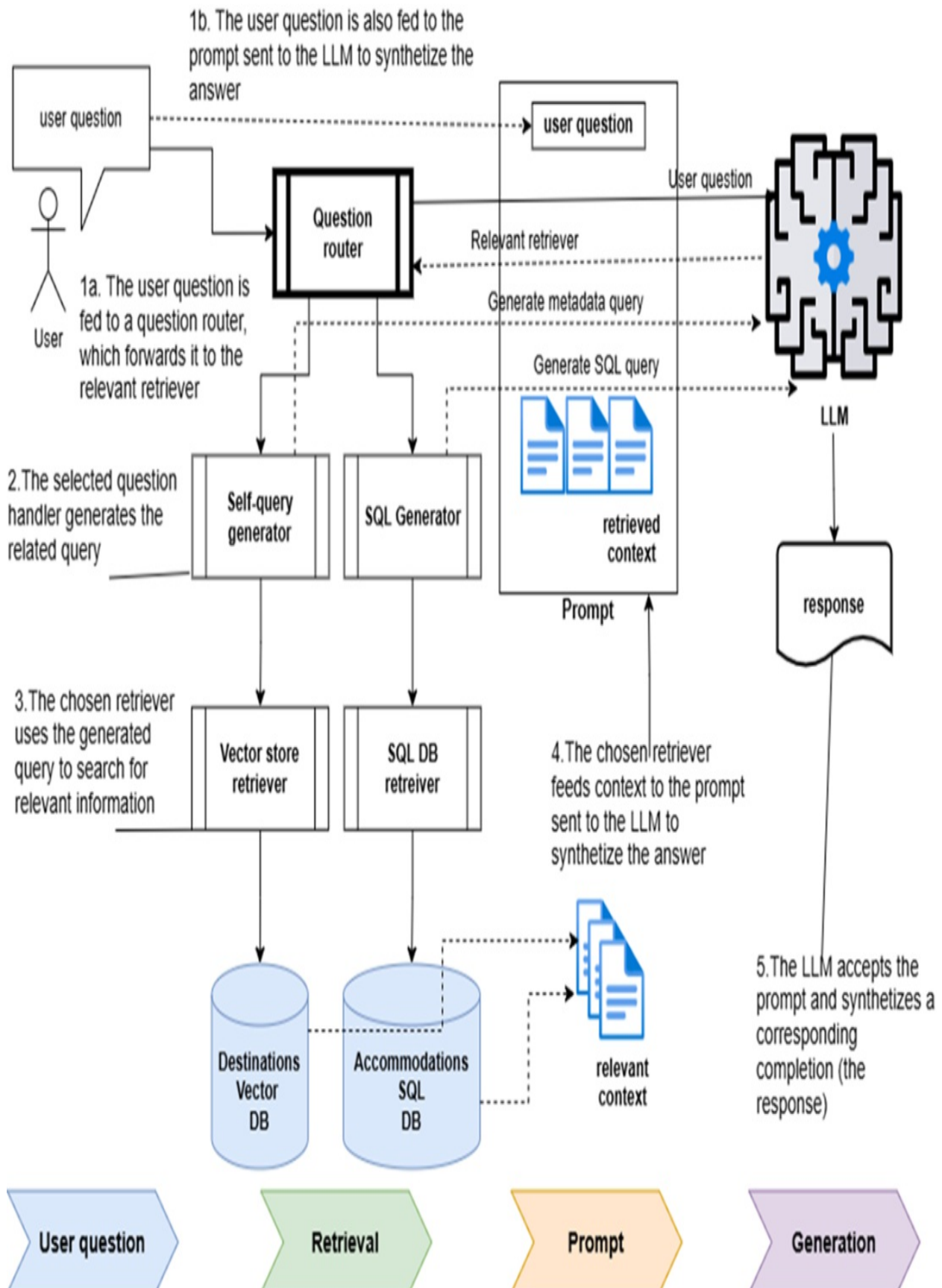
Graph DBs are evolving to meet new LLM-driven use cases, giving rise to *Knowledge Graph Embeddings*. This approach enriches knowledge graphs with textual descriptions and embeddings, supporting semantic search as a complement to traditional graph queries. For further reading on this technique, see this Nature article: <https://www.nature.com/articles/s41598-023-38857-5>. For a comprehensive guide, I recommend *Knowledge Graph Enhanced RAG* by Tomaž Bratanič and Oskar Hane.

These tools allow you to leverage the structured knowledge of graph DBs, combined with the adaptability of LLMs, for powerful retrieval-augmented generation solutions.

## 10.6 Chain Routing

An application's content might reside in multiple storage types, not just vector stores for unstructured text. You may also use relational databases for structured data, document databases for semi-structured content, and knowledge graph databases for entity relationships. Additionally, the application might need to connect to different LLMs depending on the task, as some LLMs are optimized or more cost-effective for specific functions. As a result, the RAG architecture often branches into a tree structure, with each branch tailored to specific types of queries or tasks, as shown in figure 10.6.

**Figure 10.6 Complex RAG architecture with branching pathways, each optimized for specific tasks, such as answering questions about tourist Destinations (from a vector store) or Accommodation offers (from a relational SQL DB).**



This figure illustrates a RAG setup with several branches, each designed to handle different application tasks. For example, one branch could address factual questions about tourist destinations, while another branch handles questions about available accommodation offers.

Suppose a user asks about a tourist destination. In that case, you'd likely route this query to a RAG chain based on a vector store. If the user's question is about accommodation offers, you might route it to a RAG chain connected to the UkBooking database introduced earlier.

To route each question to the correct chain, you use a routing chain. This chain analyzes the question to determine the best-suited chain for handling it. Let's go over the implementation of a routing chain for this purpose.

### 10.6.1 Setting Up Data Retrievers

To streamline setup, we'll reuse the vector store and relational database configurations from previous sections. Import the necessary libraries:

```
from typing import Literal
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field
from langchain.schema.runnable import RunnableLambda
```

Now create the corresponding retriever chains:

```
tourist_info_retriever_chain = RunnableLambda(lambda x: x['question'])
uk_accommodation_retriever_chain = full_sql_gen_chain | sql_query
```

These retriever chains direct questions to the appropriate data source. Next, we'll build a router to direct user questions to one of these retriever chains.

### 10.6.2 Setting Up the Query Router

We'll implement a question router using an LLM. The LLM will analyze each question and determine the best retriever chain based on its content. The

prompt will specify the function of each retriever: the vector store for general tourist information and the relational database for accommodation bookings.

The router function, defined in listing 10.11, binds the LLM's response to a typed object, instantiating the datasource attribute with either "tourist\_info\_store" or "uk\_booking\_db" depending on the question's intent.

**Listing 10.11 Routing the query to the correct retriever**

```
class RouteQuery(BaseModel):
 """Route a user question to the most relevant datasource."""
 datasource: Literal["tourist_info_store", "uk_booking_db"] =
 '...'
 description="Given a user question, route it either to a
)"

llm = ChatOpenAI(openai_api_key=OPENAI_API_KEY, model="gpt-4o-mini")
structured_llm_router = llm.with_structured_output(RouteQuery) #
system = """You are an expert at routing a user question to a tou
or to an UK accommodation booking relational database.
The vector store contains tourist information about UK destinatio
Use the vectorstore for general tourist information questions on
For questions about accommodation availability or booking, use th
route_prompt = ChatPromptTemplate.from_messages(
 [
 ("system", system),
 ("human", "{question}"),
]
)

question_router = route_prompt | structured_llm_router
```

This setup enables the LLM to intelligently route each question to the appropriate data source, improving response accuracy based on question intent.

## Testing the Router Chain

Let's test the router chain with a question about tourist information and another about accommodation booking:

```
selected_data_source = question_router.invoke(
 {"question": "Have you got any offers in Brighton?"}
)

print(selected_data_source)
```

Expected output:

```
datasource='uk_booking_db'
```

Then test with a tourist-related question:

```
selected_data_source = question_router.invoke(
 {"question": "Where are the best beaches in Cornwall?"}
)

print(selected_data_source)
```

Expected output:

```
datasource='tourist_info_store'
```

The router correctly identifies the appropriate data source!

## Setting Up the Retriever Chooser

Now, let's implement a function to select the correct retriever based on the chosen data source ('uk\_booking\_db' or 'tourist\_info\_store').

### Listing 10.12 Retriever Chooser Function

```
retriever_chains = {
 'tourist_info_store': tourist_info_retriever_chain,
 'uk_booking_db': uk_accommodation_retriever_chain
}

def retriever_chooser(question):
 selected_data_source = question_router.invoke(
 {"question": question})

 return retriever_chains[selected_data_source.datasource]
```

Let's test the retriever chooser function with a sample question:

```
chosen = retriever_chooser('Tell me about events or festivals in
print(chosen)
```

Expected output:

```
first=RunnableLambda(lambda x: x['question']) last=VectorStoreRet
```

The output confirms that the correct retriever chain instance is selected based on the question's intent. This setup ensures the question is routed to the best-matched data source for accurate results.

### 10.6.3 Integrating the Chain Router into a Full RAG Chain

The final step is to integrate the chain router into a complete RAG chain, enabling the workflow of retriever selection, query execution, and answer synthesis. See listing 10.13 for an example.

**Listing 10.13 Full RAG Chain for Routing, Retrieval, and Answer Synthesis**

```
from langchain_core.runnables import RunnablePassthrough

rag_prompt_template = """
Given a question and some context, answer the question.
If you get a structured context, like a tuple, try to infer the m
typically they refer to accommodation offers, and the number is a
If you do not know the answer, just say I do not know.

Context: {context}
Question: {question}
"""

rag_prompt = ChatPromptTemplate.from_template(rag_prompt_template)

def execute_rag_chain(question, chosen_retriever):
 full_rag_chain = (
 {
 "context": {"question": RunnablePassthrough()} | chos
 "question": RunnablePassthrough(), #B
 }
 | rag_prompt
 | llm
```

```
 | StrOutputParser()
)

 return full_rag_chain.invoke(question)
```

Let's test the RAG chain with both an accommodation query and a tourist information query.

### **Example: Asking about Accommodation Offers**

```
question = 'Give me some offers for Cardiff, including the accomm
chosen_retriever = retriever_chooser(question)
answer = execute_rag_chain(question, chosen_retriever)
```

Expected output:

One offer for Cardiff is the "Early Bird Discount" at Cardiff Cam

### **Example: Asking about Tourist Information**

```
question_2 = 'Tell me about events or festivals in the UK town of
chosen_retriever_2 = retriever_chooser(question_2)
answer2 = execute_rag_chain(question_2, chosen_retriever_2)
```

Expected output:

In Newquay, the **Cornish Film Festival** is held annually each N

In both cases, the RAG chain correctly routes each question to the appropriate retriever, performs the retrieval, and synthesizes a coherent answer by feeding the context and original question to the LLM.

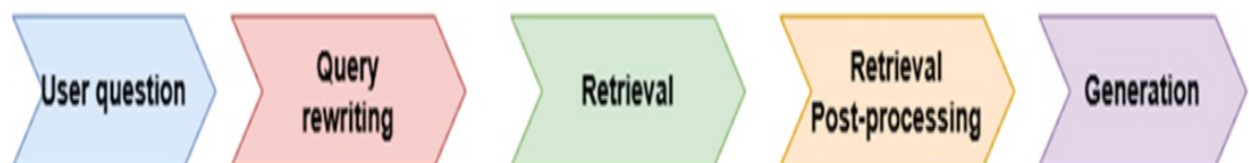
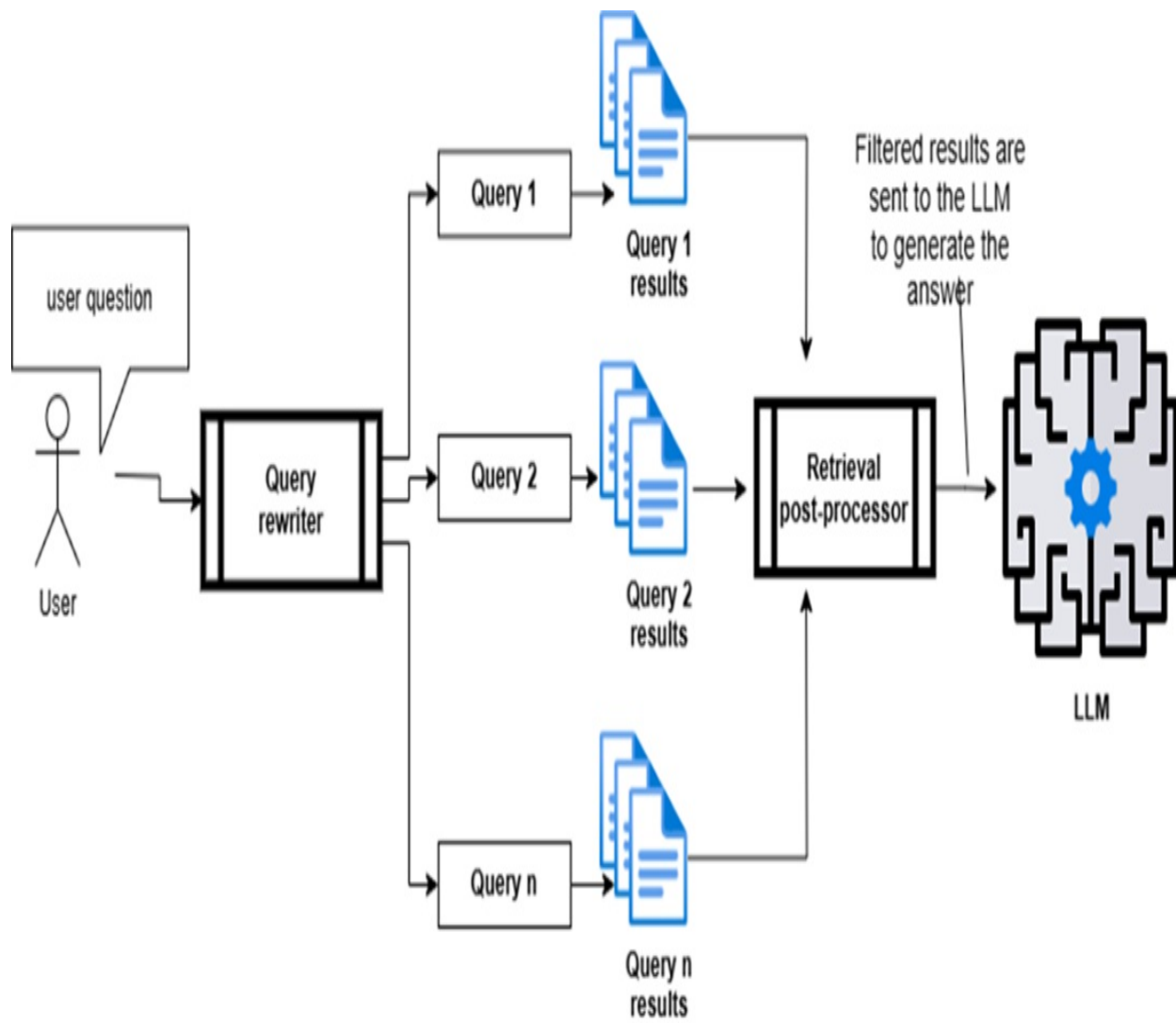
We're nearing the end of our exploration of advanced RAG techniques. One more topic remains: *retrieval post-processing*, which focuses on refining the chunks sent as context to the LLM, further optimizing the relevance and clarity of responses.



## 10.7 Retrieval Post-Processing

After using the techniques discussed so far to improve the effectiveness and accuracy of RAG retrieval, you'll likely have a list of document chunks (or nodes) from the content store. Before passing these to the LLM for answer synthesis, you may want to perform post-processing to filter out less relevant content, ensuring the LLM produces a concise and accurate response, as shown in figure 10.7.

**Figure 10.7 Retrieval Post-processing: Retrieved chunks from the vector store are filtered to remove irrelevant content, ensuring only high-quality chunks are sent to the LLM for answering the user's question.**



In the following sections, I'll introduce some key post-processing techniques.

### 10.7.1 Similarity Postprocessor

A straightforward way to reduce the number of chunks returned by a similarity retriever (often a vector-based retriever using semantic distance) is to apply a cutoff to similarity scores. Chunks below a specific similarity score, or above a certain distance, are discarded.

In LangChain, you can set this similarity threshold before executing the search by instantiating a “score threshold” similarity retriever from the vector store. Set the `search_type` to `"similarity_score_threshold"` and specify the threshold in `search_kwargs`:

```
score_threshold_similarity_retriever = vector_store.as_retriever(
 search_type="similarity_score_threshold", search_kwargs={"sco
)
```

After instantiating this retriever, you can execute the search with its `get_relevant_documents()` method:

```
doc_chunks = score_threshold_similarity_retriever.get_relevant_d
```

This retrieves only documents with similarity scores above the specified threshold, ensuring that only highly relevant content is passed to the LLM.

## 10.7.2 Keyword Postprocessors

Another post-processing approach is to filter retrieved document chunks by keywords. This allows you to include or exclude chunks based on the presence of specific terms.

While LangChain does not provide a built-in keyword post-processor, you can implement one in Python as follows (for demonstration purposes):

```
selected_chunks = [c for c in chunks
 if set(c.split()).intersection(required_keywords)
 and not set(c.split()).intersection(excluded_keywords)
```

This code filters chunks to include only those containing `required_keywords` and excludes any that contain `excluded_keywords`.

## 10.7.3 Time Weighting

You might also want to prioritize chunks based on how recently they were accessed. To do this, include a `last_accessed_at` timestamp in each chunk’s metadata and update it with each access:

```
[Document(page_content='this is some content of a chunk', metadat
```

Once timestamps are in place, you can use a `TimeWeightedVectorStoreRetriever`, which applies a time decay factor to rank chunks by both similarity score and recency:

```
retriever = TimeWeightedVectorStoreRetriever(
 vectorstore=vectorstore,
 decay_rate=1e-10, k=3
)
```

The `TimeWeightedVectorStoreRetriever` adjusts similarity scores based on recency:

```
adjusted_similarity_score = similarity_score + (1.0 - decay_rate)
```

This adjusted score allows recent, relevant content to rank higher, ensuring more timely responses. Next, we'll discuss one of the most essential post-processing techniques.

### 10.7.4 RAG Fusion (Reciprocal Rank Fusion)

In the previous chapter, we discussed “multiple query generation,” where multiple queries are generated from a user’s question, and a subset of relevant results is selected from the combined results across all queries. Using LangChain’s `MultiQueryRetriever`, we automated this process to select the most relevant answers. However, if you want finer control over result ranking, consider an approach called *Reciprocal Rank Fusion (RRF)*.

*RRF or Reciprocal Rank Fusion*, detailed by Cormack, Clarke, and Buttcher in their paper "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods"

(<https://plg.uwaterloo.ca/~gvcormac/cormacksigir09-rrf.pdf>), reranks retrieved documents based on a specific scoring formula:

```
rrfscore = 1 / (rank + k)
```

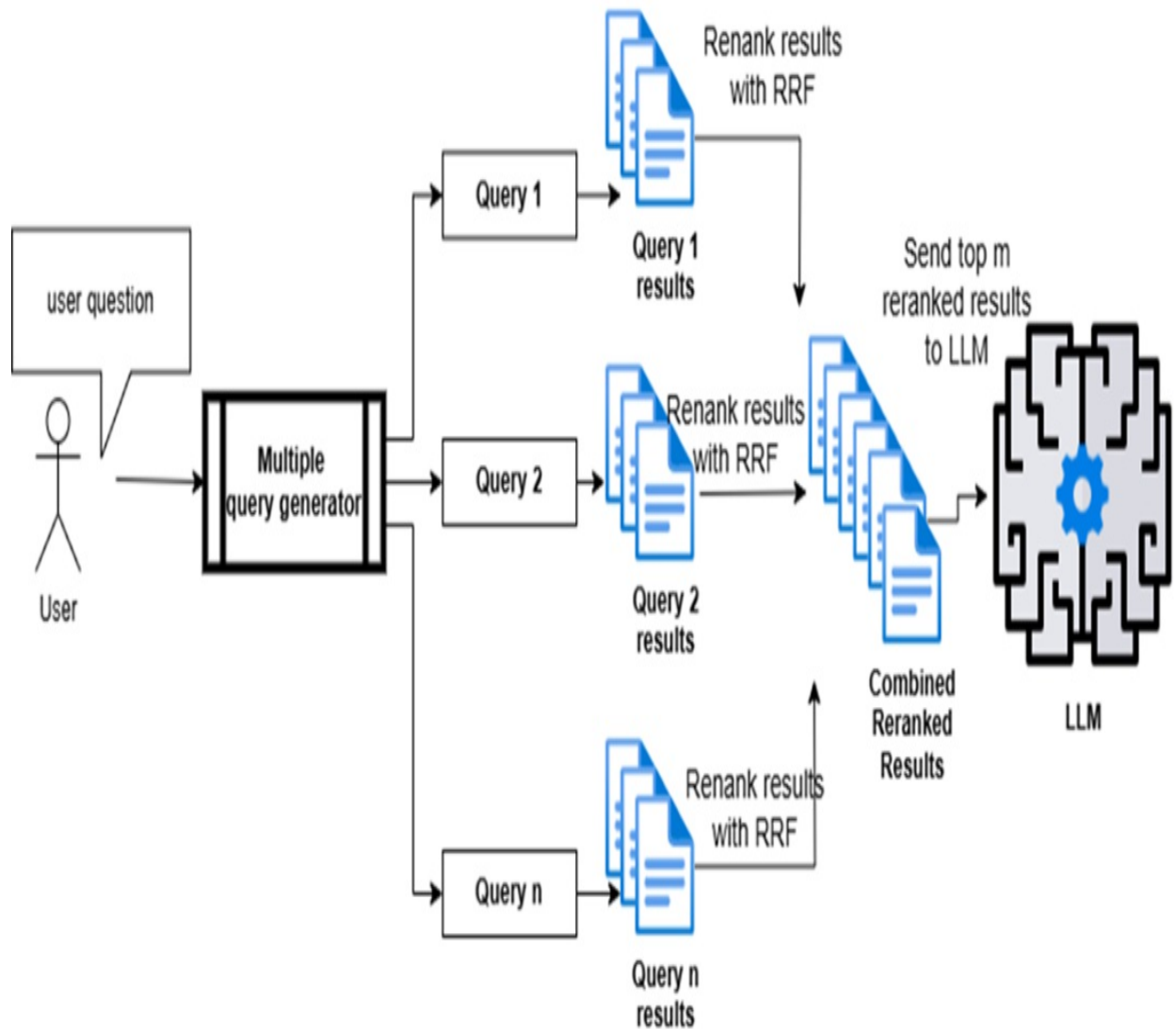
where:

- rank: the document's current rank based on similarity or relevance.
- k: a smoothing constant to control the weight of existing ranks.

As each document is processed, its RRF score accumulates across all generated queries. Once all scores are calculated, documents are reranked by their cumulative RRF scores, and the top-ranked results are sent to the LLM for answer synthesis.

You can get a visual idea of how the overall process work by following through figure 10.8.

**Figure 10.8 Reciprocal Rank Fusion Workflow: Multiple queries are generated from the initial user question. Each query retrieves a set of results, such as from a vector store. All results are then reranked using RRF scores, with only the top results sent to the LLM for final answer synthesis.**



At a high level, implementing RAG fusion involves generating multiple queries, as shown in chapter 9, and ranking the results with the RRF algorithm. This retrieval workflow can then be embedded within a larger RAG chain, as demonstrated in previous examples.

## Generating Multiple Queries

You can generate multiple queries from an initial question using a chain, as shown in Chapter 8. For convenience, the setup is repeated here in Listing 10.14.

**Listing 10.14 Multi-Query Generation**

```
from langchain_core.prompts import ChatPromptTemplate

from typing import List
from langchain_core.output_parsers import BaseOutputParser
from pydantic import BaseModel, Field

multi_query_gen_prompt_template = """
You are an AI language model assistant. Your task is to generate
different versions of the given user question to retrieve relevant
database. By generating multiple perspectives on the user question,
the user can overcome some of the limitations of the distance-based
search. Provide these alternative questions separated by newlines.
Original question: {question}
"""

multi_query_gen_prompt = ChatPromptTemplate.from_template(multi_q

class LineListOutputParser(BaseOutputParser[List[str]]):
 """Parse out a question from each output line."""

 def parse(self, text: str) -> List[str]:
 lines = text.strip().split("\n")
 return list(filter(None, lines))

questions_parser = LineListOutputParser()

llm = ChatOpenAI(model="gpt-4o", openai_api_key=OPENAI_API_KEY)

multi_query_gen_chain = multi_query_gen_prompt | llm | questions_
```

With this setup, you can now generate multiple alternative queries from a single question, helping to capture varied perspectives and nuances that improve document retrieval accuracy.

**Note**

I've chosen to use gpt-4o instead of gpt-4o-mini, as it is more likely to

produce higher-quality queries and generate more accurate, well-synthesized responses

Now that multiple queries can be generated, the next step is to implement a ranking mechanism to sort and prioritize the retrieved results.

## Reciprocal Rank Fusion Algorithm

The core of this workflow is the Reciprocal Rank Fusion (RRF) algorithm, which assigns scores to documents retrieved by multiple queries. Using the RRF formula, each document is scored based on its rank, then reranked by total RRF score. See Listing 10.15 for implementation details.

### Listing 10.15 Reciprocal Rank Fusion (RRF) Algorithm

```
def reciprocal_rank_fusion(results_groups: list[list], k=60): #A
 """ Reciprocal_rank_fusion that takes multiple groups of rank
 and an optional parameter k used in the Reciprocal Rank F

 indexed_results = {} #B

 for group_id, results_group in enumerate(results_groups): #C
 for local_rank, doc in enumerate(results_group):
 indexed_results[(group_id, local_rank)] = doc

 fused_scores = {} #D

 for key, doc in indexed_results.items(): #E
 group_id, local_rank = key

 if key not in fused_scores:
 fused_scores[key] = 0 #F

 doc_current_score = fused_scores[key]
 fused_scores[key] += 1 / (local_rank + k) #G

 reranked_results = [#H
 (indexed_results[key], score)
 for key, score in sorted(fused_scores.items(), key=lambda
]

 return reranked_results
```



With the RRF algorithm in place, let's create a RAG Fusion retrieval chain, as shown below:

```
retriever = uk_with_metadata_collection.as_retriever(search_kwargs
top_three_results = RunnableLambda(lambda x: x[0:3])) #A

rag_fusion_retrieval_chain = multi_query_gen_chain | retriever.map

docs = rag_fusion_retrieval_chain.invoke({"question": question})
len(docs)
```

The final step is to integrate this RAG Fusion retrieval chain into a larger RAG chain for end-to-end question routing, retrieval, and answer synthesis.

## Incorporating RAG Fusion into the RAG Chain

As we've seen before, integrating a retrieval chain into a broader RAG chain is straightforward. For completeness, Listing 10.16 shows how to incorporate the RAG Fusion retrieval chain within a RAG chain.

**Listing 10.16 Integrating the RAG Fusion Retrieval Chain into a RAG Chain**

```
rag_prompt_template = """
Given a question and some context, answer the question.
If you do not know the answer, just say I do not know.

Context: {context}
Question: {question}
"""

rag_prompt = ChatPromptTemplate.from_template(rag_prompt_template)

rag_chain = (
 {
 "context": {"question": RunnablePassthrough()} | rag_fusion_retrieval_chain,
 "question": RunnablePassthrough(), #B
 }
 | rag_prompt
 | llm
 | StrOutputParser()
)
```

Now, let's test the complete RAG chain with an example question:

```
user_question = "Can you give me some tips for a trip to Brighton"
answer = rag_chain.invoke(user_question)
print(answer)
```

Expected output:

Here are some tips for a trip to Brighton:

1. **Visit During Festivals**: If you can, plan your visit in May
2. **Enjoy the Beach**: Brighton boasts a beautiful stretch of sh
3. **Explore Local Culture**: Brighton has a vibrant cultural sce
4. **Transportation**: Brighton is well-connected by train, makin
5. **Seasonal Work Opportunities**: If you're looking for tempora
6. **Plan for All Budgets**: Whether you're looking for budget op
7. **Stay Safe**: Like any city, it's important to stay aware of
8. **Local Council Resources**: Check out the Brighton and Hove C

Enjoy your trip!

Congratulations! You've completed a comprehensive guide to advanced RAG techniques, making you well-equipped to tackle complex RAG tasks.

## 10.8 Summary

- Use structured data from SQL and document databases to enhance LLM responses, complementing unstructured data in vector stores.
- Structured queries are often required for these databases, and LLMs can generate them from natural language inputs.
- Vector stores support dense searches with embeddings and sparse searches using keyword-based indexing.
- Keywords for sparse searches can be added via metadata, extracted with algorithms like TF-IDF, or generated by the LLM.
- LLMs can also generate metadata queries directly from user questions.
- Many LLMs can translate user questions into SQL queries, enabling

direct access to relational databases.

- SQL-based retrievals rely on exact matching, but semantic SQL expands searches to find approximate matches aligned with user intent.
- Graph databases represent data as nodes and relationships, making them ideal for complex, interconnected information.
- Graph databases excel in LLM use cases requiring advanced reasoning and relationship understanding, as they can be used to create knowledge graphs with the help of LLMs or answer questions based on knowledge graphs through KG-RAG.
- Content for LLM applications often spans vector stores, relational databases, document stores, and knowledge graphs.
- Routing chains ensure queries are directed to the appropriate storage type for optimized retrieval.
- - Reciprocal Rank Fusion (RRF) improves query relevance by reranking documents from multiple queries using cumulative scores.
- RRF enhances answer quality by feeding the most relevant results to the LLM for synthesis.

# 11 Building Tool-based Agents with LangGraph

## This chapter covers

- Building LLM-powered agents using LangGraph
- Registering and using tools for dynamic agent execution
- Debugging agent execution and tool calls
- Simplifying agents with pre-built LangGraph components
- Observing agent execution with LangSmith

In chapter 5, you explored the distinction between agentic workflows and agents. You learned that agentic workflows are fundamentally deterministic: their logic is based on flows with conditional paths that depend on the current application state. These workflows can be elegantly modeled using node-based graphs in LangGraph, and you saw a complete, hands-on example of such a system.

Agents, however, operate differently. Rather than following a predetermined flow, agents rely on dynamic, context-sensitive decision-making. With the help of a language model (LLM), an agent chooses which tools to use—and in what order—based on the evolving context of the task at hand. These decisions aren't pre-scripted; instead, they unfold step by step, as the agent continually evaluates the outputs of previous actions and adapts accordingly.

In this chapter, you'll put these ideas into practice by building a multi-tool travel information agent. We'll begin simply, implementing an agent that provides destination information using a single tool. From there, we'll extend it into a true multi-tool agent, able to answer questions about both travel destinations and their current weather conditions.

As we progress, I'll introduce you to the core concepts necessary for constructing multi-tool agents, with particular focus on the tool-calling protocol. You'll first implement this protocol from scratch to grasp every

detail, then see how LangGraph's built-in capabilities can streamline and simplify your agent's architecture.

This chapter's multi-tool agent will serve as the foundation for the more advanced, multi-agent systems you'll build in the chapters ahead.

Let's dive in—there's a lot to discover.

## **11.1 Starting Simple: Building a Single-Tool Travel Info Agent**

In this section, we'll lay the groundwork for our agent-based applications by building a straightforward travel information agent. This first agent will use just one tool: a vector store retriever that answers questions about Cornwall's destinations and resorts, leveraging content from [wikivoyage.org](https://www.wikivoyage.org). The content is split into chunks and stored in a vector store for efficient retrieval.

If you've followed the advanced RAG (Retrieval-Augmented Generation) chapters earlier in this book, you should already be comfortable with sourcing content and populating a vector store. Here, we'll build on that foundation, keeping the focus on agent mechanics.

### **11.1.1 Project Setup**

Let's set up a new Python project using VS Code (this works seamlessly with Cursor as well).

#### **Create a virtual environment and install dependencies**

First, set up your Python virtual environment and install all necessary dependencies. You'll find the `requirements.txt` file either on the Manning website for this book or in the cloned GitHub repository that accompanies chapter 11.

Open a new PowerShell terminal in VS Code (from the menu: Terminal > New Terminal), navigate to the `ch11` project folder, then create and activate a

new virtual environment:

```
PS C:\Github\building-llm-applications\ch11> python -m venv env_c
PS C:\Github\building-llm-applications\ch11> .\env_ch11\Scripts\activate
(env_ch11) PS C:\Github\building-llm-applications\ch11>
```

Once your environment is activated, install the required dependencies:

```
(env_ch11) PS C:\Github\building-llm-applications\ch11> pip install
```

Your project environment is now ready for development.

## Add your OpenAI API key

Create a `.env` file in your project root and add your OpenAI API key:

```
OPENAI_API_KEY=<Your OPENAI_API_KEY>
```

## Configure VS Code debugging

For smooth debugging, add the following `launch.json` to your `.vscode` directory:

```
{
 "version": "0.2.0",
 "configurations": [
 {
 "name": "Python Debugger: Current File",
 "type": "debugpy",
 "request": "launch",
 "program": "${file}",
 "console": "integratedTerminal"
 }
]
}
```

Organize your implementation files:

We'll use the naming convention `main_x_y.py` for our scripts. The “x” represents the feature (e.g., `main_1_1.py` for the initial travel info agent, `main_2_1.py` when adding weather), while “y” is the version of that feature

as we iterate. This will make it easy for you to compare versions and follow the progression of the implementation.

#### **Note**

The code in these examples is intentionally simplified to focus on core functionality. Error handling and defensive programming are omitted for clarity and learning purposes.

### **11.1.2 Loading Environment Variables**

Once your `.env` file is ready, you can load your API key at the top of your script as follows (see Listing 11.1):

#### **Listing 11.1 Loading environment variables**

```
load_dotenv() #A
```

### **11.1.3 Preparing the Travel Information Vector Store**

To enable our travel information agent to answer queries about Cornwall destinations, we first need a way to store and efficiently retrieve relevant information. The approach here draws on techniques you saw in the Retrieval-Augmented Generation (RAG) section, but streamlines them for this agent-centric context. At a high level, we'll download travel content for a set of Cornwall destinations from WikiVoyage, break the text into manageable chunks, embed those chunks into vector representations, and then store everything in a Chroma vector store. We'll also encapsulate the initialization logic in a singleton pattern to ensure the vector store is only built once during the agent's lifetime.

The following code (listing 11.2) sets up this vector store, making it easy to retrieve relevant travel information as the agent operates.

#### **Listing 11.2 Preparing the Travel Information Vector Store**

```
UK_DESTINATIONS = [#A
 "Cornwall",
```

```

 "North_Cornwall",
 "South_Cornwall",
 "West_Cornwall",
]

async def build_vectorstore(destinations: Sequence[str]) -> Chroma:
 """Download WikiVoyage pages and create a Chroma vector store
 urls = [f"https://en.wikivoyage.org/wiki/{slug}" for slug in destinations]
 loader = AsyncHtmlLoader(urls) #C
 print("Downloading destination pages ...") #C
 docs = await loader.aload() #C

 splitter = RecursiveCharacterTextSplitter(chunk_size=1024, chunk_overlap=50)
 chunks = sum([splitter.split_documents([d]) for d in docs], [])

 print(f"Embedding {len(chunks)} chunks ...") #E
 vectordb_client = Chroma.from_documents(chunks, embedding=OpenAIEmbedder())
 print("Vector store ready.\n")
 return vectordb_client #F

Singleton pattern (build once)
_tivectorstore_client: Chroma | None = None #G

def get_travel_info_vectorstore() -> Chroma: #H
 global _tivectorstore_client
 if _tivectorstore_client is None:
 if not os.environ.get("OPENAI_API_KEY"):
 raise RuntimeError("Set the OPENAI_API_KEY env variable")
 _tivectorstore_client = asyncio.run(build_vectorstore(destinations))
 return _tivectorstore_client #I

tivectorstore_client = get_travel_info_vectorstore() #J
ti_retriever = tivectorstore_client.as_retriever() #K

```

This setup starts by defining a list of Cornwall-related destinations that serve as the basis for information retrieval. The `build_vectorstore()` asynchronous function constructs URLs for each destination and uses an asynchronous loader to fetch the corresponding WikiVoyage pages. Once the pages are downloaded, the text is split into overlapping chunks to ensure that information remains contextually meaningful. These chunks are then embedded using OpenAI's embedding models and stored in a Chroma vector store, making them quickly searchable by semantic similarity.

To prevent unnecessary recomputation and data downloads, the singleton



pattern is used for the vector store client. The `get_travel_info_vectorstore()` function ensures that the vector store is only built once and is reused for all future retrievals. At the end, the vector store client is instantiated, and a retriever object is created from it—this retriever will be the agent’s interface for accessing Cornwall travel information. This foundation allows the agent to efficiently answer user queries about destinations, using up-to-date knowledge sourced directly from WikiVoyage.

## 11.2 Enabling Agents to Call Tools

Now that we have our vector store retriever ready, the next step is to expose this retrieval capability as a tool the agent can use. This is where the concept of tool calling—a major advance in modern agent frameworks—enters the picture.

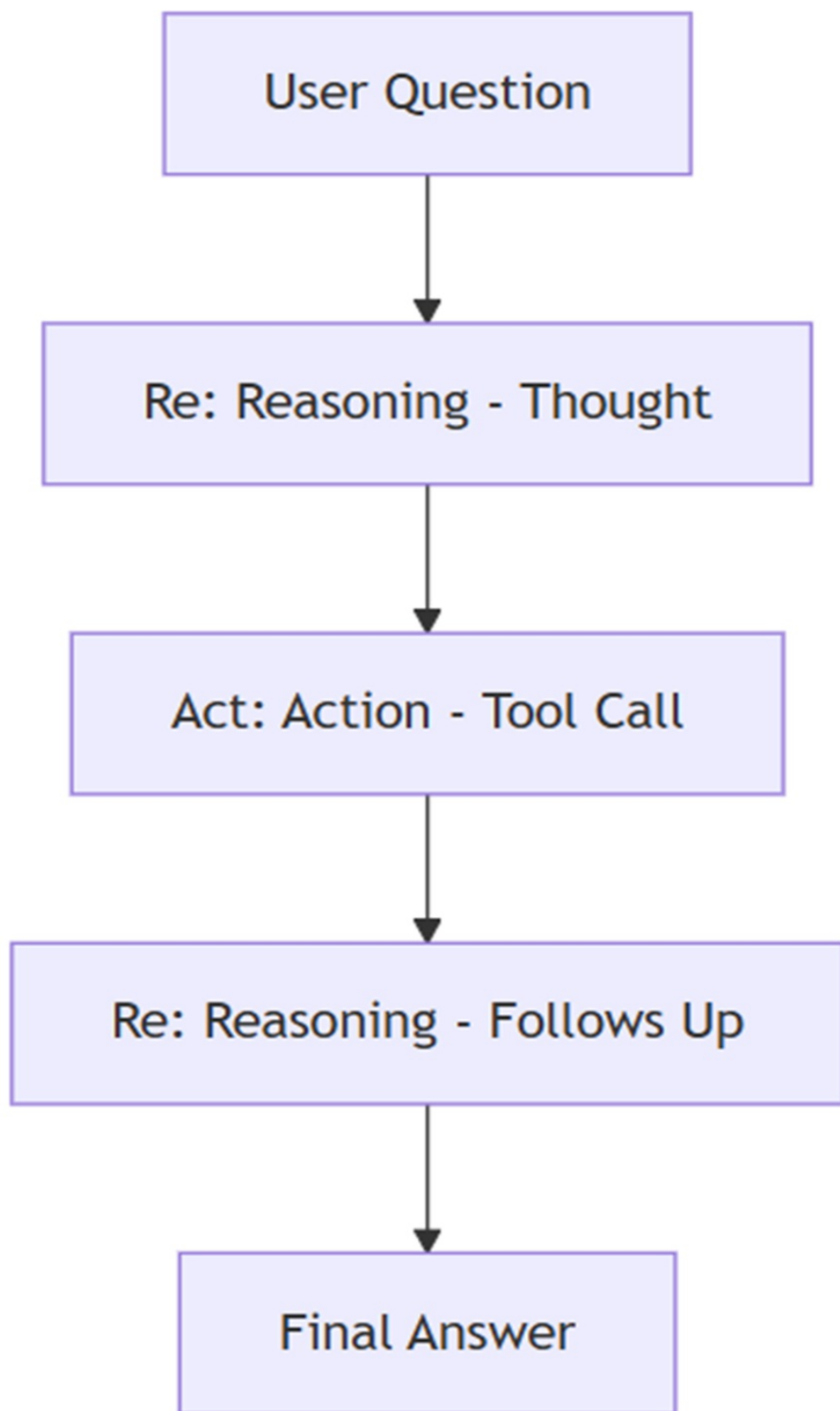
### 11.2.1 From Function Calling to Tool Calling

LLMs like those from OpenAI initially introduced function calling: a mechanism that allows the model to request specific functions, passing structured arguments, based on the needs of a given prompt. This concept quickly evolved into the more general tool calling protocol, now widely supported by major LLM providers. With tool calling, models can invoke not just custom functions but also a variety of built-in or external “tools,” handling everything from code execution to external API lookups.

This evolution has made agent implementations dramatically simpler and more robust, particularly when using the ReAct (Reasoning and Acting) design pattern. The ReAct pattern, introduced by Yao et al. in their 2022 paper "ReAct: Synergizing Reasoning and Acting in Language Models", enables LLMs to interleave reasoning steps (“thoughts”) with tool invocations (“actions”). In effect, the model can break a task into smaller pieces, use tools where appropriate, and synthesize an answer step by step.

Here’s a simplified view of the ReAct pattern as a process diagram (see figure 11.1):

**Figure 11.1. The ReAct pattern alternates between reasoning (“Re”) and action (“Act”), enabling the agent to process a user question by thinking, calling tools as needed, and following up with further reasoning before delivering the final answer.**



The ReAct pattern, as illustrated in the diagram, begins with a user question that initiates the process. The agent first enters a Reasoning phase (labeled as “Re: Reasoning - Thought”), where the language model considers the user’s input and determines the necessary next steps. If additional information or actions are needed, the agent transitions to the Action phase (“Act: Action - Tool Call”), where it calls one or more tools—such as performing a semantic search or invoking an API.

After the tool call is completed, the agent returns to another Reasoning phase (“Re: Reasoning - Follows Up”), in which it incorporates the results from the tool into its ongoing line of reasoning. This may lead to further tool calls, or the agent may determine that it now has enough information to formulate a response. Finally, the process concludes with the agent providing a Final Answer to the user.

This stepwise interplay between reasoning (“Re”) and acting (“Act”) embodies the core of the ReAct pattern, allowing agents to dynamically combine thought and action as they work toward solving the user’s query.

#### **Note**

With the introduction of the OpenAI Responses API, the transition from function calling to tool calling has accelerated. The Responses API is designed specifically for structured tool use, and has largely superseded the older Completion API for agentic applications.

### **11.2.2 How Tool Calling Works with LLMs**

OpenAI’s tool calling supports several types of tools—including user-defined functions, code interpreters, and native capabilities like web browsing. At a high level, when you register a tool with the LLM, you expose both the function signature (name and parameters) and a textual description. The model then decides, at runtime, when and how to use each tool based on the user’s input and the context of the conversation.

In LangGraph, tools can be registered using either a class-based or decorator-based approach. We’ll use the decorator-based approach for clarity and

conciseness.

Let's implement our semantic search tool as a function decorated with `@tool`, making it available to the agent for tool calling, as shown in listing 11.3:

**Listing 11.3 LangGraph attribute-based tool definition**

```
@tool #A
def search_travel_info(query: str) -> str: #B
 """Search embedded WikiVoyage content for information about d
 docs = ti_retriever.invoke(query) #C
 top = docs[:4] if isinstance(docs, list) else docs #C
 return "\n---\n".join(d.page_content for d in top) #D
```

This decorated function, `search_travel_info()`, is now recognized as a tool: it takes a user query, searches the vector store for relevant WikiVoyage content, and returns up to four top results as a single string. The `@tool` decorator ensures that the function's name, description, and parameter schema are all available to the LLM for tool calling.

### 11.2.3 Registering Tools with the LLM

To enable the agent to use our semantic search tool, we must register it with the LLM so that the model knows both the function's signature and its intended purpose. In LangChain, this is achieved using the `bind_tools` protocol, but it is instructive to see how this works at the OpenAI API level first.

With the introduction of the OpenAI Responses API, the way tools (and functions) are registered has become more standardized and explicit. Now, you provide a structured definition of your tool—specifying its name, description, and parameter schema. The model can then return responses that explicitly request tool invocations, passing arguments as needed. For further details, you can consult the OpenAI function calling documentation (<https://platform.openai.com/docs/guides/function-calling?api-mode=responses>).

#### **Example: Manual Tool Registration with OpenAI API**

Suppose you wanted to expose our `search_travel_info` function directly to the OpenAI API (without using LangChain). You would define the tool's schema as shown in Listing 11.4. Note that this example is for illustration only and is not included in the book's provided source code.

**Listing 11.4 Manual tool registration with OpenAI API**

```
search_travel_info_tool = {
 "type": "function", #A
 "function": {
 "name": "search_travel_info", #B
 "description": "Search embedded WikiVoyage content for in",
 "parameters": { #D
 "type": "object", #E
 "properties": {
 "query": { #F
 "type": "string",
 "description": "A natural language query about",
 }
 },
 "required": ["query"] #H
 }
 }
}

Register the tool (function) when making a chat completion request
response = client.chat.completions.create(
 model="gpt-4o", #I
 messages=[
 {"role": "user", "content": "Tell me about surfing in Cor"},
],
 tools=[search_travel_info_tool], #K
 tool_choice="auto", # let the model decide when to use the tool
)
```

In this example, you explicitly define the tool's metadata and parameters, and then pass it in the `tools` argument of your API call. When the model decides it needs to call `search_travel_info`, it will return a structured tool call in its response. Your application must then handle the invocation of the Python function, passing the model-generated arguments, and send the results back to the LLM if the conversation continues.

## Registering Tools in LangChain

LangChain automates much of this process. You simply define your tool as a decorated Python function, and then register it with the LLM. You can see in listing 11.5 how this looks in code:

**Listing 11.5 Registering tools in LangChain**

```
TOOLS = [search_travel_info] #A

llm_model = ChatOpenAI(temperature=0, model="gpt-4.1-mini", #B
 use_responses_api=True) #B
llm_with_tools = llm_model.bind_tools(TOOLS) #C
```

Here, we list the available tools (currently just `search_travel_info`), instantiate the GPT-4.1-mini chat model (with Responses API support), and use `.bind_tools(TOOLS)` to expose those tools to the model for tool calling.

LangChain handles the translation between your Python code and the OpenAI function/tool-calling protocol, including automatically generating the appropriate JSON schema from your function signature and docstring.

This setup ensures that whenever the model receives a user message, it can autonomously decide if and when to call any registered tool, structuring its responses according to the tool calling protocol.

**Note**

If you're using a model other than OpenAI, or you prefer not to use the Responses API, tool calling can still work—though the capabilities and structure of the responses may differ. The older Completion API is still available, but for most agentic use cases, the newer Responses API is recommended for its clarity, power, and alignment with evolving best practices.

## 11.2.4 Agent State: Tracking the Conversation

In our implementation, the agent's state is simply a collection of LLM messages that track the entire conversation. This is defined as follows:

```
class AgentState(TypedDict):
```

```
messages: Annotated[Sequence[BaseMessage], operator.add]
```

Here, AgentState only contains the sequence of messages exchanged between the user, the agent, and any tool responses. This keeps the design simple.

## 11.2.5 Executing Tool Calls

The core of the tool execution logic is implemented in a node that examines the LLM's most recent message, extracts any tool calls requested by the model, and invokes the corresponding functions with the provided arguments. Each tool's output is then wrapped in a message and appended to the conversation state.

A simplified implementation is shown in listing 11.6.

**Listing 11.6** Tool execution implemented from scratch

```
class ToolsExecutionNode: #A
 """Execute tools requested by the LLM in the last AIMessage."""
 def __init__(self, tools: Sequence): #B
 self._tools_by_name = {t.name: t for t in tools}

 def __call__(self, state: dict): #C
 messages: Sequence[BaseMessage] = state.get("messages", [

 last_msg = messages[-1] #D
 tool_messages: list[ToolMessage] = [] #E
 tool_calls = getattr(last_msg, "tool_calls", []) #F

 for tool_call in tool_calls: #G
 tool_name = tool_call["name"] #H
 tool_args = tool_call["args"] #I
 tool = self._tools_by_name[tool_name] #J
 result = tool.invoke(tool_args) #K
 tool_messages.append(
 ToolMessage(
 content=json.dumps(result), #L
 name=tool_name,
 tool_call_id=tool_call["id"],
)
)
)
```



```

 return {"messages": tool_messages} #M
tools_execution_node = ToolsExecutionNode(TOOLS) #N

```

This pattern allows the agent to handle multiple tool calls in a single step. The tools execution node inspects the LLM’s latest response, invokes each requested tool by name, collects the results, and formats them for the next stage in the conversation. In a typical agent workflow, these tool results are sent back to the LLM, which incorporates the new information to either reason further or generate a direct answer to the user’s query.

Although I’ve shown you how to implement tool calling from scratch for learning purposes, you usually won’t need to write this logic yourself in practice, as shown in the sidebar.

#### **The Built-in ToolNode Class**

LangGraph provides a built-in class, ToolNode, which performs the same function as our custom ToolsExecutionNode. For most applications, you can use:

```
tools_execution_node = ToolNode(TOOLS)
```

This saves you from having to implement the tools execution logic manually, streamlining your agent development process.

Now that you understand how tool execution is managed, let’s explore how the agent, guided by the LLM, determines which tools to call—or whether it already has enough information to generate the final answer.

## **11.2.6 The LLM Node: Coordinating Reasoning and Action**

Next, we add an LLM node to our LangGraph workflow, as shown in listing 11.7.

#### **Listing 11.7 LLM Node**

```

def llm_node(state: AgentState): #A
 """LLM node that decides whether to call the search tool."""

```

```

current_messages = state["messages"] #B
response_message = llm_with_tools.invoke(current_messages) #C

return {"messages": [response_message]} #D

```

This node takes the agent state (a list of messages) and forwards them to the LLM for processing. Because we are using an LLM with tool calling enabled (`llm_with_tools`), the model automatically understands when to issue tool calls and when to produce a final answer:

- If the last message is a user question, the LLM can decide to request a tool call (or multiple calls) to retrieve relevant information.
- If the last message(s) are tool results, the LLM integrates those responses, reasons further, and may either produce a final answer or request additional tool calls if needed.

The model returns either an `AIMessage` with a final answer in the `content` field, or additional tool calls in the `tool_calls` field, depending on the situation. This flexible, reactive loop is at the heart of all modern agent implementations—and it's what makes today's LLM-based agents so capable.

## 11.3 Assembling the Agent Graph

With our LLM node and tool execution node ready, the next step is to assemble them into a working agent graph. In `LangGraph`, an agent is structured as a directed graph, where each node represents a component (such as an LLM or a tool executor), and edges represent the possible flow of data and control between these nodes.

The code in Listing 11.8 demonstrates how we assemble our single-tool travel information agent as a graph. Each node in the graph corresponds to either the LLM (language model) or the tool execution logic.

**Listing 11.8 Graph of the tool-based agent**

```

builder = StateGraph(AgentState) #A
builder.add_node("llm_node", llm_node) #B
builder.add_node("tools", tools_execution_node) #B

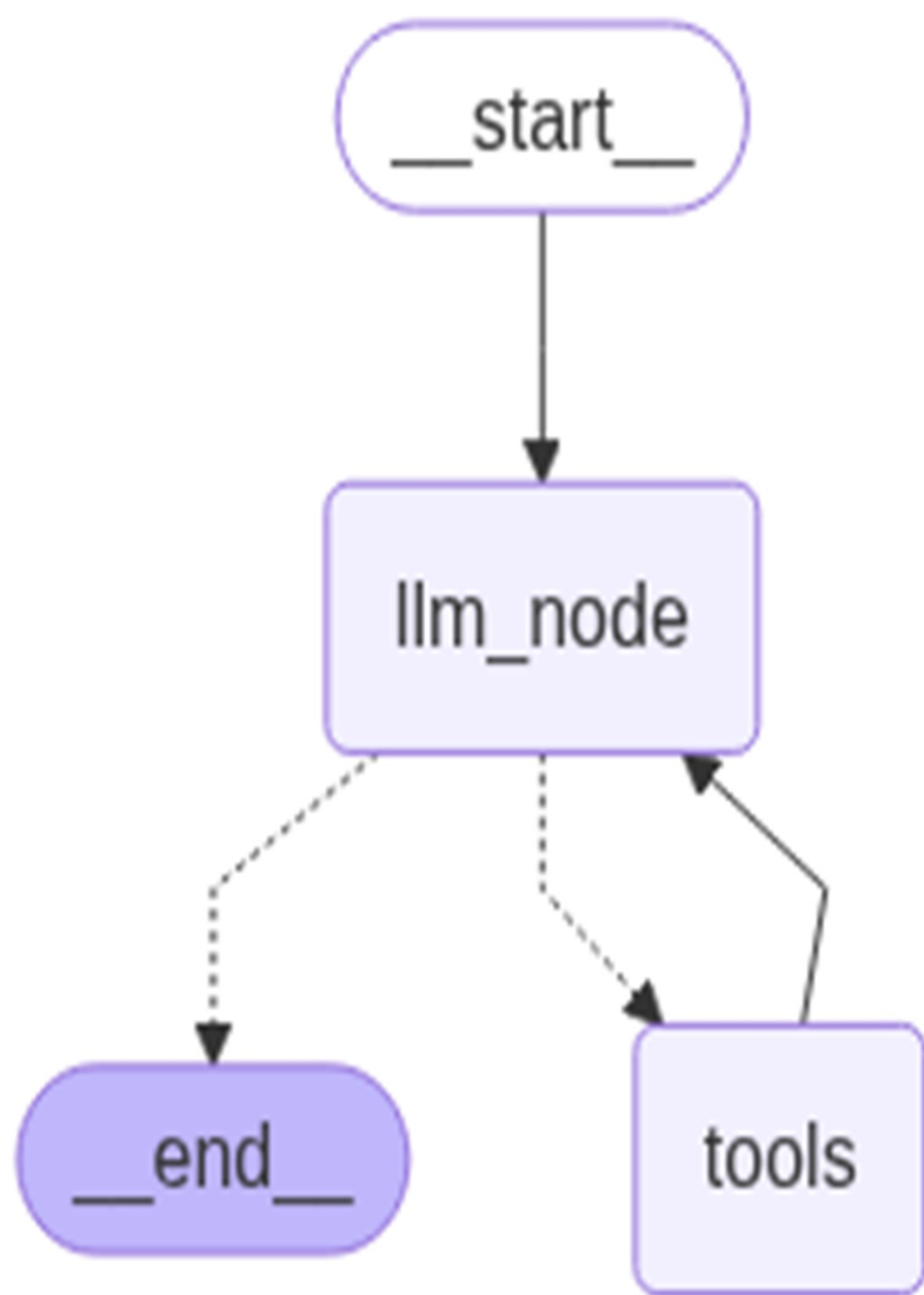
```

```
builder.add_conditional_edges("llm_node", tools_condition) #C
builder.add_edge("tools", "llm_node") #D
builder.set_entry_point("llm_node") #E
travel_info_agent = builder.compile() #F
```

### 11.3.1 Understanding the Agent Graph Structure

Our agent graph consists of two main nodes, as shown in figure 11.2: the LLM node, responsible for reasoning and generating tool calls, and the tools node, responsible for executing the requested tools and returning results. The flow of the conversation alternates between these two nodes, reflecting the ReAct pattern described earlier in this chapter.

**Figure 11.2 Conditional graph logic routes each user query through the LLM node, then dynamically directs flow to the tools node or ends the process, depending on whether tool calls are required**



A critical aspect of this setup is the conditional edge that connects the LLM node to the next step. This is controlled by the `tools_condition` function—a prebuilt utility in LangGraph. This function examines the latest message from the LLM:

- If the message contains the `tool_calls` property (meaning the LLM is requesting one or more tool invocations), the graph routes the flow to the node named "tools".
- If there are no tool calls present, the flow is directed to the END node, terminating the graph and producing a final answer for the user.

This mechanism lets the agent dynamically decide, at each turn, whether to continue reasoning, take action, or conclude the interaction.

We explicitly set the entry point of the graph to `"llm_node"`, ensuring that each user question is first processed by the language model. (As an alternative, you could achieve the same effect by adding an edge from the START node to `"llm_node"` with `graph_builder.add_edge(START, "llm_node")`.)

By compiling the graph with `builder.compile()`, we finalize our travel information agent, ready to receive queries and intelligently use its tool to find relevant travel information.

## 11.4 Running the Agent Chatbot: The REPL Loop

The final step in building our agent-powered chatbot is to implement the user interface—a simple loop that continuously accepts user questions and returns answers, until the user chooses to exit. This classic pattern, known as a REPL (Read-Eval-Print Loop), is the main bridge between the user and your travel information agent.

At a high level, the chat loop listens for input, wraps the user's question in a message structure, invokes the agent graph, and prints the assistant's reply. This interaction continues indefinitely, enabling a true conversational experience.

Listing 11.9 shows how you can implement this chat loop in Python:

**Listing 11.9 Chatbot REPL loop**

```
def chat_loop(): #A
 print("UK Travel Assistant (type 'exit' to quit)")
 while True:
 user_input = input("You: ").strip() #B
 if user_input.lower() in {"exit", "quit"}: #C
 break
 state = {"messages": [HumanMessage(content=user_input)]}
 result = travel_info_agent.invoke(state) #E
 response_msg = result["messages"][-1] #F
 print(f"Assistant: {response_msg.content}\n") #G

if __name__ == "__main__":
 chat_loop()
```

This loop welcomes the user, waits for their question, and continues processing until the user types “exit” or “quit.” Each input is packaged as a `HumanMessage` and passed to the travel information agent you just built. The agent’s reply is extracted from the graph’s output and displayed back to the user.

With this in place, you are now ready to run your first agent-based chatbot. Try asking it about destinations or activities in Cornwall—and experience how the agent reasons, retrieves information, and converses, all within the framework you’ve just constructed, as we are about to see.

## 11.5 Executing a Request

Now that your travel information agent is ready, let’s step through the agent in action by running and debugging your implementation. This hands-on walkthrough will show you how the agent orchestrates the flow between the LLM and the tool, helping you see the LangGraph framework in motion.

### 11.5.1 Step-by-Step Debugging

Begin by opening your `main_01_01.py` file and running it in debug mode, using the Python Debug configuration you set up in your `launch.json`

earlier. To trace the agent's flow, place breakpoints at the beginning of the following functions:

- `search_travel_info()`
- `ToolsExecutionNode.__call__()`
- `llm_node()`

Ready? Hit F5 (or click the play icon in your IDE), and let's walk through the agent's workflow together.

## Vector Store Creation

When you start the script, you'll see the vector store being created and populated with travel information. In your debug console, you'll see the output similar to that in figure 11.3.

**Figure 11.3 Output at startup, during vector store creation**

```
USER_AGENT environment variable not set, consider setting it to identify your requests.
Downloading destination pages ...
Fetching pages: 100%|#####|
#####| 4/4 [00:01<00:00, 3.96it/s]
Embedding 1118 chunks ...
Vector store ready.

UK Travel Assistant (type 'exit' to quit)
```

## Chatbot Loop Launch

Next, chatbot loop starts up, waiting for your input:

```
UK Travel Assistant (type 'exit' to quit)
You:
```

Enter your question and press Enter:

You: Suggest three towns with a nice beach in Cornwall

## LLM Node Activation

Your breakpoint inside `llm_node()` will trigger. Inspect the `state.messages` property:

```
HumanMessage(content='Suggest three towns with a nice beach in Co
```

Step over (F10) to the next line to send this message to the LLM. Since the LLM is configured for tool calling, examine the resulting `response_message`:

```
 AIMessage(content=[],
...
tool_calls=[
{'name': 'search_travel_info', 'args': {'query': 'beach towns in
{'name': 'search_travel_info', 'args': {'query': 'best beaches in
{'name': 'search_travel_info', 'args': {'query': 'top seaside tow
...)
```

Here, the LLM has generated three tool calls—each targeting your semantic search tool with a slightly different query. Notice how the model rewrites the queries to maximize information coverage, as discussed in the Read-Rewrite pattern from the Advanced RAG section. You do not need to manually handle query rewriting: the LLM handles it.

## Tools Execution Node

Continue execution (F5). The list of tool calls is added to the message list. The conditional edge's `tools_condition` detects tool calls, routing execution to `ToolsExecutionNode.__call__()`. Your breakpoint will trigger there.

Inspect `state.messages`:

```
[
HumanMessage(content='Suggest three towns with a nice beach in Co
AIMessage(content=[], ... ,
tool_calls=[
{'name': 'search_travel_info', 'args': {'query': 'beach towns in
{'name': 'search_travel_info', 'args': {'query': 'best beaches in
{'name': 'search_travel_info', 'args': {'query': 'top seaside tow
```



]

The last message (from the LLM) has no content (the LLM hasn't answered yet because it needs more information), but it contains the tool calls that must be executed. The node extracts these, then iterates through each one, extracting the tool name and arguments, and invokes the corresponding tool:

```
result = tool.invoke(tool_args)
```

Step through this to watch `search_travel_info()` execute. Each semantic search result (a document returned from the vector store) is collected as a `ToolMessage` and added to the list for the next LLM step.

## Tool Call Results Passed Back to LLM

Continue (F5), and you'll return to `llm_node()`. Now, `state.messages` contains your original question, the LLM's tool call instructions, and the results from each tool execution:

```
\[HumanMessage(content='Suggest three towns with a nice beach
AIMessage(content='\[, ...
tool_calls=\[{'name': 'search_travel_info', 'args': {'query
ToolMessage(content='...', name='search_travel_info', tool_
ToolMessage(content='...', name='search_travel_info', tool_
ToolMessage(content='...', name='search_travel_info', tool_
```

The content of each tool message gives the LLM the facts it needs to synthesize a final answer. Step over the following line:

```
response_message = llm_with_tools.invoke(current_messages)
```

Inspect `response_message`:

```
AIMessage(content=\[{'type': 'text', 'text': "Three towns in Corn
```

Now, the content field is populated with the LLM's answer. The `tool_calls` field is gone—the LLM no longer needs external tools and has synthesized a response.

## Completing the Request

As execution leaves `llm_node()`, the `tools_condition` on the conditional edge checks for further tool calls. Finding none, it ends the conversation.

When your main invocation returns:

```
result = travel_info_agent.invoke(state)
```

The `result.messages` list concludes with the final `AIMessage` containing the answer:

```
{'messages': [
 HumanMessage(content='Suggest three towns with a nice beach in
 AIMessage(content=[], tool_calls=[...]),
 ToolMessage(...), ToolMessage(...), ToolMessage(...),
 AIMessage(content=[{'type': 'text', 'text': "Three towns in Co
]
```

The chatbot will now display the answer and prompt for your next question:

```
UK Travel Assistant (type 'exit' to quit)
You: Suggest three towns with a nice beach in Cornwall
Assistant: [{'type': 'text', 'text': "Three towns in Cornwall wit
You:
```

You have now observed, step by step, how your agent processes a user request: interpreting the question, generating tool calls, executing them, and synthesizing a final, well-informed answer. This debug-driven walkthrough offers deep insight into the mechanics of agentic reasoning and action in LangGraph.

## 11.6 Expanding Your Agent: Adding a Weather Forecast Tool

So far, our agent has been able to answer travel-related queries using semantic search over curated travel content. But real-world travel advice often depends on dynamic, real-time information—like the weather! In this section, you’ll learn how to extend your agent by adding a second tool, enabling it to respond with context-aware answers based on current conditions.

## 11.6.1 Implementing a Mock Weather Service

To illustrate the process, start by copying your `main_01_01.py` file to a new script called `main_02_01.py`. This will help you track each evolutionary step in your agent's development.

First, let's introduce a mock weather service. This service will simulate real-time weather data for any given town, returning both a weather condition (such as "sunny" or "rainy") and a temperature. You can see the implementation of the `WeatherForecastService` in listing 11.10:

**Listing 11.10** `WeatherForecastService`

```
class WeatherForecast(TypedDict):
 town: str
 weather: Literal["sunny", "foggy", "rainy", "windy"]
 temperature: int

class WeatherForecastService:
 _weather_options = ["sunny", "foggy", "rainy", "windy"]
 _temp_min = 18
 _temp_max = 31

 @classmethod
 def get_forecast(cls, town: str) -> Optional[WeatherForecast]:

 weather = random.choice(cls._weather_options)
 temperature = random.randint(cls._temp_min, cls._temp_max)
 return WeatherForecast(town=town, weather=weather, temper
```

This mock service chooses a random weather condition and temperature within a typical summer range for Cornwall. Later, you can swap this out for a real-world weather API if desired.

## 11.6.2 Creating the Weather Forecast Tool

With the mock service in place, the next step is to create a tool that wraps it, making weather data available to the agent. Here's how you can define the tool function and add it to your agent's toolkit:

```
@tool
```

```
def weather_forecast(town: str) -> dict:
 """Get a mock weather forecast for a given town. Returns a We
 forecast = WeatherForecastService.get_forecast(town)
 if forecast is None:
 return {"error": f"No weather data available for '{town}'"}
 return forecast
```

The `weather_forecast` tool provides a mock weather forecast for a given town. When called with a town name, it returns a dictionary containing the weather conditions (such as sunny, foggy, rainy, or windy) and temperature for that location. If no data is available, it returns an error message instead. This tool allows the agent to incorporate simulated real-time weather information into its responses.

### 11.6.3 Updating the Agent for Multi-Tool Support

Finally, make sure your LLM model is set up to use tool calling and recognizes both available tools:

```
TOOLS = [search_travel_info, weather_forecast] #A

llm_model = ChatOpenAI(
 temperature=0,
 model="gpt-4.1-mini", #B
 use_responses_api=True #B
)
llm_with_tools = llm_model.bind_tools(TOOLS) #C
```

With this setup, your agent is now equipped to handle both travel queries and weather checks, giving it the foundation to provide much more accurate and useful responses. In the next sections, you'll see how to guide the LLM to use both tools effectively, and observe the agent's new capabilities in action.

This workflow not only shows how to extend your agent's toolset, but also demonstrates the modular, incremental approach that makes agentic systems with LangGraph and LangChain so powerful.

## 11.7 Executing the Multi-Tool Agent

With your weather forecast tool registered, your agent can now synthesize

answers that combine travel information and real-time weather conditions. The beauty of this modular approach is that adding a new tool requires no changes to your agent’s graph structure. All orchestration is handled by the LLM and the tool calling protocol.

### 11.7.1 Running the Multi-Tool Agent (Initial Behavior)

Let’s put the new capabilities to the test. Set the same breakpoints as before—especially in the tool execution and LLM nodes—and add a breakpoint at the start of `weather_forecast()`.

Run `main_02_01.py` in debug mode (F5) and enter the following prompt:

You: Suggest two Cornwall beach towns with nice weather

#### Incorrect Behavior: LLM Defaults to Internal Knowledge

After submitting your query, your first breakpoint will hit in `llm_node()`. Step through to the last line and inspect the value of `response_message.tool_calls`:

```
[{'name': 'weather_forecast', 'args': {'town': 'Newquay'}, 'id':
{'name': 'weather_forecast', 'args': {'town': 'Falmouth'}, 'id':
```

Here, the LLM has simply picked two towns it “knows” (Newquay and Falmouth) and asked for their weather, without consulting your semantic search tool at all. This is typical of a model using its internal knowledge base rather than the tools provided—something we want to avoid for reliability and accuracy.

Why does this happen? The LLM is acting on its pre-training and defaulting to what it already “knows” about Cornwall, rather than querying your up-to-date data.

### 11.7.2 Improving LLM Tool Usage with System Guidance

To nudge the LLM towards tool use and away from hallucinations, let’s make its instructions and tool descriptions clearer. Make a copy of your script as

main\_02\_02.py, and update the tool definitions:

```
@tool(description="Search travel information about destinations i
def search_travel_info(query: str) -> str:
 ...

@tool(description="Get the weather forecast, given a town name.")
def weather_forecast(town: str) -> dict:
 ...
```

Next, introduce a guiding SystemMessage in your llm\_node():

```
system_message = SystemMessage(content="You are a helpful assista
current_messages.append(system_message)
```

You can see the amended llm\_node() function in listing 11.11:

#### Listing 11.11 Guiding tool selection

```
def llm_node(state: AgentState): #A
 """LLM node that decides whether to call the search tool."""
 current_messages = state["messages"] #B
 system_message = SystemMessage(content="You are a helpful ass
 current_messages.append(system_message) #D
 response_message = llm_with_tools.invoke(current_messages) #E

 return {"messages": [response_message]} #F
```

Restart your application in debug mode and ask:

You: Suggest two Cornwall beach towns with nice weather

At your first breakpoint in llm\_node(), examine current\_messages before and after appending the system message. Then, check response\_message—now, you’ll see the following tool call:

```
{'name': 'search_travel_info', 'args': {'query': 'beach towns in
```

What does this mean?

The LLM is now forced to use your semantic search tool for candidate beach towns, and won’t pick them “from its own knowledge.” This minimizes

hallucinations and guarantees the data used comes from your knowledge base.

Continue stepping through the code (F5). Right before you submit the `current_messages` to the LLM again, your messages will look like:

```
[HumanMessage(content='Suggest 2 Cornwall beach towns with nice w
SystemMessage(content='You are a helpful assistant that can search
AIMessage(content=[], ..., tool_calls=[{'name': 'search_travel_in
ToolMessage(content='"<p id=\\\"mwrq\\\">Cornwall, in particular Ne
SystemMessage(content='You are a helpful assistant that can search
```

From the `ToolMessage`, the LLM now receives a list of possible beach towns from your vector store.

Next, as you step through and reach `llm_node()` again, the LLM will issue calls for the weather forecast in two of these towns:

```
AIMessage(content=[], ..., tool_calls=[
{'name': 'weather_forecast', 'args': {'town': 'Newquay'}, 'id': '
{'name': 'weather_forecast', 'args': {'town': 'St Ives'}, 'id': '

```

The towns selected by the LLM might differ in your run, but they'll always come from the results of your semantic search tool.

Step through the weather tool calls—each `ToolMessage` you inspect should look like:

```
ToolMessage(content='{"town": "Newquay", "weather": "foggy", "tem
ToolMessage(content='{"town": "St Ives", "weather": "windy", "tem
```

### What If the Weather Isn't Good?

Suppose the weather is less than ideal in those towns. Continue to the next LLM response and you'll see:

```
AIMessage(content=[], ...,
tool_calls=[
{'name': 'weather_forecast', 'args': {'town': 'Perranporth'}, 'id'
{'name': 'weather_forecast', 'args': {'town': 'Falmouth'}, 'id':
```

Here, the LLM is asking for the weather in two more towns, likely because

the previous results didn't meet the “nice weather” requirement. This process continues until the agent finds two towns with suitable conditions.

After the final tool calls, the LLM generates a synthesized, fact-based answer:

```
AIMessage(content=[{'type': 'text', 'text': 'Two beach towns in C
```

Which appears to the user as:

```
UK Travel Assistant (type 'exit' to quit)
You: Suggest two Cornwall beach towns with nice weather
Assistant: [{'type': 'text', 'text': 'Two beach towns in Cornwall
```

In summary, by enhancing tool descriptions and providing explicit instructions via system prompts, you can guide the LLM to chain tool use in a multi-step, fact-grounded workflow—first searching for beach towns, then filtering by real-time weather—producing answers that are both dynamic and reliable.

### Exercise

Try replacing the mock weather tool with a real API, like OpenWeatherMap, using LangChain's OpenWeatherMap integration. This will make your agent truly real-time!

## 11.8 Using Pre-Built Components for Rapid Development

Up to this point, you've built an agent from the ground up: wiring together the graph, orchestrating tool calls, and stepping through every detail in the debugger. You now have a solid grasp of how tool calling works under the hood.

However, for most production scenarios, you'll want to reduce boilerplate and move faster—while still retaining transparency and observability when needed. The LangGraph library provides pre-built agent components, such as the react agent, that encapsulate much of this orchestration logic for you.



Let's see how dramatically you can simplify your agent by switching to a pre-built approach.

### **11.8.1 Refactoring to Use the LangGraph React Agent**

Start by copying your previous script (`main_02_02.py`) to a new file: `main_03_01.py`.

#### **Import the RemainingSteps Utility**

At the top of your script, add the following import:

```
from langgraph.managed.is_last_step import RemainingSteps
```

#### **Remove Manual Tool Binding**

You can now delete the line where you bound tools to the LLM:

```
llm_with_tools = llm_model.bind_tools(TOOLS)
```

The pre-built agent will handle this for you internally.

#### **Update the AgentState**

Modify your `AgentState` definition to include a `remaining_steps` field. This field allows the agent to manage how many tool-calling rounds are left in a controlled way:

```
class AgentState(TypedDict):
 messages: Annotated[Sequence[BaseMessage], operator.add]
 remaining_steps: RemainingSteps
```

#### **Remove Node and Graph Construction**

Now for the biggest simplification: delete your custom `ToolsExecutionNode`, `llm_node`, and any explicit graph wiring code.

Replace it all with a single instantiation of the built-in LangGraph React agent:

```
travel_info_agent = create_react_agent(
 model=llm_model,
 tools=TOOLS,
 state_schema=AgentState,
 prompt="You are a helpful assistant that can search travel in
```

That's it! The React agent now orchestrates the flow, tool calling, and synthesis for you.

## 11.8.2 Running the Pre-Built Agent

When you run `main_03_01.py` and ask your usual test question:

You: Suggest two Cornwall beach towns with nice weather

Assistant: [{'type': 'text', 'text': 'Two beach towns in Cornwall

You get the correct, grounded answer—with much less code.

## 11.8.3 Observing and Debugging with LangSmith

A common concern when switching to high-level abstractions is loss of visibility: How do you know the agent is actually following the right reasoning steps?

While you can still debug tool functions directly, the flow inside the agent itself is less exposed.

This is where LangSmith comes in. LangSmith enables full tracing and inspection of agent behavior, including tool calls, LLM reasoning, and intermediate states.

### Enabling LangSmith Tracing

To enable tracing, add the following lines to your `.env` file:

```
LANGSMITH_TRACING=true
```

```
LANGSMITH_ENDPOINT="https://api.smith.langchain.com"
LANGSMITH_API_KEY="<your-langsmith-api-key>"
LANGSMITH_PROJECT="langchain-in-action-react-agent"
```

After re-running your application and submitting a question, you can log into LangSmith (<https://smith.langchain.com>):

1. Select Tracing projects in the Observability menu (left-hand sidebar).
2. Click your project (langchain-in-action-react-agent) in the right-hand panel.
3. Open the latest trace.

You'll see the full execution trace, as shown in figure 11.4.

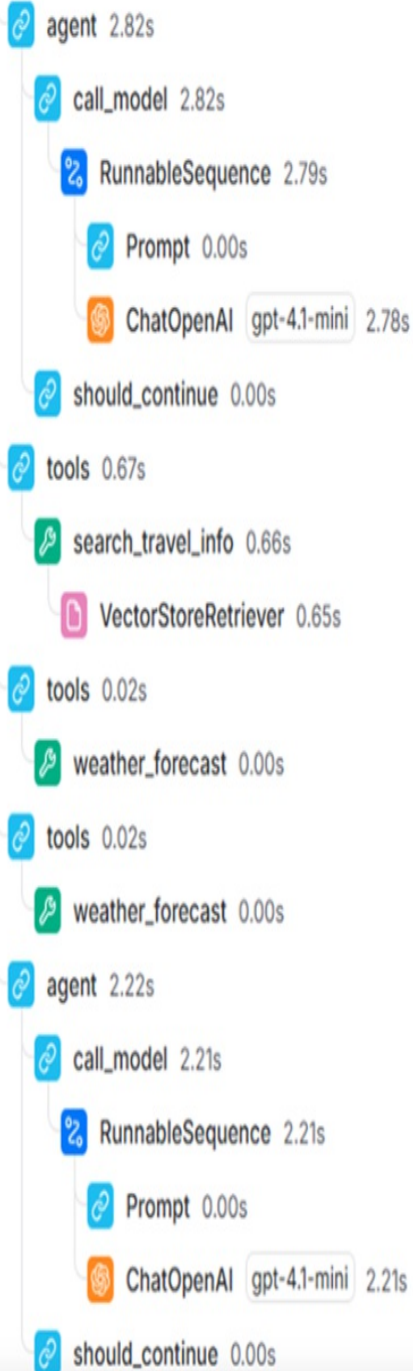
**Figure 11.4** LangSmith agent execution trace: this trace visualizes each step of the agent's workflow—including LLM calls, tool invocations, and message flow—when answering the question, "Suggest two Cornwall beach towns with nice weather."

TRACE



LangGraph

5.82s 1,267



LangGraph ID

Run Feedback Metadata

Input

HUMAN

Suggest two Cornwall beach towns with nice weather

Output

HUMAN

Suggest two Cornwall beach towns with nice weather

AI

search\_travel\_info call\_gnt74CvMudgjk9bsLU0f8BU5

query: beach towns in Cornwall

weather\_forecast call\_oj6cK0c4LtmcgEwsv1e2bMwU

town: St Ives

weather\_forecast call\_aYw57JluKlHwJi3sYxXyKUz9

The LangSmith graphical trace shows every tool call, every LLM step, and the flow of messages—so even when you use a pre-built agent, you retain the ability to audit, debug, and understand exactly what the agent is doing at every stage.

In summary, by combining pre-built LangGraph agent components with LangSmith for observability, you can quickly build robust, production-ready agentic applications—while still maintaining full transparency and control when needed. This workflow has become the foundation for many modern AI agent systems in real-world use today.

## 11.9 Summary

- LangGraph enables you to build both deterministic workflows and dynamic LLM agents using node-based and tool-based architectures.
- You can register tools—like semantic search and weather APIs—so that LLMs can choose and call them step by step.
- By stepping through code with breakpoints, you can trace agent reasoning, tool invocation, and message flow for debugging and understanding.
- Clear tool descriptions and system messages help control LLM behavior and reduce hallucination.
- Multi-tool agents can chain together tool outputs (e.g., finding towns, then filtering by weather), all orchestrated by the LLM.
- Using pre-built agent components, like the LangGraph React agent, dramatically reduces boilerplate and speeds up development.
- LangSmith provides rich observability, showing each LLM step and tool call even for high-level agent abstractions.
- This agent-centric pattern lays the groundwork for more advanced agent and multi-agent systems explored in later chapters.

# 12 Multi-agent Systems

## This chapter covers

- Connecting tools to data sources
- Composing multi-agent systems using router and supervisor patterns
- Debugging, testing, and tracing multi-agent interactions

In chapter 11, we explored the foundations of building AI agents by creating a travel information agent capable of answering user queries about destinations, routes, and transportation options. While a single, specialized agent can be powerful, real-world applications often require the coordination of multiple agents, each handling a distinct area of expertise. In this chapter, we'll embark on that journey—transforming our travel information agent into a robust, multi-agent travel assistant system.

Imagine planning a trip where you not only need up-to-date travel information but also want to seamlessly book your accommodation. Our enhanced multi-agent travel assistant will do just that: it will be able to answer travel questions and help you reserve hotels or bed and breakfasts in your chosen destination. To achieve this, we'll begin by building a new agent—the accommodation booking agent.

The accommodation booking agent will empower users to book lodgings from two different sources. First, it will interface with a local accommodation database, which mainly features hotel deals and is exposed via a dedicated tool. Second, it will connect to an external B&B REST API, providing access to a wider selection of bed and breakfast options, also accessible through its own tool. Depending on user requests, the agent will use one or both of these tools to deliver relevant accommodation options.

Once we have our new agent in place, we'll combine it with the travel information agent from the previous chapter. The result will be a unified, multi-agent travel assistant capable of fielding a wide variety of travel-related queries, handling both information requests and accommodation bookings,

and even combining both for a more streamlined experience.

Let's begin by constructing our new accommodation booking agent.

## 12.1 Building an Accommodation Booking Agent

To build a practical, helpful travel assistant, we need more than just information retrieval—we need the ability to act. In this section, we'll develop an accommodation booking agent from the ground up, starting by building the tools it needs: one for hotel bookings based on a local room availability database, and another for B&B bookings from an external REST API. By the end of this section, you'll have a ReAct-style agent that can check and book both hotel and B&B rooms in Cornwall.

### 12.1.1 Hotel Booking Tool

Let's start by creating the hotel booking tool. To enable our agent to retrieve hotel offers and availability, we'll use the LangChain SQL Database Toolkit, which exposes a SQL database as a set of agent tools. This toolkit makes it straightforward for an agent to run queries, retrieve hotel details, and check room availability—all through tool calls, without needing to write raw SQL in your prompts.

The hotel data, including current offers and availability, is stored in a local SQLite database `cornwall_hotels.db` which is kept up-to-date by our backend partners. We don't need to worry about how the data is pushed—just trust that it's there and refreshed as needed.

First, copy the latest script, `main_03_01.py`, to a new script, `main_04_01.py`. Then, prepare your environment:

1. Create a folder named `hotel_db`.
2. Place the provided SQL schema file `cornwall_hotels_schema.sql` into that folder.
3. Open a terminal (inside VS Code or standalone), navigate to the folder, and create the database with (I have omitted the root of the `ch11` folder for convenience):

```
\ch11>cd hotel_db
\ch11>sqlite3 cornwall_hotels.db < cornwall_hotels_schema.sql
```

Now, let's check that the database is working. Open the SQLite shell:

```
\ch11>sqlite3 cornwall_hotels.db
```

Within the SQLite shell, run these queries to verify your setup:

```
sqlite> .tables
sqlite> SELECT * FROM hotels;
sqlite> SELECT * FROM hotel_room_offers;
```

With the database ready, let's move on to the Python implementation. Import the necessary LangChain SQL integration libraries:

```
from langchain_community.utilities.sql_database import SQLDatabase
from langchain_community.agent_toolkits import SQLDatabaseToolkit
```

Instantiate the SQLite database:

```
hotel_db = SQLDatabase.from_uri("sqlite:///hotel_db/cornwall_hote
```

Now, create an instance of the SQL Database toolkit:

```
hotel_db_toolkit = SQLDatabaseToolkit(db=hotel_db, llm=llm_model)
```

That's it! Now you can access the toolkit's tools with:

```
hotel_db_toolkit_tools = hotel_db_toolkit.get_tools()
```

### 12.1.2 B&B Booking Tool

Next, let's create a Bed & Breakfast booking tool. This tool will retrieve B&B room availability from a REST service. For development and testing, we'll mock this service.

First, we'll define the return type for our tool, and then create a mock implementation of the BnB booking service, as shown in Listing 12.1. (For



convenience, the example here uses a reduced set of mock data. You can find the complete implementation in the code files provided with this book.).

#### Listing 12.1 BnB Booking Service

```
class BnBBookingService: #A
 @staticmethod
 def get_offers_near_town(town: str, num_rooms: int) -> List[B
 # Mocked REST API response: multiple BnBs per destination
 mock_bnb_offers = [#C
 # Newquay
 {"bnb_id": 1, "bnb_name": "Seaside BnB", "town": "New
 {"bnb_id": 2, "bnb_name": "Surfside Guesthouse", "tow
 # Falmouth
 {"bnb_id": 3, "bnb_name": "Harbour View BnB", "town":
 {"bnb_id": 4, "bnb_name": "Seafarer's Rest", "town":
 # St Austell
 {"bnb_id": 5, "bnb_name": "Garden Gate BnB", "town":
 {"bnb_id": 6, "bnb_name": "Coastal Cottage BnB", "tow
 ...
 # Port Isaac
 {"bnb_id": 27, "bnb_name": "Port Isaac View BnB", "to
 {"bnb_id": 28, "bnb_name": "Fisherman's Cottage BnB",
 # Fowey
 {"bnb_id": 29, "bnb_name": "Fowey Quay BnB", "town":
 {"bnb_id": 30, "bnb_name": "Riverside Rest BnB", "tow
]
 offers = [offer for offer in mock_bnb_offers if offer["to
 return offers
```

Now we can define the `check_bnb_availability` tool:

#### Listing 12.2 BnB Availability tool

```
@tool(description="Check BnB room availability and price for a de
def check_bnb_availability(destination: str, num_rooms: int) -> L
 """Check BnB room availability and price for the requested de
 offers = BnBBookingService.get_offers_near_town(destination,
 if not offers:
 return [{"error": f"No available BnBs found in {destinati
 return offers
```

### 12.1.3 ReAct Accommodation Booking Agent

With both the hotel and B&B booking tools ready, it's time to build the ReAct accommodation booking agent. This agent will use both tools in response to user requests. If the user doesn't specify a preference, the agent will search both hotels and B&Bs for available rooms.

```
BOOKING_TOOLS = hotel_db_toolkit_tools + [check_bnb_availability]

accommodation_booking_agent = create_react_agent(#B
 model=llm_model,
 tools=BOOKING_TOOLS,
 state_schema=AgentState,
 prompt="You are a helpful assistant that can check hotel and
)
```

You can now try out the agent by replacing the agent line in `chat_loop()` with:

```
...
result = accommodation_booking_agent.invoke(state)
...
```

Let's run the `main_04_01.py` script in debug mode and ask the following question:

```
UK Travel Assistant (type 'exit' to quit)
You: Are there any rooms available in Penzance?
```

After you hit ENTER, you might see an answer similar to the following one:

```
I have found Penzance Pier BnB with available rooms at £95 per ro
For hotels, Penzance Palace has 3 available rooms with prices of
```

As you can see, the agent used both tools to retrieve results from both the hotel database and the mock B&B service.

At this point, your accommodation booking agent is working as expected. It's strongly recommended to debug the execution and inspect the LangSmith traces to better understand how the agent is reasoning and acting step by step.

Although you now have both a travel information agent and an accommodation booking agent, they are still disconnected. You can use

either one or the other, but not both in a unified experience. In the next section, we'll build a multi-agent travel assistant that brings these capabilities together, providing a seamless experience for travel planning and accommodation booking.

## 12.2 Building a router-based Travel assistant

So far, we have developed two independent agents: a travel information agent and an accommodation booking agent, each with specialized capabilities. While this modular approach is powerful, it raises an essential design question: how can we combine these agents to deliver a seamless user experience—one that can answer travel information queries and handle accommodation bookings in a single conversation?

A common and effective solution is to introduce a router agent. This agent acts as an intelligent entry point: it receives the user's message, determines which specialized agent should handle the request, and dispatches the task accordingly.

### 12.2.1 Designing the Router Agent

To implement our multi-agent travel assistant, begin by copying your previous script, `main_04_01.py`, to `main_05_01.py`. Next, we need to bring in some extra libraries to support graph-based workflows:

```
from langgraph.graph import StateGraph, END
from langgraph.types import Command
```

The next step is to clearly define the set of agents available for routing. We do this by declaring an enumeration for the two agent types:

```
class AgentType(str, Enum):
 travel_info_agent = "travel_info_agent"
 accommodation_booking_agent = "accommodation_booking_agent"
```

To ensure our router agent receives clear and structured decisions from the LLM, we define a Pydantic model that captures the LLM's output—specifying which agent should handle each query:

```
class AgentTypeOutput(BaseModel):
 agent: AgentType = Field(..., description="Which agent should
```

By configuring the OpenAI LLM client to produce responses in this structured format, we eliminate any need for string parsing or manual post-processing. The router will always produce a result of either "travel\_info\_agent" or "accommodation\_booking\_agent".

## 12.2.2 Routing Logic

The heart of the router agent is its system prompt, which concisely instructs the LLM how to classify each user request:

```
ROUTER_SYSTEM_PROMPT = (
 "You are a router. Given the following user message, decide i
 "or an accommodation booking question (about hotels, BnBs, ro
 "If it is a travel information question, respond with 'travel
 "If it is an accommodation booking question, respond with 'ac
)
```

With this system prompt, the router agent evaluates each user input and decides which specialist agent should take over. The router is implemented as the entry node of our LangGraph workflow, with the travel information agent and the accommodation booking agent as subsequent nodes. You can see the router implementation in listing 12.2.

### Listing 12.3 Router Agent node

```
def router_agent_node(state: AgentState) -> Command[AgentType]:
 """Router node: decides which agent should handle the user qu
 messages = state["messages"] #A
 last_msg = messages[-1] if messages else None #B
 if isinstance(last_msg, HumanMessage): #C
 user_input = last_msg.content #D
 router_messages = [#E
 SystemMessage(content=ROUTER_SYSTEM_PROMPT),
 HumanMessage(content=user_input)
]
 router_response = llm_router.invoke(router_messages) #F
 agent_name = router_response.agent.value #G
 return Command(update=state, goto=agent_name) #H
```

```
return Command(update=state, goto=AgentType.travel_info_agent
```

If you examine the implementation in listing 12.2, you'll notice that the router extracts the user's message and submits it to the LLM along with the system prompt. The LLM returns a structured output of type `AgentTypeOutput`, which contains the agent name to which the request should be routed. The router then uses a `Command` to redirect the conversation flow to the selected agent node in the graph. In simple workflows, the `Command` can hand off the unchanged state to the new node, but it also allows for state updates in more complex flows.

### 12.2.3 Building the Multi-Agent Graph

At this point, you have all the components needed to assemble the graph-based multi-agent system. You can see the graph implementation in listing 12.3.

**Listing 12.4 Router-based multi-agent travel assistant graph**

```
graph = StateGraph(AgentState) #A
graph.add_node("router_agent", router_agent_node) #B
graph.add_node("travel_info_agent", travel_info_agent) #C
graph.add_node("accommodation_booking_agent", accommodation_booki

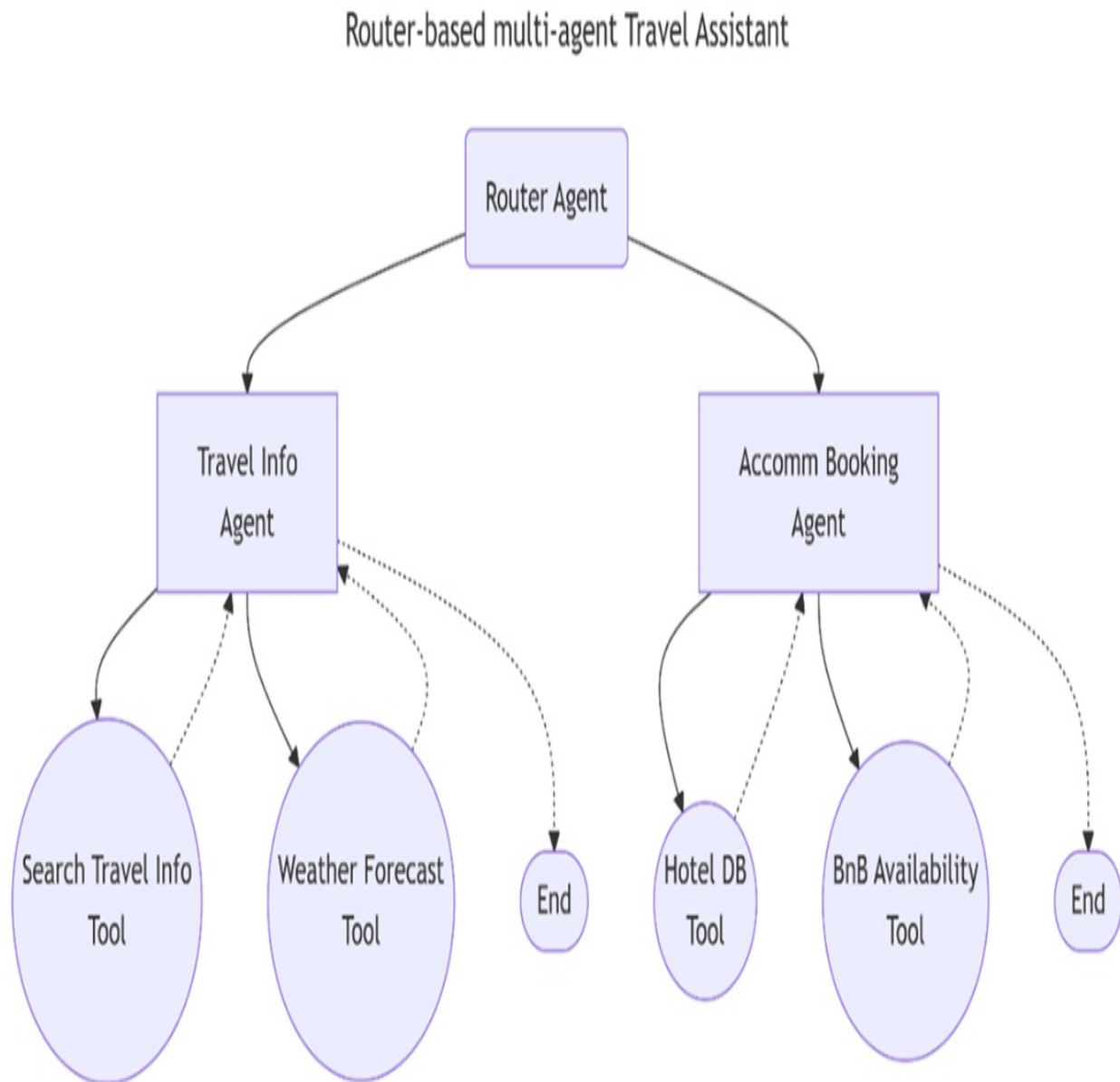
graph.add_edge("travel_info_agent", END) #E
graph.add_edge("accommodation_booking_agent", END) #F

graph.set_entry_point("router_agent") #G
travel_assistant = graph.compile() #H
```

The workflow graph connects the router agent to the two specialized agents. Notably, the only explicit edges you define are those from the travel information agent and the accommodation booking agent to the end of the workflow. The connection from the router to the specialized agents is determined dynamically at runtime by the LLM's response and is handled via `Command`.

Below is a graphical representation of the current multi-agent travel assistant graph:

**Figure 12.1 Router-based multi-agent Travel assistant: the router agent dispatches user queries to either the travel information agent or the accommodation booking agent, each equipped with their own specialized tools**



As the diagram in figure 12.1 shows, this is a hybrid architecture. At the top, the system exhibits a deterministic, workflow-driven routing logic. At the

lower level, each specialist agent uses its own set of tools (such as travel data APIs or accommodation booking interfaces) and follows an agentic, tool-based decision process, which is inherently more flexible and dynamic.

### 12.2.4 Trying Out the Router Agent

To see the system in action, run your multi-agent travel assistant by starting `main_05_01.py` in debug mode and try the following two user queries:

- What are the main attractions in St Ives?
- Are there any rooms available in Penzance this weekend?

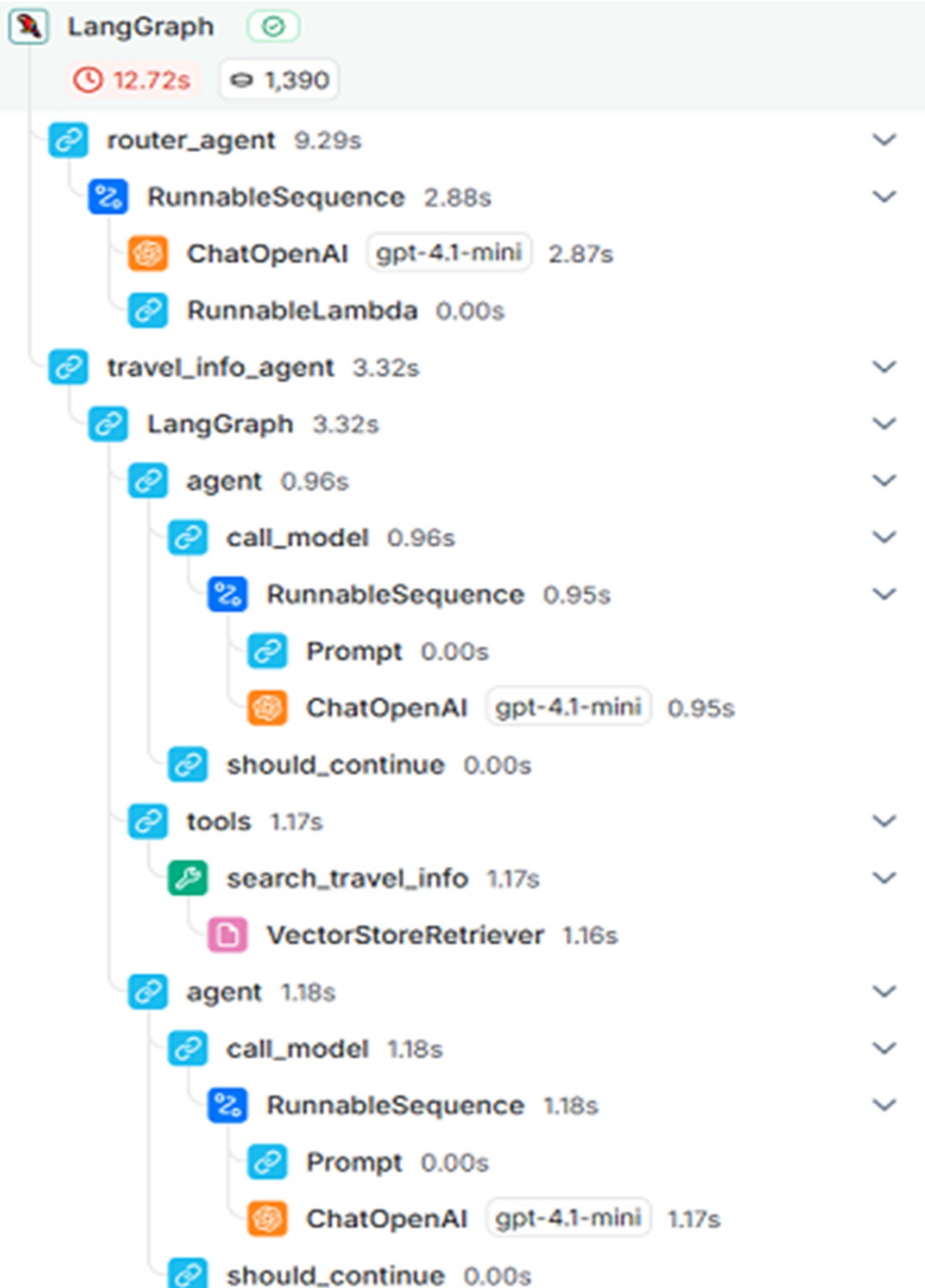
One important thing to note with this design is that each user question is routed to a single agent for handling—in other words, each query takes a "one-way ticket" through the workflow. The router makes a clean and unambiguous handoff, and the selected agent responds directly to the user before the workflow ends.

For example, when you ask:

What are the main attractions in St Ives?

the request is routed to the travel information agent. You can see the related LangSmith execution trace in figure 12.2.

**Figure 12.2 LangSmith execution trace of a travel information question**





If you ask:

Are there any rooms available in Penzance this weekend?

the system dispatches the query to the accommodation booking agent.

In both cases, the workflow (which can be followed in figure 12.2) is clear: the router agent evaluates the intent and hands the query off to the most appropriate specialist agent, which then handles the response and ends the session.

This modular, graph-based design provides a strong foundation for more advanced workflows. In later sections, you'll see how you can evolve this system to handle more complex, multi-step, or even collaborative agentic scenarios.

## **12.3 Handling Multi-Agent Requests with a Supervisor**

The workflow-based multi-agent architecture we developed in the previous section works well for simple, single-purpose queries—questions that can be clearly routed to either the travel information agent or the accommodation booking agent. But what happens when a user asks for something that spans both domains? Consider a query like:

"Can you find a nice seaside Cornwall town with good weather right now and find availability and price for one double hotel room in that town?"

With our previous router-based design, such a question cannot be answered effectively, as it requires both agents to work together and share intermediate results.

To solve this, we need to shift our architecture towards a more flexible, collaborative agent system—one where multiple specialized agents can be coordinated as “sub-tools” under a higher-level manager. In LangGraph, this is exactly the use case for the Supervisor: a built-in component designed to

orchestrate multiple agents, allowing them to collaborate on complex requests.

### 12.3.1 The Supervisor Pattern: An Agent of Agents

Conceptually, the Supervisor is an “agent of agents.” It acts as an orchestrator, managing a collection of other agents (which themselves may use tools) and deciding which agent to activate, possibly multiple times in a single workflow. Each agent acts as a specialized tool that the Supervisor can invoke as needed.

Let’s see how to set up this pattern in your multi-agent travel assistant.

Start by copying one of your previous implementation, `main_04_01.py`, to a new script, `main_06_01.py`. Next, install the necessary package:

```
pip install langgraph-supervisor
```

Then import the Supervisor:

```
from langgraph_supervisor.supervisor import create_supervisor
```

When defining agents to be managed by the Supervisor, it’s important to assign each a unique name. You can see how you instantiate your agents with names in listing 12.4.

#### Listing 12.5 Setting up the leaf agents

```
travel_info_agent = create_react_agent(
 model=llm_model,
 tools=TOOLS,
 state_schema=AgentState,
 name="travel_info_agent",
 prompt="You are a helpful assistant that can search travel in
)

accommodation_booking_agent = create_react_agent(
 model=llm_model,
 tools=BOOKING_TOOLS,
 state_schema=AgentState,
 name="accommodation_booking_agent",
```

```
) prompt="You are a helpful assistant that can check hotel and
)
```

Now you can implement your travel assistant as a Supervisor, as shown in listing 12.5:

**Listing 12.6 Setting up the Supervisor agent**

```
travel_assistant = create_supervisor(#A
 agents=[travel_info_agent, accommodation_booking_agent], #B
 model= ChatOpenAI(model="gpt-4.1", use_responses_api=True), #
 supervisor_name="travel_assistant",
 prompt=(#D
 "You are a supervisor that manages two agents: a travel i
 "You can answer user questions that might require calling
 "Decide which agent(s) to use for each user request and c
)
) .compile() #E
```

You’ll notice that configuring a Supervisor is much like setting up a ReAct agent, but instead of passing a list of tools, you provide a list of agents. Since the Supervisor needs to analyze complex multi-step requests and coordinate several agents, it’s best to use a more powerful LLM model (like gpt-4.1 or even o3) to maximize accuracy and task decomposition.

**Tip**

Try experimenting with different models for the Supervisor, such as gpt-4.1, o3 or o4-mini, and compare how well the assistant handles increasingly complex, multi-faceted questions.

As in previous designs, simply update your chat loop to invoke the supervisor-based travel assistant:

```
result = travel_assistant.invoke(state)
```

### 12.3.2 From “One-Way” to “Return Ticket” Interactions

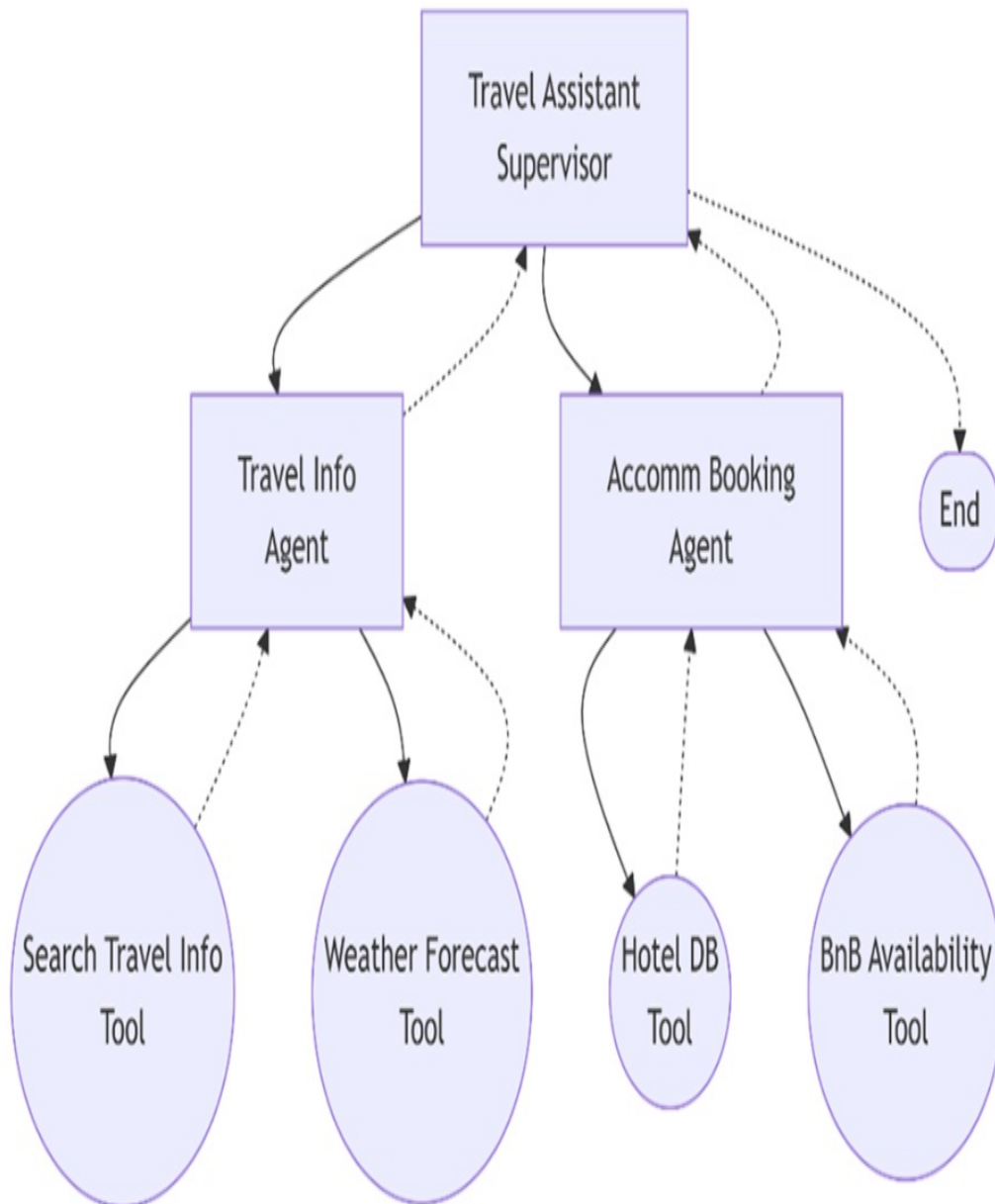
Unlike the workflow-based router—where every user question was routed once and only once to a specific agent (a “one-way ticket”)—the Supervisor

enables a much richer interaction. The Supervisor can invoke each agent as needed, potentially revisiting agents multiple times (“return tickets”) in a single session to collect, combine, and reason over intermediate results. This enables the system to handle more sophisticated, open-ended, and multi-part queries.

Below, you can see a diagram representing this supervisor-based architecture:

**Figure 12.3 Router-based multi-agent Travel assistant: the router agent dispatches user queries to either the travel information agent or the accommodation booking agent, each equipped with their own specialized tools**

## Supervisor-based multi-agent Travel Assistant



As shown in the diagram in figure 12.3, both the high-level (supervisor) and low-level (agent/tool) orchestration follow a tool-based approach, maximizing flexibility and composability. The Supervisor becomes the central decision-maker, ensuring the right agent (or sequence of agents) is activated for every complex travel request.

This Supervisor-driven architecture unlocks a new level of multi-agent collaboration, laying the groundwork for more advanced, open-ended AI travel assistants capable of addressing real-world, multi-step needs.

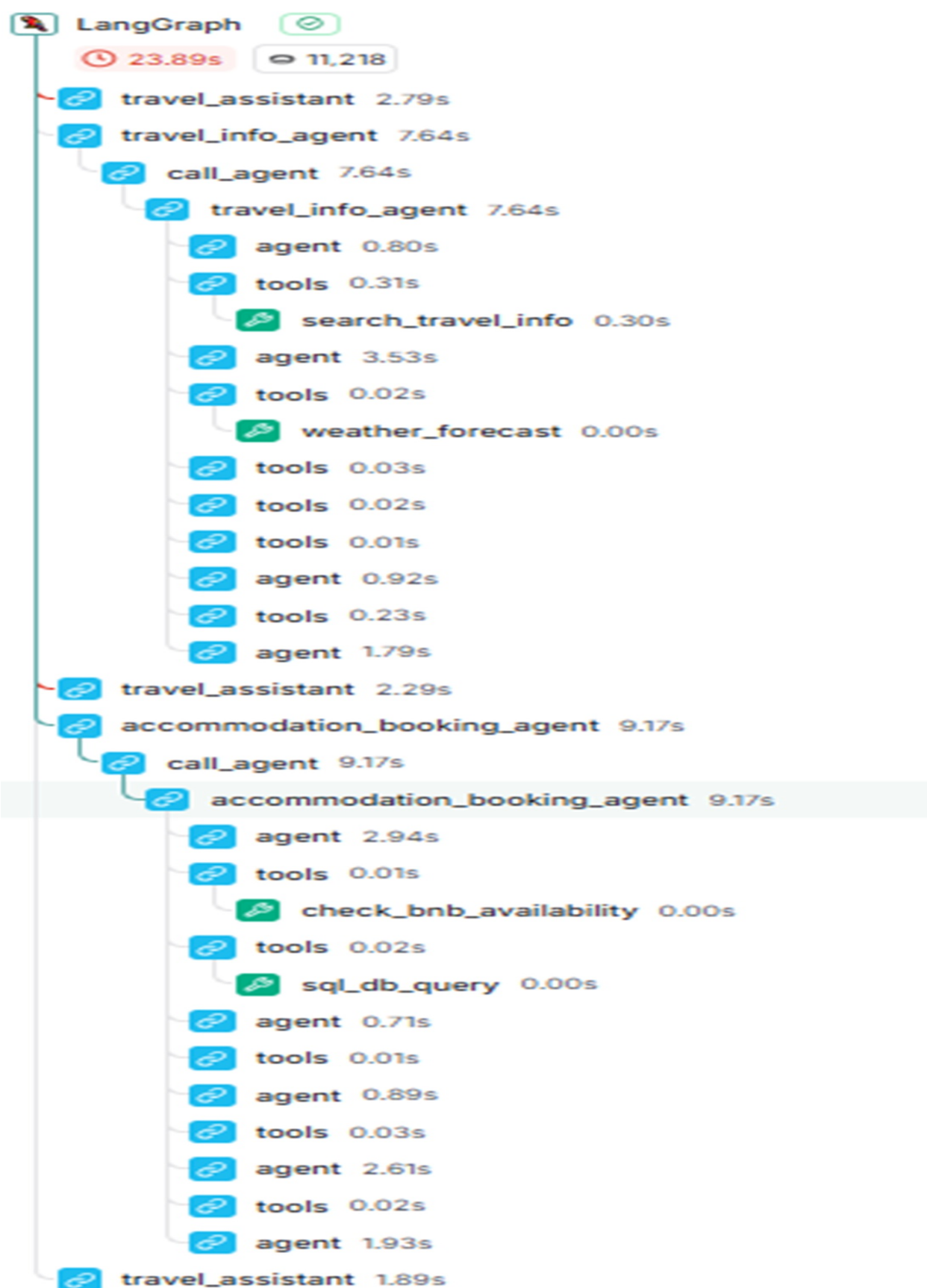
### 12.3.3 Trying out the Supervisor agent

Now, run the travel assistant by starting `main_06_01.py` in debug mode (with LangSmith tracing enabled), and try entering a complex, multi-part question such as:

```
UK Travel Assistant (type 'exit' to quit)
You: Can you find a nice seaside Cornwall town with good weather
```

When you examine the LangSmith trace, you'll notice a more intricate agent utilization trajectory, similar to that in figure 12.4, in which the `travel_assistant` is the supervisor agent.

**Figure 12.4** LangSmith execution trace of a combine travel information and booking question



I have summarized below the key steps from the execution trace below, so you can understand the flow better (remember the `travel_assistant` is the supervisor):

- `travel assistant`
  - `tools`
    - `transfer_to_travel_info_agent`
- `travel_info_agent`
  - `tools`
    - `search_travel_info`
  - `tools`
    - `weather_forecast`
- `travel_assistant`
  - `tools`
    - `transfer_to_accommodation_booking_agent`
- `accommodation_booking_agent`
  - `tools`
    - `check_bnb_availability`
  - `tools`
    - `sql_db_query`

This shows that the supervisor-based travel assistant was able to coordinate both agents, using each one and its underlying tools as needed to fully answer the user's question. Technically, agents are now orchestrated as tools, and the Supervisor manages this collaboration dynamically.

## 12.4 Summary

- By integrating external tools—such as SQL databases and REST APIs—agents can retrieve real-time data and perform actions beyond simple information retrieval.
- Router-based agent design enable modularity by directing each user query to the most appropriate specialist agent, simplifying single-task scenarios.
- The supervisor pattern goes further, orchestrating collaboration among agents, so multiple agents can be used together to answer complex,



multi-part questions.

- Tracing and debugging with LangSmith provides valuable visibility into agent decisions and tool usage, making it easier to optimize and extend your system.

# 13 Building and consuming MCP servers

## This chapter covers

- The purpose and architecture behind the Model Context Protocol (MCP)
- Building and exposing your own MCP server, with a practical weather tool example
- Testing and consuming MCP servers and related tools in applications
- Integrating remote MCP tools into agents alongside local tools

Building AI agents that can effectively access and use external context is a central challenge for application developers. Previously, integrating context from different sources meant wrapping each one as a tool, following specific protocols, a time-consuming and repetitive process repeated by countless teams. Imagine if, instead, data and service providers could expose their resources as ready-made tools, instantly available for any agent or application. This is the promise of the Model Context Protocol (MCP), introduced by Anthropic.

MCP defines a standardized way for services to expose , tools, through MCP servers. Agents, or , MCP hosts,, connect to these servers via MCP clients, discovering and using remote tools as easily as local ones. This approach moves much of the integration work to where it belongs, at the source, allowing developers to focus on building more capable agents rather than reinventing the same plumbing. Once the connection is set up, tools from MCP servers work seamlessly with existing agent architectures.

Since its release in late 2024, MCP has rapidly become a de-facto standard. Major LLM providers like OpenAI and Google have adopted MCP in their APIs and SDKs, while a growing number of companies and communities are making services available as MCP servers. Today, thousands of tools are listed on public MCP server portals, ready to be integrated into new or existing projects with minimal effort.

In this chapter, you,Äôll get hands-on experience with MCP. We,Äôll introduce the core protocol and architecture, highlight the growing ecosystem, and show you where to find both official and community MCP servers. Then, you,Äôll learn to build your own MCP server,Äôstarting with a practical weather data example using AccuWeather,Äôtest it, and integrate it into an agent application. You,Äôll also see how to combine remote MCP tools with local logic, configure clients, and follow best practices for robust integration.

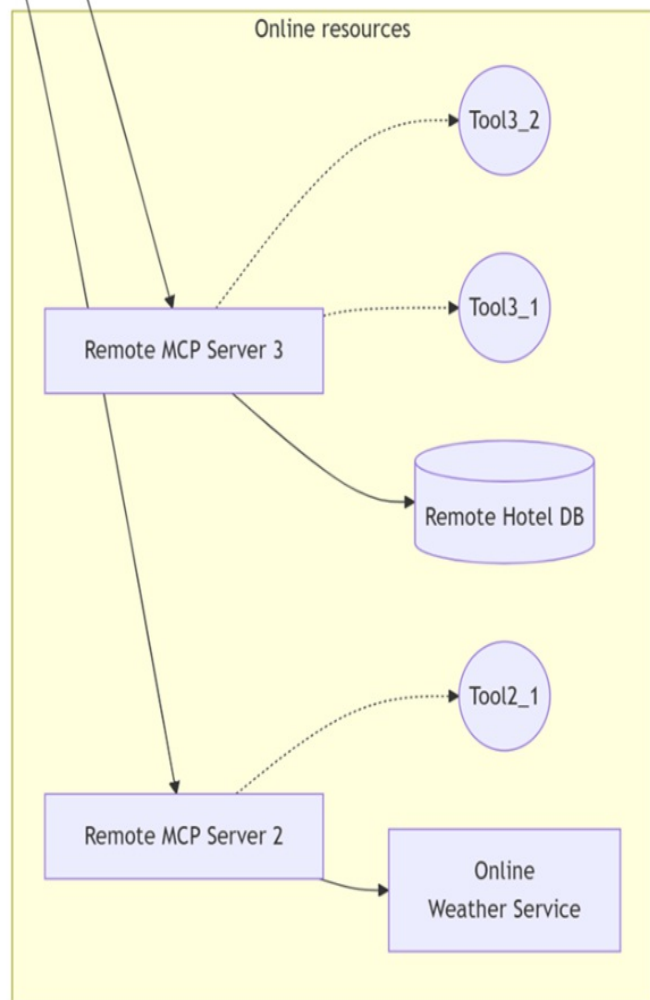
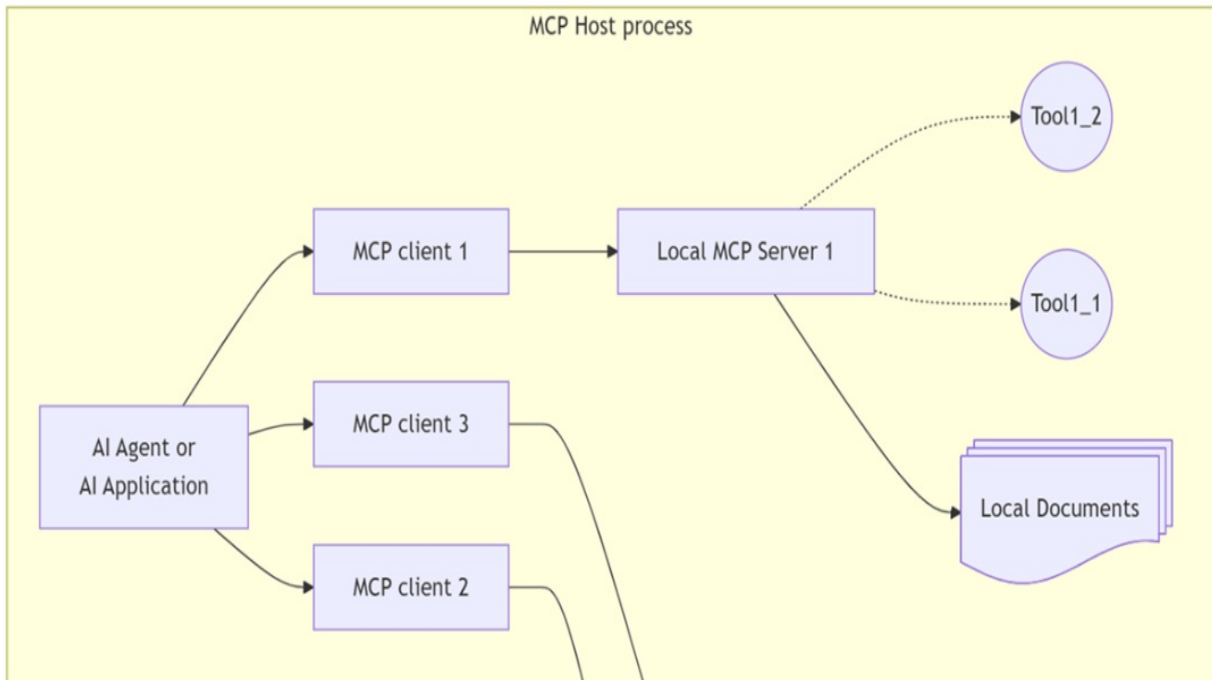
As the MCP ecosystem grows, understanding how to build and consume MCP servers is becoming essential for developers and service providers alike. Let,Äôs dive in and explore how MCP is shaping modern, context-rich AI applications.

## 13.1 Introduction to MCP Servers

Building advanced AI agents increasingly involves giving them access to external sources of context, such as databases, APIs, or other services. Traditionally, this is achieved by wrapping each data source as a ,Äútool,Äù that agents can call using a protocol supported by most large language models (LLMs). However, this approach quickly leads to repeated work,Äôevery developer has to build and maintain similar wrappers for common sources, resulting in duplicated effort across the ecosystem.

What if there was a better way? Instead of every developer reinventing the wheel, what if the providers of context could directly expose their data and services as tools, ready to be consumed by any agent? This is the central motivation behind the Model Context Protocol (MCP), an open standard introduced by Anthropic. MCP allows for context and capabilities to be exposed in a unified, tool-based way, as you can appreciate in figure 13.1,Äôeliminating much of the friction in agent development and encouraging a richer, more reusable ecosystem.

**Figure 13.1 MCP host process connecting to multiple local and remote MCP servers via MCP clients, each exposing tools backed by various resources. This architecture enables agents to flexibly access both local documents and online services through standardized tool interfaces**



### 13.1.1 The Problem: Context Integration at Scale

As we saw in previous chapters, LLM agents typically consume external data via tools injected into their requests. Each new context source, *À* weather API, a document database, a search service, *À* means creating yet another wrapper tool and integrating it with the agent, often following slightly different conventions and protocols. This process is not only repetitive but also results in the same work being duplicated by countless teams.

Imagine every agent developer spending time wrapping the same set of public APIs, or every organization hand-rolling integrations for standard services. This approach simply doesn't scale, especially as the range of possible tools and context sources grows.

### 13.1.2 The Solution: The Model Context Protocol (MCP)

MCP addresses this challenge by providing a standardized way for data and service providers to expose tools via MCP servers. Instead of each agent individually wrapping every service, developers can simply *,À* subscribe, *À* to MCP servers and use the tools they expose, with minimal additional work.

The protocol defines a classic client-server architecture, as you can see in figure 13.1.

Here, the MCP host process, *À* the agent or application, *À* connects to one or more MCP servers using an MCP client. Each MCP server can expose a collection of tools, and the agent subscribes only to the tools it needs. This architecture is familiar from that of REST APIs and WebSockets, which expose endpoints for consumption through a client, and makes it easy to add or swap out new sources of context as requirements change.

As you can also see on figure 13.1, MCP servers can be either remote (accessed over StreamableHTTP, usually in production) or local (accessed via STDIO, typically for development or for accessing local resources such as files). In most cases, and throughout this chapter, we, *À* ll assume you are working with remote MCP servers.

Once configured, tools from MCP servers integrate with your agent in exactly the same way as local tools, adhering to the same tool calling protocols you learned in earlier chapters. If your agent is modular and well-architected, you often don't need to change any logic at all to support remote MCP tools.

**Note**

MCP goes beyond just tools, it can also standardize how prompts, files, and other resources are shared. For our purposes, we'll focus on tool exposure, but for a deeper dive, see the original MCP article by Anthropic (<https://www.anthropic.com/news/model-context-protocol>).

**13.1.3 The MCP Ecosystem**

Since its introduction in late 2024, MCP has rapidly gained traction. Leading LLM providers like OpenAI and Google have both adopted MCP, integrating support into their APIs and agent SDKs. On the provider side, companies are increasingly wrapping their services and data with MCP servers, making them instantly AI-ready for a wide range of agents and applications.

A number of public MCP server portals have emerged, making it easy to find tools for almost any need. Table 13.1 highlights some of the most prominent directories:

**Table 13.1 MCP server portals**

Portal	Description
<a href="https://github.com/modelcontextprotocol/servers">https://github.com/modelcontextprotocol/servers</a>	Anthropic's official MCP portal, listing both official and community servers
<a href="https://mcp.so/">https://mcp.so/</a>	A community-driven directory featuring over 16,000 servers
<a href="https://smithery.ai/">https://smithery.ai/</a>	A portal with over 5,000 tools, mostly MCP-

	compliant
<a href="https://mcpservers.org/">https://mcpservers.org/</a>	A collection of around 1,500 servers

With such a broad and growing ecosystem, agents can increasingly draw on a shared library of tools, whether built in-house, offered by major companies, or shared by the community.

After this overview of the motivation, architecture, and ecosystem behind MCP servers, we are ready to look at how to build, expose, and consume these tools in real-world applications. In the next sections, we'll explore how to create your own MCP server and how to integrate MCP tools seamlessly into your agents.

## 13.2 How to Build MCP Servers

With a clear understanding of what MCP servers are and their role in the modern AI agent ecosystem, the next step is to learn how to actually build, deploy, and integrate MCP servers. This section will guide you through the key resources, official tools, and best practices to help you create robust MCP servers and make them available to agents and applications.

### 13.2.1 Essential Resources for MCP Server Development

Before diving into code, it's important to familiarize yourself with the foundational documentation and tools for MCP development. The official hub for the protocol is [modelcontextprotocol.io](https://modelcontextprotocol.io), which provides a wealth of information on the architecture, design principles, tutorials, and the complete protocol specification. Whether you are new to MCP or looking for advanced features, this site should be your starting point.

#### Tip

Pay particular attention to the MCP protocol specification (<https://modelcontextprotocol.io/specification>), which is regularly updated and details every aspect of the protocol, including transport mechanisms, security considerations, and best practices. The specification itself is

technology-agnostic, making it valuable no matter which language or platform you use.

While understanding the protocol is crucial, implementing MCP from scratch in your own application is not recommended. Doing so would mean duplicating a significant amount of effort, and you,Äôd likely end up reimplementing features already solved in mature SDKs. Instead, you should leverage one of the official language-specific SDKs available for MCP.

### **13.2.2 Official Language-Specific MCP SDKs**

The official MCP GitHub repository aggregates a variety of resources, with particular emphasis on the official SDKs for several major programming languages,Äôincluding Python, JavaScript, Java, Kotlin, and C#. This book will primarily focus on Python, but the same general principles apply to the other supported languages.

#### **FastMCP 1**

The initial Python SDK, often referred to as FastMCP 1, is available at <https://github.com/modelcontextprotocol/python-sdk> . This library provided the community with the first robust framework for building and consuming MCP servers in Python.

#### **FastMCP 2**

Building on the experience with FastMCP 1, the MCP community released FastMCP 2,Äôa major upgrade that addresses limitations of the original implementation and aligns more closely with the latest protocol specifications. FastMCP 2 offers significant improvements, such as:

- Easier deployment and server composition
- Enhanced security features
- Improved client connectivity and advanced capabilities, like dynamic tool rewriting
- Built-in testing utilities and integration hooks for other libraries



FastMCP 2 is actively maintained at <https://github.com/jlowin/fastmcp>, and you,Äôll find comprehensive documentation and tutorials at <https://gofastmcp.com/getting-started/welcome>. We,Äôll use FastMCP 2 for hands-on examples, so keep these resources handy as you follow along.

### 13.2.3 Consuming MCP Servers in LLM Applications and Agents

While the previous sections focused on building MCP servers, it,Äôs equally important to understand how to consume these servers in your own AI-powered applications. Thanks to wide adoption, integrating MCP tools has become remarkably straightforward,Äêespecially with leading platforms like OpenAI.

OpenAI,Äôs API natively supports tools provided by public MCP servers via the Responses API. Not only can you discover and reference these tools, but OpenAI will also execute them for you,Äêeliminating the need for manual client code in many cases.

#### Tip

Review the OpenAI documentation on remote MCP tool integration. The process is simple, but it,Äôs recommended to carefully consider your authorization strategy,Äêdeciding whether to approve such calls automatically or interactively.

In many enterprise environments, MCP servers might be available only within organizational boundaries, so you cannot expect the OpenAI Responses API to execute the remote tools for you. For these scenarios, there are two primary approaches:

1. Use the FastMCP Client: The official FastMCP SDK provides client facilities to connect, authenticate, and consume tools from MCP servers directly.
2. Take advantage of LangChain/LangGraph Integrations: If you are developing agents with LangGraph or LangChain, you can use the LangChain MCP client library,Äêparticularly the

MultiServerMCPClient class, to easily aggregate and consume tools from multiple MCP servers through a simple configuration interface.

In the following sections, we'll demonstrate both approaches in practice. You'll learn how to build a practical MCP server, test it, and integrate it into an agent workflow, whether you're working directly with SDKs or leveraging modern agent frameworks like LangChain.

## 13.3 Building a Weather MCP Server

After learning about what MCP Servers are, their purpose, and the available libraries and ecosystem, it is time to build one! In this section, we'll replace the mockup weather tool that you used to build agents in previous chapters with a real-world weather MCP server based on the AccuWeather REST API. We'll also integrate this server into one of the agent-based solutions built earlier. Step by step, you'll see how to build, test, and connect the MCP server to your agent.

### 13.3.1 Implementing the MCP Server

We begin by replacing our previous mock weather tool with a proper MCP server that exposes live weather data from AccuWeather.

Before implementing the code, go to the AccuWeather developer portal and register (for free at: <https://developer.accuweather.com/user/register>) to claim an API key.

- After registering, navigate to the My Apps tab and click "Add a new App".
- Fill in the details, such as:
  - App name: accu-mcp-server
  - Products: Core Weather Limited Trial
  - Where will the app be used: Desktop Website
  - What will you be creating with this API: Weather App
  - Programming language: Python
  - Choose other settings as desired, then click Create App.
- Once your new app is registered, locate the API key in the My Apps

dashboard and copy it.

- Add the API key to your .env file in your Python project, as shown below (replace the API key accordingly):

```
ACCUWEATHER_API_KEY=<Your API key>
```

You are now ready to implement a real MCP server that exposes the weather service. Create a folder named `mcp` within the `ch11` folder, and create an empty Python script called `accuweather_mcp.py`.

You can see the MCP server implementation in listing 14.1, adapted from this GitHub repository: <https://github.com/adhikasp/mcp-weather>.

#### **Listing 13.1 Accuweather MCP server**

```
import os
import json
from typing import Dict
from fastmcp import FastMCP
from dotenv import load_dotenv
from aiohttp import ClientSession

load_dotenv() #A

mcp = FastMCP("mcp-accuweather") #B

@mcp.tool(description="Get weather conditions for a location.") #
async def get_weather_conditions(location: str) -> Dict:
 """Get weather conditions for a location."""
 api_key = os.getenv("ACCUWEATHER_API_KEY") #D
 base_url = "http://dataservice.accuweather.com"

 async with ClientSession() as session:
 location_search_url = f"{base_url}/locations/v1/cities/se
 params = { #F
 "apikey": api_key,
 "q": location,
 }
 async with session.get(location_search_url, params=params)
 locations = await response.json() #G
 if response.status != 200:
 raise Exception(f"Error fetching location data: {
 if not locations or len(locations) == 0:
 raise Exception("Location not found")
```

```

location_key = locations[0]["Key"] #H

Get current conditions
current_conditions_url = f"{base_url}/currentconditions/v
params = { #J
 "apikey": api_key,
 "details": "true"
}
async with session.get(current_conditions_url, params=params) as response:
 current_conditions = await response.json() #K

Format current conditions
if current_conditions and len(current_conditions) > 0:
 current = current_conditions[0] #L
 current_data = {
 "temperature": {
 "value": current["Temperature"]["Metric"]["Value"],
 "unit": current["Temperature"]["Metric"]["Unit"],
 },
 "weather_text": current["WeatherText"],
 "relative_humidity": current.get("RelativeHumidity"),
 "precipitation": current.get("HasPrecipitation"),
 "observation_time": current["LocalObservationDate"]
 }
else:
 current_data = "No current conditions available" #M

return { #N
 "location": locations[0]["LocalizedNames"],
 "location_key": location_key,
 "country": locations[0]["Country"]["LocalizedNames"],
 "current_conditions": current_data,
}

if __name__ == "__main__":
 mcp.run(transport="streamable-http", host="127.0.0.1", #O
 port=8020, path="/accu-mcp-server")

```

This implementation relies on the `fastmcp` package (which is FastMCP 2). Make sure to install the required package in your virtual environment (or add `fastmcp` to your `requirements.txt` and run `pip install -r requirements.txt`).

The core logic of the implementation is simple: it searches the underlying AccuWeather locations REST endpoint to resolve the user's input, then

queries current weather conditions using AccuWeather,Â’s API.

```
(env_ch11) C:\Github\building-llm-applications\ch11> pip install
```

Now, open a new terminal (within VS Code or otherwise), activate your virtual environment, move into the mcp folder, and run the server:

```
C:\Github\building-llm-applications\ch11>env_ch11\Scripts\activat
(env_ch11) C:\Github\building-llm-applications\ch11>cd mcp
(env_ch11) C:\Github\building-llm-applications\ch11\mcp>python ac
```

You will see the MCP server starting up, and finally you'll see this output:

```
,Üê[32mINFO,Üê[0m: Started server process [,Üê[36m20712,Üê[0m
,Üê[32mINFO,Üê[0m: Waiting for application startup.
,Üê[32mINFO,Üê[0m: Application startup complete.
,Üê[32mINFO,Üê[0m: Uvicorn running on ,Üê[1mhttp://0.0.0.0:80
```

Congratulations! Your first MCP server is up and running!

### 13.3.2 Trying out the MCP Server with MCP Inspector

One of the fastest ways to interactively test your newly created MCP server is by using the MCP Inspector. This tool provides a user-friendly interface that lets you connect to any MCP server, explore its tools, and run live queries without needing to write any client code. The process is straightforward, and the Inspector is a great way to build confidence before integrating the server into your applications.

#### Installing MCP Inspector

To get started, you,Â’ll need to install the MCP Inspector on your computer. MCP Inspector is a Node.js application, so ensure you have Node.js installed. If not, you can download and install it from [nodejs.org](https://nodejs.org).

Once Node.js is ready, open a new command prompt or terminal. It,Â’s a good practice to keep your tools organized, so create a new folder under your project directory,Â’such as mcp-inspector inside ch11. Then run the following command to launch MCP Inspector using npx:

```
c:\Github\building-llm-applications\ch11\mcp-inspector>npx @model
```

During installation, you will be asked to confirm before proceeding:

```
Need to install the following packages:
@modelcontextprotocol/inspector@0.16.212
Ok to proceed? (y) y
```

After confirmation, the installation will proceed, and MCP Inspector will automatically launch in your browser:

# 14 Productionizing AI Agents: memory, guardrails, and beyond

## **This chapter covers:**

- Adding short-term memory with LangGraph checkpoints
- Implementing guardrails at multiple workflow stages
- Additional considerations for production deployment

Building AI agents that behave reliably in real-world environments is about more than just connecting a language model to some tools. Production systems need to maintain context across turns, respect application boundaries, handle edge cases gracefully, and keep operating even when something unexpected happens. Without these safeguards, even the most capable model will eventually produce errors, off-topic answers, or inconsistent behavior that undermines user trust.

In this chapter, we'll focus on two of the most important capabilities for making AI agents production-ready: memory and guardrails. Memory allows an agent to “remember” past interactions, enabling it to hold natural conversations, answer follow-up questions, and recover from interruptions. Guardrails keep the agent within its intended scope and policy framework, filtering out irrelevant or unsafe requests before they reach the model—and, if needed, catching inappropriate responses after the model has generated them.

We'll explore how these features work in LangGraph, using our travel information assistant as the running example. You'll see how to persist conversation state using checkpoints, enforce scope restrictions at both the router and agent level, and extend the design to include post-model checks and human-in-the-loop review. By the end of the chapter, you'll have the tools to build assistants that are not only smart, but also safe, focused, and resilient.

## 14.1 Memory

When designing AI agent-based systems—especially those exposed via chatbots—one of the most important steps towards production readiness is adding memory. Memory allows the system to maintain context between user interactions, enabling stateful workflows and natural conversations.

Without memory, each interaction between the user and the LLM starts fresh, with no knowledge of what was said before. For simple Q&A this might be acceptable, but for anything that requires follow-up, refinement, or reference to past turns, it quickly becomes frustrating.

LangGraph provides a powerful and flexible mechanism for implementing memory: checkpoints. In this section we will explore short-term memory using checkpoints, and see how to integrate them into our existing `travel_assistant` agent.

### 14.1.1 Types of Memory

In AI agents, memory can exist at different scopes:

- *Short-term memory* — Context retained during a single session between a user and the LLM. This is ideal for ongoing conversations until the user achieves their goal. Typically stored in-memory or in a session-scoped store.
- *Long-term user memory* — Persistent across multiple sessions for the same user, enabling the system to remember preferences or past activities.
- *Long-term application-level memory* — Persistent across all users and sessions, storing general knowledge useful for everyone (e.g., current exchange rates).

Long-term user and application memory tend to be highly system-specific, so in this section we will focus exclusively on short-term conversational memory.

### 14.1.2 Why Short-Term Memory is Needed



In a conversational application that uses the tool-calling protocol (as described in earlier chapters), a typical interaction works like this:

1. The user sends a message.
2. The LLM may issue tool calls.
3. The application executes these tools and returns results to the LLM.
4. The LLM synthesises a final answer.

If the user follows up with a clarifying or related question, a stateless system would lose all prior context, forcing the conversation to restart. This is both inefficient and unnatural.

The solution is to store the entire conversation history after each interaction and feed it back to the LLM on subsequent turns. This makes the system stateful and allows the model to resolve references like “same town” or “that hotel” based on prior turns.

### 14.1.3 Checkpoints in LangGraph

Although you could implement short-term memory simply by storing the list of exchanged messages after each LLM response, LangGraph provides a more powerful and generalized mechanism: the checkpoint.

A checkpoint is a snapshot of the graph’s execution state taken at a specific moment in the flow.

In practice, LangGraph takes these snapshots at each node in the graph—starting from the entry point (also called the START node) and continuing until the exit point (END node).

This approach is more flexible than storing messages alone, because it preserves all aspects of the graph state, not just the conversation text.

That flexibility enables a range of use cases beyond chat memory:

- *State rehydration after failure* — If an execution fails partway through, you can resume from the last successful checkpoint without re-running the entire process. This is especially valuable when some steps are

expensive or slow.

- *Human-in-the-loop workflows* — You can pause at a checkpoint to collect manual input or approval, then resume exactly where you left off using the stored state.
- *Multi-turn conversational context* — For chatbots, checkpoints ensure the conversation history is preserved across turns without having to reconstruct it manually.

In this chapter we will focus solely on the last use case—maintaining conversational state for the entire duration of a user–LLM session—while keeping in mind that the same mechanism is equally capable of powering the others.

## What is a Checkpoint?

A checkpoint represents the saved state of the graph at a given super-step, where:

- *A super-step* corresponds to the execution of a single graph node.
- *Parallel nodes* processed within the same step are grouped together.

By saving a checkpoint at each super-step, we can:

- Reuse the current execution state in a follow-up question.
- Replay the persisted execution up to a specific point—useful when recovering from errors or resuming after manual intervention.

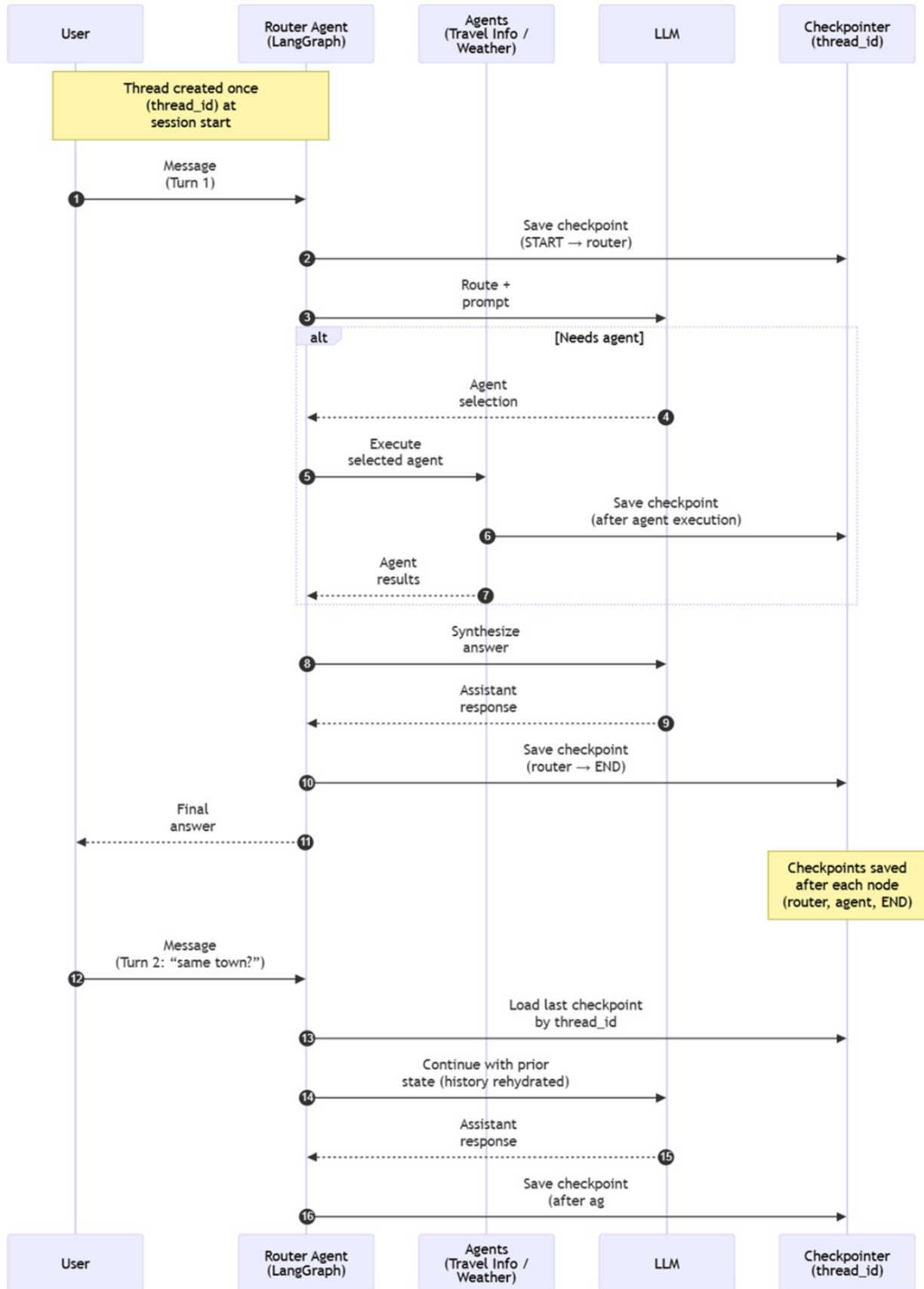
## How Checkpoints Work in LangGraph

LangGraph’s checkpointing system is built around two main concepts:

1. *Checkpoint* — The component responsible for capturing and storing state snapshots. After each super-step (i.e., completion of a node), the checkpoint records the current graph state, as illustrated in figure 14.1. This snapshot can include conversation history, tool outputs, intermediate variables, and execution metadata—ensuring the workflow can be resumed or inspected at any point.

2. *Configuration* — Defines the session that checkpoints belong to. In LangGraph, a session is called a thread.
  - a. Each thread is identified by a thread ID (a unique value, typically a UUID).
  - b. A single user may have multiple threads (sessions) active or saved at different times.
  - c. The configuration links the checkpoint to a specific thread so the system knows which session state to load later.

**Figure 14.1** Sequence diagram of the travel assistant with checkpoints saved after each node execution, allowing state restoration across turns using a shared `thread_id`.



When you pass a thread ID to the graph on subsequent invocations, LangGraph retrieves the stored state from the last checkpoint in that thread and resumes execution from there—or, in a conversational context, uses it to provide the LLM with the correct history.

#### 14.1.4 Adding Short-Term Memory to Our Travel Assistant

To demonstrate how LangGraph’s persistence and checkpointing work, we’ll enhance our router-based `travel_assistant` from section 12.2 with short-term memory.

For clarity, we’ll walk through the changes step-by-step.

Start by making a copy of your existing `main_05_01.py` and name it `main_08_01.py`.

We’ll add persistence features to this copy so you can compare with the original.

##### Step 1 — The Original Stateless Chat Loop

First, let’s revisit the existing chat loop. In this version, each time the user interacts, we only pass the new message as the state to `travel_assistant`. There is no concept of session or continuity—each turn is treated as completely separate.

##### Listing 14.1 Original chat loop from router based agent solution

```
def chat_loop(): #A
 print("UK Travel Assistant (type 'exit' to quit)")
 while True:
 user_input = input("You: ").strip() #B
 if user_input.lower() in {"exit", "quit"}: #C
 break
 state = {"messages": [HumanMessage(content=user_input)]}
 result = travel_assistant.invoke(state) #E
 response_msg = result["messages"][-1] #F
 print(f"Assistant: {response_msg.content}\n") #G
```

In this approach, `travel_assistant.invoke()` receives only the state—a fresh message each time:

```
state = {"messages": [HumanMessage(content=user_input)]}
result = travel_assistant.invoke(state)
```

## Step 2 — Introducing a Thread ID

To persist state across turns, we need a way to uniquely identify the conversation.

LangGraph does this using a thread ID, passed in a `RunnableConfig`.

We can generate one at the start of the chat loop:

```
import uuid
thread_id = uuid.uuid1()
config = {"configurable": {"thread_id": thread_id}}
```

Listing 14.2 shows the revised chat loop implementation, where the thread ID is created once, printed, and passed with every `invoke()` call

### Listing 14.2 Revised chat loop implementation with `thread_id`

```
def chat_loop(): #A
 thread_id=uuid.uuid1() #B
 print(f'Thread ID: {thread_id}')
 config={"configurable": {"thread_id": thread_id}} #B

 print("UK Travel Assistant (type 'exit' to quit)")
 while True:
 user_input = input("You: ").strip() #C
 if user_input.lower() in {"exit", "quit"}: #D
 break
 state = {"messages": [HumanMessage(content=user_input)]}
 result = travel_assistant.invoke(state, config=config) #F
 response_msg = result["messages"][-1] #G
 print(f"Assistant: {response_msg.content}\n") #H
```

Now, instead of just passing state, we pass two arguments:

```
result = travel_assistant.invoke(state, config)
```

The config ensures that all checkpoints and state belong to this specific session (`thread_id`).

### Step 3 — Adding a Checkpointer

With the thread ID in place, we can add an in-memory checkpointer to store state snapshots.

```
from langgraph.checkpoint.memory import InMemorySaver
checkpointer = InMemorySaver()
```

#### Note

`InMemorySaver` is ideal for development, testing, and quick proof-of-concepts. In a production environment, you should use a persistent storage backend to ensure state survives restarts and can be shared across multiple application instances. LangGraph provides built-in options such as a `SqliteSaver` (from the `langgraph-checkpoint-sqlite` package, backed by a SQLite database) and a `PostgresSaver` (from the `langgraph-checkpoint-postgres` package, backed by PostgreSQL). It also provides the async versions, `SqliteSaverAsync` and `PostgresSaverAsync` (from the same packages). For production deployments, the PostgreSQL-based checkpointer is generally recommended for its scalability, reliability, and concurrency support.

When compiling the graph, pass the checkpointer, as shown in Listing 14.3.

#### Listing 14.3 Graph enriched with checkpointer

```
graph = StateGraph(AgentState) #A
graph.add_node("router_agent", router_agent_node) #B
graph.add_node("travel_info_agent", travel_info_agent) #C
graph.add_node("accommodation_booking_agent", accommodation_booki

graph.add_edge("travel_info_agent", END) #E
graph.add_edge("accommodation_booking_agent", END) #F

graph.set_entry_point("router_agent") #G

checkpointer = InMemorySaver() #H
```

```
travel_assistant = graph.compile(checkpointer=checkpointer) #I
```

This enables the graph to save a snapshot after every node execution, so it can resume with full context on the next turn.

## Step 4 — Configuring the LLM for Conversation Continuity

Finally, we need to configure the LLM to reference previous turns without resending the entire conversation history every time.

OpenAI's Responses API allows this using the `use_previous_response_id` flag.

```
llm_model = ChatOpenAI(model="gpt-5", #A
 use_responses_api=True, #B
 use_previous_response_id=True) #C
```

With `use_previous_response_id=True`, the LangChain ChatOpenAI wrapper sends only the ID of the previous response, and OpenAI rehydrates the full history internally.

### Important

If you enable `use_responses_api=True` but not `use_previous_response_id=True`, LangChain will try to resend the full history on every turn. The Responses API treats this as a duplicate submission and returns an error. When using LangGraph memory with the Responses API, enabling `use_previous_response_id` is mandatory.

## 14.1.5 Executing the Checkpointer-Enabled Assistant

With the checkpointer integrated into our `travel_assistant`, we can now see short-term conversational memory in action.

Run the updated router-based assistant and enter a typical question:

```
Thread ID: e683b337-752b-11f0-84a9-34f39a8d3195
```

```
You: What's the weather like in Penzance?
```

```
Assistant: [{'type': 'text', 'text': 'The weather in Penzance is
```



Now, enter a follow-up question that refers to the previous turn:

You: What's the weather in the same town now?

Assistant: `[{'type': 'text', 'text': 'Current weather in Penzance`

Notice how the assistant correctly resolves “same town” to Penzance—it was able to do this because the LangGraph checkpointer supplied the LLM with the entire conversation history, not just the latest user input.

Congratulations—you’ve just implemented a stateful conversational chatbot!

Although this is satisfying, it’s worth taking a deeper look at what’s happening under the bonnet so you can understand exactly how the checkpointer maintains short-term memory.

### 14.1.6 Rewinding the State to a Past Checkpoint

To better understand how LangGraph manages conversational memory, we’ll simulate what happens internally when restoring from a checkpoint. This exercise will:

1. Ask the chatbot a question.
2. Retrieve the last checkpoint from the checkpointer.
3. Rehydrate the graph state to that checkpoint.
4. Ask a follow-up question that depends on that restored context.

This is effectively what LangGraph does automatically when you pass the same `thread_id` on subsequent turns.

#### Step 1 — Updating the Chat Loop for State Inspection

Create a copy of `main_08_01.py` and name it `main_08_02.py`.

Replace the `chat_loop()` with the implementation shown in listing 14.4.

##### Listing 14.4 Step-by-step state inspection

```
def chat_loop():
```

```

thread_id=uuid.uuid1() #A
print(f'Thread ID: {thread_id}')
config={"configurable": {"thread_id": thread_id}} #B

user_input = input("You: ").strip() #C

question = {"messages": [HumanMessage(content=user_input)]} #
result = travel_assistant.invoke(question, config=config) #E
response_msg = result["messages"][-1] #F
print(f"Assistant: {response_msg.content}\n") #G

state_history = travel_assistant.get_state_history(config) #H
state_history_list = list(state_history) #I
print(f'State history: {state_history_list}') #I

last_snapshot = list(state_history_list)[0] #J
print(f'Last snapshot: {last_snapshot.config}')

thread_id = last_snapshot.config["configurable"]["thread_id"]
last_checkpoint_id = last_snapshot.config["configurable"]["ch

new_config = {"configurable": #M
 {"thread_id": thread_id,
 "checkpoint_id": last_checkpoint_id}}

retrieved_snapshot = travel_assistant.get_state(new_config) #
print(f'Retrieved snapshot: {retrieved_snapshot}') #O

travel_assistant.invoke(None, config=new_config) #P

new_question = {"messages": [HumanMessage(content="What is th
result = travel_assistant.invoke(new_question, config=new_con
response_msg = result["messages"][-1] #R

print(f"Assistant: {response_msg.content}\n") #S

```

## Step 2 — Running the Example and Viewing State History

Run `main_08_02.py` in debug mode and enter the usual question:

You: What's the weather like in Penzance?

Assistant: [{ 'type': 'text', 'text': 'It's currently foggy in Pen

Retrieve the state history:

```
state_history = travel_assistant.get_state_history(config)
state_history_list = list(state_history)
print(f'State history: {state_history_list}')
```

You'll see multiple `StateSnapshot` entries, each representing a checkpoint, starting from the most recent and moving backwards to the initial `START` node. Each snapshot contains:

- The messages exchanged so far.
- Tool call information (if any).
- Metadata about the graph execution step.

### Step 3 — Rehydrating from a Specific Checkpoint

From the most recent snapshot:

```
last_snapshot = list(state_history_list)[0]
print(f'Last snapshot: {last_snapshot.config}')
```

Extract the `thread_id` and `checkpoint_id`:

```
thread_id = last_snapshot.config["configurable"]["thread_id"]
last_checkpoint_id = last_snapshot.config["configurable"]["checkp
```

Build a new config pointing to that checkpoint:

```
new_config = {"configurable":
 {"thread_id": thread_id,
 "checkpoint_id": last_checkpoint_id}}
```

Retrieve the state at this checkpoint to confirm it matches expectations:

```
retrieved_snapshot = travel_assistant.get_state(new_config)
print(f'Retrieved snapshot: {retrieved_snapshot}')
```

You should see the full conversation history up to that checkpoint, including user messages, tool outputs, and assistant responses.

### Step 4 — Resuming from the Restored State

To rewind the graph to that point:

```
travel_assistant.invoke(None, config=new_config)
```

Now ask a follow-up that depends on that past context:

```
new_question = {"messages": [HumanMessage(content="What is the we
result = travel_assistant.invoke(new_question, config=new_config)
response_msg = result["messages"][-1]
```

You might see output like:

```
Assistant: [{'type': 'text', 'text': 'In Penzance it's currently
```

The assistant correctly infers “same town” as Penzance, confirming that the rehydrated state was passed back to the LLM.

By stepping through this manual rewind, you now have a low-level understanding of how LangGraph’s checkpointers powers short-term conversational memory—storing the entire execution context at each node, and restoring it later to continue exactly where the conversation left off.

## 14.2 Guardrails

Guardrails are application-level mechanisms that keep an AI agent operating within a defined scope, policy framework, and intended purpose. They serve as the “rules of the road” for agent behavior, inspecting and validating both inputs and outputs at critical points to ensure the system stays safe, relevant, compliant, and efficient. Without guardrails, an agent might drift into unrelated subject matter, produce unsafe or non-compliant responses, or waste processing resources on unnecessary actions.

A strong guardrail design can stop a travel assistant from giving stock market tips, prevent a support bot from revealing confidential information, or block poorly formed requests before they reach an expensive language model. In practice, guardrails often fall into three broad categories:

- *Rule-based* — explicit conditions or regular expressions that catch prohibited patterns or topics.

- *Retrieval-based* — checks against approved data sources to confirm that a request is relevant and in scope.
- *Model-based* — compact classification or moderation models that assess intent, safety, or adherence to policy.

These controls can be introduced at multiple points in the agent workflow:

- *Pre-model checks* — reject invalid or irrelevant queries before the LLM runs.
- *Post-model checks* — verify generated responses against policy or safety guidelines.
- *Routing-stage checks* — decide whether a query should trigger certain tools or branches.
- *Tool-level checks* — block unsafe or unauthorized tool actions.

Functionally, guardrails act like validation layers—if a check fails, the agent’s normal flow is adjusted, whether by refusing the request, asking for clarification, or redirecting to a safer response path.

In this section, we’ll integrate custom guardrails into the router node of the travel information assistant we built earlier (now enhanced with memory) so it can immediately reject irrelevant or out-of-scope requests—such as non-travel queries. We’ll also explore LangGraph’s pre-model hook, which lets us enforce guardrails before any LLM call, ensuring that both the travel and weather agents remain within coverage boundaries—for example, only handling destinations in Cornwall.

### **14.2.1 Implementing Guardrails to Reject Non-Travel-Related Questions**

The first and most obvious guardrail for our UK travel information assistant is a domain relevance check—a pre-filter that screens the user’s question before any agent reasoning occurs. If the question falls outside the assistant’s remit, we intercept it early and politely refuse to answer. This prevents the system from attempting to handle queries about unrelated topics such as sports results, financial markets, or celebrity gossip.

Introducing this guardrail delivers two key benefits:

1. *Improved Accuracy* — Our agents are trained and configured only for travel and weather information. If we attempt to answer unrelated questions, the results will almost certainly contain inaccuracies or hallucinations. By rejecting irrelevant queries outright, we keep the conversation aligned with the agent’s actual capabilities.
2. *Cost Control* — Without this filter, some users might deliberately use our assistant as a free gateway to an expensive LLM, bypassing subscription costs for unrelated questions. By blocking non-travel topics early, we prevent this form of resource abuse and avoid unnecessary processing costs.

## Defining the Guardrail Policy

Our first task is to clearly define what qualifies as in-scope for this assistant. This ensures the guardrail has unambiguous decision criteria.

To begin, make a copy of the `main_08_01.py` script from the previous section and save it as `main_09_01.py`. Then, implement the guardrail policy as shown in Listing 14.5.

**Listing 14.5** Guardrail policy to restrict questions to travel-related topics

```
class GuardrailDecision(BaseModel): #A
 is_travel: bool = Field(
 ...,
 description=(
 "True if the user question is about travel informatio
 "lodging (hotels/BnBs), prices, availability, or weat
),
)
 reason: str = Field(..., description="Brief justification for

GUARDRAIL_SYSTEM_PROMPT = (#B
 "You are a strict classifier. Given the user's last message,
 "travel-related. Travel-related queries include destinations,
 "room availability, prices, or weather in Cornwall/England."
)

REFUSAL_INSTRUCTION = (#C
```

```

 "You can only help with travel-related questions (destination
 "availability, or weather in Cornwall/England). The user's re
 "Politely refuse and briefly explain what topics you can help
)
 llm_guardrail = llm_model.with_structured_output(GuardrailDecisio

```

This snippet defines a guardrail policy that uses a lightweight LLM classifier to determine whether a user’s question is travel-related.

`GuardrailDecision` is a Pydantic model that structures the classification output. The `is_travel` flag indicates whether the request falls within the travel domain, and `reason` provides a brief justification for the decision.

`GUARDRAIL_SYSTEM_PROMPT` instructs the model to classify strictly, giving a precise definition of “travel-related” for our purposes.

`REFUSAL_INSTRUCTION` contains a fixed, polite message to explain to the user why their question can’t be answered.

`llm_guardrail` wraps the base LLM with structured output formatting, enabling fast, consistent decision-making before the main routing logic runs.

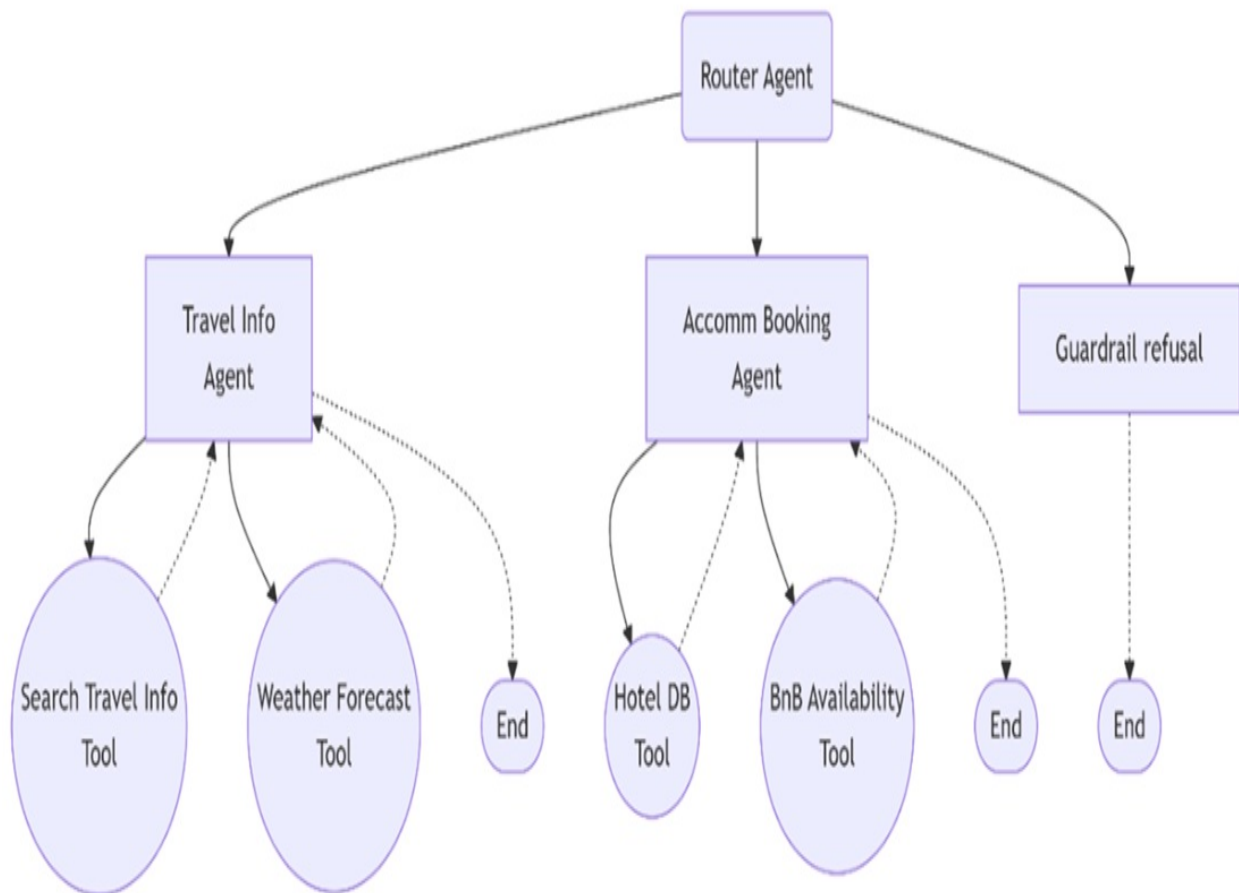
## Updating the Router Graph

At a high level, we want irrelevant queries to exit the workflow immediately—without touching any of the downstream agents. That means introducing a dedicated `guardrail_refusal` node in our `LangGraph` routing structure. This node simply redirects to the graph’s `END` without performing any work.

The updated workflow is shown in Figure 14.2.

**Figure 14.2 Updated travel assistant workflow: A guardrail checks if the user’s question is in scope. Irrelevant queries are routed to `guardrail_refusal`, which sends them directly to `END`, bypassing all other agents.**

### Router-based multi-agent Travel Assistant with Guardrail



The router agent now checks whether a question is in scope. If it is, the query is routed to either the travel information agent or the weather agent as before. If it isn't, the router sends it to the new `guardrail_refusal` node, which is linked directly to the `END` node.

The corresponding graph definition is shown in Listing 14.6.



#### Listing 14.6 Adding a `guardrail_refusal` node to the router graph

```
def guardrail_refusal_node(state: AgentState): #A
 return {}

graph = StateGraph(AgentState)
graph.add_node("router_agent", router_agent_node)
graph.add_node("travel_info_agent", travel_info_agent)
graph.add_node("accommodation_booking_agent", accommodation_booking_agent)
graph.add_node("guardrail_refusal", guardrail_refusal_node) #B

graph.add_edge("travel_info_agent", END)
graph.add_edge("accommodation_booking_agent", END)
graph.add_edge("guardrail_refusal", END) #C

graph.set_entry_point("router_agent")

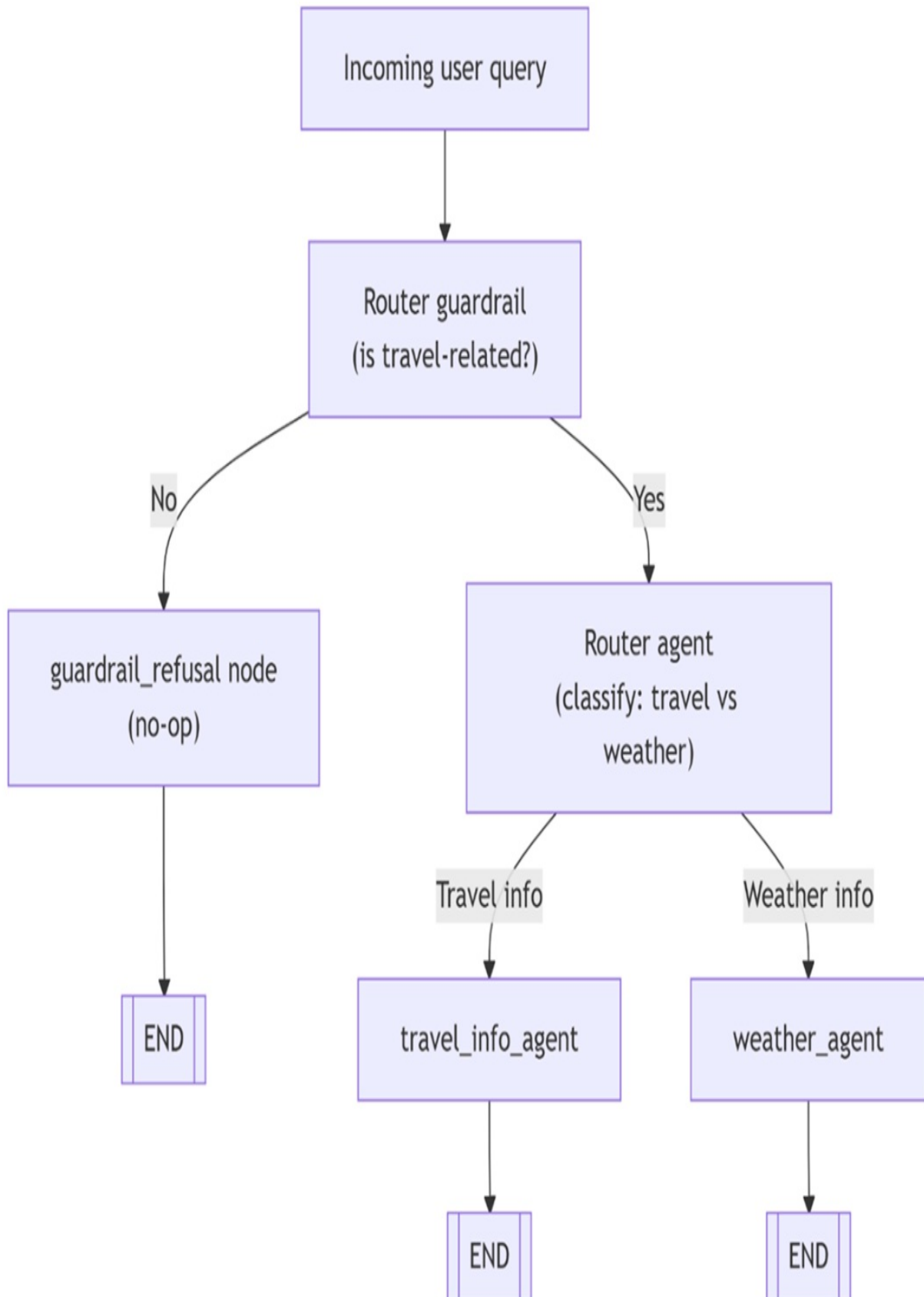
checkpointer = InMemorySaver()
travel_assistant = graph.compile(checkpointer=checkpointer)
```

As you can see, the `guardrail_refusal` node is intentionally a no-op—its only role is to create a clean shortcut to `END`.

### Updating the Router Agent

With the guardrail policy defined and the graph updated, the final step is to implement the logic that enforces it. Previously, our router agent only decided whether to send a query to the travel or weather agent. Now it must first run the guardrail check, as illustrated in the diagram of figure 14.3.

**Figure 14.3** Flowchart of updated router logic with guardrail check: queries first pass a relevance filter, and irrelevant ones are sent with a refusal message directly to the `guardrail_refusal` node.



If the guardrail check fails—meaning the question is judged irrelevant to the travel assistant—the router generates a refusal message and routes execution directly to the `guardrail_refusal` node, as shown in Figure 14.3.

This logic is implemented in Listing 14.7.

**Listing 14.7 Router agent with guardrail enforcement**

```
def router_agent_node(state: AgentState) -> Command[AgentType]:
 """Router node: decides which agent should handle the user qu
 messages = state["messages"]
 last_msg = messages[-1] if messages else None
 if isinstance(last_msg, HumanMessage):
 user_input = last_msg.content

 classifier_messages = [
 SystemMessage(content=GUARDRAIL_SYSTEM_PROMPT), #A
 HumanMessage(content=user_input),
]
 decision = llm_guardrail.invoke(classifier_messages) #B
 if not decision.is_travel: #C
 refusal_text = (#D
 "Sorry, I can only help with travel-related quest
 "lodging, prices, availability, or weather in Cor
 "Please rephrase your request to be travel-relate
)
 return Command(#E
 update={"messages": [AIMessage(content=refusal_te
 goto="guardrail_refusal",
)

 router_messages = [
 SystemMessage(content=ROUTER_SYSTEM_PROMPT),
 HumanMessage(content=user_input)
]
 router_response = llm_router.invoke(router_messages)
 agent_name = router_response.agent.value
 return Command(update=state, goto=agent_name)

 return Command(update=state, goto=AgentType.travel_info_agent
```

In this revised logic:

- The router first invokes `llm_guardrail` with the latest user query.

- If the classification result indicates the question is not travel-related, the router constructs a fixed refusal message, stores it in the state, and sends execution to the `guardrail_refusal` node—bypassing all normal routing.
- If the check passes, the router continues with its usual process of selecting the most appropriate agent.

## Testing the Guardrail

To see it in action, run `main_09_01.py` in debug mode and place a breakpoint on the line where `llm_guardrail` is invoked. Then enter the following query:

```
UK Travel Assistant (type 'exit' to quit)
You: Can you give me the latest results of Inter Milan?
```

When execution stops at the breakpoint, inspect `decision.is_travel`. You should see `False`, since football scores are outside the allowed travel domain. Resuming execution (F5) will produce:

```
Assistant: Sorry, I can only help with travel-related questions (
Congratulations—you've successfully implemented your first guardrail!
```

However, our work isn't done. Remember, one of our agents only has coverage for Cornwall, not the whole of the UK. This means we'll want to implement further scope restrictions at the agent level, which we'll tackle in the next section.

### 14.2.2 Implementing More Restrictive Guardrails at Agent Level

In traditional software development, it's considered best practice for each class or component to validate its own data rather than relying solely on validations at higher levels such as the UI. The same principle applies to agent-based systems: each agent should enforce its own input guardrails, even if broader checks are already in place at the chatbot entry point.

These agent-level guardrails are often more restrictive than system-wide ones, because they can account for the specific capabilities and data scope of

the individual agent.

In our case:

- The travel information agent can only handle queries about Cornwall, because its vector store contains data exclusively from that region.
- The accommodation booking agent will also be limited to Cornwall for now, to keep the assistant's scope consistent.

Why have two levels of guardrails?

- *Router-level guardrail* — Acts as an early fail-fast filter before any agent logic or tool invocation.
- *Agent-level guardrails* — Provide a “belt-and-suspenders” safeguard to catch out-of-scope requests if the agent is ever called directly or reused in a different context.

## Defining the Cornwall-Restricted Guardrail Policy

We start by explicitly defining the policy: only travel-related questions about Cornwall are permitted. This ensures that both travel and accommodation booking agents reject queries for other regions or countries.

Make a copy of `main_09_01.py` and rename it `main_09_02.py`. Then add the following system prompts to define the classification and refusal behavior.

### Listing 14.8 System prompts for classification and refusal behavior

```
AGENT_GUARDRAIL_SYSTEM_PROMPT = (
 "You are a strict classifier. Given the user's last message,
 "travel-related. Travel-related queries include destinations,
 "room availability, prices, or weather in Cornwall/England."
 "Only accept travel-related questions covering Cornwall (Engl
 "from other areas in England and from other countries"
)

AGENT_REFUSAL_INSTRUCTION = (
 "You can only help with travel-related questions (destination
 "availability, or weather in Cornwall/England). The user's re
 "Or it might be a travel related question but not focusing on
```

```
) "Politely refuse and briefly explain what topics you can help
```

## Creating the Agent-Level Guardrail Function

The agent guardrail is implemented as a Python function that takes the current graph state and returns either an unchanged state (for valid input) or one modified to instruct the LLM to issue a refusal.

### Listing 14.9 Agent-level guardrail function

```
def pre_model_guardrail(state: dict):
 messages = state.get("messages", [])
 last_msg = messages[-1] if messages else None
 if not isinstance(last_msg, HumanMessage): #A
 return {}

 user_input = last_msg.content
 classifier_messages = [#B
 SystemMessage(content=AGENT_GUARDRAIL_SYSTEM_PROMPT),
 HumanMessage(content=user_input),
]
 decision = llm_guardrail.invoke(classifier_messages)

 if decision.is_travel: #C
 # Allow normal flow; do not modify inputs
 return {}

 return {"llm_input_messages": [SystemMessage(content=AGEN
```

The `pre_model_guardrail()` function works as a pre-processing filter before the LLM sees the user's query:

- It verifies that the latest message is indeed from the user.
- It sends the query, along with a strict classification system prompt, to the guardrail LLM.
- If the query is in-scope (travel-related and Cornwall-specific), it passes through unchanged.
- Otherwise, the function prepends a refusal instruction so the agent politely declines the request.

## Injecting the Guardrail into the Agents

LangGraph's ReAct agents support pre-model hooks (`pre_model_hook`) and post-model hooks (`post_model_hook`), allowing you to intercept and manipulate inputs or outputs. While these hooks can be used for tasks like summarizing long inputs or sanitizing outputs, here we'll focus solely on input-side guardrails.

To enable the Cornwall restriction, we simply pass `pre_model_guardrail` to both the travel information agent and the accommodation booking agent.

### Listing 14.10 Travel information agent with Cornwall guardrail

```
travel_info_agent = create_react_agent(
 model=llm_model,
 tools=TOOLS,
 state_schema=AgentState,
 prompt="You are a helpful assistant that can search travel in
 pre_model_hook=pre_model_guardrail, #A
)
```

### Listing 14.11 Accommodation booking agent with Cornwall guardrail

```
accommodation_booking_agent = create_react_agent(#A
 model=llm_model,
 tools=BOOKING_TOOLS,
 state_schema=AgentState,
 prompt="You are a helpful assistant that can check hotel and
 pre_model_hook=pre_model_guardrail,
)
```

## Testing the Cornwall Guardrail

Run `main_09_02.py` in debug mode, placing a breakpoint on the `llm_guardrail` invocation inside `pre_model_guardrail()`. Then try:

```
UK Travel Assistant (type 'exit' to quit)
You: Can you give me some travel tips for Liverpool (UK)?
```

When paused at the breakpoint, inspect `decision.is_travel`—it should be `False` because the query is not Cornwall-specific. Execution will then prepend the

refusal instruction, resulting in output like:

Assistant: [{"type": "text", "text": "Sorry—I can only help with

With this, we now have two layers of defense:

1. A router-level guardrail that quickly rejects any non-travel queries.
2. Agent-level guardrails that enforce Cornwall-specific scope for travel and accommodation requests.

Our entire agentic workflow is now protected from both irrelevant and out-of-coverage queries, making the system safer, more reliable, and more cost-efficient.

## 14.3 Beyond this chapter

By this point, you’ve seen how to equip AI agents with memory and guardrails—two of the most critical building blocks for making them production-ready. But depending on your application’s domain, scale, and compliance requirements, there are additional considerations you may need to address before deploying to real users.

Some of these areas have been mentioned in passing throughout the book but not explored in depth, either because they are highly domain-specific or because they deserve their own dedicated treatment. Below are a few directions worth exploring further.

### 14.3.1 Long-term user and application memory

In this chapter, we focused on short- to medium-term memory—keeping track of relevant state within a single conversation or across a limited session history.

However, production agents often benefit from persistent, long-term memory that stores user preferences, past interactions, and contextual information across weeks, months, or even years. This might involve:

- Dedicated vector stores for each user.



- Periodic summarization and pruning to keep memory manageable.
- Privacy and compliance controls for personally identifiable information (PII).
- Long-term memory can dramatically improve personalization, but it also brings engineering, scaling, and regulatory challenges you’ll need to plan for.

Table 14.1 summarizes the various types of memory your AI agent-based system might need to accommodate user needs.

**Table 14.1 Type of memory in AI agents**

<u>Memory type</u>	<u>Scope</u>	<u>Persistence</u>	<u>Example (Travel Assistant)</u>	<u>Challenges</u>
Short-term	Single user session	Until session ends	Remembering “same town” in a weather follow-up within one conversation	Limited continuity; lost when session closes
Long-term (user)	Across multiple sessions for one user	Weeks, months, or years	Remembering a user’s preferred Cornwall destinations or accommodation types	Privacy compliance
Long-term (application)	Across all users and sessions	Ongoing	Storing general travel updates (e.g., Cornwall event calendar, seasonal attraction schedules)	Keeping data fresh; avoiding outdated or incorrect info

### 14.3.2 Human-in-the-loop (HITL)

Even a well-designed travel information assistant will encounter situations where automated handling is not enough—cases where the query is ambiguous, the available data is incomplete, or a decision could have a significant real-world impact. A human-in-the-loop approach allows such requests to be escalated to a human travel expert for review before a response is sent.

For our Cornwall-focused travel assistant, common HITL scenarios could include:

- Requests for personalized itineraries involving unusual combinations of activities where safety, feasibility, or timing is uncertain.
- Queries about real-time disruptions—such as severe weather, transport strikes, or event cancellations—where up-to-date human judgment is needed.
- Special accommodation or accessibility requests that require confirming details with local providers.

In early production deployments, HITL is especially valuable: it helps ensure accuracy, prevents reputational damage, and provides real-world feedback to refine the assistant’s automated policies. Over time, insights from human reviews can be incorporated into improved guardrails, better prompts, or expanded knowledge bases—reducing the number of cases that need escalation.

### **14.3.3 Post-model guardrails**

In this chapter, we focused on guardrails that run before invoking the LLM—screening queries for relevance and redirecting out-of-scope requests. In a production travel information assistant, you may also want post-model guardrails that inspect the model’s output before it is shown to the user or sent to a downstream service (such as a booking API).

For our Cornwall-focused assistant, post-model guardrails could include:

- Filtering outdated or incorrect details—for example, removing references to attractions that have permanently closed or events that have already passed.

- Redacting sensitive information—such as private contact numbers for small BnBs that should only be shared after confirmed bookings.
- Enforcing brand tone and style—ensuring that all travel advice is given in a warm, welcoming, and concise manner consistent with the assistant’s persona.
- Verifying structured output—making sure any booking recommendations, price quotes, or itineraries follow the correct format expected by other systems.

Post-model guardrails act as a final safety net, catching cases where the LLM’s answer might look reasonable but contains factual errors, tone mismatches, or details that are inappropriate for immediate user delivery.

### **14.3.4 Evaluation of AI agents and applications**

One of the most overlooked—but absolutely essential—steps in preparing an agent for production is systematic evaluation. This is a broad and evolving discipline, complex enough to deserve an entire book of its own, but it is critical for ensuring that your travel assistant remains accurate, safe, and efficient once deployed.

For our Cornwall-focused travel information assistant, evaluation might include:

- Functional testing – Verifying that the assistant provides correct, relevant, and complete answers across a wide set of test queries, such as “Best family-friendly beaches in Cornwall” or “Current weather forecast for St Ives.” This ensures it stays within scope and retrieves accurate, up-to-date information.
- Behavioral testing – Confirming that the assistant follows policy and safety rules, avoids giving irrelevant or unsafe travel advice, and maintains a consistent, friendly tone suitable for tourism and customer service.
- Performance testing – Measuring latency and API costs under realistic user loads, such as peak summer tourist season when requests might surge.
- Regression testing – Ensuring the assistant’s reliability is not

compromised when prompts are refined, tools are updated, or the underlying LLM is replaced.

While this book does not cover evaluation frameworks and methodologies in detail, you should treat evaluation as a core part of your pre-production checklist and as an ongoing process after launch. Continuous evaluation helps you catch issues before users do, adapt to changes in local events or services, and maintain the trustworthiness of your travel assistant over time.

Equipping your agents with memory and guardrails is an important milestone, but it's only part of the journey. True production readiness requires a holistic approach that includes safety, compliance, reliability, and continuous evaluation. The good news is that the foundations you've built in this chapter will make it much easier to layer in these additional capabilities as your applications grow in scope and complexity.

### **14.3.5 Deployment on LangGraph Platform and Open Agent Platform (OAP)**

The final step in preparing agents for production is deployment. How you deploy depends heavily on your organization's infrastructure strategy—whether applications run on-premises or in the cloud, and on IT policies around privacy, security, and compliance. It also reflects organizational choices: some teams favor local DevOps and SRE-driven deployments, while others rely on SaaS-based hosting for simplicity and scalability.

Once you've developed your LangGraph-based multi-agent system, a natural path is to deploy it on the *LangGraph Platform*. This is LangChain's fully managed hosting solution for agentic applications. It provides built-in features such as horizontal scalability, persistent state management, and end-to-end monitoring through the familiar LangSmith dashboard. The platform abstracts away much of the operational overhead, letting you move from prototype to production with minimal friction while still retaining observability and fine-grained debugging tools.

Another powerful option is to integrate your agents into the *Agent Open Platform (OAP)*. OAP is a flexible runtime and orchestration layer for AI

agents that comes with a set of prebuilt agent patterns—including the multi-tool agent and the supervisor agent. These can be customized and extended to fit enterprise use cases, whether by plugging into MCP servers, local vector stores, or other enterprise data sources. OAP is designed to act as a bridge between bespoke LangGraph agents and a broader ecosystem of composable, interoperable agents, making it especially valuable for organizations planning to run multiple agents in coordination.

Both LangGraph Platform and OAP are available as fully managed SaaS offerings, but can also be deployed into a client’s own cloud environment for teams that need to maintain tighter control over data residency and compliance. This dual deployment model means you can start quickly with managed hosting and later migrate to a private setup if regulatory or operational needs demand it.

Together, these deployment paths provide a smooth continuum—from development on your laptop, to scalable production hosting, to enterprise-wide agent orchestration—allowing you to choose the right trade-off between control, convenience, and operational complexity.

## 14.4 Summary

- Memory enables natural conversation — Short-term conversational memory preserves context across turns, allowing the assistant to handle follow-up questions without repeating information.
- Checkpoints store full execution state — LangGraph saves snapshots after each node, enabling both conversation continuity and recovery from failures.
- Guardrails maintain scope and safety — They filter or redirect requests that fall outside the assistant’s domain or policy guidelines.
- Multiple guardrail layers are valuable — Use router-level checks for fast rejection and agent-level checks for specific data or domain constraints.
- Post-model guardrails act as a final safety net — They verify output correctness, tone, and format before the user sees it or a downstream system processes it.
- Human-in-the-loop improves reliability — Escalating certain cases to human review can prevent costly mistakes and provide training data for

future automation.

- Evaluation is ongoing, not one-off — Regular functional, behavioral, performance, and regression testing ensures the assistant remains trustworthy over time.
- Production readiness is holistic — Combining memory, guardrails, human review, and evaluation sets the foundation for safe, effective, and scalable AI agents.

# Appendix A. Trying out LangChain

In this appendix, you will get a first hands-on experience with LangChain by performing simple sentence completions and experimenting with basic prompt engineering in a Jupyter Notebook.

## A.1 Trying out LangChain in a Jupyter Notebook environment

We'll begin with a straightforward example: completing a sentence using an OpenAI model and refining its output through prompt engineering. OpenAI's models are a convenient starting point because they are accessible through a public REST API and require no local infrastructure setup.

If you prefer, you can use an open-source LLM inference engine such as Ollama. However, since many readers may not yet have run an open-source LLM locally, we will cover that setup in detail in Appendix E. For now, we will keep things simple and use the OpenAI REST API.

Before you proceed, ensure that you have Python 3.11 or later installed on your local machine and that the following prerequisites are met:

1. Already own or generate an OpenAI key.
2. Know how to set up a Python Jupyter notebook environment.

If you're unfamiliar with these tasks, check the sidebar for guidance on "Creating an OpenAI key" and refer to Appendix B for instructions on setting up a Jupyter notebook environment. Since most readers use Windows, I'll be providing instructions specifically for Windows and a Python virtual environment. If you're using Linux or Anaconda, I assume you're advanced enough to adapt these instructions to your setup. If you prefer, you can also run these examples in an online notebook environment like Google Colab, as long as you install the required packages.

### Creating an OpenAI key

Assuming you've registered with OpenAI, which is necessary to explore the ChatGPT examples discussed at the beginning of the chapter:

- Log in to your OpenAI account at <https://platform.openai.com/> and navigate to the API section.
- Access the API Keys by clicking your profile icon > Your profile > User



API key (tab).

- Create a new secret key by clicking the "Create Secret Key" button, naming it (e.g., BuildingLLMApps), and confirming.
- Safely save the key, for example in a secure local file or a password vault tool, as retrieval is not possible later.

Additionally, set a spending limit (e.g., \$10) to control costs. Configure this under Settings > Limits in the left-hand menu based on your usage preferences.

To establish the virtual environment for the code in this chapter, follow the steps below. Open a new terminal in your operating system, create a folder like `c:\Github\building-llm-applications\ch01` navigate into it, and execute:

```
C:\>cd Github\building-llm-applications\ch01
```

```
C:\Github\building-llm-applications\ch01>python -m venv env_ch01
```

```
C:\Github\building-llm-applications\ch01>.\env_ch01\Scripts\activ
(env_ch01) C:\Github\building-llm-applications\ch01>
```

#### NOTE

I am running these commands on a Windows cmd shell. If you are on a Linux computer, you might have to adapt things slightly. For example, you can activate a virtual environment with: `./env_ch01/bin/activate` or `./env_ch01/Scripts/activate`. If you are using Powershell, you should use `Activate.ps1`.

Now install the notebook, langchain (and indirectly openai) packages:

```
(env_ch01) C:\Github\building-llm-applications\ch01>pip install n
```

#### Note

Alternatively, if you have cloned the Github code from the repo associated with this book, you can install all the packages in this way:

```
(env_ch01) C:\Github\building-llm-applications\ch01>pip install -r requirements.txt
```

Once the installation is complete, start up a Jupyter notebook:

```
(env_ch01) C:\Github\building-llm-applications\ch01>jupyter noteb
```

Create a notebook with: File > New > Notebook, then rename it with File > Rename ... to 01-langchain\_examples.ipynb.

If you have cloned the Github repository and want to use the notebook directly, start it up as follows:

```
(env_ch01) C:\Github\building-llm-applications\ch01>jupyter noteb
```

### **A.1.1 Sentence completion example**

Now, you are ready to execute LangChain code in the notebook. Start by importing the LangChain library and configuring an OpenAI Language Model (LLM) instance.

In the initial cell of your notebook, start by importing the necessary libraries, by running the cell:

```
from langchain_openai import ChatOpenAI
import getpass
```

#### **NOTE**

If you are unfamiliar with Jupyter notebooks, you can run a cell by pressing Shift+Enter or clicking the play button located next to the top menu.

Now add and execute a cell to grab your OpenAI API key (just hit Enter after inserting the key):

```
OPENAI_API_KEY = getpass.getpass('Enter your OPENAI_API_KEY')
```

**Setting the OpenAI API key with an environment variable**

Another way to set the OpenAI key is by using an environment variable. For example, in Windows, you can set it in the command shell like this, before launching the notebook:

```
set OPENAI_API_KEY=your_openai_api_key
```

Then, in your Python code, retrieve it with (make sure you import os):

```
api_key = os.getenv("OPENAI_API_KEY")
```

Since this method depends on the operating system and some readers may not be familiar with it, I'll focus on the previous approach. However, if you're comfortable using environment variables, feel free to use them.

Once you have entered your OpenAI key, add and execute this cell:

```
llm = ChatOpenAI(openai_api_key=OPENAI_API_KEY,
 model_name="gpt-4o-mini")
llm.invoke("It's a hot day, I would like to go to the...")
```

You will see output similar to the example below, which represents a return message from the LLM:

```
AIMessage(content="...beach to cool off and relax in the refreshing")
```

As you can see in the content property, the completion generated by the LLM is "...beach to cool off [...]".

## **A.1.2 Prompt engineering examples**

I've mentioned prompts several times, but I haven't shown you any examples yet. A prompt is the instruction you give the LLM to complete a task and generate a response. It's a key part of any LLM application, so much so that developing LLM applications often involves spending a lot of time designing and refining prompts through trial and error. Various patterns and techniques are already forming around prompts, leading to the emergence of a discipline called prompt engineering. This field focuses on crafting prompts that yield the best possible answers. I'll dedicate the next chapter to teaching you the fundamentals of prompt engineering. For now, let's start with a

straightforward prompt.:

```
prompt_input = """Write a short message to remind users to be
vigilant about phishing attacks."""
response = llm.invoke(prompt_input)

print(response.content)
```

Output:

Just a friendly reminder to stay vigilant against phishing attack

## Prompt template

In Chapter 2 on prompt engineering, you'll learn about prompt templates. They're structured prompts that allow you to run various versions of the same theme. LangChain offers a class called `PromptTemplate` for this. Its job is to generate prompts from template structures and input parameters. Below is an example of how to create and execute a prompt from a template, which you can place in a single notebook cell:

### Listing 1.1 Creating a prompt from a `PromptTemplate`

```
from langchain_core.prompts import PromptTemplate

segovia_aqueduct_text = "The Aqueduct of Segovia (Spanish: Acuedu
prompt_template = PromptTemplate.from_template("You are an experi

prompt_input = prompt_template.format(
 text=segovia_aqueduct_text,
 num_words=20,
 tone="knowledgeable and engaging")
response = llm.invoke(prompt_input)
print(response.content)
```

When executing the code above you should get this output or something similar:

The Aqueduct of Segovia, a Roman marvel in Spain, dates back to t

In the next chapter, I'll show you how to replicate the examples we've just

implemented in LangChain using the plain OpenAI REST API, so you can see the differences.

### A.1.3 Creating chains and executing them with LCEL

One of the benefits of using LangChain is its processing technique built around the concept of a "chain." A chain is a pipeline of components put together to achieve a particular outcome. For example, to illustrate (but do not execute this code snippet), you could create a chain to scrape the latest news from a website, summarize it, and email it to someone, like this:

```
chain = web_scraping | prompt | llm_model | email_text
```

This declarative, intuitive, and readable method of defining a chain showcases the LangChain Expression Language (LCEL), which I'll cover extensively in a later chapter. For now, let's walk through an example by reimplementing the previous summarization task using LCEL.

First, set up the chain in a new notebook cell as follows:

```
prompt_template = PromptTemplate.from_template("You are an experi
llm = ChatOpenAI(openai_api_key=OPENAI_API_KEY,
 model_name="gpt-4o-mini")

chain = prompt_template | llm
```

Now, this chain is ready to accept any text, target number of words, and target tone. Execute it as shown below:

```
response = chain.invoke({"text": segovia_aqueduct_text,
 "num_words": 20,
 "tone": "knowledgeable and engaging"})
print(response.content)
```

You'll get this output, or similar:

The Aqueduct of Segovia: A Roman marvel channeling water to the c

As you can see, this way of setting up and executing the processing is somewhat simpler than the original imperative approach. The chain pipeline works because both `PromptTemplate` and `ChatOpenAI` objects implement a

common interface (`Runnable`) that allows them to be linked. The `|` syntax is sugar for creating a `Chain` object behind the scenes.

So far, I've provided a light introduction to `LangChain`. My goal was to quickly show you what `LangChain` is, its object model, and how to code with it, knowing you're eager to dive in. I've also shared examples of common LLM applications to give you an idea of what people are building with LLMs. However, if you're not familiar with LLMs, you might want some background information before moving forward. I'll cover that in the next section. If you already have this knowledge, feel free to skip to the next chapter.

# Appendix B. Setting up a Jupyter Notebook environment

If you're just starting with Python Jupyter notebooks, think of it as an interactive space where you can type and run code, see the results, and tweak the code to get the outcomes you want in real-time. The code snippets I'll be sharing aren't exclusive to a particular Python version, but for the smoothest experience, it's suggested to run them on Python 3.11 or a newer version if you can.

## Installing the Python interpreter or a Python distribution

If you haven't actively used Python lately, I suggest installing the latest version of the Python 3 interpreter. You have a few options:

1. **Standalone Python Interpreter:** Download and install Python 3 from [python.org](https://python.org). Choose the installer appropriate for your operating system and follow the setup instructions. You can find OS-specific guidance in the official documentation here: <https://docs.python.org/3/using/index.html> (you may need to scroll to locate your platform). This option is ideal if you prefer a lightweight Python installation without the additional libraries and tools bundled with distributions like Anaconda or Miniconda.
2. **Anaconda:** Download Anaconda from [anaconda.com/download](https://anaconda.com/download). It comes with Python and a wide range of data science libraries, including tools for visualization, data analysis, and numerical/statistical work. This option is perfect for data science and machine learning projects if you have enough disk space. Anaconda also makes it easy to create virtual environments for specific projects. It includes Anaconda Navigator, a user-friendly graphical interface, for those who prefer not to use command-line tools.
3. **Miniconda:** Miniconda is a lighter alternative to Anaconda. It allows you to manage virtual environments for specific applications without taking

up much disk space. It only includes essential data science libraries.  
Learn more at: [docs.conda.io/projects/miniconda/](https://docs.conda.io/projects/miniconda/).

For the remainder of this appendix, let's assume you have Python installed (as described in option 1 above). If you've chosen Miniconda or Anaconda, I assume you're familiar with Python and can adapt my instructions as needed.

Now, you're ready to set up a virtual environment using venv, which is a tool for managing virtual environments, and to update pip, the Python package installer.

### **Creating a virtual environment with venv and upgrading pip**

Open the operating system terminal shell (e.g., cmd on Windows), create a project folder, and navigate into it:

```
C:\Github\building-llm-applications\ch01>
```

Create a virtual environment with venv. A virtual environment serves as a self-contained Python installation, specifically for the ch01 folder you've recently created. This ensures that package version conflicts with other projects on your machine are avoided. Create a virtual environment for ch01 using the following command:

```
C:\Github\building-llm-applications\ch01>python -m venv env_ch01
```

You've successfully set up a virtual environment named "env\_ch01." Now, activate it using the following command:

```
C:\Github\building-llm-applications\ch01>.\env_ch01\Scripts\activ
```

You should now observe the updated operating system prompt, displaying the environment name in front as "(env\_ch01)":

```
(env_ch01) C:\Github\building-llm-applications\ch01>
```

Before proceeding further, it is useful to upgrade pip, the Python package management tool, as follows, so you will be able to install the necessary Python packages with no issues:



```
(env_ch01) C:\Github\building-llm-applications\ch01>python -m pip
```

For additional details on the steps you've taken, refer to the following documentation on the python.org website:

<https://packaging.python.org/en/latest/guides/installing-using-pip-and-virtual-environments/>

## Setting up a Jupyter notebook

Having activated the virtual environment, you're now prepared to configure a Jupyter notebook for executing prompts with OpenAI models. Proceed to install Jupyter and the LangChain packages using the following steps:

```
(env_ch01) C:\Github\building-llm-applications\ch01>pip install n
```

After about a minute, the installation of the notebook and LangChain packages should be finished. If you'd like, you can confirm it by using the following command:

```
(env_ch01) C:\Github\building-llm-applications\ch01>pip list
```

You can start the Jupyter notebook by executing the following command:

```
(env_ch01) C:\Github\building-llm-applications\ch01>jupyter noteb
```

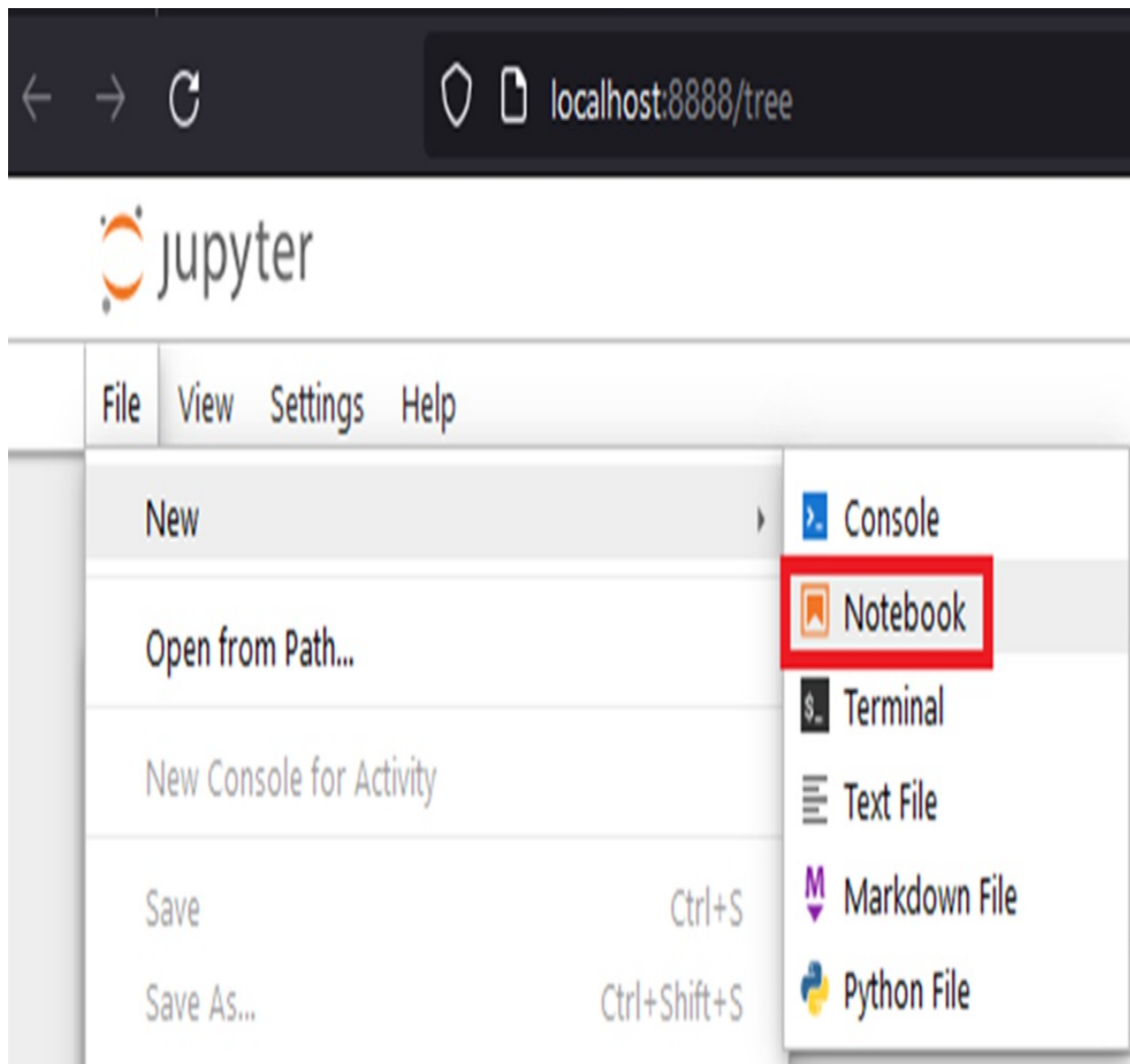
After a few seconds, you should see this output in the terminal:

```
[I 2023-10-23 22:43:26.870 ServerApp] Jupyter Server 2.8.0 is run
[I 2023-10-23 22:43:26.870 ServerApp] http://localhost:8888/tree?
[I 2023-10-23 22:43:26.871 ServerApp] http://127.0.0.1:8888/t
[I 2023-10-23 22:43:26.872 ServerApp] Use Control-C to stop this
[C 2023-10-23 22:43:26.978 ServerApp]
```

Subsequently a browser window will open up at this URL:  
<http://localhost:8888/tree>

Once you have created the notebook, select: File > Rename ' and name the notebook file: prompt\_examples.ipynb.

**Figure B.1 Creating a new Jupyter notebook: create the notebook with the menu File > New > Notebook and then rename the file to langchain\_examples.ipynb**



Now, you're prepared to input code into the notebook cells.

**TIP**

If you are unfamiliar with Jupyter notebook, remember to press Ctrl + Enter to execute the code in each cell.

# Appendix C. Choosing an LLM

## C.1 Popular Large Language Models

This appendix outlines the key features of the most popular large language models available at the time of writing. It also highlights the criteria to consider when selecting the most suitable LLM for your project.

### C.1.1 OpenAI GPT-4o and GPT-4.1 series

OpenAI's GPT-4, launched in March 2023, marked a significant milestone as the first "frontier model" to demonstrate advanced reasoning capabilities. It was also one of the earliest multi-modal models, capable of processing text, images, and later video, enabling diverse applications. While its architecture details remain undisclosed, GPT-4 was rumored to feature 1.8 trillion parameters and employed a "Mixture of Experts" design to enhance accuracy across various use cases.

The GPT-4o series succeeded GPT-4, offering specialized options for different tasks. GPT-4o was optimized for complex, high-accuracy tasks like multi-step reasoning, while GPT-4o-mini targeted simpler applications such as summarization, editing, and translation, replacing GPT-3.5 for these use cases.

The GPT-4.1 series, released in mid-2025, followed GPT-4o and introduced refinements in performance, speed, and cost-efficiency. It retained GPT-4o's strength in complex reasoning while offering better responsiveness and adaptability across a wide range of tasks. With improved balance between accuracy and resource usage, GPT-4.1 became a strong fit for both advanced and general-purpose applications.

LangChain integrates seamlessly with OpenAI models through the `langchain-openai` package and also supports open-source models using inference engines compatible with the OpenAI REST API, as detailed in

## Chapter 11.

### **C.1.2 OpenAI o1 and o3 series**

In September 2024, OpenAI introduced o1, designed for advanced reasoning and multi-step problem-solving with higher accuracy but at significantly higher costs — over 30 times that of GPT-4o-mini.

In December 2024, OpenAI announced the upcoming release of o3 and GPT-o3-mini, scheduled for early 2025, as an evolution of the o1 model with enhanced performance and capabilities.

o3 demonstrates exceptional results across various benchmarks. On the ARC-AGI test, which evaluates “general AI” capabilities, it scored 87.5%, surpassing the 87% achieved by top human performers. For comparison, GPT-3 scored 0%, and GPT-4o scored around 5%. In the Frontier Math benchmark, o3 solved 25% of complex problems, a ground-breaking achievement since no prior LLM managed more than 2%. Its programming abilities were equally remarkable, achieving 99% on a competitive programming benchmark where only top human programmers reach 100%.

o3 is positioned as the first LLM nearing AGI-level (Artificial General Intelligence) capabilities, setting new standards in reasoning, problem-solving, and programming.

### **C.1.3 Gemini**

In December 2023, Google introduced Gemini 1.5, a family of multimodal models capable of processing text, audio, images, and code. The lineup ranged from Gemini Ultra, designed for advanced tasks using a Mixture of Experts architecture, to Gemini Nano, optimized for lightweight mobile applications. Gemini Ultra surpasses PaLM-2 in benchmarks, excelling in reasoning, math comprehension, and code generation.

Gemini Ultra’s standout feature is its large context window, accommodating vast data inputs: up to 1 hour of video, 11 hours of audio, codebases exceeding 30,000 lines, or over 700,000 words. Research tests extended to 10

million tokens, demonstrating its capacity for extensive workloads.

Gemini 2.0, introduced in December 2024, marks a significant advancement in AI, enabling sophisticated reasoning for tasks like multimodal queries, coding, and complex math, powered by Trillium TPUs. Its leading model, Gemini 2.0 Flash, offers fast, high-performance capabilities with support for multimodal input and output, including text, images, video, audio, and steerable multilingual TTS. Enhancing its predecessor, 2.0 Flash operates at twice the speed of 1.5 Pro, integrates native tools, and sets new standards in AI performance.

Gemini 2.5, launched in May 2025, builds on the foundation of Gemini 2.0 with enhanced performance, efficiency, and broader multimodal capabilities. Designed for both advanced and real-time applications, Gemini 2.5 Pro offers improved reasoning, longer context handling, and faster response times across text, image, audio, and video inputs. It introduces better integration with Google's ecosystem, tighter tool use, and more reliable multilingual support, positioning it as a competitive alternative to other state-of-the-art models.

LangChain offers seamless access to Gemini models through the `langchain-google-genai` package, enabling developers to integrate these powerful models into their applications effortlessly.

### **C.1.4 Gemma**

In February 2024, Google rolled out Gemma, an open-source counterpart to Gemini, designed with a focus on lightweight functionality. Built on the same research and tech-stack as Gemini, Gemma offers model weight files at 2B and 7B parameters, freely accessible for use. The models are optimized to run on NVIDIA GPUs and Google Cloud TPUs. Additionally, Gemma is provided with a toolkit to facilitate effective utilization for both fine-tuning and inference tasks.

### **C.1.5 Claude**

In March 2023, Anthropic and Google launched Claude, a language model

designed with a strong emphasis on honesty and safety. Claude excels in summarization, coding, writing, and chatbot-based question answering. Although Anthropic has not shared details about its architecture or training, Claude set a new standard by supporting 100,000 tokens, which at the time was the biggest context window, enabling it to handle complex tasks. To improve speed, Anthropic released Claude Instant, optimized for faster performance.

Subsequent updates, including Claude 2 in July 2023, Claude 3 in March 2024, Claude 3.5 in October 2024, and Claude 3.7 in February 2025, enhanced accuracy, safety, and versatility across diverse language tasks. Claude 3.7 is available in the bigger accurate version, called Sonnet and the smaller faster version called Haiku. Claude 3.7 Sonnet is in the same segment as GPT-4.1 and Gemini 2.5 Pro. Haiku can be considered in the same group as GPT-4o-mini.

Anthropic models, including the Claude family, are accessible by installing the `langchain-anthropic` package.

### **C.1.6 Cohere**

Cohere, backed by former Google Brain employees, including Aidan Gomez, co-author of the influential "Attention Is All You Need" paper, focuses exclusively on enterprise needs. Known for precision and consistency, Cohere offers models ranging from 6 billion to 52 billion parameters, allowing organizations to tailor their approach.

Cohere has achieved the highest accuracy among large language models at some point, making it a dependable choice for businesses. Major corporations like Jasper and Spotify have adopted Cohere for advanced natural language processing, highlighting its practical applicability. However, it's worth noting that Cohere's technology comes at a higher cost compared to more widely recognized models from OpenAI. You can access Cohere models from LangChain by installing the `langchain-cohere` package.

### **C.1.7 Llama**

Meta's Llama series of large language models has played a major role in advancing open-access AI. First introduced in 2023 with a 70B-parameter model, Llama quickly evolved into a flexible, open-source platform with smaller variants to suit different computational needs. Built on transformer architecture and trained on diverse public datasets, Llama inspired derivative models like Vicuna and Orca. Over 2023 and 2024, Meta released Llama-2 and Llama-3, including a 405B-parameter model and the introduction of vision capabilities. In April 2025, Meta launched Llama 4, featuring major architectural improvements. The Llama 4 lineup includes Scout (109B parameters, multimodal, 10M token context), Maverick (400B parameters, optimized for reasoning), and Behemoth (a 2T-parameter model still in training, aimed at outperforming current benchmarks). While open-weight, these models come with licensing constraints for commercial use. Llama's scalability, openness, and compatibility with tools like LangChain—via wrappers such as GPT4All and Ollama—make it a powerful resource for both researchers and developers in AI and NLP.

### **C.1.8 Falcon**

Developed by the Technology Innovation Institute, Falcon is a transformer-based, causal decoder-only model designed for natural language processing tasks.

Falcon 3, released in December 2024, demonstrates strong performance compared to its peers. For instance, Falcon 3 10B achieves results comparable to Qwen 2.5 7B and Gemma 9B across benchmarks like MUSR and MMLU\_PRO, while outperforming them on the MATH benchmark. This positions Falcon 3 as a competitive option among contemporary open-source LLMs.

### **C.1.9 Mistral**

Founded in 2023, Mistral AI has rapidly become a leader in open-source large language models, known for its efficient and high-performing designs. It introduced the Mistral 7B model, followed by the Mixtral 8x7B and 8x22B models—both using Sparse Mixture-of-Experts (MoE) architectures that activate only a subset of parameters per token to improve cost-efficiency and

performance. In May 2025, the company released Mistral Medium 3, a mid-sized model optimized for enterprise use, offering strong performance at lower cost and supporting hybrid deployments. Mistral's ongoing innovation and open-source approach position it as a major competitor in the AI space, balancing scalability, affordability, and flexibility across a range of applications.

### **C.1.10 Qwen**

Qwen, open-sourced on GitHub in August 2023, is a family of LLMs developed by Alibaba Cloud. The models range from 1.8 billion to 72 billion parameters, trained on datasets between 2.2 trillion and 3 trillion tokens. While Qwen models are designed for general purposes, there are fine-tuned versions for specific tasks, such as Code-Qwen for coding and Math-Qwen for mathematics. Qwen-Chat has been fine-tuned using Reinforcement Learning from Human Feedback (RLHF), similar to ChatGPT. The models support a context length of around 30,000 tokens and perform particularly well in English and Chinese.

### **C.1.11 Grok**

Grok, developed by xAI, has rapidly advanced since its debut in late 2023. Initially focused on conversational tasks and integrated with the X platform, Grok used a mixture-of-experts architecture and supported up to 128,000 tokens. Subsequent versions—Grok-2 and Grok-2 Mini—added features like image generation and improved performance and speed. In February 2025, xAI released Grok 3, significantly upgraded with 10x more compute, new reasoning modes ("Think" and "Big Brain"), real-time data retrieval via "DeepSearch," and image editing capabilities. In May 2025, Grok 3.5 launched in beta with enhanced technical reasoning and a redesigned RAG system. Microsoft also partnered with xAI to host Grok 3 and Grok 3 Mini on Azure AI Foundry, bringing the models to a broader enterprise audience.

### **C.1.12 Phi**

The Phi-3 family comprises small language models (SLMs) designed to provide many capabilities of large language models while being more



resource-efficient. These models outperform others of the same and next size up in benchmarks for language, coding, and math tasks, thanks to Microsoft's innovative training methods. The Phi-3-mini (3.8 billion parameters) delivers exceptional performance, rivaling models twice its size, with future releases including Phi-3-small (7 billion parameters) and Phi-3-medium (14 billion parameters). The models are accessible through Microsoft Azure AI Model Catalog, Hugging Face, Ollama for local deployment, and as NVIDIA NIM microservices with standard API interfaces.

Phi-3.5-MoE, the latest addition to the Phi family, is a lightweight, state-of-the-art open model optimized for reasoning-intensive tasks such as code, math, and logic. It supports multilingual applications with a 128K token context length. Developed using high-quality datasets, it incorporates advanced fine-tuning and optimization techniques for precise instruction adherence and robust safety. Designed for memory-constrained and latency-sensitive environments, Phi-3.5-MoE powers general-purpose AI systems and is accessible through Azure AI Studio and GitHub via a serverless API, offering scalable and cost-efficient deployment.

### **C.1.13 DeepSeek**

DeepSeek, a Chinese AI company founded in 2023 by Liang Wenfeng, has developed open-source large language models (LLMs) that rival leading systems in performance and cost-efficiency. Its DeepSeek-V3 model, a Mixture-of-Experts (MoE) architecture with 671 billion parameters (activating 37 billion per token), was trained on 14.8 trillion tokens using supervised fine-tuning and reinforcement learning. Despite its scale, training required only 2.788 million GPU hours on Nvidia H800 chips, costing less than \$6 million. It outperforms other open-source models and matches top proprietary systems, excelling in mathematics, programming, reasoning, and multilingual tasks, with its AI assistant surpassing ChatGPT as the top free app on Apple's App Store.

DeepSeek-R1 also competes with high-end LLMs like OpenAI's o1, specializing in complex reasoning and coding through "chain-of-thought" reasoning. Trained with 2,000 Nvidia H800 chips at a cost of \$5.6 million, R1's efficiency sparked debates on sustainable AI training. While both

models are open-source, they avoid politically sensitive topics, raising privacy concerns and prompting global discussions about China's growing influence in AI and the shifting balance of technological power.

Table C.1 provides a summary of the key characteristics of the LLMs discussed in this appendix.

**Table C.1 Comparison among LLM models (\*sizes with an asterisk are estimated)**

Model	Developer	Launch	Num params	Max tokens	Open source
GPT-4o	OpenAI	May 24	200B*	128K	No
GPT-4.1	OpenAI	Apr 25	N/A	1M	No
o1	OpenAI	Oct 24	400B*	128K	No
o3	OpenAI	Dec 24	200B*	128K	No
Gemini 2.5 Pro	Google	May 25	N/A	1M	No
Gemma-3	Google	Mar 25	1B-27B	128K	Yes
Claude Sonnet-3.7	Anthropic	Feb 25	N/A	200K	No
Command R+	Cohere	Aug 24	100B	128K	No
Llama-4	Meta	Sep 24	90B	128K	Yes
Falcon3-10B-Base	TII-UAE	Dec 24	10B	32K	Yes
Mixtral-8x22B-Instruct-v0.1	Mistral AI	Mar 24	141B	64K	Yes
Qwen2.5 72B	Alibaba Cloud	Sep 24	72B	128K	Yes
Grok 3.5	xAI	Feb 25	314B	1M	No
DeepSeek-V3	Deepseek AI	Dec 24	671B	128K	Yes
DeepSeek-R1	Deepseek AI	Jan 25	671B	132K	Yes
Phi-4.0	Microsoft	Dec 24	14B	128K	Yes

## C.2 How to choose a model

Each use case has its unique requirements. Selecting the ideal LLM for your specific needs can be a complex task. It often involves testing multiple models to identify the most suitable one. However, you can employ certain criteria to streamline the selection process.

Several key factors to consider include:

- Model Purpose
- Proprietary vs. Open Source
- Model size
- Context Window size
- Supported Languages
- Accuracy vs. Speed
- Cost and Hardware Requirements
- Task suitability
- Safety and Bias

In the following sections, I'll explore these factors in greater depth to help you make a well-informed decision.

### C.2.1 Model Purpose

Some LLMs are flexible and can manage various tasks, while others are tailored for specific functions. For instance, OpenAI's GPT-4 or Gemini 1.5 Pro are versatile and adaptable models, suitable for a range of tasks, while Code Llama, or Qwen Coder as their names suggest, are specialized for generating programming code.

Specialized models are usually smaller than their more general counterparts. If you have a clear use case in mind, it's a good idea to choose an LLM specifically created and trained for that purpose. There are common LLM specializations, ranging from simpler tasks like text classification and sentiment analysis to more advanced functions like text summarization, translation, code analysis, text generation, question-answering, and logic and reasoning.

Many foundational LLMs can handle combinations of these functions, offering a wide range of possibilities to meet your specific needs.

## **C.2.2 Proprietary vs. Open-Source**

Many language models are proprietary, meaning their developers keep the internal details private. Information such as architecture, parameter count (discussed in the next section), or training specifics is often undisclosed. Proprietary models from providers like OpenAI, Gemini, Cohere, Anthropic and Stability AI are typically offered as cloud-based subscription services accessed through REST APIs. Pricing depends on the accuracy of the model used and the number of tokens processed. This approach provides convenience, allowing users to access ready-to-use services without managing hardware or infrastructure. However, data submitted to these services is retained and potentially processed by the provider, which can be a concern for sensitive data.

In contrast, open-source models offer full transparency, providing access to their underlying implementation as well as detailed information about their architecture and training processes. This transparency brings two key advantages. First, it enables you to deploy a fully private solution, avoiding the need to send sensitive data to third-party vendors—an important consideration for privacy and security. Second, it allows for fine-tuning on domain-specific datasets, giving you the flexibility to adapt the model to your unique needs. The trade-off, however, is that you typically need to host and manage the infrastructure yourself, which involves provisioning GPUs and dedicating IT resources to maintain and scale the service. As an alternative, you can opt for managed LLM hosting platforms such as IBM Watsonx, Amazon Bedrock, Azure AI Studio, or Google Vertex AI. These services provide a scalable, secure environment for running both open-source and proprietary models—eliminating much of the operational complexity associated with self-hosting.

LangChain supports both proprietary models, such as OpenAI, Gemini, and Cohere, and open-source models through inference engine wrappers like Ollama.

### **C.2.3 Model size (Number of Parameters)**

The A model's parameters represent its internal variables, specifically the weights in the artificial neural network. These weights are adjusted during training and enable the model to learn patterns from data. More parameters allow a model to capture greater complexity and nuance, potentially increasing accuracy. However, larger models with more parameters require substantial hardware resources—such as increased memory and high-performance GPUs—and often result in higher latency during inference. Compression techniques can reduce a model's size without major loss of accuracy, which we'll discuss in the chapter on open-source LLMs.

Language models vary widely in parameter count, from trillions in GPT-4 and Gemini 1.5 Ultra to a few billion in Mistral and Gemma. Parameters form the foundation of a model's ability to process text tasks.

### **C.2.4 Context Window size**

The number of input tokens allowed in Large Language Models (LLMs) directly impacts their functionality and suitability for different tasks. Token limits vary across models, influencing the complexity of prompts they can handle.

Models with smaller token limits are well-suited for concise prompts and straightforward interactions. These limitations often stem from design choices or computational constraints, and such models are typically specialized for a narrow set of tasks. In contrast, LLMs with higher token capacities—often designed as generalists—can handle more detailed, context-rich inputs, making them better suited for complex, multi-step tasks.

Choosing the right token limit depends on the task. For short, straightforward inputs, smaller token allowances may suffice. For applications requiring detailed interactions or extended context, models with larger token limits are more appropriate.

In summary, token capacity is a key consideration when selecting an LLM. Aligning the token limit with your project's requirements ensures effective

use of the model's capabilities.

### **C.2.5 Multilingual Support**

When considering an LLM for multilingual support, it's essential to research which one aligns with your requirements. Some LLMs are proficient in multiple languages, including ancient ones like Latin, ancient Greek, and even Phoenician, while others are primarily English-focused. Typically, LLMs excel with languages that have extensive training data, like English, Chinese, and Spanish. If your needs involve a less common language, you might need to seek a specialized LLM or even undertake fine-tuning yourself. It's crucial to match your language requirements with the capabilities of the chosen LLM for optimal results.

### **C.2.6 Accuracy vs. Speed**

Selecting Choosing an LLM often requires balancing accuracy and processing speed. Larger models with more parameters deliver higher precision and nuanced responses, especially for complex language tasks. However, this accuracy comes at the cost of slower processing and significant computational requirements.

Smaller models, with fewer parameters, are faster and better suited for real-time applications. While they sacrifice some accuracy, they excel in tasks requiring quick responses. The decision between a large or small model depends on the specific needs of the application—detailed comprehension favors larger models, while speed-sensitive tasks benefit from smaller ones.

Advances in compression techniques, discussed in Chapter 10 on open-source models, have bridged this gap. Compact models with lower parameter counts now achieve accuracy comparable to much larger models, making the trade-off between speed and precision less of a limitation.

### **C.2.7 Cost and Hardware Requirements**

Cost and hardware requirements are key factors when deploying Large Language Models (LLMs). Organizations must carefully weigh financial and

technical considerations to ensure effective use of these models.

Proprietary LLMs are typically priced based on their accuracy and the number of tokens processed. While they deliver high precision, enhanced capabilities result in higher costs, requiring organizations to balance accuracy against budget constraints.

Open-source LLMs, though more affordable in terms of licensing, shift the cost burden to hardware and infrastructure. Deploying these models demands powerful GPUs, sufficient RAM, and reliable virtual machines. Additionally, organizations need skilled IT staff to manage deployment, maintenance, and support.

The choice between proprietary and open-source LLMs depends on organizational goals and resources. Proprietary models offer convenience and performance for a fee, while open-source models provide flexibility and cost savings at the expense of hardware and IT investment. Careful evaluation ensures the solution aligns with both objectives and available resources.

### **C.2.8 Task suitability (standard benchmarks)**

The effectiveness of a language model for specific tasks depends on its architecture, size, training data, and fine-tuning. Different models are optimized for different use cases, and standardized benchmarks help evaluate their performance across various tasks.

Leaderboards like the Hugging Face Open LLM Leaderboard and LMSYS Chatbot Arena Leaderboard offer a centralized way to compare models across benchmarks, streamlining the evaluation process. Below are some widely recognized benchmarks, with descriptions drawn from the corresponding research publications.

- **Instruction-Following Evaluation (IFEval):** IFEval is an evaluation benchmark for Large Language Models (LLMs) designed to assess their ability to follow natural language instructions. It uses 25 types of "verifiable instructions," such as word count or keyword mentions, and includes 500 prompts. IFEval provides a simple, reproducible alternative to human evaluations, which are costly and inconsistent, and LLM-

based evaluations, which may be biased. Paper:

<https://arxiv.org/abs/2311.07911>.

- **Big Bench Hard (BBH):** The BIG-Bench Hard (BBH) benchmark is a subset of 23 challenging tasks from the BIG-Bench evaluation suite, designed to test areas where language models have historically underperformed compared to average human raters. These tasks often require multi-step reasoning and highlight the limitations of language models without advanced prompting techniques. Using chain-of-thought (CoT) prompting, models like PaLM and Codex demonstrated significant improvements, surpassing human performance on 10 and 17 tasks, respectively. The benchmark emphasizes that few-shot prompting alone underestimates the potential of language models, and CoT prompting reveals their advanced reasoning capabilities, particularly as model scale increases. Paper: <https://arxiv.org/abs/2210.09261>.
- **Mathematics Aptitude Test for Euristics - Level 5 (MATH, L5):** The MATH benchmark is a dataset of 12,500 challenging mathematics problems designed to evaluate the mathematical problem-solving abilities of machine learning models. Each problem includes a detailed step-by-step solution, enabling models to learn answer derivations and explanations. Alongside MATH, an auxiliary pretraining dataset is provided to help models grasp fundamental mathematical concepts. Despite improvements, model accuracy on MATH remains low, indicating that scaling Transformer models alone is insufficient for strong mathematical reasoning. Advancing this capability will likely require novel algorithms and research breakthroughs. Paper: <https://arxiv.org/abs/2103.03874>.
- **Google Proof Q&A (GPQA):** GPQA is a dataset of 448 multiple-choice questions in biology, physics, and chemistry, created by domain experts to be exceptionally difficult. Expert participants with advanced degrees achieve 65% accuracy (74% excluding identified errors), while non-experts with unrestricted web access average only 34%, making the questions effectively "Google-proof." GPQA is designed to aid in developing scalable oversight methods, enabling humans to supervise and extract reliable, truthful information from AI systems, even those surpassing human capabilities. Paper: <https://arxiv.org/abs/2311.12022>.
- **Multistep Soft Reasoning (MuSR):** MuSR is a benchmark designed to evaluate LLMs on multistep reasoning tasks framed within natural



language narratives. It features complex reasoning scenarios, such as 1000-word murder mysteries, generated using a neurosymbolic synthetic-to-natural algorithm. These tasks are challenging for advanced models and can scale to match future LLM advancements. MuSR emphasizes real-world reasoning, offering narratives that are more realistic and difficult than typical synthetic benchmarks while remaining accessible for human annotation. The benchmark highlights limitations in current reasoning techniques, such as chain-of-thought prompting, and helps identify areas for improvement in robust reasoning capabilities. Paper: <https://arxiv.org/abs/2310.16049>.

- More Robust Massive Multitask Language Understanding (MMLU-PRO): MMLU-Pro is an advanced benchmark that builds on the Massive Multitask Language Understanding (MMLU) dataset by introducing more challenging, reasoning-focused questions and expanding answer options from four to ten. It removes trivial and noisy questions, resulting in a dataset that better evaluates complex reasoning capabilities. Compared to MMLU, MMLU-Pro decreases model accuracy by 16% to 33% and reduces sensitivity to prompt variations, improving stability. Models using Chain of Thought (CoT) reasoning perform better on MMLU-Pro, highlighting its emphasis on complex reasoning over direct answering. This benchmark provides a more effective tool for tracking advancements in AI reasoning and comprehension. Paper: <https://arxiv.org/abs/2406.01574>.

The table below provides a snapshot of how leading LLMs perform across key benchmarks at the time of publication. For the most accurate insights, check for updated scores on the latest versions of the models you plan to use.

**Table C.2 Benchmark scores (%) of the most popular LLMs**

Model	IfEval	BBH	MATH-L5	GPQA	MuSR	MMLU-PRO
GPT-4o	85.60	83.10	76.60	53.60	N/A	74.68
Gemini 2.0 Flash	N/A	86.80	89.70	62.10	N/A	76.40
Claude 3.5 Sonnet	88.00	N/A	71.1	59.40	N/A	78.00

Command R+	N/A	N/A	44.0	N/A	N/A	N/A
Grok 2	75.50	N/A	76.1	N/A	N/A	N/A
DeepSeek-V3	86.10	N/A	N/A	59.10	N/A	75.90
Qwen2.5-72B-Instruct	86.38	61.87	1.21	16.67	11.74	51.40
Qwen2.5-Coder-32B	43.63	48.51	30.59	12.86	15.87	47.81
Qwen2.5-Math-7B	24.60	22.01	30.51	5.82	5.00	19.09
Llama-3.3-70B-Instruct	89.98	56.56	0.23	10.51	15.57	48.13
Mixtral-8x22B-Instruct-v0.1	71.84	44.11	18.73	16.44	13.49	38.70
Mistral-7B-v0.3	22.66	23.95	3.02	5.59	8.36	21.70
Gemma-2-27b-it	79.78	49.27	0.76	16.67	9.11	38.35
Falcon3-10B-Base	36.48	41.38	24.77	12.75	14.17	36.00
Phi-3.5-MoE-instruct	69.25	48.77	22.66	14.09	17.33	40.64

### C.2.9 Safety and Bias

Ensuring the safety and fairness of Large Language Models (LLMs) is essential for ethical and responsible AI use. The focus is on preventing harmful, offensive, or biased outputs that could negatively impact individuals or reinforce stereotypes.

To improve safety, LLMs must avoid generating content that is sexist, racist, or hateful. Since these models learn from extensive internet text, which can contain harmful material, implementing robust filtering and monitoring is critical to align outputs with ethical standards.

Reducing bias is equally important. Without careful oversight, LLMs can unintentionally perpetuate stereotypes. For example, assuming doctors are male and nurses are female reinforces outdated gender biases, undermining inclusivity and fairness.

This challenge is especially important in fields like healthcare, customer service, and education, where unbiased and accurate responses are vital. Regularly refining training data and fine-tuning models are necessary steps to ensure fairness and inclusivity.

In summary, promoting safety and reducing bias in LLMs requires continuous effort and a commitment to ethical AI practices. As you explore and implement AI solutions, prioritize testing and refinement to address potential issues, as covered in the next section.

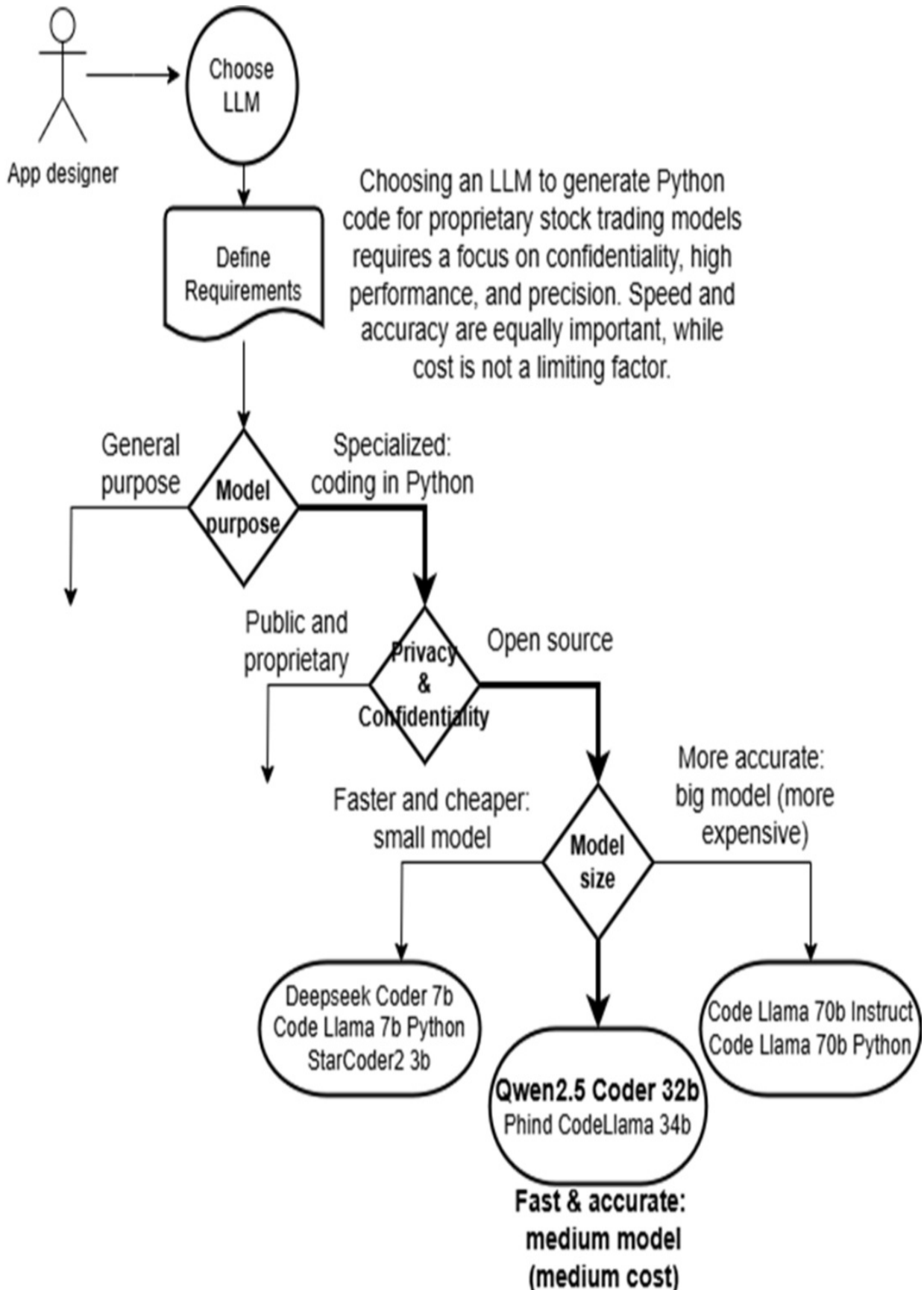
### **C.2.10 A practical example**

Let's integrate all the key factors and decisions involved in selecting an LLM for a specific application, using a practical example. Imagine implementing a code assistant chatbot to generate Python snippets for stock trading strategies. The target users are software developers supporting traders in a financial organization.

The code generator must utilize proprietary in-house trading libraries, making confidentiality a critical concern. Company policy strictly prohibits submitting any code referencing proprietary trading libraries to external systems. The assistant must generate code quickly while maintaining high accuracy to minimize bugs and avoid delays, especially during volatile trading periods. As this is for a financial company, cost is not a limiting factor.

The flowchart below illustrates the decision-making workflow for selecting an LLM for this use case.

**Figure C.1 Flowchart for selecting an LLM for a Python coding assistant designed to generate code for trading strategies. The process begins with choosing between a general-purpose and a specialized model, with a preference for LLMs tailored for code generation, particularly in Python. The next step focuses on selecting open-source models to comply with strict IT policies requiring all proprietary code to remain confidential. The final decision balances speed and accuracy, leading to the selection of Qwen2.5 Coder 32b based on its strong performance, including a high Python HumanEval score for accurate Python code generation.**



The process begins with defining requirements and progresses through the following decision points:

1. General-purpose vs. Specialized Models: The focus is on specialized LLMs optimized for code generation, particularly for Python. These models are better suited to the task than general-purpose models.
2. Privacy and Confidentiality: Given the sensitivity of the proprietary trading libraries, company policy mandates that the code cannot be exposed externally. This requires selecting an open-source model to ensure data remains within the organization.
3. Model Size: While cost is not a concern, the model must balance accuracy and speed. A medium-sized model is ideal to achieve this trade-off. The selection process narrows the options to a shortlist of seven LLMs, including general-purpose code generators and Python-specific models.
4. Accuracy Evaluation: The shortlisted models are evaluated using the Python HumanEval score (refer to Table C.3, sourced from Hugging Face leaderboard at: <https://huggingface.co/spaces/bigcode/bigcode-models-leaderboard>), which measures Python code generation accuracy. Qwen2.5-Coder-32b is chosen for its superior balance of accuracy and speed compared to other options.

**Table C.3 Python HumanEval rankings, sourced from the Hugging Face leaderboard**

Model	humaneval-python
Qwen2.5-Coder-32B-Instruct	83.20
DeepSeek-Coder-7b-instruct	80.22
CodeLlama-70b-Instruct	75.60
Phind-CodeLlama-34B-v2	71.95
CodeLlama-70b-Python	55.49
CodeLlama-7b-Python	40.48
StarCoder2-7B	34.09

Although Qwen2.5-Coder-32b has been selected as the top candidate, it is valuable to evaluate other strong alternatives, including Phind CodeLlama 34b, Deepseek Coder 7b, and Code Llama 7b Python.

## C.3 A Word of Caution

LLMs Large Language Models (LLMs) have revolutionized natural language processing, but they come with limitations and risks you need to understand.

- **Bias:** LLMs can inherit and reproduce biases present in their training data, which may result in harmful stereotypes or unfair outcomes. Addressing these issues is crucial for building ethical and trustworthy AI systems. If your application involves sensitive user profiles, it's important to implement guardrails—such as content filtering, human oversight, or prompt engineering—and to evaluate multiple LLMs to identify the one that exhibits the least bias in your specific use case.
- **Privacy and Security:** Proprietary LLMs may use prompt data for model improvement, which can raise privacy and confidentiality concerns. To mitigate this risk, consider deploying open-source models in a private environment where you retain full control over data handling. Some providers, such as OpenAI, offer enterprise plans that guarantee prompt data will not be used for training. However, it ultimately comes down to whether you trust the provider to uphold their data privacy commitments.
- **Hallucinations:** LLMs sometimes produce incorrect or fabricated answers, known as hallucinations. Transparency and explainability are key when users question an output. Tools like LangChain's evaluation framework help reduce hallucinations by improving response accuracy.
- **Responsibility and Liability:** The legal landscape for LLMs remains unclear. Determining who is responsible for errors or harm caused by LLM outputs is complex. Providers often include disclaimers in their terms and chatbot interfaces to limit liability. When deploying an LLM, consider accountability standards and define clear usage guidelines.
- **Intellectual Property (IP) Rights:** LLMs trained on unrestricted data can inadvertently generate content that violates IP laws. Some models address this by training exclusively on public domain material to avoid legal issues. If your application requires strict IP compliance, verify that your chosen LLM avoids proprietary or copyrighted content.
- Understanding these challenges ensures you deploy LLMs responsibly and avoid unintended risks.

# Appendix D. Installing SQLite on Windows

SQLite does not require a full installation. Simply unzip the package, place it in a folder, and add the folder to your system's Path environment variable. Follow these steps for Windows setup. For other operating systems, refer to the SQLite documentation.

1. Download SQLite:
  - Go to the SQLite download page:  
<https://www.sqlite.org/download.html>.
  - Download the latest zipped tools package, e.g., `sqlite-tools-win-x64-3460100.zip`.
2. Extract Files:
  - Unzip the downloaded file to a folder, for instance, `C:\sqlite`.
  - After unzipping, you should see the files, including the SQLite executable, in `C:\sqlite\sqlite-tools-win-x64-3460100`.
3. Add SQLite to the System Path:
  - Open the Start menu, go to *Control Panel*, and search for “edit system environment variables.”
  - In *System Properties*, click the *Environment Variables* button.
  - In the System variables section, select *Path* and click *Edit*.
  - Add `C:\sqlite\sqlite-tools-win-x64-3460100` at the end of the list, then click *OK* to close all dialog boxes.
4. Verify Installation:
  - Open a new command shell and type `sqlite3`. If everything is set up correctly, you’ll enter the SQLite prompt where you can start creating and managing databases.

With SQLite installed and configured, you can now create the database for the examples in Chapter 9.



# Appendix E. Open-source LLMs

## **This appendix covers**

- Advantages of open-source LLMs: flexibility, transparency, and control.
- Performance benchmarks and key features of leading open-source LLMs.
- Challenges of local deployment and strategies to address them.
- Selecting an optimal inference engine for your use case.

In earlier chapters, you worked with OpenAI's public REST API. It's a straightforward way to build LLM applications since you don't need to set up a local LLM host. After signing up with OpenAI and generating an API key, you can send requests to their endpoints and access LLM capabilities. This quick setup lets you work with state-of-the-art models like GPT-4o, 4o-mini, or o1 and o3 efficiently. The main drawback is cost—running examples like summarization might cost a few cents or even dollars. If you're working on projects for your company, privacy might also be a concern. Some employers block OpenAI entirely to avoid the risk of leaking sensitive or proprietary data.

This chapter introduces open-source LLMs, a practical solution for reducing costs and addressing privacy concerns. These models are especially appealing to individuals and organizations that prioritize data confidentiality or are new to AI. I'll guide you through the most popular open-source LLM families, their features, and the advantages they offer. The focus will be on running these models, ranging from high-performance, advanced setups to user-friendly tools ideal for learning and experimentation.

Finally, I'll show you how to transition the summarization and QA systems you built earlier to a local open-source LLM. By the end of this chapter, you'll understand open-source LLMs well and feel confident using them when they're the right choice.

## **E.1 Benefits of open-source LLMs**

Open-source Large Language Models (LLMs) offer clear advantages in cost, privacy, and flexibility. They provide control over data, lower costs by avoiding licensing fees, and allow customization. Community-driven development fuels innovation, making these models competitive with proprietary ones. This section explores the key benefits of open-source LLMs.

### **E.1.1 Transparency**

Closed-source models often function as "black boxes," making them difficult to understand and potentially problematic for compliance, especially in industries like healthcare and finance. Their lack of transparency limits customizability and creates challenges for auditing and accountability.

Open-source LLMs, by contrast, are transparent in their architecture, training data, and methods. Organizations can inspect, validate, and modify the code to suit specific needs, ensuring compliance and fostering trust. Developers gain flexibility to extend and adapt models for unique applications, improving control and oversight.

Transparent origins enhance trust in the model's integrity, offering verifiable assurances instead of relying on blind faith. This clarity also helps address potential privacy risks. However, with transparency comes responsibility—users bear accountability for flaws and financial impacts. Open access can pose cybersecurity risks if not adequately protected.

### **E.1.2 Privacy**

Privacy and security are critical, particularly in regulated industries like finance and healthcare. Leaks of sensitive data or Personally Identifiable Information (PII) can harm trust and reputation. Using proprietary public-cloud LLMs may raise concerns about security and intellectual property risks, especially for companies processing sensitive data.

Open-source LLMs mitigate these concerns by enabling on-premises or

private cloud deployment. Data stays within the corporate network, reducing exposure risks. These models can be customized to meet privacy needs, including implementing security protocols, content filtering, and data anonymization. They also support compliance with privacy laws and industry standards.

As privacy regulations grow stricter, open-source LLMs become increasingly appealing to organizations seeking control over their data.

### **E.1.3 Community driven**

Proprietary LLMs are typically developed with a fixed vision by a single organization. Open-source LLMs, however, benefit from contributions by a diverse community of developers, ranging from individuals to enterprises. These contributions improve features, provide flexibility, and foster growth through public forks and private customizations.

The open-source community produces a wide variety of models and training methods, advancing the field rapidly. While prominent contributors influence direction, the open nature allows anyone to contribute, driving collaboration and innovation. This dynamic reduces the performance gap with proprietary models as advancements in architecture, datasets, and training methods evolve.

However, community projects can face challenges, including disputes or mismanagement, which may slow progress. Despite these risks, open-source models empower smaller organizations to compete with industry leaders, leveling the playing field.

Choosing between proprietary and open-source LLMs depends on your priorities. If control, customization, and community engagement are essential, open-source models are ideal. If strict SLAs or turnkey solutions are more critical, proprietary models might be better suited. The decision depends on your project's needs and goals.

### **E.1.4 Cost Savings**

Open-source LLMs are typically more cost-effective than proprietary models due to the lack of licensing fees. However, deploying them on-premises may involve significant upfront capital costs, and running them in the cloud can incur ongoing operational expenses. Even with these factors, the total cost of ownership (TCO) for open-source models is usually lower over the medium to long term compared to the recurring fees of proprietary LLMs. The right choice depends on your use case, expected text processing volume, and readiness to handle initial deployment costs.

For low initial usage, a pay-as-you-go proprietary model might be more practical. As usage grows and the client base justifies investment in infrastructure, transitioning to an open-source model can save money. If you already have the skills to deploy and manage an open-source LLM, starting locally might be cost-effective. However, you must consider hidden costs, such as time spent by staff on setup and maintenance.

The cost-effectiveness of an LLM also depends on its application. Proprietary vendors charge based on the number of tokens processed, which can become expensive for tasks like summarizing large amounts of text. In such cases, an open-source model may reduce costs. On the other hand, for applications using efficient strategies like Retrieval Augmented Generation (RAG) with minimal text processing, proprietary models might be more economical.

An additional advantage of open-source LLMs is the ability to fine-tune them through specialized training, creating a custom model. However, this involves expenses such as consultancy fees, staff time, computational resources, and extended timelines. In cases where no proprietary model suits your specific domain, building and fine-tuning an open-source LLM may be the only option, albeit at your own expense.

## **E.2 Popular open-source LLMs**

The world of open-source Large Language Models (LLMs) is moving fast, making it tricky to figure out which ones stand out. To help, I've pulled together key details about popular open-source LLMs into a series of tables, starting with Table E.1. This table gives a straightforward overview of some of the most interesting open-source LLMs available just before this book was

published. Models are grouped by their type, listed under the “Model Type” column:

- **Foundation:** General-purpose models trained on raw data. Great for research, experimenting, or fine-tuning, but not usually ready for direct use with end-users.
- **Instruction:** Models fine-tuned to follow instructions or handle question answering.
- **Chat:** Models tailored for interactive conversations, similar to ChatGPT.
- **Code:** Built to generate, explain, or debug code.
- **Domain-Specific:** Designed for specialized industries or business needs.

This setup makes it easier to see what each type of model is good at and how it fits your needs.

**Table E.1 Most popular open-source LLMs at the time of publication**

Model	Developer	HuggingFace URL	Model Type	Size	Context window
DeepSeek-V3	Deepseek AI	/deepseek-ai/DeepSeek-V3	Foundation	671B	128K
Qwen2.5-72B-Instruct	Qwen	/Qwen/Qwen2.5-72B-Instruct	Instruct	72B	128K
Qwen2.5-Coder-32B	Qwen	/Qwen/Qwen2.5-Coder-32B	Code	32B	32K
Qwen2.5-Math-7B	Qwen	Qwen/Qwen2.5-Math-7B	Domain specific (Math)	7B	32K
Llama-3.3-70B-Instruct	Meta Llama	/meta-llama/Llama-3.3-70B-Instruct	Instruct	70B	128K
Mixtral-8x22B-Instruct-v0.1	Mistral AI	/mistralai/Mixtral-8x22B-Instruct-v0.1	Instruct	141B	64K

Mistral-7B-v0.3	Mistral AI	mistralai/Mistral-7B-v0.3	Foundation	7B	32K
gemma-2-27b-it	Google	google/gemma-2-27b-it	Foundation	27B	8K
Falcon3-10B-Base	TII-UAE	tiiuae/Falcon3-10B-Base	Foundation	10B	32K
Phi-3.5-MoE-instruct	Microsoft	microsoft/Phi-3.5-MoE-instruct	Instruct	41.9B	128K

Table E.2 shows the performance of these models based on standard benchmarks, fully defined in Appendix C.

**Table E.2 Extract from the HuggingFace Open LLM Leaderboard, showing standard performance benchmarks on the most popular open-source LLMs at the time of publication:**  
[https://huggingface.co/spaces/open-llm-leaderboard/open\\_llm\\_leaderboard](https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard)

Model	IfEval	BBH	MATH-L5	GPQA	MuSR	MMLU-PRO
DeepSeek-V3	86.10	N/A	N/A	59.10	N/A	75.90
Qwen2.5-72B-Instruct	86.38	61.87	1.21	16.67	11.74	51.40
Qwen2.5-Coder-32B	43.63	48.51	30.59	12.86	15.87	47.81
Qwen2.5-Math-7B	24.60	22.01	30.51	5.82	5.00	19.09
Llama-3.3-70B-Instruct	89.98	56.56	0.23	11.51	15.57	48.13
Mixtral-8x22B-Instruct-v0.1	71.84	44.11	18.73	16.44	13.49	38.70
Mistral-7B-	22.66	23.95	3.02	5.59	8.36	21.70

v0.3						
Gemma-2-27b-it	79.78	49.27	0.76	16.67	9.11	38.35
Falcon3-10B-Base	36.48	41.38	24.77	12.75	14.17	36.00
Phi-3.5-MoE-instruct	69.25	48.77	22.66	14.09	17.33	40.64

As shown in the table table E.2, models from the same family can vary in suitability depending on the use case. For instance, if you need an LLM for an instruction-based chatbot, you should look for one with a high IFEval score. In that case, Qwen2.5-72B-Instruct is a strong option with an impressive IFEval score of 86.38. On the other hand, if your focus is solving math problems, Qwen2.5-Math-7B might be a better choice due to its high MATH score of 30.51, combined with the advantage of being smaller in size.

Now that you know about popular open-source LLMs and their features, I'll show you how to run these models on your own computer. This will let you experiment and explore their capabilities firsthand.

## E.3 Considerations on running open-source LLMs locally

Large language models operate in two main phases: training and inference. During training, the model learns patterns from the training set, adjusting its weights (parameters). These weights are then used during inference to make predictions or respond to new inputs. Open-source LLM weights are usually easy to access, often available on platforms like Hugging Face or through tools like LM Studio or Ollama. Running the inference phase locally requires suitable hardware. This section covers hardware requirements and how modern techniques such as quantization make it feasible to run models even on consumer-grade machines. An important consideration when hosting an LLM locally is whether it provides an OpenAI-compatible API, such as a REST API. Using OpenAI during the Proof of Concept (PoC) phase and transitioning to an open-source LLM later can reduce costs or address privacy

concerns. If the inference engine supports an OpenAI-compatible API, you can switch seamlessly to production without rewriting or retesting code.

### **E.3.1 Limitations of consumer hardware**

Hosting a large LLM in production demands powerful hardware. For example, Llama-70B requires 140GB of RAM and disk space when loaded in single-precision (float16). High-end GPUs, such as NVIDIA's A100 (priced at \$10,000 or more), are essential for efficient inference. Cloud options are available, but they are expensive, costing about \$1.50 per hour for A100 rentals.

For local experimentation, however, consumer hardware can still be viable. Techniques like quantization and specialized inference engines reduce the hardware burden, enabling LLMs to run on laptops or desktops with as little as 16GB or 32GB of RAM and no dedicated GPU. These methods make it possible to explore LLM functionality on modest machines.

### **E.3.2 Quantization**

Quantization reduces LLM size by lowering the precision of weights, trading some accuracy for faster and more efficient inference. Large models typically use 16-bit or 32-bit floating-point precision (2 to 4 bytes per parameter). Quantization compresses this to 4-bit integer precision (0.5 bytes per parameter). Quantization offers a balance between size, speed, and accuracy. For example, LLAMA-2-7B, originally 13.5GB with 16-bit precision, can be reduced to 3.9GB with 4-bit quantization, allowing it to run on modest laptops.

The most common quantization techniques used are:

- **Post-Training Quantization (PTQ):** Applied after training; simpler but may reduce accuracy.
- **Quantization-Aware Training (QAT):** Integrated during training for better accuracy but more complex.
- **GPTQ:** Combines GPT with PTQ techniques.
- **NF4:** Uses 4-bit Normal Float precision, often outperforming standard



4-bit integer quantization.

- GGML and GGUF: Tools by Georgi Gerganov for running models on CPUs. GGUF, an enhanced version, supports a wider range of open-source models.

Some quantization tools like `bitsandbytes` (by Tim Dettmers) are available through Python's `pip install` and are well-documented on Hugging Face. Quantized models can be found on Hugging Face by searching terms like GGML or GGUF, or directly through inference engines such as Ollama or LM Studio.

#### **TIP**

For a deeper dive into quantization, check out the following blog posts “What are Quantized LLMs” by Miguel Carreira Neves (<https://www.tensorops.ai/post/what-are-quantized-llms>) and “How is llama.cpp possible?” by Finbarr Timbers (<https://finbarr.ca/how-is-llama-cpp-possible>)

Modern advancements in quantization and optimized engines have significantly lowered the barrier to running open-source LLMs locally, making them accessible even for users with limited hardware, especially for learning and experimentation.

### **E.3.3 OpenAI REST API compatibility**

Many inference engines include an embedded HTTP server that accepts requests through endpoints compatible with the OpenAI REST API. This compatibility allows immediate use with standard OpenAI libraries, simplifying the engine's codebase and supporting quick adoption. For users, the benefits include minimal learning requirements compared to learning proprietary APIs and the ability to swap engines seamlessly. With this standardization, you can efficiently test multiple engines using the same client code or replace an engine with one better suited to your needs.

In this section, I'll provide code snippets which you can run against each inference engine I will introduce later. These snippets require minimal

modifications—often just adjusting the port number. They demonstrate how to make requests using raw HTTP (e.g., `curl`), the OpenAI Python library and LangChain. Let's start with direct HTTP requests.

## Direct curl invocation

The easiest way to test an OpenAI-compatible REST API endpoint is by using the `/chat/completions` endpoint. Below is an example with OpenAI's public service. Open a shell (on Windows you can open a command line window or a Git Bash shell) and replace `YOUR-OPENAI-KEY` with your actual OpenAI API key:

```
$ curl https://api.openai.com/v1/chat/completions -H "Content-Type: application/json" \
 -d '{
 "model": "gpt-4o-mini",
 "messages": [
 { "role": "system", "content": "You are an helpful AI assistant." },
 { "role": "user", "content": "How many Greek temples are in Paestum?" }
],
 "temperature": 0.7
 }'
```

The response might look like this:

```
% Total % Received % Xferd Average Speed Time Time Time
100 810 100 578 100 232 209 83 0:00:02 0:00:02 0:00:00
{
 "id": "chatcmpl-AjANT94Lel2oZIOz8kmF0PKWfRuGa",
 "object": "chat.completion",
 "created": 1735328015,
 "model": "gpt-4o-mini-2024-07-18",
 "choices": [
 {
 "index": 0,
 "message": {
 "role": "assistant",
 "content": "Paestum, an ancient Greek city located in southern Italy.",
 "refusal": null
 },
 "logprobs": null,
 "finish_reason": "stop"
 }
],
}
```

```

 "usage": {
 "prompt_tokens": 28,
 "completion_tokens": 163,
 "total_tokens": 191,
 "prompt_tokens_details": {
 "cached_tokens": 0,
 "audio_tokens": 0
 },
 "completion_tokens_details": {
 "reasoning_tokens": 0,
 "audio_tokens": 0,
 "accepted_prediction_tokens": 0,
 "rejected_prediction_tokens": 0
 }
 },
 "system_fingerprint": "fp_0aa8d3e20b"
 }
}

```

To make the same request to a local open-source LLM, adjust the command as follows:

```

curl https://localhost:8000/v1/chat/completions -H "Content-Type: application/json" \
 -d '{
 "model": "mistralai/Mistral-7B-v0.3",
 "messages": [
 { "role": "system", "content": "You are a helpful AI assistant" },
 { "role": "user", "content": "How many Greek temples are there in Athens?" }
],
 "temperature": 0.7
 }'

```

#### Note

For local inference engines, you do not need to include an OpenAI key in the request header, so it has been omitted in the example. However, you need to adjust two key details in the `curl` request based on the inference engine and LLM you are using:

1. **Port Number:** Each inference engine's local HTTP server is set to a default port, which is often different from 8000. Refer to the engine's documentation and update the port number as needed.
2. **Model Name:** Inference engines use specific naming conventions for models. Some follow Hugging Face conventions (e.g., 'mistralai/Mistral-7B-v0.3'), while others may not. Check the engine's documentation to

ensure you provide the correct model name.

## Python openai library

To use the OpenAI library for requests, create a virtual environment, install the library with `pip install openai`, create a Jupyter notebook, as you have seen in the previous chapters, and use the code from listing E.1 (after replacing YOUR-OPENAI-KEY with your OpenAI key).

### Listing E.1 Calling the OpenAI completions endpoint

```
import getpass
from openai import OpenAI

OPENAI_API_KEY = getpass.getpass('Enter your OPENAI_API_KEY')
client = OpenAI(
 api_key = OPENAI_API_KEY
)

completion = client.chat.completions.create(
 model="gpt-4o-mini",
 messages=[
 { "role": "system", "content": "You are a helpful AI assistant" },
 { "role": "user", "content": "How many Greek temples are there in Paestum?" }
],
 temperature=0.7
)

print(completion.choices[0].message.content)
```

You will get output similar to this:

Paestum, an ancient Greek city located in present-day Italy, is r

In order to run the code above against a local open-source LLM, you need to adapt it as follows (note the extra `base_url` parameter when instantiating the OpenAI client):

```
from openai import OpenAI

port_number = '8080' #A

client = OpenAI(
```

```

 base_url=f'http://localhost:{port_number}/v1',
 api_key = "NO_KEY_NEEDED"
)

completion = client.chat.completions.create(
 model="mistral",
 messages=[
 { "role": "system", "content": "You are a helpful AI assistant" },
 { "role": "user", "content": "How many Greek temples are there in Paestum?" },
],
 temperature=0.7
)

print(completion.choices[0].message.content)

```

### Note

To adapt the example above for your inference engine and local open-source LLM, update the port number to match the engine's local HTTP server and set the model name according to the engine's specific naming convention. For example, if using LM studio, use port 8080.

## LangChain's openai wrapper

For LangChain, direct its OpenAI wrapper to the local inference engine by updating the base URL:

```

from langchain_openai import ChatOpenAI

port_number = '8080'

llm = ChatOpenAI(openai_api_base=f'http://localhost:{port_number}')
response = llm.invoke("How many Greek temples are there in Paestum?")

print(response.content)

```

### NOTE

Ensure the port matches the local engine configuration.

Now, let's explore the essential component required to execute open-source models on regular consumer hardware: local inference engines.

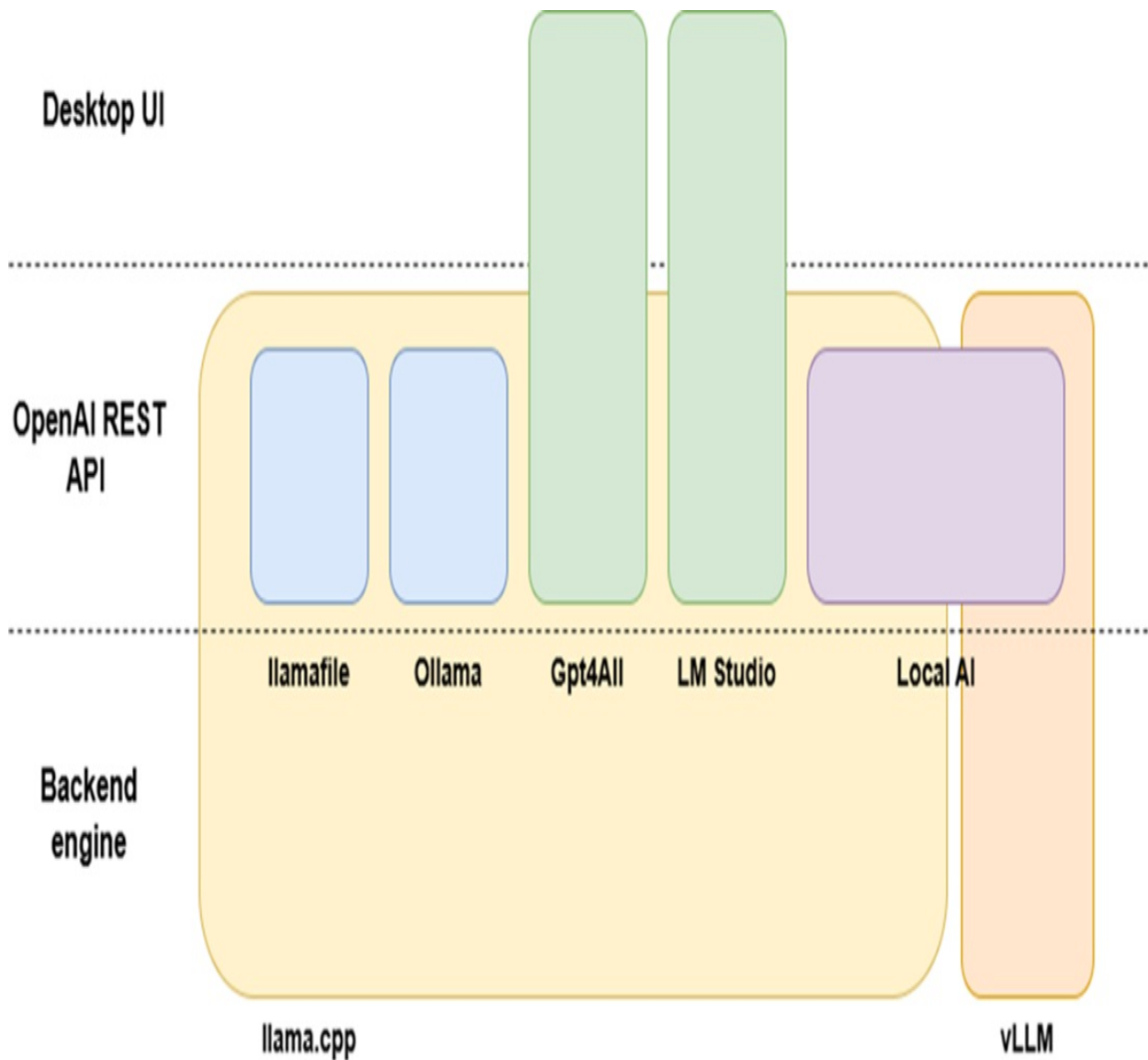
## E.4 Local inference engines

Running an open-source LLM on consumer hardware is most practical with an inference engine. These engines host the local model and handle requests from client applications, often through native bindings for languages like Python, JavaScript, Java, C++, Go, or Rust. Many inference engines also include a local HTTP server with OpenAI REST API-compatible endpoints, allowing you to use familiar OpenAI libraries or frameworks like LangChain without significant changes. This flexibility lets you switch between local open-source LLMs and OpenAI's public service with minimal effort.

In the following sections, I will introduce several inference engines, beginning with foundational tools like llama.cpp, optimized for high performance, and moving to user-friendly options like Ollama and production-grade solutions such as vLLM. I will also cover consumer-focused alternatives, including LocalAI (a simplified wrapper for engines like llama.cpp or vLLM), GPT4All, and LM Studio, which are notable for their intuitive user interfaces. These options will help you select the best engine based on your hardware, experience, and project requirements.

As shown in Figure E.1, which serves as a guide for this section, llama.cpp and vLLM are foundational backends for many inference engines. Higher-level tools like Ollama, llamafile, and LocalAI, along with user-friendly engines such as GPT4All and LM Studio, are built on llama.cpp. Meanwhile, vLLM functions independently, with LocalAI being the only tool currently building on it.

**Figure E.1 Lineage and functionality of local inference engines**



Let's start our tour with llama.cpp.

### E.4.1 Llama.cpp

*llama.cpp* was one of the first engines designed to run open-source models efficiently on consumer hardware. Initially developed to support the LLaMA model using 4-bit integer quantization for Apple silicon GPUs, it has since expanded to support Linux, Windows (x86 processors), and even Raspberry Pi. It now handles a wide range of quantized models, including Mistral, Gemma, Phi and Falcon. It supports 2-bit to 8-bit integer quantization and offers bindings for Python, Java, C#, Go, Scala, and Ruby.

## Setting up llama.cpp

To use *llama.cpp*, follow these steps:

1. **Build the Executable:** Download the source code from GitHub and build it using your preferred strategy, such as `make`, `CMake`, `Zig`, or `gmake`. Advanced build options include Metal, MPI, and BLAS for enhanced performance.
2. **Prepare the Model Weights:** Obtain a quantized version of the model (e.g., `Mistral-7B-Instruct-v0.2-GGUF`) from Hugging Face or generate it using GitHub instructions. Ensure the model fits within your system's disk and RAM capacity.
3. **Run Inference:** Execute the inference command, pointing to the quantized model file:

```
./main -m ./models/mistral-7b-instruct-v0.2.Q2_K.gguf -p "How
```

### NOTE

For detailed instructions, refer to the official GitHub page:

<https://github.com/ggerganov/llama.cpp>.

## Python bindings

If you prefer a higher-level API, links to relevant bindings are available on the llama.cpp GitHub page. Before installation, review both the llama.cpp documentation and the selected bindings' documentation to prevent redundant setup.

To install the Python bindings from PyPI, use the following command:

```
pip install llama-cpp-python
```

This command not only downloads the library but also tries to build *llama.cpp* from source using `CMake` and your system's C compiler. For GPU support, follow additional instructions and configurations.

After setup, you can use Python to interact with your local quantized LLM



instance. Listing E.2 shows an adapted example from the official llama-cpp-python bindings documentation.

**Listing E.2 Executing prompts against a local quantized Mistral-7B-Instruct instance**

```
Adapted from official documentation at
https://github.com/abetlen/llama-cpp-python
from llama_cpp import Llama
llm = Llama(
 model_path="./models/mistral-7b-instruct-v0.2.Q2_K.gguf"
)
output = llm(
 "Q: What are the planets in the solar system? A: ", # Prompt
 max_tokens=32, # Generate up to 32 tokens, set to None to g
 stop=["Q:", "\n"], # Stop generating just before the model
 echo=True # Echo the prompt back in the output
)
print(output)
```

You get the following output:

```
{
 "id": "cmpl-xxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
 "object": "text_completion",
 "created": 1679561337,
 "model": "./models/mistral-7b-instruct-v0.2.Q2_K.gguf",
 "choices": [
 {
 "text": "Q: Name the planets in the solar system? A: Mercur",
 "index": 0,
 "logprobs": None,
 "finish_reason": "stop"
 }
],
 "usage": {
 "prompt_tokens": 14,
 "completion_tokens": 28,
 "total_tokens": 42
 }
}
```

## LangChain integration

LangChain's LlamaCpp client can connect directly to a quantized local

instance of Llama-based models, provided you have installed the `llama-cpp-python` library (<https://github.com/abetlen/llama-cpp-python>):

```
from langchain_community.llms import LlamaCpp
llm = LlamaCpp(model_path="./models/llama-2-7b-chat.ggmlv3.q2_K.b
```

## OpenAI API compatible endpoints

*llama.cpp* includes a local HTTP server that provides OpenAI REST API-compatible endpoints. To try it out, you can use the ``curl`` script or Python examples from section 11.3.3, ensuring the port number and model name are correctly configured.

While detailed installation and setup instructions for *llama.cpp* are not included here, it is important to understand its basics. Many of the local inference engines discussed later are built on *llama.cpp* or draw inspiration from its design. Next, I'll introduce *Ollama*.

## E.4.2 Ollama

*Ollama* is a hosting environment for open-source LLMs, available for Windows, macOS, and Linux. Unlike *llama.cpp*, *Ollama* does not require you to build executables from source; installation packages are readily available.

### Interactive mode

After downloading and installing Ollama from the Ollama.ai homepage (<https://ollama.ai>), launch the Ollama client (in Windows, launch it from the Start menu). This opens a terminal (or PowerShell window on Windows), where you can immediately run an LLM using the following command:

```
ollama run mistral
```

This command downloads a quantized version of the selected model (Mistral in this example) from the remote Ollama library. The terminal will display the download progress, as shown in Figure 11.2.

**Figure 11.2 Screenshot of ollama terminal while installing the Mistral LLM**

```
C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe

Welcome to Ollama!

Run your first model:

ollama run llama3.2

PS C:\WINDOWS\system32> ollama run mistral
pulling manifest
pulling ff82381e2bea... 100% 4.1 GB
pulling 43070e2d4e53... 100% 11 KB
pulling 491dfa501e59... 100% 801 B
pulling ed1leda7790d... 100% 30 B
pulling 42347cd80dc8... 100% 485 B
verifying sha256 digest
writing manifest
success
>>> Send a message (/? for help)
```

At this stage, you can send a prompt and receive a response, as shown below (note the response may be slow):

```
>>> How many Greek temples are in Paestum?
There are three Greek temples in Paestum, which is a town on the
```

1. Temple of Hera I (also known as the Basilica) - This temple was dedicated to the goddess Hera. It is the largest and most impressive of the three
2. Temple of Neptune - This temple was also built around 550 BC (dedicated to Neptune in Roman mythology). It is smaller than the Temple of Hera
3. Temple of Athena - This temple was built around 460 BC and is the smallest of the three temples and is incomplete due to damage sustained over time

All three temples are well-preserved and are a popular tourist destination.

*Ollama* natively supports several popular open-source LLMs, including Llama, Phi, and Gemma. You can browse the available models on the library page: <https://ollama.com/library>. To run a GGUF model from another source, such as Hugging Face, copy the model to a local folder and import it

following the instructions in the Ollama documentation.

## Server mode

Once the server is running, it exposes a REST API on port 11434, ready to accept requests for LLM interactions.

### Note

Ollama's REST API endpoints are proprietary and not compatible with OpenAI's specifications. Code examples from earlier sections cannot be used directly.

Below is an examples using Ollama's API from curl (or Postman, if you prefer).

```
curl http://localhost:11434/api/generate -d '{
 "model": "mistral",
 "prompt": "How many Greek temples are in Paestum?"
}'
```

By default, the output streams one word at a time, each included in a separate JSON object, as shown below:

```
{
 "model": "mistral",
 "created_at": "2024-12-28T11:52:26.1375731Z",
 "response": " There",
 "done": false
}
{
 "model": "mistral",
 "created_at": "2024-12-28T11:52:26.5276658Z",
 "response": " are",
 "done": false
}
{
 "model": "mistral",
 "created_at": "2024-12-28T11:52:26.9259504Z",
 "response": " three",
 "done": false
}
```

...

If you prefer to wait for the full response, you must set the “stream” attribute to false:

```
curl http://localhost:11434/api/generate -d '{
 "model": "mistral",
 "prompt": "How many Greek temples are in Paestum?",
 "stream": false
}'
```

You will get a single response object:

```
{
 "model": "mistral",
 "created_at": "2024-12-28T13:07:03.1862002Z",
 "response": " There are three Doric temples in Paestum, locat",
 "done": true,
 "done_reason": "stop",
 "context": [
 3,
 29473,
 ... SHORTENED,
 1102,
 29499
],
 "total_duration": 40777816100,
 "load_duration": 13459239600,
 "prompt_eval_count": 16,
 "prompt_eval_duration": 1454000000,
 "eval_count": 123,
 "eval_duration": 25791000000
}
```

This is a request to the /chat endpoint, using a payload format compatible with OpenAI's corresponding request structure:

```
curl http://localhost:11434/api/chat -d '{
 "model": "mistral",
 "messages": [
 { "role": "system", "content": "You are a helpful AI assistant" },
 { "role": "user", "content": "How many Greek temples are there in Paestum?" }
],
 "temperature": 0.7
}'
```

These API calls allow interaction with the LLM using any programming language. However, for Python or JavaScript, you can use libraries provided by *Ollama* that wrap the low-level REST API, simplifying client development.

## Native Python library

To get started, install the Ollama Python library:

```
pip install ollama
```

You can then interact with the LLM's /chat endpoint as shown:

```
import ollama
response = ollama.chat(model='mistral', messages=[
 { "role": "system", "content": "You are an helpful AI assistant" },
 { "role": "user", "content": "How many Greek temples are in Paris?" }
])
print(response['message']['content'])
```

Alternatively, you can send instructions to the /generate endpoint:

```
ollama.generate(model='mistral', prompt='How many Greek temples are in Paris?')
```

To enable streaming responses, set `stream=True` in the chat or generate call.

If you need to specify a custom host or timeout, create a `Client` object:

```
from ollama import Client
client = Client(host='http://mylocalserver:8000')
response = client.chat(model='mistral', messages=[
 { "role": "system", "content": "You are an helpful AI assistant" },
 { "role": "user", "content": "How many Greek temples are in Paris?" }
])
```

For asynchronous execution, use `AsyncClient` with `asyncio`:

```
import asyncio
from ollama import AsyncClient

async def chat():
```

```
message = {'role': 'user', 'content': 'How many Greek temples'}
response = await AsyncClient().chat(model='mistral', messages=[message])
print(response['message']['content'])

asyncio.run(chat())
```

## LangChain integration

LangChain provides a wrapper for the Ollama Python library, integrating it with LangChain interfaces such as the Runnable interface. Below is an example of re-implementing a basic synchronous LLM invocation using LangChain:

```
from langchain_community.llms import Ollama
ollama = Ollama(model='mistral')

query = 'How many Greek temples are in Paestum?'
response = llm.invoke(query)
print(response)
```

To handle streaming responses, you can use the following approach:

```
for chunks in llm.stream(query):
 print(chunks)
```

For more information, I recommend consulting the *Ollama* documentation on GitHub. It includes community-driven integrations, such as web and desktop UIs, terminal plugins for Emacs and Vim, and other useful extensions.

*Ollama* is designed to help you explore open-source models locally. If you decide to move beyond a proof of concept on a Linux system and build a production-grade solution using non-quantized LLMs on powerful GPU hardware, potentially in the cloud, consider exploring *vLLM*. While *vLLM* is not specifically for consumer-grade hardware, it demonstrates the capabilities of inference engines designed to host open-source models of any size on various types of hardware.

## E.4.3 vLLM

*vLLM* is a high-performance Python library for LLM inference, built on a

C++ core and CUDA binaries. It targets Linux systems and high-grade GPUs, including V100, T4, RTX20xx, A100, L4, and H100. According to the official web page (<https://docs.vllm.ai>), vLLM offers the following advanced performance features:

- High-Throughput Serving: State-of-the-art inference speed.
- PagedAttention: Efficient memory management for attention key and value data.
- Continuous Batching: Dynamically batches incoming requests for better efficiency.
- CUDA/HIP Graph Execution: Speeds up model execution.
- Quantization Support: Includes GPTQ, AWQ, and SqueezeLLM methods.

vLLM is designed with flexibility and usability in mind. Key features include:

- HuggingFace Integration: Seamless support for popular HuggingFace models.
- Advanced Decoding Algorithms: Supports parallel sampling, beam search, and other methods.
- Tensor Parallelism: Enables distributed inference across multiple GPUs.
- Streaming Outputs: Provides real-time response generation.
- OpenAI-Compatible API: Allows easy integration with existing applications.
- GPU Support: Works with both NVIDIA and AMD GPUs.

vLLM supports a variety of architectures, including BLOOM, Falcon, GPT-J, GPT-NeoX, Vicuna, LLaMA, and Mistral. While it can handle quantized models, vLLM is optimized for larger model versions, such as LLaMA-2-70B-hf and Falcon-180B. For a detailed list of supported models, refer to the *Supported Models* section of the vLLM documentation.

## Installation

To install vLLM, ensure you are using Linux with one of the recommended high-grade GPUs. It is best to use a dedicated virtual environment created with venv. Install the library with:



```
pip install vllm
```

Once installed, you can activate the server mode.

Server mode

Start a local OpenAI-compatible HTTP server with the following command in a separate terminal:

```
$ python -m vllm.entrypoints.openai.api_server --model mistralai/
```

This will download the model from Hugging Face (if not already downloaded) and activate an HTTP server on port 8000, exposing OpenAI-compatible endpoints.

## OpenAI REST API compatible endpoints

Refer to section E.3.3 for examples of `curl`, Python, and LangChain code that you can use with vLLM. Ensure you set the port to 8000 and use the correct model name. Consult the vLLM documentation for further details.

## Offline batched inference

*vLLM* offers a unique capability for high-performance offline batched inference, allowing you to process multiple requests in a single operation. This feature sets it apart from other local inference engines. To use it, import the `LLM` and `SamplingParams` modules, create a list of prompts, and process them using an `LLM` instance. The results can be iterated and reviewed, as shown in Listing E.3.

### Listing E.3 vLLM offline batched inference

```
from vllm import LLM, SamplingParams

prompts = [#A
 "How many temples are there in Paestum?",
 "Who built the aqueduct in Segovia?",
 "Is LeBron James better than Michael Jordan?",
 "Summarize the Lord of The Rings in three sentences.",
```

```

]
sampling_params = SamplingParams(temperature=0.7, top_p=0.95) #B

llm = LLM(model="mistralai/Mistral-7B-v0.1") #C

outputs = llm.generate(prompts, sampling_params) #D

for output in outputs: #E
 prompt = output.prompt
 llm_response = output.outputs[0].text
 print(f"Prompt: {prompt!r}, LLM response: {llm_response!r}")

```

After presenting the advanced capabilities of *vLLM*, designed for deploying open-source LLMs in professional production environments, I will now return to exploring local inference engines built for consumer-grade hardware.

### E.4.4 llamafile

One of the simplest ways to run an LLM locally is with *llamafile*. According to its GitHub description, *llamafile* allows you to "distribute and run an LLM with a single file." This executable combines the quantized weights of an LLM in GGUF format with the *llama.cpp* C++ executable, packaged using Cosmopolitan-Libc. Cosmopolitan-Libc, described as "making C a build-once, run-anywhere language," enables the *llamafile* to run on macOS, Linux, and Windows (with a .exe extension) without requiring any installation. This approach sets an incredibly low barrier for experimenting with local LLMs.

The *llamafile* GitHub page offers a variety of prebuilt LLMs, including recent versions of LLaMA, Gemma, and Phi.

#### Server mode

Running an LLM with *llamafile* is straightforward. Follow these steps (refer to the GitHub project for detailed instructions):

4. Download a *llamafile*: Obtain a prebuilt file, such as `mistral-7b-instruct-v0.2.Q5_K_M.llamafile`, from the GitHub project page (<https://github.com/Mozilla-Ocho/llamafile>).

5. Place the File in a Folder: Copy the *llamafile* to a directory on your system.

6. Set Permissions:

- On macOS or Linux: Grant execute permissions using:

```
chmod +x mistral-7b-instruct-v0.2.Q5_K_M.llamafile
```

- On Windows: Rename the file to include the .exe extension:

```
mistral-7b-instruct-v0.2.Q5_K_M.llamafile.exe
```

4. Run the File: Launch the *llamafile*. For example, on Windows:

```
c:\temp>mistral-7b-instruct-v0.2.Q5_K_M.llamafile.exe -ngl 999
```

The *llamafile* will start a web server at `http://127.0.0.1:8080` and open a browser pointing to a chat web interface at the same address. You can begin interacting with the LLM immediately, as shown in Figure E.3.

**Figure E.3** *llamafile* chat web UI pointing to the local web server communicating with the local open-source LLM

```
Command Prompt - Llama-3.2-3B-Instruct.Q6_K.llamafile.exe -ngl 9999
Microsoft Windows [Version 10.0.19045.5247]
(c) Microsoft Corporation. All rights reserved.

C:\Users\rober>cd c:\temp

c:\Temp>Llama-3.2-3B-Instruct.Q6_K.llamafile.exe -ngl 9999

LLAMAFILE

software: llamafile 0.8.17
model: Llama-3.2-3B-Instruct.Q6_K.gguf
compute: Intel Core i7-6700 CPU @ 3.40GHz (skylake)
server: http://127.0.0.1:8080/

A chat between a curious human and an artificial intelligence assistant. The assistant
always answers to the human's questions.
>>> say something (or type /help for help)
```

For example you can enter the same prompt you entered into *Ollama* previously:

```
>>> How many Greek temples are in Paestum?
```

## OpenAI API compatible endpoints

The web server also includes an OpenAI API-compatible /chat/completions endpoint. Use the examples from section E.3.3 for curl, Python, or LangChain, ensuring you set the port to 8080.

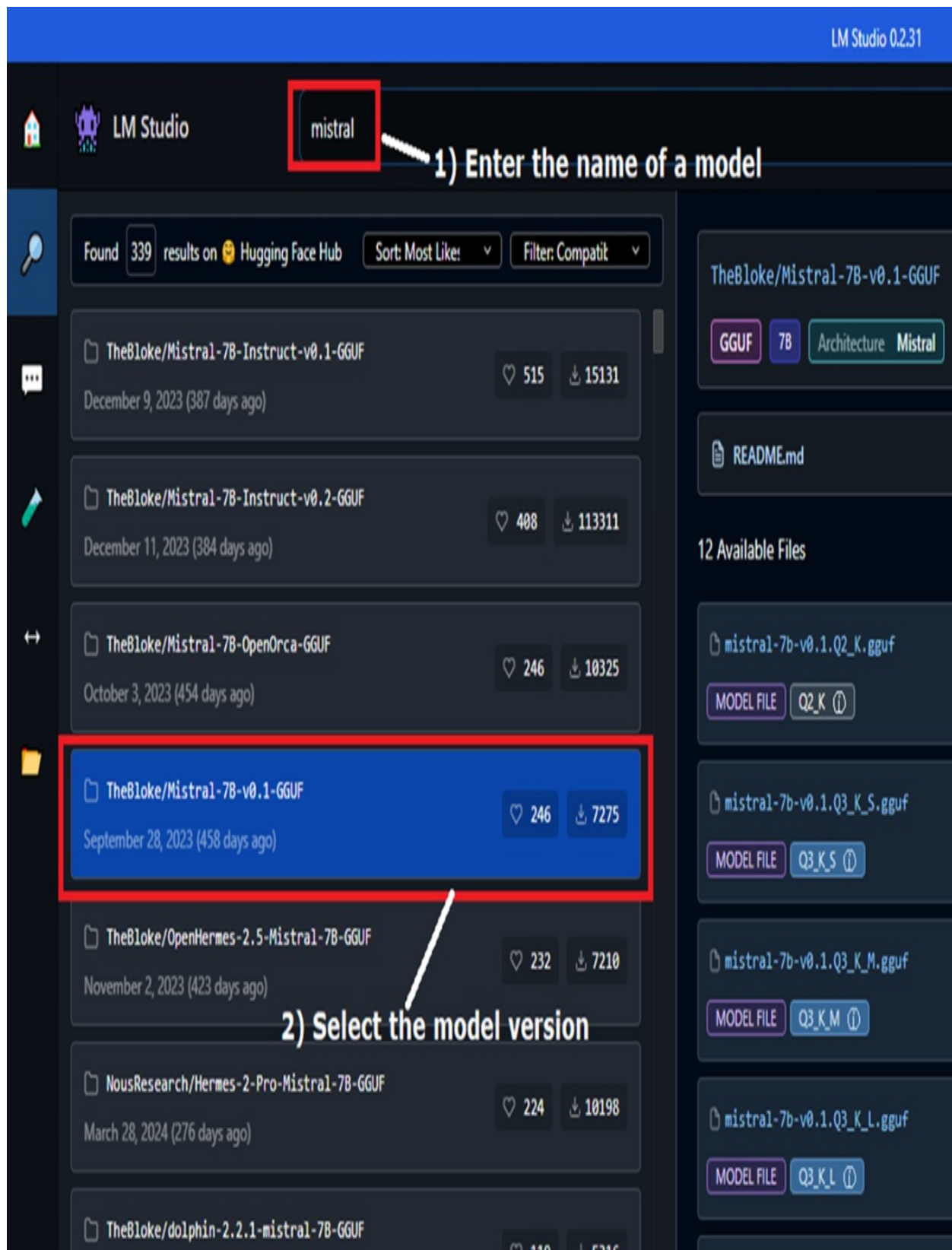
#### **TIP**

To learn more about llamafire, a project by Justine Tunney in collaboration with Mozilla, visit the GitHub page. For additional insights, check out Justine's blog post "Bash One-Liners for LLMs" (<https://justine.lol/oneliners/>).

## **E.4.5 LM Studio**

*LM Studio* is a user-focused local inference engine and a direct competitor to *GPT4All*. Executables for macOS, Windows, and Linux can be downloaded from the LM Studio website (<https://lmstudio.ai>). Once launched, LM Studio provides a seamless experience for searching, selecting, and downloading GGUF models, thanks to its integration with Hugging Face. Figure E.4 shows the model search interface.

**Figure E.4 GGUF Search Screen: 1) search for a model using the search box (in our case, Mistral), 2) select a model variant from the left panel, and download a specific quantized version from the right panel by clicking the "Download" button (not shown in the screenshot). After downloading a model, you can either chat directly through the chat screen or activate the backend server for programmatic interaction.**



In the figure above, you can select a model by entering its name in the search

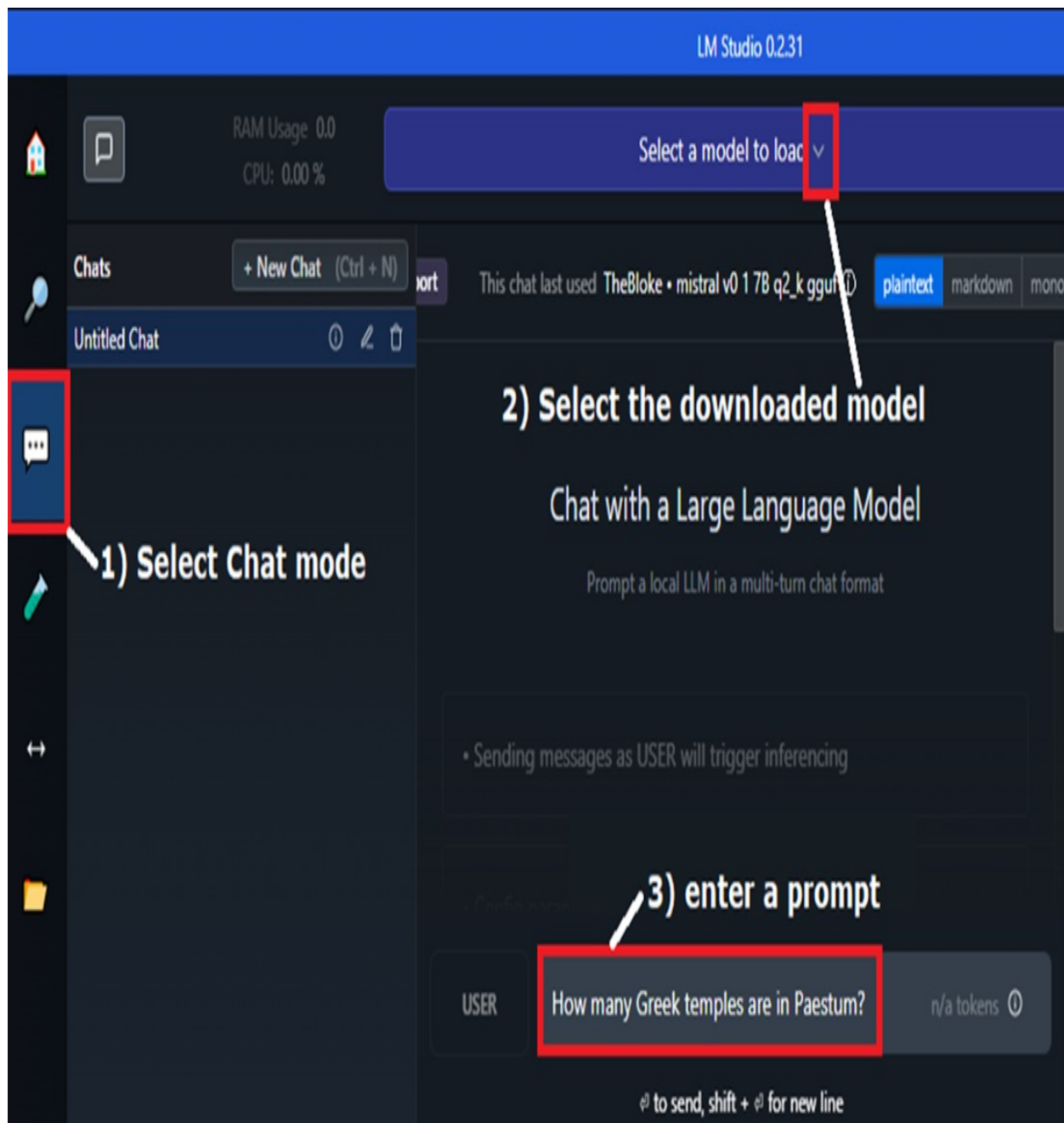
box, choosing a variant from the left panel, and then selecting a specific quantized version (in our case we select a 2-bit quantization, which has very low fidelity) from the right panel by clicking the Download button.

Once downloaded, you can either interact with the model directly through the chat screen or activate the backend server for programmatic communication.

## **Chat screen**

To send interactive prompts, open the Chat menu on the left bar, select the downloaded model from the dropdown at the top, and type your prompt in the text box at the bottom, as shown in Figure E.5.

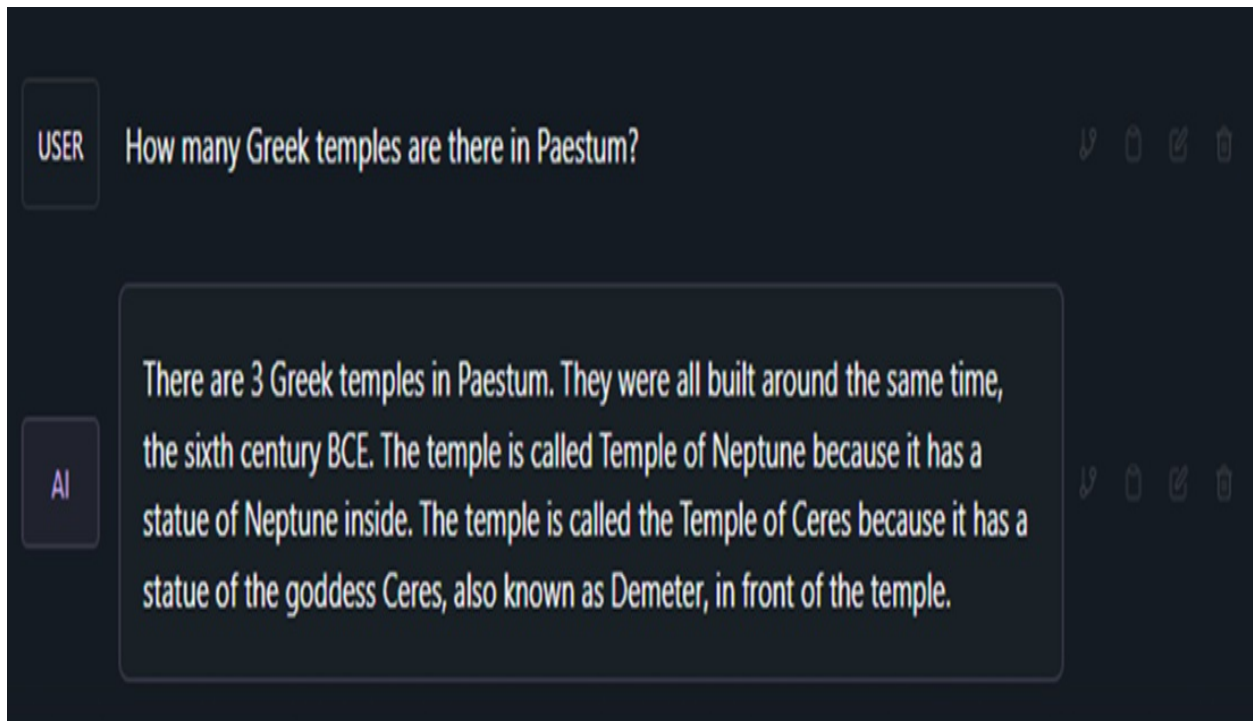
**Figure E.5 Chat Screen: Select the chat menu on the left, choose the model from the dropdown at the top, and type a prompt in the text box at the bottom.**



The response time will vary based on the model, its quantization level, and your computer's hardware. Expect a delay of a few seconds before the LLM provides an answer, as shown in Figure E.6.

**Figure E.6 Prompt Response:** The time to receive an answer depends on the model, its quantization, and your computer's hardware.





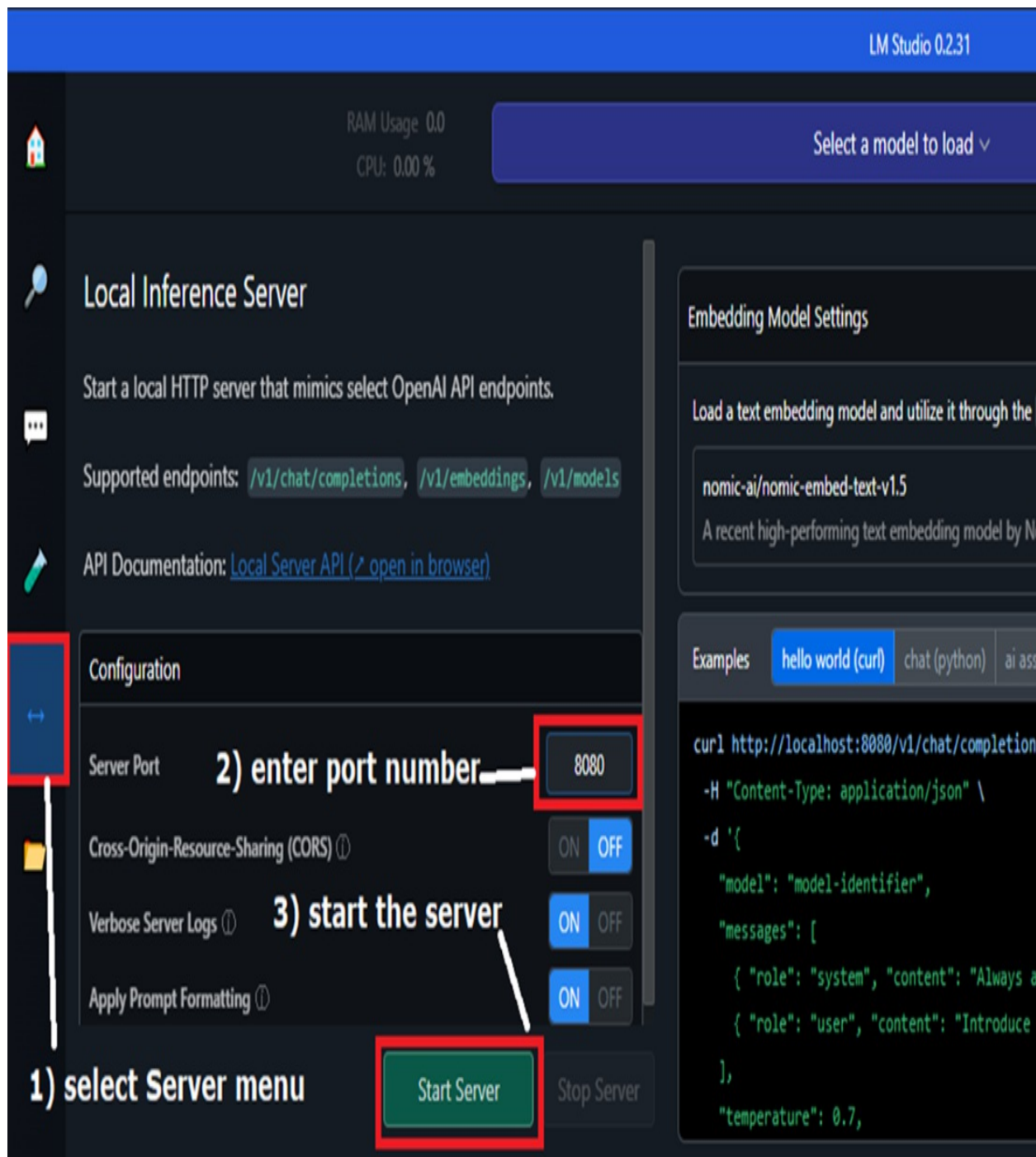
Once you've explored the UI, it's time to examine the server API mode.

## Server mode

To activate the local HTTP server and expose OpenAI-compatible REST API endpoints:

1. Click the Server icon in the left-hand menu.
2. Set the desired port number (e.g., 8080).
3. Click the Start Server button, as shown in Figure E.7.

**Figure E.7 Activating the Local HTTP Server: click the Server menu on the left, enter (or accept) the port number and click the Start Server button.**



After starting the server, the logs will appear in the terminal panel, confirming the server is running. Example logs:

```
[2024-12-30 13:42:41.709] [INFO] [LM STUDIO SERVER] Verbose serve
[2024-12-30 13:42:41.721] [INFO] [LM STUDIO SERVER] Success! HTTP
[2024-12-30 13:42:41.721] [INFO] [LM STUDIO SERVER] Supported end
[2024-12-30 13:42:41.721] [INFO] [LM STUDIO SERVER] -> GET ht
```

```
[2024-12-30 13:42:41.722] [INFO] [LM STUDIO SERVER] -> POST ht
[2024-12-30 13:42:41.722] [INFO] [LM STUDIO SERVER] -> POST ht
[2024-12-30 13:42:41.722] [INFO] [LM STUDIO SERVER] -> POST ht
[2024-12-30 13:42:41.723] [INFO] [LM STUDIO SERVER] Model loaded:
[2024-12-30 13:42:41.723] [INFO] [LM STUDIO SERVER] Logs are save
```

To test the server, copy the sample `curl` request provided in the left-hand panel and paste it into a terminal (on Windows, use Git Bash for easier handling of escaping), or alternatively run on Postman against the specified URL. As the terminal begins processing the request, the LM Studio logs panel will display activity, providing real-time updates during the request's execution:

```
[2024-12-30 13:46:12.782] [INFO] Received POST request to /v1/cha
 "model": "TheBloke/Mistral-7B-v0.1-GGUF",
 "messages": [
 {
 "role": "system",
 "content": "Always answer in rhymes."
 },
 {
 "role": "user",
 "content": "Introduce yourself."
 }
],
 "temperature": 0.7,
 "max_tokens": -1,
 "stream": true
}
[2024-12-30 13:46:12.783] [INFO] [LM STUDIO SERVER] Context Overf
[2024-12-30 13:46:12.783] [INFO] [LM STUDIO SERVER] Streaming res
[2024-12-30 13:46:20.630] [INFO] [LM STUDIO SERVER] First token g
[2024-12-30 13:46:52.409] [INFO] Finished streaming response
```

You will now be able to interact with the OpenAI compatible REST API endpoints. As usual, refer to section E.3.3 for code examples and make sure you use the correct port number and model name.

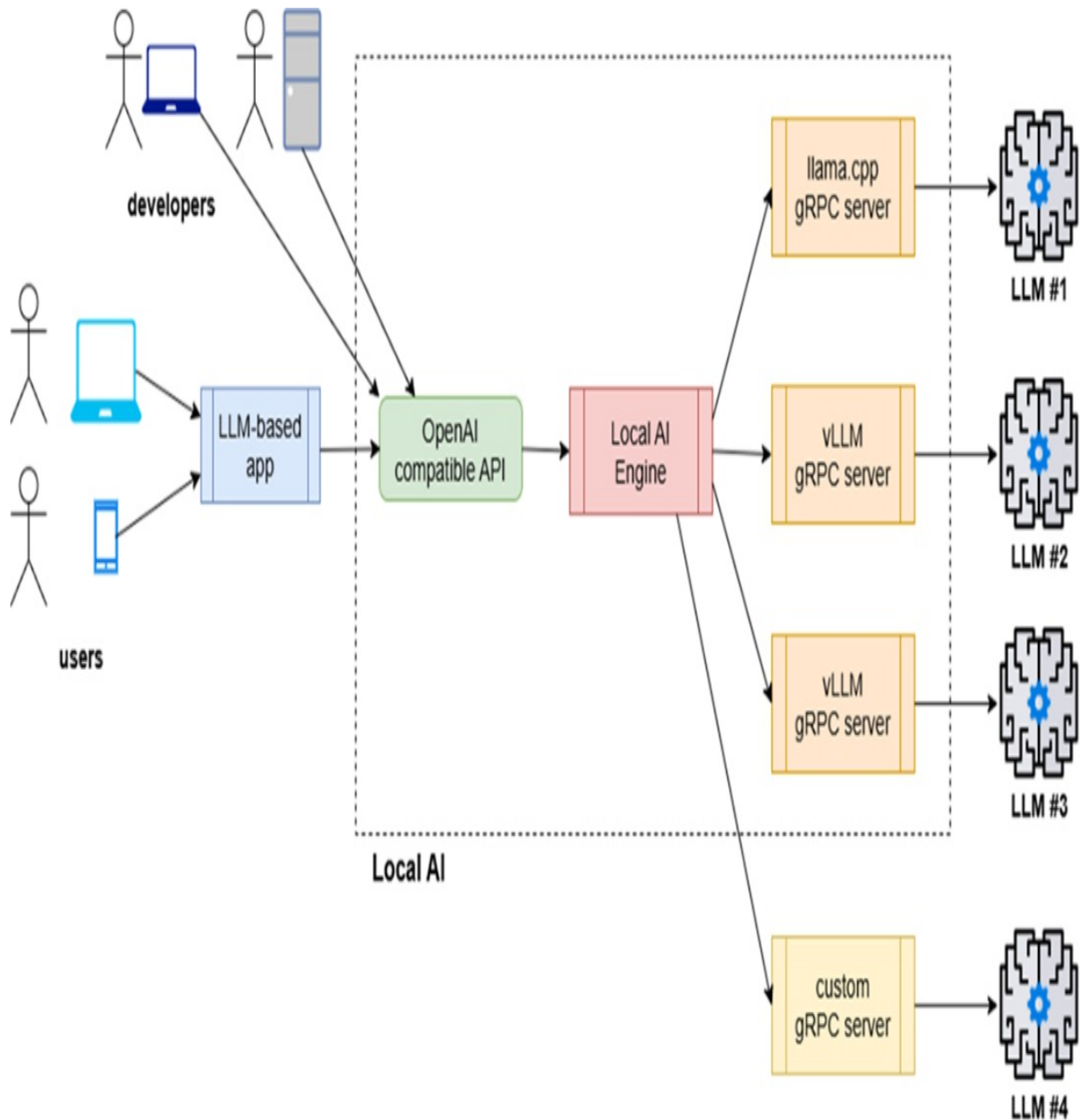
## E.4.6 LocalAI

*LocalAI* is a free inference engine designed to run OpenAI REST API-compatible LLMs on consumer hardware, including systems with plain CPUs or low-grade GPUs. It supports various quantized open-source LLMs and can

also handle audio-to-text, text-to-audio, and multi-modal models. Here, the focus is on its text LLM capabilities.

Written in Go, *LocalAI* serves as a higher-level inference engine that routes OpenAI-like REST API calls to back-end engines like *llama.cpp* or *vLLM*. Figure E.1, adapted from the *LocalAI* documentation, illustrates this architecture.

**Figure E.1: LocalAI Architecture: *LocalAI* routes OpenAI-like REST API calls to inference engines like *llama.cpp*, *vLLM*, or other custom backends.**



## Server mode

The primary distribution method for LocalAI is through container images, which can be deployed using Docker, Podman, or Kubernetes. Popular models are automatically downloaded when starting the container.

For example, to run `Mistral-openorca` on CPU:

```
docker run -ti -p 8080:8080 localai/localai:v2.7.0-ffmpeg-core mi
```

If you have CUDA-12 GPU you can run the related image with the `-gpu` option:

```
docker run -ti -p 8080:8080 --gpus all localai/localai:v2.7.0-cub
```

Container images for CPU, CUDA-11, and CUDA-12 are available on Docker Hub and Quay.io. The LocalAI documentation (<https://localai.io/docs>) provides the appropriate Docker commands for each model and hardware configuration.

If you have a custom quantized model in GGUF format, you can place it in a local folder (e.g., `local_models`) and reference it when starting the container:

```
docker run -p 8080:8080 -v $PWD/local_models:/local_models -ti --
```

## OpenAI REST API compatible endpoints

When a model is started using Docker, *LocalAI* launches an HTTP server on port 8080 with OpenAI-compatible endpoints. Refer to section 10.3.3 for examples of `curl`, Python, and LangChain code you can use with LocalAI. Ensure the port is set to 8080 and verify the model name in the LocalAI documentation for accurate configuration.

## E.4.7 GPT4All

*GPT4All* is an inference engine built on *llama.cpp* that allows you to run GGUF quantized LLM models on consumer hardware, including CPUs and low-grade GPUs. It improves on *llamafile* with a more user-friendly graphical interface, making it accessible to non-technical users.

Installation packages for Windows, macOS, and Ubuntu Linux are available on the *GPT4All* website (<https://gpt4all.io>). After installation, you'll see a desktop client shown in figure E.2, with an interface which allows you to download a model, chat with it and also upload your documents to enable out-of-the box RAG Q&A.

**Figure E.2 GPT4All desktop application home screen**



GPT4All includes the following components, visible and hidden:

1. **Backend Inference Engine:** Built on *llama.cpp*, the engine supports GGUF quantized LLM models (typically under 8GB) for architectures like Falcon, LLAMA, MPT, GPT-J, and Mistral.
2. **Language-Specific Bindings:** High-level API libraries are available for

C++, Python, Go, NodeJS, and more, enabling programmatic access to the inference engine.

3. **Local Web Server:** The desktop application can start a local web server that exposes chat completions through OpenAI-compatible REST API endpoints.
4. **Desktop GUI:** The graphical client lets you interact with an LLM through a user-friendly interface. Models can be selected from a dropdown or added from the Model Explorer on the GPT4All homepage or Hugging Face.
5. **LocalDocs Plugin (Optional):** This plugin allows you to import files containing data or unstructured text and chat with the content. It implements a local Retrieval-Augmented Generation (RAG) architecture using SBERT and an embedded vector database, enabling basic Q&A functionality with zero coding.

## **Server mode**

To enable the local HTTP server for REST API access (port 4891), navigate to GPT4All > Settings and enable the web server option.

## **OpenAI REST API compatible endpoints**

The server exposes OpenAI-compatible endpoints. Use the examples from section 10.3.3 to send requests with `curl`, Python, or LangChain. Ensure the port is set to 4891, and verify the correct model name in the GPT4All documentation.

## **GPT4All Python bindings**

You can create a Python client using the GPT4All Python generation API. Install the package from PyPI:

```
pip install gpt4all
```

Then you can invoke the `generate()` function as follows:

```
from gpt4all import GPT4All
```



```
model = GPT4All('mistral-7b-instruct-v0.1.Q4_0.gguf')
output = model.generate('How many Greek temples are in Paestum?')
print(output)
```

When you instantiate the model, the GGUF file will download automatically to a local directory if it is not already available. The Python API also supports streaming responses.

## LangChain GPT4all python library

LangChain integrates with the native GPT4All Python bindings, providing an advantage by implementing standard LangChain Python interfaces, such as Runnable. This allows you to build LCEL-based solutions, as demonstrated in previous chapters.

First, install the gpt4all package:

```
pip install gpt4all
```

Next, test the GPT4All LangChain integration by running the code in Listing E.1. Use a dedicated environment created with venv for best practice.

### NOTE

Before running the code, ensure you download a GGUF quantized model of your choice into the specified local directory: open the GPT4All application and navigate to the Models section in the left-hand menu. Click on + Add Model to browse the available models, then search for those labeled with the .gguf extension. Once you find the model you want, simply click Download to save it to your device.

### Listing E.1 Connecting to a local open-source model via LangChain's GPT4All Wrapper

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_community.llms import GPT4All

prompt = ChatPromptTemplate.from_messages([#A
 ("system", "You are a helpful AI assistant."),
 ("user", "{input}")
])
```

```
model_path = ('./models/mistral-7b-instruct-v0.1.Q4_0.gguf') #B
llm = GPT4All(model=model_path)

chain = prompt | llm #C

response = chain.invoke({"input": "How many Greek temples are the
print(response)
```

This code demonstrates how to access a local open-source model through GPT4All using LangChain’s wrapper. With this integration, you can seamlessly implement complex workflows.

### E.4.8 Comparing local inference engines

Before closing this section, I want to provide a comparison of the local inference engines discussed.

Some engines are optimized for consumer-grade hardware, while others support advanced configurations, including high-end GPUs. Most engines include an OpenAI-compatible HTTP server, but only a few provide dedicated bindings for specific programming languages. A summary of the main characteristics of each engine is shown in Table E.3.

Table E.3 Summary of the characteristics of local LLM inference engines

Inference engine	Backend engine	Installer available	Supported OS	OpenAI REST API compatibility	Native bindings	
llama.cpp	llama.cpp	No (must build from source)	MacOS, Linux, Windows	Yes	Python, Java, C#, Go, Scala, Ruby, Rust, Scala, JavaScript, node.js	I
ollama	llama.cpp	Yes	MacOS, Linux;	Yes	Python, Javascript	I

			Windows			
vLLM	vLLM	Yes (pip)	Linux	Yes	Python	1
llamafile	llama.cpp	Yes	MacOS, Linux, Windows	Yes	N/A	1
LocalAI	Llama.cpp and vLLM	Docker image	Any hardware running Docker	Yes	N/A	1
GPT4All	Llama.cpp	Yes	MacOS, Linux Ubuntu, Windows	Yes	Python	1
LM Studio	Llama.cpp	Yes	MacOS, Linux Ubuntu, Windows	Yes	Python	1

By now, you should have a clear understanding of how to use local inference engines to run open-source LLMs. However, inference engines are not the only option for serving local LLMs. In the next section, I'll briefly explain how to perform inference using the Hugging Face Transformers library.

### E.4.9 Choosing a local inference engine

If you are new to local LLM inference engines, start with a simple option like *llamafile*. It offers an easy way to experiment with lightweight models. Once you're comfortable, consider moving to *ollama*, which simplifies downloading, configuring, and testing additional models.

If you prefer a graphical user interface, consider upgrading to *LM Studio* or *GPT4All* instead.

For production-grade solutions, *vLLM* is a strong choice. If you need maximum control over all aspects of hosting an LLM, consider using *llama.cpp*. It provides advanced customization options but requires a deeper

understanding of the hosting process.

## E.5 Inference via the HuggingFace Transformers library

For full control over a model's architecture and the ability to modify it, you can run a pre-trained model using the Hugging Face Transformers library. This library is not only designed for experimentation and configuration but also supports fine-tuning models, enabling you to share your improvements with the Hugging Face community. Built on deep learning frameworks like JAX, PyTorch, and TensorFlow, it allows you to use one framework for training and another for inference.

### E.5.1 Hugging Face Transformers library

Before installing the transformers package, set up a virtual environment with venv and install one or more of the following backends: Flax, PyTorch, or TensorFlow. Backend installation can vary depending on your hardware, so it is not covered here. Assuming the backends are installed, you can install transformers via PyPI:

```
pip install transformers
```

Once installed, you can interact with pre-trained models stored locally. For example, with the PyTorch backend, you can run inference on a quantized 4-bit version of Mistral Instruct as shown in Listing E.4 (which I have adapted from the Hugging Face documentation):

**Listing E.4 Performing inference through the Hugging Face transformers library**

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

model_id = "mistralai/Mixtral-8x7B-Instruct-v0.1" #A
tokenizer = AutoTokenizer.from_pretrained(model_id) #B

model = AutoModelForCausalLM.from_pretrained(model_id, load_in_4b

text = "Hello my name is"
```

```
inputs = tokenizer(text, return_tensors="pt").to(0) #C
outputs = model.generate(**inputs, max_new_tokens=20) #D
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

When using the Transformers library, you must explicitly handle tokenization. While this approach provides significant flexibility, it requires a deeper understanding of transformer architecture, making it well-suited for advanced use cases, research, and experimentation.

Let's see how you'd write the same code with LangChain.

## E.5.2 LangChain's HuggingFace Pipeline

LangChain offers a wrapper for the Hugging Face Transformers pipeline, simplifying integration into LangChain applications. If you want to use Hugging Face with LangChain, you can implement the code shown in listing E.5.

**Listing E.5 HuggingFace transformers via LangChain**

```
from langchain_community.llms.huggingface_pipeline import Hugging
from transformers import AutoModelForCausalLM, AutoTokenizer, pip
from langchain.prompts import PromptTemplate

model_id = "mistralai/Mixtral-8x7B-Instruct-v0.1" #A

tokenizer = AutoTokenizer.from_pretrained(model_id) #B
model = AutoModelForCausalLM.from_pretrained(model_id)

pipe = pipeline("text-generation",
 model=model, tokenizer=tokenizer, max_new_tokens=50) #C
hf_pipeline = HuggingFacePipeline(pipeline=pipe)

prompt_template = """Question: {question}

Answer: Let's think step by step.""" #D
prompt = PromptTemplate.from_template(prompt_template)

llm_chain = prompt | hf #E

question = "How many Greek temples are there in Paestum?"
```

```
print(llm_chain.invoke({"question": question})) #F
```

Now that I've covered how to run an open-source LLM locally, the next step is to explore practical applications. In the following section, I'll show how to redirect the research summarization engine we built in Chapter 4 from public OpenAI endpoints to a local open-source LLM like Mistral.

## **E.6 Building a local summarization engine**

Revisit the research summarization engine from Chapter 4 to compare the original OpenAI-based solution with a local open-source LLM. To do this, duplicate the Visual Studio Code project folder by copying the `ch04` folder and renaming it `apE`. This setup allows you to test both versions side by side, making it easier to evaluate accuracy and performance.

Keep in mind that processing with a local LLM will take significantly longer compared to OpenAI's API. However, this exercise will provide valuable insights into how hardware affects inference speed. It will also give you hands-on experience in evaluating whether and how to implement a project using a local open-source LLM.

### **E.6.1 Choosing the inference engine**

First, decide which inference engine to use. Options like GPT4All and LM Studio are user-friendly and suitable for those with basic LLM experience.

To minimize code changes, avoid rewriting the application with native Python bindings like those offered by GPT4All. Instead, use the OpenAI-compatible REST API endpoints provided by both GPT4All and LM Studio. Choosing between them is a matter of preference since both meet the requirements of usability and OpenAI compatibility. For this example, I'll use LM Studio.

### **E.6.2 Starting up the OpenAI compatible server**

If LM Studio is not already installed, follow the installation steps covered earlier. Open the Server menu, select a port (e.g., 8080), and start the server.

LM Studio will now accept OpenAI-compatible requests.

### E.6.3 Modifying the original solution

Since requests will be routed through an OpenAI-compatible endpoint, you only need to make a small change in the `llm_models.py` file, shown in Listing E.6.

**Listing E.6 Original `llm_models.py` code**

```
from langchain.llms.openai import OpenAI

openai_api_key = 'YOUR_OPENAI_API_KEY'

def get_llm():
 return OpenAI(openai_api_key=openai_api_key, model="gpt-4o-mini")
```

You simply need to replace the implementation of `get_llm()` as shown in listing E.7:

**Listing E.7 Modified `llm_models.py` code, using a local open-source LLM**

```
def get_llm():
 port_number = '8080'

 client = OpenAI(
 base_url=f'http://localhost:{port_number}/v1',
 api_key = 'NO-KEY-NEEDED',
 model='mistral'
)

 return client
```

This approach avoids changes to the rest of the code. However, for flexibility, you could make `get_llm()` configurable, as shown in Listing E.8.

**Listing E.8 configurable `get_llm()`**

```
def get_llm(is_llm_local=False):
 port_number = '8080'

 if is_llm_local:
```

```

 client = OpenAI(
 base_url=f'http://localhost:{port_number}/v1',
 api_key='NO-KEY-NEEDED',
 model='mistral'
)
 else:
 client = OpenAI(openai_api_key=openai_api_key, model="gpt-4o")

 return client

```

This version allows you to switch between OpenAI and local LLMs. Keep in mind that some inference engines have fixed port numbers or specific model naming conventions, which may require you to further generalize the code and make it more configurable.

## E.6.4 Running the summarization engine through the local LLM

To test the summarization engine, execute `chain_try_5_1.py`. The LM Studio server logs panel should display activity as the request is processed. When the output is generated (this may take some time, as noted earlier), it should closely match the results from the original OpenAI-based summarization.

## E.6.5 Comparison between OpenAI and local LLM

When running the summarization engine with a local LLM, you'll notice:

1. **Accuracy:** The output is similar for this use case (web scraping and summarization).
2. **Performance:** Local LLM inference is slower, especially on CPUs. Without a GPU, processing times can be significantly longer.

This suggests that while a quantized local open-source LLM is sufficient for development, it may not meet production requirements. Consider deploying LM Studio on hardware with an NVIDIA GPU or using a more advanced inference engine like vLLM. vLLM supports unquantized models and high-end GPUs, offering better performance for demanding use cases.



## E.7 Summary

- Open-source LLMs provide potential cost savings, privacy control, and flexibility, with community-driven innovation ensuring competitive performance and customization compared to proprietary models. These features make open-source LLMs an appealing option for both experimentation and production.
- Transparency in architecture, training data, and methods allows open-source LLMs to be validated, modified, and tailored for compliance and trust. This clarity ensures businesses can meet regulatory requirements while customizing models for specific needs.
- Open-source LLMs empower organizations with lower total cost of ownership and fine-tuning capabilities, enabling tailored solutions while maintaining long-term affordability. This combination helps businesses achieve both financial efficiency and functional scalability.
- Comparative tables simplify the evaluation of open-source LLMs, showcasing their unique features and optimal use scenarios. By comparing models side-by-side, you can quickly identify the best fit for your project goals.
- Open-source LLM weights enable local inference with accessible tools and modern techniques like quantization, supporting seamless OpenAI-compatible API integration for cost-effective and private deployments. These methods make running powerful models on consumer hardware more practical and affordable.
- Start with a simple inference engine like llamafile for lightweight models, progress to tools like Ollama or LM Studio for ease of use, and choose llama.cpp or vLLM for advanced, production-grade customization. This progression helps you build confidence through simple user interfaces and scale capabilities as your project grows.
- Applications built for OpenAI can be adapted to open-source LLMs using compatible REST APIs, offering insights into hardware impacts on inference speed and feasibility. This adaptation enables cost-effective experimentation and highlights areas for optimization.

# welcome

Dear Reader,

Thank you so much for purchasing "*AI Agents and Applications*". Your interest in this cutting-edge topic means a great deal to me, and I am excited to guide you through the expansive world of large language models (LLMs).

This book emerged from my professional journey as a software developer, where my main focus has been quantitative development in finance. Over the years, I occasionally delved into AI projects aimed at tackling problems where traditional mathematical computations fell short. My fascination with LLMs began when I encountered ChatGPT in November 2022. Its capabilities inspired me to explore further, leading me to experiment with the OpenAI APIs, delve into prompt engineering, and design applications using Retrieval Augmented Generation (RAG). My involvement with LangChain, an open-source LLM application framework, significantly accelerated my learning and allowed me to witness its rapid adoption within the community.

"*AI Agents and Applications*" is structured to cater both to beginners and seasoned professionals. It offers a comprehensive look into the foundational technologies and advanced techniques in the realm of LLMs. Whether you are new to programming or an experienced developer, you will find the content approachable yet enriching, especially with practical code examples to enhance your engagement.

The insights in this book are drawn from my real-world experiences and continuous learning in a field that evolves almost daily. I cover a range of topics from running open source LLMs locally to advanced RAG techniques and fine-tuning methods. My goal is to provide a resource that I wish had been available to me—an accessible, practical guide that empowers you to navigate and excel in this emerging domain.

As this field is still in its infancy, your feedback is incredibly valuable. I encourage you to actively participate in our online [discussion forum at](#)

[liveBook](#). Your questions, comments, and suggestions not only help improve this book but also contribute to the broader community exploring LLM applications.

Thank you once again for joining me on this journey. I am eager to hear about your experiences and look forward to any insights you might share.

Warm regards,

— Roberto Infante

**In this book**

[welcome](#) [1 Introduction to AI Agents and Applications](#) [2 Executing prompts programmatically](#) [3 Summarizing text using LangChain](#) [4 Building a research summarization engine](#) [5 Agentic Workflows with LangGraph](#) [6 RAG fundamentals with Chroma DB](#) [7 Q&A chatbots with LangChain and LangSmith](#) [8 Advanced indexing](#) [9 Question transformations](#) [10 Query generation, routing and retrieval post-processing](#) [11 Building Tool-based Agents with LangGraph](#) [12 Multi-agent Systems](#) [13 Building and consuming MCP servers](#) [14 Productionizing AI Agents: memory, guardrails, and beyond](#) [Appendix A. Trying out LangChain](#) [Appendix B. Setting up a Jupyter Notebook environment](#) [Appendix C. Choosing an LLM](#) [Appendix D. Installing SQLite on Windows](#) [Appendix E. Open-source LLMs](#)